A FRIENDLY, HANDS-ON GUIDE

Go for Java Developers

Building enterprise applications in Go, one step at a time, using the Java and Spring Boot you already know as the map.

What you know

Spring Boot · JPA · Maven

JUnit · Mockito · Micrometer

Threads & CompletableFuture

→

Where you're going

net/http · chi · sqlc · pgx

testing · testify · slog

goroutines & channels

Language fundamentals · Concurrency · REST & gRPC · Databases · Messaging · Security · Observability · Resilience · Testing · Performance · Docker & Kubernetes · Capstone project

Go 1.25+ (examples verified on 1.25 and 1.27) · Java 21 · Spring Boot 3

November 2026

Contents

Welcome 8

- Who this guide is for 8
- What you'll build 8
- How each chapter works 9
- How to read the boxes 9
- How to read the code 10
- Three ways to read this guide 10
- Setting up the examples 11
- A note on versions 11
- Before you start 11

Chapter 1: The Mindset Shift: From Java to Go 12

- The big picture 12
- Why Go exists 13
- Where Go shines, and where Java still shines 13
- Go's philosophy in eight ideas 14
- Spring Boot magic vs Go's explicit wiring 16
- The same program, two ways 18
- Java habits to unlearn 20
- How to think in Go: a checklist 25
- Recap 26
- Try it yourself 26
- Further reading 27

Chapter 2: Setting Up and Your First Go Program 28

- The big picture 28
- Installing Go 29
- Modules: Go's answer to pom.xml 30
- The go command: your whole build tool 33
- Go versions: the go line, toolchain and GOTOOLCHAIN 34
- Tools in go.mod (Go 1.24+) 35
- Workspaces: go.work for multi-module work 36
- Editor setup and debugging 37
- Hello, Orderly: your first program 38
- Anatomy of a Go project folder 43
- Cross-compiling: one command, any platform 43
- A tiny Makefile 44
- Recap 44
- Try it yourself 45
- Further reading 45

Chapter 3: Language Basics: Types, Structs, Pointers, Slices and Maps 47

- The big picture 47
- Variables and constants 48
- Control flow 51
- Functions 52
- Structs: Go's POJOs and records 54
- Pointers 57
- Arrays, slices and ArrayList 59

Maps and HashMap 62
Strings, bytes and runes 64
Equality: == instead of equals and hashCode 66
A preview of struct tags 67
Time: time.Time vs java.time 67
Recap 69
Try it yourself 70
Further reading 70

Chapter 4: Interfaces, Errors, Generics and Packages 72

The big picture 72
Interfaces: satisfied, not declared 73
Embedding: composition, not inheritance 78
Errors are values 79
defer: cleanup you can see 84
panic and recover 86
Enums with typed constants and iota 87
Generics 88
Iterators: range over functions 91
Packages and visibility 92
Recap 93
Try it yourself 94
Further reading 95

Chapter 5: Concurrency: Goroutines, Channels and Context 96

The big picture 96
Goroutines 97
Channels 100
select: waiting on several things at once 101
Shared memory: mutexes, Once and atomics 102
context.Context: the stop button that travels with the request 105
errgroup: structured concurrency for Go 107
Concurrency patterns 108
Goroutine leaks 113
Data races and the race detector 115
The memory model in plain English 116
Testing time-based code with testing/synctest 116
Recap 117
Try it yourself 118
Further reading 118

Chapter 6: Structuring an Enterprise Project 120

The big picture 120
Project layout 121
Package by feature, not by layer 123
Layers: handler, service, repository 124
Hexagonal architecture without the ceremony 126
DDD-lite: aggregates, value objects and domain errors 127
Dependency injection 130
The application lifecycle 135
Modular monolith or microservices? 137
Anti-patterns to avoid 138
Recap 139
Try it yourself 139
Further reading 140

Chapter 7: Building REST APIs 141

The big picture 141
net/http fundamentals 142
JSON with encoding/json 144
Validation 148
Error responses with problem+json 150
Middleware 152
chi: the router we recommend 155
Gin, Echo and Fiber 156
Pagination and API versioning 158
http.Server timeouts 159
OpenAPI: spec-first with oapi-codegen 160
The HTTP client side 163
A first taste of handler testing 164
Recap 165
Try it yourself 165
Further reading 166

Chapter 8: gRPC, GraphQL and Real-Time APIs 167

The big picture 167
Choosing the right style 168
Protocol Buffers and buf 168
gRPC with grpc-go 170
Connect: gRPC that also speaks HTTP/JSON 176
grpc-gateway, briefly 178
GraphQL with gqlgen 178
WebSockets 182
Server-Sent Events 184
Recap 186
Try it yourself 187
Further reading 187

Chapter 9: Working with Databases and Caches 188

The big picture 188
database/sql: Go's JDBC 189
pgx and pgxpool: the Postgres driver to use 192
sqlc: write SQL, get type-safe Go 193
Other approaches: sqlx, GORM and ent 196
Transactions without @Transactional 199
Migrations with goose 204
Pool tuning and query timeouts 206
The repository pattern in Go 208
Caching 208
MongoDB, briefly 212
Recap 212
Try it yourself 213
Further reading 213

Chapter 10: Configuration, Secrets and Feature Flags 215

The big picture 215
Twelve-factor config in one minute 216
Environment variables into structs with caarlos0/env 216
Fail fast with validation at startup 218
Passing config explicitly 220
Profiles without Spring profiles 220
Layered file and env config with koanf 222
Viper: popular, with caveats 224

The flag package for CLIs 225
Secrets 226
Dynamic reload: mostly, don't 229
Feature flags with OpenFeature 230
Recap 234
Try it yourself 234
Further reading 235

Chapter 11: Messaging, Background Jobs and Workflows 236

The big picture 236
Delivery semantics in plain English 237
Kafka with franz-go 238
RabbitMQ with amqp091-go 243
NATS, JetStream and cloud brokers, briefly 245
Watermill: a Spring Cloud Stream for Go 246
The transactional outbox 247
Idempotent consumers 252
Background jobs 253
Workflows and sagas with Temporal 257
Recap 260
Try it yourself 261
Further reading 262

Chapter 12: Security 263

The big picture 263
Authentication vs authorization 264
Validating JWTs: your API as a resource server 265
OpenID Connect: keys from your identity provider 269
Authorization: who may do what 271
Passwords and randomness 274
TLS and mutual TLS 276
Defending against common web attacks 277
The OWASP Top 10 in Go 282
Supply chain security 282
Keeping secrets out of logs 284
Recap 285
Try it yourself 286
Further reading 286

Chapter 13: Observability: Logs, Metrics, Traces and Profiles 287

The big picture 287
Logs with log/slog 288
Metrics with Prometheus 293
Distributed tracing with OpenTelemetry 297
Health endpoints: liveness vs readiness 302
Profiling: seeing inside a running program 303
Running the example 307
Recap 307
Try it yourself 307
Further reading 308

Chapter 14: Resilience and Graceful Shutdown 309

The big picture 309
Timeouts everywhere 310
Retries with exponential backoff and jitter 311
Circuit breakers 314
Rate limiting 317

Bulkheads, load shedding and fallbacks 319
Recovering panics in goroutines 322
Graceful shutdown done right 323
Microservice frameworks vs Spring Cloud 327
Recap 328
Try it yourself 329
Further reading 329

Chapter 15: Testing 330

The big picture 330
Your first Go test 331
Table-driven tests: Go's @ParameterizedTest 335
Assertions: plain Go or testify? 336
Test doubles: fakes, mocks and when to use which 339
Testing HTTP 344
Integration tests with testcontainers-go 346
Testing time 349
Fuzzing 351
Benchmarks 353
Coverage 355
The race detector 356
Example tests that double as documentation 357
Black-box and white-box tests 357
The test pyramid in Go 358
Recap 358
Try it yourself 359
Further reading 360

Chapter 16: Tooling and Code Quality 361

The big picture 361
Formatting: gofmt and goimports 362
go vet: the careful friend 363
staticcheck: the deeper review 363
golangci-lint: one runner for everything 364
Your editor: gopls 366
Debugging with Delve 367
Code generation with go generate 368
Pinning tools with tool directives (Go 1.24) 369
Task runners: Makefile, Taskfile and Maven phases 370
Pre-commit hooks 372
CI pipelines 373
Dependency management 377
go fix and the modernize analyzers 379
Style guides and naming 379
Recap 380
Try it yourself 381
Further reading 381

Chapter 17: Performance and How Go Works Inside 383

The big picture 383
When to care, and how to measure 384
Stack, heap and escape analysis 385
The garbage collector 386
Reducing allocations 388
Struct layout and alignment, briefly 391
The scheduler: goroutines, threads and G-M-P 391

GOMAXPROCS in containers 393
Startup time and memory footprint 393
Profile-guided optimisation 394
Inlining and bounds-check elimination, briefly 395
I/O, HTTP and JSON 395
Common performance mistakes 397
Recap 398
Try it yourself 398
Further reading 399

Chapter 18: Build, Containers and Deployment 400

The big picture 400
Building binaries 401
Container images 404
Running on Kubernetes 408
CI/CD pipelines 415
Releasing Go modules 417
Writing Kubernetes operators 418
Serverless, briefly 419
Recap 421
Try it yourself 421
Further reading 421

Chapter 19: More Enterprise Essentials 423

The big picture 423
Command-line tools with cobra 424
Embedding files with go:embed 426
Money and decimals 428
Multi-tenancy 430
Audit logging 432
Internationalisation 434
Plugin architectures 436
Documentation: doc comments and runnable examples 438
Private modules in companies 439
Time zones and UTC in enterprise data 441
Recap 442
Try it yourself 443
Further reading 443

Chapter 20: Capstone: Building Orderly End to End 444

The big picture 444
The life of a "place order" request 445
A tour of the folders 446
main.go: the composition root 447
The heart: the PlaceOrder transaction 449
The outbox relay 453
Turning errors into problem+json 455
Authentication, and "only your own orders" 456
Graceful shutdown, in the right order 458
Run it yourself 459
Where each chapter shows up 462
The same app in Spring Boot 463
Recap 465
Try it yourself 466
Further reading 466

Chapter 21: Your Path to Mastery 468

The big picture 468
The four levels of Go skill 468
A 12-week plan 469
Five milestone projects 470
Habits that build mastery 471
Recommended resources 472
Recap 473
Try it yourself 473
Further reading 474

Appendix A: Java to Go Cheat Sheet 475

Language concepts 475
Concurrency 476
Toolchain 477
Spring annotations and their Go equivalents 477
Frameworks and libraries 478
Everyday snippets, side by side 479
The ten biggest mindset shifts, in one line each 483

Appendix B: Common Mistakes Java Developers Make in Go 484

Design and style 484
Errors 486
Data types 487
Concurrency 488
Resources and I/O 490
Data access 490
Testing 491
Tooling 491

Appendix C: Go Code Review Checklist 492

Readability and naming 492
Design 492
Errors 493
Concurrency 493
Context 493
Resources and I/O 493
Data access 494
Security 494
Observability 494
Resilience and lifecycle 494
Testing 495
Performance 495
Dependencies 495

Appendix D: Glossary 496

Go language and runtime 496
Enterprise and architecture terms 498

Welcome

> *You already know how to build serious software. This guide helps you do it in Go, without throwing away anything you've learned.*

If you've spent years with Java and Spring Boot, your first week with Go can feel strange. There are no classes, no annotations and no exceptions. Nobody wires your beans for you. You might catch yourself thinking, "*Where is everything?*"

That feeling is normal, and it passes faster than you'd expect. Most of what makes you a good Java engineer carries straight over: clean boundaries, testing discipline, transactions, observability, operational thinking. Go asks you to express those ideas more directly. This guide shows you how, one familiar concept at a time.

Who this guide is for

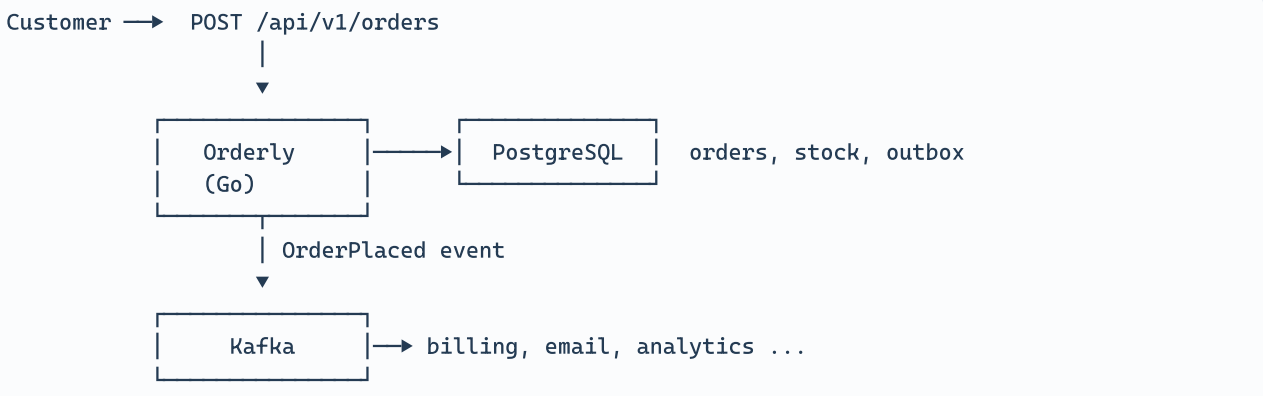
You'll get the most out of this guide if:

- you're comfortable with **Java** (records, generics, streams, `CompletableFuture`)
- you've built services with **Spring Boot** (controllers, JPA, `@Transactional`, Spring Kafka, Actuator)
- you know the basics of **Docker**, **Kubernetes** and **CI/CD**
- you want to become *genuinely good* at Go, not just get Java code to compile in Go syntax

You don't need any prior Go experience. We start from `go mod init`.

What you'll build

Every chapter uses the same small business so you never have to learn a new domain: **Orderly**, an order and inventory service.



Customers place orders, stock gets reserved, and events flow out to other systems. It's small enough to hold in your head but real enough to need transactions, security, messaging, metrics and graceful shutdown. Chapter 20 puts all of it together into one service you can run.

How each chapter works

Every chapter follows the same rhythm, so you always know where you are:

1. **Why it matters.** The problem in plain English, often with an everyday analogy.
2. **How you'd do it in Java.** Real Spring Boot code you'll recognise.
3. **How Go does it.** The idiomatic version, with every non-obvious line explained.
4. **What's different and why.** The design thinking behind Go's choice, which is the part that makes it stick.
5. **Gotchas, tips and a recap.** These are the mistakes every Java developer makes once, so you can skip them.
6. **Try it yourself.** Short exercises on the Orderly domain.

How to read the boxes

You'll see a few kinds of coloured boxes. Each one has a job:

> 🎯 **What you'll learn**
>
> Appears at the top of each chapter so you know what's coming.

> 🧉 **Coming from Java**
>
> How you'd approach this in Java or Spring, and which habits to leave behind.

> 🐭 **The Go way**
>
> The mindset that experienced Go developers bring to the topic.

> 🧩 **Think of it like…**
>
> An everyday comparison to make an abstract idea concrete.

> 💡 **Tip**
>
> Practical advice, and usually a clear recommendation when there are several options.

> ⚠️ **Gotcha**
>
> A common trap. Read these twice.

> 🧠 **Key idea**
>
> The one thing to remember if you forget everything else.

> 🏋️ **Exercise**
>
> Something to try with your hands on the keyboard.

How to read the code

Code blocks are labelled so you can tell at a glance which world you're in:

JAVA `OrderController.java`

```java
// Java code has an orange badge
```

GO `internal/order/service.go`

```go
// Go code has a blue badge
```

TERMINAL

```
# Terminal commands look like this
go test ./...
```

OUTPUT

```
ok      example.com/orderly/internal/order    0.214s
```

When a file name appears in the header, that's where the code lives. When you see **Avoid** and **Prefer** labels, the first block shows a common mistake and the second shows the better way.

> 💡 **Every Go example really runs**
>
> All Go code in this guide comes from runnable programs in the companion `examples/` folder, one module per chapter, plus the full `capstone/` service. Each one was built, vetted and tested on Go 1.25 and Go 1.27. If a snippet looks odd out of context, open the folder and run it.

Three ways to read this guide

You don't have to read cover to cover. Pick the path that fits your situation:

If you...	Read these, in order
need to ship a Go service soon	1, 2, 3, 4, 6, 7, 9, 15, 18, then 20
want to really master Go	Everything, front to back, doing the exercises
mostly review other people's Go code	1, 4, 5, 16, Appendix B, Appendix C

If you...	Read these, in order
are curious how Go compares to Java	1, then Appendix A (the cheat sheet)

Chapter 21 turns all of this into a week-by-week plan.

Setting up the examples

The companion folder looks like this:

```
go-enterprise-guide/
  examples/
    ch02-toolchain/     one Go module per chapter
    ch03-basics/
    ...
  capstone/             the complete Orderly service (Chapter 20)
```

To run any chapter's examples:

TERMINAL

```
cd examples/ch07-rest
go test ./...          # run the tests
go run ./chi           # run one of the example programs
```

Each example folder has a short `README.md` explaining what's inside.

A note on versions

This guide targets **Go 1.25 or newer**, and every example was also verified on **Go 1.27**, the latest release at the time of writing. On the Java side, examples use **Java 21 and Spring Boot 3**, because that's what most enterprise teams run today.

Go keeps a strong compatibility promise: code written for Go 1.25 keeps compiling on every later 1.x release. Everything you learn here will stay useful.

Before you start

Learning a new language as an experienced engineer is humbling. You'll write code that feels clumsy, then a week later it'll feel natural. Be patient with yourself. You're not starting from zero; you're translating years of hard-won judgement into a new, simpler vocabulary.

Let's begin with the biggest shift of all: how Go *thinks*.

CHAPTER 1

The Mindset Shift: From Java to Go

> *Go is easy to read in an afternoon, but thinking in Go takes a small, deliberate shift, and this chapter makes that shift feel natural.*

🎯 **What you'll learn**

- Why Go was created, and which problems it was built to solve
- Where Go shines, and where Java is still the better tool
- The handful of ideas that make up Go's philosophy, with small examples
- How Spring Boot's "magic" maps to Go's explicit wiring
- Eleven Java habits to unlearn, and what to do instead
- A short checklist to help you "think in Go" from day one

The big picture

You already know how to build serious systems. You know transactions, REST, messaging, testing and deployment. None of that knowledge is wasted in Go. What changes is the *style*: Go asks you to say more things out loud in code, and to lean less on frameworks that do things for you behind the scenes.

That can feel like a step backwards for the first week or two. Then something clicks. You open a Go file you've never seen before, read it top to bottom, and understand exactly what it does. No proxies, no annotation processors, no "where does this bean come from?". That feeling is what this chapter is about.

🧩 **Think of it like moving from an automatic car to a manual one**

Spring Boot is an automatic gearbox: smooth, clever, and it decides a lot for you. Go is a manual: you shift the gears yourself. It's a little more work at first, but you always know which gear you're in, and nothing surprises you on a steep hill.

Here is the whole chapter in one table. Don't worry if some rows are new; we'll cover each one.

Concept	Java / Spring	Go
Wiring objects	IoC container, `@Autowired`	Constructors called in `main`
Errors	Exceptions, `try` / `catch`	Returned `error` values
Reuse	Inheritance, abstract classes	Composition, small interfaces
Cross-cutting code	Annotations, AOP proxies	Middleware, plain functions

Concept	Java / Spring	Go
Request scope	`ThreadLocal`, MDC	`context.Context` passed along
Code style	Team style guides, Checkstyle	`gofmt`, one style for everyone
Building blocks	Frameworks (Spring, Hibernate)	Standard library plus small libraries

Why Go exists

Go was conceived at Google in late 2007 by Robert Griesemer, Rob Pike and Ken Thompson. It went public in 2009, and Go 1.0 shipped in 2012 with a promise that Go 1 code would keep compiling. That promise still holds today.

The designers were not trying to invent a clever language. They were trying to fix the day-to-day pain of building large software with large teams. In his 2012 talk "Go at Google", Rob Pike listed the problems they saw:

- **Slow builds.** One 2007 Google binary took 45 minutes to build, even on a distributed build system. Header files expanded about 4 MB of C++ source into more than 8 GB of compiler input.
- **Uncontrolled dependencies.** Nobody could tell what a program really depended on.
- **Everyone using a different subset of the language.** C++ and Java are big, so each team wrote its own dialect.
- **Code that is hard to read**, hard to tool, and expensive to update.
- **Concurrency that is hard to get right.** Google's servers handle huge numbers of simultaneous requests, and threads plus locks made that painful.

Go's answers are deliberately plain. The compiler is fast because the language is small and imports are strict. Unused imports are compile errors, so dependencies stay honest. There is one formatting style, enforced by a tool. And concurrency is built into the language with goroutines and channels (Chapter 5).

> **Coming from Java**
>
> Java solved many of the same problems in its own way: the JIT for speed, Maven for dependencies, and virtual threads (Java 21) for cheap concurrency. Go is not "better Java". It made different trade-offs: smaller language, faster builds, single static binaries, and less magic.

Where Go shines, and where Java still shines

Being fair here matters. You'll make better architecture decisions if you know both languages' strengths, rather than picking a team.

Go has become the language of cloud infrastructure. **Docker, Kubernetes, Terraform, Prometheus, etcd, Helm, Consul and Vault are all written in Go.** If you have run a container this week, you've used Go code.

Go shines at	Why
Microservices and APIs	Fast start-up, small memory use, one binary to ship

Go shines at	Why
Cloud-native tools and operators	The Kubernetes ecosystem speaks Go
CLIs and developer tools	Cross-compiles to a single file for every OS
Network services and proxies	Goroutines make thousands of connections cheap
Serverless and scale-to-zero	Starts in milliseconds, no warm-up

Java is still an excellent choice in plenty of places, and nobody should rewrite a working system only to use Go.

Java shines at	Why
Large, rich domain models	Mature ORMs, JPA, and decades of patterns
Big existing enterprise estates	Your libraries, skills and tooling are already there
Batch and data processing	Spring Batch, Spark, Flink, Kafka Streams
Long-running, CPU-heavy work	The JIT can beat ahead-of-time code after warm-up
"Batteries included" platforms	Spring covers almost every enterprise need

> 💡 **If you're unsure, start with a new, small service**
>
> The best way to bring Go into a Java shop is a new, well-bounded service: a gateway, a worker, an internal tool. Keep your big domain monolith in Java and let the two talk over HTTP, gRPC or Kafka.

Go's philosophy in eight ideas

Go has a culture as well as a syntax. Rob Pike collected much of it as the "Go Proverbs". You'll hear these quoted in code reviews, so let's go through the most useful ones, each with a tiny example.

Simplicity

Go has 25 keywords. Java has around 50 (counting the reserved ones), and Java's feature list keeps growing. Go adds features slowly and on purpose: generics took more than ten years to arrive.

Simplicity here doesn't mean "toy". It means there is usually one obvious way to write something, so code written by a new joiner looks like code written by the tech lead. That makes a codebase of a million lines feel familiar everywhere you open it.

Explicit over implicit

In Go, things happen because a line of code says so. There is no field injection, no proxy that opens a transaction around your method, and no exception that silently jumps up five frames.

GO

```go
order, err := repo.FindOrder(ctx, id)
if err != nil {
    return fmt.Errorf("find order %s: %w", id, err)
}
```

Yes, this is more typing than a `throws` clause. In return, you can see every place a function can fail by reading it.

Composition over inheritance

Go has no `extends`. You build bigger things from smaller things: a struct can contain (embed) another struct, and a type satisfies an interface by having the right methods. We'll see this in the "habits" section below and in depth in Chapter 4.

Clear is better than clever

Go code is read far more often than it is written, so Go culture rewards boring, obvious code. A plain `for` loop beats a chain of five stream operations if the loop is easier to follow. If a reviewer has to stop and think "wait, how does this work?", that's a reason to rewrite it.

A little copying is better than a little dependency

Pulling in a library for one small helper adds a long-term cost: upgrades, security patches, licence checks and a bigger build. Go culture says: if it's ten lines, copy or write the ten lines.

Coming from Java

In the Maven world it's normal to pull in Guava or Apache Commons for a single utility method. In Go, the standard library (`strings`, `slices`, `maps`, `time`, `net/http`, `encoding/json`, `log/slog`) covers far more than Java's does, so you'll need far fewer small dependencies.

Share memory by communicating

The full proverb is "Don't communicate by sharing memory; share memory by communicating." Instead of several threads poking at one shared map under a lock, goroutines pass data to each other through channels, so only one goroutine owns a piece of data at a time.

Think of it like a kitchen pass

In a busy restaurant, cooks don't all reach into the same pan. Each cook finishes a plate and puts it on the pass, and the next person picks it up. The pass is a channel: it hands over ownership cleanly. Chapter 5 is all about this.

Go still has mutexes, and they're the right tool for protecting a small piece of shared state. The proverb is a default, not a ban.

Errors are values

A Go error is an ordinary value of type `error`, returned like any other result. You can store it, wrap it, compare it and test it.

GO

```go
var ErrOrderNotFound = errors.New("order not found")
```

Because errors are values, you handle them with normal code (`if`, `switch`, `errors.Is`) instead of special control flow. Chapter 4 covers wrapping and custom error types.

One formatting style, and libraries over frameworks

`gofmt` formats every Go file the same way: tabs, brace positions, spacing. There's nothing to configure and nothing to argue about. A proverb sums it up: "Gofmt's style is no one's favorite, yet gofmt is everyone's favorite."

The same spirit applies to architecture. Instead of one big framework that owns your application, Go programs combine the standard library with small, focused libraries: a router here, a database driver there. You call them; they don't call you.

> 🧠 **Key idea**
>
> Go optimises for the *reader* and for the *team*, not for the cleverness of a single line. When in doubt, choose the version a tired colleague could understand at 2 a.m.

Spring Boot magic vs Go's explicit wiring

Let's look at the thing you'll notice first: there is no IoC container. Here is a small Spring Boot catalog with a repository and a service.

JAVA Catalog.java (Spring Boot)

```java
@Repository
public class JpaProductStore implements ProductStore {
    // Spring Data or JPA code here
}

@Service
public class Catalog {
    private final ProductStore store;

    public Catalog(ProductStore store) { // injected by Spring
        this.store = store;
    }
}

@SpringBootApplication
```

```java
public class OrderlyApplication {
    public static void main(String[] args) {
        SpringApplication.run(OrderlyApplication.class, args);
    }
}
```

Spring scans the classpath, finds the annotated classes, works out the dependency graph, and builds everything for you. It's convenient, and it's also invisible: to find out which `ProductStore` you got, you need to know Spring's rules.

In Go, the same graph is built by hand, in `main`, in plain sight:

GO wiring/main.go

```go
func main() {
    ctx := context.Background()

    // The whole "application context", built by hand, top to bottom.
    logger := slog.New(slog.NewTextHandler(os.Stdout, nil))
    store := NewMemoryStore()
    catalog := NewCatalog(store, logger)

    mug := Product{SKU: "MUG-BLUE-01", Name: "Blue Mug", PriceCents: 1299}
    if err := catalog.Add(ctx, mug); err != nil {
        logger.Error("add failed", "err", err)
        os.Exit(1)
    }
    // ...
}
```

`NewCatalog` is an ordinary function. It receives its dependencies as parameters, which is exactly the constructor injection you already use in Spring, minus the container.

GO wiring/main.go

```go
// productStore is what Catalog needs. The consumer declares it, and
// MemoryStore satisfies it without saying so.
type productStore interface {
    Save(ctx context.Context, p Product) error
    Find(ctx context.Context, sku string) (Product, error)
}

// NewCatalog is plain constructor injection.
func NewCatalog(store productStore, logger *slog.Logger) *Catalog {
    return &Catalog{store: store, logger: logger}
}
```

Running it prints:

OUTPUT

```
time=2026-11-02T23:10:45.244+05:30 level=INFO msg="product added" sku=MUG-BLUE-01
TEE-RED-XL is not in the catalog (yet)
```

Full code: `examples/ch01-mindset/wiring/`

> **The Go way**
>
> Wire your application by hand in `main` (or a small `run` function it calls). Even large services have a wiring section of only 50 to 150 lines, and it doubles as a map of the whole system. Chapter 6 shows how to keep it tidy as it grows. Teams that really want a DI framework can use Uber Fx; Google's Wire, once popular, is now archived.

A mental model map

When you meet a Java concept in your head, this table tells you where to look in Go.

Java / Spring	Go	Where in this guide
Class	Struct type with methods	Chapter 3
`new Order(...)`, constructor	`NewOrder(...)` function	Chapter 3
Interface (declared up front)	Interface (satisfied implicitly)	Chapter 4
`extends`	Embedding a struct	Chapters 3 and 4
Checked/unchecked exceptions	`error` values; `panic` for bugs	Chapter 4
Generics	Generics (type parameters)	Chapter 4
`Thread`, `ExecutorService`	Goroutines, `sync.WaitGroup`	Chapter 5
`BlockingQueue`	Channels	Chapter 5
`ThreadLocal`, MDC	`context.Context`	Chapter 5
Maven/Gradle module	Go module (`go.mod`)	Chapter 2
Java package	Go package (one directory)	Chapters 2 and 6
`@RestController`	`http.Handler` and a router	Chapter 7
JPA / Hibernate	sqlc, pgx, or GORM	Chapter 9
`application.yml`	Env vars into a struct	Chapter 10
Servlet filter, AOP aspect	HTTP middleware	Chapters 7 and 13
JUnit + Mockito	`testing` + fakes	Chapter 15
Fat JAR	Single static binary	Chapter 18

The same program, two ways

Let's make this concrete. Here is a greeting service written with every Java habit we're about to discuss: an `I`-prefixed interface, an `Impl`, getters and setters, a builder and a factory. It's valid Go, and it works.

GO javaish/main.go (trimmed)

```go
type IGreetingService interface {
    Greet(name string) string
}

type GreetingServiceImpl struct {
```

```
    prefix string
    shout  bool
}

func (s *GreetingServiceImpl) GetPrefix() string  { return s.prefix }
func (s *GreetingServiceImpl) SetPrefix(p string) { s.prefix = p }

// ... IsShout, SetShout, Greet, a builder with WithPrefix/WithShout ...

func (GreetingServiceFactory) CreateDefault() IGreetingService {
    return NewGreetingServiceBuilder().
        WithPrefix("Welcome to Orderly").
        WithShout(false).
        Build()
}
```

The full file is about 70 lines. Here's the idiomatic version, complete:

GO idiomatic/main.go

```
// Greeter builds welcome messages for Orderly customers.
// The zero value works: Greeter{} greets with an empty prefix.
type Greeter struct {
    Prefix string
    Shout  bool
}

// Greet returns a greeting for name.
func (g Greeter) Greet(name string) string {
    msg := g.Prefix + ", " + name + "!"
    if g.Shout {
        return strings.ToUpper(msg)
    }
    return msg
}

func main() {
    g := Greeter{Prefix: "Welcome to Orderly"}
    fmt.Println(g.Greet("Ada"))
}
```

Both programs print the same thing:

OUTPUT

```
Welcome to Orderly, Ada!
```

Full code: `examples/ch01-mindset/javaish/` *and* `examples/ch01-mindset/idiomatic/`

The struct literal `Greeter{Prefix: "Welcome to Orderly"}` *is* the builder: named fields, any order, and anything left out gets a sensible zero value. There's no interface because nothing needs one yet. When a test or a second implementation needs one, the consumer can declare it in a few lines.

Java habits to unlearn

This is the heart of the chapter. Every Java developer brings some of these habits along, and that's completely normal; they were good habits in Java. For each one we'll look at the Java way, why it doesn't fit Go, and what to do instead.

Habit 1: Getters and setters for everything

The Java way: private fields, plus `getX()` and `setX()` for each one (or Lombok's `@Data`). Frameworks like JPA and Jackson expect JavaBean accessors.

Why it doesn't fit: Go's visibility rule is about packages, not classes. A field with a capital letter is visible outside the package; a lower-case one is not. `encoding/json` and most libraries work directly with exported fields, so accessors add nothing but noise.

The Go way: export the field. When you need to protect an invariant, keep the field unexported and add a method named after the field, *without* `Get`.

GO habits/getters.go

```go
// Product shows the Go default: exported fields, no getters or setters.
type Product struct {
    SKU        string
    Name       string
    PriceCents int64
}

// Customer keeps its id private, so it offers a read-only accessor.
// Note the name: ID(), not GetID().
type Customer struct {
    id    string
    Email string
}

// ID returns the customer's id.
func (c Customer) ID() string { return c.id }
```

Habit 2: An interface for every class

The Java way: `UserService` plus `UserServiceImpl`, or `IUserService` plus `UserService`. The interface exists up front so Spring can proxy it and Mockito can mock it.

Why it doesn't fit: Go interfaces are satisfied *implicitly*. A type never says "implements"; if it has the methods, it fits. That means the *consumer* can declare the small interface it needs, at the moment it needs it. A one-to-one interface declared next to its only implementation is pure ceremony. And Go never uses an `I` prefix.

The Go way: return concrete types, and declare tiny interfaces where they're used. A popular saying is "accept interfaces, return structs".

GO habits/consumer.go

```go
// orderFinder is declared by the code that uses it, and it lists only
// the one method that code calls. Any type with that method fits.
type orderFinder interface {
    FindOrder(ctx context.Context, id string) (Order, error)
}

// Receipts depends on a small interface, not a concrete type.
type Receipts struct {
    orders orderFinder
}
```

`OrderMap`, defined further down in the same file, has a `FindOrder` method and never mentions `orderFinder`. It fits anyway. Another proverb says it well: "The bigger the interface, the weaker the abstraction."

Habit 3: Deep package trees and `util` packages

The Java way: `com.acme.orderly.service.impl.order`, plus `common`, `util` and `helpers` packages that grow forever.

Why it doesn't fit: a Go package is one directory, and its *name* appears at every call site (`order.New`, `money.Format`). Long paths add nothing. Import cycles are compile errors, so a giant `common` package that everybody imports becomes a trap. And a name like `util` tells the reader nothing about what's inside.

The Go way: a shallow layout organised by what the code is *about*:

```
orderly/
  cmd/orderly/main.go     <- wiring lives here
  internal/
    order/                <- order rules, types, repository interface
    inventory/            <- stock reservation
    postgres/             <- database code
    httpapi/              <- handlers and middleware
```

`internal/` is special: only packages inside the directory that contains it may import it. It's how Go keeps implementation details private. Chapter 6 covers project layout properly.

Habit 4: Exceptions (and reaching for `panic`)

The Java way: `throw new OrderNotFoundException(id)` and a `@ControllerAdvice` somewhere far away that catches it.

Why it doesn't fit: Go has no exceptions. It does have `panic`, which looks similar, so many newcomers use it as `throw`. Please don't. A panic means "the program is broken", like an `Error` in Java, not "the order wasn't found".

The Go way: return an `error` as the last result, check it straight away, and add context as it travels up.

GO habits/consumer.go

```go
func (r *Receipts) Print(ctx context.Context, id string) (string, error) {
    o, err := r.orders.FindOrder(ctx, id)
    if err != nil {
        return "", fmt.Errorf("receipt for order %s: %w", id, err)
    }
    return fmt.Sprintf("Order %s: %d cents", o.ID, o.TotalCents), nil
}
```

The `%w` verb wraps the original error, so a caller can still ask `errors.Is(err, ErrOrderNotFound)` and turn it into a 404. Keep `panic` for truly impossible states and start-up failures.

> **Why all the `if err != nil`?**
>
> It's the most common complaint from newcomers, and it fades quickly. Each check is a decision point you can see: retry, wrap, translate or give up. After a few weeks, code that hides its failure paths starts to feel uncomfortable.

Habit 5: Factories, builders and abstract classes

The Java way: `OrderServiceFactory`, Lombok `@Builder`, and `AbstractBaseService` with template methods for subclasses to fill in.

Why it doesn't fit: Go has no constructors, no overloading and no abstract classes, so those patterns lose their reason to exist. Struct literals with named fields already read like a builder.

The Go way: a plain `NewX` function, and a config struct whose zero values mean "use the default".

GO habits/options.go

```go
// ServerConfig replaces a builder. A struct literal with named fields
// is already readable, and zero values mean "use the default".
type ServerConfig struct {
    Addr        string
    ReadTimeout time.Duration
}

// NewServer is a plain constructor that takes a config struct.
func NewServer(cfg ServerConfig) *Server {
    return &Server{cfg: cfg.withDefaults()}
}
```

Calling it reads nicely: `NewServer(ServerConfig{Addr: ":9090"})`. Some libraries use "functional options" instead, as in `grpc.NewClient(addr, opts...)`; they're fine to use, but a config struct is the simpler default for your own code. Instead of an abstract base class, pass in a function or a small interface for the part that varies.

Habit 6: Annotation-driven behaviour

The Java way: `@Transactional`, `@Cacheable`, `@Retryable`, `@Scheduled`, `@Valid`. You add a word above a method and a proxy does the work.

Why it doesn't fit: Go has no annotations and no runtime proxies. Struct tags (like `` `json:"sku"` ``) look a bit similar, but they're only labels that a library can read; they never add behaviour by themselves.

The Go way: behaviour is code you can see and step through:

- **Cross-cutting HTTP concerns** (logging, auth, metrics) are middleware: functions of type `func(http.Handler) http.Handler` (Chapter 7).
- **Transactions** are an explicit helper that runs your function inside `BEGIN` / `COMMIT` (Chapter 9).
- **Retries and timeouts** are explicit calls with a `context` deadline (Chapter 14).
- **Validation** is a function call, or `go-playground/validator` called on purpose (Chapter 7).

It's more visible, and that's the point: a debugger steps straight into it.

Habit 7: Name stutter

The Java way: `com.acme.order.OrderService`, imported and used as `OrderService`.

Why it doesn't fit: in Go you always see the package name in front: `order.OrderService` reads "order, order service". The package name already gives the context.

The Go way: let the package name do half the work.

Avoid	Prefer
`order.OrderService`	`order.Service`
`order.NewOrderService()`	`order.NewService()`
`http.HTTPClient`	`http.Client` (the real name)
`user.UserRepository`	`user.Repository` or `postgres.UserStore`

Habit 8: Over-abstracting early

The Java way: Controller, `OrderService` interface, `OrderServiceImpl`, `OrderRepository` interface, `OrderRepositoryImpl`, a mapper and three DTOs, all on day one, "in case we need them".

Why it doesn't fit: in Go, adding an interface later is cheap, because implementations never have to declare it. Abstractions built before you have two real uses usually guess wrong, and then they get in the way.

The Go way: start concrete. Write the struct and its methods. Extract an interface when a second implementation appears, or when a test needs a fake. Remember "a little copying is better than a little dependency": two similar functions are fine until a third shows you the real pattern.

Habit 9: Inheritance hierarchies

The Java way: `BaseEntity` with `id` and timestamps, `AuditableEntity extends BaseEntity`, `Product extends AuditableEntity`.

Why it doesn't fit: Go has no inheritance at all.

The Go way: embed a struct to reuse its fields and methods (composition), and use interfaces for polymorphism.

GO habits/embedding.go

```go
// Audit holds fields that many types share.
type Audit struct {
    CreatedAt time.Time
    UpdatedAt time.Time
}

// Touch records a change.
func (a *Audit) Touch(now time.Time) { a.UpdatedAt = now }

// CatalogItem embeds Audit. This is composition, not inheritance:
// a CatalogItem *has an* Audit, and Go promotes its fields and methods
// so you can write item.Touch(now) and item.UpdatedAt.
type CatalogItem struct {
    Audit
    SKU  string
    Name string
}
```

> ⚠️ **Gotcha: embedding is not subclassing**
>
> A `CatalogItem` is not an `Audit`. You can't pass a `CatalogItem` to a function that expects an `Audit`, and there's no `super` or method overriding. If you need "anything that can be touched", define an interface with a `Touch` method.

Habit 10: Ignoring context propagation

The Java way: request-scoped data lives in `ThreadLocal`s: the security context, the MDC for logging, the current transaction. Timeouts are configured on each client.

Why it doesn't fit: goroutines have no identity you can attach data to, and one request often fans out into several goroutines. A `ThreadLocal` equivalent would break constantly.

The Go way: pass a `context.Context` as the **first parameter** of every function that does I/O or might take a while, all the way down. It carries cancellation, deadlines and a few request-scoped values.

GO habits/consumer.go

```go
func (m OrderMap) FindOrder(ctx context.Context, id string) (Order, error) {
    if err := ctx.Err(); err != nil {
        return Order{}, err // the caller gave up: stop early
    }
    // ...
}
```

> 🧩 **Think of it like a stop button that travels with the request**
>
> When a user closes their browser, or a deadline passes, someone presses the stop button. Every function holding the context can notice and stop work, including database queries and outgoing HTTP calls. Chapter 5 explains it fully.

Habit 11: Expecting GORM to be Hibernate

The Java way: JPA with Hibernate gives you a persistence context, automatic dirty checking, lazy-loading proxies, cascades and JPQL.

Why it doesn't fit: GORM is the most popular Go ORM, and it's handy, but it isn't Hibernate. There's no persistence context that tracks changes and flushes them at commit, no lazy-loading proxies (you load associations explicitly with `Preload`), and when you update with a struct, GORM skips zero-value fields. That last one surprises everyone: setting a quantity to `0` silently does nothing.

The Go way: most Go teams write SQL and let a tool generate type-safe code around it. This guide's default is **sqlc** with the **pgx** driver; GORM, sqlx, ent and Bun are compared in Chapter 9. If you like JPA, give sqlc a fair try. Seeing your actual SQL is a relief once you get used to it.

How to think in Go: a checklist

You don't need to memorise this chapter. Keep this list nearby for your first few weeks, and glance at it during code reviews.

When you feel like...	Try this instead
Writing an interface first	Write the struct; add a small interface at the consumer later
Adding getters and setters	Export the field
Throwing an exception	Return an `error` and wrap it with `%w`
Adding an annotation	Write middleware or call a helper function
Creating a `util` package	Name the package after what it provides
Using a `ThreadLocal`	Pass `ctx context.Context` first
Adding a library for one helper	Write or copy the few lines
Arguing about formatting	Run `gofmt` and move on

And a few positive habits to build:

- Start with a concrete type and a function.
- Make the zero value useful, so `var x T` is ready to use.
- Handle every error where it happens, and add context.
- Pass `context.Context` as the first parameter for I/O.
- Wire dependencies by hand in `main`.
- Prefer the standard library, then small, well-maintained libraries.

- Choose the clearest code, even if it's a few lines longer.

> 💡 **Be patient with yourself**
>
> For the first couple of weeks, your Go will look like Java with different keywords. That's how everyone starts. Read some well-written Go (the standard library's `net/http` and `strings` packages are lovely) and the style will rub off on you faster than you expect.

Recap

- Go was designed at Google to fix slow builds, tangled dependencies, hard-to-read code and painful concurrency in large teams.
- Go shines for cloud-native services, CLIs and network code; Java still shines for rich domain models, big existing estates and data processing.
- Go's culture values simplicity, explicit code, composition, clarity and few dependencies, with errors as ordinary values and one formatting style.
- Instead of an IoC container, Go wires dependencies by hand in `main` using plain constructor functions.
- Interfaces are satisfied implicitly, so declare small ones where they're used, not next to every type.
- Replace exceptions with returned errors, annotations with middleware and helpers, inheritance with embedding, and `ThreadLocal`s with `context.Context`.
- Name packages by what they provide, avoid stutter (`order.Service`), and start concrete before abstracting.

Try it yourself

> 🏋️ **Exercise 1: Spot the habits**
>
> Open `examples/ch01-mindset/javaish/main.go` and list every Java habit you can find (there are at least five). For each, write one sentence on what the Go version does instead. *Hint:* compare it line by line with `idiomatic/main.go`.

> 🏋️ **Exercise 2: Add a feature, the Go way**
>
> Add an optional `Suffix string` field to `Greeter` in `idiomatic/`, so that `Greeter{Prefix: "Hi", Suffix: " :)"}` produces `Hi, Ada! :)`. Add a case to the table-driven test. *Hint:* the zero value of `Suffix` is `""`, so existing callers keep working without changes.

> 🏋️ **Exercise 3: Swap the store**
>
> In `wiring/`, write a second store type, `LoggingStore`, that wraps any `productStore` and logs each call before passing it on. Wire it in `main` between `NewMemoryStore()` and `NewCatalog`. *Hint:* `LoggingStore` holds a `productStore` field and has the same two methods. Nothing else needs to change. That's the payoff of small interfaces.

> 🏋️ **Exercise 4 (stretch): Translate a Spring service**
>
> Pick a small `@Service` class from a Spring project you know. Sketch the Go version: one struct, a `NewX` constructor taking its dependencies, methods that take `ctx context.Context` first and return `error`, and the lines in `main` that wire it. *Hint:* map each `@Autowired` dependency to a constructor parameter, and each exception to a returned error.

Further reading

- Go at Google: Language Design in the Service of Software Engineering: Rob Pike explains the problems Go was built to solve.
- Effective Go: the classic guide to idiomatic Go, including names and getters.
- Go Proverbs: the short sayings quoted throughout this chapter.
- Package names: the Go blog on naming packages and avoiding stutter.
- Errors are values: why Go treats errors as ordinary values.
- Go Code Review Comments: the checklist many teams use in code reviews.

CHAPTER 2

Setting Up and Your First Go Program

> *One small download gives you a compiler, a build tool, a test runner, a formatter and a documentation viewer, so you'll be running real Go code within the hour.*

🎯 **What you'll learn**

- How to install Go on Windows, macOS or Linux and check that it works
- How Go modules compare with Maven's `pom.xml` and Gradle builds
- The everyday `go` commands, mapped to the Maven and Gradle tasks you know
- How Go pins its own version (`toolchain`, `GOTOOLCHAIN`) and its tools (`tool` lines)
- How to set up VS Code or GoLand, and debug with Delve
- How to build "Hello, Orderly": a CLI and a tiny JSON web server

The big picture

In Java, a typical setup is a JDK, Maven or Gradle, a pile of plugins (Surefire, Checkstyle, Spotless, Javadoc), and an IDE that understands all of it. Each piece has its own configuration and its own version to manage.

Go ships all of that as **one command**: `go`. It compiles, downloads dependencies, runs tests, formats code, checks for bugs and shows documentation. There's no XML or Groovy build script, because the source code and one small `go.mod` file already contain everything the tool needs.

🧩 **Think of it like a Swiss Army knife**

Maven plus its plugins is a full toolbox: powerful, but you have to pick the right tool and keep each one sharp. The `go` command is a Swiss Army knife: fewer, simpler blades that all fold into one handle and always fit together.

Concept	Java / Spring	Go
Install	JDK + Maven or Gradle	One Go download
Project file	`pom.xml`, `build.gradle`	`go.mod` (plus `go.sum`)
Dependency coordinates	`groupId:artifactId:version`	Module path + version
Artifact repository	Maven Central, Nexus	Module proxy, or the Git repo itself
Build output	JAR that needs a JVM	One native binary, no runtime needed
Formatter	Spotless, IDE settings	`gofmt` (built in)

Concept	Java / Spring	Go
Multi-module build	Parent POM, `settings.gradle`	`go.work` workspace

Installing Go

The official downloads live at https://go.dev/dl/. At the time of writing the newest release is **Go 1.27.3**; this guide's examples need Go 1.25 or newer. Pick whichever install route suits you.

Windows

Download the `.msi` installer from go.dev/dl and run it. It installs Go into `C:\Program Files\Go` and adds it to your `PATH`. If you prefer a package manager, winget works too:

TERMINAL

```
winget install GoLang.Go
```

Open a *new* terminal afterwards, so it picks up the updated `PATH`.

macOS

Download the `.pkg` installer for your chip (Apple silicon is `arm64`, Intel is `amd64`) and run it. Or use Homebrew:

TERMINAL

```
brew install go
```

Linux

Download the tarball, unpack it into `/usr/local`, and add Go's `bin` folder to your `PATH`. These are the commands from the official install page:

TERMINAL

```
sudo rm -rf /usr/local/go
sudo tar -C /usr/local -xzf go1.27.3.linux-amd64.tar.gz
echo 'export PATH=$PATH:/usr/local/go/bin' >> ~/.profile
source ~/.profile
```

Removing the old folder first matters: unpacking on top of an older Go can leave a broken mix of files.

Check that it works

TERMINAL

```
go version
```

OUTPUT

```
go version go1.25.0 windows/amd64
```

That's your first small win. The last part, `windows/amd64`, is the operating system and CPU architecture the tool is running on.

`go env`: the settings worth knowing

`go env` prints every setting the Go tool uses. You can ask for specific ones:

TERMINAL

```
go env GOROOT GOPATH GOMODCACHE GOPROXY GOSUMDB
```

OUTPUT

```
C:\Program Files\Go
C:\Users\pshrivastava7\go
C:\Users\pshrivastava7\go\pkg\mod
https://proxy.golang.org,direct
sum.golang.org
```

Variable	What it means	Java cousin
`GOROOT`	Where Go itself is installed	`JAVA_HOME`
`GOPATH`	Go's home folder (`~/go`); `go install` puts binaries in its `bin`	`~/.m2`
`GOMODCACHE`	Downloaded dependencies	`~/.m2/repository`
`GOPROXY`	Where modules are downloaded from	Maven mirror
`GOOS`, `GOARCH`	Target OS and CPU for builds	(no equivalent)

You can change a setting permanently with `go env -w`, for example `go env -w GOPRIVATE=dev.azure.com/acme`. It's stored in Go's own config file, so you don't have to edit your shell profile.

> **You don't need to set GOPATH**
>
> Old tutorials tell you to put all your code inside `GOPATH`. That was before modules (Go 1.11 in 2018). Today you can keep projects anywhere on disk. Add `$GOPATH/bin` (usually `~/go/bin`) to your `PATH` so tools you install are easy to run.

Modules: Go's answer to `pom.xml`

A **module** is a collection of Go packages that are versioned and released together. It's the unit of dependency management, like a Maven artifact. A module is defined by a `go.mod` file at its root.

Creating a module

TERMINAL

```
mkdir orderly && cd orderly
go mod init example.com/orderly
```

OUTPUT

```
go: creating new go.mod: module example.com/orderly
```

The resulting `go.mod` is three lines:

GO.MOD go.mod

```
module example.com/orderly

go 1.25.0
```

The **module path** (`example.com/orderly`) does two jobs. It's the import prefix for every package in the module, and, for public code, it's where the Go tool can find the source: `github.com/google/uuid` really is a GitHub repository. Compare that with Maven, where `groupId` and `artifactId` are names that a repository *maps* to files.

Maven	Go
`<groupId>com.google.guava</groupId>`	(part of the module path)
`<artifactId>guava</artifactId>`	(part of the module path)
`<version>33.0.0-jre</version>`	`v1.6.0`, from a Git tag
Published to Maven Central	Published by pushing a Git tag

Use `github.com/yourname/project` (or your company's Git host) for real code, and `example.com/...` for experiments.

> 💡 **Pin the go line to your team's baseline**
>
> The `go` line says which Go version the module needs at minimum. Depending on your Go version, `go mod init` writes either your exact version or a slightly older one. Set it to your team's baseline with `go mod edit -go=1.25.0`. Every example in this guide uses `go 1.25.0`.

Adding a dependency

You don't edit `go.mod` by hand to add a dependency. Ask for it with `go get`:

TERMINAL

```
go get github.com/google/uuid
```

OUTPUT

```
go: added github.com/google/uuid v1.6.0
```

GO.MOD go.mod

```
module example.com/orderly

go 1.25.0

require github.com/google/uuid v1.6.0 // indirect
```

The `// indirect` comment appears because no code imports `uuid` yet. Once you `import "github.com/google/uuid"` in a file and run `go mod tidy`, the comment disappears.

`go mod tidy` is the command you'll run most. It reads your source code, adds any missing requirements, removes unused ones, and updates `go.sum`. Run it after you add or remove imports; there's no separate "install dependencies" step.

Versions, and v2 and beyond

Go modules use semantic versioning: `v1.6.0` means major 1, minor 6, patch 0. Go adds one rule Maven doesn't have, called **semantic import versioning**: from v2 onwards, the major version becomes part of the module path.

GO

```
import "github.com/jackc/pgx/v5" // the Postgres driver, major version 5
```

Why? A new major version is allowed to break its API, so Go treats it as a *different module*. Your program can even use `pgx/v4` and `pgx/v5` side by side during a migration. You'll see `/v2`, `/v5` and so on in import paths all the time; now you know why.

> **Coming from Java**
>
> Maven resolves version conflicts with "nearest definition wins", and Gradle picks the highest version it sees. Go uses **minimal version selection**: for each module it takes the highest version that anything in your build *requires*, and never a newer one. Builds are repeatable without a lock file, and an upgrade only happens when someone asks for it.

`go.sum` and the checksum database

`go.sum` records a cryptographic hash for each dependency version:

go.sum

```
github.com/google/uuid v1.6.0 h1:NIvaJDMOsjHA8n1jAhLSgzrAzy1Hgr+hNrb57e+94F0=
github.com/google/uuid v1.6.0/go.mod h1:TIyPZe4MgqvfeYDBFedMoGGpEw/LqOeaOT+nhxU+yHo=
```

The first line covers the module's files, the second its `go.mod`. If a download ever doesn't match, the build stops. Commit `go.sum` to Git, always.

On first download, Go also checks the hash against a public, append-only **checksum database** (`sum.golang.org`). That protects everyone from a tag being quietly moved to different code. Three environment variables control where modules come from:

- `GOPROXY`: the download mirror. The default, `https://proxy.golang.org,direct`, tries Google's public proxy and falls back to the Git repository. Companies often point it at Artifactory or Athens, like a Nexus mirror.
- `GOSUMDB`: the checksum database to verify against (default `sum.golang.org`).
- `GOPRIVATE`: module path patterns that should skip both the public proxy and the checksum database, such as `GOPRIVATE=github.com/acme/*`. You need this for company code. Chapter 19 covers private modules in detail.

The `go` command: your whole build tool

Here are the commands you'll use every day, with their closest Maven and Gradle equivalents.

Go	What it does	Maven / Gradle
`go run ./cmd/app`	Compile and run in one step	`spring-boot:run` / `bootRun`
`go build ./...`	Compile every package	`mvn compile` / `gradle build`
`go test ./...`	Run all tests	`mvn test` / `gradle test`
`gofmt -w .` (or `go fmt ./...`)	Format all code	Spotless apply
`go vet ./...`	Find likely bugs	SpotBugs, Error Prone
`go install pkg@version`	Build and install a binary	(none; like a global CLI install)
`go doc strings.Cut`	Show documentation	Javadoc in the IDE
`go mod tidy`	Sync dependencies with imports	`dependency:analyze` + edit
`go generate ./...`	Run code generators	`generate-sources` phase

The `./...` pattern means "this folder and everything below it". You'll type it many times a day.

`go vet` deserves a special mention. It's fast, built in, and catches real mistakes. Given this deliberately wrong line:

GO Avoid

```go
fmt.Printf("SKU: %d\n", sku) // sku is a string
```

`go vet` replies:

OUTPUT

```
.\main.go:7:19: fmt.Printf format %d has arg sku of wrong type string
```

And `go doc` brings documentation to your terminal, with no browser needed:

TERMINAL

```
go doc strings.Cut
```

OUTPUT

```
package strings // import "strings"

func Cut(s, sep string) (before, after string, found bool)
    Cut slices s around the first instance of sep, returning the text before and
    after sep. The found result reports whether sep appears in s. If sep does
    not appear in s, cut returns s, "", false.
```

Go versions: the `go` line, `toolchain` and `GOTOOLCHAIN`

Java teams manage JDK versions with SDKMAN, toolchains in Maven, or Docker images. Go builds this into the `go` command itself.

- The **`go` line** in `go.mod` is the *minimum* Go version the module needs.
- An optional **`toolchain` line** (for example `toolchain go1.27.3`) says which Go release the team *prefers* for working on the module.
- The `GOTOOLCHAIN` environment variable decides what to do when the installed Go is older than the module needs. The default, `auto`, downloads the right release, caches it and switches to it automatically.

Here's what that looks like in practice. This module says `go 1.25.0`, but the newest `golang.org/x/text` needs Go 1.26:

TERMINAL

```
go get golang.org/x/text@latest
```

OUTPUT

```
go: golang.org/x/text@v0.43.0 requires go >= 1.26.0; switching to go1.26.10
go: upgraded go 1.25.0 => 1.26.0
go: added golang.org/x/text v0.43.0
```

Go fetched a newer toolchain *and* raised your module's `go` line. Handy, but it means a dependency upgrade quietly raised the minimum Go version for everyone using your module.

> ⚠️ **Gotcha: `@latest` can raise your Go version**
>
> If your team must stay on a baseline (this guide uses 1.25), check `go.mod` after every `go get`. To stay put, ask for a version that supports your baseline, such as `go get golang.org/x/text@v0.41.0`. You can also set `GOTOOLCHAIN=local` in CI, so builds fail loudly instead of switching.

GOTOOLCHAIN value	Behaviour
`auto` (default)	Use the local Go, or download a newer one if `go.mod` needs it
`local`	Always use the installed Go; fail if it's too old
`go1.27.3`	Always use exactly this release (downloaded if needed)

That last form is handy for trying a newer Go without installing it: `GOTOOLCHAIN=go1.27.3 go test ./...` is how this guide checks every example on two Go versions.

Tools in `go.mod` (Go 1.24+)

Java projects pin build-time tools as Maven plugins: a code generator, a linter, an OpenAPI generator. Go 1.24 added the same idea to `go.mod` with **`tool` lines**. Let's use `stringer`, a small official generator that writes `String()` methods for constants.

TERMINAL

```
go get -tool golang.org/x/tools/cmd/stringer@v0.49.0
```

GO.MOD go.mod

```
module example.com/goguide/ch02

go 1.25.0

tool golang.org/x/tools/cmd/stringer

require (
    golang.org/x/mod v0.39.0 // indirect
    golang.org/x/sync v0.22.0 // indirect
    golang.org/x/tools v0.49.0 // indirect
)
```

The tool's version is now locked like any other dependency, and every teammate runs the same one with `go tool stringer`. (We pinned v0.49.0 because newer `x/tools` releases need Go 1.26, as in the gotcha above.)

Code generators usually run through `go generate`, which looks for special comments:

GO shipping/shipping.go

```go
// Package shipping holds delivery options for Orderly orders.
package shipping

//go:generate go tool stringer -type=Speed

// Speed is how fast an order is delivered.
type Speed int

const (
    Standard Speed = iota
    Express
    Overnight
)
```

Running `go generate ./...` creates `shipping/speed_string.go`, so `Express.String()` returns `"Express"`. Generated files are committed to Git, so a normal `go build` never needs the generator.

Coming from Java

`go generate` is never run automatically by `go build`, unlike a Maven plugin bound to `generate-sources`. That's deliberate: builds stay fast and predictable, and anyone can build your code without installing generators.

Workspaces: `go.work` for multi-module work

Sometimes you work on two modules at once: say, `orderly-api` and a shared `orderly-shared` library it imports. In Maven you'd use a parent POM with `<modules>`. In Go you create a **workspace**:

TERMINAL

```
go work init ./orderly-api ./orderly-shared
```

go.work

```
go 1.25.0

use (
    ./orderly-api
    ./orderly-shared
)
```

Now, inside the workspace, `orderly-api` uses your *local* copy of `orderly-shared` instead of the published version. Edit both, run tests, and commit each module when you're done.

> 💡 **Usually, don't commit `go.work`**
>
> A workspace is a personal, local convenience. Most teams add `go.work` to `.gitignore`, so CI builds each module exactly as `go.mod` describes. If you're unsure, start with a single module; most services never need more.

Editor setup and debugging

Every good Go editor talks to **gopls** (say "go please"), the official Go language server. It powers completion, go-to-definition, rename, inline errors and automatic imports.

- **VS Code** is free and very popular. Install the "Go" extension published by the Go team at Google. When it offers to install tools (gopls, Delve, staticcheck), say yes.
- **GoLand** from JetBrains will feel like home if you use IntelliJ IDEA. It has its own Go engine, great refactoring, and a familiar debugger. IntelliJ IDEA Ultimate can use the same Go plugin.

Either way, turn on **format on save**. Go code is always gofmt-formatted, and gopls also tidies your imports when you save.

Debugging with Delve

Delve (`dlv`) is Go's debugger, and both editors use it under the hood. Pressing F5 in VS Code creates a launch configuration like this:

JSON .vscode/launch.json

```json
{
  "version": "0.2.0",
  "configurations": [
    {
      "name": "Launch hello-http",
      "type": "go",
      "request": "launch",
      "mode": "auto",
      "program": "${workspaceFolder}/hello-http"
    }
  ]
}
```

You can also use Delve from the terminal. Install it once, then debug any package:

TERMINAL

```
go install github.com/go-delve/delve/cmd/dlv@latest
dlv debug ./hello-cli
```

A short session looks like this (output trimmed and paths shortened):

OUTPUT

```
(dlv) break main.main
Breakpoint 1 set at 0x7ff7e65a4c76 for main.main() ./hello-cli/main.go:23
(dlv) continue
> [Breakpoint 1] main.main() ./hello-cli/main.go:23
(dlv) next
(dlv) next
(dlv) print p.SKU
"MUG-BLUE-01"
(dlv) print p.PriceCents
1299
```

`break`, `continue`, `next`, `step` and `print` cover most debugging sessions, much like the buttons in IntelliJ's debugger.

Hello, Orderly: your first program

Time to write code. We'll build two small programs for Orderly: a CLI that prints a product, then a web server that returns it as JSON.

Step 1: A program that prints a product

Create `hello-cli/main.go` inside your module:

GO hello-cli/main.go

```go
// Command hello-cli prints one Orderly product to the terminal.
package main

import (
    "fmt"
    "time"
)

// Product is something Orderly sells.
type Product struct {
    ID         string
    SKU        string
    Name       string
    PriceCents int64 // money as whole cents, never float64
    CreatedAt  time.Time
}

// formatPrice turns 1299 into "$12.99".
func formatPrice(cents int64) string {
    return fmt.Sprintf("$%d.%02d", cents/100, cents%100)
}
```

Let's walk through it:

- `package main` marks this package as a program, not a library. Every Go file starts with a `package` line, and all files in one folder share the same package.

- `import` lists the packages this file uses. `fmt` handles formatted printing (like `String.format`) and `time` handles dates. Unused imports are compile errors, which keeps files tidy.
- `type Product struct` declares a struct: a named group of fields, like a Java record. The type comes *after* the name: `Name string`.
- **Capital letters matter.** `Product` and `PriceCents` start with upper case, so other packages can use them, like `public`. Lower-case names, like `formatPrice`, are private to the package.
- `int64` **for money.** Floating-point numbers can't represent most decimal fractions exactly, so Orderly stores whole cents (Chapter 19 covers money properly).
- `func formatPrice(cents int64) string` is a function taking an `int64` and returning a `string`. `%02d` pads the cents to two digits.

Now the `main` function, where the program starts:

GO hello-cli/main.go (continued)

```go
func main() {
    p := Product{
        ID:         "8f14e45f-ceea-4e7a-9b1c-3d2a6c1e9b01",
        SKU:        "MUG-BLUE-01",
        Name:       "Blue Coffee Mug",
        PriceCents: 1299,
        CreatedAt:  time.Date(2026, time.November, 2, 9, 30, 0, 0, time.UTC),
    }

    fmt.Println("Hello, Orderly!")
    fmt.Printf("%s (%s) costs %s\n", p.Name, p.SKU, formatPrice(p.PriceCents))
    fmt.Println("In the catalog since", p.CreatedAt.Format("2 January 2006"))
}
```

- `func main()` takes no arguments and returns nothing. It's `public static void main(String[] args)` without the ceremony; command-line arguments live in `os.Args` if you need them.
- `p := Product{...}` creates a value. `:=` declares a variable and infers its type, like Java's `var`. Named fields make the literal read like a builder.
- **The trailing commas** on each line are required when a literal spans several lines. They make diffs cleaner, and gofmt expects them.
- `fmt.Printf` works like `System.out.printf`: `%s` for strings, `%d` for integers.
- `Format("2 January 2006")` looks strange. Go formats dates by example, using one fixed reference date; Chapter 3 explains this.

Step 2: Run it, build it, test it

TERMINAL

```
go run ./hello-cli
```

OUTPUT

```
Hello, Orderly!
Blue Coffee Mug (MUG-BLUE-01) costs $12.99
In the catalog since 2 November 2026
```

Your first Go program works. `go run` compiled it to a temporary folder and ran it. To get a real binary instead:

TERMINAL

```
go build -o bin/ ./hello-cli
./bin/hello-cli
```

The result is a single native executable (`hello-cli.exe` on Windows). It needs no JVM and no runtime: copy it to another machine with the same OS and it runs.

Tests live next to the code, in files ending with `_test.go`:

GO hello-cli/main_test.go

```go
package main

import "testing"

func TestFormatPrice(t *testing.T) {
    cases := map[int64]string{
        1299: "$12.99",
        500:  "$5.00",
        7:    "$0.07",
    }
    for cents, want := range cases {
        if got := formatPrice(cents); got != want {
            t.Errorf("formatPrice(%d) = %q, want %q", cents, got, want)
        }
    }
}
```

TERMINAL

```
go test -v ./hello-cli
```

OUTPUT

```
=== RUN   TestFormatPrice
--- PASS: TestFormatPrice (0.00s)
PASS
ok      example.com/goguide/ch02/hello-cli  3.758s
```

No JUnit, no assertion library, no runner plugin: a test is any function named `TestXxx` that takes `*testing.T`. Chapter 15 covers testing in depth.

Step 3: A tiny JSON web server

Now let's serve that product over HTTP, using only the standard library. First the data:

GO hello-http/main.go

```go
// Product is something Orderly sells. The tags name the JSON fields.
type Product struct {
    ID         string    `json:"id"`
    SKU        string    `json:"sku"`
    Name       string    `json:"name"`
    PriceCents int64     `json:"price_cents"`
    CreatedAt  time.Time `json:"created_at"`
}

// products is a pretend database, keyed by product ID.
var products = map[string]Product{
    "p-1001": {
        ID:         "p-1001",
        SKU:        "MUG-BLUE-01",
        Name:       "Blue Coffee Mug",
        PriceCents: 1299,
        CreatedAt:  time.Date(2026, 11, 2, 9, 30, 0, 0, time.UTC),
    },
}
```

- **The back-quoted strings** after each field are *struct tags*. `encoding/json` reads them to name the JSON fields, much like Jackson's `@JsonProperty`.
- `map[string]Product` is Go's built-in hash map, like `HashMap<String, Product>`. Real Orderly IDs are UUIDs; we use a short one so the commands below stay readable.

Next, the handler and the routes:

GO hello-http/main.go (continued)

```go
// getProduct handles GET /api/v1/products/{id}.
func getProduct(w http.ResponseWriter, r *http.Request) {
    id := r.PathValue("id")
    p, ok := products[id]
    if !ok {
        http.Error(w, "product not found", http.StatusNotFound)
        return
    }
    w.Header().Set("Content-Type", "application/json")
    if err := json.NewEncoder(w).Encode(p); err != nil {
        log.Printf("encode product %s: %v", id, err)
    }
}

// newMux registers every route. Tests call it too.
func newMux() *http.ServeMux {
    mux := http.NewServeMux()
    mux.HandleFunc("GET /api/v1/products/{id}", getProduct)
    return mux
}
```

- **A handler** is any function with the signature `func(http.ResponseWriter, *http.Request)`. Think of it as a `@GetMapping` method without the annotation. You write the response to `w`.
- `"GET /api/v1/products/{id}"` registers both the method and a path pattern with a wildcard (Go 1.22+). `r.PathValue("id")` reads it, like `@PathVariable`.
- `p, ok := products[id]` is the "comma-ok" form: `ok` is `false` if the key is missing.
- `json.NewEncoder(w).Encode(p)` streams the struct straight to the response as JSON. It returns an error, and Go asks you to deal with it, so we log it.

Finally, `main` starts the server:

GO hello-http/main.go (continued)

```go
func main() {
    srv := &http.Server{
        Addr:              ":8080",
        Handler:           newMux(),
        ReadHeaderTimeout: 5 * time.Second,
    }
    log.Printf("Orderly is listening on http://localhost%s", srv.Addr)
    log.Fatal(srv.ListenAndServe())
}
```

- `http.Server` is the server itself; there's no Tomcat to embed. Setting `ReadHeaderTimeout` protects against clients that open a connection and send headers very slowly. Chapter 14 covers timeouts and graceful shutdown properly.
- `log.Fatal` prints the error and exits if the server stops, for example because port 8080 is already taken.

Step 4: Try it

Start the server with `go run ./hello-http`, then call it from a second terminal:

TERMINAL

```
curl -s localhost:8080/api/v1/products/p-1001
```

OUTPUT

```
{"id":"p-1001","sku":"MUG-BLUE-01","name":"Blue Coffee Mug","price_cents":1299,
"created_at":"2026-11-02T09:30:00Z"}
```

(The response is one line; we wrapped it to fit the page.) An unknown ID gets a 404, and the wrong method gets a 405 for free:

TERMINAL

```
curl -si -X POST localhost:8080/api/v1/products/p-1001 | head -3
```

OUTPUT

```
HTTP/1.1 405 Method Not Allowed
Allow: GET, HEAD
Content-Type: text/plain; charset=utf-8
```

Congratulations: that's a working JSON API in about 60 lines, with no framework at all. The example folder also has tests that call `newMux()` with `httptest`, without opening a real port.

Full code: *`examples/ch02-toolchain/hello-cli/`* *and* *`examples/ch02-toolchain/hello-http/`*

Anatomy of a Go project folder

Here's the example folder for this chapter, plus the folders you'll add as a real service grows:

```
ch02-toolchain/
  go.mod              module path, Go version, dependencies, tools
  go.sum              checksums for every dependency (commit it)
  Makefile            short names for common commands
  hello-cli/
    main.go           package main: a program
    main_test.go      tests sit next to the code
  hello-http/
    main.go
    main_test.go
  shipping/           a library package (not main)
    shipping.go
    speed_string.go   generated by stringer
  # in a bigger service you'll also see:
  cmd/orderly/        one folder per binary, each with a main.go
  internal/           private packages only this module can import
  migrations/         SQL files, often embedded into the binary
```

A few rules make this work:

- **One folder is one package.** All `.go` files in a folder share a `package` name. There's no `src/main/java`.
- **The import path is the module path plus the folder:** `example.com/goguide/ch02/shipping`.
- **`internal/` is enforced by the compiler**, so other modules can't import your private code.
- **`cmd/<name>/main.go`** is the convention for binaries, so one module can build several programs.

Chapter 6 goes much deeper into structuring an enterprise project.

Cross-compiling: one command, any platform

Here's a feature Java developers tend to love. Go can build a binary for another operating system or CPU from your laptop, with no extra toolchain. Set two environment variables:

TERMINAL

```
GOOS=linux GOARCH=arm64 go build -o bin/hello-http-linux-arm64 ./hello-http
```

That produces a Linux binary for ARM servers (like AWS Graviton) while you're sitting on Windows or a Mac. `go tool dist list` shows all supported targets; there are dozens. This is why Go is such a good fit for CLIs and containers. Chapter 18 builds tiny container images this way.

> 📝 **One condition: no cgo**
>
> Cross-compiling is this easy when your code is pure Go. If a dependency uses cgo (Go calling C code), you need a C cross-compiler too. Most Go libraries avoid cgo for exactly this reason.

A tiny Makefile

Go doesn't need a build script, but teams like short names for long commands. A small Makefile is the most common choice:

MAKEFILE Makefile

```
.PHONY: all fmt vet test build run generate clean

all: fmt vet test build

fmt:
    gofmt -l -w .

vet:
    go vet ./...

test:
    go test ./...

build:
    go build -o bin/ ./...

run:
    go run ./hello-http

generate:
    go generate ./...

clean:
    rm -rf bin
```

Now `make` (or `make all`) formats, vets, tests and builds everything. Recipe lines must start with a **tab**, not spaces; that's the classic Makefile trap. On Windows, `make` comes with Git Bash setups, WSL or Chocolatey. Chapter 16 builds a fuller Makefile with linting and coverage.

Full code: `examples/ch02-toolchain/`

Recap

- Install Go from go.dev/dl (or winget, Homebrew, or the Linux tarball), then check it with `go version`.

- A module is defined by `go.mod`; its path is both the import prefix and, for public code, where the source lives.
- Use `go get` to add dependencies and `go mod tidy` to keep `go.mod` and `go.sum` in sync. Commit both files.
- From v2 onwards, the major version is part of the import path (`pgx/v5`).
- `go run`, `build`, `test`, `vet`, `fmt`, `doc`, `install` and `generate` replace most of Maven and its plugins.
- The `go` line sets the minimum Go version; `GOTOOLCHAIN` controls switching. Watch out for `@latest` raising your `go` line.
- `tool` lines in `go.mod` pin build tools; `go.work` lets you develop several modules together.
- One `GOOS=linux GOARCH=arm64 go build` gives you a binary for another platform.

Try it yourself

Exercise 1: A second product

Add a second product (`TEE-RED-XL`, "Red T-Shirt", 2500 cents) to the `products` map in `hello-http`, and fetch it with curl. *Hint:* the map key must match the ID in the URL.

Exercise 2: List all products

Add a `GET /api/v1/products` route that returns every product as a JSON array. Add a test for it next to the existing ones. *Hint:* build a `[]Product` by ranging over the map, then encode the slice. Map order is random, so sort the slice by ID with `slices.SortFunc` if you want stable output.

Exercise 3: Explore the tools

Run `go doc net/http.Server`, then `go vet ./...` and `GOTOOLCHAIN=go1.27.3 go version` (or another release) in the example folder. What did `GOTOOLCHAIN` do the first time? *Hint:* look in `go env GOMODCACHE` for a `golang.org/toolchain@...` folder.

Exercise 4 (stretch): Ship it everywhere

Add a `release` target to the Makefile that builds `hello-http` for `linux/amd64`, `linux/arm64`, `darwin/arm64` and `windows/amd64` into `bin/`, with the OS and architecture in each file name. *Hint:* use one `GOOS=... GOARCH=... go build -o ...` line per target.

Further reading

- Download and install: the official install steps for every platform.
- Tutorial: Get started with Go: a gentle first module, from the Go team.
- Go Modules Reference: everything about `go.mod`, `go.sum`, proxies and private modules.

- Go Toolchains: how the `go` and `toolchain` lines and `GOTOOLCHAIN` work.
- Managing dependencies: day-to-day `go get` and `go mod tidy`.
- gopls documentation: features and settings of the Go language server.
- Delve: the Go debugger's home page and command reference.

CHAPTER 3

Language Basics: Types, Structs, Pointers, Slices and Maps

> *This is the chapter you'll come back to most: the everyday building blocks of Go, each one explained next to the Java you already know.*

🎯 **What you'll learn**

- How Go declares variables and constants, and why zero values replace `null`
- Go's control flow: `if` with a short statement, `for` as the only loop, and `switch`
- Functions with multiple results, closures, and when to use named results
- Structs and methods, value vs pointer receivers, and the `NewX` constructor convention
- Slices and maps in depth, including the two gotchas that bite every newcomer
- Strings, equality and the `time` package, compared with `java.time`

The big picture

Go's basic toolbox is small. There are structs instead of classes, slices instead of `ArrayList`, maps instead of `HashMap`, and a handful of basic types. You can hold the whole language in your head, which is exactly what makes Go code so easy to read later.

But "small" doesn't mean "the same as Java". Assignment copies whole structs, slices share memory, and most types have no `null`. We'll go through each one slowly, using the Orderly types from the rest of the book.

🧩 **Think of it like a well-organised workshop**

Java hands you a huge catalogue of power tools. Go gives you a small workbench with a dozen excellent hand tools, each with a clear job. You'll reach for the same few every day, so it's worth learning exactly how each one feels in your hand.

Concept	Java	Go
Data type	Class, record (POJO)	`struct`
Missing value	`null`	Zero value (`0` , `""` , `false` , `nil`)
Constructor	`new Order(...)`	`NewOrder(...)` function
Growable list	`ArrayList<T>`	Slice `[]T`

Concept	Java	Go
Hash map	`HashMap<K, V>`	`map[K]V`
Set	`HashSet<T>`	`map[T]struct{}`
Lambda	`x -> x * 2`	`func(x int) int { return x * 2 }`
Object equality	`equals()` + `hashCode()`	`==` on comparable types
Date and time	`java.time.Instant`, `Duration`	`time.Time`, `time.Duration`

Full code for this chapter: `examples/ch03-basics/`, one small program per section.

Variables and constants

Declaring variables

Go has two ways to declare a variable. The long form uses `var`; the short form uses `:=` and only works inside functions.

GO variables/main.go

```go
var sku string = "MUG-BLUE-01" // full form
var quantity = 3               // type inferred (int)
price := int64(1299)           // short form, only inside functions
fmt.Println(sku, quantity, price)
```

`:=` is like Java's `var`: it declares and infers in one step. You'll use it for almost every local variable. Use `var` when you want the zero value (`var total int64`) or at package level.

> **Coming from Java**
>
> Notice the order: the name comes first, then the type (`sku string`, not `String sku`). It reads left to right as "sku is a string". It feels backwards for about a day, and then it feels natural.

Go is strict about unused things. An unused local variable or import is a compile error, not a warning:

OUTPUT

```
.\main.go:21:2: declared and not used: x
```

It's annoying for a week and lovely after that.

Basic types

Java	Go	Notes
`int` (32-bit)	`int32`	Go's plain `int` is 64-bit on 64-bit machines
`long`	`int64`	Use for money in cents and IDs
`double`, `float`	`float64`, `float32`	Prefer `float64`

Java	Go	Notes
`boolean`	`bool`	
`char` (UTF-16 unit)	`rune` (= `int32`)	One Unicode code point
`byte` (signed)	`byte` (= `uint8`)	Unsigned in Go: 0 to 255
`String`	`string`	Immutable UTF-8 bytes
(none)	`uint`, `uint8` … `uint64`	Unsigned integers exist

If you're unsure, use `int` for counts and indexes, `int64` for money and anything stored in a `bigint` column, and `float64` for measurements. Integer overflow wraps around silently, as in Java: an `int8` holding 127 becomes -128 after `++`.

Zero values instead of null

Every Go variable starts with a usable **zero value**: `0` for numbers, `""` for strings, `false` for bools, and `nil` for pointers, slices, maps, channels, functions and interfaces.

GO variables/main.go

```go
var (
    name    string
    count   int
    ok      bool
    ratio   float64
    pointer *int
)
fmt.Printf("%q %d %t %g %v\n", name, count, ok, ratio, pointer)
```

OUTPUT

```
"" 0 false 0 <nil>
```

A `string` can never be `null`, and neither can an `int` or a struct. That removes a whole family of `NullPointerException`s. Go's culture builds on this with a proverb: **make the zero value useful**. A `strings.Builder`, a `sync.Mutex` or an empty slice are all ready to use the moment you declare them.

> ⚠️ **Gotcha: zero can be a real value**
>
> Because `0` and `""` are real values, you can't tell "not set" from "set to zero" in a plain `int` field. When the difference matters (an optional discount, a nullable column), use a pointer (`*int64`) or a type like `sql.NullInt64`. Chapter 9 shows both.

Constants and `iota`

Constants are values fixed at compile time. Untyped constants are flexible: `MaxItemsPerOrder` below can be used as an `int`, an `int64` or a `float64` without conversion.

GO variables/main.go

```go
const (
    StoreName               = "Orderly" // untyped string constant
    MaxItemsPerOrder        = 50        // untyped integer constant
    FreeShippingCents int64 = 5000      // typed constant
)

// Priority is an enum-like type. iota counts 0, 1, 2, ...
type Priority int

const (
    Low Priority = iota
    Normal
    High
)
```

`iota` is a counter that starts at 0 in each `const` block and goes up by one per line. It's the usual way to build an integer enum. For enums that are stored or sent over the wire, Orderly uses a named *string* type instead (`OrderStatus`, coming up below), so the values stay readable in JSON and the database.

> **Coming from Java**
>
> Go has no `enum` keyword with methods and fields per constant. A named type plus constants covers most uses, and you can still add methods to the named type, as we'll do with `OrderStatus.IsFinal()`.

Explicit conversions

Go never converts between types on its own, not even from `int` to `int64`. You convert explicitly with `T(value)`:

GO variables/main.go

```go
var qty int = 3
var unit int64 = 1299
total := int64(qty) * unit // int * int64 would not compile
```

If you forget, the compiler tells you plainly:

OUTPUT

```
.\main.go:20:14: invalid operation: qty * unit (mismatched types int and int64)
```

Strings and numbers convert with the `strconv` package, not with a cast:

GO variables/main.go

```go
n, err := strconv.Atoi("42")
fmt.Println(n, err, strconv.Itoa(7)+" items") // 42 <nil> 7 items
```

> ⚠️ **Gotcha: `string(65)` is not `\"65\"`**
>
> Converting an integer to `string` gives the character with that code point, so `string(rune(65))` is `"A"`. To turn a number into its digits, use `strconv.Itoa` or `fmt.Sprint`. `go vet` warns you if you write `string(someInt)`.

Control flow

`if` with a short statement

Go's `if` needs no parentheses, always needs braces, and can start with a short statement. Variables declared there only exist inside the `if` and its `else`:

GO control/main.go

```go
if qty, err := strconv.Atoi("3"); err != nil {
    fmt.Println("bad quantity:", err)
} else {
    fmt.Println("quantity is", qty)
}
```

You'll see this shape everywhere with errors: `if err := doThing(); err != nil { ... }`. It keeps `err` tightly scoped.

`for` is the only loop

Go has one loop keyword, `for`, and it covers every shape you know from Java:

GO control/main.go

```go
for i := 0; i < 3; i++ { // classic three-part loop
    fmt.Print(i, " ")
}

stock := 3
for stock > 0 { // a "while" loop
    stock--
}

for i := range 3 { // range over an int (Go 1.22+): 0, 1, 2
    fmt.Print(i, " ")
}

skus := []string{"MUG-BLUE-01", "TEE-RED-XL"}
for i, sku := range skus { // like for-each, with the index too
    fmt.Println(i, sku)
}
```

`for { ... }` with no condition loops forever until you `break` or `return`. If you don't need the index in a `range` loop, write `for _, sku := range skus`. The underscore, called the *blank identifier*, means "I'm ignoring this value".

> **Version note: one variable per iteration**
>
> Since Go 1.22, each loop iteration gets a fresh copy of the loop variables. Before that, closures and goroutines started inside a loop all shared one variable, a famous source of bugs. With `go 1.22` or later in `go.mod`, that bug is gone.

`switch` without fall-through

Go's `switch` doesn't fall through, so there's no `break` to forget. One case can match several values, and cases can be any expression:

GO control/main.go

```go
switch status {
case "PENDING", "CONFIRMED":
    fmt.Println(status, "-> can still be cancelled")
case "SHIPPED":
    fmt.Println(status, "-> on its way")
default:
    fmt.Println(status, "-> unknown status")
}
```

A `switch` with no value at all (`switch { case total >= 5000: ... }`) is a neat replacement for a long `if` / `else if` chain. If you ever really want C-style fall-through, there's a `fallthrough` keyword, but you'll rarely need it.

Functions

Multiple results

A Go function can return several values. The most common pair is "a result and an error":

GO functions/main.go

```go
// splitSKU returns two values. Callers must deal with both.
func splitSKU(sku string) (product, variant string, err error) {
    before, after, found := strings.Cut(sku, "-")
    if !found {
        return "", "", errors.New("sku has no dash: " + sku)
    }
    return before, after, nil
}
```

The caller receives them all at once: `product, variant, err := splitSKU("MUG-BLUE-01")` gives `MUG`, `BLUE-01` and a `nil` error. In Java you'd return a small record or throw an exception. In Go, multiple results are so cheap that you'll use them all the time, especially `(value, error)`. Chapter 4 is all about errors.

Named results and naked returns

The results in `splitSKU` have names (`product, variant string, err error`). Names document what each result means, and they start as zero values. A bare `return` (a *naked return*) then returns their current values:

GO functions/main.go

```go
func priceRange(prices ...int64) (lowest, highest int64) {
    if len(prices) == 0 {
        return // returns 0, 0
    }
    return slices.Min(prices), slices.Max(prices)
}
```

Use named results when they make a signature clearer, and naked returns only in very short functions; in a long one, the reader has to scroll up to see what's returned.

Variadic functions

`prices ...int64` makes `priceRange` variadic, like Java's `long... prices`. Inside the function, `prices` is a slice. To pass an existing slice, spread it with `...`:

GO functions/main.go

```go
fmt.Println(priceRange(1299, 2500, 499)) // 499 2500
prices := []int64{1299, 2500}
fmt.Println(priceRange(prices...)) // 1299 2500
```

Functions are values, and closures

Functions in Go are ordinary values: you can store them in variables, pass them as arguments and return them. A *function type* names a signature, a bit like a one-method Java functional interface:

GO functions/main.go

```go
// discount is a function type, like a one-method Java interface.
type discount func(cents int64) int64

// percentOff returns a closure that remembers pct.
func percentOff(pct int64) discount {
    return func(cents int64) int64 {
        return cents - cents*pct/100
    }
}

// apply takes a function as a parameter.
func apply(cents int64, rules ...discount) int64 {
    for _, rule := range rules {
        cents = rule(cents)
    }
    return cents
}
```

A **closure** is a function that remembers variables from the place it was created; `percentOff(10)` returns a function that remembers `pct = 10`. Calling `apply(10000, percentOff(10), minusOneDollar)` gives `8900`.

Here's the big difference from Java lambdas. A Java lambda can only capture *effectively final* variables. A Go closure captures the variable itself and can change it:

GO functions/main.go

```go
counter := 0
next := func() int {
    counter++ // changes the outer variable
    return counter
}
next()
next()
fmt.Println("counter:", counter) // counter: 2
```

That's powerful, and it's also why you need care when closures run on several goroutines at once (Chapter 5).

`defer`: Go's `finally`

`defer` schedules a call to run when the surrounding function returns, however it returns. You'll use it to release resources right next to where you acquire them:

GO

```go
f, err := os.Open("orders.csv")
if err != nil {
    return err
}
defer f.Close() // runs when the function returns
```

It plays the role of `finally` and try-with-resources, and several deferred calls run last in, first out.

Structs: Go's POJOs and records

Declaring a struct

A **struct** is a named group of fields. It's Go's answer to the POJO and the record. Here is Orderly's product:

GO orderly/orderly.go

```go
// Product is something Orderly sells.
type Product struct {
    ID         string    `json:"id"`
    SKU        string    `json:"sku"`
    Name       string    `json:"name"`
    PriceCents int64     `json:"price_cents"` // never float64 for money
    CreatedAt  time.Time `json:"created_at"`
}
```

The back-quoted text after each field is a *struct tag*; we'll preview those near the end of the chapter. Here's the Java record it corresponds to:

JAVA

```
public record Product(UUID id, String sku, String name,
                      long priceCents, Instant createdAt) {}
```

Create a struct with a **composite literal**. Name the fields; any you leave out get their zero value:

GO structs/main.go

```
mug := orderly.Product{
    ID:         "p-1",
    SKU:        "MUG-BLUE-01",
    Name:       "Blue Mug",
    PriceCents: 1299,
}
fmt.Printf("%+v\n", mug)
```

OUTPUT

```
{ID:p-1 SKU:MUG-BLUE-01 Name:Blue Mug PriceCents:1299
CreatedAt:0001-01-01 00:00:00 +0000 UTC}
```

(The output is one line; we wrapped it to fit the page.)

See that `CreatedAt`? We didn't set it, so it holds the zero `time.Time`: midnight on 1 January in year 1. It's not `null`, it's a very old date. We'll meet it again in the time section.

Unlike a record, a struct is mutable: you can write `mug.Name = "Big Blue Mug"`. Go has no `final` fields; immutability is a convention, usually achieved with unexported fields.

⚠️ **Gotcha: assignment copies the whole struct**

In Java, `b = a` copies a *reference*, so both names point to one object. In Go, `b := a` copies *every field* into a new value:

GO

```
copyOfMug := mug
copyOfMug.PriceCents = 999
fmt.Println(mug.PriceCents, copyOfMug.PriceCents) // 1299 999
```

The same happens when you pass a struct to a function. If you want to share one value, pass a pointer (more on that shortly).

Methods

A method is a function with a **receiver**, written in brackets before the name. Methods are declared outside the struct, anywhere in the same package:

GO orderly/orderly.go

```go
// OrderStatus is a named string type, so the compiler can tell a
// status apart from any other string.
type OrderStatus string

const (
    StatusPending   OrderStatus = "PENDING"
    StatusConfirmed OrderStatus = "CONFIRMED"
    StatusShipped   OrderStatus = "SHIPPED"
    StatusCancelled OrderStatus = "CANCELLED"
)

// IsFinal reports whether no further changes are allowed.
// Value receiver: OrderStatus is a small value and this only reads it.
func (s OrderStatus) IsFinal() bool {
    return s == StatusShipped || s == StatusCancelled
}
```

You can add methods to any type you declare, not only structs. `OrderStatus` is a string underneath, but it has its own methods and the compiler won't let you mix it up with an ordinary string by accident.

Value receivers vs pointer receivers

This is the most important design decision in this section. A method's receiver can be a value (`s OrderStatus`) or a pointer (`o *Order`):

- A **value receiver** gets a *copy*. Changes inside the method are thrown away.
- A **pointer receiver** gets the *address* of the original, so it can change it.

Here's Orderly's order, whose methods change it:

GO orderly/orderly.go

```go
// AddItem adds a product to the order.
// Pointer receiver: it changes the order.
func (o *Order) AddItem(p Product, qty int) error {
    if o.Status != StatusPending {
        return ErrNotPending
    }
    // ... check that qty is positive ...
    o.Items = append(o.Items, OrderItem{
        ProductID:      p.ID,
        Quantity:       qty,
        UnitPriceCents: p.PriceCents,
    })
    return nil
}
```

`Order.TotalCents()` only reads the order, but it uses a pointer receiver too (`func (o *Order) TotalCents() int64`), so all of `Order`'s methods are consistent.

> 💡 **Rule of thumb for receivers**
>
> Use a **pointer receiver** if the method changes the value, if the struct is large, or if it contains a `sync.Mutex`. Use a **value receiver** for small, read-only types like `OrderStatus` or `time.Time`. Once any method needs a pointer, make *all* of that type's methods pointers. If you're unsure, use a pointer receiver.

You don't need to write `(&order).AddItem(...)`. If `order` is a variable, Go takes its address for you when the method needs a pointer, and dereferences it when the method needs a value.

Constructors: the `NewX` convention

Go has no constructors. Instead, by convention, a plain function named `NewX` builds a ready-to-use `X`:

GO orderly/orderly.go

```go
// NewOrder creates a pending order. This is Go's constructor
// convention: a plain function named NewX that returns a ready value.
func NewOrder(id, customerID string, now time.Time) *Order {
    return &Order{
        ID:         id,
        CustomerID: customerID,
        Status:     StatusPending,
        CreatedAt:  now,
    }
}
```

Returning `&Order{...}`, the address of a local value, is perfectly safe in Go. The compiler's **escape analysis** (the compiler deciding whether a variable can live on the fast, short-lived stack or has to move to the heap) notices that the order outlives the function and puts it on the heap. You never manage memory yourself.

Write a `NewX` function when a type needs set-up (defaults, validation, internal maps). When the zero value or a plain literal is enough, skip it. The `structs/` example puts it all together: it creates an order with `NewOrder`, adds items, prints `TotalCents()` and cancels it.

Pointers

`&`, `*` and no arithmetic

A **pointer** holds the address of a value. `&x` takes the address of `x`; `*p` follows the pointer to the value. Unlike C, there's no pointer arithmetic, so pointers are safe.

GO pointers/main.go

```go
qty := 2
ptr := &qty // & takes the address
*ptr = 5    // * follows the pointer
fmt.Println("qty:", qty)
```

OUTPUT

```
qty: 5
```

In Java, every object variable is secretly a pointer (a reference), and every primitive is a value. In Go, *you* choose, for every type. The difference shows up when you pass things to functions:

GO pointers/main.go

```go
// raiseByValue gets a copy. The caller's product does not change.
func raiseByValue(p orderly.Product, cents int64) {
    p.PriceCents += cents
}

// raiseByPointer gets the address. The caller's product changes.
func raiseByPointer(p *orderly.Product, cents int64) {
    p.PriceCents += cents // Go dereferences p for you
}
```

OUTPUT

```
after raiseByValue: 1299
after raiseByPointer: 1399
```

Notice `p.PriceCents` works on a pointer without writing `(*p).PriceCents`. Go follows the pointer for you when you access a field.

When to use pointers

Use a pointer when...	Example
A function must change the caller's value	`raiseByPointer(&mug, 100)`
The type has pointer-receiver methods	`*Order`, `*strings.Builder`
The struct is large and copied often	Big config or aggregate structs
You need "no value" as distinct from zero	`Discount *int64` in an optional field
Many places must share one instance	A service, a repository, a pool

Small values like `int`, `string`, `time.Time` and small structs are usually passed by value. Copying them is cheap, and values can't be changed behind your back.

`nil` and the nil pointer panic

The zero value of a pointer is `nil`, Go's closest thing to `null`. It's how a function says "nothing found":

GO pointers/main.go

```go
// findProduct returns nil when nothing matches.
func findProduct(products []orderly.Product, sku string) *orderly.Product {
    for i := range products {
        if products[i].SKU == sku {
            return &products[i] // points into the slice
        }
    }
    return nil
}
```

Following a `nil` pointer panics, much like a `NullPointerException`:

OUTPUT

```
recovered: runtime error: invalid memory address or nil pointer dereference
```

(The example program uses `recover` so it can print the panic and keep going; you'll rarely do that in real code.) Check for `nil` before you use a pointer that might be empty. That said, for "not found", Go code more often returns a value plus an error, `(Product, error)`, which is harder to misuse.

Arrays, slices and `ArrayList`

Arrays: fixed size, rarely used directly

An array has a fixed length that's part of its type: `[3]string` and `[4]string` are different types. Arrays are values, so assigning one copies every element.

You'll rarely declare arrays yourself, but they're the storage underneath every slice.

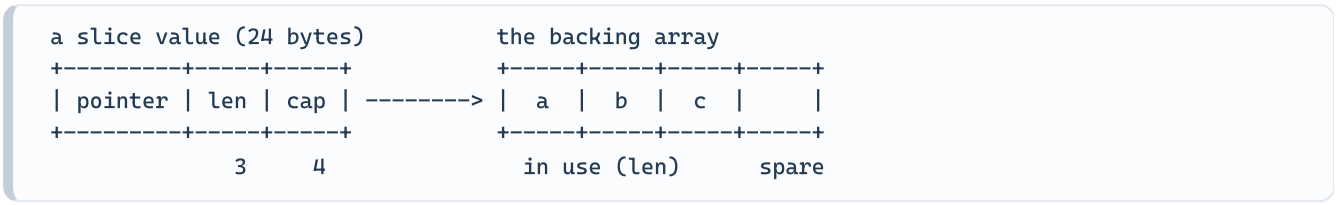
Slices: Go's `ArrayList`

A **slice** is a growable, ordered list: `[]string`, `[]Product`, `[]int64`. You'll use slices everywhere Java uses `List`.

GO slices/main.go

```go
skus := []string{"MUG-BLUE-01", "TEE-RED-XL"}
skus = append(skus, "CAP-GRN-01") // always use the result
fmt.Println(skus, "len:", len(skus))
// [MUG-BLUE-01 TEE-RED-XL CAP-GRN-01] len: 3
```

Under the hood, a slice is a tiny struct of three fields: a pointer to an array, a **length** (how many elements are in use) and a **capacity** (how many fit before a bigger array is needed).



`append` adds to the end. When there's no spare capacity, it allocates a bigger array, copies everything over and returns a slice pointing at the new array. That's why you must always write `s = append(s, x)`: the result may point somewhere new.

GO slices/main.go

```go
var qtys []int // nil slice: len 0, cap 0, ready to append
for i := range 5 {
    qtys = append(qtys, i)
    fmt.Printf("len=%d cap=%d\n", len(qtys), cap(qtys))
}
```

OUTPUT

```
len=1 cap=4
len=2 cap=4
len=3 cap=4
len=4 cap=4
len=5 cap=8
```

The exact capacities are an implementation detail and change between Go versions; what matters is that growth happens in jumps. When you know the final size, preallocate with `make([]T, length, capacity)` so the slice never has to regrow:

GO slices/main.go

```go
totals := make([]int64, 0, 100) // len 0, room for 100
```

Coming from Java

A `nil` slice is not a problem the way a `null` list is. `len(nilSlice)` is 0, `range` over it does nothing, and `append` works on it. Declaring `var items []OrderItem` and appending is perfectly normal Go.

The aliasing gotcha

Here's the gotcha every Java developer meets once. Slicing an existing slice with `s[low:high]` doesn't copy anything. The new slice points into the *same* backing array:

GO slices/main.go

```go
all := []string{"a", "b", "c", "d"}
firstTwo := all[0:2] // shares memory with all
firstTwo[0] = "A"
fmt.Println("all after write:", all) // [A b c d]

// firstTwo has spare capacity, so append overwrites all[2]!
firstTwo = append(firstTwo, "X")
fmt.Println("all after append:", all) // [A b X d]
```

Let's draw what happened:

```
             index:   0     1     2     3
backing array:      +-----+-----+-----+-----+
                    | "A" | "b" | "c" | "d" |
                    +-----+-----+-----+-----+
all       --> starts at 0, len 4, cap 4
firstTwo  --> starts at 0, len 2, cap 4   (two spare slots!)

append(firstTwo, "X") finds spare room, so it writes into index 2:
                    +-----+-----+-----+-----+
                    | "A" | "b" | "X" | "d" |   <- all sees "X" too
                    +-----+-----+-----+-----+
```

Writes through one slice show up in the other, and an `append` can silently overwrite data that "belongs" to someone else. The first fix is a *full slice expression*, `s[low:high:max]`, which also limits the capacity:

GO slices/main.go

```go
// A full slice expression caps the capacity, forcing a copy.
safe := all[0:2:2]
safe = append(safe, "Y")
fmt.Println("all after safe append:", all, "safe:", safe)
```

OUTPUT

```
all after safe append: [A b X d] safe: [A b Y]
```

With no spare capacity, `append` had to allocate a new array, so `all` was left alone. The second fix is even simpler: make an independent copy up front with `slices.Clone(all[:2])`.

> **Key idea**
>
> A slice is a *view* onto an array, not the array itself. Sub-slices share memory. When a function keeps a slice it was given, or returns part of a slice it owns, think about whether it should copy first.

Copying and the `slices` package

The built-in `copy(dst, src)` copies elements between existing slices and returns how many it copied. Most of the time, though, you'll reach for the `slices` package (Go 1.21+), which covers what `Collections` and many `Stream` operations do in Java:

GO slices/main.go

```go
prices := []int64{2500, 499, 1299}
slices.Sort(prices)
fmt.Println("sorted:", prices)

i, found := slices.BinarySearch(prices, 1299)
fmt.Println("1299 at", i, found)

fmt.Println("contains 499?", slices.Contains(prices, 499))
fmt.Println("index of 2500:", slices.Index(prices, 2500))

prices = slices.DeleteFunc(prices, func(p int64) bool {
    return p < 1000
})
fmt.Println("without cheap items:", prices)
```

OUTPUT

```
sorted: [499 1299 2500]
1299 at 1 true
contains 499? true
index of 2500: 2
without cheap items: [1299 2500]
```

There's also `slices.Max`, `slices.Equal`, `slices.Reverse`, `slices.SortFunc` and more. For filtering and mapping, a plain `for` loop is idiomatic and often clearer than a chain of calls.

Maps and `HashMap`

The basics and comma-ok

A **map** is Go's built-in hash table: `map[KeyType]ValueType`.

GO maps/main.go

```go
stock := map[string]int{
    "MUG-BLUE-01": 12,
    "TEE-RED-XL":  0,
}
stock["CAP-GRN-01"] = 7     // insert or overwrite
delete(stock, "CAP-GRN-01") // no error if the key is missing
```

Reading a missing key doesn't throw and doesn't return `null`; it returns the zero value. That's convenient, but it can't tell "zero" apart from "absent". The **comma-ok** form can:

GO maps/main.go

```go
if qty, ok := stock["TEE-RED-XL"]; ok {
    fmt.Println("TEE-RED-XL is listed, qty", qty) // listed, qty 0
}
if _, ok := stock["HAT-01"]; !ok {
    fmt.Println("HAT-01 is not listed")
}
```

Iteration order is random, on purpose

Ranging over a map visits keys in a deliberately unpredictable order, which changes from run to run. Go does this so nobody accidentally depends on an order that was never promised. When you need stable output, sort the keys:

GO maps/main.go

```go
for _, sku := range slices.Sorted(maps.Keys(stock)) {
    fmt.Println(sku, stock[sku])
}
```

This is like iterating a `HashMap` in Java, except Go makes the randomness obvious from day one. If you need insertion order, keep a separate slice of keys.

The nil map trap

A map variable declared with `var` is `nil`. Reading from a nil map is fine, but writing to one panics:

GO maps/main.go

```go
var prices map[string]int64        // nil: declared but never made
fmt.Println(prices["MUG-BLUE-01"]) // 0, reading is fine
prices["MUG-BLUE-01"] = 1299       // panics
```

OUTPUT

```
recovered: assignment to entry in nil map
```

Create maps with a literal (`map[string]int64{}`) or `make(map[string]int64)` before writing. This bites everyone once, usually with a struct field that was never initialised, which is a good reason to give such structs a `NewX` constructor.

The `maps` package and sets

The `maps` package (Go 1.21+) has helpers like `maps.Keys`, `maps.Values`, `maps.Clone` and `maps.Equal`. Go has no built-in set type; a map with empty-struct values does the job:

GO maps/main.go

```go
tags := map[string]struct{}{} // a set of strings
for _, t := range []string{"kitchen", "gift", "kitchen"} {
    tags[t] = struct{}{}
}
_, isGift := tags["gift"]
fmt.Println("tags:", len(tags), "gift?", isGift) // tags: 2 gift? true
```

`struct{}` is a type with no fields, and it takes up no memory. Some people use `map[string]bool` instead because it reads more easily; both are fine.

> ⚠️ **Gotcha: maps are not safe for concurrent writes**
>
> Like `HashMap`, a Go map must not be written by one goroutine while others read or write it. Go detects many such cases at run time and crashes with "concurrent map writes". Protect shared maps with a `sync.Mutex` (Chapter 5).

Strings, bytes and runes

Strings are immutable UTF-8 bytes

A Go `string` is an immutable sequence of **bytes**, almost always UTF-8 text. Java strings are sequences of UTF-16 `char`s. That difference shows up as soon as you leave plain ASCII:

GO strings/main.go

```go
name := "Café Crème" // Go source is UTF-8
fmt.Println("bytes:", len(name), "runes:", utf8.RuneCountInString(name))
// bytes: 12 runes: 10
```

`len` counts bytes. An `é` takes two bytes in UTF-8, so "Café Crème" has 12 bytes but only 10 characters. Go calls a Unicode code point a **rune**.

```
"Café" as UTF-8 bytes:
  byte index:   0      1      2      3      4
  byte value:  0x43   0x61   0x66   0xC3   0xA9
  rune:         C      a      f    \_____é_____/
  len("Café") == 5, but it has 4 runes
```

Indexing a string gives you a byte; ranging over it gives runes, together with each rune's byte offset:

GO strings/main.go

```go
fmt.Println("name[3] is byte", name[3]) // name[3] is byte 195
for i, r := range "Café" {
    fmt.Printf("%d:%c ", i, r) // 0:C 1:a 2:f 3:é
}
```

When you need character positions, convert to `[]rune` first: `string([]rune(name)[3])` is `"é"`.

`strings`, and `strings.Builder` vs `StringBuilder`

The `strings` package holds what Java puts on `String` itself: `strings.ToUpper`, `strings.TrimSpace`, `strings.HasPrefix`, `strings.Split`, `strings.Contains`, `strings.ReplaceAll` and friends.

Because strings are immutable, building one with `+=` in a loop creates garbage, exactly as in Java. Use `strings.Builder`, Go's `StringBuilder`:

GO strings/main.go

```go
var b strings.Builder // the zero value is ready to use
for i, item := range []string{"Blue Mug", "Red Tee"} {
    fmt.Fprintf(&b, "%d. %s\n", i+1, item)
}
fmt.Print(b.String())
```

OUTPUT

```
1. Blue Mug
2. Red Tee
```

`fmt` verbs you'll use every day

`fmt.Printf` and `fmt.Sprintf` work like Java's `String.format`, with some very handy extras for debugging. Here's what each verb prints for an `OrderItem` and a few simple values:

Verb	Meaning	Example output
`%v`	Any value, default format	`{p-1 2 1299}`
`%+v`	Struct with field names	`{ProductID:p-1 Quantity:2 ...}`
`%#v`	Go syntax for the value	`orderly.OrderItem{ProductID:"p-1", ...}`
`%s`, `%q`	String; quoted string	`MUG-BLUE-01`, `"MUG-BLUE-01"`
`%d`, `%5d`	Integer; padded to width 5	`42`, `42`
`%.2f`	Float with 2 decimals	`13.00`
`%t`, `%x`	Bool; hexadecimal	`true`, `ff`
`%T`	The value's type	`orderly.OrderItem`

`%v` and `%+v` are the ones you'll reach for most when debugging. `go vet` checks that your verbs match your arguments.

Equality: `==` instead of `equals` and `hashCode`

In Java, `==` compares references and you override `equals` and `hashCode` for value equality. In Go, `==` compares **values**, and it works on structs out of the box, field by field:

GO structs/main.go

```go
a := orderly.OrderItem{ProductID: "p-1", Quantity: 2, UnitPriceCents: 1299}
b := orderly.OrderItem{ProductID: "p-1", Quantity: 2, UnitPriceCents: 1299}
fmt.Println("a == b:", a == b) // true

pa, pb := &a, &b
fmt.Println(pa == pb, *pa == *pb) // false true: pointers compare addresses

seen := map[orderly.OrderItem]bool{a: true}
fmt.Println("seen b?", seen[b]) // true: structs can be map keys
```

No `hashCode` to write, no Lombok, no chance of `equals` and `hashCode` disagreeing. A struct whose fields are all comparable is automatically usable as a map key.

There's one catch: slices, maps and functions aren't comparable. A struct that contains one, like `Order` with its `Items` slice, can't use `==` at all. The compiler tells you:

OUTPUT

```
.\main.go:17:14: invalid operation: a == b (struct containing []OrderItem
cannot be compared)
```

For those, compare the parts you care about, use `slices.Equal` and `maps.Equal`, or in tests use `reflect.DeepEqual` or the `go-cmp` library (Chapter 15).

> ⚠️ **Gotcha: don't compare times with `==`**
>
> `time.Time` is a struct that includes a location and, for `time.Now()`, a monotonic clock reading. Two `time.Time` values for the same instant can be `!=`. Always use `t1.Equal(t2)`, `Before` and `After`. More on this below.

A preview of struct tags

Struct tags are short strings attached to fields. They do nothing on their own; libraries read them at run time through *reflection* (code inspecting types while the program runs). `encoding/json` is the one you'll meet first:

GO tags/main.go

```go
// Customer shows a few common json tag options.
type Customer struct {
    ID       string `json:"id"`
    Email    string `json:"email"`
    Nickname string `json:"nickname,omitempty"` // left out when empty
    Password string `json:"-"`                  // never serialised
}
```

Marshalling `Customer{ID: "c-42", Email: "ada@example.com", Password: "s3cret"}` produces:

OUTPUT

```
{"id":"c-42","email":"ada@example.com"}
```

The empty `Nickname` was omitted and the password never left the struct. The Orderly types use the same kind of tags, which is why an `Order` marshals to the snake_case JSON (`"customer_id"`, `"unit_price_cents"`) that Orderly's REST API uses throughout this guide. The `tags/` example prints a full order.

You'll see other tags later: `db:"..."` for SQL libraries, `validate:"required"` for validation (Chapter 7) and `env:"PORT"` for configuration (Chapter 10). Think of them as Jackson or JPA annotations that only carry data and never add behaviour.

Time: `time.Time` vs `java.time`

The types

Go's `time` package is smaller than `java.time`, and two types do most of the work:

java.time	Go	Notes
`Instant`	`time.Time` (in UTC)	An instant with nanosecond precision
`ZonedDateTime`, `OffsetDateTime`	`time.Time` with a `*time.Location`	Same type, different zone
`Duration`	`time.Duration`	An `int64` of nanoseconds
`ZoneId.of("Europe/Berlin")`	`time.LoadLocation("Europe/Berlin")`	IANA zone names
`LocalDate`	(none built in)	Often a `time.Time` at midnight UTC
`DateTimeFormatter`	Layout strings	Format by example

Durations are typed, so you write `90 * time.Second` instead of `90000` and hope everyone knows it's milliseconds:

GO times/main.go

```go
placed := time.Date(2026, time.November, 2, 14, 30, 0, 0, time.UTC)

timeout := 90 * time.Second
fmt.Println("timeout:", timeout, "=", timeout.Seconds(), "seconds")

shipBy := placed.Add(48 * time.Hour)
fmt.Println("ship by:", shipBy)
fmt.Println("gap:", shipBy.Sub(placed))
fmt.Println("next month:", placed.AddDate(0, 1, 0).Format(time.DateOnly))
```

OUTPUT

```
timeout: 1m30s = 90 seconds
ship by: 2026-11-04 14:30:00 +0000 UTC
gap: 48h0m0s
next month: 2026-12-02
```

`Add` adds a duration; `Sub` gives the duration between two times; `AddDate(years, months, days)` does calendar arithmetic.

Formatting with the reference date

This is the part that makes every Java developer blink. Go has no `yyyy-MM-dd` patterns. Instead, you write *an example* of the format you want, using one fixed reference time:

```
Mon Jan 2 15:04:05 MST 2006
(month 1, day 2, hour 3 PM, minute 4, second 5, year 06, zone -07)
```

The numbers count up 1, 2, 3, 4, 5, 6, 7, which is the trick to remembering it. Write the reference date the way you want your dates to look:

GO times/main.go

```go
fmt.Println(placed.Format(time.RFC3339))
fmt.Println(placed.Format("02/01/2006 15:04"))
fmt.Println(placed.Format("Monday, 2 January 2006 at 3:04 PM"))

t, err := time.Parse(time.DateOnly, "2026-12-24")
fmt.Println("parsed:", t, err)
```

OUTPUT

```
2026-11-02T14:30:00Z
02/11/2026 14:30
Monday, 2 November 2026 at 2:30 PM
parsed: 2026-12-24 00:00:00 +0000 UTC <nil>
```

The `time` package has constants for common layouts: `time.RFC3339` (use it for APIs and logs), `time.DateOnly` (`2006-01-02`), `time.DateTime` (`2006-01-02 15:04:05`) and `time.Kitchen` (`3:04PM`).

> 💡 **If you're unsure, use RFC 3339 in UTC**
>
> For anything machines read (JSON, logs, messages), use RFC 3339 with UTC times. `encoding/json` already marshals a `time.Time` as RFC 3339. Save friendly formats for what humans see.

Time zones

A `time.Time` is an instant plus a location used for display. `In` shows the same instant on a different wall clock:

GO times/main.go

```go
berlin, err := time.LoadLocation("Europe/Berlin")
if err != nil {
    panic(err)
}
tokyo, _ := time.LoadLocation("Asia/Tokyo")
inBerlin := placed.In(berlin)
fmt.Println("Berlin:", inBerlin.Format(time.DateTime+" MST"))
fmt.Println("Tokyo: ", placed.In(tokyo).Format(time.DateTime+" MST"))
fmt.Println("same instant?", inBerlin.Equal(placed),
    "== says:", inBerlin == placed)
```

OUTPUT

```
Berlin: 2026-11-02 15:30:00 CET
Tokyo:  2026-11-02 23:30:00 JST
same instant? true == says: false
```

There's the `==` gotcha from the equality section, live: same instant, different location, so `==` says `false` while `Equal` says `true`.

`LoadLocation` reads the IANA time zone database from the operating system. Minimal container images often don't have it, so the example adds one import that embeds the database into the binary (about 450 KB):

GO

```go
import _ "time/tzdata" // the binary carries its own zone database
```

Finally, remember the zero `time.Time` from the struct section. Check for it with `IsZero()`: a `ShippedAt` field that's still zero means "not shipped yet". Chapter 19 covers time zones in enterprise data: storing UTC, daylight saving traps and "today" in the user's zone.

Recap

- Declare with `:=` inside functions and `var` for zero values or package level; unused variables and imports are compile errors.
- Every type has a zero value (`0`, `""`, `false`, `nil`), so there's no `null` for strings, numbers or structs. Make zero values useful.

- Go never converts types implicitly: write `int64(qty)`, and use `strconv` for strings and numbers.
- `for` is the only loop (three-part, condition-only, `range`, `range 3`), and `switch` never falls through.
- Structs replace POJOs and records. Assignment copies the whole struct; use pointers to share.
- Use pointer receivers for methods that change state (and then for all of that type's methods); value receivers for small read-only types. Constructors are plain `NewX` functions.
- Slices are views onto arrays: always use `s = append(s, x)`, and remember that sub-slices share memory. Writing to a `nil` map panics; map order is random.
- Strings are UTF-8 bytes (`len` counts bytes, `range` gives runes); `==` compares structs by value; compare times with `Equal`; format dates with the `2006-01-02` reference layout.

Try it yourself

Exercise 1: Order summary

In `examples/ch03-basics/orderly`, add a method `func (o *Order) ItemCount() int` that returns the total quantity across all items. Add a test next to `TestTotalCents`. *Hint:* it's the same loop as `TotalCents`, summing `item.Quantity`.

Exercise 2: Stock report

Write a program that takes a `map[string]int` of SKU to stock level and prints the SKUs that are out of stock, sorted alphabetically, using a `strings.Builder` to build the report. *Hint:* collect matching keys into a slice, then `slices.Sort` it.

Exercise 3: Break the slice

Write a function `firstN(items []orderly.OrderItem, n int) []orderly.OrderItem` that returns `items[:n]`. Show, with a small `main`, how appending to the result can change the original slice. Then fix it two ways: with a full slice expression, and with `slices.Clone`. *Hint:* print `len` and `cap` of both slices before and after the `append`.

Exercise 4 (stretch): Delivery estimate

Write `func deliveryDate(placed time.Time, loc *time.Location) string` that adds three calendar days in the customer's time zone and formats the result like `Thursday, 5 November`. Test it with an order placed late in the evening UTC for a customer in `Asia/Tokyo`. *Hint:* convert with `placed.In(loc)` first, then use `AddDate(0, 0, 3)` and the layout `"Monday, 2 January"`.

Further reading

- A Tour of Go: interactive exercises for everything in this chapter.
- Go Slices: usage and internals: the classic Go blog post on how slices work.

- Go maps in action: maps, zero values, iteration order and concurrency.
- Strings, bytes, runes and characters in Go: a careful explanation of UTF-8 in Go.
- Effective Go: idiomatic use of structs, methods, slices and maps.
- Package time: the full reference, including all layout constants.

CHAPTER 4

Interfaces, Errors, Generics and Packages

> *These four features are where Go's design thinking shows most clearly. Once they click, Go code stops feeling foreign and starts feeling calm.*

🎯 **What you'll learn**

- How Go interfaces work without `implements`, and why small interfaces owned by the consumer make code easy to test
- Why struct embedding looks like inheritance but isn't, and the surprise that proves it
- How to create, wrap, inspect and handle errors, from the repository all the way to an HTTP status code
- How `defer`, `panic` and `recover` replace `finally`, try-with-resources and `RuntimeException`
- How to write enums, generics and iterators that feel like Go rather than translated Java
- How exported names, `internal/` and the no-cycles rule shape your packages

The big picture

In Java, much of your design lives in type hierarchies: interfaces you implement, classes you extend, exceptions you throw up the stack. Go keeps the same goals (abstraction, reuse, solid failure handling, modularity) but reaches them with smaller, more explicit tools. They're the everyday vocabulary of every service in this guide, so we'll take our time.

🧩 **Think of it like LEGO instead of a family tree**

Java's class hierarchies work like a family tree: what you are depends on who your parents were. Go works like LEGO. Pieces click together because their shapes fit, not because they're related. An interface is the shape of a socket, and any brick with the right studs fits, whoever made it.

Java ↔ Go at a glance

Concept	Java / Spring	Go
Implement an interface	`implements OrderFinder`	Have the methods; no keyword
Reuse code	`extends` (inheritance)	Embedding (composition)
Report failure	Exceptions, `throws`	Return an `error` value
Clean up	`finally`, try-with-resources	`defer`
Unrecoverable bug	`RuntimeException`, `Error`	`panic` (rare)

Concept	Java / Spring	Go
Enum	`enum` class	Typed constants and `iota`
Generics	Erased `List<T>`, wildcards	Type parameters, constraints
Lazy sequences	`Iterator`, `Stream`	`for range` over `iter.Seq`
Visibility	`public`, `private`, package	Capital letter means exported

Interfaces: satisfied, not declared

Implicit satisfaction

In Java, a class has to name every interface it implements, up front. That ties the class to the interface's package, and you can't make someone else's class implement *your* interface later. Go turns this around: a type satisfies an interface automatically if it has the right methods.

JAVA

```java
public interface OrderFinder {
    Order findOrder(String id);
}

public class MemoryStore
        implements OrderFinder {
    @Override
    public Order findOrder(String id) {
        return orders.get(id);
    }
}
```

GO

```go
type OrderFinder interface {
    FindOrder(ctx context.Context,
        id string) (Order, error)
}

// No "implements" anywhere.
type MemoryStore struct {
    orders map[string]Order
}

func (m *MemoryStore) FindOrder(
    ctx context.Context, id string,
) (Order, error) { /* ... */ }
```

`*MemoryStore` satisfies `OrderFinder` because it has a `FindOrder` method with exactly that signature. This is called **structural typing**: a type fits an interface because of its shape, not because of a declared relationship. It's like duck typing, except the compiler checks it.

The check happens where you *use* the type, for example when you pass a `*MemoryStore` to a function that wants an `OrderFinder`. If you'd like the compiler to check it right next to the type, add this one-liner:

GO

```go
// The build fails if *MemoryStore ever stops satisfying OrderFinder.
var _ OrderFinder = (*MemoryStore)(nil)
```

Coming from Java

You'll miss `implements` for about a week. After that, you'll enjoy fitting any type, even one from another library, to an interface you wrote, without touching the type.

Small interfaces are the norm

Because types don't have to opt in, Go interfaces tend to be tiny. The most important ones in the standard library have a single method:

GO

```go
// package io
type Reader interface {
    Read(p []byte) (n int, err error)
}
type Writer interface {
    Write(p []byte) (n int, err error)
}

// package fmt
type Stringer interface {
    String() string
}

// built into the language
type error interface {
    Error() string
}
```

Files, network connections, HTTP bodies, `bytes.Buffer` and `strings.Builder` all satisfy `io.Writer`, so a function that accepts one works with all of them. A Go proverb puts it well: "*The bigger the interface, the weaker the abstraction.*"

Giving `Order` a `String()` method makes it a `fmt.Stringer`, so `fmt` prints it nicely everywhere:

GO interfaces/main.go

```go
func (o Order) String() string {
    return fmt.Sprintf("order %s (%s, %d items, %d cents)",
        o.ID, o.Status, len(o.Items), o.TotalCents())
}
```

Accept interfaces, return structs

In Spring, the interface usually lives with the provider: `OrderRepository extends JpaRepository<Order, UUID>` has dozens of methods, and every consumer depends on all of them.

Go code usually does the opposite. **The consumer declares the small interface it needs**, right where it uses it. Here's a receipt service that only needs to find orders:

GO interfaces/main.go

```go
// OrderFinder is declared where it is used, and it is tiny.
type OrderFinder interface {
    FindOrder(ctx context.Context, id string) (Order, error)
}

type ReceiptService struct {
    orders OrderFinder
}
```

```go
// NewReceiptService accepts an interface and returns a concrete struct.
func NewReceiptService(f OrderFinder) *ReceiptService {
    return &ReceiptService{orders: f}
}

func (s *ReceiptService) Print(ctx context.Context, w io.Writer,
    id string) error {
    o, err := s.orders.FindOrder(ctx, id)
    if err != nil {
        return fmt.Errorf("print receipt: %w", err)
    }
    _, err = fmt.Fprintf(w, "Receipt for %v\n", o)
    return err
}
```

The `MemoryStore` has `Save`, `FindOrder` and `Delete`, and it has never heard of `OrderFinder`. It fits anyway. We can print to the terminal or into a string with the same code:

GO interfaces/main.go

```go
svc := NewReceiptService(store) // *MemoryStore used as OrderFinder

_ = svc.Print(ctx, os.Stdout, "ord-1") // *os.File is an io.Writer

var sb strings.Builder // so is *strings.Builder
_ = svc.Print(ctx, &sb, "ord-1")
fmt.Print("captured: ", sb.String())
```

OUTPUT

```
Receipt for order ord-1 (PENDING, 2 items, 4499 cents)
captured: Receipt for order ord-1 (PENDING, 2 items, 4499 cents)
```

"Accept interfaces, return structs" is the rule of thumb behind this. Taking an interface keeps your function flexible for callers. Returning a concrete type keeps every method available and lets callers decide which interface, if any, they need.

The payoff shows up in tests, where a one-method interface is easy to fake by hand:

GO interfaces/main_test.go

```go
// finderFunc lets a plain function satisfy OrderFinder, the same trick
// net/http uses with http.HandlerFunc.
type finderFunc func(ctx context.Context, id string) (Order, error)

func (f finderFunc) FindOrder(ctx context.Context, id string) (Order, error) {
    return f(ctx, id)
}
```

> 💡 **If you're unsure, start without an interface**
>
> Write concrete types first. Introduce an interface when a second implementation or a test fake appears, and declare it in the package that *uses* it. Interfaces created "in case we need one", next to their only implementation, are a Java habit worth leaving behind.

Full code: `examples/ch04-interfaces-errors-generics/interfaces/`

Interface embedding and `any`

You can build bigger interfaces out of smaller ones by listing them inside another interface. The standard library does this with `io.ReadWriter`, and we can do the same:

GO interfaces/main.go

```go
type OrderSaver interface {
    Save(ctx context.Context, o Order) error
}

// OrderStore is "everything an OrderFinder and an OrderSaver can do".
type OrderStore interface {
    OrderFinder
    OrderSaver
}
```

The empty interface, `interface{}`, has no methods, so every type satisfies it. Since Go 1.18 it has a friendlier name: `any`. It's Go's `Object`. Use it at true boundaries (JSON decoding, logging, `fmt`) and reach for a real type or generics everywhere else.

Type assertions and type switches

A **type assertion** asks an interface "is the thing inside you a `*os.File`?". Always use the two-value "comma ok" form, which reports failure instead of panicking:

GO interfaces/main.go

```go
var w io.Writer = os.Stdout
if f, ok := w.(*os.File); ok {
    fmt.Println("w is a file named", f.Name())
}
```

A **type switch** checks several types at once. It's the closest thing to Java 21's pattern-matching `switch`:

GO interfaces/main.go

```go
func describe(v any) string {
    switch x := v.(type) {
    case nil:
        return "nothing at all"
    case Order: // must come before fmt.Stringer: Order is one too
        return fmt.Sprintf("an order worth %d cents", x.TotalCents())
    case fmt.Stringer:
        return "something printable: " + x.String()
    case error:
        return "an error: " + x.Error()
    case int, int64: // with two types, x stays an `any`
        return fmt.Sprintf("a whole number: %v", x)
    default:
        return fmt.Sprintf("something else (%T)", x)
    }
}
```

Cases are tried in order, and a case can be an interface (a `time.Duration` lands in the `fmt.Stringer` case). Many type switches usually mean a method on an interface would serve you better.

The nil interface gotcha

This one trips up every Java developer, so let's look at it slowly. An interface value is really **two** things: the dynamic type of what's inside, and the value itself. It only equals `nil` when *both* are empty.

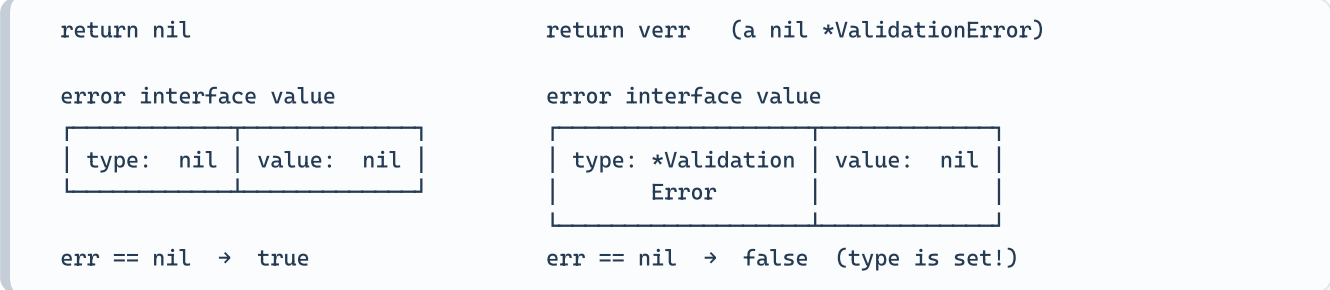
GO nilinterface/main.go

```go
func validateBuggy(customerID string) error {
    var verr *ValidationError // nil pointer of a concrete type
    if customerID == "" {
        verr = &ValidationError{Field: "customer_id"}
    }
    return verr // BUG: wraps a (*ValidationError)(nil) in a non-nil error
}
```

OUTPUT

```
buggy: err == nil? false
buggy: dynamic type *main.ValidationError, dynamic value <nil>
```

The customer was valid, yet the caller sees an error. Here's why:



> ⚠️ **Gotcha: the nil interface trap**
>
> Never return a nil *pointer of a concrete type* through an interface. When there's no error, write `return nil` literally. The same applies to any interface, not only `error`.

Method sets, briefly

A method with a pointer receiver, `func (e *EmailNotifier) Notify(...)`, belongs only to `*EmailNotifier`. So the *value* `EmailNotifier{}` doesn't satisfy an interface that needs `Notify`, and the compiler tells you exactly that:

OUTPUT

```
cannot use EmailNotifier{} (value of struct type EmailNotifier) as Notifier
value in variable declaration: EmailNotifier does not implement Notifier
(method Notify has pointer receiver)
```

The fix is to use `&EmailNotifier{}`. As a rule of thumb, if any method on a type has a pointer receiver, pass that type around as a pointer.

Embedding: composition, not inheritance

Sometimes several types share fields and behaviour. Java reaches for a base class. Go offers **embedding**: put a type inside a struct without a field name, and its fields and methods are *promoted*, so you can use them as if they belonged to the outer struct.

GO embedding/main.go

```go
type Entity struct {
    ID        string
    CreatedAt time.Time
}

func (e Entity) Describe() string { return "entity " + e.ID }

// Summary calls Describe. Which Describe? Always Entity's.
func (e Entity) Summary() string { return "[" + e.Describe() + "]" }

type Order struct {
    Entity // embedded: no field name
    Status string
}

func (o Order) Describe() string {
    return "order " + o.ID + " (" + o.Status + ")"
}
```

`o.ID` works, and so does `o.Summary()`. It looks like inheritance. Now for the surprise. In Java, a base class method that calls an overridden method runs the subclass version (this is called *virtual dispatch*):

JAVA

```java
class Entity {
    String describe() { return "entity " + id; }
    String summary() { return "[" + describe() + "]"; }
}
class Order extends Entity {
    @Override String describe() { return "order " + id; }
}
// new Order().summary()  ->  "[order ...]"
```

Go doesn't do that:

GO embedding/main.go

```go
fmt.Println(o.Describe())        // Order's own method
fmt.Println(o.Summary())         // promoted from Entity: surprise!
fmt.Println(Summary(o))          // interface: uses Order.Describe
```

OUTPUT

```
order ord-1 (PENDING)
[entity ord-1]
[order ord-1 (PENDING)]
```

`o.Summary()` is shorthand for `o.Entity.Summary()`. Inside it, the receiver is the embedded `Entity`, which has no idea it lives inside an `Order`. There's no virtual dispatch through embedding, ever. When you need behaviour that varies by type, pass it in through an interface, like the `Summary(d Describer)` function in the example.

> 🧠 **Key idea**
>
> Embedding is a shortcut for delegation, not a type relationship: you can't pass an `Order` where an `Entity` is expected. Interfaces give you polymorphism; embedding only saves typing.

Where embedding really shines is wrapping. Embed an interface, override one method, and every other method passes straight through. Here's a logging decorator:

GO embedding/main.go

```go
type loggingFinder struct {
    OrderFinder // every OrderFinder method, for free
}

func (l loggingFinder) FindOrder(ctx context.Context,
    id string) (Order, error) {
    fmt.Println("log: finding order", id)
    return l.OrderFinder.FindOrder(ctx, id) // call the wrapped one
}
```

Full code: `examples/ch04-interfaces-errors-generics/embedding/`

Errors are values

Why Go chose values over exceptions

Java has checked exceptions you must declare and unchecked ones you mustn't, and Spring mostly converts the former into the latter (think `DataAccessException`). Either way, an exception is invisible control flow: from reading a method call, you can't tell whether it might jump straight out of your method.

Go's answer is plain. An error is an ordinary value of the built-in `error` interface (anything with an `Error() string` method), returned as the last result:

GO

```go
func (r *MemoryRepo) Find(ctx context.Context, id string) (Order, error)
```

The designers chose this on purpose:

- **Failure is visible.** Every call that can fail says so in its signature, and every place that handles it is right there in the code.
- **Failure is normal.** In networked services, timeouts and missing rows aren't exceptional. They deserve ordinary code.

- **Errors are cheap.** No stack trace is captured, so returning an error costs about the same as returning any other value.

People have proposed shorter syntax many times. In 2025 the Go team announced that, for the foreseeable future, they'll stop pursuing syntax changes for error handling. What you learn here is how Go will look for a long time.

The `if err != nil` rhythm

Here's the shape you'll write hundreds of times a week:

GO errors/order.go

```go
func (s *Service) Get(ctx context.Context, id string) (Order, error) {
    o, err := s.repo.Find(ctx, id)
    if err != nil {
        return Order{}, fmt.Errorf("get order %s: %w", id, err)
    }
    return o, nil
}
```

It feels repetitive at first, and every Java developer grumbles about it. Here's how most people make peace with it:

- **Read the left edge.** Go code keeps the happy path unindented and handles each failure with an early return. Skim down the left margin to follow the story.
- **Let your editor type it.** GoLand and VS Code both expand a snippet for `if err != nil`.
- **Treat each check as a decision point.** "What should happen if this fails?" is a question Java let you skip. Go makes you answer it, and your on-call self will thank you.

Wrapping with `%w`, `errors.Is` and `errors.As`

A bare "insufficient stock" in a log is hard to act on. Which product? Which order? So each layer **wraps** the error with context using `fmt.Errorf` and the `%w` verb:

GO errors/order.go

```go
func (r *MemoryRepo) Reserve(ctx context.Context, productID string,
    qty int) error {
    have := r.stock[productID]
    if have < qty {
        return fmt.Errorf("product %s: want %d, have %d: %w",
            productID, qty, have, ErrInsufficientStock)
    }
    r.stock[productID] = have - qty
    return nil
}
```

The service wraps it again with `fmt.Errorf("place order %s: %w", o.ID, err)`, and the final message reads like a trail of breadcrumbs:

OUTPUT

```
place order ord-2: product MUG-BLUE-01: want 9, have 5: insufficient stock
```

> 🧩 **Think of it like labels on a parcel**
>
> Each depot a parcel passes through sticks a new label on top. `%w` adds a label without hiding the ones underneath. `errors.Is` and `errors.As` peel back the labels to look for the original sender.

Because the original error is still inside, callers can ask about it:

You want to know	Use	Java equivalent
Is this a specific error value?	`errors.Is(err, ErrOrderNotFound)`	`catch (OrderNotFoundException e)`
Is there an error of this type? Give it to me	`errors.As(err, &verr)`	`catch (ValidationException e)` and read its fields

Both search the whole chain of wrapped errors, so they keep working no matter how many layers added context. Never compare error messages with `strings.Contains`.

> 💡 **`%w` or `%v`?**
>
> Use `%w` when callers may need to inspect the cause. Use `%v` when you deliberately want to hide an implementation detail, for example so that callers can't depend on a Postgres driver error leaking out of your repository.

Sentinel errors vs custom error types

There are two common ways to define errors. A **sentinel error** is a fixed, exported value that callers compare against. A **custom error type** is a struct that carries details.

GO errors/order.go

```go
// Sentinel errors: fixed values that callers compare against.
var (
    ErrOrderNotFound     = errors.New("order not found")
    ErrInsufficientStock = errors.New("insufficient stock")
)

// ValidationError is a custom error type: it carries details.
type ValidationError struct {
    Field   string
    Problem string
}

func (e *ValidationError) Error() string {
    return e.Field + " " + e.Problem
}
```

If you're unsure which to use, start with a sentinel. Switch to a custom type when callers need data from the error, such as which field failed validation. Check sentinels with `errors.Is`, and types with `errors.As(err, &verr)` where `verr` is a `*ValidationError` variable that receives the match.

> **Newer Go: errors.AsType**
>
> Go 1.26 added a generic shortcut, `verr, ok := errors.AsType[*ValidationError](err)`. It's tidier, but the examples in this guide stay on Go 1.25, so we use `errors.As`.

Reporting several errors at once with `errors.Join`

Validation should report *every* problem, not only the first. `errors.Join` (Go 1.20) bundles errors into one, and returns `nil` if they're all `nil`:

GO errors/order.go

```go
func validate(o Order) error {
    var errs []error
    if o.CustomerID == "" {
        errs = append(errs, &ValidationError{"customer_id", "is required"})
    }
    if len(o.Items) == 0 {
        errs = append(errs, &ValidationError{"items", "must not be empty"})
    }
    for i, it := range o.Items {
        if it.Quantity <= 0 {
            field := fmt.Sprintf("items[%d].quantity", i)
            errs = append(errs, &ValidationError{field, "must be positive"})
        }
    }
    return errors.Join(errs...) // nil when errs is empty
}
```

OUTPUT

```
2: place order: customer_id is required
items[0].quantity must be positive
   first bad field: customer_id
```

The joined message puts each error on its own line, and `errors.Is` and `errors.As` search inside every one of them.

Where to handle, where to propagate

A good mental model: **lower layers add context and pass errors up; the edge handles them once.**

```
repository     return ErrOrderNotFound
    |
    ▼
service        return fmt.Errorf("get order %s: %w", id, err)
    |
    ▼
HTTP handler   writeError(w, r, err)   → log once, map to 404
```

The most common mistake is logging an error *and* returning it. Every layer does the same, and one failure produces five log lines with no extra information:

GO Avoid

```go
o, err := s.repo.Find(ctx, id)
if err != nil {
    slog.Error("find failed", "err", err) // logged here...
    return Order{}, err                   // ...and again by every caller
}
```

Instead, either *handle* the error (log it, retry, fall back to a default, turn it into a response) or *return* it with context. Never both.

Mapping domain errors to HTTP status at the edge

In Spring you'd write a `@RestControllerAdvice` with `@ExceptionHandler` methods. In Go, it's an ordinary function in your HTTP layer, the only place that knows about both domain errors and status codes:

GO errors/httpapi.go

```go
func statusFor(err error) int {
    var verr *ValidationError
    switch {
    case errors.As(err, &verr):
        return http.StatusBadRequest // 400
    case errors.Is(err, ErrOrderNotFound):
        return http.StatusNotFound // 404
    case errors.Is(err, ErrInsufficientStock):
        return http.StatusConflict // 409
    default:
        return http.StatusInternalServerError // 500
    }
}

func writeError(w http.ResponseWriter, r *http.Request, err error) {
    status := statusFor(err)
    msg := err.Error()
    if status == http.StatusInternalServerError {
        slog.Error("request failed", "path", r.URL.Path, "err", err)
        msg = "internal error" // don't leak internals to clients
    }
    http.Error(w, msg, status)
}
```

Every handler calls `writeError` and returns. Here's the demo sending three requests through the router:

OUTPUT

```
4: GET /api/v1/orders/ord-404 -> 404
   get order ord-404: order not found
4: POST /api/v1/orders -> 400
   place order: customer_id is required | items must not be emp...
4: POST /api/v1/orders -> 201
   {"id":"ord-4","customer_id":"cus-9","items":[{"product_id":"...
```

Chapter 7 looks at returning proper `application/problem+json` responses.

Full code: `examples/ch04-interfaces-errors-generics/errors/`

defer: cleanup you can see

Every resource you open needs closing, even when something fails halfway. Java gives you `finally` and try-with-resources. Go gives you `defer`: it schedules a function call to run when the surrounding function returns, however it returns.

JAVA

```java
try (var in = Files.newBufferedReader(p)) {
    return parse(in);
} // in.close() runs here
```

GO

```go
f, err := os.Open(path)
if err != nil {
    return err
}
defer f.Close() // runs on return
return parse(f)
```

The win is locality: cleanup sits next to acquisition, so a `return` added later can't skip it.

Order and timing

Deferred calls run in **last-in, first-out** order, like a stack of plates. And there's a subtle point: the *arguments* of a deferred call are evaluated immediately, while a deferred *closure* reads variables when it finally runs.

GO deferring/main.go

```go
func argumentsAreEvaluatedEarly() {
    status := "PENDING"
    defer fmt.Println("argument saw:", status) // status is read NOW
    defer func() {
        fmt.Println("closure saw: ", status) // status is read LATER
    }()
    status = "CONFIRMED"
}
```

OUTPUT

```
closure saw:  CONFIRMED
argument saw: PENDING
```

The defer-in-a-loop gotcha

`defer` belongs to the *function*, not to the loop body. Open a file per loop iteration with `defer f.Close()`, and nothing closes until the whole loop finishes. With ten thousand files, you run out of file handles.

GO Avoid

```go
for _, n := range names {
    c := open(n)
    defer c.Close() // runs when the function returns, not per loop!
    fmt.Println("use  ", n)
}
```

GO Prefer

```go
for _, n := range names {
    if err := exportOne(n); err != nil { // the defer lives in exportOne
        return err
    }
}
```

The output shows the difference: in the first version every `close` waits until the end.

OUTPUT

```
--- avoid                --- prefer
open  a.csv              open  a.csv
use   a.csv              use   a.csv
open  b.csv              close a.csv
use   b.csv              open  b.csv
loop done                use   b.csv
close b.csv              close b.csv
close a.csv              loop done
```

Keeping the error from Close

When you write a file, `Close` can fail, and that may mean your data never reached the disk. A plain `defer f.Close()` throws that error away. The fix combines a **named result** (a return value with a name, which a deferred closure can change) with `errors.Join`:

GO deferring/main.go

```go
func writeReport(path string, lines []string) (err error) {
    f, err := os.Create(path)
    if err != nil {
        return fmt.Errorf("create report: %w", err)
    }
    defer func() {
        err = errors.Join(err, f.Close()) // nil only if both are nil
    }()
    for _, line := range lines {
        if _, err := fmt.Fprintln(f, line); err != nil {
            return fmt.Errorf("write report: %w", err)
        }
    }
    return nil
}
```

It's Go's version of Java's suppressed exceptions: if the write and the close both fail, the caller sees both. For files you only *read*, a plain `defer f.Close()` is fine.

Full code: `examples/ch04-interfaces-errors-generics/deferring/`

panic and recover

Go does have a crash mechanism. `panic` stops normal execution, runs deferred calls on the way up, and kills the program unless something calls `recover`. Think of it as a `RuntimeException` you aren't supposed to catch for business logic.

Panic is acceptable for:

- **Programmer bugs**, like an index out of range or a nil pointer. The runtime panics for you.
- **Impossible states** that mean the code itself is wrong, such as an unknown enum value in a `switch` you control.
- **Startup checks**, where failing fast beats running broken. `regexp.MustCompile` and `template.Must` panic on bad input, which is fine for package-level values.

It's never right for expected failures: not found, invalid input, timeouts. Those are `error` values.

The one place you *do* recover is at the edge of a server. Left alone, `net/http` recovers a handler panic itself, logs it and drops the connection, so the client gets no response at all. A small middleware turns it into a clean 500:

GO panics/main.go

```go
func recoverer(next http.Handler) http.Handler {
    return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
```

```
        defer func() {
            rec := recover()
            if rec == nil {
                return // no panic: nothing to do
            }
            if rec == http.ErrAbortHandler {
                panic(rec) // a deliberate abort: let net/http handle it
            }
            slog.Error("panic in handler", "path", r.URL.Path,
                "panic", rec, "stack", string(debug.Stack()))
            http.Error(w, "internal error", http.StatusInternalServerError)
        }()
        next.ServeHTTP(w, r)
    })
}
```

OUTPUT

```
level=ERROR msg="panic in handler" path=/buggy panic="runtime error: invalid
memory address or nil pointer dereference"
GET /buggy -> 500 internal error
GET /sku/MUG-BLUE-01 -> 200 valid: true
```

> ⚠️ **Gotcha: recover only works in its own goroutine**
>
> `recover` only stops a panic when called from a deferred function in the *same goroutine*. A panic inside a goroutine you started with `go` crashes the whole program, middleware or not. Chapter 5 shows how to keep goroutines safe.

Enums with typed constants and iota

Java enums are full classes with fields, methods, `values()` and exhaustive `switch` checks. Go has no `enum` keyword. Instead, you define a named type and a block of constants. `iota` is a counter that starts at 0 in each `const` block and goes up by one per line.

JAVA

```
public enum ShippingSpeed {
    STANDARD, EXPRESS, OVERNIGHT;
}
```

GO enums/main.go

```
type ShippingSpeed int

const (
    SpeedUnknown   ShippingSpeed = iota // 0
    SpeedStandard                       // 1
    SpeedExpress                        // 2
    SpeedOvernight                      // 3
)

// String makes the value print nicely (it satisfies fmt.Stringer).
func (s ShippingSpeed) String() string {
```

```
    switch s {
    case SpeedStandard:
        return "standard"
    case SpeedExpress:
        return "express"
    case SpeedOvernight:
        return "overnight"
    default:
        return fmt.Sprintf("ShippingSpeed(%d)", int(s))
    }
}
```

Reserving 0 for "unknown" is a good habit, because 0 is what an unset field holds. For values that travel through JSON, logs and databases, a **string-based** type reads better. That's what Orderly uses for statuses, and you can hang methods on it like a Java enum:

GO enums/main.go

```go
type OrderStatus string

const (
    StatusPending   OrderStatus = "PENDING"
    StatusConfirmed OrderStatus = "CONFIRMED"
    StatusShipped   OrderStatus = "SHIPPED"
    StatusCancelled OrderStatus = "CANCELLED"
)

func (s OrderStatus) CanCancel() bool {
    return s == StatusPending || s == StatusConfirmed
}
```

Be honest with yourself about what you lose. `ShippingSpeed(7)` and `OrderStatus("SHIPPPED")` both compile. There's no `values()`, and a `switch` that forgets a case compiles without a warning. Three habits cover the gap:

- Add a `Valid()` method and call it wherever outside data comes in.
- Run the **exhaustive** linter (included in golangci-lint, see Chapter 16) to flag `switch` statements that miss a constant.
- Generate `String()` methods with the `stringer` tool if you have many integer enums.

Full code: `examples/ch04-interfaces-errors-generics/enums/`

Generics

Before Go 1.18 you either copied a function per type or used `any` and lost type safety. Generics fix that for containers and algorithms that work the same way for many types.

Type parameters and constraints

A type parameter goes in square brackets. Its **constraint** says which types are allowed, and therefore which operations the function may use:

GO generics/main.go

```go
func Map[T, U any](in []T, f func(T) U) []U {
    out := make([]U, 0, len(in))
    for _, v := range in {
        out = append(out, f(v))
    }
    return out
}

// Number is a constraint: the set of types Sum accepts. The ~ means
// "this type, or any named type built on it", so Cents qualifies.
type Number interface {
    ~int | ~int64 | ~float64
}

func Sum[T Number](xs []T) T {
    var total T // the zero value of whatever T is
    for _, x := range xs {
        total += x // allowed: every type in Number supports +
    }
    return total
}

type Cents int64 // underlying type int64
```

The constraints you'll use most:

- `any`: no requirements. You can store and pass values, but not compare or add them.
- `comparable`: works with `==` and as a map key. Built into the language.
- `cmp.Ordered`: works with `<` and `>` (numbers and strings). From the `cmp` package.
- Your own type sets, like `Number`, with `~` to include named types such as `Cents`.

The compiler infers type arguments from the call, so you rarely write them:

GO generics/main.go

```go
ids := Map(orders, func(o Order) string { return o.ID }) // T, U inferred
fmt.Println("revenue:", Sum(totals), "| ints:", Sum([]int{1, 2, 3}))
```

OUTPUT

```
ids: [ord-1 ord-2 ord-3]
revenue: 18498 | ints: 6
```

> **The Go way: loops are fine**
>
> The standard library has no `Map` or `Filter`, on purpose. A `for` loop is clear, fast and easy to debug, and Go developers reach for it first. Use helpers like these when they genuinely remove duplication, not to recreate Java streams.

Generic types

Types can have type parameters too. A set needs `comparable` keys, and a page of results works for anything:

GO generics/main.go

```go
type Set[T comparable] struct {
    items map[T]struct{}
}

func (s *Set[T]) Add(v T)      { s.items[v] = struct{}{} }
func (s *Set[T]) Has(v T) bool { _, ok := s.items[v]; return ok }
func (s *Set[T]) Len() int     { return len(s.items) }

// Page wraps any list response: Page[Order], Page[Product], ...
type Page[T any] struct {
    Items      []T    `json:"items"`
    NextCursor string `json:"next_cursor,omitempty"`
}
```

`json.Marshal(FirstPage(ids, 2))` on a `Page[string]` gives:

OUTPUT

```
{"items":["ord-1","ord-2"],"next_cursor":"2"}
```

How Go generics differ from Java's

Topic	Java	Go
At runtime	Erased to `Object`	Real types; compiled per type "shape"
Primitives	Boxed: `List<Integer>`	Unboxed: `Set[int]` stores ints
Variance	`? extends`, `? super`	No wildcards, no variance
Generic methods	`<R> R map(...)`	Not before Go 1.27 (see note)
Operators	Not allowed on `T`	Allowed if the constraint permits

Instead of `List<? extends Number>`, you write a type parameter with a constraint. And up to Go 1.26 only a *type* can have type parameters, not a method, which is why `Map` above is a plain function.

> **Newer Go: generic methods**
>
> Go 1.27 lets a method declare its own type parameters, for example `func (s *Set[T]) Map[U any](...)`. Interface methods still can't. Our examples target Go 1.25, so they use functions.

When not to use generics

The Go team's own advice is to write the code first and introduce a type parameter only when you notice you're about to write the exact same code twice for different types. Skip generics when:

- **An interface does the job.** If the code only calls methods, accept an interface like `io.Writer`.
- **There's only one type today.** Don't generalise for an imagined future.
- **The behaviour differs per type.** That's what methods are for.

Iterators: range over functions

Java has `Iterator` and `Stream` for lazy sequences. Since Go 1.23, a plain function can feed a `for range` loop. The `iter` package names the two shapes:

GO

```go
type Seq[V any] func(yield func(V) bool)        // one value per step
type Seq2[K, V any] func(yield func(K, V) bool) // two values per step
```

Your function calls `yield` for each element. If the loop body hits `break`, `yield` returns `false` and you stop. Here's a repository method that streams pending orders lazily:

GO iterators/main.go

```go
func (r *OrderRepo) ByStatus(status string) iter.Seq[Order] {
    return func(yield func(Order) bool) {
        for _, o := range r.orders {
            r.scanned++
            if o.Status != status {
                continue
            }
            if !yield(o) { // false means the caller broke out
                return
            }
        }
    }
}
```

GO iterators/main.go

```go
for o := range repo.ByStatus("PENDING") {
    fmt.Println("first pending:", o.ID)
    break
}
fmt.Println("scanned", repo.scanned, "of", len(repo.orders), "orders")
```

OUTPUT

```
first pending: ord-2
scanned 2 of 5 orders
```

Only two orders were looked at, like a Java `Stream` with `findFirst()`. An `iter.Seq2[Order, error]` is a handy shape for database-style cursors, where any row might fail. The example's `Rows` method shows it.

The standard library speaks iterators too:

Expression	What you get
`slices.All(s)`	index and value pairs (`iter.Seq2`)
`slices.Values(s)`	values only (`iter.Seq`)
`maps.Keys(m)` , `maps.Values(m)`	keys or values, in random order
`slices.Collect(seq)`	a new slice from any `iter.Seq`
`slices.Sorted(maps.Keys(m))`	sorted keys in one line

> ⚠️ **Gotcha: respect yield's answer**
>
> When `yield` returns `false` , return immediately. If your iterator keeps calling `yield` after that, the program panics at runtime.

Full code: `examples/ch04-interfaces-errors-generics/iterators/`

Packages and visibility

Capital letters are the access modifier

Go has no `public` or `private` . A name that starts with an upper-case letter (`Order` , `NewService` , `ErrOrderNotFound`) is **exported**, visible to other packages. A lower-case name (`statusConfirmed` , `validate`) is visible only inside its own package. That's the whole system: package-level privacy, which matches Java's package-private more than `private` .

`internal/` directories

Sometimes a package should be shared inside your project but not by anyone else. Put it under a directory named `internal` , and only code rooted at the parent of `internal/` can import it. Try it from another module and the build stops:

OUTPUT

```
main.go:3:8: use of internal package example.com/orderly/internal/order not
allowed
```

This is why the capstone (Chapter 20) keeps almost everything in `internal/` . You can refactor freely without breaking anyone outside.

Naming packages

The package name is part of every call site, so choose it with care:

- **Short, lower case, one word:** `order` , `inventory` , `httpapi` . No underscores or `camelCase` .
- **Named for what it provides,** never `util` , `common` or `helpers` .
- **No stutter:** callers write `order.Service` , so don't call it `order.OrderService` .

`init()` and why to avoid it

A function named `init` runs automatically when its package loads, before `main`. It's tempting for setup, but it hides side effects, can't return an error and makes tests unpredictable. Prefer explicit constructors called from `main`. The one common, acceptable use is a library registering itself, such as a database driver imported with a blank import: `import _ "github.com/jackc/pgx/v5/stdlib"`.

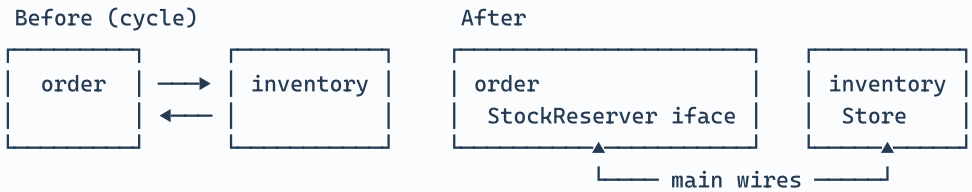
No import cycles, and how to break them

Java happily compiles packages that depend on each other in a circle. Go refuses:

OUTPUT

```
package example.com/orderly/internal/inventory
    imports example.com/orderly/internal/order from inventory.go
    imports example.com/orderly/internal/inventory from order.go: import cycle
not allowed
```

This feels strict at first, but it keeps dependencies flowing one way. The usual fix is our friend the consumer-side interface. `order` declares the tiny interface it needs, `inventory` satisfies it without importing `order`, and `main` connects them:



GO packages/internal/order/order.go

```go
// StockReserver is what package order needs from inventory. It lives
// here, on the consumer side, so order never imports inventory.
type StockReserver interface {
    Reserve(ctx context.Context, productID string, qty int) error
}
```

GO packages/main.go

```go
stock := inventory.NewStore(map[string]int{"MUG-BLUE-01": 3})
orders := order.NewService(stock) // *inventory.Store fits StockReserver
```

Two other fixes help when the interface trick doesn't fit: move the shared types into a small third package that both import, or merge two packages that are really one idea.

Full code: `examples/ch04-interfaces-errors-generics/packages/`

Recap

- A type satisfies an interface by having its methods. Keep interfaces small and declare them where they're used: accept interfaces, return structs.

- An interface holding a nil pointer is not nil. Return a literal `nil` when there's no error.
- Embedding promotes fields and methods, but it is not inheritance: there's no virtual dispatch. Use interfaces for polymorphism.
- Errors are values. Wrap them with `fmt.Errorf("...: %w", err)` at each layer, inspect them with `errors.Is` and `errors.As`, and handle them once at the edge.
- `defer` runs cleanup in LIFO order when the function returns. Its arguments are evaluated immediately, and it doesn't run per loop iteration.
- `panic` is for bugs and startup failures. Recover only in server middleware, and remember it doesn't cross goroutines.
- Enums are typed constants. Add `String()` and `Valid()`, and let the exhaustive linter guard your switches.
- Use generics for containers and algorithms. Prefer loops and interfaces for everything else; iterators give you lazy, `Stream`-like sequences with plain `for range`.

Try it yourself

Exercise 1: A new sentinel error

In `examples/ch04-interfaces-errors-generics/errors/`, add `ErrOrderAlreadyShipped` and a `Service.Cancel` method that returns it (wrapped with the order ID) for shipped orders. Map it to `409 Conflict` in `statusFor` and add a test case.

Hint: add a `case errors.Is(...)` line before the default.

Exercise 2: A custom error type with data

Replace `ErrInsufficientStock` in the `Reserve` method with an `*InsufficientStockError` holding `ProductID`, `Wanted` and `Available`. Keep `errors.Is(err, ErrInsufficientStock)` working for existing callers.

Hint: give your type an `Is(target error) bool` method that returns `target == ErrInsufficientStock`.

Exercise 3: GroupBy

Write `GroupBy[T any, K comparable](items []T, key func(T) K) map[K][]T` and use it to group orders by status. Then ask yourself honestly whether the plain loop would have been clearer.

Exercise 4 (stretch): An iterator adapter

Write `FilterSeq[T any](seq iter.Seq[T], keep func(T) bool) iter.Seq[T]` and use it to rewrite `ByStatus` on top of `slices.Values(r.orders)`. Add a test that proves it stops early when the caller breaks.

Further reading

- Effective Go: the classic tour of interfaces, embedding, `defer` and errors in idiomatic style.
- Working with Errors in Go 1.13: the original explanation of wrapping, `errors.Is` and `errors.As`.
- Errors are values: Rob Pike on reducing `if err != nil` noise without new syntax.
- On | No syntactic support for error handling: why the Go team stopped pursuing new error syntax in 2025.
- When To Use Generics: practical guidance from the Go team on when type parameters help.
- Range Over Function Types: how and why iterators were added in Go 1.23.
- Go FAQ: Why is my nil error value not equal to nil?: the official explanation of the nil interface trap.

CHAPTER 5

Concurrency: Goroutines, Channels and Context

> *Concurrency is the reason many teams pick Go, and it's friendlier than you might fear: a handful of small tools covers almost everything an enterprise service needs.*

🎯 **What you'll learn**

- What a goroutine is, how it compares with platform and virtual threads, and how to wait for a group of them
- How channels and `select` move data and signals between goroutines, and when a mutex is the better tool
- How `context.Context` carries cancellation, deadlines and request values through your whole call chain
- How to run tasks concurrently with `errgroup` so the first failure cancels the rest
- Six production patterns: worker pools, pipelines, fan-out/fan-in, semaphores, rate limiting and timeouts
- How to find goroutine leaks and data races, and how to test time-based code instantly with `testing/synctest`

The big picture

A web service is concurrent whether you like it or not. Hundreds of requests arrive at once, each one calls a database and two other services, and background jobs run alongside. In Spring, a thread pool hides most of this from you until something goes wrong.

Go puts concurrency in the language itself, and keeps it small. You'll learn one keyword (`go`), one data type (channels), one statement (`select`), a few helpers from `sync`, and `context`. That's the toolkit.

🧩 **Think of it like a restaurant kitchen**

Goroutines are cooks: cheap to hire, and you can have thousands. Channels are the pass where finished plates are handed from one station to the next. A mutex is the single key to the walk-in fridge. And `context` is the head chef shouting "table 12 left, stop cooking their order!"

Java ↔ Go at a glance

Concept	Java 21 / Spring	Go
Lightweight task	Virtual thread	Goroutine
Start one	`executor.submit(...)`	`go f()`
Wait for many	`CountDownLatch`	`sync.WaitGroup`
Hand data over	`BlockingQueue`	Channel
Wait on several events	`CompletableFuture.anyOf`	`select`
Lock	`synchronized`, `ReentrantLock`	`sync.Mutex`
Lazy singleton	Holder idiom	`sync.OnceValue`
Atomic counter	`AtomicLong`	`atomic.Int64`
Cancel and deadlines	`Future.cancel`, interrupts	`context.Context`
Request-scoped data	`ThreadLocal`, `ScopedValue`	`context.Context` values
Structured tasks	`StructuredTaskScope` (preview)	`errgroup`

Goroutines

What a goroutine is

A **goroutine** is a function running independently of the code that started it, managed by the Go runtime rather than the operating system. You start one by putting `go` in front of a function call. That's all the syntax there is.

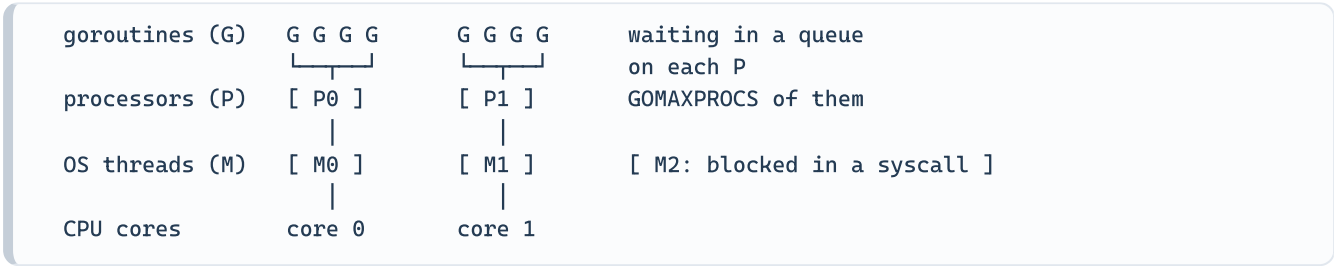
If you've used Java 21's virtual threads, you already have the right intuition. Here's how the three options compare:

	Platform thread	Java 21 virtual thread	Goroutine
Starting size	About 1 MB stack reserved	Small, grows on the heap	A few KB stack, grows
Scheduled by	The OS kernel	JVM, onto carrier threads	Go runtime, onto OS threads
Practical count	Thousands	Millions	Millions
Blocking I/O	Blocks the OS thread	Unmounts from its carrier	Parks; the OS thread moves on
Preemption	By the OS	Only when it blocks	Yes, since Go 1.14

The two lightweight options are close cousins. The differences are history: virtual threads were added to a 25-year-old platform, so older libraries can still *pin* a carrier thread (fixed for `synchronized` in Java 24). Goroutines have existed since Go 1.0, so every Go library expects them.

The scheduler in one picture

The runtime runs many goroutines on a few OS threads. This is called **M:N scheduling**: M goroutines on N threads.



Each **P** is a "licence to run Go code", and there are `GOMAXPROCS` of them (by default, the number of CPUs; since Go 1.25 it also respects container CPU limits on Linux). When a goroutine waits on the network, the runtime parks it and runs another. When an OS thread blocks in a system call, its P moves to another thread. You don't manage any of this. Chapter 17 goes deeper.

Starting goroutines and waiting with WaitGroup

Let's send order confirmation emails concurrently. In Java you might use a virtual-thread executor and a `CountDownLatch`:

JAVA

```
var latch = new CountDownLatch(orders.size());
try (var executor = Executors.newVirtualThreadPerTaskExecutor()) {
    for (Order o : orders) {
        executor.submit(() -> {
            try {
                sendConfirmation(o);
            } finally {
                latch.countDown();
            }
        });
    }
    latch.await();
}
```

Go's `sync.WaitGroup` is a counter you can wait on. Here's the classic form:

GO goroutines/main.go

```go
func notifyAllClassic(orders []Order) []string {
    results := make([]string, len(orders))
    var wg sync.WaitGroup
    for i, o := range orders {
        wg.Add(1) // count it BEFORE the goroutine starts
        go func() {
            // Done says "I'm finished", even if we return early.
            defer wg.Done()
            // Each goroutine writes only its own slot: no sharing.
            results[i] = sendConfirmation(o)
        }()
    }
    wg.Wait() // block until the counter is back to zero
    return results
}
```

Go 1.25 added `wg.Go`, which does the `Add` and `Done` for you. Prefer it in new code:

GO goroutines/main.go

```go
for i, o := range orders {
    wg.Go(func() {
        results[i] = sendConfirmation(o)
    })
}
wg.Wait()
```

Each email takes 100ms, and goroutines are so cheap that 100,000 sleeping ones are no trouble:

OUTPUT

```
emailed ana@example.com about ord-1
emailed ben@example.com about ord-2
emailed chen@example.com about ord-3
3 emails took 100ms
100,000 goroutines finished in 1.91s
```

Your timing will differ, but notice the scale. A hundred thousand platform threads would exhaust most machines.

> ⚠️ **Gotcha: Add before go, and never panic**
>
> Call `wg.Add(1)` *before* the `go` statement, never inside the goroutine, or `Wait` may return too early (`go vet` flags this since Go 1.25). And remember from Chapter 4: a panic in any goroutine crashes the whole program, so goroutines that run risky code need their own `recover`.

Since Go 1.22, each loop iteration gets fresh `i` and `o` variables, so capturing them in the closure is safe. Older blog posts show a `o := o` line; you no longer need it.

Full code: `examples/ch05-concurrency/goroutines/`

Channels

Passing data safely

Goroutines often need to hand work or results to each other. Go's motto is *"Do not communicate by sharing memory; instead, share memory by communicating."* A **channel** is a typed pipe: one goroutine sends a value in, another receives it.

> 🧩 **Think of it like a baton or a conveyor belt**
>
> An **unbuffered** channel is a relay baton: the runner can't let go until the next runner grabs it, so both meet at the same moment. A **buffered** channel is a conveyor belt with a few slots: the sender drops items and walks away until the belt is full, and the receiver picks them up when ready.

```
unbuffered: make(chan Order)        buffered: make(chan Order, 3)

sender ──▶ ● ──▶ receiver           sender ──▶ [ ▪ ▪ ▫ ] ──▶ receiver
both wait until the hand-off        sender waits only when full,
happens                             receiver only when empty
```

Java developers know this as `BlockingQueue`. An unbuffered channel behaves like a `SynchronousQueue`, and a buffered one like an `ArrayBlockingQueue`. Two things are new: a channel can be **closed** to say "no more values", and `for range` reads until that happens.

Close, range and directional types

Here's an order-packing conveyor belt. Notice the arrow in the parameter types: `chan<- string` can only send, `<-chan string` can only receive, and the compiler enforces it.

GO channels/main.go

```go
// produce only sends: chan<- string is a send-only channel.
func produce(orderIDs []string, out chan<- string) {
    for _, id := range orderIDs {
        out <- id // blocks if the buffer is full
    }
    close(out) // "no more orders": ends the consumer's range loop
}

// consume only receives: <-chan string is a receive-only channel.
func consume(in <-chan string) int {
    packed := 0
    for id := range in { // runs until in is closed AND empty
        fmt.Println("packing", id)
        packed++
    }
    return packed
}
```

GO channels/main.go

```go
belt := make(chan string, 2)
go produce([]string{"ord-1", "ord-2", "ord-3"}, belt)
fmt.Println("packed", consume(belt), "orders")
```

OUTPUT

```
packing ord-1
packing ord-2
packing ord-3
packed 3 orders
```

Channels have a few sharp edges, and this table is worth bookmarking:

Operation	Open channel	Closed channel	`nil` channel
Send `ch <- v`	Waits for room	**Panics**	Blocks forever
Receive `<-ch`	Waits for a value	Remaining values, then zero value and `ok == false`	Blocks forever
`close(ch)`	Closes it	**Panics**	**Panics**

> 💡 **Who closes a channel?**
>
> The **sender** closes, and only when receivers need to know that no more values are coming (for example, to end a `range` loop). Receivers never close. A channel nobody closes is fine; the garbage collector cleans it up.

A channel of `struct{}` (an empty struct, which takes no memory) makes a neat signal. Closing it wakes every goroutine waiting on it at once, which is how "done" and "stop" signals work throughout Go.

Full code: `examples/ch05-concurrency/channels/`

select: waiting on several things at once

Often a goroutine needs to wait for whichever happens *first*: a result, a timeout, or a cancellation. `select` is a `switch` for channel operations. It blocks until one case can proceed, then runs that case.

> 🧩 **Think of it like a receptionist with several phone lines**
>
> The receptionist answers whichever line rings first. With a `default` case, they glance at the phones and, if nothing is ringing, carry on with other work.

Here are the three shapes you'll use constantly:

GO selecting/main.go

```go
// A timeout: whichever happens first wins.
select {
case p := <-fetchPrice(sku, delay):
    return p, nil
case <-time.After(timeout):
    return 0, errors.New("pricing service timed out")
}
```

GO selecting/main.go

```go
// Non-blocking: if the buffer is full, drop the event and move on.
select {
case events <- e:
    return true
default: // runs immediately if no other case is ready
    return false
}
```

GO selecting/main.go

```go
// Stop waiting when the request is cancelled or times out.
select {
case id := <-paid:
    return id, nil
case <-ctx.Done():
    return "", ctx.Err()
}
```

OUTPUT

```
1250 <nil>
0 pricing service timed out
true true false
payment "": context deadline exceeded
```

That third shape, with `ctx.Done()`, is the most important line in Go concurrency. We'll meet it again in every pattern below.

One more trick: a `nil` channel blocks forever, so setting a finished channel variable to `nil` switches its `select` case off. The example's `mergeUntilDone` uses this to read two channels until both are closed.

Full code: `examples/ch05-concurrency/selecting/`

Shared memory: mutexes, Once and atomics

Channels aren't the only tool. When several goroutines share *state* (a stock map, a cache, a counter), protecting it with a lock is often simpler.

sync.Mutex vs synchronized

Java's `synchronized` locks an object's built-in monitor; `ReentrantLock` is the explicit version. Go's `sync.Mutex` is like a `ReentrantLock` without the re-entry: locking it twice from the same goroutine deadlocks. Its zero value is ready to use.

GO locks/main.go

```go
type Inventory struct {
    mu        sync.Mutex
    available map[string]int
}

func (inv *Inventory) Reserve(productID string, qty int) error {
    inv.mu.Lock()
    defer inv.mu.Unlock() // unlocks on every return path
    if inv.available[productID] < qty {
        return ErrInsufficientStock
    }
    inv.available[productID] -= qty
    return nil
}
```

The demo sends 100 customers after 50 mugs at once. Exactly 50 succeed:

OUTPUT

```
reserved: 50 rejected: 50 left: 0
```

For data that's read far more often than written, `sync.RWMutex` lets many readers in at once (`RLock` / `RUnlock`) but gives a writer exclusive access (`Lock` / `Unlock`), like Java's `ReentrantReadWriteLock`.

> ⚠️ **Gotcha: never copy a mutex**
>
> A struct containing a mutex must be passed by pointer. A copy gets its own, separate lock, and the protection silently disappears. `go vet` reports these copies (its `copylocks` check).

sync.OnceValue vs the lazy holder idiom

Java's classic lazy singleton is the holder idiom, which leans on class loading for thread safety:

JAVA

```java
class TaxRates {
    private static class Holder {
        static final Map<String, Integer> RATES = load();
    }
    static Map<String, Integer> get() { return Holder.RATES; }
}
```

Go says it directly. `sync.OnceValue` (Go 1.21) wraps a function so it runs once, on first use, however many goroutines call it at the same moment:

GO locks/main.go

```go
var taxRates = sync.OnceValue(func() map[string]int {
    fmt.Println("loading tax rates (this prints once)")
    return map[string]int{"UK": 20, "DE": 19} // percent
})
```

Call it as `taxRates()["UK"]`. There's also `sync.Once` for functions without a result, and `sync.OnceValues` for a value plus an error.

Atomics and sync.Map

For a single number, a lock is overkill. `sync/atomic` has typed atomics such as `atomic.Int64` with `Add`, `Load`, `Store` and `CompareAndSwap`, the same idea as Java's `AtomicLong`:

GO locks/main.go

```go
var ordersPlaced atomic.Int64 // a lock-free counter

ordersPlaced.Add(1)
fmt.Println(ordersPlaced.Load())
```

`sync.Map` looks like Java's `ConcurrentHashMap`, but it isn't the default choice. It's untyped (keys and values are `any`) and tuned for two cases: entries written once and read many times, or goroutines working on separate keys. For everything else, a plain map guarded by a `sync.Mutex`, like `Inventory` above, is typed, clear and usually as fast.

Channels or mutexes? A clear recommendation

This question causes more debate than it deserves. Here's a rule that works:

Situation	Reach for
Handing work or results from one goroutine to another	A channel
Signalling "done" or "stop"	`context`, or closing a channel
Protecting shared state (map, cache, counter)	A mutex or an atomic
Waiting for a group of tasks	`sync.WaitGroup` or `errgroup`

> 💡 **If you're unsure**
>
> Use a **mutex for state** and **channels for flow**. If you find yourself building a channel-based "manager goroutine" whose only job is to guard a map, swap it for a mutex. If you're passing a lock around so goroutines can take turns handing data over, use a channel.

Full code: `examples/ch05-concurrency/locks/`

context.Context: the stop button that travels with the request

Why context exists

A customer clicks "Place order", then closes the browser. In a well-behaved service, every database query and downstream call for that request should stop. And if the whole request has a two-second budget, every call inside it should respect what's left.

In Java, that information is scattered: thread interrupts, `Future.cancel`, per-client timeout settings, and `ThreadLocal` for request data. Go puts all of it in one value, `context.Context`, passed explicitly as the first parameter of every function that does I/O.

> 🧩 **Think of it like a ticket with a stop button**
>
> Every request carries a ticket. Printed on it: a deadline, and a few labels such as the request ID. Attached to it: a big red stop button. Anyone holding the ticket can check the time left or see if the button was pressed. Pass the ticket to a helper, and the helper can't miss the stop signal.

Contexts form a tree. Cancelling a parent cancels all its children, but never the other way round:

```
context.Background()
  └── r.Context()                  cancelled if the client disconnects
        └── WithTimeout(2s)        the whole request's budget
              ├── WithTimeout(200ms)   call the inventory service
              └── WithTimeout(500ms)   call the payment provider
```

Deadlines and timeouts

Here's a payment call with a 100ms budget, against a provider that needs 500ms:

GO contexts/main.go

```go
func chargeCard(ctx context.Context, cents int64,
    delay time.Duration) error {
    select {
    case <-time.After(delay):
        fmt.Printf("[%s] charged %d cents\n", RequestID(ctx), cents)
        return nil
    case <-ctx.Done():
        return fmt.Errorf("charge card: %w", ctx.Err())
    }
}
```

GO contexts/main.go

```go
ctx, cancel := context.WithTimeout(base, 100*time.Millisecond)
defer cancel() // always release the timer, even on success
err := chargeCard(ctx, 4499, 500*time.Millisecond)
fmt.Println(err, "| deadline?", errors.Is(err, context.DeadlineExceeded))
```

OUTPUT

```
charge card: context deadline exceeded | deadline? true
[req-42] charged 1250 cents
```

`ctx.Err()` returns `context.DeadlineExceeded` when time runs out and `context.Canceled` when someone pressed stop. Real libraries (`net/http`, pgx, the Kafka clients) all watch `ctx.Done()` for you, so passing the context along is usually all you need to do. For your own loops, `context.WithCancel` gives you the stop button directly, and the example's `pollOutbox` shows a background loop that exits when it's pressed.

Request-scoped values

A context can also carry values. Use an unexported key type so no other package can collide with your key:

GO contexts/main.go

```go
type ctxKey int

const requestIDKey ctxKey = 0

func WithRequestID(ctx context.Context, id string) context.Context {
    return context.WithValue(ctx, requestIDKey, id)
}

func RequestID(ctx context.Context) string {
    id, _ := ctx.Value(requestIDKey).(string) // "" if not set
    return id
}
```

Coming from Java: ThreadLocal and ScopedValue

`ThreadLocal` is implicit and tied to a thread, so it breaks when work hops threads (async code, reactive code, thread pools). Java 25's `ScopedValue` is closer to Go: immutable and bound to a scope. Go's context goes one step further: it's an ordinary parameter, so it flows into goroutines, across function boundaries and into tests without any magic.

The rules

These conventions are followed by every serious Go codebase:

- **First parameter, named `ctx`:** `func (s *Service) Place(ctx context.Context, o Order)`.
- **Never store it in a struct.** A context belongs to one operation; pass it to each call.
- **Always call `cancel`**, usually with `defer`, or the context's resources stay alive until its timer fires or its parent ends. `go vet` warns you if you forget (its `lostcancel` check).
- **Values only for request-scoped data** such as request IDs, the authenticated user or trace IDs. Never for optional parameters or dependencies like a database pool.
- **Never pass `nil`.** If you don't have one yet, use `context.TODO()` as a visible reminder.

WithoutCancel and AfterFunc

Two Go 1.21 helpers solve specific problems. `context.WithoutCancel` keeps a context's values but drops its cancellation: handy for an audit write that must finish even if the client disconnected. `context.AfterFunc` runs a function when a context is done, for cleanup such as closing a connection:

GO contexts/main.go

```
reqCtx, endRequest := context.WithCancel(base)
auditCtx := context.WithoutCancel(reqCtx)
endRequest()
fmt.Println(reqCtx.Err(), "|", auditCtx.Err(), "|", RequestID(auditCtx))
```

OUTPUT

```
context canceled | <nil> | req-42
```

Full code: `examples/ch05-concurrency/contexts/`

errgroup: structured concurrency for Go

Placing an order needs a stock check and a price for every item. Running them one by one is slow; running them all at once is fast. And if any item is out of stock, there's no point waiting for the rest.

In Java you might reach for `CompletableFuture.allOf`:

JAVA

```
List<CompletableFuture<?>> tasks = new ArrayList<>();
for (OrderItem item : items) {
    tasks.add(CompletableFuture.runAsync(
        () -> inventory.check(item), executor));
    tasks.add(CompletableFuture.supplyAsync(
        () -> pricing.price(item.productId()), executor));
}
// Waits for ALL tasks, even after one fails, and cancels nothing.
CompletableFuture.allOf(tasks.toArray(CompletableFuture[]::new)).join();
```

Java 21's preview `StructuredTaskScope.ShutdownOnFailure` fixes that: the first failure cancels its siblings. Go's `errgroup` package (from `golang.org/x/sync`, maintained by the Go team) has worked that way for years:

GO errgroups/main.go

```
func PrepareOrder(ctx context.Context, inv *Inventory, pr *Pricing,
    items []Item) ([]PricedItem, error) {
    g, ctx := errgroup.WithContext(ctx) // ctx is cancelled on first error
    g.SetLimit(4)                       // at most 4 calls in flight

    priced := make([]PricedItem, len(items))
    for i, it := range items {
        g.Go(func() error {
            return inv.Check(ctx, it)
        })
```

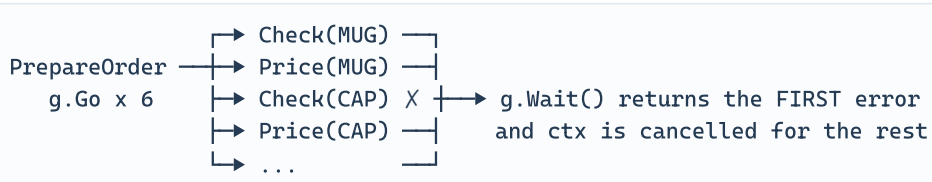
```
        g.Go(func() error {
            cents, err := pr.Price(ctx, it.ProductID)
            if err != nil {
                return err
            }
            priced[i] = PricedItem{Item: it, UnitPriceCents: cents}
            return nil
        })
    }
    if err := g.Wait(); err != nil { // waits for all, returns first error
        return nil, fmt.Errorf("prepare order: %w", err)
    }
    return priced, nil
}
```

Here's what happens when one item, `CAP`, is out of stock:

OUTPUT

```
happy path: [{{MUG 2} 1250} {{TEE 1} 1999}] <nil>
took about 100ms
sad path: prepare order: check CAP: insufficient stock
out of stock? true | gave up early? true
```

The happy path ran four 100ms calls in about 100ms total. The sad path gave up almost immediately: the failing check cancelled `ctx`, and every other call saw `ctx.Done()` and returned.



> 💡 **If you're unsure, use errgroup**
>
> Whenever you'd write a `WaitGroup` plus an error variable plus a mutex, use `errgroup` instead. `SetLimit` turns it into a bounded worker pool too. Use the `ctx` that `WithContext` returns inside the tasks, and don't use it after `Wait`: it's cancelled by then.

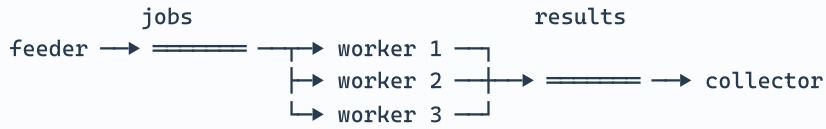
Full code: `examples/ch05-concurrency/errgroups/`

Concurrency patterns

These six patterns cover most of what enterprise services need. Each one is in the examples with tests.

Worker pool

A fixed number of workers share one jobs channel. It's Go's version of a `ThreadPoolExecutor` with a fixed size, and it's useful when each job is heavy, like rendering invoice PDFs.



GO workerpool/main.go

```go
func RunPool(n int, orderIDs []string) []Result {
    jobs := make(chan Job)
    results := make(chan Result)

    var wg sync.WaitGroup
    for w := 1; w <= n; w++ {
        wg.Go(func() { worker(w, jobs, results) }) // range over jobs
    }
    go func() { // feed the jobs, then say "no more work"
        for _, id := range orderIDs {
            jobs <- Job{OrderID: id}
        }
        close(jobs)
    }()
    go func() { // close results once every worker has returned
        wg.Wait()
        close(results)
    }()

    var out []Result
    for r := range results {
        out = append(out, r)
    }
    return out
}
```

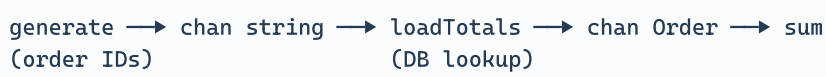
OUTPUT

```
invoices: 12 | workers used: 3
took about 80ms (12 jobs x 20ms / 3 workers = 80ms)
```

Notice the two closing goroutines. Closing `jobs` lets workers' `range` loops end; closing `results` after `wg.Wait()` lets the collector's loop end. For many cases, `errgroup` with `SetLimit(n)` gives you the same pool in fewer lines.

Pipeline

A pipeline is a chain of stages joined by channels, like an assembly line. Each stage is a goroutine that reads from the previous channel and writes to the next.



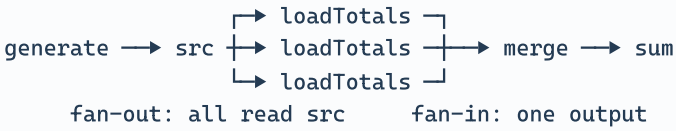
Every stage follows one rule: whenever it sends, it also watches `ctx.Done()`, so the whole line stops if the consumer walks away.

GO pipeline/main.go

```go
func generate(ctx context.Context, ids ...string) <-chan string {
    out := make(chan string)
    go func() {
        defer close(out)
        for _, id := range ids {
            select {
            case out <- id:
            case <-ctx.Done(): // downstream gave up: stop early
                return
            }
        }
    }()
    return out
}
```

Fan-out and fan-in

Fan-out means several goroutines read from the same channel to share slow work. **Fan-in** means merging their output channels back into one.



GO pipeline/main.go

```go
func merge(ctx context.Context, ins ...<-chan Order) <-chan Order {
    out := make(chan Order)
    var wg sync.WaitGroup
    for _, in := range ins {
        wg.Go(func() {
            for o := range in {
                select {
                case out <- o:
                case <-ctx.Done():
                    return
                }
            }
        })
    }
    go func() {
        wg.Wait()  // every input is drained...
        close(out) // ...so nobody will send on out again
    }()
    return out
}
```

With three loaders instead of one, six 30ms lookups take a third of the time:

OUTPUT

```
1 loader(s): 6 orders, 29247 cents, about 180ms
3 loader(s): 6 orders, 29247 cents, about 60ms
```

Full code: `examples/ch05-concurrency/pipeline/`

Semaphore

A **semaphore** limits how many things happen at once, for example "no more than 3 calls to the supplier API". A buffered channel makes a perfectly good one: its capacity is the number of permits.

GO semaphore/main.go

```go
sem := make(chan struct{}, limit)
var wg sync.WaitGroup
for _, sku := range skus {
    wg.Go(func() {
        sem <- struct{}{}        // acquire: blocks while all are taken
        defer func() { <-sem }() // release
        t.callSupplier(sku)
    })
}
wg.Wait()
```

When waiting should respect a context, or some jobs need more than one permit, use `golang.org/x/sync/semaphore`:

GO semaphore/main.go

```go
sem := semaphore.NewWeighted(limit)
// ...
if err := sem.Acquire(ctx, 1); err != nil { // gives up if ctx is done
    return fmt.Errorf("waiting for a slot: %w", err)
}
wg.Go(func() {
    defer sem.Release(1)
    t.callSupplier(sku)
})
```

OUTPUT

```
channel semaphore, peak in flight: 3
x/sync/semaphore, peak in flight: 3 <nil>
```

Rate limiting

A semaphore limits *how many at once*; a rate limiter limits *how many per second*. For steady pacing, `time.Ticker` delivers a tick at a fixed interval. For the common "average rate plus small bursts" rule, use `golang.org/x/time/rate`, a **token bucket**: tokens drip into a bucket at a fixed rate, and each call takes one.

GO ratelimit/main.go

```go
// 10 calls per second on average, with bursts of up to 3.
lim := rate.NewLimiter(rate.Limit(10), 3)
for i := range n {
    if err := lim.Wait(ctx); err != nil { // waits for a token
        return err
    }
    call(i)
}
```

OUTPUT

```
rate.Limiter, 10/s with a burst of 3:
  call 0 at 0s
  call 1 at 0s
  call 2 at 0s
  call 3 at 100ms
  call 4 at 200ms
```

`lim.Wait` queues the caller. For incoming HTTP traffic you usually want `lim.Allow()` instead, which never waits, so you can reply `429 Too Many Requests`. The example includes that middleware, and Chapter 14 covers resilience patterns in more depth.

Timeouts

The timeout pattern is a context deadline per outgoing call, so one slow dependency can't eat the whole request's budget. Set `http.Client.Timeout` too, as a safety net for calls that forget.

GO timeouts/main.go

```go
func fetchStock(ctx context.Context, client *http.Client, baseURL,
    productID string) (string, error) {
    ctx, cancel := context.WithTimeout(ctx, 200*time.Millisecond)
    defer cancel()

    url := baseURL + "/api/v1/inventory/" + productID
    req, err := http.NewRequestWithContext(ctx, http.MethodGet, url, nil)
    if err != nil {
        return "", err
    }
    resp, err := client.Do(req)
    if err != nil {
        return "", fmt.Errorf("fetch stock %s: %w", productID, err)
    }
    defer resp.Body.Close()
    body, err := io.ReadAll(resp.Body)
    return strings.TrimSpace(string(body)), err
}
```

OUTPUT

```
MUG-1: "5 available", err: <nil>
SLOW-1 deadline exceeded? true | after 200ms
```

Because the child context can never outlive its parent, a caller with only 20ms left still wins over the 200ms budget here. The test file checks exactly that.

Full code: `examples/ch05-concurrency/` (`workerpool/`, `semaphore/`, `ratelimit/`, `timeouts/`)

Goroutine leaks

A **goroutine leak** is a goroutine that blocks forever: waiting to send on a channel nobody reads, or to receive from one nobody closes. Each one holds memory, and a leak in a request path grows until the pod is killed. It's the concurrency version of a memory leak.

Here's a classic. Ask three database replicas and take the fastest answer:

GO leaks/main.go

```go
func firstAnswerLeaky(replicas int) string {
    ch := make(chan string) // unbuffered
    for r := 1; r <= replicas; r++ {
        go func() { ch <- query(r) }()
    }
    return <-ch // we read ONE answer; the others block forever
}
```

OUTPUT

```
leaky: goroutines before 1 after 201
fixed: goroutines before 201 after 201
```

A hundred calls leaked two goroutines each. There are two fixes, and the second is better in services:

- **Make room:** `make(chan string, replicas)` lets every sender finish, even if nobody reads.
- **Cancel the losers:** create a context with `WithCancel`, `defer cancel()`, and have each query watch `ctx.Done()`. Now the slow queries stop working too. The example's `firstAnswerCtx` shows the full version.

The rule behind both: **whenever you start a goroutine, know how it will end.**

Catching leaks in tests with goleak

Uber's `goleak` checks that no unexpected goroutines are left when a test finishes. Add one line per test, or use `goleak.VerifyTestMain(m)` for a whole package:

GO leaks/main_test.go

```go
func TestFirstAnswerDoesNotLeak(t *testing.T) {
    defer goleak.VerifyNone(t) // retries briefly, then fails on leftovers
    if got := firstAnswer(3); got != "answer from replica 1" {
        t.Fatalf("got %q", got)
    }
}
```

Point the same test at the leaky version, and it fails with the stuck goroutine and the line it's stuck on (trimmed and wrapped to fit):

OUTPUT

```
--- FAIL: TestFirstAnswerDoesNotLeak (0.46s)
    leak_test.go:12: found unexpected goroutines:
        [Goroutine 9 in state chan send, with
        example.com/goguide/ch05/leaks.firstAnswerLeaky.func1 on top of
        the stack:
        example.com/goguide/ch05/leaks.firstAnswerLeaky.func1()
            .../leaks/main.go:24 +0x3e
```

In production, watch the goroutine count as a metric (Chapter 13) and look at the goroutine profile from `net/http/pprof` (Chapter 17).

> **Newer Go: a leak profile built in**
>
> Go 1.26 added an experimental `goroutineleak` profile, and Go 1.27 made it generally available, including a `/debug/pprof/goroutineleak` endpoint. It uses the garbage collector to find goroutines blocked on something no running goroutine can reach.

Full code: `examples/ch05-concurrency/leaks/`

Data races and the race detector

A **data race** happens when two goroutines touch the same memory at the same time, at least one of them writes, and nothing synchronises them. The result is unpredictable: lost updates, torn values, or a crash. Maps are especially strict: concurrent writes make the runtime stop the program with `fatal error: concurrent map writes`.

GO races/main.go

```go
func countRacy(n int) int {
    count := 0
    var wg sync.WaitGroup
    for range n {
        wg.Go(func() { count++ }) // RACE: read, add, write
    }
    wg.Wait()
    return count
}
```

OUTPUT

```
racy:   95290 (wrong, and different each run)
mutex:  100000
atomic: 100000
```

Go ships a **race detector**. Add `-race` to `go test`, `go run` or `go build`, and the program watches every memory access while it runs. When it sees a race, it prints a report headed `WARNING: DATA RACE`, showing the two conflicting accesses (for example a read and an earlier write) with a stack trace for each, plus where each goroutine was started. Under `go test`, the test then fails.

TERMINAL

```
go test -race ./...
```

A few things to know:

- It finds races that **actually happen** during the run, so it's only as good as your tests. When it reports one, treat it as a real bug.
- It costs roughly 5 to 10 times the memory and 2 to 20 times the CPU, so use it in tests and CI, not production.
- It needs **cgo**. On Windows that means `CGO_ENABLED=1` and a C compiler such as gcc from MinGW-w64; without them Go stops with `go: -race requires cgo; enable cgo by setting CGO_ENABLED=1`. Most teams run `go test -race ./...` in their Linux CI pipeline (Chapter 16 covers CI tooling).

Full code: `examples/ch05-concurrency/races/`

The memory model in plain English

Why did the racy counter lose updates? Because without synchronisation, nothing guarantees that one goroutine ever sees another's writes, or sees them in order. The **memory model** is the rulebook for when a write in one goroutine is guaranteed visible in another. It uses one idea: **happens-before**. If A happens before B, B sees everything A did.

Java has the same concept in the Java Memory Model. Go's version is short. These are the guarantees you'll lean on:

- A **send** on a channel happens before the matching **receive** completes.
- **Closing** a channel happens before a receive that returns because it's closed.
- `mu.Unlock()` happens before the next `mu.Lock()` returns.
- A goroutine's work happens before `wg.Wait()` returns for it.
- The function passed to `sync.Once` / `OnceValue` finishes before any caller gets its result.
- Atomic operations behave like Java `volatile`: everyone sees them in a single order.

```
goroutine A                          goroutine B
order.Status = "CONFIRMED"
done <- struct{}{}   ──────────────►  <-done
                                     fmt.Println(order.Status)
                                     // guaranteed to print CONFIRMED
```

The official document says it best: "*If you must read the rest of this document to understand the behavior of your program, you are being too clever. Don't be clever.*" Use channels, locks, `WaitGroup` and atomics, and the guarantees come for free.

Testing time-based code with testing/synctest

Code with timeouts and expiry is painful to test. Real sleeps make tests slow, and short sleeps make them flaky. In Java you'd inject a `java.time.Clock` and advance it by hand. Go 1.25 offers something smoother: `testing/synctest`.

Orderly holds reserved stock for 15 minutes. If the order isn't confirmed by then, a `time.AfterFunc` timer releases the units:

GO synctesting/holds.go

```go
func (h *Holds) Hold(orderID string, units int, ttl time.Duration) {
    h.mu.Lock()
    h.held[orderID] = units
    h.mu.Unlock()
    time.AfterFunc(ttl, func() { h.expire(orderID) }) // runs later
}
```

`synctest.Test` runs a test inside a **bubble**, an isolated world with a fake clock. When every goroutine in the bubble is blocked, the clock jumps straight to the next timer. `synctest.Wait` waits until every other goroutine in the bubble has settled.

GO synctesting/holds_test.go

```go
func TestHoldExpiresAfterFifteenMinutes(t *testing.T) {
    synctest.Test(t, func(t *testing.T) {
        h := NewHolds()
        h.Hold("ord-1", 2, 15*time.Minute)

        // The bubble's fake clock jumps ahead: this takes no real time.
        time.Sleep(15*time.Minute - time.Second)
        synctest.Wait() // let every goroutine in the bubble settle
        if got := h.Released(); got != 0 {
            t.Fatalf("released %d units a second too early", got)
        }

        time.Sleep(time.Second)
        synctest.Wait() // the AfterFunc callback has now run
        if got := h.Released(); got != 2 {
            t.Fatalf("released %d units, want 2", got)
        }
    })
}
```

OUTPUT

```
=== RUN   TestHoldExpiresAfterFifteenMinutes
--- PASS: TestHoldExpiresAfterFifteenMinutes (0.00s)
```

Fifteen simulated minutes, zero real seconds, and no clock interface in the production code. That's a small win worth celebrating.

> **Newer Go: synctest.Sleep**
>
> Go 1.27 adds `synctest.Sleep`, which combines `time.Sleep` and `synctest.Wait` in one call. On Go 1.25 and 1.26, write the two calls as above.

Full code: `examples/ch05-concurrency/synctesting/`

Recap

- A goroutine is a cheap, runtime-scheduled function call started with `go`. Wait for groups with `sync.WaitGroup` (`wg.Go` since 1.25), and know how every goroutine will end.
- Channels hand data between goroutines: unbuffered for a hand-off, buffered for a queue. The sender closes; `range` reads until closed.
- `select` waits on several channel operations; combine it with `time.After`, `default` and, above all, `ctx.Done()`.

- Use a mutex for shared state and channels for flow. `sync.OnceValue` replaces the holder idiom, and `atomic.Int64` replaces `AtomicLong`.
- `context.Context` carries cancellation, deadlines and request values. Pass it first, never store it, always call `cancel`.
- `errgroup` runs tasks concurrently, returns the first error and cancels the rest. `SetLimit` bounds it.
- Worker pools, pipelines, fan-out/fan-in, semaphores, rate limiters and timeouts all follow the same rules: close what you own, and watch `ctx.Done()` when sending.
- Catch leaks with `goleak`, races with `go test -race` in CI, and test timers instantly with `testing/synctest`.

Try it yourself

Exercise 1: A cancellable worker pool

Add a `ctx context.Context` parameter to `RunPool` in `examples/ch05-concurrency/workerpool/`. When the context is cancelled, the feeder should stop sending and the workers should stop. Add a test with `goleak.VerifyNone` that cancels halfway through.

Hint: replace each plain send with a `select` that also watches `ctx.Done()`.

Exercise 2: One more call per item

In `errgroups/`, add a `Shipping.Estimate(ctx, productID)` call for each item, with a 150ms fake delay. Try `SetLimit(2)`, `SetLimit(6)` and no limit, and explain the timings you see.

Exercise 3: A rate limit per customer

Change the `limitRequests` middleware in `ratelimit/` to keep one `*rate.Limiter` per customer, read from an `X-Customer-ID` header. Protect the map with a `sync.Mutex` and test that one busy customer can't block another.

Exercise 4 (stretch): Test the limiter without waiting

Write a `testing/synctest` test for `withLimiter` that proves calls 4 and 5 happen exactly 100ms and 200ms after the start, using `time.Now()` inside the bubble.

Hint: inside a bubble, `time.Now()` reads the fake clock, so the timings are exact.

Further reading

- The Go Memory Model: the official rules for happens-before, with that famous "don't be clever" advice.
- Go Concurrency Patterns: Pipelines and cancellation: the classic article on pipelines, fan-out and fan-in.
- Go Concurrency Patterns: Context: how and why context came to be.

- Contexts and structs: why a context belongs in parameters, not struct fields.
- errgroup package docs: the full API, including `TryGo` and `SetLimit`.
- Data Race Detector: how to read race reports, with typical races and their fixes.
- Testing concurrent code with testing/synctest: the Go team's introduction to bubbles and fake time.
- goleak: Uber's goroutine leak detector for tests.
- Java virtual threads guide: useful for comparing the JVM's model with goroutines.

CHAPTER 6

Structuring an Enterprise Project

> *A clear structure is what lets a Go service grow from one file to fifty developers without turning into a maze.*

🎯 **What you'll learn**

- Lay out a Go service with `cmd/`, `internal/` and friends, and why most teams skip `pkg/`
- Organise code by feature (`internal/order`) instead of by layer, and why Go rewards it
- Map handler, service and repository onto `@RestController`, `@Service` and `@Repository`
- Model an aggregate with real behaviour, value objects and domain errors (DDD-lite)
- Wire dependencies by hand in `main.go`, and know when Uber Fx or Wire are worth it
- Start and stop an application in the right order, and spot the classic structure mistakes

The big picture

Spring Initializr hands you a folder structure, a build file and a container that connects your beans. Go hands you an empty folder and a `go.mod` file. That freedom feels like vertigo at first, and every Java developer feels it.

The good news: the Go community has settled on a few conventions, and once you know them, Go projects are very easy to navigate. Most of the "framework" is a handful of folder names and one well-organised `main.go`.

🧩 **Think of it like moving into an unfurnished house**

Spring gives you a fully furnished flat: comfortable, but moving the sofa means reading the landlord's manual. Go gives you an empty house with a few labelled rooms (the front door is `cmd/`, the private rooms are `internal/`). You choose the furniture, and you always know where everything is because you put it there.

Here's the whole chapter in one table:

Concept	Java / Spring	Go
Project skeleton	Initializr, Maven layout	`cmd/`, `internal/` conventions
Unit of organisation	package + Maven module	package (a folder) + Go module
Hiding code	`private`, package-private, JPMS	lowercase names, `internal/` folders

Concept	Java / Spring	Go
Layers	`@RestController`, `@Service`, `@Repository`	plain structs: handler, service, repository
Dependency injection	Spring container at runtime	constructor calls in `main.go`
Lifecycle	`@PostConstruct`, `SmartLifecycle`	explicit start and stop in `main`
Multi-module	Maven reactor, Gradle composite	one module, or `go.work`

Full code: `examples/ch06-architecture/`. The example folder is itself laid out the way this chapter recommends, so you can open it next to the text.

Project layout

A predictable layout means a new teammate can find the entry point, the business rules and the database code in under a minute. Let's look at the folders you'll see in almost every Go service.

The folders everyone recognises

```
orderly/
├── go.mod                  module example.com/orderly
├── go.sum                  checksums of every dependency
├── cmd/
│   └── orderly/main.go     the program: wiring and startup only
├── internal/               private to this module (compiler-enforced)
│   ├── order/              the order feature: domain, service, ports
│   ├── inventory/          the inventory feature
│   ├── httpapi/            HTTP handlers (an adapter)
│   ├── postgres/           database code (an adapter)
│   └── platform/           config, logging, server setup
├── api/openapi.yaml        API contracts (OpenAPI, protobuf)
├── migrations/             SQL migrations
├── deploy/                 Dockerfile, Helm chart, compose file
└── Makefile
```

- **`cmd/<name>/`** holds one folder per executable, each a small `package main`. A server and an admin tool become `cmd/orderly/` and `cmd/orderly-admin/`.
- **`internal/`** holds everything else, behind a fence the compiler enforces (more in a moment).
- **`api/`** holds contracts other teams read: OpenAPI specs, `.proto` files, GraphQL schemas.
- **`migrations/`** and **`deploy/`** hold SQL migrations (Chapter 9) and container files (Chapter 18).

Compared with a Maven project

Here's the Spring Boot project you know:

```
orderly/
├── pom.xml
└── src/
    ├── main/
    │   ├── java/com/example/orderly/
    │   │   ├── OrderlyApplication.java
    │   │   ├── controller/OrderController.java
    │   │   ├── service/OrderService.java
    │   │   ├── repository/OrderRepository.java
    │   │   └── model/Order.java
    │   └── resources/
    │       ├── application.yml
    │       └── db/migration/V1__init.sql
    └── test/java/com/example/orderly/...
```

And here's how the pieces line up:

Maven / Spring	Go
`pom.xml`	`go.mod` (plus `go.sum` for checksums)
`OrderlyApplication.java`	`cmd/orderly/main.go`
`src/main/java/...`	`internal/...`
`src/test/java/...`	`*_test.go` files next to the code
`resources/application.yml`	environment variables (Chapter 10)
`resources/db/migration`	`migrations/`

Go has no `src/main/java/com/example` tunnel, so paths stay short. Tests live **next to** the code they test, which you'll come to love.

`internal/`: a fence the compiler enforces

In Java, a `public` class is visible to anyone with your JAR, unless you adopt the Java module system (JPMS). In Go, a package under a folder named `internal` can only be imported by code rooted at that folder's parent. So `example.com/orderly/internal/order` is usable inside `example.com/orderly/` and nowhere else. Try it from another module and the build fails:

OUTPUT

```
package example.com/b
  main.go:3:8: use of internal package example.com/a/internal/secret not allowed
```

> **The Go way**
>
> Put your code under `internal/` by default. It costs nothing, and it means you can rename, split or delete any package without breaking somebody else's build. You only promise stability for what you deliberately expose.

The `pkg/` debate

Many Go repositories have a `pkg/` folder, often copied from a popular GitHub template called "golang-standards/project-layout". Despite the name, it isn't an official standard, and the Go team's own guide, Organizing a Go module, doesn't use `pkg/` at all.

The idea was "code that other projects may import". That describes almost nothing in a service, so `pkg/` usually becomes a second `internal/` without the protection.

> 💡 **If you're unsure, skip pkg/**
>
> For services, use `cmd/` plus `internal/`. Only expose packages when you're truly publishing a library for other modules, and then put them at the module root (`example.com/orderly/client`) rather than under `pkg/`.

You also don't need every folder on day one. The official guide starts with a single `main` package and splits it as it grows: a flat package for a few files, `cmd/` plus `internal/` once there's a second binary, then one package per feature.

Package by feature, not by layer

The most important structural choice is how you split code into packages. Java projects usually split by technical layer. Go projects work best when you split by business feature.

> 🧩 **Think of it like a restaurant kitchen**
>
> A good kitchen is organised by station: the pastry station has its own flour, oven and knives; the grill station has its own. Nobody builds a "knife room" that every cook must run to. Package by feature is organising by station. Package by layer is the knife room.

Here's the layout most Java developers reach for first, followed by the one Go rewards:

Avoid

```
internal/
├── controllers/   order_controller.go, inventory_controller.go
├── services/      order_service.go, inventory_service.go
├── repositories/  order_repo.go, inventory_repo.go
└── models/        order.go, stock.go, product.go
```

Prefer

```
internal/
├── order/         order.go, service.go, service_test.go
├── inventory/     inventory.go, inventory_test.go
├── httpapi/       handler.go (all HTTP endpoints)
├── memory/        in-memory adapters (postgres/ in production)
└── platform/      config, logger, http.Server
```

Why does this matter more in Go than in Java?

1. **Visibility is per package.** If `services` needs a type from `models`, that type must be exported (capitalised) to the whole module. By-layer packages end up exporting nearly everything.
2. **Import cycles are forbidden.** If package `a` imports `b`, then `b` can never import `a`. By-layer packages hit this wall fast.
3. **The package name is part of every call.** `services.OrderService` says "service" twice and tells you nothing; `order.Service` reads well.

Repeating the package name inside an identifier is called **stutter**, and Go style avoids it. The shared Orderly domain calls the error `ErrOrderNotFound`, but inside package `order` we name it `ErrNotFound`, because callers already write `order.` in front.

> 🧠 **Key idea**
>
> Name packages after what they provide (`order`, `inventory`, `httpapi`), never after what kind of code they hold (`models`, `services`, `utils`). Short, lowercase, singular.

Layers: handler, service, repository

Separating layers still matters in Go, even without a package per layer. The classic three layers map neatly onto what you know from Spring.

In Spring, a service looks like this:

JAVA

```java
@Service
public class OrderService {
    private final OrderRepository repo;
    private final InventoryClient inventory;

    public OrderService(OrderRepository repo, InventoryClient inventory) {
        this.repo = repo;
        this.inventory = inventory;
    }

    @Transactional
    public Order place(UUID customerId, List<OrderItem> items) { ... }
}
```

In Go, it's a plain struct with a constructor function:

GO internal/order/service.go

```go
// Service holds the use cases (the "application service").
type Service struct {
    repo   Repository
    stock  StockReserver
    events EventPublisher
    log    *slog.Logger
    newID  func() string
    now    func() time.Time
}

// NewService takes every dependency as an argument: no globals,
// no container, no magic.
func NewService(repo Repository, stock StockReserver,
    events EventPublisher, log *slog.Logger) *Service {
    return &Service{
        repo: repo, stock: stock, events: events, log: log,
        newID: uuid.NewString,
        now:   time.Now,
    }
}
```

What's different? No annotations, no proxies generated at startup, no classpath scanning. `NewService` is the constructor injection you already practise, minus the container calling it for you. Transactions are explicit too (Chapter 9).

The `newID` and `now` fields let tests make IDs and time predictable, like injecting a `Clock` in Java.

Interfaces live where they're used

Here's the habit that surprises Java developers most. In Spring, `OrderRepository` is an interface next to its implementation (or generated by Spring Data). In Go, the **consumer** declares the interface, listing only the methods it needs:

GO internal/order/service.go

```go
// Repository is what the order service needs from storage. It lives
// here, next to the code that uses it, not next to the database code.
type Repository interface {
    Save(ctx context.Context, o *Order) error
    FindByID(ctx context.Context, id string) (*Order, error)
}

// StockReserver is the small slice of inventory this service uses.
type StockReserver interface {
    Reserve(ctx context.Context, productID string, qty int) error
    Release(ctx context.Context, productID string, qty int) error
}
```

Because Go interfaces are satisfied implicitly (Chapter 4), `*inventory.Service` fits `StockReserver` without ever mentioning it. The `order` package doesn't import `inventory` at all; only `main.go` knows that one plugs into the other.

The same happens one layer up: `httpapi` declares its own `OrderService` interface with the three methods its handlers call, and `*order.Service` satisfies it.

If you want the compiler to confirm that an adapter satisfies a port, add this one-line check next to the adapter. It costs nothing at runtime:

GO internal/memory/orders.go

```go
// Compile-time check: *OrderStore satisfies order.Repository.
var _ order.Repository = (*OrderStore)(nil)
```

> **Coming from Java**
>
> Unlearn the `IOrderService` / `OrderServiceImpl` pair. In Go, a struct doesn't need an interface until someone needs to swap it. The Go proverb is **"accept interfaces, return structs"**: constructors return concrete types, and the code that receives them declares the small interface it wants.

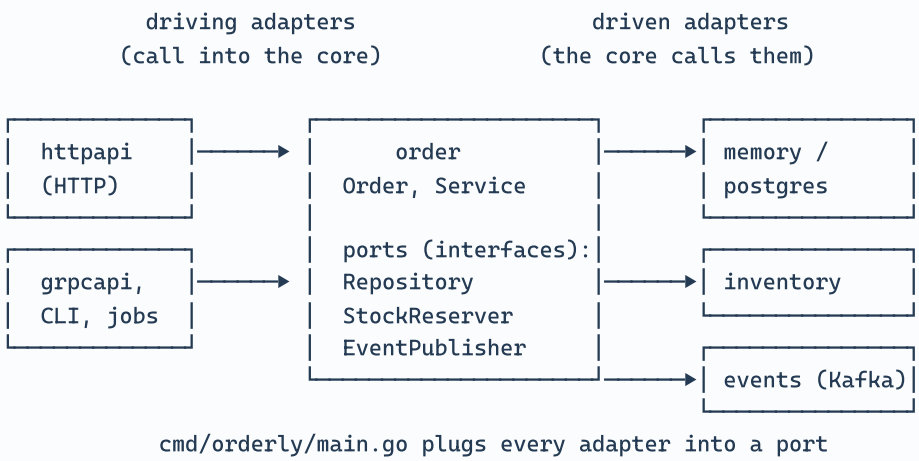
The payoff shows up in tests. The fake `StockReserver` in `service_test.go` is a dozen lines, with no mocking library.

Hexagonal architecture without the ceremony

Hexagonal architecture, also called **ports and adapters**, keeps business rules independent of technology. A **port** is an interface the core needs ("save an order"). An **adapter** is a concrete piece of technology that fulfils a port, such as a Postgres repository, an HTTP handler or a Kafka publisher.

> 🧩 **Think of it like a travel plug adapter**
>
> Your laptop (the core) has one kind of plug (the port). In each country you buy a cheap adapter that fits the local socket. You never rewire the laptop to travel. Swap Postgres for an in-memory store in tests the same way: change the adapter, keep the core.

Here's how the example's packages arrange themselves:



In Go this needs no framework. Ports are small interfaces in the feature package, adapters are structs in their own packages, the core imports neither HTTP nor SQL, and `main.go` plugs adapters into ports.

> ⚠️ **Gotcha: don't over-engineer it**
>
> You don't need `ports`, `domain`, `application` and `infrastructure` packages per feature, or a DTO and mapper for every layer. If adding one field means editing six files, the structure is working against you. Add layers only when a real need appears.

DDD-lite: aggregates, value objects and domain errors

Many Spring codebases end up with an **anaemic domain model**: entities that are bags of getters and setters, while a service class holds every rule. You don't need the full Domain-Driven Design (DDD) toolbox to do better; three ideas carry most of the value.

Aggregates with behaviour

An **aggregate** is a cluster of objects that changes together and protects its own rules, entered through its **aggregate root**. For us, `Order` (with its items) is the aggregate, and one rule is "you can cancel an order that hasn't shipped". In Java, you'd write something like this:

JAVA

```java
@Entity
public class Order {
    @Id private UUID id;
    @Enumerated(EnumType.STRING) private OrderStatus status;

    public void cancel() {
        switch (status) {
            case SHIPPED -> throw new IllegalStateException("shipped");
            case CANCELLED -> throw new IllegalStateException("cancelled");
            default -> status = OrderStatus.CANCELLED;
        }
    }
}
```

In Go, the rule lives in a method, and the status field is unexported (lowercase) so no other package can set it directly:

GO internal/order/order.go

```go
type Order struct {
    ID         string
    CustomerID string
    Items      []Item
    CreatedAt  time.Time
    status     Status // unexported: only methods below may change it
}

// Cancel enforces the business rule: you can cancel any order that
// hasn't shipped yet, and cancelling twice is an error.
func (o *Order) Cancel() error {
    switch o.status {
    case StatusShipped:
        return ErrAlreadyShipped
    case StatusCancelled:
        return ErrAlreadyCancelled
    }
    o.status = StatusCancelled
    return nil
}

// Status reports the current status.
func (o *Order) Status() Status { return o.status }
```

The method has a **pointer receiver** (`*Order`) because it changes the order, and a broken rule comes back as an `error` value rather than an exception. The service stays thin because the rule lives in the aggregate:

GO internal/order/service.go

```go
func (s *Service) Cancel(ctx context.Context, id string) (*Order, error) {
    o, err := s.repo.FindByID(ctx, id)
    if err != nil {
        return nil, err
    }
    if err := o.Cancel(); err != nil { // the rule lives in the aggregate
        return nil, err
    }
    if err := s.repo.Save(ctx, o); err != nil {
        return nil, fmt.Errorf("save order: %w", err)
    }
    s.release(ctx, o.Items)
    s.publish(ctx, "OrderCancelled", o.ID)
    return o, nil
}
```

> **Should every field be unexported?**
>
> No. Unexport the fields that carry rules (like `status`) and keep plain data exported. A database adapter then needs a small constructor to rebuild an order from a row, a fair price for rules nobody can bypass.

Value objects

A **value object** has no identity: two of them with the same contents are interchangeable, like two five-pound notes. Go's named types and structs make them natural:

GO internal/order/order.go

```go
// Money is an amount in cents. It is a value object: it has no
// identity, and two Money values with the same cents are equal.
type Money int64

func (m Money) String() string {
    return fmt.Sprintf("$%d.%02d", m/100, m%100)
}

// NewItem is the only way to build a valid Item.
func NewItem(productID string, qty int, unitPrice Money) (Item, error) {
    if productID == "" {
        return Item{}, ErrMissingProduct
    }
    if qty < 1 {
        return Item{}, ErrInvalidQuantity
    }
    return Item{ProductID: productID, Quantity: qty, UnitPrice: unitPrice}, nil
}
```

`Money` is a distinct type, so a quantity can't sneak in where an amount belongs. Value objects are passed by value (copied), which gives you the immutability of a Java `record`.

Domain errors

Instead of an exception hierarchy (`OrderNotFoundException extends RuntimeException`), Go uses **sentinel errors**: package-level error values that callers compare with `errors.Is`.

GO internal/order/order.go

```go
// Domain errors. Callers check them with errors.Is.
var (
    ErrNotFound         = errors.New("order not found")
    ErrMissingCustomer  = errors.New("customer id is required")
    ErrNoItems          = errors.New("order needs at least one item")
    ErrAlreadyShipped   = errors.New("order has already shipped")
    ErrAlreadyCancelled = errors.New("order is already cancelled")
)
```

Errors travel up the layers, often wrapped with `%w`, and the HTTP adapter turns them into status codes in one place. It's the Go cousin of `@ControllerAdvice`:

GO internal/httpapi/handler.go

```go
status := http.StatusInternalServerError
switch {
case errors.Is(err, order.ErrNotFound):
    status = http.StatusNotFound
case errors.Is(err, order.ErrAlreadyShipped),
    errors.Is(err, order.ErrAlreadyCancelled),
    errors.Is(err, inventory.ErrInsufficientStock):
    status = http.StatusConflict
}
```

Chapter 7 grows this into RFC 9457 problem responses. And because the rules live in plain structs, testing them needs no Spring context, no database and no mocks:

TERMINAL

```
go test ./internal/order -run TestCancelRules -v
```

OUTPUT

```
=== RUN   TestCancelRules
--- PASS: TestCancelRules (0.00s)
    --- PASS: TestCancelRules/pending_can_cancel (0.00s)
    --- PASS: TestCancelRules/confirmed_can_cancel (0.00s)
    --- PASS: TestCancelRules/shipped_cannot_cancel (0.00s)
    --- PASS: TestCancelRules/cancel_twice (0.00s)
PASS
ok      example.com/goguide/ch06/internal/order 1.737s
```

Dependency injection

The handler needs the service, the service needs a repository, and so on. Spring connects them with a container that scans and reflects at startup. Go teams usually connect them with plain function calls. Here are the three options worth knowing.

Manual wiring: `main.go` as the composition root

The **composition root** is the single place in a program that creates the concrete objects and plugs them together. In Go, that's `main`, or a function it calls.

> 🧩 **Think of it like a theatre's stage manager**
>
> The actors (your packages) only know their own lines. Before the curtain goes up, the stage manager places every prop and tells each actor who they're talking to. Nobody on stage needs to know where the props came from.

Here's the example's composition root, building the graph from the edges inwards:

GO cmd/orderly/main.go

```go
// build wires the object graph by hand, from the edges inwards.
func build(logger *slog.Logger) *app {
    // 1. Adapters: storage, other modules, messaging.
    orderRepo := memory.NewOrderStore()
    stock := inventory.NewService(inventory.SampleStock()...)
    outbox := events.NewOutbox(logger)

    // 2. Application services: the business use cases.
    orders := order.NewService(orderRepo, stock, outbox, logger)

    // 3. Delivery: HTTP handlers.
    api := httpapi.NewHandler(orders, logger)

    return &app{handler: api.Routes(), outbox: outbox}
}
```

`main` itself stays tiny. It turns operating system signals into a cancelled context and hands everything to `run`, which returns an error instead of exiting deep inside:

GO cmd/orderly/main.go

```go
func main() {
    ctx, stop := signal.NotifyContext(context.Background(),
        os.Interrupt, syscall.SIGTERM)
    defer stop()

    if err := run(ctx, os.Getenv, os.Stdout); err != nil {
        fmt.Fprintln(os.Stderr, "error:", err)
        os.Exit(1)
    }
}
```

Passing `getenv` and the output writer into `run` lets a test start the whole program with fake settings and read its logs. Let's run it and place an order:

TERMINAL

```
ORDERLY_ADDR=127.0.0.1:18081 go run ./cmd/orderly
curl -X POST localhost:18081/api/v1/orders -d '{"customer_id":"c-42",
  "items":[{"product_id":"MUG-BLUE-01","quantity":2,"unit_price_cents":1250}]}'
```

OUTPUT

```
level=INFO msg="outbox relay started"
level=INFO msg="http server started" addr=127.0.0.1:18081
level=INFO msg="order placed" order_id=7d84d51f-... total=$25.00
level=INFO msg="event published" event=OrderPlaced order_id=7d84d51f-...
```

Cancel the same order twice and the aggregate's rule comes all the way out as `409 Conflict` with the body `{"error":"order is already cancelled"}`.

Watching `main.go` grow gracefully

Java developers often worry that manual wiring won't scale. In practice it grows calmly with a few habits:

- **Keep construction in a readable order**: config, infrastructure, repositories, services, handlers, servers.
- **Extract a `build` function** (as above) so tests reuse the real wiring.
- **Group by feature** when it gets long: a `newOrderModule(db, log)` helper keeps `main` scannable.
- **Let the compiler help**: a forgotten dependency fails the build, where Spring throws `NoSuchBeanDefinitionException` at startup.

A real composition root is often 100 to 200 lines. That's fine: it's the most honest wiring diagram you'll ever have, and "Go to definition" works on every line.

> 💡 **If you're unsure, wire by hand**
>
> Manual wiring is the default in the Go community, and it's what the capstone in Chapter 20 uses. Reach for a DI tool only when you feel real pain, not in anticipation of it.

Uber Fx: a container like Spring's

Uber Fx is a dependency-injection framework built on Uber's reflection-based library `dig`. It's the closest thing Go has to the Spring container: you register constructors, Fx works out the order to call them in, and it runs start and stop hooks. Here's the same graph:

GO cmd/orderly-fx/main.go

```go
func main() {
    fx.New(appOptions()).Run() // Run blocks until Ctrl+C or SIGTERM
}

func appOptions() fx.Option {
    return fx.Options(
        fx.Provide(
            func() platform.Config { return platform.LoadConfig(os.Getenv) },
            func(cfg platform.Config) *slog.Logger {
                return platform.NewLogger(os.Stdout, cfg.LogLevel)
            },
            // fx.As binds a concrete type to the interface consumers want.
            fx.Annotate(memory.NewOrderStore, fx.As(new(order.Repository))),
            fx.Annotate(newInventory, fx.As(new(order.StockReserver))),
            fx.Annotate(newOutbox, fx.As(new(order.EventPublisher))),
            fx.Annotate(order.NewService, fx.As(new(httpapi.OrderService))),
            httpapi.NewHandler,
```

```
            newHTTPServer,
        ),
        // Ask for the server so Fx builds it (and everything it needs).
        fx.Invoke(func(*http.Server) {}),
    )
}
```

Lifecycle hooks are where Fx shines. A constructor receives an `fx.Lifecycle` and registers what to do on start and on stop:

GO cmd/orderly-fx/main.go

```
func newHTTPServer(lc fx.Lifecycle, cfg platform.Config,
    h *httpapi.Handler, log *slog.Logger) *http.Server {
    srv := platform.NewHTTPServer(cfg.Addr, h.Routes())
    lc.Append(fx.Hook{
        OnStart: func(context.Context) error {
            ln, err := net.Listen("tcp", srv.Addr)
            if err != nil {
                return err // Fx aborts startup and runs OnStop hooks
            }
            log.Info("http server started", "addr", ln.Addr().String())
            go srv.Serve(ln)
            return nil
        },
        OnStop: func(ctx context.Context) error {
            log.Info("http server stopping")
            return srv.Shutdown(ctx)
        },
    })
    return srv
}
```

Fx runs `OnStart` hooks in the order the constructors ran (dependencies first) and `OnStop` hooks in reverse. With `ORDERLY_DEBUG=true` you can watch it start the outbox relay before the HTTP server, without being told to:

OUTPUT

```
level=DEBUG msg="OnStart hook executing" callee=main.newOutbox.func1() ...
level=INFO msg="outbox relay started"
level=DEBUG msg="OnStart hook executing" callee=main.newHTTPServer.func1() ...
level=INFO msg="http server started" addr=127.0.0.1:18082
level=DEBUG msg=started
```

If you know Spring, this table is your Fx phrasebook:

Spring	Uber Fx
`@Bean`, `@Component`	`fx.Provide(constructor)`
binding to an interface	`fx.Annotate(ctor, fx.As(new(Iface)))`
`ApplicationRunner`	`fx.Invoke(func(...))`
`@PostConstruct`, `SmartLifecycle.start`	`fx.Hook{OnStart: ...}`
`@PreDestroy`, `SmartLifecycle.stop`	`fx.Hook{OnStop: ...}`

Spring	Uber Fx
`@Configuration` class	`fx.Module("orders", ...)`
context startup test	`fx.ValidateApp`, `fxtest.New`

> ⚠️ **Gotcha: Fx errors arrive at runtime**
>
> Like Spring, Fx resolves the graph at startup, so a missing dependency compiles fine and fails on launch. "Go to definition" also stops telling you who provides a type. The example's `TestGraphIsValid` calls `fx.ValidateApp` so CI catches a broken graph early.

Fx (v1.24.0 at the time of writing) is actively maintained and runs most of Uber's Go services. It earns its keep in organisations with many services that share modules such as "logging plus metrics plus tracing".

google/wire: compile-time injection (now archived)

Wire works like Dagger in the Java world: you describe the graph, and a code generator writes plain Go wiring code at build time. You write an **injector** file, guarded by a build tag so only the generator sees it:

GO cmd/orderly-wire/wire.go

```go
//go:build wireinject

package main

func initializeApp(log *slog.Logger) *app {
    panic(wire.Build(
        memory.NewOrderStore,
        wire.Bind(new(order.Repository), new(*memory.OrderStore)),
        newInventory,
        wire.Bind(new(order.StockReserver), new(*inventory.Service)),
        events.NewOutbox,
        wire.Bind(new(order.EventPublisher), new(*events.Outbox)),
        order.NewService,
        wire.Bind(new(httpapi.OrderService), new(*order.Service)),
        httpapi.NewHandler,
        newApp,
    ))
}
```

Running `go run github.com/google/wire/cmd/wire@v0.7.0 ./cmd/orderly-wire` writes `wire_gen.go`, which is exactly the code you would have written by hand:

GO cmd/orderly-wire/wire_gen.go (generated)

```go
func initializeApp(log *slog.Logger) *app {
    orderStore := memory.NewOrderStore()
    service := newInventory()
    outbox := events.NewOutbox(log)
    orderService := order.NewService(orderStore, service, outbox, log)
    handler := httpapi.NewHandler(orderService, log)
    mainApp := newApp(handler, outbox)
    return mainApp
}
```

Wire has no lifecycle support, so `main` still starts and stops things itself.

> 📝 **Maintenance status: archived**
>
> Google archived google/wire on 25 August 2025, and its README says the project is no longer maintained. The last release, v0.7.0, still works: our example builds on Go 1.25 and 1.27. If you inherit Wire, there's no rush to migrate, because the generated code is plain Go you can keep. For new projects, write that same code by hand.

Which one should you use?

Situation	Use	Why
Most services, most teams	manual wiring	readable, compile-checked, no magic
Many services sharing infrastructure modules	Uber Fx	`fx.Module` reuse, lifecycle hooks
Existing codebase already on Wire	keep Wire	works, but archived; don't adopt it new
You miss Spring's container	manual wiring first	the feeling passes within weeks

The application lifecycle

Spring orders startup and shutdown through `SmartLifecycle` phases and `@DependsOn`. In Go you own the order, which turns out to be refreshingly clear: **start things in dependency order, stop them in reverse.**

> 🧩 **Think of it like opening and closing a shop**
>
> In the morning you switch on the lights and the till before you unlock the front door. At closing time you lock the front door first, serve the customers still inside, and only then cash up and switch off the lights.

```
STARTUP (top to bottom)          SHUTDOWN (bottom to top)
1. load config                   6. flush logs and telemetry
2. logger, telemetry             5. close DB pool, clients
3. DB pool, clients              4. stop workers, drain queues
4. background workers            3. finish in-flight requests
5. listen on the port            2. stop accepting connections
6. report "ready"                1. report "not ready"
```

Go's `defer` fits this beautifully. Deferred calls run **last-in, first-out**, like a stack of plates, so what you set up first is torn down last. Here's the start of the lifecycle code in `run`:

GO cmd/orderly/main.go

```go
// Start background workers before the server takes traffic.
workerCtx, stopWorkers := context.WithCancel(context.Background())
var wg sync.WaitGroup
wg.Go(func() { a.outbox.Run(workerCtx) })
defer func() { // deferred, so it runs after the server has stopped
    stopWorkers()
    wg.Wait()
}()
```

`sync.WaitGroup.Go` is new in Go 1.25: it starts a goroutine and tracks it in one call. Next, `run` opens the port with `net.Listen` (so "address already in use" fails fast), starts `srv.Serve` in a goroutine, and waits for either a server failure or a shutdown signal:

GO cmd/orderly/main.go

```go
select {
case err := <-serveErr: // the server died on its own
    return err
case <-ctx.Done(): // Ctrl+C or SIGTERM
}

logger.Info("shutting down")
shutdownCtx, cancel := context.WithTimeout(context.Background(),
    cfg.ShutdownTimeout)
defer cancel()
if err := srv.Shutdown(shutdownCtx); err != nil {
    return fmt.Errorf("http shutdown: %w", err)
}
logger.Info("http server stopped")
return nil
```

`srv.Shutdown` stops accepting connections and waits for in-flight requests, up to the timeout. Only after it returns does the deferred call stop the outbox relay, which drains any queued events. The logs from a test run show the order:

OUTPUT

```
level=INFO msg="outbox relay started"
level=INFO msg="http server started" addr=127.0.0.1:18083
level=INFO msg="shutting down"
level=INFO msg="http server stopped"
level=INFO msg="outbox relay stopped"
```

Chapter 14 builds on this with readiness probes, Kubernetes grace periods and draining strategies.

Modular monolith or microservices?

Package by feature has a lovely side effect: each feature is already a candidate service. That makes the **modular monolith** (one deployable with strong internal boundaries) the sensible starting point for most new systems.

	Modular monolith	Microservices
Deploy	one binary	many binaries
Calls between features	function calls	network calls (REST, gRPC, events)
Transactions	one database transaction	sagas, outbox (Chapter 11)
Team fit	one to a few teams	many independent teams
Cost of a wrong boundary	move a package	migrate data and APIs

To keep a monolith modular:

- Features talk through small interfaces or service methods, never through each other's database tables.
- Shared code goes in a small, boring `platform` package, not a growing `common` one.
- Nested `internal/` folders add stricter fences: `internal/order/internal/pricing` can only be imported by code under `internal/order`.

When a feature truly needs its own scaling, give it its own `cmd/` and swap the in-process implementation for a gRPC client (Chapter 8). The rest of the code doesn't notice.

Multi-module repositories and `go.work`

In Maven, multi-module builds are normal. In Go, **one module per repository** is the norm, even for a large service. A **module** is a versioned unit with its own `go.mod`; you need several only when parts are released independently, such as a client library other teams import.

To change several modules together locally, a **workspace** file tells the `go` command to use your local copies. It's much like a Gradle composite build with `includeBuild`:

TERMINAL

```
go work init ./orders ./inventory
```

GO.MOD go.work

```
go 1.25.0

use (
    ./inventory
    ./orders
)
```

> ⚠️ **Gotcha: go.work and CI**
>
> The Go modules reference generally advises against committing `go.work`, because it can make CI test a different combination of module versions than your users will get. Keep it for local development unless every module in the repository is always developed and released together.

Anti-patterns to avoid

Every Java developer brings a few habits that fight Go's grain. Here are the ones that hurt most, and what to do instead.

A giant `models` package, or `utils`

A `models` package holding every struct becomes the centre of a dependency web, and it slowly fills with logic that belongs elsewhere. Names like `utils`, `common` and `helpers` describe nothing, so everything ends up in them.

Instead: keep types with the feature that owns them (`order.Order`), keep JSON structs in the adapter that needs them, and name helper packages after what they provide (`money`, `retry`).

Circular dependencies

Go refuses to compile an import cycle:

OUTPUT

```
package example.com/shop/x
    imports example.com/shop/y from x.go
    imports example.com/shop/x from y.go: import cycle not allowed
```

This feels harsh at first, and every Java developer hits it. Think of it as a free design review from the compiler.

Instead: let the consumer define an interface (as `order.StockReserver` does), move the shared type to the package that owns the concept, or let `main` connect the two.

An interface for everything

GO Avoid

```go
type OrderService interface { Place(...); Get(...); Cancel(...) }
type orderServiceImpl struct{ ... }
func NewOrderService(...) OrderService { return &orderServiceImpl{...} }
```

An interface defined next to its only implementation adds indirection, hides the real type from "Go to definition" and grows big because it must serve every caller.

Instead: return the concrete `*order.Service`, and let each consumer declare the small interface it needs.

A service-locator singleton

GO Avoid

```go
var Registry = map[string]any{} // filled in init(), read everywhere

func (h *Handler) get(w http.ResponseWriter, r *http.Request) {
    svc := Registry["orders"].(*order.Service) // hidden dependency
}
```

Global registries and package variables like `var DB *sql.DB` recreate `ApplicationContext.getBean()`. They hide dependencies, make tests interfere with each other and turn startup order into a mystery.

Instead: pass dependencies through constructors and build everything in `main`. If a function needs something, it belongs in its parameter list or its struct.

Recap

- Use `cmd/<binary>/` for entry points and `internal/` for everything else; skip `pkg/` unless you publish a library.
- Organise packages by feature (`order`, `inventory`), name them after what they provide, and avoid stutter (`order.ErrNotFound`).
- Handler, service and repository are still the right layers; they're plain structs with constructors, not annotated beans.
- Define small interfaces where they're consumed, and return concrete structs. Fakes then take a few lines.
- Hexagonal architecture in Go is ports (interfaces in the core) plus adapters (structs elsewhere), plugged together in `main`.
- Put rules in aggregates (`Order.Cancel()`), use value objects (`Money`, `Item`), and map sentinel domain errors to status codes in one place.
- Wire by hand in the composition root; consider Uber Fx for large fleets; don't adopt the archived google/wire for new work.
- Start in dependency order and stop in reverse; `defer` and `srv.Shutdown` make this natural.

Try it yourself

🏋 **Exercise 1: Add a stock-level endpoint**

Add `GET /api/v1/inventory/{productId}` to the example. Define a small `StockLevels` interface in `httpapi` with one method, and pass `*inventory.Service` to the handler in `build`. *Hint:* `inventory.Service` already has a `Level` method, so the compiler will tell you whether your interface matches.

> **🏋️ Exercise 2: A rule for shipping**
>
> Add a `Ship` use case to `order.Service` and an endpoint for it. Then add a rule to the aggregate: an order can only ship if its total is above zero. Write a table-driven test like `TestCancelRules`. *Hint:* the rule belongs in `Order.Ship()`, not in the service or the handler.

> **🏋️ Exercise 3: A product feature**
>
> Create `internal/product` with a `Product` type, a `Catalog` service and an in-memory store. Make `order.Service` look up unit prices through a `PriceLookup` port instead of trusting the request. *Hint:* define `PriceLookup` in package `order`; only `main.go` should import both `order` and `product`.

> **🏋️ Exercise 4 (stretch): Fx modules**
>
> In `cmd/orderly-fx`, group the order-related providers into `fx.Module("orders", ...)` and the platform providers into another module. Keep `TestGraphIsValid` passing. *Hint:* read the "Modules" page of the Fx documentation linked below.

Further reading

- Organizing a Go module: the Go team's own guidance on layouts, from one file to many binaries
- Package names: the Go blog post behind "short, lowercase, no stutter"
- Go Code Review Comments: includes the advice to define interfaces where they're used
- Uber Fx documentation: getting started, lifecycle hooks and modules
- google/wire on GitHub: the archived repository, useful if you inherit a Wire codebase
- Tutorial: Getting started with multi-module workspaces: `go.work` step by step
- Anemic Domain Model: Martin Fowler on why behaviour belongs with data

CHAPTER 7

Building REST APIs

> *Go's standard library already contains a production-grade HTTP server, so building a REST API is mostly about learning a few small, sharp patterns.*

🎯 **What you'll learn**

- Serve JSON endpoints with `net/http` and Go 1.22's method-aware `ServeMux`
- Decode and validate requests safely, and return friendly RFC 9457 problem responses
- Write middleware for logging, request IDs, panic recovery, timeouts and CORS, and order it correctly
- Build the same API with chi, and compare Gin, Echo and Fiber
- Configure `http.Server` and `http.Client` timeouts so production traffic can't hurt you
- Generate a typed server from an OpenAPI spec with oapi-codegen

The big picture

In Spring, a REST endpoint involves a stack of cooperating parts: embedded Tomcat, the `DispatcherServlet`, `@RestController`, Jackson, Bean Validation, `@ControllerAdvice` and servlet filters. You annotate a method, and the framework does the rest.

Go ships most of that stack in the standard library, as plain types you call yourself. There's no dispatcher to configure and no annotation scanning. The whole ecosystem, from routers to frameworks to middleware, agrees on one tiny interface: `http.Handler`.

🧩 **Think of it like a standard power socket**

`http.Handler` is the socket shape every Go web library agrees on. A router is a power strip, a middleware is a surge protector, your endpoint is the lamp. Because they all share one plug shape, you can mix pieces from different makers freely.

Concept	Java / Spring	Go
Server	embedded Tomcat	`http.Server` (standard library)
Endpoint	`@RestController` method	`http.Handler` / `http.HandlerFunc`
Routing	`@GetMapping("/orders/{id}")`	`mux.Handle("GET /orders/{id}", h)`
Path variable	`@PathVariable`	`r.PathValue("id")`
JSON	Jackson annotations	`encoding/json` struct tags

Concept	Java / Spring	Go
Validation	`@Valid`, `@NotNull`	go-playground/validator tags
Error mapping	`@ControllerAdvice`, `ProblemDetail`	one `problemFor(err)` function
Filters	`Filter`, `HandlerInterceptor`	`func(http.Handler) http.Handler`
HTTP client	`RestClient`, `WebClient`	`http.Client`
API docs	springdoc-openapi	oapi-codegen (spec-first)

Full code: `examples/ch07-rest/`. The orders API from the shared Orderly domain is built several ways there, all sharing one in-memory service.

net/http fundamentals

Before reaching for any library, it pays to understand the four types everything else is built on. Once these click, every Go web framework becomes easy to read.

Handler and HandlerFunc

Here is the complete definition of a handler in the standard library:

GO

```go
type Handler interface {
    ServeHTTP(ResponseWriter, *Request)
}

type HandlerFunc func(ResponseWriter, *Request)
```

`Handler` is an interface with one method. `HandlerFunc` is a function type that has a `ServeHTTP` method of its own, so any ordinary function with the right signature can be used as a `Handler`. It's a classic Go trick: an adapter that turns a function into an interface value.

Most of the time you'll write functions and register them like this:

GO stdlib/main.go

```go
mux.HandleFunc("GET /healthz", func(w http.ResponseWriter, _ *http.Request) {
    w.WriteHeader(http.StatusNoContent)
})
```

ResponseWriter and Request

`*http.Request` is everything the client sent. `http.ResponseWriter` is how you answer. These are the members you'll use daily:

You want	Use
method, path, query	`r.Method`, `r.URL.Path`, `r.URL.Query().Get("limit")`
a path variable	`r.PathValue("id")`

You want	Use
a header	`r.Header.Get("X-Request-ID")`
the body	`r.Body` (an `io.ReadCloser`; the server closes it)
cancellation and deadlines	`r.Context()`
set a response header	`w.Header().Set("Location", "/api/v1/orders/42")`
set the status	`w.WriteHeader(http.StatusCreated)`
write the body	`w.Write(b)` or `json.NewEncoder(w)`

> ⚠️ **Gotcha: headers, then status, then body**
>
> The order matters. Headers set after `WriteHeader` are silently ignored, and the first `Write` sends a `200 OK` if you haven't set a status. Calling `WriteHeader` twice logs "superfluous response.WriteHeader call". Every Go developer hits this once.

Routing with ServeMux (Go 1.22+)

Here's a familiar Spring controller:

JAVA

```java
@RestController
@RequestMapping("/api/v1/orders")
public class OrderController {
    private final OrderService orders;

    public OrderController(OrderService orders) { this.orders = orders; }

    @GetMapping("/{id}")
    public OrderResponse get(@PathVariable UUID id) {
        return OrderResponse.from(orders.get(id));
    }

    @PostMapping
    public ResponseEntity<OrderResponse> place(
            @Valid @RequestBody PlaceOrderRequest req) { ... }
}
```

Since Go 1.22, the standard library's `ServeMux` understands HTTP methods and path wildcards, which covers most REST routing:

GO stdlib/main.go

```go
mux := http.NewServeMux()
mux.Handle("POST /api/v1/orders", h.Handle(h.PlaceOrder))
mux.Handle("GET /api/v1/orders", h.Handle(h.ListOrders))
mux.Handle("GET /api/v1/orders/{id}", h.Handle(h.GetOrder))
mux.Handle("POST /api/v1/orders/{id}/cancel", h.Handle(h.CancelOrder))
```

Inside a handler, `r.PathValue("id")` returns the matched segment. A few pattern rules worth knowing:

- `{id}` matches one path segment; `{path...}` matches the rest of the path.

- A trailing slash pattern like `/files/` matches everything below it; `/files/{$}` matches only `/files/` itself.
- When two patterns overlap, the more specific one wins, and conflicting patterns panic at registration, so you find out at startup.
- `GET` patterns also match `HEAD` requests.

If the path matches but the method doesn't, `ServeMux` answers `405` and lists the allowed methods for you:

OUTPUT

```
HTTP/1.1 405 Method Not Allowed
Allow: GET, HEAD
Content-Type: text/plain; charset=utf-8

Method Not Allowed
```

The 404 and 405 bodies are plain text, though. If you want them as JSON too, that's one of the reasons to pick chi, which we'll meet shortly.

> **Newer Go**
>
> Go 1.26 changed `ServeMux`'s automatic trailing-slash redirects from `301 Moved Permanently` to `307 Temporary Redirect`. Our examples don't rely on either.

JSON with encoding/json

Jackson turns objects into JSON through annotations and an `ObjectMapper`. Go's `encoding/json` does the same job using **struct tags**: short strings attached to struct fields that libraries read at runtime.

Struct tags vs Jackson annotations

In Spring, a response type might look like this:

JAVA

```java
@JsonInclude(JsonInclude.Include.NON_NULL)
public record OrderResponse(
        String id,
        @JsonProperty("customer_id") String customerId,
        OrderStatus status,
        @JsonProperty("total_cents") long totalCents,
        @JsonProperty("created_at") Instant createdAt,
        @JsonProperty("cancelled_at") Instant cancelledAt) {}
```

In Go, the tag sits after the field's type, in backquotes:

GO internal/httpapi/dto.go

```go
// OrderResponse is the JSON shape of an order. It is separate from
// order.Order so the API can stay stable while the domain evolves.
type OrderResponse struct {
    ID          string          `json:"id"`
    CustomerID  string          `json:"customer_id"`
    Status      order.Status    `json:"status"`
    Items       []ItemResponse  `json:"items"`
    TotalCents  int64           `json:"total_cents"`
    CreatedAt   time.Time       `json:"created_at"`
    CancelledAt time.Time       `json:"cancelled_at,omitzero"` // Go 1.24+
}
```

Jackson	`encoding/json`
`@JsonProperty("customer_id")`	`` `json:"customer_id"` ``
`@JsonIgnore`	`` `json:"-"` ``
`@JsonInclude(NON_EMPTY)`	`` `json:",omitempty"` ``
`@JsonInclude(NON_DEFAULT)`	`` `json:",omitzero"` `` (Go 1.24+)
`FAIL_ON_UNKNOWN_PROPERTIES`	`decoder.DisallowUnknownFields()`

Two surprises catch every Java developer. First, **only exported (capitalised) fields** are serialised; a lowercase field is invisible to `encoding/json`. Second, a **nil slice becomes `null`**, while an empty one (`make([]T, 0)`) becomes `[]`. That's why `NewOrderResponse` always allocates the items slice.

omitempty vs omitzero

`omitempty` leaves out `false`, `0`, `""`, nil pointers and empty slices or maps. It never leaves out a struct, so a zero `time.Time` is sent as `"0001-01-01T00:00:00Z"`, which confuses every client.

`omitzero`, added in Go 1.24, leaves out any zero value, including structs, and respects a type's `IsZero()` method. With it, `cancelled_at` appears only once an order is cancelled:

OUTPUT

```
  "status": "CANCELLED",
  "total_cents": 2500,
  "created_at": "2026-11-02T18:05:24.2361884Z",
  "cancelled_at": "2026-11-02T18:05:26.0725627Z"
```

Time formats

`time.Time` marshals to RFC 3339 with up to nanosecond precision, such as `2026-11-02T18:05:24.2361884Z`. That matches what Spring Boot's auto-configured Jackson produces for an `Instant`, so Java and Go services interoperate out of the box. Store and send times in UTC (`time.Now().UTC()`); the example service does.

> **Newer Go: encoding/json/v2**
>
> Go 1.25 has an experimental `encoding/json/v2` behind `GOEXPERIMENT=jsonv2`. In Go 1.27 it became a regular package, and the classic `encoding/json` is now backed by the new implementation. Behaviour is preserved, but error message text may differ, so never match on JSON error strings. Our helpers check error types instead.

A decodeJSON and writeJSON helper pair

`json.NewDecoder(r.Body).Decode(&req)` on its own is too trusting for production. It accepts unknown fields, reads bodies of any size, and ignores trailing data after the first object. Spring handles these through configuration; in Go, you write one small helper and reuse it everywhere:

GO internal/httpapi/json.go

```go
const maxBodyBytes = 1 << 20 // 1 MiB is plenty for an order

func decodeJSON(w http.ResponseWriter, r *http.Request, dst any) error {
    if ct := r.Header.Get("Content-Type"); ct != "" {
        mt, _, _ := mime.ParseMediaType(ct)
        if mt != "application/json" {
            return newError(http.StatusUnsupportedMediaType,
                "Unsupported media type",
                "Content-Type must be application/json")
        }
    }
    r.Body = http.MaxBytesReader(w, r.Body, maxBodyBytes)
    dec := json.NewDecoder(r.Body)
    dec.DisallowUnknownFields() // like Jackson's FAIL_ON_UNKNOWN_PROPERTIES

    if err := dec.Decode(dst); err != nil {
        return decodeError(err)
    }
    if err := dec.Decode(&struct{}{}); !errors.Is(err, io.EOF) {
        return badRequest("body must contain a single JSON object")
    }
    return nil
}
```

`http.MaxBytesReader` wraps the body so that reading more than the limit fails with an `*http.MaxBytesError`. The second `Decode` call checks that nothing follows the first object. `decodeError` turns each failure into a friendly 4xx error by checking its type:

GO internal/httpapi/json.go

```
    switch {
    case errors.As(err, &tooBig):
        return newError(http.StatusRequestEntityTooLarge, "Body too large",
            fmt.Sprintf("body must not exceed %d bytes", tooBig.Limit))
    case errors.As(err, &syntaxErr):
        return badRequest(fmt.Sprintf("malformed JSON at byte %d",
            syntaxErr.Offset))
    case errors.As(err, &typeErr):
        return badRequest(fmt.Sprintf("field %q has the wrong type",
            typeErr.Field))
    case errors.Is(err, io.EOF):
        return badRequest("body must not be empty")
    default: // for example an unknown field
        return badRequest(err.Error())
    }
```

The writing side encodes **before** writing anything, so an encoding failure can still become a clean 500 instead of half a response:

GO internal/httpapi/json.go

```
func writeJSON(w http.ResponseWriter, status int, v any) error {
    body, err := json.Marshal(v)
    if err != nil {
        return fmt.Errorf("encode response: %w", err)
    }
    w.Header().Set("Content-Type", "application/json")
    w.WriteHeader(status)
    _, _ = w.Write(append(body, '\n')) // a failed write means the client left
    return nil
}
```

Send a typo in a field name and the client learns about it straight away:

OUTPUT

```
{"type":"about:blank","title":"Bad request","status":400,
 "detail":"json: unknown field \"itemz\"","instance":"/api/v1/orders",...}
```

Validation

Decoding checks the *shape* of JSON. Validation checks the *values*: is the customer ID a UUID, is the quantity between 1 and 100? In Spring, Bean Validation annotations and `@Valid` do this:

JAVA

```java
public record PlaceOrderRequest(
        @NotNull UUID customerId,
        @NotEmpty @Size(max = 50) List<@Valid Line> items) {}

public record Line(
        @NotBlank String productId,
        @Min(1) @Max(100) int quantity) {}
```

Go has no built-in equivalent, and the de facto standard is **go-playground/validator** (v10.30.4 in our examples), which reads `validate` struct tags:

GO internal/httpapi/dto.go

```go
type PlaceOrderRequest struct {
    CustomerID string     `json:"customer_id" validate:"required,uuid"`
    Items      []LineJSON `json:"items" validate:"required,min=1,max=50,dive"`
}

type LineJSON struct {
    ProductID string `json:"product_id" validate:"required"`
    Quantity  int    `json:"quantity" validate:"gte=1,lte=100"`
}
```

`dive` means "validate each element of this slice", like `List<@Valid Line>`. Here's how the common annotations translate:

Bean Validation	validator tag
`@NotNull`, `@NotBlank`	`required`
`@Size(min=1, max=50)` on a list	`min=1,max=50`
`@Min(1) @Max(100)`	`gte=1,lte=100`
`@Valid` on nested items	`dive`
custom `ConstraintValidator`	`RegisterValidation("name", fn)`

Create the validator once, because it caches struct metadata and is safe for concurrent use. Two setup steps make its errors useful to API clients: report JSON field names instead of Go names, and register any custom rules:

GO internal/httpapi/validate.go

```go
var validate = newValidator()

func newValidator() *validator.Validate {
    v := validator.New(validator.WithRequiredStructEnabled())
    // Report JSON names ("customer_id"), not Go names ("CustomerID").
    v.RegisterTagNameFunc(func(f reflect.StructField) string {
        name, _, _ := strings.Cut(f.Tag.Get("json"), ",")
        if name == "-" {
            return ""
        }
        return name
    })
    // A custom rule, like a custom ConstraintValidator in Java.
    _ = v.RegisterValidation("orderstatus", validOrderStatus)
    return v
}
```

Friendly field errors

Out of the box, validator's error text reads like `Key: 'PlaceOrderRequest.Items[0].Quantity' Error:Field validation for 'Quantity' failed on the 'gte' tag`. That's fine for logs, not for your API's users. So we convert each failure into a field path and a human sentence:

GO internal/httpapi/validate.go

```go
func validateStruct(v any) error {
    err := validate.Struct(v)
    var ves validator.ValidationErrors
    if !errors.As(err, &ves) {
        return err // nil, or a programming error such as a bad tag
    }
    fields := make([]FieldError, 0, len(ves))
    for _, fe := range ves {
        fields = append(fields, FieldError{
            Field:   fieldPath(fe), // "items[0].quantity"
            Message: message(fe),   // "must be at least 1"
        })
    }
    e := newError(http.StatusUnprocessableEntity, "Validation failed",
        "one or more fields are invalid")
    e.Problem.Errors = fields
    return e
}
```

`message` is a `switch` on `fe.Tag()` that returns phrases like "is required" and "must be a UUID". Here's what a client sees for a request with three mistakes:

OUTPUT

```
{
    "type": "about:blank",
    "title": "Validation failed",
    "status": 422,
    "detail": "one or more fields are invalid",
    "instance": "/api/v1/orders",
    "request_id": "4d47ea4f-8043-46aa-8369-87457392b801",
    "errors": [
        {"field": "customer_id", "message": "must be a UUID"},
        {"field": "items[0].product_id", "message": "is required"},
        {"field": "items[0].quantity", "message": "must be at least 1"}
    ]
}
```

We use `422 Unprocessable Content` for well-formed JSON with invalid values, and `400 Bad Request` for JSON we can't parse. Spring returns `400` for both by default; either convention is fine as long as you're consistent.

Error responses with problem+json

Every API needs one consistent error format. **RFC 9457** ("Problem Details for HTTP APIs") defines it: a JSON object with `type`, `title`, `status`, `detail` and `instance`, sent as `application/problem+json`. Spring 6 supports it natively with `ProblemDetail`:

JAVA

```
@RestControllerAdvice
public class ApiErrors {
    @ExceptionHandler(OrderNotFoundException.class)
    ProblemDetail notFound(OrderNotFoundException e) {
        ProblemDetail pd = ProblemDetail.forStatusAndDetail(
                HttpStatus.NOT_FOUND, e.getMessage());
        pd.setTitle("Order not found");
        return pd;
    }
}
```

Go has no exceptions to intercept, so we make handlers **return** errors and handle them in one place. First, the problem type:

GO internal/httpapi/problem.go

```go
// Problem is an RFC 9457 "problem details" body.
type Problem struct {
    Type      string       `json:"type"`
    Title     string       `json:"title"`
    Status    int          `json:"status"`
    Detail    string       `json:"detail,omitempty"`
    Instance  string       `json:"instance,omitempty"`
    RequestID string       `json:"request_id,omitempty"` // extension
    Errors    []FieldError `json:"errors,omitempty"`     // extension
}
```

Next, the central mapping from domain errors to problems. This one function is your `@ControllerAdvice`:

GO internal/httpapi/problem.go

```go
func problemFor(err error) Problem {
    var apiErr *Error
    switch {
    case errors.As(err, &apiErr): // decoding and validation errors
        return apiErr.Problem
    case errors.Is(err, order.ErrNotFound):
        return problem(http.StatusNotFound, "order-not-found",
            "Order not found", err)
    case errors.Is(err, order.ErrInsufficientStock):
        return problem(http.StatusConflict, "insufficient-stock",
            "Not enough stock", err)
    case errors.Is(err, context.DeadlineExceeded):
        return Problem{Type: "about:blank", Title: "Request timed out",
            Status: http.StatusServiceUnavailable}
    default: // never leak internal details to clients
        return Problem{Type: "about:blank", Title: "Internal Server Error",
            Status: http.StatusInternalServerError}
    }
}
```

The full version has a few more cases. Finally, a tiny adapter lets handlers return an error, and routes every error through the mapping:

GO internal/httpapi/handlers.go

```go
// APIFunc is a handler that returns an error instead of writing it.
type APIFunc func(w http.ResponseWriter, r *http.Request) error

// Handle adapts an APIFunc to http.Handler. Every error goes through
// WriteError, so error handling lives in one place.
func (h *Handler) Handle(fn APIFunc) http.HandlerFunc {
    return func(w http.ResponseWriter, r *http.Request) {
        if err := fn(w, r); err != nil {
            h.WriteError(w, r, err)
        }
    }
}
```

`WriteError` calls `problemFor`, logs anything in the 5xx range with the request ID, and writes the problem. Now a handler reads top to bottom with no error-formatting noise:

GO internal/httpapi/handlers.go

```go
func (h *Handler) PlaceOrder(w http.ResponseWriter, r *http.Request) error {
    var req PlaceOrderRequest
    if err := decodeJSON(w, r, &req); err != nil {
        return err
    }
    if err := validateStruct(req); err != nil {
        return err
    }
    o, err := h.orders.Place(r.Context(), req.CustomerID, req.lines())
    if err != nil {
        return err
    }
    w.Header().Set("Location", "/api/v1/orders/"+o.ID)
    return writeJSON(w, http.StatusCreated, NewOrderResponse(o))
}
```

Ask for more T-shirts than we have, and the domain error arrives as a tidy problem:

OUTPUT

```
{
    "type": "https://orderly.example/problems/insufficient-stock",
    "title": "Not enough stock",
    "status": 409,
    "detail": "insufficient stock for TEE-GOPHER-L",
    "instance": "/api/v1/orders",
    "request_id": "679edfb3-03d4-4b4b-b9b9-0f7dd9ef6034"
}
```

> 🐭 **The Go way**
>
> Errors are values that flow back up the call stack. Handlers return them, a single function decides what each one means over HTTP, and nothing is thrown. You can read the whole error policy of your API on one screen.

Middleware

Some behaviour applies to every request: logging, request IDs, panic recovery, timeouts, CORS. Spring splits this between servlet `Filter`s and `HandlerInterceptor`s. Go has one concept: **middleware**, a function that takes a handler and returns a new handler wrapping it.

GO

```go
type Middleware func(http.Handler) http.Handler
```

> 🧩 **Think of it like layers of an onion**
>
> Each middleware is a layer around your handler. A request travels inwards through every layer to reach the handler; the response travels back outwards through the same layers in reverse.

Request IDs

Here's a Spring filter that adds a request ID:

JAVA

```java
@Component
public class RequestIdFilter extends OncePerRequestFilter {
    @Override
    protected void doFilterInternal(HttpServletRequest req,
            HttpServletResponse res, FilterChain chain)
            throws ServletException, IOException {
        String id = Optional.ofNullable(req.getHeader("X-Request-ID"))
                .orElse(UUID.randomUUID().toString());
        res.setHeader("X-Request-ID", id);
        chain.doFilter(req, res);
    }
}
```

And the Go middleware. Instead of a thread-local or MDC, the ID travels in the request's `context`:

GO internal/httpapi/middleware.go

```go
type ctxKey struct{} // unexported type: no other package can collide

func RequestID(next http.Handler) http.Handler {
    return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
        id := r.Header.Get("X-Request-ID")
        if id == "" || len(id) > 64 {
            id = uuid.NewString()
        }
        w.Header().Set("X-Request-ID", id)
        ctx := context.WithValue(r.Context(), ctxKey{}, id)
        next.ServeHTTP(w, r.WithContext(ctx)) // call the next layer
    })
}
```

Logging, recovery, timeouts and CORS

A logging middleware needs the status code, but `ResponseWriter` doesn't expose it. The fix is to wrap the writer:

GO internal/httpapi/middleware.go

```go
type statusRecorder struct {
    http.ResponseWriter
    status int
}

func (s *statusRecorder) WriteHeader(code int) {
    s.status = code
    s.ResponseWriter.WriteHeader(code)
}

// Unwrap lets http.ResponseController reach the real writer (for
// Flush, deadlines and so on).
func (s *statusRecorder) Unwrap() http.ResponseWriter {
    return s.ResponseWriter
}
```

Embedding `http.ResponseWriter` means the recorder gets every method for free and overrides only `WriteHeader`. The `Unwrap` method matters for streaming in Chapter 8. Each request then produces one structured log line:

OUTPUT

```
level=INFO msg=request method=POST path=/api/v1/orders status=201
  duration=761µs request_id=1fd2fba1-0f7a-4c28-b862-b1662fb96164
```

The other middleware in the example follow the same shape:

- **`Recover`** uses `defer` and `recover()` to turn a panic into a logged 500 problem. Without it, `net/http` still catches the panic so the server survives, but the client sees a dropped connection and the stack trace goes to standard error instead of your logs.
- **`Timeout`** gives every request a deadline with `context.WithTimeout`. Handlers pass `r.Context()` down, so database calls give up when time runs out. The standard library's `http.TimeoutHandler` is an alternative that also cuts off the response.
- **`CORS`** answers browser preflight `OPTIONS` requests for allowed origins. For credentials, wildcards and other options, use `github.com/rs/cors` or `github.com/go-chi/cors` rather than growing your own.

Ordering middleware

Order matters here, as it does for filters in Spring. A small `Chain` helper applies middleware so that the first one listed is the outermost:

GO stdlib/main.go

```go
// The first middleware listed is the outermost layer.
return httpapi.Chain(mux,
    httpapi.RequestID,
    httpapi.Logger(log),
    httpapi.Recover(log),
    httpapi.CORS("https://app.orderly.example"),
    httpapi.Timeout(5*time.Second),
)
```

```
request → RequestID → Logger → Recover → CORS → Timeout → mux → handler
                                                              |
response ← RequestID ← Logger ← Recover ← CORS ← Timeout ← mux ←┘

RequestID first    so every later layer (and every log line) has the ID
Logger outside     so it records the 500 that Recover writes after a panic
Recover outside    so a panic anywhere further in is caught
CORS before mux    so preflight OPTIONS requests never hit a 405
Timeout innermost  so the deadline covers the real work only
```

A test in the example proves the onion shape:

OUTPUT

```
order = A in, B in, handler, B out, A out
```

chi: the router we recommend

The standard `ServeMux` covers a lot, but bigger APIs want route groups, middleware on some routes only, and JSON 404s. **chi** (v5.5.0 at the time of writing) adds exactly that while staying 100% `net/http`: its handlers are `http.Handler`s and its middleware has the standard `func(http.Handler) http.Handler` shape. chi has no external dependencies.

Here's the same API on chi. Notice that the handlers and middleware are the **very same** code as the stdlib version:

GO chi/main.go

```go
r := chi.NewRouter()
r.Use(middleware.Heartbeat("/healthz")) // answers before logging
r.Use(httpapi.RequestID)
r.Use(httpapi.Logger(log))
r.Use(httpapi.Recover(log))
r.Use(httpapi.CORS("https://app.orderly.example"))

r.NotFound(httpapi.NotFound) // problem+json for unknown routes
r.MethodNotAllowed(httpapi.MethodNotAllowed)
```

```
    r.Route("/api/v1", func(r chi.Router) {
        r.Use(httpapi.Timeout(5 * time.Second)) // only for API routes
        r.Route("/orders", func(r chi.Router) {
            r.Get("/", h.Handle(h.ListOrders))
            r.Post("/", h.Handle(h.PlaceOrder))
            r.Route("/{id}", func(r chi.Router) {
                r.Get("/", h.Handle(h.GetOrder))
                r.Post("/cancel", h.Handle(h.CancelOrder))
            })
        })
    })
```

`r.Route` creates a sub-router, like a class-level `@RequestMapping`. `r.Use` inside it applies middleware to that group only, and `r.With(...)` applies middleware to a single route. URL parameters are read with `chi.URLParam(r, "id")`; recent chi versions also fill in `r.PathValue`, which is why our handlers work unchanged on both routers.

chi also ships a catalogue of ready-made middleware in `github.com/go-chi/chi/v5/middleware`:

Middleware	What it does
`RequestID`, `Logger`, `Recoverer`	chi's own versions of the basics
`Timeout`, `Throttle`	request deadlines, concurrency limits
`Compress`, `CleanPath`, `StripSlashes`	gzip, path tidying
`ClientIPFromXFF` and friends	trusted client IPs behind proxies
`Heartbeat`	a cheap liveness endpoint
`Deprecation`, `Sunset`	retirement headers (RFC 9745, RFC 8594)

Because the example's contract tests run against both versions, you can be sure they behave the same. The one visible difference: chi's 404 and 405 answers are problem+json too.

> 💡 **If you're unsure, use chi**
>
> Start with the standard library for small services. Move to chi when you want route groups, per-route middleware or custom 404 and 405 responses. Your handlers don't change, so it's a cheap decision to revisit.

Gin, Echo and Fiber

You'll meet three other popular frameworks in job ads and existing codebases. They're more "batteries included" and each has its own context type instead of plain `http.ResponseWriter` and `*http.Request`.

Framework	Built on	Handler signature	Good to know
Gin v1.12	`net/http`	`func(*gin.Context)`	most popular; big dependency tree
Echo v5	`net/http`	`func(*echo.Context) error`	v5 released January 2026; v4 in maintenance
Fiber v3	fasthttp	`func(fiber.Ctx) error`	Express-like; not `net/http` compatible

Here's `GET /api/v1/orders/{id}` in Gin, from the example's `gin/` folder. Gin validates with the same validator library, but reads a `binding` tag:

GO gin/main.go

```
v1 := r.Group("/api/v1")
v1.GET("/orders/:id", func(c *gin.Context) {
    o, err := svc.Get(c.Request.Context(), c.Param("id"))
    if errors.Is(err, order.ErrNotFound) {
        c.JSON(http.StatusNotFound, gin.H{"error": err.Error()})
        return
    }
    if err != nil {
        c.JSON(http.StatusInternalServerError, gin.H{"error": "internal"})
        return
    }
    c.JSON(http.StatusOK, httpapi.NewOrderResponse(o))
})
```

In Echo v5, handlers return an error, which feels close to our `APIFunc` pattern:

GO

```
e := echo.New()
e.GET("/api/v1/orders/:id", func(c *echo.Context) error {
    return c.JSON(http.StatusOK, Order{ID: c.Param("id")})
})
```

And in Fiber v3:

GO

```
app := fiber.New()
app.Get("/api/v1/orders/:id", func(c fiber.Ctx) error {
    return c.JSON(Order{ID: c.Params("id")})
})
```

> ⚠️ **Gotcha: Fiber is not net/http**
>
> Fiber runs on **fasthttp**, a separate HTTP engine. Standard `http.Handler` middleware, `httptest`, and many observability libraries don't plug in directly; you go through Fiber's `adaptor` package. Fiber also reuses request buffers between requests for speed, so values read from the context are only valid inside the handler unless you copy them or enable the `Immutable` setting. Fiber v3 also requires Go 1.26 or newer.

Gin pulls in a surprisingly large dependency tree: adding Gin v1.12 to our module brought in QUIC and MongoDB BSON packages, among others. Echo v5 and chi are much lighter.

> 💡 **Our recommendation**
>
> For most enterprise teams, use the standard library or chi: you stay on the interfaces every Go library understands, and you never need a framework migration. If your team already knows Gin well, Gin is a perfectly reasonable choice. Avoid Fiber unless you have measured a need for fasthttp.

Pagination and API versioning

Two design questions come up in every API review: how clients page through large lists, and how the API evolves without breaking them.

Cursor vs offset pagination

	Offset (`?page=3&size=20`)	Cursor (`?cursor=...&limit=20`)
Spring equivalent	`Pageable` , `Page<T>`	`Window<T>` , `ScrollPosition`
SQL	`LIMIT 20 OFFSET 40`	`WHERE id > $last LIMIT 20`
Deep pages	slow: the database skips rows	fast: an index seek
Inserts while paging	items shift, duplicates appear	stable
Jump to page 7	yes	no

Offset pagination is fine for small admin tables. For feeds, exports and anything large, prefer **cursor** pagination, also called keyset pagination. The cursor is an opaque token: we base64-encode the last ID so clients don't build cursors themselves.

GO internal/httpapi/handlers.go

```
    after, err := decodeCursor(r.URL.Query().Get("cursor"))
    if err != nil {
        return err
    }
    page, err := h.orders.List(r.Context(), order.ListFilter{
        Status: order.Status(q.Status), Limit: q.Limit, After: after,
    })
    if err != nil {
        return err
    }
    resp := ListResponse{Items: make([]OrderResponse, 0, len(page.Orders))}
    for _, o := range page.Orders {
        resp.Items = append(resp.Items, NewOrderResponse(o))
    }
    resp.NextCursor = encodeCursor(page.Next)
```

A client keeps following `next_cursor` until it disappears (it's `omitempty`):

TERMINAL

```
curl "localhost:8080/api/v1/orders?limit=2"
curl "localhost:8080/api/v1/orders?limit=2&cursor=NDlkZDZkYTQtZjdi..."
```

With Postgres, the query behind this is a keyset condition such as the one below; Chapter 9 builds it with sqlc.

SQL

```sql
SELECT id, customer_id, status, created_at FROM orders
WHERE (created_at, id) > ($1, $2)
ORDER BY created_at, id
LIMIT $3;
```

Versioning

Put the major version in the path (`/api/v1/...`). It's visible in logs, easy to route, and what most enterprise API guidelines recommend. Header or media-type versioning works too, but it's harder to debug from a browser or a log line.

When you retire an old version, tell clients in machine-readable headers before you switch it off. chi v5.5 has middleware for both standard headers:

GO chi/main.go

```go
    r.With(
        middleware.Deprecation(time.Date(2026, 6, 1, 0, 0, 0, 0, time.UTC)),
        middleware.Sunset(time.Date(2027, 6, 1, 0, 0, 0, 0, time.UTC)),
    ).Get("/api/v0/orders/{id}", h.Handle(h.GetOrder))
```

OUTPUT

```
HTTP/1.1 404 Not Found
Content-Type: application/problem+json
Deprecation: @1780272000
Sunset: Tue, 01 Jun 2027 00:00:00 GMT
```

http.Server timeouts

Here's a line you'll see in many tutorials, and should never ship:

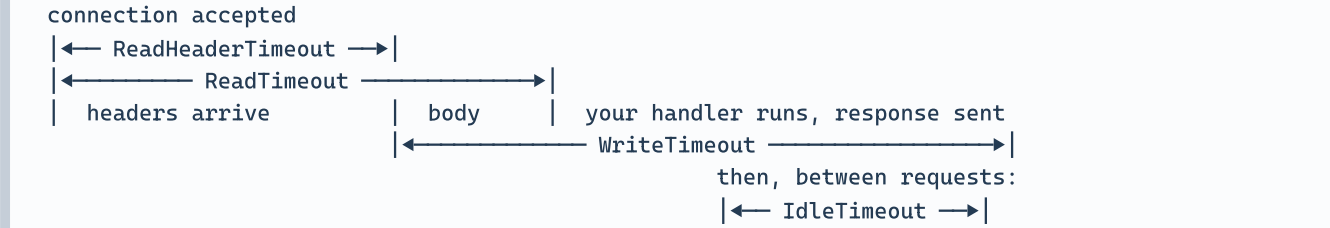
GO Avoid

```go
log.Fatal(http.ListenAndServe(":8080", handler)) // no timeouts at all
```

`http.ListenAndServe` uses a zero-value `http.Server`, and **zero means "no timeout"**. A client that sends its headers one byte per minute (the "Slowloris" attack) holds a connection and a goroutine forever. Thousands of them exhaust your file descriptors. Tomcat protects Spring apps with a connection timeout out of the box; in Go, you set the limits yourself:

GO stdlib/main.go

```go
func newServer(addr string, h http.Handler, log *slog.Logger) *http.Server {
    return &http.Server{
        Addr:              addr,
        Handler:           h,
        ReadHeaderTimeout: 5 * time.Second,  // slow-header (Slowloris) guard
        ReadTimeout:       10 * time.Second, // whole request, body included
        WriteTimeout:      15 * time.Second, // from end of headers to response
        IdleTimeout:       60 * time.Second, // keep-alive between requests
        MaxHeaderBytes:    1 << 20,
        ErrorLog:          slog.NewLogLogger(log.Handler(), slog.LevelWarn),
    }
}
```



Field	Protects against	A sensible start
`ReadHeaderTimeout`	clients that send headers slowly	5s
`ReadTimeout`	slow request bodies	10 to 30s
`WriteTimeout`	handlers or clients that never finish	a bit above your slowest endpoint
`IdleTimeout`	idle keep-alive connections piling up	60 to 120s

> ⚠️ **Gotcha: WriteTimeout and streaming**
>
> `WriteTimeout` also cuts off long-lived responses such as Server-Sent Events. Don't remove it globally; instead, lift it per request with `http.NewResponseController(w).SetWriteDeadline(...)`. Chapter 8 shows how.

OpenAPI: spec-first with oapi-codegen

OpenAPI documents are how other teams discover and trust your API. There are two ways to keep the document and the code in sync:

Approach	Go tools	Java tool	Trade-off
Code-first	swaggo/swag (comment annotations)	springdoc-openapi	quick start; the spec follows the code

Approach	Go tools	Java tool	Trade-off
Spec-first	oapi-codegen, ogen	openapi-generator	the spec is the contract; code must match it

springdoc works well in Spring because it reads annotations at runtime. Go has no runtime annotations, so swaggo parses special comments instead, and swag v1 still produces the older Swagger 2.0 format (an OpenAPI 3 version, swag v2, has been in release-candidate stage for a long time).

> 💡 **If you're unsure, go spec-first with oapi-codegen**
>
> Write `openapi.yaml` first, review it like code, and generate Go types and a server interface from it. The compiler then refuses to build until every operation is implemented. ogen is a strong alternative that also generates validation, but it's more opinionated.

The example configures oapi-codegen (v2.8.2) to generate models and a **strict server** for the standard library router:

YAML oapi/oapi-codegen.yaml

```yaml
package: api
output: api/api.gen.go
generate:
  models: true
  std-http-server: true # Go 1.22+ net/http ServeMux
  strict-server: true   # typed request and response objects
```

oapi-codegen is installed as a Go 1.24 **tool dependency**, pinned in `go.mod` like any library, and run through `go generate`:

TERMINAL

```
go get -tool github.com/oapi-codegen/oapi-codegen/v2/cmd/oapi-codegen@v2.8.2
go generate ./oapi/...   # runs: go tool oapi-codegen -config ... openapi.yaml
```

From the spec, oapi-codegen generates this interface (reformatted to fit the page):

GO oapi/api/api.gen.go (generated)

```go
type StrictServerInterface interface {
    // (POST /api/v1/orders)
    PlaceOrder(ctx context.Context, request PlaceOrderRequestObject) (
        PlaceOrderResponseObject, error)

    // (GET /api/v1/orders/{id})
    GetOrder(ctx context.Context, request GetOrderRequestObject) (
        GetOrderResponseObject, error)
}
```

Every response in the spec becomes its own type, such as `GetOrder200JSONResponse` and `GetOrder404ApplicationProblemPlusJSONResponse`. You can only return responses the spec declares:

GO oapi/server.go

```go
// The build fails if an operation in openapi.yaml is not implemented.
var _ api.StrictServerInterface = (*server)(nil)

func (s *server) GetOrder(ctx context.Context,
    req api.GetOrderRequestObject) (api.GetOrderResponseObject, error) {
    o, err := s.orders.Get(ctx, req.Id)
    if errors.Is(err, order.ErrNotFound) {
        p := problem(http.StatusNotFound, "Order not found", err)
        return api.GetOrder404ApplicationProblemPlusJSONResponse(p), nil
    }
    if err != nil {
        return nil, err // becomes a 500 via ResponseErrorHandlerFunc
    }
    return api.GetOrder200JSONResponse(toAPI(o)), nil
}
```

Wiring it up takes two lines:

GO oapi/main.go

```go
strict := api.NewStrictHandler(&server{orders: svc}, nil)
return api.HandlerFromMux(strict, http.NewServeMux())
```

📝 Validation is a separate step

oapi-codegen generates types, not checks: `minItems` and `maximum` in the spec are not enforced by the generated code. Add the `OapiRequestValidator` middleware from `github.com/oapi-codegen/nethttp-middleware` to validate requests against the spec, or keep validating in your handlers.

The HTTP client side

Services call other services, and the client side has its own traps. Spring developers know `RestTemplate` (now in maintenance mode), its modern replacement `RestClient` (Spring 6.1+), and the reactive `WebClient`:

JAVA

```java
var factory = new SimpleClientHttpRequestFactory();
factory.setConnectTimeout(Duration.ofSeconds(2));
factory.setReadTimeout(Duration.ofSeconds(5));

RestClient client = RestClient.builder()
        .baseUrl("http://orders:8080")
        .requestFactory(factory)
        .build();

OrderResponse o = client.get().uri("/api/v1/orders/{id}", id)
        .retrieve().body(OrderResponse.class);
```

Go has one type for all of this: `http.Client`. It's safe for concurrent use and pools connections, so create one per downstream service and share it:

GO client/client.go

```go
func New(baseURL string) *Client {
    t := http.DefaultTransport.(*http.Transport).Clone()
    t.MaxIdleConnsPerHost = 32 // the default of 2 is low for services
    return &Client{
        baseURL: baseURL,
        http: &http.Client{
            Timeout:   10 * time.Second, // hard cap for the whole exchange
            Transport: t,
        },
    }
}
```

> ⚠️ **Gotcha: http.DefaultClient has no timeout**
>
> `http.Get(url)` and `http.DefaultClient` wait forever for a server that accepts the connection but never answers. One slow dependency can then tie up every goroutine in your service. Always build your own client with a `Timeout`, and pass a `context` with a deadline for each call.

A call then follows a fixed rhythm: build the request with a context, send it, **always** close the body, check the status, decode:

GO client/client.go

```go
    req, err := http.NewRequestWithContext(ctx, http.MethodGet, u, nil)
    if err != nil {
        return Order{}, err
    }
    req.Header.Set("Accept", "application/json")
```

```
    resp, err := c.http.Do(req)
    if err != nil {
        return Order{}, fmt.Errorf("get order %s: %w", id, err)
    }
    defer drainAndClose(resp.Body) // always, or the connection leaks

    if resp.StatusCode != http.StatusOK {
        apiErr := &APIError{Status: resp.StatusCode}
        _ = json.NewDecoder(resp.Body).Decode(apiErr) // best effort
        return Order{}, apiErr
    }
```

Unlike `RestTemplate`, a non-2xx status is **not** an error in Go: `err` is nil and you check `resp.StatusCode` yourself. `drainAndClose` reads any leftover bytes before closing, so the connection can go back to the pool for reuse.

> **Newer Go**
>
> From Go 1.27, closing an HTTP/1 response body drains a limited amount of unread content automatically. The explicit drain is still harmless, and needed on Go 1.25.

Retries, circuit breakers and rate limits belong around this client; Chapter 14 covers them.

A first taste of handler testing

Because handlers are plain `http.Handler` values, you can test them without starting a server. `httptest.NewRequest` builds a request, and `httptest.NewRecorder` captures the response:

GO stdlib/main_test.go

```
func TestPlaceOrder(t *testing.T) {
    body := `{"customer_id":"6f1c2a52-6c39-4b8e-9f43-0f7a3b1d2e10",
        "items":[{"product_id":"MUG-BLUE-01","quantity":2}]}`
    req := httptest.NewRequest("POST", "/api/v1/orders",
        strings.NewReader(body))
    rec := httptest.NewRecorder()

    newTestRouter().ServeHTTP(rec, req)

    if rec.Code != http.StatusCreated {
        t.Fatalf("status = %d, body = %s", rec.Code, rec.Body)
    }
    if !strings.Contains(rec.Body.String(), `"total_cents":2500`) {
        t.Errorf("unexpected body: %s", rec.Body)
    }
}
```

This runs the full stack, router and middleware included, in microseconds. The example also has a shared contract suite that both the stdlib and chi routers pass:

OUTPUT

```
--- PASS: TestPlaceOrder (0.00s)
--- PASS: TestContract (0.01s)
    --- PASS: TestContract/place,_get_and_cancel (0.00s)
    --- PASS: TestContract/errors_are_problem+json (0.00s)
    --- PASS: TestContract/status_codes (0.01s)
    --- PASS: TestContract/validation_lists_every_field (0.00s)
    --- PASS: TestContract/cursor_pagination (0.00s)
    --- PASS: TestContract/CORS_preflight (0.00s)
PASS
```

Chapter 15 goes much deeper: table-driven tests, `httptest.Server` for clients, and testcontainers for real databases.

Recap

- `http.Handler` is the one interface every Go web library shares; Go 1.22's `ServeMux` handles methods and path variables (`r.PathValue`).
- Map Jackson annotations to struct tags; remember that only exported fields are serialised, nil slices become `null`, and `omitzero` (Go 1.24) drops zero times.
- Decode with a helper that limits body size, rejects unknown fields and trailing data, and classifies errors by type.
- Validate with go-playground/validator, reporting JSON field names and friendly messages in a 422 problem.
- Let handlers return errors, and map them to RFC 9457 problems in one function, your `@ControllerAdvice`.
- Middleware is `func(http.Handler) http.Handler`; list it outermost first, with request ID and logging outside recovery.
- Use chi when you outgrow `ServeMux`; Gin is fine if your team knows it; Fiber isn't `net/http`.
- Always set `http.Server` timeouts and use your own `http.Client` with a timeout; close every response body.

Try it yourself

🏋 **Exercise 1: Filter by customer**

Add a `customer_id` query parameter to `GET /api/v1/orders` that returns only that customer's orders. Validate it as a UUID with the existing validator. *Hint:* add a field to `listQuery` and to `order.ListFilter`, then extend the contract suite in `internal/apitest`.

🏋️ Exercise 2: A product endpoint

Add `GET /api/v1/products` and `POST /api/v1/products` to both the stdlib and chi routers. A product needs a SKU, a name and a positive `price_cents`. *Hint:* reuse `decodeJSON`, `validateStruct` and the `APIFunc` pattern; the handlers should compile unchanged on both routers.

🏋️ Exercise 3: Slow-request test

Write a test that proves the `Timeout` middleware works: a handler that waits on `r.Context().Done()` should produce a `503` problem. *Hint:* wrap a test handler with `httpapi.Timeout(50 * time.Millisecond)` and return `r.Context().Err()` from an `APIFunc`.

🏋️ Exercise 4 (stretch): Spec-first cancel

Add `POST /api/v1/orders/{id}/cancel` to `oapi/openapi.yaml`, regenerate with `go generate`, and implement it. Notice how the build fails until you do. *Hint:* declare `200`, `404` and `409` responses in the spec, and the generator will create a type for each.

Further reading

- net/http package documentation: `ServeMux` patterns, `Server` fields and `ResponseController`
- Routing enhancements for Go 1.22: the Go blog post introducing method and wildcard patterns
- encoding/json package documentation: struct tags, `omitempty` and `omitzero`
- RFC 9457: Problem Details for HTTP APIs: the error format used throughout this chapter
- go-playground/validator: every built-in tag, and how to write custom ones
- chi documentation: routing, sub-routers and the middleware catalogue
- oapi-codegen: configuration options and the strict server
- Spring Framework: Error Responses: how `ProblemDetail` works on the Java side, for comparison

CHAPTER 8

gRPC, GraphQL and Real-Time APIs

> *REST is a great default, but some conversations need a faster line, a smarter question or a live feed, and Go handles all three beautifully.*

🎯 **What you'll learn**

- Choose between REST, gRPC, GraphQL, WebSockets and Server-Sent Events with a clear decision table
- Define a service in Protocol Buffers and generate Go code with buf
- Build a grpc-go server and client with interceptors, deadlines, status codes and streaming
- Serve gRPC, gRPC-Web and plain HTTP/JSON from one handler with Connect
- Build a schema-first GraphQL API with gqlgen and fix the N+1 problem with a dataloader
- Push live order-status updates with WebSockets and Server-Sent Events

The big picture

In Chapter 7 we built a REST API, and it remains the right choice for most public endpoints. But Orderly has other conversations too: the warehouse service calls the order service thousands of times a second, the mobile app wants an order with its products in one round trip, and the customer's browser wants to see "SHIPPED" appear without refreshing.

Each of those needs a different tool. The good news is that Go's ecosystem for all of them is mature, and they all build on what you already know: `net/http`, `context` and code generation.

🧩 **Think of it like choosing how to send a message**

REST is a letter: a standard format every post office accepts. gRPC is a pneumatic tube between two offices: very fast and precise, but both ends need the tube. GraphQL is a personal shopper: you hand over one list and get back exactly what you asked for. A WebSocket is a phone call: both sides can talk at any time. Server-Sent Events are a radio broadcast: the station talks and you listen.

Concept	Java / Spring	Go
Protobuf code generation	`protoc` + Maven/Gradle plugin	buf + `protoc-gen-go`
gRPC server	grpc-java + Spring gRPC	grpc-go, or Connect
Interceptors	`ServerInterceptor`	`grpc.UnaryServerInterceptor`
GraphQL	Spring for GraphQL, Netflix DGS	gqlgen (schema-first codegen)

Concept	Java / Spring	Go
N+1 fix	`@BatchMapping`, `DataLoader`	dataloadgen
WebSockets	Spring WebSocket, STOMP	`github.com/coder/websocket`
Server-Sent Events	`SseEmitter`, `Flux<ServerSentEvent>`	`net/http` + `ResponseController`

Full code: `examples/ch08-grpc-graphql-realtime/`, with generated code committed so everything builds out of the box.

Choosing the right style

Teams lose weeks arguing about API styles. Most of the decision comes down to who the client is and which direction the data flows.

Style	Best for	Think twice when
REST + JSON	public APIs, simple CRUD, wide tooling	clients need many round trips per screen
gRPC	internal service-to-service calls	browsers must call it directly
Connect	new internal services that also need curl and browsers	you must support exotic gRPC features
GraphQL	many client types needing different data shapes	simple CRUD, or heavy caching at the CDN
WebSockets	two-way, low-latency traffic (chat, collaboration)	updates only flow server to client
Server-Sent Events	live server-to-browser updates	clients must send a stream back

> 💡 **If you're unsure**
>
> Use REST for anything public, Connect (or gRPC) between your own services, and SSE for live updates to browsers. Reach for GraphQL when several front ends need different views of the same data, and WebSockets when the client really needs to talk back continuously.

Protocol Buffers and buf

gRPC and Connect both describe services in **Protocol Buffers** (protobuf): a language-neutral schema plus a compact binary format. You write a `.proto` file once and generate types and client/server code for Go, Java and many other languages. If you've used Avro or generated Java classes from WSDL, the idea is familiar.

PROTOBUF proto/orderly/v1/orders.proto

```protobuf
syntax = "proto3";

package orderly.v1;

import "google/protobuf/timestamp.proto";
```

```
// OrderService places, fetches, cancels and watches orders.
service OrderService {
  rpc PlaceOrder(PlaceOrderRequest) returns (PlaceOrderResponse);
  rpc GetOrder(GetOrderRequest) returns (GetOrderResponse);
  rpc CancelOrder(CancelOrderRequest) returns (CancelOrderResponse);
  // WatchOrder streams status changes until the order is final.
  rpc WatchOrder(WatchOrderRequest) returns (stream WatchOrderResponse);
}

enum OrderStatus {
  ORDER_STATUS_UNSPECIFIED = 0;
  ORDER_STATUS_PENDING = 1;
  ORDER_STATUS_CONFIRMED = 2;
  ORDER_STATUS_SHIPPED = 3;
  ORDER_STATUS_CANCELLED = 4;
}

message Order {
  string id = 1;
  string customer_id = 2;
  repeated OrderItem items = 3;
  OrderStatus status = 4;
  int64 total_cents = 5;
  google.protobuf.Timestamp created_at = 6;
}
```

The numbers (`= 1`, `= 2`) are **field numbers**: they identify fields on the wire, not their position. Three rules keep you out of trouble:

- Never change or reuse a field number once it has shipped; mark retired ones `reserved`.
- Adding a new field is always safe; old clients ignore it.
- Enums start with an `_UNSPECIFIED = 0` value, because zero is what an unset field reads as.

Why buf instead of protoc

The classic `protoc` compiler works, but you have to manage its plugins, include paths and flags yourself. **buf** (v1.75.0 at the time of writing) is a modern, single-binary toolchain for protobuf:

Task	protoc	buf
Generate code	long command lines	`buf generate` with a YAML file
Lint style	no	`buf lint`
Catch breaking changes	no	`buf breaking --against .git#branch=main`
Third-party protos	copy files around	dependencies in `buf.yaml`

> 💡 **If you're unsure, use buf**
>
> buf is the de facto standard for new protobuf projects. `buf breaking` alone, run in CI, prevents the most painful class of production incidents: a field change that silently breaks older clients.

The generation config lists the plugins. Managed mode sets `go_package` for us, so the `.proto` file stays free of Go-specific options:

YAML buf.gen.yaml

```yaml
version: v2
managed:
  enabled: true # buf sets go_package for us, so the .proto stays clean
  override:
    - file_option: go_package_prefix
      value: example.com/goguide/ch08/gen
plugins:
  - local: protoc-gen-go
    out: gen
    opt: paths=source_relative
  - local: protoc-gen-go-grpc
    out: gen
    opt: paths=source_relative
  - local: protoc-gen-connect-go
    out: gen
    opt: paths=source_relative
```

The three plugins are ordinary Go programs, installed with `go install` (versions are in the example's README). Then:

TERMINAL

```
buf lint && buf generate
```

OUTPUT

```
gen/orderly/v1/orderlyv1connect/orders.connect.go
gen/orderly/v1/orders.pb.go
gen/orderly/v1/orders_grpc.pb.go
```

Coming from Java

Generated Java messages are immutable and built with `Order.newBuilder().setId(...).build()`. Generated Go messages are plain structs you fill with a literal, `&orderlyv1.Order{Id: "o-1"}`. Read fields through the generated getters, such as `req.GetId()`: they're safe to call on a nil message and return the zero value instead of panicking.

gRPC with grpc-go

gRPC is a remote procedure call framework from Google that sends protobuf messages over HTTP/2. The official Go implementation, grpc-go (v1.84.0 in our examples), is used by Kubernetes, etcd and most cloud-native infrastructure.

The server

In Java with grpc-java, you extend a generated base class and push results into a `StreamObserver`:

JAVA

```java
@Service // auto-registered by Spring gRPC
public class OrderGrpcService extends OrderServiceGrpc.OrderServiceImplBase {
    @Override
    public void getOrder(GetOrderRequest req,
            StreamObserver<GetOrderResponse> responseObserver) {
        Order order = orders.find(req.getId())
                .orElseThrow(() -> Status.NOT_FOUND
                        .withDescription("order not found")
                        .asRuntimeException());
        responseObserver.onNext(GetOrderResponse.newBuilder()
                .setOrder(toProto(order)).build());
        responseObserver.onCompleted();
    }
}
```

Spring gRPC (1.0 since December 2025, for Spring Boot 4) registers such beans automatically; many Spring Boot 3 teams use the community `grpc-spring-boot-starter` and its `@GrpcService` annotation instead.

In Go, a unary method is an ordinary function that takes a request and returns a response or an error:

GO grpcsvc/server.go

```go
// orderServer implements the generated orderlyv1.OrderServiceServer.
type orderServer struct {
    // Embedding makes new RPCs added to the .proto return Unimplemented
    // instead of breaking the build. It must be embedded by value.
    orderlyv1.UnimplementedOrderServiceServer
    store *orderstore.Store
}

func (s *orderServer) GetOrder(ctx context.Context,
    req *orderlyv1.GetOrderRequest) (*orderlyv1.GetOrderResponse, error) {
    if req.GetId() == "" {
        return nil, status.Error(codes.InvalidArgument, "id is required")
    }
    o, err := s.store.Get(ctx, req.GetId())
    if err != nil {
        return nil, toStatus(err)
    }
    return &orderlyv1.GetOrderResponse{Order: protoconv.Order(o)}, nil
}
```

No observers and no builders: return a value or an error. The server is assembled in one function, which is also what the tests use:

GO grpcsvc/main.go

```go
func newServer(store *orderstore.Store, log *slog.Logger,
    token string) *grpc.Server {
    srv := grpc.NewServer(
        grpc.ChainUnaryInterceptor(loggingUnary(log), authUnary(token)),
        grpc.ChainStreamInterceptor(authStream(token)),
    )
    orderlyv1.RegisterOrderServiceServer(srv, &orderServer{store: store})
    healthpb.RegisterHealthServer(srv, health.NewServer()) // for k8s probes
    reflection.Register(srv)                               // for grpcurl
    return srv
}
```

The standard **health service** lets Kubernetes probe the server natively, and **reflection** lets tools like `grpcurl` discover your API, much like browsing a REST API with curl.

Status codes and errors

gRPC has its own set of status codes instead of HTTP statuses. As with REST in Chapter 7, map domain errors to them in one place:

GO grpcsvc/server.go

```go
func toStatus(err error) error {
    switch {
    case errors.Is(err, orderstore.ErrNotFound):
        return status.Error(codes.NotFound, err.Error())
    case errors.Is(err, orderstore.ErrInvalidArgument),
        errors.Is(err, orderstore.ErrUnknownProduct):
        return status.Error(codes.InvalidArgument, err.Error())
    case errors.Is(err, orderstore.ErrInsufficientStock),
        errors.Is(err, orderstore.ErrNotCancellable):
        return status.Error(codes.FailedPrecondition, err.Error())
    case errors.Is(err, context.Canceled),
        errors.Is(err, context.DeadlineExceeded):
        return status.FromContextError(err).Err()
    default:
        return status.Error(codes.Internal, "internal error")
    }
}
```

gRPC code	Closest HTTP status	Use it when
`InvalidArgument`	400	the request itself is wrong
`Unauthenticated`	401	no or bad credentials
`PermissionDenied`	403	authenticated, but not allowed
`NotFound`	404	the thing doesn't exist
`AlreadyExists`	409	a create would duplicate something
`FailedPrecondition`	400 / 409	the system's state forbids it

gRPC code	Closest HTTP status	Use it when
`Unavailable`	503	try again later (safe to retry)
`DeadlineExceeded`	504	the deadline passed
`Internal`	500	a bug; don't leak details

On the client, `status.Code(err)` returns the code, so callers can branch on `codes.NotFound` the way Java code catches `StatusRuntimeException`. Here's what `grpcurl` shows:

TERMINAL

```
grpcurl -plaintext -H 'authorization: Bearer dev-token' \
  -d '{"id":"nope"}' localhost:9090 orderly.v1.OrderService/GetOrder
```

OUTPUT

```
ERROR:
  Code: NotFound
  Message: order not found
```

Deadlines travel with the context

Every gRPC call should have a **deadline**: a point in time after which the caller stops waiting. In grpc-java you'd write `stub.withDeadlineAfter(2, TimeUnit.SECONDS)`. In Go, the deadline lives in the `context` you pass:

GO

```
ctx, cancel := context.WithTimeout(ctx, 2*time.Second)
defer cancel()
resp, err := client.GetOrder(ctx, &orderlyv1.GetOrderRequest{Id: id})
```

grpc-go sends the remaining time to the server, where it appears as the server-side `ctx`'s deadline. Pass that `ctx` into your database calls and the whole chain gives up together. If time runs out, the client gets `codes.DeadlineExceeded`, which the example's tests check.

> ⚠️ **Gotcha: gRPC has no default deadline**
>
> A call without a deadline can wait forever if the server hangs. Always use `context.WithTimeout` on the client side, as you set `http.Client.Timeout` in Chapter 7.

Server streaming

`WatchOrder` is a **server-streaming** RPC: one request, then a stream of responses until the server returns. Here's the heart of it:

GO grpcsvc/server.go

```
    ev := orderstore.Event{OrderID: o.ID, Status: o.Status,
        At: time.Now().UTC()} // start with the current status
    for {
        if err := stream.Send(protoconv.Event(ev)); err != nil {
            return err
        }
        if ev.Status.Final() {
            return nil // returning ends the stream with status OK
        }
        select {
        case ev = <-events:
        case <-ctx.Done(): // client cancelled or deadline passed
            return status.FromContextError(ctx.Err()).Err()
        }
    }
```

Notice the `select` from Chapter 5: the loop waits for either the next event or the client going away. On the client side, you call `Recv()` until it returns `io.EOF`, which means the server finished cleanly:

GO grpcsvc/server_test.go

```
    next, err := stream.Recv()
    if err != nil ||
        next.GetStatus() != orderlyv1.OrderStatus_ORDER_STATUS_CANCELLED {
        t.Fatalf("next = %v, %v", next, err)
    }
    if _, err := stream.Recv(); err != io.EOF { // server ended the stream
        t.Fatalf("want io.EOF, got %v", err)
    }
```

Interceptors

Interceptors are gRPC's middleware, like a grpc-java `ServerInterceptor` or a servlet filter. A unary interceptor receives the request and a `handler` that calls the real method:

GO grpcsvc/interceptors.go

```
func loggingUnary(log *slog.Logger) grpc.UnaryServerInterceptor {
    return func(ctx context.Context, req any, info *grpc.UnaryServerInfo,
        handler grpc.UnaryHandler) (any, error) {
        start := time.Now()
        resp, err := handler(ctx, req) // call the actual RPC method
        log.InfoContext(ctx, "rpc", "method", info.FullMethod,
            "code", status.Code(err).String(),
            "duration", time.Since(start).Round(time.Microsecond))
        return resp, err
    }
}
```

gRPC headers are called **metadata**. The auth interceptor reads the `authorization` entry and compares it in constant time, so response timing can't leak the token:

GO grpcsvc/interceptors.go

```go
func checkToken(ctx context.Context, want string) error {
    md, _ := metadata.FromIncomingContext(ctx)
    got := md.Get("authorization") // keys are always lowercase
    if len(got) == 0 || subtle.ConstantTimeCompare(
        []byte(got[0]), []byte("Bearer "+want)) != 1 {
        return status.Error(codes.Unauthenticated, "missing or invalid token")
    }
    return nil
}
```

`grpc.ChainUnaryInterceptor` runs interceptors in the order listed, so logging sees even the calls that auth rejects. Streaming RPCs need their own `StreamServerInterceptor`; it's easy to forget and leave streams unprotected. Chapter 12 replaces the shared token with JWT validation.

> ⚠️ **Gotcha: keep health checks public**
>
> Our first version of the auth interceptor also blocked the health and reflection services, so Kubernetes probes and `grpcurl list` failed with `Unauthenticated`. The example lets methods under `/grpc.health.v1.Health/` and `/grpc.reflection.` through without a token.

Testing in-process with bufconn

grpc-go ships `bufconn`, an in-memory listener. Tests start the real server, interceptors included, and connect a real generated client with no ports and no network:

GO grpcsvc/server_test.go

```go
    lis := bufconn.Listen(1 << 20)
    srv := newServer(orderstore.New(), log, "test-token")
    go srv.Serve(lis)
    t.Cleanup(srv.Stop)

    dial := func(ctx context.Context, _ string) (net.Conn, error) {
        return lis.DialContext(ctx)
    }
    conn, err := grpc.NewClient("passthrough:///bufnet",
        grpc.WithContextDialer(dial),
        grpc.WithTransportCredentials(insecure.NewCredentials()),
    )
```

`grpc.NewClient` is the current way to create a connection; it replaced the older `grpc.Dial`, which you'll still see in many blog posts. A client then sends the token as outgoing metadata with `metadata.AppendToOutgoingContext(ctx, "authorization", "Bearer test-token")`.

Connect: gRPC that also speaks HTTP/JSON

grpc-go is excellent, but it has some friction. It runs its own HTTP/2 server instead of plugging into `net/http`, so your middleware, router and `httptest` habits don't apply. Browsers can't call it without a proxy. And you can't poke it with curl.

Connect (`connectrpc.com/connect`, v1.21.0 in our examples), from the team behind buf, fixes this. It generates ordinary `net/http` handlers from the same `.proto` file, and one handler understands three protocols:

- **gRPC**, so existing gRPC clients in any language work unchanged
- **gRPC-Web**, so browsers can call it without a proxy
- **the Connect protocol**, which for unary calls is a plain HTTP POST with a JSON or protobuf body

The implementation looks almost the same as grpc-go, with generic request and response wrappers that carry headers:

GO connectsvc/server.go

```go
func (s *orderServer) GetOrder(ctx context.Context,
    req *connect.Request[orderlyv1.GetOrderRequest],
) (*connect.Response[orderlyv1.GetOrderResponse], error) {
    o, err := s.store.Get(ctx, req.Msg.GetId())
    if err != nil {
        return nil, toConnectError(err)
    }
    return connect.NewResponse(
        &orderlyv1.GetOrderResponse{Order: protoconv.Order(o)}), nil
}
```

Errors use the same codes as gRPC, for example `connect.NewError(connect.CodeNotFound, err)`. Mounting the service is two lines on a standard `ServeMux`:

GO connectsvc/main.go

```go
    mux := http.NewServeMux()
    path, handler := orderlyv1connect.NewOrderServiceHandler(
        &orderServer{store: store},
        connect.WithInterceptors(logging(log)),
    )
    mux.Handle(path, handler) // path is "/orderly.v1.OrderService/"
```

gRPC needs HTTP/2. Since Go 1.24, `http.Server` can serve HTTP/2 without TLS (called **h2c**), which is what you want behind a load balancer that terminates TLS:

GO connectsvc/main.go

```go
    protocols := new(http.Protocols)
    protocols.SetHTTP1(true)
    protocols.SetUnencryptedHTTP2(true)
```

Now anyone with curl can call your gRPC service:

TERMINAL

```
curl -X POST localhost:8080/orderly.v1.OrderService/PlaceOrder \
  -H 'Content-Type: application/json' \
  -d '{"customerId":"c-42","items":[{"productId":"MUG-BLUE-01","quantity":2}]}'
```

OUTPUT

```
{"order":{"id":"42072da2-2c62-4704-92e1-5784be7d6ad3","customerId":"c-42",
"items":[{"productId":"MUG-BLUE-01","quantity":2,"unitPriceCents":"1250"}],
"status":"ORDER_STATUS_CONFIRMED","totalCents":"2500",
"createdAt":"2026-11-02T18:23:39.187859900Z"}}
```

Two details of protobuf's JSON mapping surprise people. Field names become lowerCamelCase (`customerId`), and **64-bit integers are sent as strings** (`"2500"`), because JavaScript numbers can't hold every 64-bit value exactly. Errors come back as JSON too, with a matching HTTP status:

OUTPUT

```
HTTP/1.1 404 Not Found
Content-Type: application/json

{"code":"not_found","message":"order not found"}
```

The example's test proves the "one handler, many clients" claim by calling the same server four ways:

GO connectsvc/server_test.go

```go
    // 1. A Connect client using the Connect protocol (the default).
    connectClient := orderlyv1connect.NewOrderServiceClient(srv.Client(), srv.URL)

    // 2. The same generated client speaking the gRPC protocol.
    grpcProto := orderlyv1connect.NewOrderServiceClient(srv.Client(), srv.URL,
        connect.WithGRPC())

    // 3. A plain grpc-go client: Connect servers are real gRPC servers.
    conn, err := grpc.NewClient(srv.Listener.Addr().String(),
        grpc.WithTransportCredentials(credentials.NewClientTLSFromCert(pool, "")))

    // 4. Plain HTTP and JSON: no generated code, no gRPC library at all.
    httpResp, err := srv.Client().Post(
        srv.URL+"/orderly.v1.OrderService/GetOrder", "application/json",
        strings.NewReader(`{"id":"`+id+`"}`))
```

	grpc-go	Connect
Server	its own HTTP/2 server	standard `net/http` handler
Middleware	gRPC interceptors only	interceptors and `net/http` middleware
Browsers	need a proxy (Envoy)	gRPC-Web and JSON built in
curl and Postman	needs grpcurl	plain HTTP POST
Testing	`bufconn`	`httptest`

	grpc-go	Connect
Ecosystem maturity	the reference implementation	younger, smaller ecosystem

> 💡 **Our recommendation: Connect for new services**
>
> For new Go services, use Connect. You keep protobuf contracts, gRPC compatibility and streaming, and you gain curl-ability, browser support and all your `net/http` tooling. Stay on grpc-go when you need its more advanced features, such as xDS-based load balancing, or when a codebase already uses it.

grpc-gateway, briefly

Before Connect existed, the standard way to offer REST/JSON on top of a grpc-go service was **grpc-gateway**. You annotate each RPC with an HTTP mapping, and a plugin generates a reverse proxy that translates REST calls into gRPC calls:

PROTOBUF

```protobuf
import "google/api/annotations.proto";

service OrderService {
  rpc GetOrder(GetOrderRequest) returns (GetOrderResponse) {
    option (google.api.http) = { get: "/v1/orders/{id}" };
  }
}
```

grpc-gateway is actively maintained and a good fit when you must publish resource-style REST paths (like `GET /v1/orders/{id}`) for an existing grpc-go service. If you only need JSON access, Connect is simpler: there's no proxy layer and no extra annotations.

GraphQL with gqlgen

GraphQL lets the client describe exactly the data it wants in a query, and the server answers in that shape. It shines when a web app, a mobile app and partner integrations all need different slices of the same data.

In Spring for GraphQL, you map schema fields to controller methods:

JAVA

```java
@Controller
public class OrderGraphqlController {
    @QueryMapping
    public Order order(@Argument String id) { return orders.get(id); }

    @BatchMapping // one call for all items: avoids N+1
    public Map<OrderItem, Product> product(List<OrderItem> items) {
        return catalog.productsFor(items);
    }
}
```

Netflix DGS, the other popular Java option, now builds on Spring for GraphQL too. In Go, the leading library is **gqlgen** (v0.17.94 in our examples). It's **schema-first**: you write the GraphQL schema, and gqlgen generates type-safe Go code and resolver stubs for you to fill in.

GRAPHQL graphql/schema.graphqls

```graphql
type Query {
  order(id: ID!): Order
  orders(first: Int! = 10): [Order!]!
}

type Order {
  id: ID!
  customerId: ID!
  status: OrderStatus!
  items: [OrderItem!]!
  totalCents: Int!
}

type OrderItem {
  product: Product!
  quantity: Int!
  unitPriceCents: Int!
}
```

gqlgen is added as a Go tool dependency, then run with `go tool gqlgen generate`. Its config tells it to reuse our own Go types where the names match (`autobind`). Our `OrderItem` holds a product ID but no `Product` field:

GO graphql/graph/model/model.go

```go
// OrderItem has a ProductID but no Product field, so gqlgen generates
// a resolver for "product". That resolver is where N+1 queries hide.
type OrderItem struct {
    ProductID      string
    Quantity       int
    UnitPriceCents int
}
```

Resolvers

A **resolver** is a function that produces the value of one field. gqlgen generates a stub for every field it can't fill from a struct field or method, and you write the body. On regeneration, your code is kept (signatures wrapped here to fit the page):

GO graphql/graph/schema.resolvers.go

```go
// Order is the resolver for the order field.
func (r *queryResolver) Order(ctx context.Context,
    id string) (*model.Order, error) {
    return r.Repo.Get(id)
}

// Product is the resolver for the product field.
func (r *orderItemResolver) Product(ctx context.Context,
    obj *model.OrderItem) (*model.Product, error) {
    // Not r.Catalog directly: the loader batches all items' lookups.
    return For(ctx).Product.Load(ctx, obj.ProductID)
}
```

The server is a `net/http` handler, so it mounts on a `ServeMux` like everything else:

GO graphql/main.go

```go
    srv := handler.New(schema)
    srv.AddTransport(transport.POST{})
    srv.Use(extension.Introspection{})          // lets tools read the schema
    srv.Use(extension.FixedComplexityLimit(50)) // rejects huge queries

    mux := http.NewServeMux()
    mux.Handle("POST /query", graph.LoaderMiddleware(catalog, srv))
    mux.Handle("GET /", playground.Handler("Orderly", "/query"))
```

The complexity limit matters: without one, a client can write a deeply nested query that makes your server do enormous work. Here's a real query:

GRAPHQL

```graphql
{ order(id: "o-1") { status totalCents items { quantity product { sku } } } }
```

Wrapped in a JSON body (`graphql/order-query.json` in the example), it goes out with curl:

TERMINAL

```
curl -X POST localhost:8080/query -H 'Content-Type: application/json' \
  -d @order-query.json
```

OUTPUT

```
{"data": {"order": {"status": "CONFIRMED", "totalCents": 4500,
  "items": [{"quantity": 2, "product": {"sku": "MUG-BLUE-01"}},
            {"quantity": 1, "product": {"sku": "TEE-GOPHER-L"}}]}}}
```

GraphQL reports errors differently from REST. The HTTP status is usually `200`, and problems appear in an `errors` array next to `data`:

OUTPUT

```
{"errors":[{"message":"order has already shipped","path":["cancelOrder"],
  "locations":[{"line":1,"column":12}]}],"data":null}
```

The N+1 problem and dataloaders

Ask for 3 orders with 2 items each, including each item's product. A naive `product` resolver runs once per item: 1 query for the orders plus 6 product queries. With 100 orders, that's 201 queries. This is the **N+1 problem**, and it's the most common GraphQL performance bug.

> 🧩 **Think of it like a waiter**
>
> A bad waiter walks to the kitchen after each guest orders. A good waiter collects the whole table's order, then makes one trip. A dataloader is the good waiter: it collects every product ID requested in a short window, then makes one batched query.

```
without a dataloader                with a dataloader
--------------------                -----------------------------
orders query          1             orders query                1
product(p-mug)        1             product(p-mug) ─┐
product(p-tee)        1             product(p-tee)  │ collected
product(p-tee)        1             product(p-tee)  │ for 1ms,
product(p-cap)        1             product(p-cap)  │ then one
product(p-mug)        1             product(p-mug)  │ batched call
product(p-cap)        1             product(p-cap) ─┘           1
total                 7             total                       2
```

We use **dataloadgen** (v0.0.10), the generic dataloader the gqlgen docs recommend. A mapped loader takes a function that fetches many keys at once:

GO graphql/graph/loaders.go

```go
func NewLoaders(c *Catalog) *Loaders {
    return &Loaders{
        // Collect every product ID requested within 1ms, then fetch them
        // all with a single ProductsByIDs call.
        Product: dataloadgen.NewMappedLoader(c.ProductsByIDs,
            dataloadgen.WithWait(time.Millisecond)),
    }
}
```

Loaders also cache results, so they must live for **one request only**; otherwise one user could see another user's cached data. A small middleware creates fresh loaders per request and puts them in the context, where the resolver finds them with `For(ctx)`. The test proves the batching works:

GO graphql/main_test.go

```go
resp := query(t, h, `{ orders { id totalCents items { product { name } } } }`)
// ...
if got := catalog.Calls(); got != 1 {
    t.Fatalf("ProductsByIDs called %d times, want 1 (N+1!)", got)
}
```

Coming from Java

Spring for GraphQL solves N+1 with `@BatchMapping` or a registered `DataLoader`, wired by the framework. In gqlgen, you wire the loader yourself with a middleware and a context value. It's a few more lines, and nothing is hidden.

WebSockets

A **WebSocket** upgrades an HTTP connection into a long-lived, two-way channel. Either side can send a message at any time, which suits chat, collaborative editing and dashboards that also send commands.

Which library?

Library	Status	Notes
`github.com/coder/websocket`	maintained by Coder since 2024	formerly `nhooyr.io/websocket`; `context` everywhere, no dependencies
`github.com/gorilla/websocket`	archived Dec 2022, revived July 2023	stable, very widely used; last release v1.5.3 (June 2024)
`golang.org/x/net/websocket`	old	its own docs point to the two above

If you're unsure, use coder/websocket

It takes a `context` on every call, so cancellation and deadlines work the way the rest of your Go code does, and it's safe to write from several goroutines. The old `nhooyr.io/websocket` import path still resolves, but new releases come from `github.com/coder/websocket` (v1.8.15 in our examples). gorilla/websocket remains a reasonable choice if your codebase already uses it.

In Spring you'd implement a `WebSocketHandler` or use STOMP messaging over WebSocket. In Go, the handler is a regular `http.HandlerFunc` that **accepts** the upgrade:

GO websocket/main.go

```go
c, err := websocket.Accept(w, r, &websocket.AcceptOptions{
    // Browsers on other origins are rejected unless listed here.
    OriginPatterns: []string{"app.orderly.example"},
})
```

```
        if err != nil {
            return // Accept has already written an HTTP error
        }
        defer c.CloseNow() // a no-op if we closed cleanly below

        ctx, cancel := context.WithCancel(r.Context())
        defer cancel()
        go readCommands(ctx, cancel, c, store, id, log)

        msg := statusMessage{OrderID: o.ID, Status: o.Status}
        for {
            if err := wsjson.Write(ctx, c, msg); err != nil {
                return
            }
            if msg.Status.Final() {
                c.Close(websocket.StatusNormalClosure, "order is final")
                return
            }
            select {
            case ev := <-events:
                msg = statusMessage{OrderID: ev.OrderID, Status: ev.Status}
            case <-ctx.Done(): // the client went away
                return
            }
        }
```

The connection has two jobs, so it gets two goroutines. The handler's loop writes status updates, while `readCommands` reads messages such as `{"action":"cancel"}` from the browser. When reading fails because the client disconnected, `readCommands` cancels the shared context, and the writer stops too.

> ⚠️ **Gotcha: WebSockets in production**
>
> - **Check the origin.** Without it, any website a user visits could open a socket to your API using their cookies. coder/websocket rejects cross-origin browsers unless you list them in `OriginPatterns`.
> - **Authenticate before upgrading**, while you still have a normal HTTP request to reject.
> - **Plan for several instances.** A socket lives on one pod. To reach users connected elsewhere, publish events through a broker such as Redis or Kafka (Chapter 11).

The test dials the server, sends a command over the same connection, and checks that the update and a polite close arrive:

GO websocket/main_test.go

```go
if err := wsjson.Write(ctx, c, command{Action: "cancel"}); err != nil {
    t.Fatal(err)
}
err = wsjson.Read(ctx, c, &msg)
if err != nil || msg.Status != "CANCELLED" {
    t.Fatalf("after cancel = %+v, %v", msg, err)
}

// CANCELLED is final, so the server closes the connection politely.
err = wsjson.Read(ctx, c, &msg)
if websocket.CloseStatus(err) != websocket.StatusNormalClosure {
    t.Fatalf("want normal closure, got %v", err)
}
```

Server-Sent Events

Most "live updates" only flow one way: the server tells the browser that something changed. For that, **Server-Sent Events** (SSE) are simpler than WebSockets. SSE is a normal HTTP response that never quite ends: the server keeps writing small text messages, and the browser's built-in `EventSource` API reads them and even reconnects automatically.

In Spring MVC you'd return an `SseEmitter`, or a `Flux<ServerSentEvent<T>>` in WebFlux. In Go, you need nothing beyond `net/http`. The wire format is plain text: `field: value` lines, ending with a blank line.

```
id: 1
event: status
data: {"order_id":"57af127c-...","status":"SHIPPED","at":"2026-11-02T18:30:24Z"}
```

Here's the handler's setup. Note the first line: it lifts the server's `WriteTimeout` for this one response, as promised in Chapter 7:

GO sse/main.go

```go
    rc := http.NewResponseController(w)
    // Lift the server's WriteTimeout for this long-lived response.
    if err := rc.SetWriteDeadline(time.Time{}); err != nil {
        http.Error(w, "streaming unsupported",
            http.StatusInternalServerError)
        return
    }
    w.Header().Set("Content-Type", "text/event-stream")
    w.Header().Set("Cache-Control", "no-cache")
    w.Header().Set("X-Accel-Buffering", "no") // tell nginx not to buffer
    w.WriteHeader(http.StatusOK)
```

`http.ResponseController` (Go 1.20+) gives you per-request control over flushing and deadlines. It finds the real writer through any middleware wrappers that implement `Unwrap`, which is why Chapter 7's `statusRecorder` has that method. Each event is written and then **flushed**, so it leaves now instead of sitting

in a buffer:

GO sse/main.go

```go
func writeEvent(w http.ResponseWriter, rc *http.ResponseController,
    ev orderstore.Event) error {
    data, err := json.Marshal(statusEvent{ev.OrderID, ev.Status, ev.At})
    if err != nil {
        return err
    }
    // The format: "field: value" lines, then a blank line.
    if _, err := fmt.Fprintf(w, "id: %d\nevent: status\ndata: %s\n\n",
        ev.Seq, data); err != nil {
        return err
    }
    return rc.Flush() // push it now instead of waiting for a full buffer
}
```

The main loop then waits for an event, a heartbeat tick or the client leaving:

GO sse/main.go

```go
        for {
            select {
            case ev := <-events:
                if err := writeEvent(w, rc, ev); err != nil {
                    return // the client disconnected
                }
                if ev.Status.Final() {
                    return // ending the handler closes the stream
                }
            case <-heartbeat.C: // a comment line keeps proxies from timing out
                if _, err := fmt.Fprint(w, ": ping\n\n"); err != nil {
                    return
                }
                if err := rc.Flush(); err != nil {
                    return
                }
            case <-r.Context().Done(): // the browser tab was closed
                return
            }
        }
```

Let's watch an order ship. In one terminal, follow the stream with `curl -N` (no buffering); in another, ship the order:

TERMINAL

```
curl -N localhost:8080/api/v1/orders/57af127c-.../events
curl -X POST localhost:8080/api/v1/orders/57af127c-.../ship   # second terminal
```

OUTPUT

```
id: 0
event: status
data: {"order_id":"57af127c-...","status":"CONFIRMED","at":"2026-11-02T18:30:22Z"}

id: 1
event: status
data: {"order_id":"57af127c-...","status":"SHIPPED","at":"2026-11-02T18:30:24Z"}
```

The first event is the current state, so a fresh page never shows stale data. When the order reaches a final status, the handler returns and the stream ends. The example's test reads the stream with a `bufio.Scanner` and checks both events arrive.

> ⚠️ **Gotcha: SSE behind proxies and in browsers**
>
> - Proxies and load balancers may buffer or time out quiet responses. The `X-Accel-Buffering` header and the heartbeat comment handle the usual suspects.
> - Over HTTP/1.1, browsers allow only about six connections per site, and each open `EventSource` uses one. Serve SSE over HTTP/2, where streams share one connection.
> - On reconnect, the browser sends the last `id:` it saw in a `Last-Event-ID` header. Use it to replay missed events if they matter.

Recap

- Pick by client and direction: REST for public APIs, gRPC or Connect between services, GraphQL for many differently shaped clients, SSE for one-way live updates, WebSockets for two-way traffic.
- Define services in protobuf, never reuse field numbers, and generate code with buf; run `buf breaking` in CI.
- grpc-go methods return a response or a `status` error; deadlines travel in the `context`; interceptors are gRPC's middleware; `bufconn` gives fast in-process tests.
- Connect serves gRPC, gRPC-Web and HTTP/JSON from one `net/http` handler; prefer it for new services.
- gqlgen is schema-first: you write the schema, it generates the plumbing, you fill in resolvers. Fix N+1 with a per-request dataloader and limit query complexity.
- Use coder/websocket for WebSockets, check origins, and authenticate before upgrading.
- SSE needs only `net/http`: set `text/event-stream`, flush each event, and lift `WriteTimeout` per request with `http.ResponseController`.

Try it yourself

> 🏋️ **Exercise 1: A ListOrders RPC**
>
> Add `rpc ListOrders(ListOrdersRequest) returns (ListOrdersResponse)` with `page_size` and `page_token` fields, regenerate with `buf generate`, and implement it in the Connect server. Call it with curl. *Hint:* run `buf lint` first; it will remind you of protobuf naming conventions.

> 🏋️ **Exercise 2: Deadline propagation**
>
> Make the gRPC store's `Get` method sleep for 3 seconds when the ID is `"slow"`, but return early when `ctx.Done()` fires. Write a test that calls it with a 100ms deadline and expects `codes.DeadlineExceeded`. *Hint:* use `select` with `time.After` and `ctx.Done()`, and map `context.DeadlineExceeded` with `status.FromContextError`.

> 🏋️ **Exercise 3: Customers in GraphQL**
>
> Add `customer: Customer!` to the `Order` type, with a `Customer` that has `id` and `name`. Write a second dataloader so that listing orders makes one customer lookup, and prove it with a test like `TestDataloaderBatches`. *Hint:* follow the `Product` loader; gqlgen will generate the new resolver stub for you.

> 🏋️ **Exercise 4 (stretch): Resumable SSE**
>
> Keep the last 100 events in the store. When a client reconnects with a `Last-Event-ID` header, send any events it missed before switching to live updates. *Hint:* the event `Seq` already increases monotonically, so "missed" means `Seq > lastID`.

Further reading

- Protocol Buffers language guide (proto3): field numbers, defaults and evolution rules
- buf documentation: `buf generate`, lint rules and breaking-change detection
- gRPC Go basics tutorial: the official walkthrough, including streaming
- gRPC status codes: what every code means and when to use it
- Connect for Go: getting started: the same ideas as this chapter, step by step
- gqlgen documentation: configuration, resolvers and dataloaders
- coder/websocket: API docs and examples
- Using server-sent events (MDN): the browser side of SSE, including `EventSource`

CHAPTER 9

Working with Databases and Caches

> *Almost every enterprise service lives or dies by its data layer, so this is where Go's "explicit over magic" style pays off the most.*

🎯 **What you'll learn**

- Use `database/sql` correctly: the pool, `QueryContext`, `Scan`, `rows.Err()`, `sql.ErrNoRows` and NULLs
- Talk to Postgres with **pgx/v5** and **pgxpool**, and generate type-safe Go from plain SQL with **sqlc**
- Weigh sqlx, GORM and ent honestly against what you know from JPA, jOOQ and MyBatis
- Run transactions without `@Transactional`, and reserve stock safely with `SELECT ... FOR UPDATE`
- Migrate schemas with **goose**, tune the pool like HikariCP and put timeouts on every query
- Add Redis and in-process caching, and stop cache stampedes with `singleflight`

The big picture

In Spring Boot, the data layer arrives almost fully assembled. You add a starter, write an interface that extends `JpaRepository`, put `@Transactional` on a service method and HikariCP, Hibernate and a transaction manager quietly do the rest. It's productive, and it hides a lot.

Go takes the opposite approach. The standard library gives you a small, solid core (`database/sql`), and the community builds on it with tools that keep **SQL visible**. You'll write more SQL than you did with JPA. In return, you'll always know which query runs, when it runs and inside which transaction.

🧩 **Think of it like a taxi rank**

A database connection pool is a taxi rank. You don't buy a taxi for every trip. You take the next free one, ride, and it goes back to the rank. Forgetting to close a result set is like keeping the taxi parked outside your house with the meter running: soon the rank is empty and everyone else waits.

Here's the map for this chapter.

Concept	Java / Spring	Go
Low-level API	JDBC	`database/sql`
Postgres driver	PgJDBC	**pgx/v5**
Connection pool	HikariCP	`pgxpool` (or the pool inside `sql.DB`)

Concept	Java / Spring	Go
Type-safe SQL	jOOQ, MyBatis	**sqlc**
ORM	Hibernate / JPA	GORM, ent
Transactions	`@Transactional`	`Begin` / `Commit` / `Rollback`, a `WithTx` helper
Migrations	Flyway, Liquibase	**goose**
Caching	`@Cacheable`, Caffeine	go-redis cache-aside, otter, `singleflight`
Missing value	`Optional<T>`, `null`	`sql.Null[T]`, pointers

Full code: `examples/ch09-data/`. Most programs there run without Docker: they use a pure-Go SQLite driver or an in-memory Redis, so you can try everything on a laptop.

database/sql: Go's JDBC

`database/sql` is the standard library's database API. Like JDBC, it defines the interfaces, and a **driver** (a package you import for its side effect of registering itself) talks to the actual database. Even if you end up using pgx and sqlc, it's worth knowing, because many libraries (goose, for one) speak `database/sql`.

sql.DB is a pool, not a connection

The most common beginner misunderstanding is right here. `sql.Open` returns a `*sql.DB`, and despite the name it is **a pool of connections**, safe for many goroutines at once. You create one at startup and share it for the life of the program.

GO basics/db.go

```go
// open returns a *sql.DB. Remember: this is a pool, not one connection.
func open(ctx context.Context) (*sql.DB, error) {
    db, err := sql.Open("sqlite", "file:orderly?mode=memory&cache=shared")
    if err != nil {
        return nil, err // only fails on a bad driver name or DSN
    }
    db.SetMaxOpenConns(10)                  // like HikariCP maximumPoolSize
    db.SetMaxIdleConns(5)                   // idle connections kept around
    db.SetConnMaxLifetime(30 * time.Minute) // recycle old connections

    // sql.Open doesn't connect. Ping does, so we fail fast at startup.
    if err := db.PingContext(ctx); err != nil {
        db.Close()
        return nil, fmt.Errorf("ping: %w", err)
    }
    // ... create the table (see the full file)
```

Two things surprise Java developers:

- **`sql.Open` doesn't connect.** It only validates its arguments. The first real connection happens on first use, so call `PingContext` at startup if you want to fail fast.
- **You never "get a connection" yourself.** Each query borrows one from the pool and returns it when it's done. That's the `DataSource.getConnection()` and `close()` dance from JDBC, done for you.

> **Why SQLite here?**
>
> This demo uses `modernc.org/sqlite`, a pure-Go SQLite driver, so it runs with no database server and no C compiler. For Postgres via `database/sql`, import `_ "github.com/jackc/pgx/v5/stdlib"` and call `sql.Open("pgx", url)`. The rest of the code stays the same, including the `$1` placeholders.

Querying: the loop that matters

Here's the JDBC version you've written a hundred times:

JAVA

```java
String sql = "SELECT id, sku, name, price_cents FROM products "
        + "WHERE price_cents <= ? ORDER BY sku";
try (Connection conn = dataSource.getConnection();
     PreparedStatement ps = conn.prepareStatement(sql)) {
    ps.setLong(1, maxPrice);
    try (ResultSet rs = ps.executeQuery()) {
        while (rs.next()) {
            products.add(new Product(rs.getString("id"),
                    rs.getString("sku"), rs.getString("name"),
                    rs.getLong("price_cents")));
        }
    }
}
```

And the Go version. Every line has a job, so read it slowly once:

GO basics/db.go

```go
func listProducts(ctx context.Context, db *sql.DB,
    maxPrice int64) ([]Product, error) {
    rows, err := db.QueryContext(ctx,
        `SELECT id, sku, name, price_cents, note, discount
         FROM products WHERE price_cents <= $1 ORDER BY sku`, maxPrice)
    if err != nil {
        return nil, err
    }
    defer rows.Close() // gives the connection back to the pool

    var out []Product
    for rows.Next() {
        var p Product
        err := rows.Scan(&p.ID, &p.SKU, &p.Name, &p.PriceCents,
            &p.Note, &p.Discount)
        if err != nil {
            return nil, err
        }
        out = append(out, p)
    }
    // Next returns false on success AND on failure. Err tells them apart.
    return out, rows.Err()
}
```

What's going on:

- `QueryContext` takes a `context.Context` first. If the request is cancelled or its deadline passes, the query is cancelled too. Always use the `...Context` variants.
- `defer rows.Close()` is your try-with-resources. Without it, the connection stays checked out (the taxi with the meter running).
- `Scan` copies columns into variables by position, so you pass pointers (`&p.ID`). The column count and order must match the `SELECT`.
- `rows.Err()` is the line everyone forgets. If the network drops halfway through, `Next` returns `false` and the loop ends as if all was well. Only `rows.Err()` tells you that you got half the rows.

For statements that don't return rows, use `ExecContext`. It gives you a `sql.Result` with `RowsAffected()`, the equivalent of the `int` that `executeUpdate()` returns.

One row, and sql.ErrNoRows

For a single row, `QueryRowContext` returns a `*sql.Row` whose `Scan` also closes it. If nothing matched, `Scan` returns the sentinel error `sql.ErrNoRows`. That's Go's version of an empty `Optional` from `findById`.

GO basics/db.go

```go
// ErrProductNotFound is our domain error; callers never see sql errors.
var ErrProductNotFound = errors.New("product not found")

func productBySKU(ctx context.Context, db *sql.DB,
    sku string) (Product, error) {
    var p Product
    err := db.QueryRowContext(ctx,
        `SELECT id, sku, name, price_cents, note, discount
         FROM products WHERE sku = $1`, sku).
        Scan(&p.ID, &p.SKU, &p.Name, &p.PriceCents, &p.Note, &p.Discount)
    if errors.Is(err, sql.ErrNoRows) {
        return Product{}, ErrProductNotFound
    }
    return p, err
}
```

> 💡 **Translate driver errors at the edge**
>
> Turn `sql.ErrNoRows` (or `pgx.ErrNoRows`) into your own domain error, such as `ErrProductNotFound`, inside the data layer. Your HTTP handler can then map `ErrProductNotFound` to a 404 without importing anything database-related.

NULL handling: sql.Null[T] and pointers

Java has `null` for every object and `Optional<T>` for return values. Go's `string` and `int64` can't be nil, so a NULL column needs a type that can say "no value". You have two good options:

GO basics/main.go

```go
type Product struct {
    ID         string
    SKU        string
    Name       string
    PriceCents int64
    Note       sql.Null[string] // nullable column
    Discount   *int64           // another way to model NULL
}
```

- `sql.Null[T]` (generic, Go 1.22+) is a small struct with `V` (the value) and `Valid` (false means NULL). It reads like `Optional`: check `Valid` before using `V`. The older `sql.NullString`, `sql.NullInt64` and friends do the same job.
- **A pointer** (`*int64`) is `nil` for NULL. It's shorter to work with and maps nicely to JSON `null`, which is why sqlc can generate pointers for you (you'll see that shortly).

Running the demo shows both in action, plus the not-found path:

TERMINAL

```
go run ./basics
```

OUTPUT

```
CAP-RED-01    1800 cents  (no note)
MUG-BLUE-01   1299 cents  dishwasher safe
TEE-BLACK-M has a 10% discount
lookup NOPE-00: product not found | is ErrProductNotFound: true
rows repriced: 2
```

> ⚠️ **Gotcha: scanning NULL into a plain string**
>
> If a column can be NULL and you scan it into a plain `string`, you get a runtime error like `converting NULL to string is unsupported`. It only shows up when real data has a NULL, often in production. Make nullable columns `sql.Null[T]` or pointers from day one.

pgx and pgxpool: the Postgres driver to use

`database/sql` is deliberately generic, so it can't expose Postgres-specific features. **pgx** is a Postgres driver and toolkit written in pure Go that speaks Postgres's native protocol directly. It's the house pick for this guide, and here's why:

- **It's faster and more precise.** It uses the binary protocol and understands Postgres types natively: `uuid`, `jsonb`, arrays, `timestamptz`, `numeric`.

- **It has Postgres features** that `database/sql` can't model: `COPY` for bulk loads, `LISTEN/NOTIFY`, batched queries.
- `pgxpool` is a proper connection pool with health checks and lifetime jitter, much like HikariCP.
- **It's the default for the ecosystem.** sqlc, River (Chapter 11) and many others target pgx/v5 directly.

You'll still see `lib/pq` in older code. It works, but new projects should start with pgx.

Here's a query with pgx and no other tools. `pgx.CollectRows` runs the loop, scans each row into a struct by column name, closes the rows and checks the error, so all four lines of ceremony from the previous section disappear:

GO orderly/pgx_direct.go

```go
type ProductRow struct {
    ID         string `db:"id"`
    SKU        string `db:"sku"`
    Name       string `db:"name"`
    PriceCents int64  `db:"price_cents"`
}

func CheapProducts(ctx context.Context, pool *pgxpool.Pool,
    maxCents int64) ([]ProductRow, error) {
    rows, err := pool.Query(ctx, `
        SELECT id::text, sku, name, price_cents
        FROM products WHERE price_cents <= $1 ORDER BY sku`, maxCents)
    if err != nil {
        return nil, err
    }
    return pgx.CollectRows(rows, pgx.RowToStructByName[ProductRow])
}
```

That's pleasant for a handful of queries. For a whole service, though, you want the compiler to check that your Go types match your SQL. That's sqlc's job.

> 💡 **If you're unsure, use pgx/v5 with pgxpool, plus sqlc**
>
> That combination is the default for Postgres services in this guide and in the capstone (Chapter 20). Use `database/sql` when you need to support several databases or a library requires it; `stdlib.OpenDBFromPool` gives you a `*sql.DB` backed by your pgx pool.

sqlc: write SQL, get type-safe Go

sqlc is a code generator. You write normal SQL queries with a one-line name annotation, sqlc parses them against your schema and generates Go functions with typed parameters and result structs. If a column is renamed, regeneration fails or the compiler complains, long before production.

Coming from Java

sqlc sits between **jOOQ** and **MyBatis**. Like MyBatis, you write real SQL in its own file. Like jOOQ's code generator, you get compile-time checked types from your schema. Unlike **Spring Data**, there's no query derivation from method names: the SQL is the source of truth, and you can read exactly what runs.

The configuration: sqlc.yaml

sqlc needs to know where your schema and queries live and what to generate. It understands goose migration files directly, so your migrations *are* the schema:

YAML orderly/sqlc.yaml

```yaml
version: "2"
sql:
  - engine: "postgresql"
    schema: "migrations"      # sqlc reads the goose files, skipping Down
    queries: "queries"
    gen:
      go:
        package: "db"
        out: "db"
        sql_package: "pgx/v5"            # generate code for pgx, not database/sql
        emit_interface: true             # also generate a Querier interface
        emit_pointers_for_null_types: true   # nullable text -> *string
        overrides:
          - db_type: "uuid"
            go_type: "github.com/google/uuid.UUID"
          - db_type: "timestamptz"
            go_type: "time.Time"
```

The `overrides` swap pgx's own types (`pgtype.UUID`, `pgtype.Timestamptz`) for the friendlier `uuid.UUID` and `time.Time`.

The queries

Each query gets a `-- name:` comment with a function name and a result kind: `:one` (one row), `:many` (a slice), `:exec` (no result) or `:execrows` (rows affected).

SQL orderly/queries/orders.sql

```sql
-- name: LockStock :one
-- Locks the stock row until the transaction ends (pessimistic lock).
SELECT s.product_id, s.available, p.price_cents
FROM stock s
JOIN products p ON p.id = s.product_id
WHERE s.product_id = $1
FOR UPDATE OF s;

-- name: ReserveStock :exec
UPDATE stock
SET available = available - @quantity,
    reserved  = reserved + @quantity
WHERE product_id = @product_id;
```

```
-- name: CreateOrder :one
INSERT INTO orders (id, customer_id, status)
VALUES ($1, $2, $3)
RETURNING *;
```

`@quantity` is a named parameter, which becomes a field name in the generated params struct. Run the generator from the folder with `sqlc.yaml`:

TERMINAL

```
sqlc generate
```

The generated code

sqlc writes ordinary, readable Go into `db/`. You commit it, review it in pull requests and never edit it by hand. First, the core types:

GO orderly/db/db.go (generated)

```go
type DBTX interface {
    Exec(context.Context, string, ...interface{}) (pgconn.CommandTag, error)
    Query(context.Context, string, ...interface{}) (pgx.Rows, error)
    QueryRow(context.Context, string, ...interface{}) pgx.Row
}

func New(db DBTX) *Queries {
    return &Queries{db: db}
}

type Queries struct {
    db DBTX
}
```

Keep an eye on `DBTX`. Both `*pgxpool.Pool` and `pgx.Tx` satisfy it, which is the key to transactions later. And here's one generated query, with a struct for its result:

GO orderly/db/orders.sql.go (generated)

```go
type LockStockRow struct {
    ProductID  uuid.UUID
    Available  int32
    PriceCents int64
}

// Locks the stock row until the transaction ends (pessimistic lock).
func (q *Queries) LockStock(ctx context.Context,
    productID uuid.UUID) (LockStockRow, error) {
    row := q.db.QueryRow(ctx, lockStock, productID)
    var i LockStockRow
    err := row.Scan(&i.ProductID, &i.Available, &i.PriceCents)
    return i, err
}
```

It's exactly the code you'd write by hand, without the typos. (We wrapped one line to fit the page.) The nullable `cancel_reason` column became `CancelReason *string` in the `Order` model, thanks to `emit_pointers_for_null_types`.

Using it

Calling a query is a normal method call with typed arguments:

GO

```
q := db.New(pool)
p, err := q.CreateProduct(ctx, db.CreateProductParams{
    ID: uuid.New(), Sku: "MUG-BLUE-01", Name: "Blue mug", PriceCents: 1299,
})
```

With `emit_interface: true`, sqlc also generates a `Querier` interface listing every method. That interface is what lets us unit-test business logic with a fake, as you'll see in the transactions section.

> **The Go way**
>
> sqlc reflects a common Go attitude: generate boring code at build time rather than doing clever things at runtime. There's no reflection, no proxy and no query language to learn. If you can read SQL and Go, you can debug it.

Other approaches: sqlx, GORM and ent

sqlc is the recommendation, but you'll meet the alternatives in real codebases, so let's look at each one fairly.

sqlx, briefly

sqlx extends `database/sql` with struct scanning and named parameters. You still write SQL by hand, but scanning uses `db` struct tags, a bit like Spring's `BeanPropertyRowMapper`:

GO sqlxdemo/main.go

```
var cheap []Product // SelectContext scans every row into the slice
err = db.SelectContext(ctx, &cheap,
    `SELECT * FROM products WHERE price_cents < $1`, 2000)

var tee Product // GetContext scans one row (sql.ErrNoRows if none)
err = db.GetContext(ctx, &tee,
    `SELECT * FROM products WHERE sku = $1`, "TEE-BLACK-M")
```

It's a nice upgrade for existing `database/sql` code. Be aware that it has seen few changes since its v1.4.0 release in 2024: it's stable rather than actively developed. For new Postgres code, pgx's `CollectRows` or sqlc cover the same ground.

GORM vs Hibernate and JPA, honestly

GORM is Go's most popular ORM (object-relational mapper: a library that maps structs to tables and writes the SQL for you). A GORM model will look familiar:

GO gormdemo/main.go

```go
type Order struct {
    ID         string `gorm:"primaryKey"`
    CustomerID string
    Status     string
    Items      []OrderItem // has-many, linked by OrderItem.OrderID
    CreatedAt  time.Time   // filled in by GORM on create
}
```

What feels familiar: struct tags instead of annotations, associations, `AutoMigrate`, hooks, soft deletes, and `db.Create(&order)` saving the items too. What's **missing** matters more, because JPA habits will mislead you:

- **No persistence context.** There's no first-level cache and no "managed entity". Each call is a fresh query.
- **No dirty checking.** Changing a field does nothing until you call `Save` or `Update`.
- **No lazy-loading proxies.** An association you didn't ask for is an empty slice, not a proxy that loads on access.

The demo program prints every SQL statement GORM runs, so you can see these differences:

TERMINAL

```
go run ./gormdemo
```

OUTPUT

```
== No lazy loading: forgetting Preload gives an empty slice
  SQL: SELECT * FROM `orders` WHERE id = "o1" ORDER BY `orders`.`id` LIMIT 1
  order o1 has 0 items (really 1!)

== The N+1 trap: loading items in a loop
  SQL: SELECT * FROM `orders`
  SQL: SELECT * FROM `order_items` WHERE order_id = "o1"
  SQL: SELECT * FROM `order_items` WHERE order_id = "o2"
  SQL: SELECT * FROM `order_items` WHERE order_id = "o3"
  3 orders loaded with 4 queries

== The fix: Preload batches the children into one IN query
  SQL: SELECT * FROM `orders`
  SQL: SELECT * FROM `order_items` WHERE `order_items`.`order_id`
       IN ("o1","o2","o3")
  3 orders loaded with 2 queries
```

(We wrapped the long `IN` line to fit the page.) The **N+1 problem** means one query for the parents plus one more query per parent. In Hibernate it sneaks in through lazy loading. In GORM it appears when you notice the empty slice and "fix" it with a loop. `Preload("Items")` is the answer, like a JPA `JOIN FETCH` or an entity graph.

The last two sections of the demo show the missing dirty checking and a classic gotcha:

OUTPUT

```
== No dirty checking: changing a field saves nothing
  SQL: SELECT * FROM `orders` WHERE id = "o1" ORDER BY `orders`.`id` LIMIT 1
  SQL: SELECT * FROM `orders` WHERE id = "o1" ORDER BY `orders`.`id` LIMIT 1
  status in database: PENDING
  SQL: UPDATE `orders` SET `status`="SHIPPED" WHERE `id` = "o1"

== Gotcha: Updates with a struct skips zero values
  SQL: (none, GORM skipped the update)
  SQL: UPDATE `order_items` SET `quantity`=0 WHERE `id` = 1
```

`Updates(OrderItem{Quantity: 0})` silently does nothing, because GORM can't tell "set to zero" from "not set" in a struct. Use a map (or `Select`) when zero is a real value.

> 💡 **When GORM is fine**
>
> GORM is a reasonable choice for CRUD-heavy internal tools, admin back-offices and prototypes with simple models. For transactional core logic like reserving stock, prefer sqlc: you'll want to see and review every statement, and you'll never be surprised by what an ORM decided to do.

ent, briefly

ent (created at Facebook, now maintained by Ariga, the team behind Atlas) takes a different route: you describe your schema *as Go code*, and ent generates a fully typed client, including graph-style traversals across relationships:

GO entschema/order.go

```go
func (Order) Fields() []ent.Field {
    return []ent.Field{
        field.UUID("id", uuid.UUID{}).Default(uuid.New),
        field.UUID("customer_id", uuid.UUID{}),
        field.Enum("status").
            Values("PENDING", "CONFIRMED", "SHIPPED", "CANCELLED").
            Default("PENDING"),
        field.Time("created_at").Default(time.Now).Immutable(),
    }
}

func (Order) Edges() []ent.Edge {
    return []ent.Edge{
        edge.To("items", OrderItem.Type),
    }
}
```

It suits large domains with many relationships, where you'd otherwise hand-write lots of joins. The cost is a big generated client and an extra abstraction between you and SQL.

Choosing an approach

Approach	You write	Closest Java tool	Pick it when
`database/sql`	SQL, manual `Scan`	JDBC	libraries, several databases
pgx directly	SQL, `CollectRows`	`JdbcTemplate`	a few Postgres queries
sqlc	SQL; Go is generated	jOOQ codegen, MyBatis	**the default for services**
sqlx	SQL, struct tags	`BeanPropertyRowMapper`	existing `database/sql` code
GORM	Go method chains	Hibernate / JPA (lite)	CRUD tools, simple models
ent	schema as Go code	JPA entities + codegen	large entity graphs

Transactions without @Transactional

In Spring, transactions are declarative. Here's the core Orderly flow, placing an order and reserving stock, in idiomatic Spring Boot:

JAVA

```java
public interface StockRepository extends JpaRepository<Stock, UUID> {
    @Lock(LockModeType.PESSIMISTIC_WRITE)        // SELECT ... FOR UPDATE
    @Query("select s from Stock s where s.productId = :id")
    Optional<Stock> findByIdForUpdate(@Param("id") UUID id);
}

@Service
public class OrderService {
    // constructor injection of stockRepo and orderRepo omitted

    @Transactional
    public Order placeOrder(UUID customerId, List<ItemRequest> items) {
        for (ItemRequest item : items) {
            Stock stock = stockRepo.findByIdForUpdate(item.productId())
                    .orElseThrow(ProductNotFoundException::new);
            stock.reserve(item.quantity()); // dirty checking saves this
        }
        return orderRepo.save(Order.confirmed(customerId, items));
    }
}
```

That `@Transactional` works through a **proxy**: Spring wraps your bean, starts a transaction before the method and commits or rolls back after it. It's elegant, and it has well-known traps: calling a `@Transactional` method from the same class skips the proxy, and checked exceptions don't trigger a rollback by default.

Go has no proxies and no annotations. A transaction is a **value** (`pgx.Tx`) that you begin, use and finish. This feels like a step backwards at first, and every Java developer misses `@Transactional` for a week or two. Then you notice that you can see exactly where each transaction starts and ends, and that the self-invocation trap can't happen.

> 🧩 **Think of it like a shopping basket at the checkout**
>
> A transaction is a basket. You put things in (inserts and updates), and at the till you either pay for everything (commit) or put everything back on the shelves (rollback). Nobody else sees half a basket.

Begin, Commit, Rollback, and a WithTx helper

You could write `Begin`, `Commit` and `Rollback` by hand in every service method, but you'd repeat the error handling each time. Instead, write a small helper once. It plays the role of `TransactionTemplate.execute` in Spring:

GO orderly/store.go

```go
// WithTxOptions is WithTx with an explicit isolation level or mode.
func (s *Store) WithTxOptions(ctx context.Context, opts pgx.TxOptions,
    fn func(q db.Querier) error) error {
    tx, err := s.pool.BeginTx(ctx, opts)
    if err != nil {
        return fmt.Errorf("begin tx: %w", err)
    }
    // Rollback after a successful Commit is a harmless no-op, so we can
    // always defer it. This is our "finally" block.
    defer func() { _ = tx.Rollback(ctx) }()

    if err := fn(db.New(tx)); err != nil {
        return err
    }
    if err := tx.Commit(ctx); err != nil {
        return fmt.Errorf("commit tx: %w", err)
    }
    return nil
}
```

`WithTx(ctx, fn)` calls this with default options. Here's why the shape works:

- **`defer tx.Rollback`** runs however the function exits: an error, an early return or even a panic. After a successful `Commit` it does nothing. This one line replaces the `try/catch/finally` you'd write in Java.
- **`fn` receives `db.New(tx)`**, a `Queries` bound to the transaction. Every query inside the callback runs in the same transaction; queries outside it can't accidentally join.
- **The callback takes the `db.Querier` interface**, which `*db.Queries` satisfies. Production passes the real thing; tests pass a fake.

> 📝 **pgx has a built-in helper too**
>
> `pgx.BeginFunc(ctx, pool, func(tx pgx.Tx) error {...})` does the same begin, commit-or-rollback dance. Our helper adds the sqlc `Queries` wrapping and options, which is why many teams write their own small version.

Passing transactions through repositories

What if a repository should work both inside and outside a transaction? In Spring, the transaction is attached to the current thread, so repositories find it by magic. In Go, you pass it in. sqlc's `DBTX` interface makes that painless, because both the pool and a transaction implement it:

GO orderly/repo.go

```go
// NewOrderRepo accepts db.DBTX, so you can pass the pool (auto-commit)
// or a pgx.Tx (part of a bigger transaction). The repo doesn't care.
func NewOrderRepo(conn db.DBTX) *OrderRepo {
    return &OrderRepo{q: db.New(conn)}
}
```

So `NewOrderRepo(pool)` gives you auto-commit reads, and `NewOrderRepo(tx)` gives you a repository that takes part in your transaction. No thread-locals, no surprises.

Reserving stock with SELECT ... FOR UPDATE

Now the important part. If two customers try to buy the last blue mug at the same moment, both could read "1 available", both could subtract one, and you'd sell a mug you don't have. `SELECT ... FOR UPDATE` locks the row until the transaction ends, so the second transaction waits for the first.

> 🧩 **Think of it like the last item on the shelf**
>
> `FOR UPDATE` is putting your hand on the last mug while you check your wallet. Other shoppers can see it, but they have to wait until you either buy it or let go.

Here's the service. It depends on a tiny `TxRunner` interface rather than on `*Store`, so the business rules don't know about pgx:

GO orderly/place_order.go

```go
func (s *Service) PlaceOrder(ctx context.Context, customerID uuid.UUID,
    items []Item) (Order, error) {
    if len(items) == 0 {
        return Order{}, ErrEmptyOrder
    }
    // Lock rows in one fixed order (by product ID). Two orders for the
    // same products then queue up instead of deadlocking.
    items = slices.Clone(items)
    slices.SortFunc(items, func(a, b Item) int {
        return bytes.Compare(a.ProductID[:], b.ProductID[:])
    })

    var order Order
    err := s.tx.WithTx(ctx, func(q db.Querier) error {
        lines, err := reserve(ctx, q, items)
        if err != nil {
            return err
        }
        order, err = s.saveOrder(ctx, q, customerID, lines)
        return err
    })
```

```
    return order, err
}
```

Sorting before locking matters. If order A locks the mug then the shirt while order B locks the shirt then the mug, each waits for the other forever: a **deadlock**. Postgres detects it and kills one transaction, but it's better never to cause it. A consistent lock order prevents it.

The reservation itself checks availability under the lock:

GO orderly/place_order.go

```go
func reserve(ctx context.Context, q db.Querier,
    items []Item) ([]OrderItem, error) {
    lines := make([]OrderItem, 0, len(items))
    for _, it := range items {
        st, err := q.LockStock(ctx, it.ProductID) // SELECT ... FOR UPDATE
        if errors.Is(err, pgx.ErrNoRows) {
            return nil, fmt.Errorf("%s: %w", it.ProductID, ErrProductNotFound)
        }
        if err != nil {
            return nil, fmt.Errorf("lock stock: %w", err)
        }
        if st.Available < it.Quantity {
            return nil, fmt.Errorf("%s: want %d, have %d: %w", it.ProductID,
                it.Quantity, st.Available, ErrInsufficientStock)
        }
        err = q.ReserveStock(ctx, db.ReserveStockParams{
            ProductID: it.ProductID, Quantity: it.Quantity,
        })
        if err != nil {
            return nil, fmt.Errorf("reserve stock: %w", err)
        }
        lines = append(lines, OrderItem{ProductID: it.ProductID,
            Quantity: it.Quantity, UnitPriceCents: st.PriceCents})
    }
    return lines, nil
}
```

If any item is short, `reserve` returns an error wrapping `ErrInsufficientStock`, the callback returns it, and `WithTx` rolls back every reservation made so far. The caller can check `errors.Is(err, ErrInsufficientStock)` and return a 409 Conflict.

The integration test (`orderly/integration_test.go`) proves the locking works: ten goroutines race to buy five mugs against a real Postgres in Docker, and exactly five succeed while the stock ends at zero, never negative.

> 💡 **An alternative: one conditional UPDATE**
>
> For a single row you can skip the explicit lock: `UPDATE stock SET available = available - $2 WHERE product_id = $1 AND available >= $2`, then check that one row was affected. It's atomic and fast. `FOR UPDATE` shines when you need to read, decide in Go, then write, as we do here.

Testing the business rules with a fake

Because the service only needs a `TxRunner` and a `db.Querier`, a unit test can use an in-memory fake. Here's a handy Go trick for faking a big interface: **embed the interface** in your fake struct and implement only the methods the test needs. Any other method would panic if called, which is what you want in a focused test.

GO orderly/fakes_test.go

```go
// It embeds the db.Querier interface: methods we don't override would
// panic if called, which is exactly what we want in a focused unit test.
type fakeDB struct {
    db.Querier
    stock  map[uuid.UUID]db.Stock
    prices map[uuid.UUID]int64
    orders map[uuid.UUID]db.Order
    items  []db.OrderItem
    locks  []uuid.UUID // the order in which rows were locked

    commits, rollbacks int
}

// WithTx makes the fake behave like a transaction: it snapshots the
// state and restores it if fn fails.
func (f *fakeDB) WithTx(ctx context.Context,
    fn func(q db.Querier) error) error {
    stock, orders := maps.Clone(f.stock), maps.Clone(f.orders)
    items := len(f.items)
    if err := fn(f); err != nil {
        f.stock, f.orders, f.items = stock, orders, f.items[:items]
        f.rollbacks++
        return err
    }
    f.commits++
    return nil
}
```

The tests then check the rules (stock reserved, rollback on shortage, rows locked in ID order) with no database at all:

OUTPUT

```
--- PASS: TestPlaceOrder_ReservesStockAndSavesOrder (0.00s)
--- PASS: TestPlaceOrder_InsufficientStockRollsBack (0.00s)
--- PASS: TestPlaceOrder_Errors (0.00s)
PASS
ok      example.com/goguide/ch09/orderly    0.710s
```

Chapter 15 goes much deeper into fakes, testcontainers and test design.

Isolation levels

An **isolation level** controls how much concurrent transactions can see of each other's uncommitted or freshly committed work. Postgres defaults to **Read Committed**, the same default you get in most Spring Boot apps on Postgres. Our `FOR UPDATE` lock makes Read Committed safe for stock reservation.

When you need stronger guarantees, pass options instead of using an annotation attribute:

GO

```go
err := store.WithTxOptions(ctx,
    pgx.TxOptions{IsoLevel: pgx.Serializable}, func(q db.Querier) error {
        // ... reads and writes that must behave as if run one at a time
        return nil
    })
```

Level	In plain words	Use it for
Read Committed	each statement sees committed data	almost everything (default)
Repeatable Read	the whole transaction sees one snapshot	consistent reports
Serializable	as if transactions ran one by one	complex invariants

With Serializable, Postgres may abort a transaction with a **serialization failure** (SQLSTATE `40001`) instead of blocking. The right response is to retry the whole transaction. Spring doesn't do that for you either; in Go you write a small loop around `WithTxOptions` that retries while `IsSerializationFailure(err)` is true:

GO orderly/pgx_direct.go

```go
func IsSerializationFailure(err error) bool {
    var pgErr *pgconn.PgError
    return errors.As(err, &pgErr) && pgErr.Code == "40001"
}
```

Migrations with goose

A **migration** is a versioned script that changes the schema, applied in order and recorded in a table so it runs once. In Spring you'd use Flyway (`V1__init.sql` in `db/migration`) or Liquibase changelogs, and Spring Boot runs them at startup. In Go, the house pick is **goose**.

> 🧩 **Think of it like a ship's logbook**
>
> Migrations are numbered log entries. You only ever add new entries at the end; you never rewrite yesterday's page. The database remembers which page it has reached and reads only the new ones.

A goose migration is a SQL file with `Up` and `Down` sections:

SQL orderly/migrations/00001_products_and_stock.sql

```sql
-- +goose Up
CREATE TABLE products (
    id          uuid PRIMARY KEY,
    sku         text NOT NULL UNIQUE,
    name        text NOT NULL,
    price_cents bigint NOT NULL CHECK (price_cents >= 0),
    created_at  timestamptz NOT NULL DEFAULT now()
);
```

```
CREATE TABLE stock (
    product_id uuid PRIMARY KEY REFERENCES products (id),
    available  int NOT NULL CHECK (available >= 0),
    reserved   int NOT NULL DEFAULT 0 CHECK (reserved >= 0)
);

-- +goose Down
DROP TABLE stock;
DROP TABLE products;
```

The goose CLI creates new files with the next number (`-s` asks for sequential numbers instead of timestamps):

TERMINAL

```
go run github.com/pressly/goose/v3/cmd/goose@v3.27.0 \
    -dir migrations -s create add_order_notes sql
```

OUTPUT

```
2026/11/02 23:27:27 Created new file: 00001_add_order_notes.sql
```

(That run was in an empty folder, hence `00001`.)

Embedding migrations and running them at startup

Go's `embed` package compiles files into the binary, the way resources end up inside a Spring Boot fat jar. Two lines turn the migrations folder into a filesystem value:

GO orderly/migrations/embed.go

```
package migrations

import "embed"

// FS contains every *.sql file in this folder.
//
//go:embed *.sql
var FS embed.FS
```

Running them at startup is a short function. goose works with `database/sql`, so we wrap our pgx pool with `stdlib.OpenDBFromPool`:

GO orderly/migrate.go

```
func Migrate(ctx context.Context, pool *pgxpool.Pool) error {
    sqlDB := stdlib.OpenDBFromPool(pool) // goose speaks database/sql
    defer sqlDB.Close()

    // A Postgres advisory lock: if three pods start at once, one migrates
    // and the others wait. goose leaves locking off unless you ask.
    locker, err := lock.NewPostgresSessionLocker()
    if err != nil {
        return fmt.Errorf("goose locker: %w", err)
    }
    provider, err := goose.NewProvider(goose.DialectPostgres, sqlDB,
```

```
        migrations.FS, goose.WithSessionLocker(locker))
    if err != nil {
        return fmt.Errorf("goose provider: %w", err)
    }
    results, err := provider.Up(ctx)
    // ... return the error, or log each applied version
```

> ⚠️ **Gotcha: concurrent startups**
>
> Flyway takes a lock by default. goose doesn't: without `WithSessionLocker`, three replicas starting together can all try to apply the same migration. Always pass a locker when migrating at startup.

At startup, or as a separate job?

Both are common:

- **At startup** (the code above) is simplest and matches Spring Boot's default. It's fine for small services and when every migration is backward compatible.
- **As a separate job** (a Kubernetes `Job`, a Helm pre-upgrade hook or a CI step) keeps schema changes out of the app's startup path and lets you run them with a more privileged database user. Use the goose CLI, or a small `migrate` command in your own binary that calls the same `Migrate` function.

Either way, write migrations that old and new versions of the app can both live with during a rolling deploy (add a column first, start using it later, drop old columns last). Chapter 18 covers deployment order.

Pool tuning and query timeouts

Tuning pgxpool like HikariCP

Every HikariCP setting you know has a pgxpool cousin. Here's the Spring version first:

YAML application.yml

```yaml
spring:
  datasource:
    hikari:
      maximum-pool-size: 20
      minimum-idle: 2
      max-lifetime: 1800000     # 30 minutes, in ms
      idle-timeout: 300000      # 5 minutes
      connection-timeout: 5000
```

And the Go version, where the settings are plain struct fields:

GO orderly/pool.go

```
func PoolConfig(url string) (*pgxpool.Config, error) {
    cfg, err := pgxpool.ParseConfig(url)
    if err != nil {
        return nil, fmt.Errorf("parse database url: %w", err)
    }
    cfg.MaxConns = 20                       // HikariCP maximumPoolSize
    cfg.MinConns = 2                        // keep 2 open even when idle
```

```
    cfg.MaxConnLifetime = 30 * time.Minute   // HikariCP maxLifetime
    cfg.MaxConnLifetimeJitter = time.Minute  // avoid mass reconnects
    cfg.MaxConnIdleTime = 5 * time.Minute    // HikariCP idleTimeout
    cfg.HealthCheckPeriod = 30 * time.Second // background idle checks
    cfg.ConnConfig.ConnectTimeout = 5 * time.Second

    // A server-side safety net: Postgres cancels any statement that runs
    // longer than this, even if a caller forgot a context deadline.
    cfg.ConnConfig.RuntimeParams["statement_timeout"] = "10s"
    cfg.ConnConfig.RuntimeParams["application_name"] = "orderly"
    return cfg, nil
}
```

Setting	pgxpool default	Advice
`MaxConns`	greater of 4 and CPU count	size for the database, not the pod
`MinConns`	0	a few warm connections help latency
`MaxConnLifetime`	1 hour	below any proxy or firewall idle cut-off
`MaxConnIdleTime`	30 minutes	shorter frees database memory
`HealthCheckPeriod`	1 minute	how often idle connections are checked

`NewPool` in the same file builds the pool and pings it once, so a wrong password fails at startup rather than on the first customer request. You can also set these values in the URL (`?pool_max_conns=20`), which is handy for environment-specific tuning.

> ⚠️ **Pool size is per pod**
>
> Ten replicas with `MaxConns = 20` can open 200 connections, and Postgres's default `max_connections` is 100. Do the multiplication before you scale out. HikariCP has the same trap; it's easier to forget when Go pods start so quickly.

Query timeouts with context

In Spring you might set `@Transactional(timeout = 5)` or a JDBC query timeout. In Go, the **context** you pass to every query is the timeout. If the incoming HTTP request already has a deadline, your queries inherit it. For a tighter limit on one step, wrap the context:

GO

```
ctx, cancel := context.WithTimeout(ctx, 2*time.Second)
defer cancel() // always release the timer
order, err := repo.Get(ctx, id)
```

When the deadline passes, pgx stops waiting: it interrupts the network read, throws that connection away and returns an error (`pgconn.Timeout(err)` reports `true`). The query itself may keep running on the server for a moment, which is exactly why the `statement_timeout` safety net above is worth having. Chapter 14 shows how one deadline flows through a whole request.

The repository pattern in Go

Java codebases often have a repository interface per entity, a service interface and an implementation of each. In Go, keep the data layer **thin** and put interfaces **where they're used**:

- **The repository is a concrete struct**, like `OrderRepo` above. Its job is mapping rows to domain types and turning driver errors into domain errors. Business rules belong in the service.
- **Interfaces belong to the consumer.** The service declares the two or three methods it needs (our `TxRunner`), not the full list the repository offers. Go's implicit interfaces make this free: `*Store` satisfies `TxRunner` without saying so.
- **sqlc's `Querier` covers the rest.** You rarely need a hand-written interface that mirrors every query.

For example, an HTTP handler that only reads orders declares exactly that:

GO

```go
// in package httpapi, the consumer
type orderGetter interface {
    Get(ctx context.Context, id uuid.UUID) (orderly.Order, error)
}
```

> 🧠 **Key idea**
>
> Accept small interfaces, return concrete structs. You get easy fakes in tests without a forest of `FooRepository` / `FooRepositoryImpl` pairs. Chapter 6 discusses package layout in more depth.

Caching

A **cache** keeps a copy of data that's expensive to fetch, so most requests skip the expensive part. Product details in Orderly are read thousands of times for every change, which makes them a perfect candidate.

Redis cache-aside vs @Cacheable

In Spring you'd add an annotation and let a proxy handle it:

JAVA

```java
@Cacheable(cacheNames = "products", key = "#id")
public Product getProduct(UUID id) {
    return productRepo.findById(id)
            .orElseThrow(() -> new ProductNotFoundException(id));
}

@CacheEvict(cacheNames = "products", key = "#product.id()")
public Product updateProduct(Product product) { ... }
```

Under the hood that's the **cache-aside** pattern: look in the cache; on a miss, load from the database and store the result with a **TTL** (time to live: how long before the entry expires). In Go you write those steps yourself with **go-redis/v9**. Here's the heart of it:

GO cache/redis.go

```go
func (c *RedisCache) GetProduct(ctx context.Context,
    id string) (Product, error) {
    b, err := c.rdb.Get(ctx, key(id)).Bytes()
    switch {
    case err == nil:
        var p Product
        if json.Unmarshal(b, &p) == nil {
            return p, nil // cache hit
        }
    case errors.Is(err, redis.Nil):
        // cache miss: the normal case, carry on to the database
    default:
        // Redis is down or slow. Serve from the database, don't fail.
        slog.WarnContext(ctx, "redis get failed", "id", id, "err", err)
    }

    p, err := c.next.GetProduct(ctx, id)
    if err != nil {
        return Product{}, err
    }
    b, _ = json.Marshal(p)
    // Up to 10% random jitter, so keys cached together expire apart.
    ttl := c.ttl + rand.N(c.ttl/10+1)
    if err := c.rdb.Set(ctx, key(id), b, ttl).Err(); err != nil {
        slog.WarnContext(ctx, "redis set failed", "id", id, "err", err)
    }
    return p, nil
}
```

A few deliberate choices here:

- `redis.Nil` **means "key not found"**. It's a normal miss, not a failure.
- **A Redis outage doesn't fail the request.** A cache is an optimisation, so we log and fall through to the database.
- **Errors aren't cached.** If the product doesn't exist, we return before `Set`.
- **The TTL has jitter.** Without it, thousands of keys written in the same second expire in the same second.

`c.next` is anything with a `GetProduct` method: the database repository, or another cache layer. Invalidation (the `@CacheEvict` part) is a one-line `Invalidate` method that calls `rdb.Del`.

> 🧩 **Think of it like a sticky note on your monitor**
>
> Cache-aside is checking the sticky note before you walk to the filing room. If the note is missing, you fetch the file, write a new note and give it an expiry date so stale notes don't live forever.

A local in-process cache vs Caffeine

Redis is a network hop away. For the hottest data, an **in-process cache** (memory inside your own Go process) is a hundred times faster. In Java you'd reach for Caffeine. In Go, the house pick is **otter v2**, whose design is based on Caffeine's, so its eviction policy and API ideas will feel familiar. (ristretto from Dgraph is the other well-known option.)

JAVA

```java
Cache<UUID, Product> products = Caffeine.newBuilder()
        .maximumSize(10_000)
        .expireAfterWrite(Duration.ofSeconds(30))
        .build();
```

GO cache/local.go

```go
func NewLocalCache(size int, ttl time.Duration,
    next ProductSource) *LocalCache {
    c := otter.Must(&otter.Options[string, Product]{
        MaximumSize:      size, // evict beyond this many entries
        ExpiryCalculator: otter.ExpiryWriting[string, Product](ttl),
    })
    return &LocalCache{cache: c, next: next}
}

func (l *LocalCache) GetProduct(ctx context.Context,
    id string) (Product, error) {
    if p, ok := l.cache.GetIfPresent(id); ok {
        return p, nil
    }
    p, err := l.next.GetProduct(ctx, id)
    if err != nil {
        return Product{}, err
    }
    l.cache.Set(id, p)
    return p, nil
}
```

Each pod has its own local copy, and an eviction in Redis doesn't reach it. Keep local TTLs short (seconds, not minutes) so a pod can never be stale for long.

Preventing cache stampedes with singleflight

A **cache stampede** happens when a popular key expires and hundreds of concurrent requests all miss at once, all hit the database for the same row, and the database falls over. `golang.org/x/sync/singleflight` fixes this within a process: concurrent calls with the same key share one execution.

> 🧩 **Think of it like asking the manager**
>
> Twenty customers in a queue want to know if the blue mug is back in stock. Instead of twenty people knocking on the manager's door, one person asks and the other nineteen wait for the answer.

GO cache/dedup.go

```go
func (d *Dedup) GetProduct(ctx context.Context, id string) (Product, error) {
    v, err, _ := d.group.Do(id, func() (any, error) {
        // Many callers wait on this one load, so don't let the first
        // caller's cancellation fail everyone. Use our own timeout.
        ctx, cancel := context.WithTimeout(context.WithoutCancel(ctx),
            d.timeout)
        defer cancel()
        return d.next.GetProduct(ctx, id)
    })
    if err != nil {
        return Product{}, err
    }
    return v.(Product), nil
}
```

`group.Do(key, fn)` runs `fn` once per key at a time; anyone else calling with the same key waits and gets the same result. Spring's closest equivalent is `@Cacheable(sync = true)`.

> ⚠️ **Gotcha: one caller's context for everyone**
>
> Inside `Do`, the function runs with the *first* caller's context. If that caller gives up, everyone waiting gets a "context canceled" error. That's why we detach with `context.WithoutCancel` and set our own timeout.

Putting the layers together

Because every layer has the same `GetProduct` method, they stack like decorators, the same idea as Spring's proxy chain but written out where you can see it:

GO cache/cmd/cachedemo/main.go

```go
db := &slowDB{}
redisLayer := cache.NewRedisCache(rdb, 10*time.Minute, db)
products := cache.NewLocalCache(10_000, 30*time.Second,
    cache.NewDedup(redisLayer))
```

The demo uses a fake database with a 50 ms query. If `REDIS_ADDR` isn't set, it starts an in-memory Redis (miniredis), so it runs anywhere:

TERMINAL

```
go run ./cache/cmd/cachedemo
```

OUTPUT

```
first call (cold)        MUG-BLUE-01 in 53ms   (db loads so far: 1)
second call (local hit)  MUG-BLUE-01 in 0s     (db loads so far: 1)
another pod (Redis hit)  MUG-BLUE-01 in 1ms    (db loads so far: 1)
100 concurrent calls for p2: 1 database load(s)
```

One database load for the whole run, including a hundred concurrent requests for a cold key. That's a small win worth celebrating.

> 💡 **Tune the Redis client for a cache**
>
> A cache must be fast or be skipped. Set short timeouts on the client, for example `DialTimeout: 500 * time.Millisecond` and `ReadTimeout: 200 * time.Millisecond` in `redis.Options`, so a sick Redis costs you milliseconds, not seconds. The demo's `main.go` shows the full options.

MongoDB, briefly

Some teams use MongoDB for document-shaped data. The official Go driver (v2, `go.mongodb.org/mongo-driver/v2`) is mature and well documented. An order with its items fits naturally into one document:

GO mongo/orders.go

```go
type Order struct {
    ID         string      `bson:"_id"`
    CustomerID string      `bson:"customer_id"`
    Status     string      `bson:"status"`
    Items      []OrderItem `bson:"items"`
    CreatedAt  time.Time   `bson:"created_at"`
}

func (r *OrderRepo) Get(ctx context.Context, id string) (Order, error) {
    var o Order
    err := r.coll.FindOne(ctx, bson.D{{Key: "_id", Value: id}}).Decode(&o)
    if errors.Is(err, mongo.ErrNoDocuments) {
        return Order{}, ErrOrderNotFound
    }
    return o, err
}
```

The patterns carry over from SQL: a `mongo.Client` is a pool you create once (`mongo.Connect(options.Client().ApplyURI(uri))`), every call takes a context, and "not found" (`mongo.ErrNoDocuments`) becomes a domain error. Filters are `bson.D` values, which play the role of Spring Data's `Criteria` objects. The example folder also shows listing with sort and limit, a `$set` update and an integration test with testcontainers.

Recap

- `*sql.DB` and `*pgxpool.Pool` are **pools**: create one at startup, share it, and ping it so bad config fails fast.
- In `database/sql`, always use the `...Context` methods, `defer rows.Close()` and check `rows.Err()`; turn `ErrNoRows` into a domain error.
- Model NULL with `sql.Null[T]` or pointers, never a plain `string`.
- For Postgres, use **pgx/v5 + pgxpool + sqlc**: you write SQL, and the compiler checks the Go that calls it.

- GORM has no persistence context, dirty checking or lazy loading; use `Preload` to avoid N+1, and save the ORM for simple CRUD.
- Transactions are values: a `WithTx` helper with `defer tx.Rollback` replaces `@Transactional`, and `DBTX` lets repositories join a transaction.
- Reserve stock with `SELECT ... FOR UPDATE`, locking rows in a consistent order to avoid deadlocks.
- Migrate with goose (embedded files, with a session locker), tune the pool like HikariCP, and layer caches (otter, Redis, `singleflight`) behind one small interface.

Try it yourself

> 🏋 **Exercise 1: Cancel an order**
>
> Add a `CancelOrder` method to the service that, in one transaction, calls the generated `CancelOrder` query and releases the reserved stock for every item (you'll need a new `ReleaseStock` query in `queries/orders.sql`, then `sqlc generate`). Return `ErrOrderNotFound` when no row was updated. *Hint:* `:execrows` gives you the affected row count. Extend `fakeDB` with the two new methods and write the unit test first.

> 🏋 **Exercise 2: A migration for order notes**
>
> Add a goose migration `00003_order_notes.sql` that adds a nullable `notes text` column to `orders`. Regenerate the code and check the type sqlc picked for `Notes`. *Hint:* with `emit_pointers_for_null_types`, you should see `*string`.

> 🏋 **Exercise 3: Negative caching**
>
> Right now, repeated requests for a product that doesn't exist always reach the database. Change `RedisCache` to remember "not found" for 30 seconds, and add a test with miniredis. *Hint:* store a special marker value, and check for it before unmarshalling.

> 🏋 **Exercise 4: Retry serialization failures (stretch)**
>
> Write `WithSerializableRetry(ctx, store, attempts, fn)` that runs `fn` in a Serializable transaction and retries up to `attempts` times while `IsSerializationFailure(err)` is true, with a short random backoff. *Hint:* the integration test setup in `orderly/integration_test.go` is a good place to provoke real conflicts with two goroutines.

Further reading

- Go: Accessing relational databases: the official tutorial and guides for `database/sql`.
- pgx documentation: the driver, `pgxpool` and helpers like `CollectRows`.
- sqlc documentation: configuration, query annotations and type overrides.

- goose: migrations, embedding and the Provider API.
- GORM guides: read the sections on Preload and Updates before using it in anger.
- go-redis documentation: client options, pipelines and caching patterns.
- otter: the in-process cache inspired by Caffeine.
- singleflight: the package docs are short and worth reading in full.

CHAPTER 10

Configuration, Secrets and Feature Flags

> *Good configuration is invisible when it works and a 3 a.m. outage when it doesn't, so let's make it boring, typed and checked at startup.*

🎯 **What you'll learn**

- Load environment variables into typed structs with **caarlos0/env**, the Go cousin of `@ConfigurationProperties`
- Validate everything at startup and fail fast with clear, secret-free error messages
- Replace Spring profiles with env files, Kubernetes overlays and layered **koanf** config
- Know when Viper is fine and which of its behaviours will surprise you
- Keep secrets out of git and out of logs, using Kubernetes, Vault or a cloud secret manager
- Ship features safely behind flags with the **OpenFeature** Go SDK, and test them

The big picture

In Spring Boot, configuration feels like magic. You drop values in `application.yml`, add a profile file for production, and `@ConfigurationProperties` or `@Value` binds them into your beans. Environment variables override files through relaxed binding, and Spring Cloud Config can even refresh values at runtime.

Go has no framework doing that binding for you, and that turns out to be a good thing. Configuration becomes a plain struct that you load once in `main`, validate, and pass to the parts of your program that need it. There's no hidden global lookup, so you can always answer "where does this value come from?" by reading code.

🧩 **Think of it like a pilot's pre-flight checklist**

A pilot doesn't discover a missing fuel cap at 30,000 feet. Every item is checked on the ground, and if anything is wrong, the plane doesn't take off. Loading config into a struct and validating it at startup is your pre-flight check: a bad setting stops the pod before it ever takes traffic.

Concept	Java / Spring	Go
Typed config	`@ConfigurationProperties`	struct + **caarlos0/env**
Single value	`@Value("${x:default}")`	a struct field with `envDefault`
Validation	`@Validated` + Bean Validation	go-playground/validator, plain checks
Profiles	`application-prod.yml`	env files, Kubernetes overlays, koanf layers

Concept	Java / Spring	Go
CLI arguments	Spring Shell, picocli	`flag` package (cobra for big CLIs)
Secrets	Spring Cloud Vault, configtree	env vars or files from Kubernetes, cloud SDKs
Runtime refresh	`@RefreshScope`	rarely; `slog.LevelVar`, koanf watch
Feature flags	Unleash, LaunchDarkly, Togglz	**OpenFeature** Go SDK

Full code: `examples/ch10-config/`. Everything there runs without Docker.

Twelve-factor config in one minute

The **Twelve-Factor App** is a set of principles for building cloud services, and factor III says: *store config in the environment*. Config is everything that changes between deploys (URLs, credentials, pool sizes, feature switches). Code is everything that doesn't.

Following it gives you three practical benefits:

- **One artifact for every environment.** The same container image goes from dev to staging to prod; only the environment changes.
- **No secrets in the repository.** Credentials arrive at runtime from the platform.
- **Language-neutral operations.** Kubernetes, Docker Compose and your CI pipeline all speak environment variables.

Spring Boot already honours this through relaxed binding (`ORDERLY_DB_URL` overrides `orderly.db.url`). In Go we'll make environment variables the main source, and add files only where they earn their keep.

Environment variables into structs with caarlos0/env

The Spring version you know

Here's typed, validated configuration in Spring Boot 3, using a record:

JAVA

```java
@Validated
@ConfigurationProperties(prefix = "orderly")
public record OrderlyProperties(
        @Pattern(regexp = "dev|staging|prod") @DefaultValue("dev") String env,
        @Valid Http http,
        @Valid Db db) {

    public record Http(@Min(1) @Max(65535) @DefaultValue("8080") int port,
                       @DefaultValue("5s") Duration readTimeout) {}

    public record Db(@NotBlank String url,
                     @Min(1) @Max(200) @DefaultValue("20") int maxConns) {}
}
```

Spring binds `ORDERLY_DB_URL` to `orderly.db.url`, applies defaults, and refuses to start if validation fails. That's a great experience, and we want the same one in Go.

The Go version

caarlos0/env reads environment variables into a struct using struct tags (small annotations on fields, written in backticks). It's the house pick because it does one job well: defaults, required fields, nested prefixes, slices, durations and file-based secrets, with no global state.

GO config/config.go

```go
// Config is the whole service configuration. Every variable name gets the
// ORDERLY_ prefix, so Env comes from ORDERLY_ENV (see Load).
type Config struct {
    Env string `env:"ENV" envDefault:"dev" validate:"oneof=dev staging prod"`

    LogLevel slog.Level `env:"LOG_LEVEL" envDefault:"info"`

    // Nested structs add a prefix: DB.URL comes from ORDERLY_DB_URL.
    HTTP     HTTPConfig     `envPrefix:"HTTP_"`
    DB       DBConfig       `envPrefix:"DB_"`
    Kafka    KafkaConfig    `envPrefix:"KAFKA_"`
    Payments PaymentsConfig `envPrefix:"PAYMENTS_"`
}
```

The nested structs hold each area's settings:

GO config/config.go

```go
type HTTPConfig struct {
    Port int `env:"PORT" envDefault:"8080" validate:"min=1,max=65535"`

    ReadTimeout     time.Duration `env:"READ_TIMEOUT" envDefault:"5s"`
    WriteTimeout    time.Duration `env:"WRITE_TIMEOUT" envDefault:"10s"`
    ShutdownTimeout time.Duration `env:"SHUTDOWN_TIMEOUT" envDefault:"20s"`
}

type DBConfig struct {
    URL      Secret `env:"URL,required,notEmpty" validate:"url"`
    MaxConns int32  `env:"MAX_CONNS" envDefault:"20" validate:"min=1,max=200"`
}

type KafkaConfig struct {
    // A comma-separated list: ORDERLY_KAFKA_BROKERS=k1:9092,k2:9092
    Brokers []string `env:"BROKERS,required" validate:"dive,hostname_port"`
    Topic   string   `env:"TOPIC" envDefault:"orderly.orders"`
}
```

Reading the tags from left to right:

- `env:"PORT"` names the variable. With the `HTTP_` prefix from the parent and the global `ORDERLY_` prefix, the full name is `ORDERLY_HTTP_PORT`.
- `envDefault:"8080"` is the fallback, like `${orderly.http.port:8080}`.
- `required` fails if the variable is missing; `notEmpty` also fails if it's set to an empty string.

- **Types are parsed for you.** `time.Duration` accepts `5s` or `1m30s`, `[]string` splits on commas, and `slog.Level` accepts `debug` or `warn` because it implements `encoding.TextUnmarshaler`.
- `validate:"..."` is for go-playground/validator, which we'll run right after loading.

Loading it

Loading is a single generic call. We wrap it in a function that also accepts a map of variables, so tests never depend on the machine they run on:

GO config/load.go

```go
// LoadFrom reads the given variables instead (nil means the real
// environment), which keeps tests independent of the machine.
func LoadFrom(environ map[string]string) (Config, error) {
    cfg, err := env.ParseAsWithOptions[Config](env.Options{
        Prefix:      "ORDERLY_",
        Environment: environ,
    })
    var agg env.AggregateError
    if errors.As(err, &agg) { // env reports every missing variable at once
        return Config{}, problems(agg.Errors...)
    }
    if err != nil {
        return Config{}, fmt.Errorf("config: %w", err)
    }
    if err := check(cfg); err != nil {
        return Config{}, err
    }
    return cfg, nil
}
```

`Load()` is `LoadFrom(nil)`. A nice detail: caarlos0/env collects *every* problem before returning, so an operator fixes all missing variables in one go instead of one per restart.

> **Coming from Java**
>
> There's no `@Value` sprinkled through your services. In Go, a function receives the config values it needs as parameters, so there's exactly one place that reads the environment. That's the same discipline Spring's docs recommend with constructor-bound `@ConfigurationProperties`, made mandatory.

Fail fast with validation at startup

Struct tags catch a lot, but some rules span several fields, such as "production needs a payments API key". We run both kinds in one `check` function and report problems without ever printing values, because values might be secrets:

GO config/load.go

```go
func check(cfg Config) error {
    var errs []error
    var verrs validator.ValidationErrors
    if err := validate.Struct(cfg); errors.As(err, &verrs) {
```

```
        for _, fe := range verrs {
            // Never print the value: it might be a secret.
            errs = append(errs, fmt.Errorf("%s fails %q",
                strings.TrimPrefix(fe.Namespace(), "Config."), ruleOf(fe)))
        }
    } else if err != nil {
        return err
    }
    if cfg.Env == "prod" && cfg.Payments.APIKey == "" {
        errs = append(errs, errors.New("Payments.APIKey is required in prod"))
    }
    if len(errs) > 0 {
        return problems(errs...)
    }
    return nil
}
```

Then `main` loads once and exits with a clear message if anything is wrong:

GO cmd/orderly/main.go

```
func main() {
    cfg, err := config.Load()
    if err != nil {
        // Fail fast: a clear message and a non-zero exit code. Kubernetes
        // shows this in the pod logs and won't route traffic to us.
        fmt.Fprintln(os.Stderr, err)
        os.Exit(1)
    }
    // ... build the logger, pool, Kafka client and server from cfg
```

Here's what an operator sees when two required variables are missing:

TERMINAL

```
go run ./cmd/orderly
```

OUTPUT

```
config: 2 problem(s):
  - required environment variable "ORDERLY_DB_URL" is not set
  - required environment variable "ORDERLY_KAFKA_BROKERS" is not set
```

And with values that are present but wrong (a typo in the port, a broker without a port, and production without its API key):

OUTPUT

```
config: 3 problem(s):
  - HTTP.Port fails "max=65535"
  - Kafka.Brokers[0] fails "hostname_port"
  - Payments.APIKey is required in prod
```

That's a small win worth celebrating: a bad deploy now fails in the first second, with a message that tells you exactly what to fix, instead of failing on the first customer request.

> 💡 **Pair fail-fast config with readiness probes**
>
> When the process exits with code 1, Kubernetes keeps the old pods serving and reports `CrashLoopBackOff` for the new ones. Your rollout stops by itself. Chapter 18 covers rollouts, and Chapter 14 covers probes.

Passing config explicitly

In Spring, any bean can inject any property. In Go, resist the urge to make a global `var Cfg config.Config` that every package reads. A global is a hidden dependency: tests have to mutate it, and you can't tell what a function needs by looking at it.

Instead, load once in `main` and hand each component **only its slice** of the config:

GO cmd/orderly/main.go

```go
// newLogger needs only the level, so that's all it gets.
func newLogger(level slog.Level) *slog.Logger {
    return slog.New(slog.NewTextHandler(os.Stdout,
        &slog.HandlerOptions{Level: level}))
}

// newHTTPServer gets the HTTP slice of the config, not the whole thing.
func newHTTPServer(c config.HTTPConfig, h http.Handler) *http.Server {
    return &http.Server{
        Addr:         fmt.Sprintf(":%d", c.Port),
        Handler:      h,
        ReadTimeout:  c.ReadTimeout,
        WriteTimeout: c.WriteTimeout,
    }
}
```

A test for `newHTTPServer` builds a small `HTTPConfig` literal; it doesn't need a database URL or Kafka brokers to exist. This is the manual constructor wiring from Chapter 6, applied to configuration.

> 🐭 **The Go way**
>
> Configuration is data, not a service. Load it, validate it, pass it down. If you find yourself wanting `config.Get("db.url")` deep inside a repository, that's a sign the repository should receive the value through its constructor.

Profiles without Spring profiles

Spring profiles do two jobs: they switch configuration values (`application-prod.yml`) and they switch beans (`@Profile("prod")`). Go handles the two jobs separately, which keeps each one simple.

Switching values is the environment's job. Each environment provides its own variables:

- **Local development:** an env file per environment, loaded into your shell or by Docker Compose (`env_file:`).
- **Kubernetes:** a ConfigMap per environment, usually produced by Kustomize overlays or Helm values files.
- **Tests:** a map passed to `LoadFrom`, as you saw above.

Here's the dev file from the examples. It's safe to commit because it holds no real secrets:

PROPERTIES env/dev.env

```
# Local development settings. Safe to commit: no real secrets here.
# Load into your shell with:  set -a; . env/dev.env; set +a
ORDERLY_ENV=dev
ORDERLY_LOG_LEVEL=debug
ORDERLY_DB_URL=postgres://orderly:orderly@localhost:5432/orderly?sslmode=disable
ORDERLY_KAFKA_BROKERS=localhost:9092
```

With those loaded, the service starts and logs its config, secrets included but redacted (more on that soon). We wrapped the long first line:

OUTPUT

```
time=2026-11-02T23:52:19.783+05:30 level=INFO msg="config loaded" env=dev
  db="{URL:[REDACTED] MaxConns:20}" kafka=[localhost:9092]
time=2026-11-02T23:52:19.783+05:30 level=INFO msg="http server ready"
  addr=:8080 read_timeout=5s
```

In Kubernetes, a **Kustomize overlay** plays the role of a profile file. The base defines the Deployment; each overlay generates that environment's ConfigMap:

YAML deploy/overlays/prod/kustomization.yaml

```yaml
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization
resources:
  - ../../base
configMapGenerator:
  - name: orderly-config
    behavior: merge          # override the base's values for this env
    literals:
      - ORDERLY_ENV=prod
      - ORDERLY_LOG_LEVEL=warn
      - ORDERLY_DB_MAX_CONNS=40
      - ORDERLY_KAFKA_BROKERS=kafka-1:9092,kafka-2:9092,kafka-3:9092
```

The base Deployment pulls every key in as an environment variable with `envFrom: [{configMapRef: {name: orderly-config}}]`. Kustomize adds a content hash to the generated ConfigMap's name, so changing a value triggers a normal rolling restart.

Switching behaviour (the `@Profile` job) is plain code in `main`. If production uses Kafka and local dev uses a log-only publisher, write an `if cfg.Env == "dev"` when wiring, or better, a dedicated setting such as `ORDERLY_EVENTS_SINK=log`. Explicit settings age better than environment names.

Layered file and env config with koanf

Some teams like YAML files for the bulk of their settings, especially when there are dozens of them. **koanf** is a small, modular library that merges configuration from layers (files, env vars, flags, remote stores), with later layers winning. It maps neatly onto Spring's model:

```
base.yaml            like application.yml         (lowest priority)
    |
prod.yaml            like application-prod.yml
    |
ORDERLY_* env vars   like relaxed binding          (highest priority)
    ▼
Config struct
```

The files look exactly like you'd expect. The profile file only holds what differs:

YAML layered/config/prod.yaml

```yaml
# Like application-prod.yml: only what differs in production.
env: prod
log_level: warn
db:
  max_conns: 40
kafka:
  brokers: [kafka-1:9092, kafka-2:9092, kafka-3:9092]
```

And the loader stacks the three layers:

GO layered/layers.go

```go
func Load(dir, profile string, environ func() []string) (Config, error) {
    k := koanf.New(".")
    base := filepath.Join(dir, "base.yaml")
    if err := k.Load(file.Provider(base), yaml.Parser()); err != nil {
        return Config{}, fmt.Errorf("load %s: %w", base, err)
    }
    if profile != "" {
        path := filepath.Join(dir, profile+".yaml")
        if err := k.Load(file.Provider(path), yaml.Parser()); err != nil {
            return Config{}, fmt.Errorf("load %s: %w", path, err)
        }
    }
    err := k.Load(env.Provider(".", env.Opt{
        Prefix:        "ORDERLY_",
        TransformFunc: envKey,
        EnvironFunc:   environ, // nil means os.Environ
    }), nil)
    if err != nil {
        return Config{}, fmt.Errorf("load env: %w", err)
    }

    var cfg Config
    if err := k.Unmarshal("", &cfg); err != nil {
        return Config{}, fmt.Errorf("decode config: %w", err)
    }
```

```
    return cfg, nil
}
```

Environment variable names need a rule for nesting. YAML keys like `max_conns` already contain underscores, so we use a **double underscore** for each level:

GO layered/layers.go

```go
// envKey turns ORDERLY_DB__MAX_CONNS into db.max_conns. A double
// underscore separates levels, so keys can keep their single ones.
// A value with commas becomes a list: ORDERLY_KAFKA__BROKERS=k1,k2.
func envKey(k, v string) (string, any) {
    k = strings.ToLower(strings.TrimPrefix(k, "ORDERLY_"))
    k = strings.ReplaceAll(k, "__", ".")
    if strings.Contains(v, ",") {
        return k, strings.Split(v, ",")
    }
    return k, v
}
```

Let's watch the layers win, one after another:

TERMINAL

```
go run ./layered
go run ./layered -profile prod
ORDERLY_DB__MAX_CONNS=50 ORDERLY_KAFKA__BROKERS=k1:9092,k2:9092 \
  ORDERLY_HTTP__READ_TIMEOUT=2s go run ./layered -profile prod
```

OUTPUT

```
env=dev log_level=info port=8080 read_timeout=5s
db.max_conns=10 kafka.brokers=[localhost:9092]
env=prod log_level=warn port=8080 read_timeout=5s
db.max_conns=40 kafka.brokers=[kafka-1:9092 kafka-2:9092 kafka-3:9092]
env=prod log_level=warn port=8080 read_timeout=2s
db.max_conns=50 kafka.brokers=[k1:9092 k2:9092]
```

> 💡 **If you're unsure, use caarlos0/env**
>
> Environment variables alone cover most services, and they keep you honest about twelve-factor config. Reach for koanf when you have many settings that are easier to review as YAML, or when you need several sources merged. Either way, validate the final struct exactly as we did above.

Viper: popular, with caveats

Viper is the best-known Go config library, partly because it pairs with **cobra**, the CLI framework behind `kubectl` and `docker`. You'll meet it in many codebases, so it's worth knowing its sharp edges. The demo program shows two of them:

GO viperdemo/main.go

```go
    os.Setenv("ORDERLY_DB_URL", "postgres://from-env")

    v := viper.New() // an instance, rather than the package-level global
    v.SetConfigType("yaml")
    if err := v.ReadConfig(strings.NewReader(yamlDoc)); err != nil {
        panic(err)
    }
    v.SetEnvPrefix("ORDERLY")
    v.SetEnvKeyReplacer(strings.NewReplacer(".", "_"))
    v.AutomaticEnv()
    // ... then print some values and Unmarshal into a struct
```

OUTPUT

```
feature keys: map[expressshipping:false newcheckout:true]
Get("HTTP.PORT"): 8080
GetString("db.url"): postgres://from-env
Unmarshal -> DB.URL="" (env var ignored!)
with a default -> DB.URL="postgres://from-env"
```

What happened:

- **Keys are case-insensitive, so Viper lowercases them.** `NewCheckout` became `newcheckout`. That's harmless for most settings, but surprising for maps keyed by names, such as tenant IDs or flag names.
- **`AutomaticEnv` and `Unmarshal` don't fully cooperate.** `GetString("db.url")` finds the environment variable, but `Unmarshal` only fills keys Viper already knows from a file, a default or an explicit `BindEnv`. The classic fix is to call `SetDefault` for every key.
- **The package-level functions use global state.** `viper.GetString(...)` from anywhere is the Go equivalent of a static singleton. Prefer `viper.New()` and pass the instance (or better, the decoded struct) around.

> 📝 **When Viper is fine**
>
> For a cobra-based CLI that merges flags, a config file and environment variables, Viper is a reasonable choice and the integration saves code. For long-running services, caarlos0/env or koanf give you the same results with fewer surprises.

The flag package for CLIs

Enterprise teams write plenty of small tools: data backfills, admin commands, migration runners. Go's standard `flag` package covers most of them without any dependency. Each flag binds to a variable with a type, a default and a help text:

GO cli/parse.go

```go
func parse(args []string, stderr io.Writer) (options, error) {
    fs := flag.NewFlagSet("orderlyctl", flag.ContinueOnError)
    fs.SetOutput(stderr)
    o := options{Statuses: []string{"PENDING"}}

    fs.StringVar(&o.DSN, "dsn", "", "Postgres URL (default $ORDERLY_DB_URL)")
    fs.DurationVar(&o.Since, "since", 24*time.Hour, "how far back to look")
    fs.IntVar(&o.Batch, "batch", 500, "rows per batch")
    fs.BoolVar(&o.DryRun, "dry-run", false, "show what would happen")
    fs.Func("status", "comma-separated order statuses (default PENDING)",
        func(s string) error {
            o.Statuses = strings.Split(strings.ToUpper(s), ",")
            // ... reject anything that isn't a known status
            return nil
        })

    if err := fs.Parse(args); err != nil {
        return options{}, err
    }
    if o.DSN == "" { // read the env var after parsing, see the note below
        o.DSN = os.Getenv("ORDERLY_DB_URL")
    }
    // ... more checks: DSN required, batch within 1..10000
    return o, nil
}
```

Using our own `FlagSet` rather than the global `flag.CommandLine` makes `parse` easy to test with any argument slice. `fs.Func` lets you plug in custom parsing and validation for a flag. And you get help text for free:

TERMINAL

```
go run ./cli -h
go run ./cli -dsn postgres://localhost/orderly -since 2h -status confirmed -dry-run
```

OUTPUT

```
Usage of orderlyctl:
  -batch int
        rows per batch (default 500)
  -dry-run
        show what would happen
  -dsn string
        Postgres URL (default $ORDERLY_DB_URL)
  -since duration
        how far back to look (default 24h0m0s)
  -status value
        comma-separated order statuses (default PENDING)
dry run, would re-publish CONFIRMED orders from the last 2h0m0s (batch 500)
```

> ⚠️ **Gotcha: secrets in flag defaults**
>
> It's tempting to write `fs.StringVar(&dsn, "dsn", os.Getenv("ORDERLY_DB_URL"), ...)`. But `-h` prints every default, so the help text would show your database password to anyone looking over your shoulder or reading CI logs. Use an empty default and read the variable after parsing, as above.

For tools with many subcommands (`orderlyctl orders list`, `orderlyctl stock adjust`), cobra (`github.com/spf13/cobra`) adds subcommands, shell completion and generated docs.

Secrets

Never in git

A **secret** is any value that grants access: passwords, API keys, private keys, tokens. The rules are short:

- **Never commit secrets**, not even "temporary" ones; git history is forever. Add a secret scanner (such as gitleaks) to CI as a safety net.
- **Inject them at runtime** from the platform: Kubernetes Secrets, Vault or a cloud secret manager.
- **Never log them.** This one deserves code, because it's easy to break by accident.

A Secret type that refuses to print

Logging the whole config at startup is useful, and leaking the database password into your log platform is a security incident. A tiny named type solves both: every printing path shows `[REDACTED]`.

GO config/secret.go

```go
// Secret is a string that refuses to print itself. Logging the whole
// Config, printing it with %v or encoding it as JSON shows [REDACTED].
type Secret string

func (Secret) String() string               { return redacted }
func (Secret) GoString() string             { return redacted }
func (Secret) LogValue() slog.Value         { return slog.StringValue(redacted) }
func (Secret) MarshalText() ([]byte, error) { return []byte(redacted), nil }

// Reveal returns the real value. Call it only where the secret is used,
// such as when opening the database pool.
func (s Secret) Reveal() string { return string(s) }
```

`String` and `GoString` cover `fmt` (including nested fields), `LogValue` covers `slog`, and `MarshalText` covers JSON. A unit test in `config/config_test.go` checks all four paths. Code that really needs the value calls `cfg.DB.URL.Reveal()`, which is easy to spot in code review.

Kubernetes: env vars or mounted files

Kubernetes Secrets reach your container in one of two ways. Here's a Deployment fragment that uses both:

YAML deploy/base/deployment.yaml (fragment)

```yaml
containers:
  - name: orderly
    env:
      - name: ORDERLY_DB_URL              # 1. as an environment variable
        valueFrom:
          secretKeyRef: {name: orderly-db, key: url}
      - name: ORDERLY_PAYMENTS_API_KEY_FILE
        value: /var/run/secrets/orderly/payments-api-key
    volumeMounts:                         # 2. as a file
      - name: payments
        mountPath: /var/run/secrets/orderly
        readOnly: true
volumes:
  - name: payments
    secret:
      secretName: orderly-payments
      items:
        - {key: api-key, path: payments-api-key}
```

Mounted files are often preferred for secrets: they aren't inherited by child processes or captured by tools that dump a process's environment, and Kubernetes updates them in place when the Secret changes. caarlos0/env reads them with the `file` option. The variable holds a *path*, and the field gets the file's contents:

GO config/config.go

```go
type PaymentsConfig struct {
    BaseURL string `env:"BASE_URL" envDefault:"http://payments:9000"`

    // The file named by ORDERLY_PAYMENTS_API_KEY_FILE holds the key. This
    // is how Kubernetes mounts a Secret as a file.
    APIKey Secret `env:"API_KEY_FILE,file"`
}
```

In Spring Boot you'd get the same effect with `spring.config.import=configtree:/var/run/secrets/orderly/`.

HashiCorp Vault

If your company runs **Vault**, the simplest integration needs no Vault code in your app at all. The **Vault Agent Injector** or the **Vault Secrets Operator** writes secrets into files or Kubernetes Secrets, and your service reads them exactly as above. That's the Go equivalent of Spring Cloud Vault doing the work behind the scenes, but outside your process.

When you do want to call Vault directly (for example, for short-lived database credentials), use the official client `github.com/hashicorp/vault/api`. With rotating credentials, pgxpool's `BeforeConnect` hook lets you fetch the current password each time a new connection opens. (HashiCorp also publishes `vault-client-go`, but it's still marked beta.)

Cloud secret managers

Each cloud has an SDK for its secret store: `azsecrets` for Azure Key Vault, `secretsmanager` in aws-sdk-go-v2 for AWS Secrets Manager, and `cloud.google.com/go/secretmanager` for GCP Secret Manager. They all follow the same shape: build a credential from the workload's identity, create a client, fetch by name. Here's Azure:

GO secrets/azure.go

```go
func FetchSecret(ctx context.Context, vaultURL, name string) (string, error) {
    cred, err := azidentity.NewDefaultAzureCredential(nil)
    if err != nil {
        return "", fmt.Errorf("azure credential: %w", err)
    }
    client, err := azsecrets.NewClient(vaultURL, cred, nil)
    if err != nil {
        return "", fmt.Errorf("key vault client: %w", err)
    }
    // An empty version means "the latest one".
    resp, err := client.GetSecret(ctx, name, "", nil)
    if err != nil {
        return "", fmt.Errorf("get secret %q: %w", name, err)
    }
    return *resp.Value, nil
}
```

`DefaultAzureCredential` tries environment variables, workload identity on AKS, managed identity and finally your local `az login`, so the same code runs on a laptop and in production with no credential in your config. Call a function like this once in `main`, put the result into your `Config`, then validate as usual.

> 💡 **If you're unsure, let the platform deliver secrets**
>
> Prefer secrets as mounted files or env vars delivered by Kubernetes (fed by External Secrets Operator, the Secrets Store CSI driver or Vault's operator). Your app stays cloud-neutral and testable. Use the SDKs when you need features like rotation callbacks or per-request secret lookups.

Dynamic reload: mostly, don't

Spring Cloud's `@RefreshScope` can rebuild beans when config changes. It's clever, and it's also a common source of half-applied, hard-to-reproduce states. A connection pool can't resize safely mid-flight, and two goroutines can read the old and new values in the same request.

For most settings, the boring answer is the right one: **change the ConfigMap and do a rolling restart**. A Go service usually starts in well under a second, each pod switches over in one step, and it's exactly what Kubernetes is built for.

> ⚠️ **Gotcha: Kubernetes doesn't update env vars**
>
> Environment variables are fixed when a container starts; editing the ConfigMap changes nothing until a restart. Mounted ConfigMap and Secret files *are* updated in place (after a short delay), except when mounted with `subPath`.

A few settings are genuinely useful to change live: the **log level** (to debug a production issue for ten minutes) and **feature flags** (next section, handled by a flag provider). For the log level, Go has the perfect tool: `slog.LevelVar`, a level you can change safely while other goroutines log. koanf can watch a mounted file and update it:

GO layered/watch.go

```go
func watchLogLevel(path string, level *slog.LevelVar,
    log *slog.Logger) (*file.File, error) {
    f := file.Provider(path)
    err := f.Watch(func(_ any, err error) {
        if err != nil {
            log.Error("config watch failed", "err", err)
            return
        }
        k := koanf.New(".")
        if err := k.Load(f, yaml.Parser()); err != nil {
            log.Warn("ignoring unreadable config change", "err", err)
            return
        }
        var lvl slog.Level
        if err := lvl.UnmarshalText([]byte(k.String("log_level"))); err != nil {
            log.Warn("ignoring bad log_level", "err", err)
            return // keep the old, known-good level
        }
        level.Set(lvl) // LevelVar is safe to change while logging
        log.Info("log level changed", "level", lvl)
    })
    return f, err
}
```

Under the hood, koanf uses **fsnotify**, the standard Go library for file-change notifications. Note the defensive shape: a bad value is logged and ignored, never applied. If you ever need to reload more than one value, build a complete new `Config`, validate it, and swap it in with an `atomic.Pointer[Config]` so readers always see a consistent whole.

Feature flags with OpenFeature

A **feature flag** is a switch that turns a feature on or off at runtime, for everyone or for a targeted group, without a deploy. Flags let you merge unfinished work safely, roll out to 5% of customers first, and kill a misbehaving feature in seconds.

> 🧩 **Think of it like the light switches in a new building**
>
> The electrician wires every room (you deploy the code), but each room's lights stay off until someone flips its switch (enables the flag). If a room's wiring smokes, you flip it off again without rewiring the building.

In Java you might call the Unleash SDK (`unleash.isEnabled(...)`), LaunchDarkly (`client.boolVariation(...)`) or a Spring Cloud Config property. Each vendor has its own API, so switching vendors means touching every call site. **OpenFeature** is a CNCF standard API for feature flags with SDKs for Java, Go and many other languages. Your code talks to the OpenFeature client; a **provider** plugs in the actual vendor (flagd, LaunchDarkly, Unleash, Flagsmith and others).

The Java version, for reference:

JAVA

```java
OpenFeatureAPI api = OpenFeatureAPI.getInstance();
api.setProviderAndWait(provider);
Client client = api.getClient();

var ctx = new ImmutableContext(customer.id(),
        Map.of("tier", new Value(customer.tier())));
boolean express = client.getBooleanValue("express-shipping", false, ctx);
```

Using a flag in a service

Our checkout service depends on a one-method interface that `*openfeature.Client` already satisfies, so tests can swap in anything:

GO flags/checkout.go

```go
type FlagEvaluator interface {
    BooleanValue(ctx context.Context, flag string, defaultValue bool,
        evalCtx openfeature.EvaluationContext,
        options ...openfeature.Option) (bool, error)
}

func (c *Checkout) ShippingOptions(ctx context.Context,
    cust Customer) []string {
    // The evaluation context says who is asking, so rules can target
    // tiers, countries or a percentage of customers.
    evalCtx := openfeature.NewEvaluationContext(cust.ID, map[string]any{
        "tier":    cust.Tier,
        "country": cust.Country,
    })
    options := []string{"standard"}
    express, err := c.flags.BooleanValue(ctx, "express-shipping", false,
        evalCtx)
    if err != nil {
        // We already got the safe default (false). Note it, carry on.
        c.log.WarnContext(ctx, "flag evaluation failed",
            "flag", "express-shipping", "err", err)
    }
    if express {
        options = append(options, "express")
```

```
    }
    return options
}
```

Two OpenFeature ideas are worth noticing:

- **The default value is always returned on failure.** If the provider is down or the flag doesn't exist, you get `false` plus an error. Choose defaults that are safe when the flag system is unavailable.
- **The evaluation context** carries a **targeting key** (usually the user or customer ID, used for consistent percentage rollouts) and attributes that rules can match on.

An in-memory provider

The SDK ships an in-memory provider, which is perfect for local development, demos and tests. Here's the flag definition, with a rule that enables express shipping for gold-tier customers only:

GO flags/provider.go

```go
// goldOnly turns the flag on for gold-tier customers only.
var goldOnly = func(f memprovider.InMemoryFlag,
    ctx openfeature.FlattenedContext) (any, openfeature.ProviderResolutionDetail) {
    if ctx["tier"] == "gold" {
        return f.Variants["on"], openfeature.ProviderResolutionDetail{
            Variant: "on", Reason: openfeature.TargetingMatchReason}
    }
    return f.Variants["off"], openfeature.ProviderResolutionDetail{
        Variant: "off", Reason: openfeature.DefaultReason}
}

func orderlyFlags() map[string]memprovider.InMemoryFlag {
    return map[string]memprovider.InMemoryFlag{
        "express-shipping": {
            Key:              "express-shipping",
            State:            memprovider.Enabled,
            DefaultVariant:   "off",
            Variants:         map[string]any{"on": true, "off": false},
            ContextEvaluator: &goldOnly,
        },
    }
}
```

Wiring happens once in `main`. Registering the provider under a named **domain** ("orderly") and creating a client for that domain keeps this flag set separate from any other part of the program:

GO flags/main.go

```go
    provider := memprovider.NewInMemoryProvider(orderlyFlags())
    // Register the provider for the "orderly" domain and wait until ready.
    if err := openfeature.SetNamedProviderAndWait("orderly",
        provider); err != nil {
        log.Fatal(err)
    }
    defer openfeature.Shutdown()

    client := openfeature.NewClient("orderly")
    checkout := NewCheckout(client, slog.New(slog.NewTextHandler(os.Stderr,
        nil)))
```

TERMINAL

```
go run ./flags
```

OUTPUT

```
cust-1 (gold): [standard express]
cust-2 (basic): [standard]
details: value=true variant=on reason=TARGETING_MATCH
missing flag: value=false
  err=error code: FLAG_NOT_FOUND: flag for key no-such-flag not found
```

The `BooleanValueDetails` call (third line) tells you *why* a value was chosen, which is gold when a customer asks "why don't I see express shipping?". In production, you'd replace `memprovider` with your vendor's provider; `Checkout` doesn't change.

Testing flagged code

Tests register a fresh in-memory provider under a domain named after the test, so tests never share flag state:

GO flags/checkout_test.go

```go
func newClient(t *testing.T,
    flags map[string]memprovider.InMemoryFlag) *openfeature.Client {
    t.Helper()
    p := memprovider.NewInMemoryProvider(flags)
    require.NoError(t, openfeature.SetNamedProviderAndWait(t.Name(), p))
    return openfeature.NewClient(t.Name())
}

func TestShippingOptions_TargetsGoldTier(t *testing.T) {
    checkout := NewCheckout(newClient(t, orderlyFlags()), quiet)
    ctx := context.Background()

    assert.Equal(t, []string{"standard", "express"},
        checkout.ShippingOptions(ctx, Customer{ID: "c1", Tier: "gold"}))
    assert.Equal(t, []string{"standard"},
```

```
        checkout.ShippingOptions(ctx, Customer{ID: "c2", Tier: "basic"}))
}
```

The same file also tests the kill switch (`State: memprovider.Disabled`) and a provider outage, using a three-line fake `FlagEvaluator` that always returns an error. Both cases must fall back to standard shipping only.

> 💡 **Keep flags short-lived**
>
> Every flag is a branch in your code and a row in someone's dashboard. Give each release flag an owner and a removal date, and delete it once the feature is fully rolled out. Long-lived operational switches (kill switches, maintenance modes) are fine, but name them clearly.

Recap

- Treat config as data: one `Config` struct, loaded once in `main`, validated, and passed down explicitly. No global singletons.
- caarlos0/env maps environment variables to typed fields with defaults, `required`, nested `envPrefix` and `file` secrets; it's the Go cousin of `@ConfigurationProperties`.
- Validate at startup with go-playground/validator plus a few cross-field checks, list every problem at once, and exit non-zero.
- Replace Spring profiles with per-environment env files, Kustomize overlays or Helm values; switch behaviour with explicit settings, not environment names.
- Use koanf when you want layered YAML plus env overrides; know Viper's lowercase keys and `AutomaticEnv` / `Unmarshal` gap before you use it.
- Keep secrets out of git and out of logs: deliver them as env vars or mounted files, wrap them in a `Secret` type, and use Vault or cloud SDKs where needed.
- Prefer rolling restarts to live reload; reload only safe things like the log level (`slog.LevelVar`) and feature flags.
- Put new features behind OpenFeature flags, choose safe defaults, and test flagged code with the in-memory provider.

Try it yourself

> 🏋️ **Exercise 1: A new setting, end to end**
>
> Add `ORDERLY_ORDERS_MAX_ITEMS` (default 50, must be between 1 and 500) to `config.Config` in a new `OrdersConfig` struct with the `ORDERS_` prefix. Add a test case for the default, an override and an invalid value. *Hint:* copy the shape of `HTTPConfig.Port`, including its `validate` tag.

> 🏋️ **Exercise 2: A staging profile**
>
> Add `layered/config/staging.yaml` with three replicas' worth of brokers and `log_level: info`, then extend the table test in `layers_test.go` with a "staging profile" row. *Hint:* `TestLoad_MissingProfileFails` currently uses "staging"; switch it to a profile that still doesn't exist.

> 🏋️ **Exercise 3: A percentage rollout**
>
> Change the `express-shipping` flag so it's on for all gold customers *and* for roughly 10% of everyone else, chosen consistently by customer ID. *Hint:* in the context evaluator, hash `ctx[openfeature.TargetingKey]` with `hash/fnv` and compare `sum % 100 < 10`.

> 🏋️ **Exercise 4: Reload safely (stretch)**
>
> Build a `Reloader` that watches a YAML file, builds a complete new `Config` on change, validates it, and swaps it into an `atomic.Pointer[Config]` only if it's valid. Write a test that writes an invalid file and checks the old config is still in place. *Hint:* reuse the structure of `watchLogLevel` and `require.Eventually` from `layers_test.go`.

Further reading

- The Twelve-Factor App: Config: the short essay behind environment-based configuration.
- caarlos0/env: the README lists every tag option, including `file`, `expand` and `init`.
- go-playground/validator: the full list of built-in validation tags.
- koanf: providers, parsers and examples of merging and watching.
- Kubernetes: Secrets: how Secrets are stored, mounted and updated.
- Azure Key Vault secrets client for Go: the API used in the example.
- OpenFeature documentation: concepts like providers, evaluation context and hooks.
- OpenFeature Go SDK: the client API and the in-memory provider.

CHAPTER 11

Messaging, Background Jobs and Workflows

> *Most of the hard bugs in distributed systems hide in the gaps between services, so this chapter is about crossing those gaps safely.*

🎯 **What you'll learn**

- Explain at-most-once, at-least-once and effectively-once delivery in plain English, and pick the right one
- Produce and consume Kafka messages with **franz-go**: keys, partitions, manual commits, retries, dead-letter topics and graceful shutdown
- Use RabbitMQ with amqp091-go, and know where NATS, Azure Service Bus and Google Pub/Sub fit
- Build message handlers with Watermill's router and middleware, the Go cousin of Spring Cloud Stream
- Publish events reliably with a **transactional outbox**, and make consumers idempotent
- Run background jobs with a proper lifecycle, River and cron, and model an order saga with **Temporal**

The big picture

When a customer places an order in Orderly, several other things should happen: billing charges the card, the warehouse prepares a parcel, loyalty adds points and an email goes out. If the order API called each of those services directly, one slow email server would make checkout slow, and one outage would make checkout fail.

Asynchronous messaging fixes that. The order service records the order, publishes an `OrderPlaced` event and replies to the customer straight away. Other services consume the event in their own time, at their own pace, and a temporary outage only delays them.

🧩 **Think of it like the ticket rail in a restaurant kitchen**

The waiter doesn't stand next to the chef waiting for each dish. They clip an order ticket to the rail and go back to the customers. Cooks take tickets when they're ready, and if one station is slow, tickets queue up instead of the dining room grinding to a halt. A message broker is that ticket rail.

You've likely done all of this in Spring with Spring Kafka, Spring AMQP, `@Scheduled` and perhaps Temporal or Camunda. Here's how the pieces map:

Concept	Java / Spring	Go
Kafka producer	`KafkaTemplate`	**franz-go** `kgo.Client.ProduceSync`
Kafka consumer	`@KafkaListener`	a poll, process, commit loop
Retries and DLT	`DefaultErrorHandler` + DLT recoverer	`cenkalti/backoff` + a DLT publish
RabbitMQ	Spring AMQP, `@RabbitListener`	`rabbitmq/amqp091-go`
Broker abstraction	Spring Cloud Stream	Watermill
Reliable publishing	outbox, Debezium	outbox table + relay goroutine
Scheduled work	`@Scheduled`, Quartz	goroutines, robfig/cron, River
Workflows and sagas	Temporal Java, Camunda	Temporal Go SDK

Full code: `examples/ch11-messaging/`. The Kafka code runs against an in-memory Kafka cluster that ships with franz-go, so you can run it without Docker.

Delivery semantics in plain English

Before any code, let's settle the vocabulary, because every design decision in this chapter depends on it. Networks fail and processes crash at the worst moment, so a message system has to choose what happens to a message caught in the middle.

> 🧩 **Think of it like posting letters**
>
> **At-most-once** is dropping a postcard in the box: it usually arrives, but if it's lost, nobody resends it. **At-least-once** is registered mail with a "resend until signed for" rule: it always arrives, but if the signature slip gets lost, a second copy turns up. **Effectively-once** is the recipient keeping a list of letter numbers already read, so a second copy goes straight in the bin.

Guarantee	What can happen	How you get it
At-most-once	messages may be lost, never repeated	commit or ack *before* processing
At-least-once	never lost, may be repeated	commit or ack *after* processing
Effectively-once	repeats happen but change nothing	at-least-once + idempotent consumers

"Exactly-once delivery" across a network isn't something any broker can promise end to end. Kafka's transactions give exactly-once *processing* for Kafka-to-Kafka pipelines, but the moment your consumer writes to a database or calls an API, you're back to at-least-once plus idempotency.

> 🧠 **Key idea**
>
> Default to **at-least-once delivery with idempotent consumers**. Every pattern below (manual commits, the outbox, dedup tables, Temporal activities) is built around that one idea.

Kafka with franz-go

Choosing a Kafka client

Go has several Kafka clients, and the choice matters more than in Java, where the official client is the obvious default.

Library	Pure Go?	Notes
twmb/franz-go	yes	full protocol, transactions, fast; the house pick
segmentio/kafka-go	yes	simple API, smaller feature set
confluent-kafka-go	no (cgo)	wraps the C library librdkafka
IBM/sarama	yes	the oldest; formerly Shopify/sarama

franz-go is the recommendation: it's pure Go (so it cross-compiles and builds in tiny containers), it supports every modern Kafka feature, including idempotent and transactional producers, and it's actively maintained. confluent-kafka-go is also solid, but it needs **cgo** (Go calling C code), so you need a C toolchain at build time and you lose easy cross-compilation.

An idempotent producer with keys

Here's the Spring version you'd write today:

JAVA

```java
@Service
public class OrderEventPublisher {
    private final KafkaTemplate<String, OrderPlaced> kafka;

    public OrderEventPublisher(KafkaTemplate<String, OrderPlaced> kafka) {
        this.kafka = kafka;
    }

    public void publish(OrderPlaced event) {
        // key = order ID, so all events for one order share a partition
        kafka.send("orderly.orders", event.orderId().toString(), event)
             .join(); // wait for the broker's ack
    }
}
```

In Go, a `kgo.Client` plays the role of the `KafkaTemplate` plus its producer factory. You create one at startup and share it:

GO kafka/producer.go

```go
func NewProducer(brokers ...string) (*kgo.Client, error) {
    return kgo.NewClient(
        kgo.SeedBrokers(brokers...),
        kgo.ClientID("orderly-api"),
        kgo.DefaultProduceTopic(events.TopicOrders),
        kgo.RequiredAcks(kgo.AllISRAcks()),     // wait for all in-sync replicas
        kgo.ProducerLinger(5*time.Millisecond), // batch briefly for throughput
    )
}
```

franz-go producers are **idempotent by default**: the broker de-duplicates retried sends, so a network blip never writes a record twice or out of order within a partition. That's `enable.idempotence=true` in the Java client, without having to remember it.

Publishing one event is a record with a key, a value and optional headers:

GO kafka/producer.go

```go
func PublishOrderPlaced(ctx context.Context, cl *kgo.Client,
    e events.OrderPlaced) error {
    value, err := json.Marshal(e)
    if err != nil {
        return fmt.Errorf("encode event: %w", err)
    }
    rec := &kgo.Record{
        Key:   []byte(e.OrderID),
        Value: value,
        Headers: []kgo.RecordHeader{
            {Key: events.TypeHeader, Value: []byte("OrderPlaced")},
        },
    }
    if err := cl.ProduceSync(ctx, rec).FirstErr(); err != nil {
        return fmt.Errorf("produce: %w", err)
    }
    return nil
}
```

The **key** decides the **partition** (one of the topic's ordered sub-logs). Records with the same key always go to the same partition, so every event for order `o-1` is consumed in the order it was produced. franz-go's default partitioner hashes keys the same way as the Java client, so Go and Java producers agree on where a key belongs.

`ProduceSync` waits for the broker's acknowledgement, like `send(...).join()`. For high throughput, `cl.Produce(ctx, rec, callback)` is asynchronous, like `send` with a `whenComplete` callback.

A consumer group with manual commits

A **consumer group** is a set of consumers sharing a topic's partitions; Kafka gives each partition to one member and rebalances when members come and go. The consumer's **offset** is its bookmark in each partition, and **committing** saves that bookmark.

In Spring you'd switch to manual acknowledgement and acknowledge after the work is done:

JAVA

```java
// application.yml: spring.kafka.listener.ack-mode: manual
@KafkaListener(topics = "orderly.orders", groupId = "billing")
public void onOrderPlaced(OrderPlaced event, Acknowledgment ack) {
    loyalty.award(event);
    ack.acknowledge(); // commit only after the work succeeded
}
```

In Go there's no listener container; you write the loop yourself, and it's short. First, the client:

GO kafka/consumer.go

```go
func NewConsumer(group string, topics []string,
    brokers ...string) (*kgo.Client, error) {
    return kgo.NewClient(
        kgo.SeedBrokers(brokers...),
        kgo.ConsumerGroup(group),
        kgo.ConsumeTopics(topics...),
        kgo.DisableAutoCommit(),    // we commit after processing
        kgo.BlockRebalanceOnPoll(), // no rebalance while we hold records
        // A brand-new group starts at the oldest record, like
        // auto.offset.reset=earliest in the Java client.
        kgo.ConsumeResetOffset(kgo.NewOffset().AtStart()),
    )
}
```

`BlockRebalanceOnPoll` deserves a sentence: it stops the group from moving partitions away from us between "poll" and "commit", so we never commit offsets for a partition we no longer own. The price is that we must process each batch reasonably quickly and then call `AllowRebalance`.

Now the loop, which is the heart of every Go Kafka consumer:

GO kafka/run.go

```go
func (c *Consumer) Run(ctx context.Context) error {
    // Finish the current batch even after ctx is cancelled, so a
    // shutdown doesn't abandon half-processed work.
    work := context.WithoutCancel(ctx)
    // A poll can return records right as ctx is cancelled. We drop them
    // (they aren't committed, so they'll be redelivered), but we must
    // still unblock rebalancing or Close can't leave the group.
    defer c.cl.AllowRebalance()
    for {
        fetches := c.cl.PollRecords(ctx, 100)
        if fetches.IsClientClosed() || ctx.Err() != nil {
            return nil
        }
        fetches.EachError(func(topic string, p int32, err error) {
            c.log.Error("fetch failed", "topic", topic, "partition", p,
                "err", err)
        })

        for _, r := range fetches.Records() {
            if err := c.process(work, r); err != nil {
                return err // can't even dead-letter it: stop, don't commit
```

```
            }
        }

        commitCtx, cancel := context.WithTimeout(work, 10*time.Second)
        err := c.cl.CommitUncommittedOffsets(commitCtx)
        cancel()
        if err != nil {
            c.log.Error("commit failed; records will be redelivered",
                "err", err)
        }
        c.cl.AllowRebalance() // pairs with BlockRebalanceOnPoll
    }
}
```

Read it as three steps: **poll** up to 100 records, **process** each one, **commit** the batch. Because the commit comes after processing, a crash between the two means the batch is delivered again: that's at-least-once.

> ⚠️ **Gotcha: forgetting AllowRebalance**
>
> When we first wrote this loop for the book, the demo hung on shutdown. The last poll had returned records while the context was being cancelled, rebalancing stayed blocked, and `Close` waited forever to leave the group. The deferred `AllowRebalance` fixes it. If you use `BlockRebalanceOnPoll`, every return path needs one.

Retries, backoff and dead-letter topics

Some failures are temporary (the database is restarting), and retrying fixes them. Others are permanent (the payload isn't valid JSON), and retrying forever would block the whole partition behind one bad record, a so-called **poison message**.

Spring Kafka handles this with an error handler bean:

JAVA

```
@Bean
DefaultErrorHandler errorHandler(KafkaTemplate<Object, Object> template) {
    var backOff = new ExponentialBackOffWithMaxRetries(3);
    backOff.setInitialInterval(200);
    var handler = new DefaultErrorHandler(
            new DeadLetterPublishingRecoverer(template), backOff);
    handler.addNotRetryableExceptions(JsonProcessingException.class);
    return handler;
}
```

In Go, the same policy is a small function using **cenkalti/backoff**, the house pick for retries:

GO kafka/retry.go

```go
func (c *Consumer) process(ctx context.Context, r *kgo.Record) error {
    b := backoff.NewExponentialBackOff()
    b.InitialInterval, b.MaxInterval = c.retry.InitialDelay, c.retry.MaxDelay

    _, err := backoff.Retry(ctx, func() (struct{}, error) {
        return struct{}{}, c.handle(ctx, r)
    }, backoff.WithBackOff(b), backoff.WithMaxTries(c.retry.MaxTries))
    if err == nil {
        return nil
    }
    c.log.Warn("giving up on record, dead-lettering it", "topic", r.Topic,
        "partition", r.Partition, "offset", r.Offset, "err", err)
    return c.deadLetter(ctx, r, err)
}
```

The handler marks permanent failures by wrapping them with `backoff.Permanent(err)`, which stops retrying immediately (the `addNotRetryableExceptions` equivalent). After the last attempt, `deadLetter` copies the record to `orderly.orders.dlt` with headers recording the error and the original topic, partition and offset, so someone can inspect and replay it later. (Spring's default suffix is `.DLT`; pick one convention per company.)

> 💡 **Keep in-consumer retries short**
>
> Retrying inside the loop blocks the partition, so keep it to a few attempts over a few seconds. For longer waits (a partner API down for an hour), dead-letter quickly and replay later, or use dedicated retry topics with increasing delays.

Graceful shutdown

Kubernetes sends `SIGTERM` before killing a pod. A well-behaved consumer finishes its batch, commits, leaves the group (so its partitions move to other pods at once) and exits. In Go, that's a cancellable context and a `WaitGroup`:

GO kafka/cmd/kafkademo/main.go

```go
    // Stop on Ctrl+C or SIGTERM (what Kubernetes sends), or after 3s.
    ctx, stop := signal.NotifyContext(context.Background(),
        os.Interrupt, syscall.SIGTERM)
    defer stop()
    ctx, cancel := context.WithTimeout(ctx, 3*time.Second)
    defer cancel()
    // ... publish some events, create the consumer client and loop

    var wg sync.WaitGroup
    wg.Go(func() {
        if err := loop.Run(ctx); err != nil {
            log.Error("consumer stopped", "err", err)
        }
    })
    <-ctx.Done()
    wg.Wait()  // the loop has committed its last batch
```

```
    cl.Close() // leaves the group so partitions move on quickly
    log.Info("shut down cleanly")
```

The demo starts an in-memory Kafka, publishes three good events and one poison record, consumes them in the `billing` group and shuts down (we wrapped the long warning line):

TERMINAL

```
go run ./kafka/cmd/kafkademo
```

OUTPUT

```
level=INFO msg="published 4 records"
partition 0 offset 0: order o-1, 12 points
partition 0 offset 1: order o-2, 25 points
partition 1 offset 0: order o-3, 49 points
level=WARN msg="giving up on record, dead-lettering it" topic=orderly.orders
  partition=1 offset=1 err="bad payload: invalid character 'o' looking for
  beginning of object key string"
level=INFO msg="shut down cleanly"
```

Testing without a broker

franz-go ships `kfake`, an in-memory Kafka cluster that speaks the real protocol, including consumer groups. Tests start one in a line or two, then use the normal client against it:

GO kafka/kafka_test.go

```go
func newCluster(t *testing.T) []string {
    t.Helper()
    c, err := kfake.NewCluster(kfake.NumBrokers(1),
        kfake.SeedTopics(3, events.TopicOrders),
        kfake.SeedTopics(1, events.TopicOrders+".dlt"))
    require.NoError(t, err)
    t.Cleanup(c.Close)
    return c.ListenAddrs()
}
```

The example's tests check that one key stays on one partition, that transient errors are retried and that a poison record lands in the DLT, all in a fraction of a second. It's a big win compared with spinning up a Kafka container for every unit test. Keep a few testcontainers-based tests for things kfake can't tell you, such as broker configuration (Chapter 15).

RabbitMQ with amqp091-go

RabbitMQ is a different kind of broker. Kafka is a **log**: messages stay for a retention period, and each consumer group tracks its own position. RabbitMQ is a **queue**: a message is routed through an **exchange** to queues, delivered to one consumer and deleted once acknowledged.

In Spring AMQP, a listener acks for you when the method returns normally:

JAVA

```java
@RabbitListener(queues = "loyalty.order-placed")
public void onOrderPlaced(OrderPlaced event) {
    loyalty.award(event);
    // returns normally: acked; throws: rejected (and maybe requeued);
    // throw AmqpRejectAndDontRequeueException to dead-letter it
}
```

The Go client is `github.com/rabbitmq/amqp091-go`, maintained by the RabbitMQ team. It's lower level than Spring AMQP: you declare the topology, publish and acknowledge explicitly. Declarations are idempotent, so each service can declare what it needs at startup. Here's the queue with its dead-letter exchange:

GO rabbitmq/rabbit.go

```go
    // Messages this queue rejects are re-routed to the DLX.
    args := amqp.Table{"x-dead-letter-exchange": DLX}
    if _, err := ch.QueueDeclare(Queue, true, false, false, false,
        args); err != nil {
        return fmt.Errorf("declare queue: %w", err)
    }
    return ch.QueueBind(Queue, RoutingKey, Exchange, false, nil)
```

The positional booleans (`durable, autoDelete, exclusive, noWait`) are the least friendly part of this API; a comment or named constants help reviewers. Publishing uses **publisher confirms**, the broker's promise that it has safely stored the message:

GO rabbitmq/rabbit.go

```go
    conf, err := ch.PublishWithDeferredConfirmWithContext(ctx, Exchange,
        RoutingKey, true, false, amqp.Publishing{
            ContentType:  "application/json",
            DeliveryMode: amqp.Persistent, // survive a broker restart
            MessageId:    id,              // consumers dedupe on this
            Body:         body,
        })
    if err != nil {
        return fmt.Errorf("publish: %w", err)
    }
    ok, err := conf.WaitContext(ctx)
    // ... error if !ok: the broker refused the message
```

On the consuming side, turn off auto-ack, set a **prefetch** (how many unacknowledged messages one consumer may hold) and settle each delivery after processing:

GO rabbitmq/consume.go

```go
func settle(d amqp.Delivery, err error, log *slog.Logger) error {
    switch {
    case err == nil:
        return d.Ack(false)
    case errors.Is(err, ErrPermanent):
        log.Warn("dead-lettering message", "id", d.MessageId, "err", err)
        return d.Nack(false, false)
    default:
        log.Warn("requeueing message", "id", d.MessageId, "err", err)
        return d.Nack(false, true)
    }
}
```

The library makes this easy to test: `amqp.Delivery` has an `Acknowledger` field, so a unit test plugs in a fake and checks which call was made. The integration test then runs the whole flow against a RabbitMQ container.

> ⚠️ **Gotcha: no automatic reconnection**
>
> Spring's `CachingConnectionFactory` quietly reconnects after a network blip. amqp091-go doesn't: when the connection drops, your delivery channel closes and that's it. Watch `conn.NotifyClose(...)` and rebuild the connection and channels with backoff, or let Watermill's AMQP adapter (next sections) do it for you.

NATS, JetStream and cloud brokers, briefly

You'll meet a few other brokers in enterprise Go code. All have solid, officially supported Go clients:

Broker	Go client	Good fit
NATS + JetStream	`nats-io/nats.go` (`jetstream` package)	low latency, edge, simple ops
Azure Service Bus	`azservicebus` (Azure SDK)	Azure: sessions, DLQ built in
Google Pub/Sub	`cloud.google.com/go/pubsub/v2`	GCP: fully managed fan-out
AWS SQS / SNS	`aws-sdk-go-v2`	AWS: simple queues and topics

NATS is itself written in Go and is popular for fast service-to-service messaging; **JetStream** adds persistence, replay and at-least-once delivery on top. The cloud brokers trade some flexibility for zero operations. The patterns in this chapter (explicit acks, dead-letter queues, idempotent consumers, the outbox) apply to all of them unchanged.

Watermill: a Spring Cloud Stream for Go

Writing the poll, process, ack loop for each broker is fine for one or two consumers. When you have dozens, you want what Spring Cloud Stream gives you: handlers that don't know which broker they're on, plus reusable cross-cutting behaviour. **Watermill** fills that role in Go.

JAVA

```java
// Spring Cloud Stream: a function bean, bound to a topic in config
@Bean
public Consumer<OrderPlaced> loyaltyPoints() {
    return event -> loyalty.award(event);
}
```

Watermill has three main ideas:

- **Publisher and Subscriber** interfaces, with adapters for Kafka, RabbitMQ, NATS, SQL, Google Pub/Sub, Redis Streams and an in-memory Go channel.
- A **Router** that connects topics to **handlers**: functions that receive a `*message.Message`.
- **Middleware** that wraps handlers, exactly like HTTP middleware (Chapter 7): retries, poison queues, panic recovery, correlation IDs, throttling.

GO watermill/router.go

```go
    poison, err := middleware.PoisonQueue(pubSub, "orders.poison")
    if err != nil {
        return nil, err
    }
    // Order matters: the first middleware wraps all the others.
    router.AddMiddleware(
        middleware.CorrelationID, // copy correlation IDs to new messages
        poison,                   // 3. still failing? park it, move on
        middleware.Retry{ // 2. retry failures with backoff
            MaxRetries: 2, InitialInterval: 10 * time.Millisecond,
            Multiplier: 2, Logger: logger,
            OnRetryHook: func(n int, _ time.Duration) {
                fmt.Fprintf(out, "  retry #%d\n", n)
            },
        }.Middleware,
        middleware.Recoverer, // 1. turn a panic into an error
    )

    router.AddConsumerHandler("loyalty-points", "orders.placed", pubSub,
        func(msg *message.Message) error {
            // ... decode the event, award points, return nil or an error
        })
```

Read the middleware from the inside out: a panic becomes an error, errors are retried twice, and a message that still fails goes to the `orders.poison` topic so the rest of the stream keeps flowing. With the in-memory pub/sub, the demo runs anywhere:

TERMINAL

```
go run ./watermill
```

OUTPUT

```
order o-1: 12 points
cannot decode "{oops"
cannot decode "{oops"
  retry #1
cannot decode "{oops"
  retry #2
poison queue: "{oops" (reason: invalid character 'o' looking for beginning of
  object key string)
order o-2: 25 points
```

(We wrapped the long poison line.) Swapping the Go channel for Kafka or RabbitMQ changes only the publisher and subscriber setup; the handlers and middleware stay the same.

> 💡 **If you're unsure: franz-go directly, or Watermill?**
>
> For one or two Kafka consumers with specific needs (careful commits, high throughput), use franz-go directly as shown earlier. For many handlers, several brokers, or a team that wants one consistent pattern with retry and poison queues built in, Watermill is a good fit.

The transactional outbox

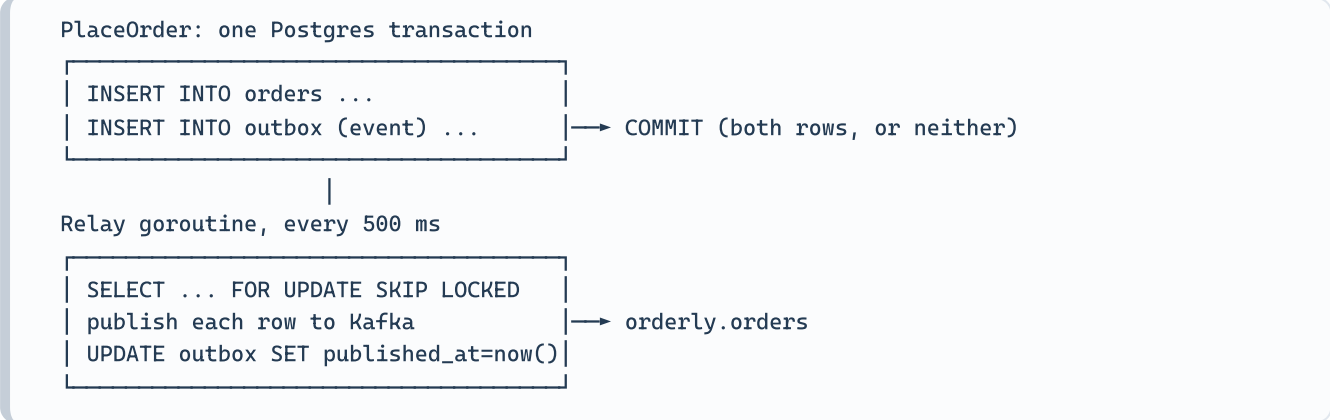
The dual-write problem

Here's a bug that hides in most first versions of event-driven services. The order service saves the order to Postgres and then publishes `OrderPlaced` to Kafka. Those are two separate systems, so there are two ways for it to go wrong:

```
save order ──► COMMIT ──► publish to Kafka ──X  (crash or Kafka down)
                          => order exists, nobody hears about it

publish to Kafka ──► save order ──X  (constraint violation, rollback)
                          => everyone hears about an order that doesn't exist
```

You can't wrap Postgres and Kafka in one transaction (and XA two-phase commit is something nobody misses). The **transactional outbox** pattern solves it with a simple trick: write the event into an `outbox` table *in the same database transaction* as the order. A separate **relay** then reads the outbox and publishes to Kafka.



> 🧩 **Think of it like the office out-tray**
>
> You don't run to the post office in the middle of writing a contract. You finish the contract and drop the envelope in your out-tray in the same motion. Later, the mail clerk empties every out-tray. If you tear up the contract, the envelope never went into the tray, so nothing gets sent.

Writing the order and the event together

The outbox table holds the topic, the Kafka key and the JSON payload, plus `published_at` (NULL until sent):

SQL outbox/schema.sql

```sql
CREATE TABLE outbox (
    id           bigserial PRIMARY KEY,
    topic        text NOT NULL,
    message_key  text NOT NULL,
    payload      jsonb NOT NULL,
    created_at   timestamptz NOT NULL DEFAULT now(),
    published_at timestamptz
);

-- Keeps "find unpublished rows" fast, however big the table grows.
CREATE INDEX outbox_unpublished ON outbox (id) WHERE published_at IS NULL;
```

`Enqueue` takes the caller's transaction, so any service method can add events to whatever it's already writing:

GO outbox/writer.go

```go
func Enqueue(ctx context.Context, tx pgx.Tx, topic, key string,
    event any) error {
    payload, err := json.Marshal(event)
    if err != nil {
        return fmt.Errorf("encode event: %w", err)
    }
    _, err = tx.Exec(ctx,
        `INSERT INTO outbox (topic, message_key, payload) VALUES ($1, $2, $3)`,
        topic, key, payload)
    return err
}
```

And `PlaceOrder` uses both inside `pgx.BeginFunc`, so the order and its event commit together or not at all:

GO outbox/writer.go

```go
func (s *OrderStore) PlaceOrder(ctx context.Context, o Order) error {
    return pgx.BeginFunc(ctx, s.pool, func(tx pgx.Tx) error {
        _, err := tx.Exec(ctx, `INSERT INTO orders
            (id, customer_id, status, total_cents) VALUES ($1, $2, $3, $4)`,
            o.ID, o.CustomerID, "CONFIRMED", o.TotalCents)
        if err != nil {
            return fmt.Errorf("insert order: %w", err)
        }
        return Enqueue(ctx, tx, events.TopicOrders, o.ID.String(),
            events.OrderPlaced{OrderID: o.ID.String(),
                CustomerID: o.CustomerID.String(),
                TotalCents: o.TotalCents, Items: o.Items})
    })
}
```

This is the same transaction helper idea you met in Chapter 9, applied to events.

The relay: FOR UPDATE SKIP LOCKED

The relay depends on two small interfaces, so its logic can be tested without a database or a broker:

GO outbox/relay.go

```go
type Publisher interface {
    Publish(ctx context.Context, m Message) error
}

// Store runs fn in a database transaction.
type Store interface {
    InTx(ctx context.Context, fn func(tx Tx) error) error
}

// Tx is what the relay can do inside that transaction.
type Tx interface {
    // ClaimBatch locks up to limit unpublished rows, oldest first, skipping
    // rows another relay has already locked (FOR UPDATE SKIP LOCKED).
    ClaimBatch(ctx context.Context, limit int) ([]Message, error)
    MarkPublished(ctx context.Context, ids []int64) error
}
```

One pass of the relay claims a batch, publishes it in order and marks what was sent, all inside one transaction:

GO outbox/relay_run.go

```go
func (r *Relay) RunOnce(ctx context.Context) (int, error) {
    sent := 0
    err := r.store.InTx(ctx, func(tx Tx) error {
        msgs, err := tx.ClaimBatch(ctx, r.batch)
        if err != nil {
            return err
        }
        var done []int64
        for _, m := range msgs {
            if err := r.pub.Publish(ctx, m); err != nil {
                // Stop at the first failure: later rows may share this
                // key, and publishing them now would break ordering.
                r.log.Warn("publish failed, will retry", "id", m.ID,
                    "err", err)
                break
            }
            done = append(done, m.ID)
        }
        sent = len(done)
        if sent == 0 {
            return nil
        }
        return tx.MarkPublished(ctx, done)
    })
    return sent, err
}
```

`Run` calls `RunOnce` on a ticker until the context is cancelled, and immediately again whenever a batch was full. The Postgres side is where the clever part lives:

GO outbox/postgres.go

```go
func (t pgTx) ClaimBatch(ctx context.Context, limit int) ([]Message, error) {
    rows, err := t.tx.Query(ctx, `
        SELECT id, topic, message_key, payload
        FROM outbox
        WHERE published_at IS NULL
        ORDER BY id
        LIMIT $1
        FOR UPDATE SKIP LOCKED`, limit)
    if err != nil {
        return nil, err
    }
    return pgx.CollectRows(rows, pgx.RowToStructByPos[Message])
}
```

`FOR UPDATE` locks the claimed rows until the transaction ends, and `SKIP LOCKED` tells other transactions to skip locked rows instead of waiting for them. So if three replicas each run a relay, they share the work without ever publishing the same row twice at the same time. The integration test proves it: two relays drain 50 outbox rows concurrently, and each row is published exactly once.

> ⚠️ **Gotcha: the relay is at-least-once too**
>
> If the relay publishes a message and then crashes before its transaction commits, the row is still unpublished and gets sent again. That's why the Kafka publisher adds a `message-id` header with the outbox row ID: it stays the same on a resend, so consumers can de-duplicate (next section). Also note that several relays process *different* batches in parallel, so strict per-key ordering across replicas needs a single active relay or partitioning by key.

Testing the relay with fakes

The unit tests use an in-memory `fakeStore` and a `fakePublisher` that can fail on chosen message IDs. One test checks that a failure stops the batch and that the next run resumes in order; another uses `testing/synctest` (Chapter 15) to drive the ticker with a fake clock; a third runs the relay against kfake with the real Kafka publisher:

GO outbox/relay_test.go

```go
func TestRunOnce_StopsAtFirstFailureAndKeepsOrder(t *testing.T) {
    store := newFakeStore(4)
    pub := &fakePublisher{failOn: map[int64]bool{3: true}}
    relay := NewRelay(store, pub, quiet)

    n, err := relay.RunOnce(context.Background())
    require.NoError(t, err)
    assert.Equal(t, 2, n, "1 and 2 sent, 3 failed, 4 not attempted")
    assert.False(t, store.published[3])
    assert.False(t, store.published[4])

    pub.failOn = nil // the broker is back
    _, err = relay.RunOnce(context.Background())
    require.NoError(t, err)
    assert.Equal(t, []int64{1, 2, 3, 4}, pub.sent, "still in order")
}
```

OUTPUT

```
--- PASS: TestRelayToKafka (0.04s)
--- PASS: TestRunOnce_PublishesAndMarks (0.00s)
--- PASS: TestRunOnce_StopsAtFirstFailureAndKeepsOrder (0.00s)
--- PASS: TestRun_PollsUntilCancelled (0.00s)
ok      example.com/goguide/ch11/outbox 1.243s
```

> **The CDC alternative**
>
> Instead of a polling relay, some teams use **change data capture** (CDC): Debezium reads Postgres's write-ahead log and publishes outbox rows to Kafka. It removes polling and keeps strict ordering, at the cost of running Kafka Connect. The write side of your Go code stays exactly the same.

Idempotent consumers

At-least-once delivery means every consumer will, sooner or later, see a message twice. An **idempotent** operation is one that has the same effect whether it runs once or several times. Some operations already are: "set order status to SHIPPED" or `INSERT ... ON CONFLICT DO NOTHING` with a natural key. Prefer those when you can.

Others aren't. "Add 49 loyalty points" run twice gives the customer 98. For those, keep a **processed-messages table** (sometimes called an inbox) and record each message ID in the same transaction as the work:

SQL inbox/schema.sql

```sql
CREATE TABLE processed_messages (
    consumer     text NOT NULL,
    message_id   text NOT NULL,
    processed_at timestamptz NOT NULL DEFAULT now(),
    PRIMARY KEY (consumer, message_id)
);
```

The handler tries to insert the ID first. If the insert finds a duplicate, the message was handled before, and we skip it quietly:

GO inbox/inbox.go

```go
func (h *PointsHandler) Handle(ctx context.Context, messageID string,
    e events.OrderPlaced) error {
    return h.store.InTx(ctx, func(tx Tx) error {
        first, err := tx.MarkProcessed(ctx, "loyalty-points", messageID)
        if err != nil {
            return fmt.Errorf("dedup check: %w", err)
        }
        if !first {
            return nil // seen it before: already done, skip quietly
        }
        // Same transaction: if this fails, the dedup row rolls back too,
        // so a retry will process the message again.
        return tx.AddPoints(ctx, e.CustomerID, e.TotalCents/100)
    })
}
```

In Postgres, `MarkProcessed` is one statement: `INSERT INTO processed_messages ... ON CONFLICT DO NOTHING`, returning whether a row was inserted. The unit tests deliver the same message three times and check the points were added once, then check that a failure rolls back the dedup row so a retry can succeed.

> 💡 **Choose the message ID carefully**
>
> Use an ID that stays the same when a message is *re-sent*, such as the outbox row ID in our `message-id` header or RabbitMQ's `MessageId`. A Kafka offset is not enough: if the relay publishes the same event twice, the copies get different offsets. Clean old rows out of `processed_messages` with a periodic job once they're older than your longest possible redelivery window.

Background jobs

Not all asynchronous work is a message between services. Expiring unpaid orders, refreshing a cache, sending a confirmation email or building a nightly report are **background jobs** inside one service.

In-process goroutines with a proper lifecycle

In Spring you'd annotate a method:

JAVA

```java
@Scheduled(fixedDelay = 500) // milliseconds
public void expireUnpaidOrders() {
    orderService.expireUnpaid();
}
```

In Go, a periodic job is a goroutine with a ticker. The important part is the **lifecycle**: it must stop when the service stops, and the service must wait for it. A goroutine nobody waits for can be killed half-way through a database write.

GO jobs/periodic.go

```go
func RunEvery(ctx context.Context, name string, interval time.Duration,
    log *slog.Logger, fn func(context.Context) error) {
    ticker := time.NewTicker(interval)
    defer ticker.Stop()
    for {
        select {
        case <-ctx.Done():
            log.Info("job stopped", "job", name)
            return
        case <-ticker.C:
            if err := fn(ctx); err != nil {
                log.Error("job run failed", "job", name, "err", err)
            }
        }
    }
}
```

`main` starts each job with `sync.WaitGroup.Go` (new in Go 1.25), and waits for all of them after the shutdown signal:

GO jobs/cmd/jobsdemo/main.go

```go
    var wg sync.WaitGroup
    wg.Go(func() {
        jobs.RunEvery(ctx, "expire-unpaid-orders", 500*time.Millisecond, log,
            func(context.Context) error {
                log.Info("expired unpaid orders", "count", 0)
                return nil
            })
    })
    // ... a second job, "refresh-stats", every 900 ms

    <-ctx.Done()
    log.Info("shutting down, waiting for jobs")
    wg.Wait() // never exit while a job is half-way through
    log.Info("bye")
```

OUTPUT

```
level=INFO msg="expired unpaid orders" count=0
level=INFO msg="refreshed sales stats"
level=INFO msg="expired unpaid orders" count=0
level=INFO msg="expired unpaid orders" count=0
level=INFO msg="refreshed sales stats"
level=INFO msg="shutting down, waiting for jobs"
level=INFO msg="job stopped" job=refresh-stats
level=INFO msg="job stopped" job=expire-unpaid-orders
level=INFO msg=bye
```

The unit test runs `RunEvery` inside `synctest.Test`, so "three minutes" of ticks pass instantly on a fake clock.

> ⚠️ **Gotcha: every replica runs it**
>
> With three pods, an in-process job runs three times, exactly like `@Scheduled` in three JVMs (Java teams add ShedLock for this). That's fine for per-pod work like refreshing a local cache. For work that must happen once, use a job queue such as River, or make the job itself safe to run concurrently.

Cron schedules with robfig/cron

For calendar schedules ("every night at 02:00 UTC"), `github.com/robfig/cron/v3` is the long-standing choice:

GO jobs/cron.go

```go
c := cron.New(cron.WithLocation(time.UTC))
_, err := c.AddFunc("0 2 * * *", func() { // minute hour day month weekday
    if err := report(ctx); err != nil {
        log.Error("nightly report failed", "err", err)
    }
})
if err != nil {
    return err
}
c.Start()
<-ctx.Done()
<-c.Stop().Done() // Stop returns a context that ends when jobs finish
```

> ⚠️ **Gotcha: five fields, not six**
>
> Spring's `@Scheduled(cron = "0 0 2 * * *")` has six fields, starting with seconds. robfig/cron's default parser uses the classic five-field format, starting with minutes. Copying a Spring expression across silently changes its meaning. Use `cron.New(cron.WithSeconds())` if you want the six-field format.

> **Maintenance status**
>
> robfig/cron's last release (v3.0.1) was in 2020, and the repository is no longer actively developed, although it's stable and very widely used. For new code, `go-co-op/gocron/v2` is an actively maintained alternative, and River (below) has periodic jobs that run on only one replica.

Durable jobs with River

In-process goroutines lose their work when a pod dies. For jobs that must not be lost, such as sending the order confirmation email, you want a **durable job queue**: jobs are stored, retried with backoff and run by any available worker. Java teams use Quartz with a JDBC job store, JobRunr or db-scheduler for this.

River is a job queue for Go built on Postgres, so if you already run Postgres, you need no new infrastructure. A job is a payload type with a `Kind`, and a worker that processes it:

GO jobs/river.go

```go
type ConfirmationEmailArgs struct {
    OrderID string `json:"order_id"`
}

// Kind names the job type; it must stay stable across deploys.
func (ConfirmationEmailArgs) Kind() string { return "confirmation_email" }

type ConfirmationEmailWorker struct {
    river.WorkerDefaults[ConfirmationEmailArgs]
    Mailer Mailer
}

func (w *ConfirmationEmailWorker) Work(ctx context.Context,
    job *river.Job[ConfirmationEmailArgs]) error {
    return w.Mailer.SendOrderConfirmation(ctx, job.Args.OrderID)
}
```

The killer feature is **transactional enqueueing**. Because the queue lives in your database, the job can be inserted in the same transaction as the order, which is the outbox idea again, for free:

GO jobs/river_client.go

```go
func PlaceOrder(ctx context.Context, pool *pgxpool.Pool,
    client *river.Client[pgx.Tx], orderID string) error {
    return pgx.BeginFunc(ctx, pool, func(tx pgx.Tx) error {
        _, err := tx.Exec(ctx,
            `INSERT INTO orders (id, status) VALUES ($1, 'CONFIRMED')`, orderID)
        if err != nil {
            return fmt.Errorf("insert order: %w", err)
        }
        _, err = client.InsertTx(ctx, tx,
            ConfirmationEmailArgs{OrderID: orderID}, nil)
        return err
    })
}
```

The client setup (in the same file) registers workers, sets the number of concurrent workers per queue and adds a periodic cleanup job. River elects a leader among your replicas, so periodic jobs run once, not once per pod. `rivermigrate` creates River's tables, and the integration test runs the whole flow against Postgres in a container.

asynq (`github.com/hibiken/asynq`) is a popular alternative if you prefer Redis: it's similar in spirit to Sidekiq and comes with a web UI. It's still pre-1.0, so check its changelog when upgrading.

Need	Use
per-pod housekeeping, cache refresh	goroutine + ticker, proper lifecycle
a calendar schedule, per pod	robfig/cron or gocron
durable jobs with retries, Postgres already there	**River**
durable jobs, Redis already there	asynq
multi-step business processes over minutes or days	Temporal (next)

Workflows and sagas with Temporal

Why sagas

Placing an Orderly order really involves three services: reserve stock, charge the payment, confirm the order. There's no distributed transaction across them, so if the payment fails, you must **undo** the stock reservation yourself. A **saga** is a sequence of local steps, each with a **compensating action** that undoes it, run in reverse when a later step fails.

You can build sagas from events (each service reacts to the previous one's event, called **choreography**), but the flow becomes hard to see and harder to change. **Orchestration** puts the flow in one place. **Temporal** is an orchestration engine where you write the flow as ordinary code, and Temporal makes it **durable**: every step's result is recorded, so if a worker crashes mid-saga, another worker replays the history and carries on from where it left off.

> 🧩 **Think of it like a travel agent with a perfect notebook**
>
> You ask an agent to book a flight, a hotel and a car. They write down each booking as it succeeds. If the car rental falls through, they look at the notebook and cancel the hotel and the flight, in that order. If the agent goes home sick, a colleague picks up the same notebook and carries on. Temporal is the notebook, and the workflow code is the agent.

The Java version

If you've used Temporal from Java, the shape is familiar:

JAVA

```
public class OrderSagaImpl implements OrderSaga {
    private final OrderActivities activities = Workflow.newActivityStub(
            OrderActivities.class,
            ActivityOptions.newBuilder()
```

```
                    .setStartToCloseTimeout(Duration.ofSeconds(10))
                    .build());

    @Override
    public OrderResult placeOrder(OrderInput input) {
        Saga saga = new Saga(new Saga.Options.Builder().build());
        try {
            activities.reserveStock(input);
            saga.addCompensation(activities::releaseStock, input);
            String paymentId = activities.chargePayment(input);
            saga.addCompensation(activities::refundPayment, paymentId);
            activities.confirmOrder(input);
            return OrderResult.confirmed(paymentId);
        } catch (ActivityFailure e) {
            saga.compensate();
            return OrderResult.cancelled(e.getMessage());
        }
    }
}
```

Camunda solves the same problem with BPMN diagrams; Temporal is code-first, which suits Go teams well.

Activities: the steps that touch the world

Activities are where side effects happen: database writes, payment calls, emails. They're normal Go methods, retried by Temporal according to a policy. A declined card isn't worth retrying, so that activity marks the error as non-retryable:

GO saga/activity_impl.go

```go
func (a *Activities) ChargePayment(ctx context.Context,
    in OrderInput) (string, error) {
    id, err := a.Payments.Charge(ctx, in.OrderID, in.TotalCents)
    if errors.Is(err, ErrCardDeclined) {
        // Tell Temporal not to retry: a declined card stays declined.
        return "", temporal.NewNonRetryableApplicationError(
            "card declined", "CardDeclined", err)
    }
    return id, err
}
```

`Activities` holds interfaces (`Inventory`, `Payments`, `Orders`), so production wires real clients and tests wire fakes.

The workflow: the saga as code

The workflow runs each step, and after each success it pushes the matching undo step onto a list:

GO saga/workflow.go

```go
func OrderSaga(ctx workflow.Context, in OrderInput) (OrderResult, error) {
    ctx = workflow.WithActivityOptions(ctx, workflow.ActivityOptions{
        StartToCloseTimeout: 10 * time.Second, // per attempt
        RetryPolicy: &temporal.RetryPolicy{
            InitialInterval:    time.Second,
            BackoffCoefficient: 2,
            MaximumAttempts:    5,
```

```
        },
    })
    var a *Activities // only used to name the activity methods
    var undo []func(workflow.Context) error

    if err := workflow.ExecuteActivity(ctx, a.ReserveStock, in).
        Get(ctx, nil); err != nil {
        return compensate(ctx, in, undo, err)
    }
    undo = append(undo, func(ctx workflow.Context) error {
        return workflow.ExecuteActivity(ctx, a.ReleaseStock, in).Get(ctx, nil)
    })

    var paymentID string
    if err := workflow.ExecuteActivity(ctx, a.ChargePayment, in).
        Get(ctx, &paymentID); err != nil {
        return compensate(ctx, in, undo, err)
    }
    // ... push RefundPayment onto undo, then run ConfirmOrder the same way
    return OrderResult{Status: "CONFIRMED", PaymentID: paymentID}, nil
}
```

`var a *Activities` looks odd the first time: it's a nil pointer used only so that `a.ReserveStock` names the activity in a type-safe way. Temporal calls the real, registered `Activities` value on the worker.

Compensation walks the list backwards, using a **disconnected context** so the clean-up still runs even if someone cancels the workflow:

GO saga/compensate.go

```
    dctx, cancel := workflow.NewDisconnectedContext(ctx)
    defer cancel()
    for i := len(undo) - 1; i >= 0; i-- {
        if err := undo[i](dctx); err != nil {
            // A failed compensation needs a human: log loudly, keep going.
            log.Error("compensation failed", "step", i, "err", err)
        }
    }
    // ... then run the CancelOrder activity and return a CANCELLED result
```

> ⚠️ **Gotcha: workflow code must be deterministic**
>
> Temporal replays workflow code to rebuild its state, so the same input must always make the same decisions. Inside a workflow, don't call `time.Now`, `time.Sleep`, `rand`, start goroutines or do I/O. Use `workflow.Now`, `workflow.Sleep`, `workflow.Go` and activities instead. This feels restrictive at first, and every Temporal beginner trips on it once; the SDK's replay tests and the `workflowcheck` static analyser help.

Testing the saga in-process

Temporal's `testsuite` package runs workflows in memory, skipping timers and retry delays, so a test that retries five times still finishes in milliseconds. Register the activities with fakes, run the workflow and check what happened:

GO saga/workflow_test.go

```go
func TestOrderSaga_CardDeclinedReleasesStock(t *testing.T) {
    fake := &fakeServices{chargeErrors: []error{ErrCardDeclined}}
    res := runSaga(t, fake, nil)
    assert.Equal(t, "CANCELLED", res.Status)
    // Charged once (no retries), then compensated in reverse order.
    assert.Equal(t, []string{"reserve", "charge", "release", "cancel"},
        fake.calls)
}
```

`runSaga` is a small helper that creates the test environment, registers `&Activities{...}` with the fakes and runs `OrderSaga`. Other tests cover the happy path, transient payment errors (charged three times, then confirmed) and a confirm step that always fails, using `env.OnActivity` to mock a single activity:

OUTPUT

```
--- PASS: TestOrderSaga_HappyPath (0.07s)
--- PASS: TestOrderSaga_CardDeclinedReleasesStock (0.00s)
--- PASS: TestOrderSaga_RetriesTransientPaymentErrors (0.00s)
--- PASS: TestOrderSaga_ConfirmFailsRefundsAndReleases (0.01s)
ok      example.com/goguide/ch11/saga   1.855s
```

Running it for real

In production, a **worker** process polls a task queue on the Temporal server and runs your workflow and activity code:

GO saga/cmd/worker/main.go

```go
    w := worker.New(c, saga.TaskQueue, worker.Options{})
    w.RegisterWorkflow(saga.OrderSaga)
    svc := logOnly{slog.Default()}
    w.RegisterActivity(&saga.Activities{Inventory: svc, Payments: svc,
        Orders: svc})
    if err := w.Run(worker.InterruptCh()); err != nil { // until Ctrl+C
        log.Fatalln("worker:", err)
    }
```

Your HTTP handler starts a saga with `c.ExecuteWorkflow(ctx, client.StartWorkflowOptions{ID: "order-" + orderID, TaskQueue: saga.TaskQueue}, saga.OrderSaga, input)`. Using the order ID as the workflow ID means a retried HTTP request can't start a second saga for the same order. Locally, `temporal server start-dev` from the Temporal CLI gives you a server and a web UI where you can watch each step.

Recap

- Default to **at-least-once delivery plus idempotent consumers**; "exactly-once" end to end is a myth.

- Use **franz-go** for Kafka: idempotent producers by default, keys for per-entity ordering, and a poll, process, commit loop with `BlockRebalanceOnPoll` and a deferred `AllowRebalance`.
- Retry briefly with backoff, mark permanent errors, and dead-letter poison messages so partitions keep moving; test it all against `kfake`.
- amqp091-go gives you RabbitMQ with publisher confirms and manual acks, but reconnection is your job (or Watermill's).
- Watermill brings Spring Cloud Stream-style handlers and middleware (retry, poison queue, recoverer) to Go.
- Never dual-write: put events in an **outbox** table in the same transaction, and relay them with `FOR UPDATE SKIP LOCKED`.
- Make consumers idempotent with a processed-messages table keyed on a stable message ID.
- Give background goroutines a lifecycle (context + `WaitGroup`), use River for durable or once-only jobs, and Temporal for multi-step sagas with compensation.

Try it yourself

Exercise 1: Publish OrderCancelled

Add `PublishOrderCancelled` to `kafka/producer.go`, keyed by order ID, with an `event-type` header of `OrderCancelled`. Extend the demo's handler to switch on the header and print something different for each type. *Hint:* loop over `r.Headers` to find `events.TypeHeader`.

Exercise 2: Clean up the outbox

Published outbox rows pile up forever. Write a periodic job with `jobs.RunEvery` that deletes rows published more than seven days ago, in batches of 1,000, and test it with a fake. *Hint:* `DELETE FROM outbox WHERE id IN (SELECT id FROM outbox WHERE published_at < now() - interval '7 days' LIMIT 1000)`.

Exercise 3: Wire the inbox to Kafka

Connect `inbox.PointsHandler` to the Kafka consumer loop: read the `message-id` header that the outbox publisher adds, decode the `OrderPlaced` payload and call `Handle`. Treat a missing header as a permanent error. *Hint:* the `TestRelayToKafka` test shows how to read headers back from kfake.

Exercise 4: A shipping step in the saga (stretch)

Add a `ScheduleShipment` activity after `ConfirmOrder`, with a `CancelShipment` compensation. Write a test where scheduling the shipment fails permanently, and check that the refund and the stock release run in reverse order, followed by `CancelOrder`. *Hint:* follow the `undo` pattern, and use `env.OnActivity` to make the new activity fail.

Further reading

- franz-go: the README, examples folder and the `kgo` package docs explain groups, commits and transactions.
- Spring for Apache Kafka reference: useful for mapping the Java concepts you know.
- amqp091-go: the RabbitMQ team's Go client, with notes on confirms and reconnection.
- Watermill documentation: routers, middleware and the list of pub/sub adapters.
- Transactional outbox pattern: Chris Richardson's short write-up of the pattern.
- River documentation: transactional enqueueing, periodic jobs and retries.
- Temporal Go SDK developer guide: workflows, activities, testing and determinism rules.
- Saga pattern: choreography versus orchestration, with diagrams.

CHAPTER 12

Security

> *Security is a set of small, boring habits, and once you know them Go makes most of them easy to keep.*

🎯 **What you'll learn**

- Tell authentication from authorization, and return 401, 403 and 404 correctly
- Validate JWTs with golang-jwt and go-oidc, and log users in with OIDC and PKCE
- Enforce "who may do what" with middleware, service-layer checks and policy engines
- Hash passwords, generate secrets, and configure TLS and mutual TLS
- Defend against injection, XSS, CSRF, path traversal, SSRF and slow clients
- Keep known vulnerabilities out of your dependencies and secrets out of your logs

The big picture

In Spring Boot, adding `spring-boot-starter-security` quietly locks every endpoint, adds security headers, turns on CSRF protection and, once you set an issuer, validates tokens. Convenient, but it can feel like magic.

Go has no standard security framework. You compose standard-library building blocks and a few focused libraries as ordinary middleware. Nothing is on by default, so you need the checklist; the good news is that every protection is a few lines you can read and test.

🧩 **Think of it like a hotel**

A hotel protects guests in layers. The front door keeps out the street (TLS, timeouts, size limits). Reception checks your ID (authentication). Your key card opens only your room (authorization). The room safe protects your passport even if someone gets in (hashing). No layer is perfect, so the hotel uses them all. Security people call this **defence in depth**.

The layers in front of an Orderly handler (the examples split them across two programs):

```
request
  v  security headers .. every response gets HSTS, nosniff, CSP...
  v  CORS .............. answers browser preflights for allowed origins
  v  CSRF check ........ rejects cross-site POSTs sent by browsers
  v  body limit ........ caps request bodies at 1 MiB
  v  JWT middleware .... who are you?                  (401 if unknown)
  v  RequireRole ....... may your role call this route? (403 if not)
  v  handler -> service: is this YOUR order?           (404 if not)
```

Java ↔ Go at a glance

Concept	Java / Spring	Go
Security framework	Spring Security filter chain	ordinary `net/http` middleware
JWT validation	`oauth2ResourceServer().jwt()`	`golang-jwt/jwt/v5` or `go-oidc`
OIDC login	`oauth2Login()`	`x/oauth2` + `go-oidc`
Method security	`@PreAuthorize`	role middleware + service checks
Password hashing	`BCryptPasswordEncoder`	`x/crypto/bcrypt`, `argon2`
Secure random	`SecureRandom`	`crypto/rand`
CSRF and CORS	`http.csrf()`, `http.cors()`	`http.CrossOriginProtection`, `rs/cors`
Dependency scan	OWASP Dependency-Check, Snyk	`govulncheck`

Full code: `examples/ch12-security/` *(one folder per topic)*

> **Version note**
>
> The examples pin the newest library releases that still support Go 1.25: golang-jwt v5.3.1, go-oidc v3.21.0, `x/oauth2` v0.36.0, `x/crypto` v0.55.0, rs/cors v1.11.1 and Casbin v3.11.0. Newer `golang.org/x` releases require Go 1.26. On Go 1.26 or later, upgrade them; the code doesn't change.

Authentication vs authorization

These two sound alike and get mixed up all the time:

- **Authentication** (AuthN) answers "who are you?" You prove an identity with a password, a token or a certificate.
- **Authorization** (AuthZ) answers "what may you do?" Given who you are, may you cancel *this* order?

HTTP has a status code for each failure:

Situation	Status	Meaning
No token, or a bad one	`401 Unauthorized`	"I don't know who you are"
Valid token, wrong role	`403 Forbidden`	"I know you, and the answer is no"
Someone else's order	`404 Not Found`	"As far as you're concerned, it doesn't exist"

Yes, `401` is named "Unauthorized" but means "unauthenticated"; everybody finds that confusing. The `404` for someone else's order is deliberate: a `403` would confirm that it exists.

Validating JWTs: your API as a resource server

Orderly clients log in at an **identity provider** (IdP) such as Keycloak, Microsoft Entra ID, Auth0 or Okta, get an **access token**, and send it with every request. Your API is a **resource server**: it logs nobody in, it only checks tokens.

A **JWT** (JSON Web Token, "jot") is three base64url chunks joined by dots: a header naming the signing algorithm, a payload of **claims** (facts about the caller) and a signature. A demo token's payload, decoded:

JSON

```json
{
  "roles": ["customer"],
  "iss": "https://id.orderly.example",
  "sub": "cus-9",
  "aud": ["orderly-api"],
  "exp": 1793646917,
  "iat": 1793643317
}
```

`iss` is the issuer, `sub` the subject (user ID), `aud` the audience (which API it's for) and `exp` the expiry in Unix seconds. A JWT is *signed, not encrypted*: anyone can read it, nobody can change it unnoticed. Never put secrets in claims.

The Spring way

Spring Boot needs the issuer, the audience and a filter chain:

YAML application.yaml

```yaml
spring:
  security:
    oauth2:
      resourceserver:
        jwt:
          issuer-uri: https://id.orderly.example
          audiences: orderly-api
```

JAVA SecurityConfig.java

```java
@Configuration
@EnableMethodSecurity
public class SecurityConfig {
    @Bean
    SecurityFilterChain api(HttpSecurity http) throws Exception {
        return http
            .authorizeHttpRequests(a -> a
                .requestMatchers("/healthz").permitAll()
                .anyRequest().authenticated())
            .oauth2ResourceServer(o -> o.jwt(Customizer.withDefaults()))
            .build();
    }
}
```

The Go way: a verifier with every check spelled out

In Go we build a `jwt.Parser` with the rules we want, one option per check:

GO jwtauth/auth.go

```go
// Claims is what Orderly expects inside an access token.
type Claims struct {
    Roles []string `json:"roles"`
    jwt.RegisteredClaims
}

// Verifier checks access tokens issued by our identity provider.
type Verifier struct {
    parser *jwt.Parser
    key    *rsa.PublicKey
}

func NewVerifier(key *rsa.PublicKey, issuer, audience string) *Verifier {
    p := jwt.NewParser(
        jwt.WithValidMethods([]string{"RS256"}), // pin the algorithm
        jwt.WithIssuer(issuer),                  // who made the token
        jwt.WithAudience(audience),              // who the token is for
        jwt.WithExpirationRequired(),            // no "exp", no entry
        jwt.WithLeeway(30*time.Second),          // tolerate clock skew
    )
    return &Verifier{parser: p, key: key}
}
```

Two of those options surprise people:

- **Algorithm pinning.** The token's header names its algorithm, and the attacker writes that header. In the classic "algorithm confusion" attack, they switch `RS256` to `HS256` and sign with your *public* key as if it were a shared secret, or they send `alg: none`. `WithValidMethods` refuses anything but the algorithm your IdP really uses.
- **Leeway** tolerates clocks that disagree. Your server and the IdP are never perfectly in sync, and 30 seconds of slack avoids rejecting a token issued "in the future".

`Verify` calls `parser.ParseWithClaims` with our public key and returns typed `*Claims`. Its errors work with `errors.Is` (`jwt.ErrTokenExpired` ...): log them, but don't send them to the caller.

Putting the claims in the context

Spring keeps the user in `SecurityContextHolder`, a thread-local. Go has no thread-locals, so the request's `context.Context` carries the claims, under a key of our own unexported type:

GO jwtauth/auth.go

```go
// claimsKey is unexported, so no other package can read or overwrite it.
type claimsKey struct{}

func WithClaims(ctx context.Context, c *Claims) context.Context {
    return context.WithValue(ctx, claimsKey{}, c)
}

// ClaimsFrom returns the verified claims stored by the middleware.
func ClaimsFrom(ctx context.Context) (*Claims, bool) {
    c, ok := ctx.Value(claimsKey{}).(*Claims)
    return c, ok
}
```

> ⚠️ **Gotcha: string keys in context**
>
> `context.WithValue(ctx, "claims", c)` compiles, but any package can reuse the string and overwrite your value. A private struct type as the key can't collide.

The middleware

The middleware finds the bearer token, verifies it and passes the claims on (`bearerToken` and `unauthorized` are tiny helpers in the same file). Notice how little it tells the caller when things go wrong:

GO jwtauth/auth.go

```go
func (v *Verifier) Middleware(next http.Handler) http.Handler {
    return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
        raw, ok := bearerToken(r)
        if !ok {
            unauthorized(w, `Bearer realm="orderly"`)
            return
        }
        claims, err := v.Verify(raw)
        if err != nil {
            // Log the real reason for us; tell the caller very little.
            slog.WarnContext(r.Context(), "token rejected", "err", err)
            unauthorized(w, `Bearer realm="orderly", error="invalid_token"`)
            return
        }
        ctx := WithClaims(r.Context(), claims)
        next.ServeHTTP(w, r.WithContext(ctx))
    })
}
```

To get Spring's "everything is locked unless I say otherwise", put public routes on an outer mux and mount the whole `/api/` tree behind the middleware. Any new endpoint under `/api/` is then protected automatically:

GO jwtauth/orders.go

```go
api := http.NewServeMux()
api.HandleFunc("GET /api/v1/orders/{id}", h.getOrder)
api.HandleFunc("POST /api/v1/orders/{id}/cancel", h.cancelOrder)
api.Handle("POST /api/v1/products",
    RequireRole("admin", http.HandlerFunc(h.createProduct)))

mux := http.NewServeMux()
mux.HandleFunc("GET /healthz", func(w http.ResponseWriter, _ *http.Request) {
    _, _ = w.Write([]byte("ok\n"))
})
mux.Handle("/api/", v.Middleware(api))
```

Trying it out

The demo creates its own signing key and prints dev tokens (in production only your IdP holds the private key). `$FORGED` is the customer's token with the payload edited to say `"admin"`:

TERMINAL

```
curl -i localhost:8080/api/v1/orders/ord-42
curl -H "Authorization: Bearer $CUS9" localhost:8080/api/v1/orders/ord-42
curl -i -X POST -H "Authorization: Bearer $FORGED" \
  localhost:8080/api/v1/products
```

OUTPUT

```
HTTP/1.1 401 Unauthorized
Www-Authenticate: Bearer realm="orderly"
unauthorized

{"id":"ord-42","customer_id":"cus-9","status":"CONFIRMED","total_cents":2598}

HTTP/1.1 401 Unauthorized
Www-Authenticate: Bearer realm="orderly", error="invalid_token"
unauthorized
```

And the server's log (timestamp trimmed) explains the forgery, for our eyes only:

OUTPUT

```
WARN token rejected err="token signature is invalid: crypto/rsa: verification error"
```

Testing the middleware

Security code deserves tests that *attack* it: start from valid claims, break one thing per case, and run the real middleware with `httptest`:

GO jwtauth/auth_test.go

```go
tests := []struct {
    name  string
    token string
    want  int
}{
    {"valid token", sign(t, rs256, testKey, validClaims("cus-9")), 200},
    {"expired", sign(t, rs256, testKey, expired), 401},
    {"wrong audience", sign(t, rs256, testKey, wrongAud), 401},
    {"signed by someone else", sign(t, rs256, otherKey,
        validClaims("cus-9")), 401},
    // The classic "algorithm confusion" attack: sign with HMAC,
    // using the server's PUBLIC key as the shared secret.
    {"HS256 with public key", sign(t, jwt.SigningMethodHS256, pubPEM,
        validClaims("cus-9")), 401},
    {"alg none", sign(t, jwt.SigningMethodNone,
        jwt.UnsafeAllowNoneSignatureType, validClaims("cus-9")), 401},
}
```

All nine cases pass (the table also covers a missing token, a wrong issuer and a missing `exp`), and `go test -v` lists each attack by name.

> **The Go way**
>
> Spring Security's defaults are good, but hidden in auto-configuration. In Go, every rule is a line you wrote, so a reviewer can see the algorithm is pinned and the audience checked. Forgetting a line is on you, which is why attack-style tests matter so much.

OpenID Connect: keys from your identity provider

The verifier above holds one public key, but identity providers **rotate** their keys, so a hard-coded key means an outage on rotation day. **OpenID Connect** (OIDC), a thin identity layer on top of OAuth 2.0, standardises two things that fix this:

- **Discovery:** the IdP publishes its settings at `<issuer>/.well-known/openid-configuration`.
- **JWKS** (JSON Web Key Set): the IdP publishes its current public keys at a URL listed there. Each token's header carries a `kid` (key ID) naming the key that signed it.

The `coreos/go-oidc` library reads both, caches the keys, and re-fetches them when it meets an unknown `kid`, exactly what Spring's `issuer-uri` does.

Verifying API tokens against JWKS

For a resource server, ask the provider for a verifier with your API's audience as `ClientID`:

GO oidc/main.go

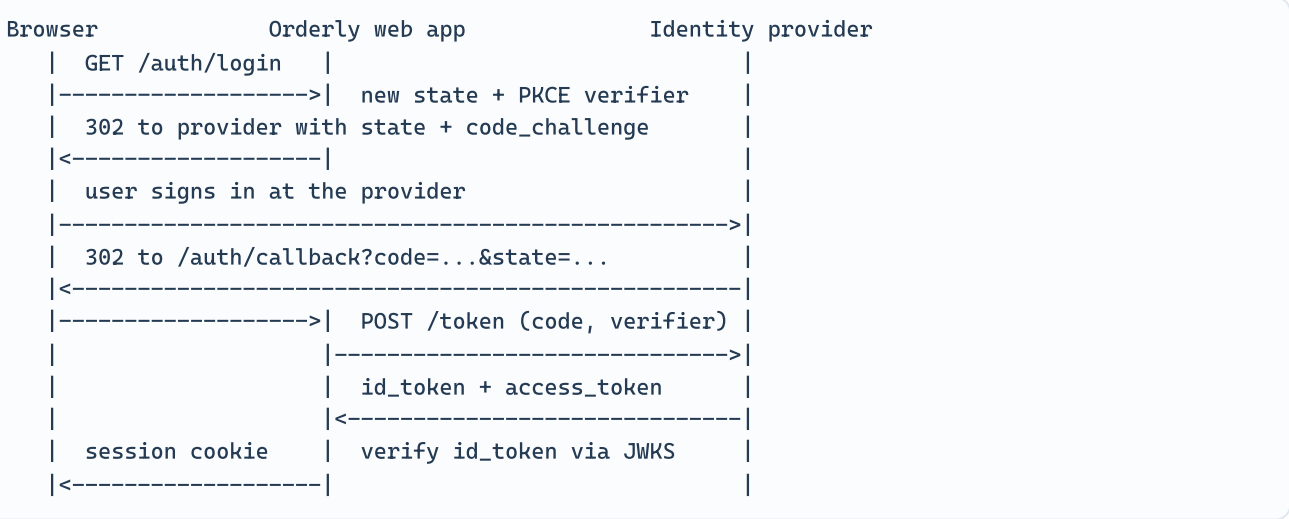
```go
// NewAPIVerifier is the resource-server side: it checks the signature
// against the provider's JWKS (cached, refreshed when a new key ID shows
// up), plus issuer, expiry and audience.
func NewAPIVerifier(ctx context.Context, issuer, audience string) (
    *oidc.IDTokenVerifier, error) {
    provider, err := oidc.NewProvider(ctx, issuer)
    if err != nil {
        return nil, err
    }
    return provider.Verifier(&oidc.Config{ClientID: audience}), nil
}
```

`verifier.Verify(ctx, raw)` returns an `*oidc.IDToken` whose `Claims(&v)` method decodes custom claims such as `roles`. Despite the name, `IDTokenVerifier` works for any JWT your provider signs, access tokens included, as long as the audience matches. It only accepts the algorithms the provider advertises.

> 💡 **If you're unsure, use go-oidc in production**
>
> Use golang-jwt when you control the keys (tokens you mint yourself, or tests). For tokens from an identity provider, use go-oidc: key rotation then needs no code change and no restart.

The login flow: authorization code + PKCE

Web apps also need to *log users in*. The recommended flow is the **authorization code flow with PKCE** (Proof Key for Code Exchange, "pixie"). Our app creates a random secret (the *verifier*), sends only its hash (the *challenge*) to the provider, and later proves it knows the original, so a stolen one-time code is useless.



In Spring this is `http.oauth2Login()`. In Go, `golang.org/x/oauth2` has PKCE helpers built in:

GO oidc/main.go

```go
// login sends the browser to the provider with a fresh state and PKCE pair.
func (a *App) login(w http.ResponseWriter, r *http.Request) {
    state := rand.Text()                  // stops forged callbacks (CSRF)
    verifier := oauth2.GenerateVerifier() // PKCE secret, kept by us
    setTempCookie(w, "oidc_state", state)
    setTempCookie(w, "oidc_pkce", verifier)
    url := a.oauth.AuthCodeURL(state, oauth2.S256ChallengeOption(verifier))
    http.Redirect(w, r, url, http.StatusFound)
}
```

`setTempCookie` writes a five-minute `HttpOnly`, `Secure`, `SameSite=Lax` cookie. The `callback` handler then checks that the returned `state` matches the cookie, calls `a.oauth.Exchange(ctx, code, oauth2.VerifierOption(verifier))` to swap the code for tokens, verifies the `id_token` with go-oidc, and starts the app's own session.

If you also send an OIDC `nonce` (`oidc.Nonce(...)`), compare `idToken.Nonce` yourself; go-oidc leaves that check to you.

> 💡 **Test against a fake provider**
>
> The example's tests start a tiny identity provider with `httptest` (discovery, JWKS and a token endpoint that checks the PKCE verifier), so the real go-oidc and oauth2 code runs end to end with no network. See `oidc/fakeidp_test.go`.

Authorization: who may do what

Now we decide what the caller may do, at two levels: coarse rules per route ("only admins create products") and fine rules per resource ("only the owner cancels an order").

Role checks per route

Spring puts the rule in an annotation:

JAVA ProductController.java

```java
@PostMapping("/api/v1/products")
@PreAuthorize("hasRole('ADMIN')")
public ProductDto create(@RequestBody @Valid NewProduct p) {
    return products.create(p);
}
```

In Go it's a small middleware that reads the claims from the context; we already wrapped the products route with it:

GO jwtauth/auth.go

```go
// RequireRole lets a request through only if the token has the role.
func RequireRole(role string, next http.Handler) http.Handler {
    return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
        c, ok := ClaimsFrom(r.Context())
        if !ok || !c.HasRole(role) {
            http.Error(w, "forbidden", http.StatusForbidden)
            return
        }
        next.ServeHTTP(w, r)
    })
}
```

The Go rule sits in the route table next to the path it protects, so one file shows every route and its guard. There's no proxy, so no self-invocation trap (in Spring, calling a `@PreAuthorize` method from the same class skips the check). The flip side: the guard belongs to the route, so code reaching the logic another way isn't covered. One more reason to keep ownership rules in the service.

Resource-level checks belong in the service

"Is this your order?" can't be answered from the URL; you need the order. Spring teams often write a bean and call it from SpEL:

JAVA OrderAccess.java

```java
@Component("orderAccess")
public class OrderAccess {
    private final OrderRepository orders; // constructor omitted

    public boolean canRead(UUID orderId, Authentication auth) {
        boolean support = auth.getAuthorities().stream()
            .anyMatch(a -> a.getAuthority().equals("ROLE_SUPPORT"));
        return support || orders.findById(orderId)
            .map(o -> o.getCustomerId().toString().equals(auth.getName()))
            .orElse(false);
    }
}
// on the controller:
// @PreAuthorize("@orderAccess.canRead(#id, authentication)")
```

In Go, the rule is plain code in the service, where business rules live anyway. The handler turns the claims into a `Principal`, a small type with no HTTP or JWT details, and passes it explicitly:

GO jwtauth/orders.go

```go
// canAccess is the resource-level rule: your own orders, or support staff.
func canAccess(p Principal, o Order) bool {
    return o.CustomerID == p.ID || p.Is("support")
}

func (s *OrderService) Get(ctx context.Context, p Principal,
    id string) (Order, error) {
```

```
    s.mu.Lock()
    defer s.mu.Unlock()
    o, ok := s.orders[id]
    if !ok || !canAccess(p, o) {
        // Same answer for "missing" and "not yours", so nobody can
        // probe for other customers' order IDs.
        return Order{}, ErrOrderNotFound
    }
    return o, nil
}
```

You can test this rule with no tokens, HTTP or framework: `svc.Get(ctx, Principal{ID: "cus-7"}, "ord-42")` must return `ErrOrderNotFound`. Over HTTP, customer `cus-7` gets `404 Not Found` for that order, and a customer posting to `/api/v1/products` gets `403 Forbidden`.

> **Key idea**
>
> Broken access control is number one in the OWASP Top 10. Route-level role checks are not enough: every query that loads a customer's data must also check *whose* data it is. In SQL, that often means `WHERE id = $1 AND customer_id = $2`.

Policy engines: Casbin and OPA

When rules multiply, or non-developers need to change them, a **policy engine** moves the rules out of Go code into data.

Casbin (now an Apache Incubator project; the module path is still `github.com/casbin/casbin/v3`) reads a model file describing *how* to decide (here: roles plus URL patterns) and a policy file listing *who* may do *what*:

casbin/policy.csv

```
p, customer, /api/v1/orders/:id, GET
p, customer, /api/v1/orders/:id/cancel, POST
p, support, /api/v1/orders/:id, GET
p, admin, /api/v1/products, POST
g, support, customer
g, adm-1, admin
```

The `g` lines are role inheritance: support staff get every customer permission, and user `adm-1` is an admin. `casbin.NewEnforcer(model, adapter)` builds an enforcer once; `e.Enforce(sub, obj, act)` answers each question:

OUTPUT

```
customer GET  /api/v1/orders/ord-42        allowed=true
customer POST /api/v1/products             allowed=false
support  POST /api/v1/orders/ord-42/cancel allowed=true
adm-1    POST /api/v1/products             allowed=true
```

Look at the third line. Support may call the cancel *route* (inherited from customer), yet our service lets only the *owner* cancel. A URL policy can't know who owns an order, so the service-layer check stays.

OPA (Open Policy Agent, a CNCF graduated project) uses a policy language called Rego. This Orderly policy was checked with OPA's own evaluator:

authz.rego

```
package orderly.authz

default allow := false

# Support staff may read any order.
allow if {
    input.action == "read"
    "support" in input.user.roles
}

# Customers may read and cancel their own orders.
allow if {
    input.action in {"read", "cancel"}
    input.order.customer_id == input.user.id
}
```

Teams run OPA as a sidecar and `POST` the input to `/v1/data/orderly/authz/allow`, or embed it with the `github.com/open-policy-agent/opa/v1/rego` package. Current OPA releases need Go 1.26, so it isn't in the tested examples.

> 💡 **If you're unsure, start with plain Go**
>
> Role middleware plus ownership checks in the service covers most services and is the easiest to test. Reach for **Casbin** when admins edit permissions stored in a database, or tenants define their own roles. Reach for **OPA** when a platform team wants one policy language across many services.

Passwords and randomness

If Orderly stores passwords, a thief who copies the table must get nothing useful. That means a **slow, salted hash**: a deliberately expensive one-way function, mixed with a random **salt** so identical passwords give different hashes. Never use fast hashes such as SHA-256 for passwords, and never encrypt them (encryption can be reversed).

JAVA

```
var encoder =
    new BCryptPasswordEncoder(12);
String hash = encoder.encode(pw);
boolean ok =
    encoder.matches(pw, hash);
```

GO

```
h, err := bcrypt.GenerateFromPassword(
    []byte(pw), 12)
err = bcrypt.CompareHashAndPassword(
    h, []byte(pw))
ok := err == nil
```

Both write the same `$2a$` format, so Go can verify hashes from your Spring app. The example's `CheckBcrypt` only has to strip the `{bcrypt}` prefix that Spring's `DelegatingPasswordEncoder` adds.

Argon2id is the newer, memory-hard alternative: every guess needs lots of RAM, which hurts attackers with GPUs. `x/crypto/argon2` provides the raw `argon2.IDKey` function, so the example stores salt and parameters with the hash in the standard PHC string format (as Spring's `Argon2PasswordEncoder` does) and checks with `subtle.ConstantTimeCompare`, so timing reveals nothing. The demo:

OUTPUT

```
bcrypt:   $2a$12$cjuAxCTUCeYd2FzDmsEDMeln/8L/HDm4vpZ8RoxdbJi.s48WIh96i
          took 329ms; right: true, wrong: false
argon2id: $argon2id$v=19$m=19456,t=2,p=1$4m/85L9ecxt6igcLwOgYIQ$PK4Tk/Lded...
          took 32ms; right: true
reset token: WDESNNEXKAE23DNCZ2THR73A25
API key:     ok_RrQpleMupOpg3gSO39w9SyHuGdcPnCCTB5z0iAZzt3k
```

> 💡 **If you're unsure, use bcrypt with cost 12**
>
> It's one call each way, has no memory knob to get wrong, and reads your existing Spring hashes. Choose Argon2id for a new system that can afford the memory per login; the OWASP minimum used here is 19 MiB, 2 passes, 1 thread.

> ⚠️ **Gotcha: bcrypt's 72-byte limit**
>
> bcrypt only reads the first 72 bytes of a password. Go returns `bcrypt.ErrPasswordTooLong` instead of silently truncating, so handle that error, or cap passphrase length in validation.

crypto/rand, never math/rand, for secrets

Reset tokens, API keys, session IDs and salts must be unguessable. Java has `SecureRandom`; Go has `crypto/rand`:

JAVA

```java
var random = new SecureRandom();
byte[] key = new byte[32];
random.nextBytes(key);
String token = Base64.getUrlEncoder()
    .withoutPadding()
    .encodeToString(key);
```

GO

```go
key := make([]byte, 32)
rand.Read(key) // crypto/rand
token := base64.RawURLEncoding.
    EncodeToString(key)

// or, since Go 1.24:
token = rand.Text()
```

Since Go 1.24, `crypto/rand.Read` never returns an error (on the rare legacy system where the OS generator could fail, the program crashes rather than continue with weak randomness), and `rand.Text()` returns base32 text with at least 128 bits of randomness.

> ⚠️ **Gotcha: two packages called rand**
>
> `math/rand` and `math/rand/v2` are fast and fine for simulations and retry jitter, but they aren't designed for secrets, and their documentation says not to use them for security-sensitive work. Check your imports: gosec flags `math/rand` as G404.

TLS and mutual TLS

TLS (Transport Layer Security, the "S" in HTTPS) encrypts traffic and proves the server's identity with a certificate. Spring Boot uses `server.ssl.*` properties; Go uses a `tls.Config`. Go's defaults are excellent, so the safest config mostly avoids weakening them:

GO mtls/tlsconfig.go

```go
// ServerTLSConfig is a sane default for an HTTPS server. Go's defaults
// are already good, so the main job is to not weaken them.
func ServerTLSConfig(cert tls.Certificate) *tls.Config {
    return &tls.Config{
        MinVersion:   tls.VersionTLS12, // the default today; say it out loud
        Certificates: []tls.Certificate{cert},
        // No CipherSuites or CurvePreferences: Go picks safe ones, and
        // you get improvements (like post-quantum key exchange in Go
        // 1.24) for free when you upgrade.
    }
}
```

TLS 1.2 has been the server default minimum since Go 1.22; stating it documents intent and guards against a `GODEBUG` override. Don't copy old blog posts that list cipher suites: TLS 1.3 suites aren't configurable, Go's TLS 1.2 list is curated, and since Go 1.24 the default key exchange includes the hybrid post-quantum `X25519MLKEM768`.

Mutual TLS between services

In normal TLS only the server shows a certificate. With **mutual TLS** (mTLS) the client shows one too, like both people showing ID at a meeting, so the inventory service knows the caller really is the billing service. In Spring:

YAML application.yaml

```yaml
server:
  ssl:
    key-store: classpath:inventory.p12
    key-store-password: ${KEYSTORE_PASSWORD}
    trust-store: classpath:internal-ca.p12
    trust-store-password: ${TRUSTSTORE_PASSWORD}
    client-auth: need
```

In Go it's two more server fields. `ClientCAs` holds your internal **CA** (certificate authority, the issuer of your certificates):

GO mtls/tlsconfig.go

```go
// MTLSServerConfig also demands a client certificate from our own CA.
func MTLSServerConfig(cert tls.Certificate,
    clientCAs *x509.CertPool) *tls.Config {
    cfg := ServerTLSConfig(cert)
    cfg.ClientAuth = tls.RequireAndVerifyClientCert
    cfg.ClientCAs = clientCAs
    return cfg
}
```

The client, `MTLSClient`, is an `http.Client` whose `tls.Config` sets `Certificates` (ours) and `RootCAs` (only the internal CA). The handler reads the caller's name from `r.TLS.PeerCertificates[0].Subject.CommonName`. The demo makes a throwaway CA in memory and calls the server three ways:

OUTPUT

```
with client cert:     200 hello, billing-service (TLS 1.3)
without client cert:  error: tls: certificate required
old TLS 1.1 client:   error: tls: protocol version not supported
```

> 💡 **Let the platform do mTLS when it can**
>
> In Kubernetes, a service mesh such as Istio or Linkerd adds mTLS between pods with no code change, and cert-manager issues and rotates certificates. Do mTLS in Go when there's no mesh or the app must act on the caller's certificate, and load certificates via `tls.Config.GetCertificate` so they rotate without a restart.

Defending against common web attacks

Now the classic attacks. Each defence is small; the skill is remembering them all. The code lives in `examples/ch12-security/webrisks/`.

SQL injection

SQL injection happens when user input becomes part of the SQL text and changes the query's meaning. The cure is the same as `JdbcTemplate` with `?`: placeholders.

GO Avoid

```go
q := fmt.Sprintf(
    "SELECT id FROM orders WHERE customer_id = '%s'", customerID)
rows, err := pool.Query(ctx, q) // customerID = "x' OR '1'='1"
```

GO Prefer

```go
rows, err := pool.Query(ctx,
    "SELECT id FROM orders WHERE customer_id = $1", customerID)
```

The value travels separately from the SQL text, so it can never become SQL. pgx uses `$1, $2`; MySQL drivers use `?`. With **sqlc** (Chapter 9) the generated code always uses placeholders. Identifiers such as column names can't be placeholders, so dynamic sorting needs an **allow-list**:

GO webrisks/sqlsafe.go

```go
// sortOrders maps user-facing sort names to SQL. Identifiers such as
// column names can't be placeholders, so they come from this allow-list.
var sortOrders = map[string]string{
    "newest": "created_at DESC",
    "oldest": "created_at ASC",
}
```

The request says `?sort=newest`; the SQL only receives a string we wrote. Shell commands are similar: `exec.Command("convert", userFile)` runs the program with no shell in between, so there's nothing to inject into. Avoid `sh -c` with user input.

Cross-site scripting (XSS)

XSS is injection into HTML: an attacker saves `<script>` as their name, and your page runs it in other users' browsers. Go's `html/template` escapes data for the exact spot it lands in, like Thymeleaf's `th:text`; its twin `text/template` escapes nothing. Same template, same hostile input:

OUTPUT

```
text/template:
<p>Hello, <script>alert("pwned")</script>!</p>
<a href="javascript:alert(1)">my site</a>
html/template:
<p>Hello, &lt;script&gt;alert(&#34;pwned&#34;)&lt;/script&gt;!</p>
<a href="#ZgotmplZ">my site</a>
```

`html/template` even knows an `href` is a URL and replaces the `javascript:` URL with the harmless marker `#ZgotmplZ`. For JSON APIs, `encoding/json` escapes `<`, `>` and `&` by default.

> ⚠️ **Gotcha: template.HTML switches escaping off**
>
> Converting a string to `template.HTML` tells the engine "this is already safe", like `th:utext`. Never do it with anything a user typed.

Cross-site request forgery (CSRF)

CSRF tricks a logged-in user's browser into sending a request they didn't intend: an evil page posts a form to `/api/v1/orders/42/cancel`, and the browser attaches the user's session cookie. Spring Security fights this with CSRF tokens, on by default.

Go 1.25 added `http.CrossOriginProtection`. It reads headers every modern browser sends (`Sec-Fetch-Site`, falling back to `Origin`) and rejects unsafe methods (POST, PUT, DELETE...) from other sites, with no tokens to embed in forms:

GO webrisks/main.go

```go
// CSRF: block state-changing requests sent by other sites' pages.
csrf := http.NewCrossOriginProtection()
if err := csrf.AddTrustedOrigin(shopOrigin); err != nil {
    return nil, err
}
```

A POST carrying `Sec-Fetch-Site: cross-site` gets `403 Forbidden` with the body "cross-origin request detected from Sec-Fetch-Site header". Requests without those headers (`curl`, other services) pass, because they aren't browsers carrying someone's cookies. GET, HEAD and OPTIONS always pass, so never change state in a GET handler.

> 📝 **Do bearer-token APIs need CSRF protection?**
>
> Browsers attach cookies automatically but never an `Authorization: Bearer` header, so a pure token API isn't exposed. Once you use session cookies (like the OIDC login above), you need it. It's cheap, so many teams turn it on everywhere.

CORS: what it is and what it isn't

Browsers enforce the **same-origin policy**: JavaScript on `shop.orderly.example` may *send* requests to `api.orderly.example` but not *read* the responses. **CORS** (Cross-Origin Resource Sharing) is how your API tells the browser "this origin may read my responses", usually after an `OPTIONS` **preflight** request.

CORS relaxes a browser rule. It is *not* a security control for your server: it doesn't stop `curl` or other servers, and it doesn't replace authentication. The `rs/cors` middleware handles the details:

GO webrisks/main.go

```go
// CORS: let our own shop front end read responses from this API.
c := cors.New(cors.Options{
    AllowedOrigins: []string{shopOrigin},
    AllowedMethods: []string{http.MethodGet, http.MethodPost},
    AllowedHeaders: []string{"Authorization", "Content-Type"},
    MaxAge:         600, // seconds a browser may cache the preflight
})
```

OUTPUT

```
HTTP/1.1 204 No Content
Access-Control-Allow-Headers: authorization,content-type
Access-Control-Allow-Methods: POST
Access-Control-Allow-Origin: https://shop.orderly.example
Access-Control-Max-Age: 600
```

List exact origins; avoid `"*"` for anything that uses cookies or returns private data.

Security headers

Spring Security adds protective response headers by default. Go adds none, so the example's `SecurityHeaders` middleware sets them with `w.Header().Set(...)` before calling the next handler:

Header	Value	Protects against
`Strict-Transport-Security`	`max-age=63072000; includeSubDomains`	downgrades to plain HTTP
`X-Content-Type-Options`	`nosniff`	JSON being sniffed as HTML
`X-Frame-Options`	`DENY`	clickjacking in iframes
`Content-Security-Policy`	`default-src 'none'; frame-ancestors 'none'`	injected scripts
`Referrer-Policy`	`no-referrer`	URLs leaking to other sites
`Cache-Control`	`no-store`	personal data in caches

Wrap it outermost so error responses get the headers too. A server-rendered site needs a looser `Content-Security-Policy` than this API-only one.

Request size limits and timeouts

Without limits, one client can send a 10 GB body or hold connections open forever. `http.MaxBytesReader` caps a body; reading past the cap returns `*http.MaxBytesError`, which becomes a `413`:

GO webrisks/limits.go

```go
// in the LimitBody middleware:
r.Body = http.MaxBytesReader(w, r.Body, maxBytes)

// in the handler, after json.Decoder.Decode fails:
var tooBig *http.MaxBytesError
if errors.As(err, &tooBig) {
    http.Error(w, "request body too large",
        http.StatusRequestEntityTooLarge)
    return
}
```

A **slowloris** attack opens many connections and sends headers a byte at a time, tying up the server. An `http.Server{}` with no timeouts waits forever, so set them:

GO webrisks/main.go

```go
srv := &http.Server{
    Addr:              "127.0.0.1:8080",
    Handler:           h,
    ReadHeaderTimeout: 5 * time.Second,  // slowloris defence
    ReadTimeout:       15 * time.Second, // whole request incl. body
    WriteTimeout:      30 * time.Second, // writing the response
    IdleTimeout:       60 * time.Second, // idle keep-alive connections
    MaxHeaderBytes:    64 << 10,         // 64 KiB of headers is plenty
}
```

gosec flags a missing `ReadHeaderTimeout` as G112. Chapter 14 extends "timeouts everywhere" to clients and databases, and adds rate limiting.

Path traversal

Path traversal means a filename like `../../etc/passwd` escapes the folder you meant to serve. `filepath.Join` won't save you: it *cleans* the path, so `filepath.Join("/invoices", "../etc/passwd")` is `/etc/passwd`. Go 1.24 added `os.Root`, a directory handle whose methods refuse to leave it, even through symbolic links:

GO webrisks/files.go

```go
// invoiceHandler serves files from one directory and nothing outside it.
func invoiceHandler(invoices *os.Root) http.HandlerFunc {
    return func(w http.ResponseWriter, r *http.Request) {
        name := r.URL.Query().Get("file") // "ord-42.txt", or an attack
        f, err := invoices.Open(name)     // "../x" fails: escapes the root
        if err != nil {
            slog.Warn("invoice not served", "err", err) // err names the file
            http.NotFound(w, r)
            return
        }
        defer f.Close()
        // ... check it's a regular file, then io.Copy(w, f)
    }
}
```

Asking for `?file=../../etc/passwd` gets a `404`, and the log (timestamp trimmed) says why:

OUTPUT

```
WARN invoice not served err="openat ../../etc/passwd: path escapes from parent"
```

Open the root once at startup with `os.OpenRoot(dir)`; for a static folder, use `http.FileServerFS(root.FS())`.

SSRF, briefly

SSRF (server-side request forgery) happens when your server fetches a user-supplied URL (a webhook, an avatar) and the user points it inside: at `localhost:6060/debug/pprof`, or the cloud metadata service at `169.254.169.254`, which hands out credentials. Prefer an allow-list of destinations; failing that, refuse internal addresses at dial time:

GO webrisks/ssrf.go

```go
// Control runs after the DNS lookup, on the exact IP about to be
// dialled, so "evil.example -> 10.0.0.5" is caught as well.
Control: func(network, address string, _ syscall.RawConn) error {
    ap, err := netip.ParseAddrPort(address)
    if err != nil {
        return err
    }
    if ip := ap.Addr().Unmap(); !isPublic(ip) {
        return fmt.Errorf("ssrf guard: %s is not public", ip)
    }
    return nil
},
```

Checking the *resolved* IP inside the dialer also defeats **DNS rebinding** (a name that resolves to a public IP when checked and a private one when used). Treat it as a solid start, not a complete defence.

The OWASP Top 10 in Go

The **OWASP Top 10** is the industry's list of the most critical web application risks. Here's the 2025 edition, mapped to Go:

OWASP 2025	Main Go defences
A01 Broken Access Control	deny by default; ownership checks in services
A02 Security Misconfiguration	headers, timeouts, private admin ports (Ch 13)
A03 Software Supply Chain Failures	go.sum, checksum DB, govulncheck in CI (Ch 18)
A04 Cryptographic Failures	TLS 1.2+, crypto/rand, bcrypt or Argon2id
A05 Injection	placeholders, sqlc (Ch 9), html/template, no shell
A06 Insecure Design	ownership rules, rate limits (Ch 14), safe defaults
A07 Authentication Failures	pinned JWT checks, OIDC with PKCE
A08 Software or Data Integrity Failures	checksum DB, verified token signatures
A09 Security Logging and Alerting Failures	structured audit logs, redaction (Ch 13)
A10 Mishandling of Exceptional Conditions	check every error, fail closed, recover panics (Ch 14)

"Fail closed" means that when something unexpected happens, such as a policy engine timing out, you deny the request rather than allow it.

Supply chain security

Your service is mostly other people's code. **Supply chain security** means making sure that code is what you think it is, and free of known holes. Out of the box, Go's module system does more here than a typical Maven setup.

> 🧩 **Think of it like a tamper-evident seal**
>
> `go.sum` lists the seals you recorded when you first opened each jar. The **checksum database** (sum.golang.org) is a public notary holding the same seals for everyone, in an append-only log. If anyone, even the author, swaps the contents of a published version, the seal no longer matches and Go refuses it.

The `go` command checks every download against `go.sum`, and checks module versions new to your `go.sum` against the checksum database. `go mod verify` re-checks your local module cache:

TERMINAL

```
go mod verify
```

OUTPUT

```
all modules verified
```

For private modules, set `GOPRIVATE` (for example `go env -w GOPRIVATE='github.com/acme-corp/*'`) so their names never reach the public proxy or checksum database.

govulncheck: only what you can reach

OWASP Dependency-Check and similar scanners match versions against CVE lists, so they flag every vulnerable jar on the classpath, whether or not you call the broken method. **govulncheck**, from the Go team, builds a call graph and reports only the vulnerabilities your code can reach. On this chapter's examples:

TERMINAL

```
go run golang.org/x/vuln/cmd/govulncheck@latest ./...
```

OUTPUT

```
=== Symbol Results ===

No vulnerabilities found.

Your code is affected by 0 vulnerabilities.
This scan also found 0 vulnerabilities in packages you import and 3
vulnerabilities in modules you require, but your code doesn't appear to call
these vulnerabilities.
```

The three unreachable findings are in `x/crypto/ssh` and `x/crypto/openpgp`, which share a module with `bcrypt` but which we never import. A version-matching scanner would have failed the build three times.

govulncheck also checks your toolchain's standard library. Scanning the same code with the original Go 1.25.0 reported 45 reachable standard-library vulnerabilities, all fixed in later 1.25 patch releases. It exits with status 3 when it finds something, which fails a CI step.

> 💡 **Keep your Go patch version current**
>
> Most Go vulnerabilities are in the standard library and get fixed in patch releases, so moving from 1.25.0 to the latest 1.25.x is the cheapest security win there is. Run govulncheck in CI, and on a schedule for code nobody touches. (The latest govulncheck and gosec need Go 1.26 to build; with the default `GOTOOLCHAIN=auto`, `go run ...@latest` fetches a newer toolchain for them.)

gosec, and fewer dependencies

gosec looks for security smells such as weak randomness, missing timeouts and loose file permissions; it also runs inside golangci-lint (Chapter 16). A confession: its first run on this chapter's examples found six issues. Two were real (a handler writing a certificate name without a `Content-Type`, which a browser could sniff as HTML, and an unchecked integer conversion), one was our deliberately outdated TLS 1.1 client, and three were minor. After the fixes, plus two `#nosec` annotations with reasons:

OUTPUT

```
Summary:
  Gosec  : dev
  Files  : 18
  Lines  : 1195
  Nosec  : 2
  Issues : 0
```

Always write the reason after `--`, as in `// #nosec G402 -- outdated demo client`.

Finally, the cheapest dependency to secure is the one you don't add. The standard library covers HTTP, TLS, crypto, JSON and templates, so ask whether 30 lines of your own code would do. `go mod why -m <module>` shows why something is in your build.

Keeping secrets out of logs

Logs get copied everywhere, so a token in a log line is a leaked token. Chapter 10 built a `Secret` type whose `String`, `GoString`, `LogValue` and JSON methods all print `[REDACTED]`, and Chapter 13 explains the `slog.LogValuer` mechanism. This chapter's example uses the same type:

GO redact/main.go (excerpt)

```go
type Secret string

// LogValue is used by slog, whatever the handler.
func (Secret) LogValue() slog.Value { return slog.StringValue(redacted) }

// Reveal is the one, easy-to-grep way to get the real value.
func (s Secret) Reveal() string { return string(s) }
```

Security adds two habits. The first is a **safety net** for plain strings that slip through: a `ReplaceAttr` function that blanks well-known keys.

GO redact/main.go

```go
func redactKeys(_ []string, a slog.Attr) slog.Attr {
    if sensitiveKeys[strings.ToLower(a.Key)] {
        return slog.String(a.Key, redacted)
    }
    return a
}
```

A config struct holding a `Secret`, a careless plain-string password, and the struct via `fmt`:

OUTPUT

```
level=INFO msg=connecting db="{Host:db User:orderly Password:[REDACTED]}"
level=INFO msg="login attempt" user=ana password=[REDACTED]
fmt: {db orderly [REDACTED]}
```

The second is a **test that hunts for the real value** in every output path, so a refactoring can't quietly undo the protection:

GO redact/main_test.go

```go
js, _ := json.Marshal(cfg)
outputs := map[string]string{
    "logs": logs.String(), // text and JSON handlers
    "%v":   fmt.Sprintf("%v", cfg),
    "%+v":  fmt.Sprintf("%+v", cfg),
    "%#v":  fmt.Sprintf("%#v", cfg),
    "json": string(js),
}
for name, out := range outputs {
    if strings.Contains(out, "hunter2") {
        t.Errorf("%s leaked the password: %s", name, out)
    }
}
```

> 💡 **A few more logging habits**
>
> Never log whole request headers (they hold `Authorization` and cookies) or bodies. Log the token's subject, not the token. Keep client-facing errors generic and put the detail in the server log.

Recap

- Authentication is "who are you?" (`401`); authorization is "what may you do?" (`403`, or `404` to hide other people's data).
- Validate JWTs with every check spelled out: pinned algorithm, issuer, audience, required expiry, small leeway. Keep claims in the context under an unexported key type.
- With an identity provider, use go-oidc (keys from JWKS, safe rotation) and the authorization code flow with PKCE.
- Check roles in middleware and ownership in the service. Use Casbin or OPA when rules outgrow code.
- Hash passwords with bcrypt (cost 12) or Argon2id; make secrets with `crypto/rand`, never `math/rand`.
- Keep Go's TLS defaults with TLS 1.2 as the minimum; use mTLS (or a mesh) between services.
- Use placeholders, `html/template`, `CrossOriginProtection`, exact CORS origins, security headers, body limits, server timeouts and `os.Root`.
- Run govulncheck and gosec in CI, stay on the latest Go patch release, and make secrets redact themselves.

Try it yourself

> 🏋️ **Exercise 1: Scopes as well as roles**
>
> OAuth 2.0 tokens often carry a `scope` claim of space-separated permissions, such as `"orders:read orders:write"`. Add it to `Claims` and write `RequireScope("orders:write")` in `jwtauth`, with a table test.
>
> *Hint:* `strings.Fields` splits the claim; `slices.Contains` checks it.

> 🏋️ **Exercise 2: My orders only**
>
> Add `GET /api/v1/orders` to `jwtauth`. Customers see only their own orders; support sees everything. Put the filter in `OrderService` and test it without HTTP.

> 🏋️ **Exercise 3 (stretch): Survive a key rotation**
>
> Let the fake identity provider in `oidc/fakeidp_test.go` switch to a new key with a new `kid`. Prove that a verifier created *before* the rotation accepts tokens signed *after* it.

Further reading

- Go Security Best Practices: the Go team's checklist, with links to govulncheck and fuzzing.
- Traversal-resistant file APIs: the Go blog post introducing `os.Root`.
- net/http CrossOriginProtection: the exact rules Go 1.25's CSRF protection applies.
- golang-jwt/jwt v5: parser options and error values.
- coreos/go-oidc: provider discovery, verifiers and key sets.
- OWASP Top 10:2025: the full descriptions behind the table above.
- OWASP Password Storage Cheat Sheet: current parameters for bcrypt and Argon2id.
- gosec: the list of rules and how to suppress findings properly.

CHAPTER 13

Observability: Logs, Metrics, Traces and Profiles

> *When something goes wrong at 3 a.m., observability is the difference between guessing and knowing.*

🎯 **What you'll learn**

- Write structured, request-scoped logs with `log/slog`, including trace IDs and redaction
- Expose Prometheus metrics with a small middleware, and avoid the label-cardinality trap
- Trace a request across services with OpenTelemetry, using automatic and manual spans
- Build liveness and readiness endpoints that Kubernetes can trust
- Profile CPU, memory, goroutines and locks with pprof, and catch rare slowdowns with the flight recorder

The big picture

In Spring Boot, observability mostly arrives with dependencies: Actuator, Micrometer and the OpenTelemetry Java agent. Go gives you excellent building blocks instead, and you wire them together yourself. It's a little more work, but nothing is hidden. Every metric, span and log field exists because a line of code you can read put it there.

Observability has four signals, and each answers a different question:

- **Logs** are timestamped records of events: "order ord-42 was placed".
- **Metrics** are numbers measured over time: "we serve 120 requests per second, and 1% fail".
- **Traces** follow one request through your services: "checkout took 900 ms, and 700 ms of that was the inventory call".
- **Profiles** show where your program spends CPU time and memory, down to the function.

🧩 **Think of it like a hospital**

Logs are the nurse's notes: a diary of what happened and when. **Metrics** are the vital-signs monitor by the bed, with live graphs and alarms. **Traces** are the barcode on the patient's wristband, scanned in every department, so you can follow the whole journey. **Profiles** are the MRI scan: an inside view you take when the other signals say something is wrong.

Java ↔ Go at a glance

Concept	Java / Spring	Go
Logging API	SLF4J + Logback	`log/slog` (standard library)
Per-request log fields	MDC (thread-local)	a logger stored in `context.Context`
Metrics	Micrometer + Actuator	`prometheus/client_golang`
Tracing	OpenTelemetry Java agent	OpenTelemetry Go SDK + `otelhttp`
Health checks	Actuator `/health` groups	your own `/healthz` and `/readyz`
Profiling	JFR, async-profiler	`pprof`, `runtime/trace`

What we'll build

The example is a small Orderly service. Each request passes through three middleware layers before it reaches a handler, and profiling lives on a separate, private port:

```
request (public port :8080)
   │
   ▼
otelhttp ........ starts a span, reads the caller's traceparent header
   │
   ▼
requestLogger ... request ID, logger in the context, access log line
   │
   ▼
metrics ......... counts and times each request by route
   │
   ▼
ServeMux ─┬─── GET /api/v1/orders/{id}
          ├─── GET /api/v1/inventory/{productId}
          ├─── GET /healthz, GET /readyz
          └─── GET /metrics

go tool pprof ──► admin port 127.0.0.1:6060 /debug/pprof/   (private)
```

Full code: *`examples/ch13-observability/orderly/`*

> **Version note**
>
> The examples pin the newest releases that still support Go 1.25: `prometheus/client_golang` v1.24.1, OpenTelemetry Go v1.46.0 and `otelhttp` v0.71.0. Newer releases of these libraries require Go 1.26. If you're on Go 1.26 or later, upgrade freely; the code doesn't change.

Logs with log/slog

A plain text line like `Order ord-42 placed by cus-9` is easy for a human to read and hard for a machine to search. **Structured logging** means each line is a message plus named fields (`order_id=ord-42`), usually as JSON, so tools like Loki, Elasticsearch or CloudWatch can filter on them.

Since Go 1.21, the standard library has a structured logger, `log/slog`. Before that, teams picked zap or zerolog. Today `slog` is the default choice, and it's ours.

Your first slog call

JAVA

```java
private static final Logger log =
    LoggerFactory.getLogger(Orders.class);

log.info("Order {} placed, total {}",
    orderId, totalCents);
```

GO

```go
slog.Info("order placed",
    "order_id", orderID,
    "total_cents", totalCents)
```

In Java, values go into a message template. In Go, the message stays constant and the values become key/value pairs. A constant message makes it easy to count "order placed" events, and the fields stay searchable.

> ⚠️ **Gotcha: keys and values must pair up**
>
> `slog.Info("placed", "order_id")` has a key with no value. slog won't crash; it logs `!BADKEY=order_id`. `go vet` has a `slog` check that catches most of these, one more reason to run it in CI (see Chapter 16).

Handlers, levels, attributes and groups

A **handler** decides how and where records are written, like a Logback appender plus encoder. The standard library ships `TextHandler` (`key=value` for humans) and `JSONHandler` (one JSON object per line for machines). Here's a short tour, starting with both handlers and a level you can change while the program runs:

GO slogdemo/main.go (excerpt)

```go
// 1. Text handler: easy for humans to read on a laptop.
text := slog.New(slog.NewTextHandler(os.Stdout, nil))
text.Info("order placed", "order_id", "ord-42", "total_cents", 2598)

// 2. JSON handler with a level we can change while running.
level := new(slog.LevelVar) // starts at Info
log := slog.New(slog.NewJSONHandler(os.Stdout,
    &slog.HandlerOptions{Level: level}))

log.Debug("hidden: below the current level")
level.Set(slog.LevelDebug)
log.Debug("visible now", "reason", "level lowered at runtime")
```

Next come typed attributes, a group, and a child logger:

GO slogdemo/main.go (excerpt)

```go
// 3. Typed attributes and a group.
log.Info("stock reserved",
    slog.String("product_id", "MUG-BLUE-01"),
    slog.Int("quantity", 2),
    slog.Group("warehouse",
        slog.String("id", "ams-1"),
        slog.Int("aisle", 7)),
)

// 4. With: a child logger that always carries request_id.
reqLog := log.With("request_id", "req-7f3a")
reqLog.Warn("payment slow", "elapsed_ms", 1840)
```

The output (long lines wrapped to fit the page):

OUTPUT

```
time=2026-11-02T23:16:02.344+05:30 level=INFO msg="order placed"
  order_id=ord-42 total_cents=2598
{"time":"2026-11-02T23:16:02.3450608+05:30","level":"DEBUG",
  "msg":"visible now","reason":"level lowered at runtime"}
{"time":"2026-11-02T23:16:02.3450608+05:30","level":"INFO",
  "msg":"stock reserved","product_id":"MUG-BLUE-01","quantity":2,
  "warehouse":{"id":"ams-1","aisle":7}}
{"time":"2026-11-02T23:16:02.3455903+05:30","level":"WARN",
  "msg":"payment slow","request_id":"req-7f3a","elapsed_ms":1840}
```

What to notice:

- **Levels** are Debug, Info, Warn and Error. A `slog.LevelVar` can change at runtime, much like Spring Boot's `/actuator/loggers`.
- **Typed attributes** (`slog.String`, `slog.Int`) are slightly faster than loose pairs and can't be mismatched.
- **Groups** nest fields under a name. `log.WithGroup("billing")` does it for every later field.
- `With` returns a child logger that adds fields to every line. You'll use it constantly.

> 💡 **If you're unsure, use JSON in production and text locally**
>
> Choose the handler from configuration (`LOG_FORMAT=text` on your laptop). Call `slog.SetDefault(logger)` once in `main`, and both `slog.Info` and the older `log.Printf` go through your handler.

A request-scoped logger: Go's answer to MDC

You want every line written during a request to carry the same `request_id`. In Spring you'd put it in the **MDC** (Mapped Diagnostic Context), a thread-local map that Logback adds to every line:

JAVA RequestIdFilter.java

```java
@Component
public class RequestIdFilter extends OncePerRequestFilter {
    @Override
    protected void doFilterInternal(HttpServletRequest req,
            HttpServletResponse res, FilterChain chain)
            throws ServletException, IOException {
        String id = Optional.ofNullable(req.getHeader("X-Request-ID"))
                .orElseGet(() -> UUID.randomUUID().toString());
        MDC.put("request_id", id);
        try {
            chain.doFilter(req, res);
        } finally {
            MDC.remove("request_id"); // threads are pooled: clean up!
        }
    }
}
```

Go has no thread-locals, and a request can hop between goroutines. Request-scoped data travels in the `context.Context` instead (see Chapter 5). So we build a child logger and put it in the context, under a **typed key**:

GO orderly/logging.go (excerpt)

```go
// loggerKey is an unexported type, so no other package can collide
// with (or overwrite) our context value.
type loggerKey struct{}

// loggerFrom returns the request-scoped logger, or the default one.
func loggerFrom(ctx context.Context) *slog.Logger {
    if l, ok := ctx.Value(loggerKey{}).(*slog.Logger); ok {
        return l
    }
    return slog.Default()
}
```

A plain string key like `"logger"` could be read or overwritten by any package. An unexported struct type is unique to yours.

The middleware reuses an incoming `X-Request-ID` (from your API gateway, say) or creates one, stores the child logger, and writes one access-log line at the end:

GO orderly/logging.go (excerpt)

```go
func requestLogger(base *slog.Logger, next http.Handler) http.Handler {
    return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
        start := time.Now()
        id := r.Header.Get("X-Request-ID")
        if id == "" || len(id) > 64 {
            id = rand.Text() // 26 random characters (Go 1.24+)
        }
```

```
        w.Header().Set("X-Request-ID", id)

        log := base.With("request_id", id)
        ctx := context.WithValue(r.Context(), loggerKey{}, log)
        inner := r.WithContext(ctx)
        rec := &statusRecorder{ResponseWriter: w, status: http.StatusOK}

        next.ServeHTTP(rec, inner)
        // The mux records the matched route on the request it was given.
        // Copy it back so outer middleware (otelhttp) can see it too.
        r.Pattern = inner.Pattern

        log.LogAttrs(ctx, levelFor(rec.status), "request completed",
            slog.String("method", r.Method),
            slog.String("path", r.URL.Path),
            slog.String("route", routeOf(inner)),
            slog.Int("status", rec.status),
            slog.Float64("duration_ms", millis(time.Since(start))),
        )
    })
}
```

A few helpers live in the example code. `statusRecorder` wraps the `ResponseWriter` to remember the status code. `levelFor` logs 5xx as Error and 4xx as Warn. `routeOf` returns `r.Pattern`, the route the `ServeMux` matched, such as `GET /api/v1/orders/{id}` (Go 1.23+). We'll see why the pattern is copied back in the tracing section.

Handlers then fetch the logger:

GO orderly/orders.go (excerpt)

```
log := loggerFrom(ctx)
// ...
log.InfoContext(ctx, "order fetched",
    "order_id", order.ID, "items", len(order.Items))
```

Passing `ctx` to `InfoContext` looks redundant, but it lets the handler add the trace ID too, as you'll see shortly.

> **The Go way: explicit beats ambient**
>
> Java's MDC works by magic: values appear because of a thread-local. In Go, the logger lives in the context, and the context is a parameter. It feels verbose for a week. Then you notice that background goroutines and worker pools carry the right fields with no extra effort, because the context goes wherever you pass it.

LogValuer: let types redact themselves

Sooner or later someone logs a struct containing an email address. Instead of hoping every caller remembers to mask it, let the type decide. Implement `slog.LogValuer`, and slog calls `LogValue` whenever the value is logged:

GO slogdemo/main.go (excerpt)

```go
// LogValue decides how a Customer appears in every log line.
// slog calls it automatically, so callers cannot forget to mask.
func (c Customer) LogValue() slog.Value {
    return slog.GroupValue(
        slog.String("id", c.ID),
        slog.String("email", maskEmail(c.Email)),
    )
}
```

Logging `"customer", c` now prints a safe version:

OUTPUT

```
{"time":"2026-11-02T23:16:02.3455903+05:30","level":"INFO",
  "msg":"customer signed in","request_id":"req-7f3a",
  "customer":{"id":"cus-9","email":"a***@example.com"}}
```

That's the mechanism; Chapter 12 applies it to secrets such as passwords and tokens. As a second safety net, `slog.HandlerOptions.ReplaceAttr` lets a handler rewrite any attribute by key, for example blanking anything called `password`.

zap, zerolog and slog adapters

zap (from Uber) and **zerolog** were the popular fast loggers before slog, and both are still maintained. They allocate less per line, which matters only when you log tens of thousands of lines per second on a hot path.

> 💡 **If you're unsure, use slog**
>
> Write your code against `*slog.Logger`. If profiling ever shows logging is a bottleneck, keep the API and swap the handler: `go.uber.org/zap/exp/zapslog` provides a slog handler backed by zap. Use zap or zerolog directly when you join a codebase that already does.

Metrics with Prometheus

Logs describe *one* request. Metrics describe *all* of them, cheaply: incrementing a counter costs nanoseconds, however busy you are. That's why alerts are built on metrics.

Prometheus is the most common metrics system in Kubernetes shops. It **scrapes**: every 15 seconds or so it calls `GET /metrics` on your service and stores the numbers. The Go library is `github.com/prometheus/client_golang`.

Counter, gauge and histogram in plain words

Type	What it is	Orderly example	Analogy
Counter	only goes up	requests served	a car's odometer
Gauge	goes up and down	requests in flight	a fuel gauge
Histogram	observations sorted into buckets	request duration	parcels sorted by size

A histogram counts how many requests took under 5 ms, under 10 ms, under 25 ms and so on. From those buckets, Prometheus estimates percentiles like "99% finished within 120 ms" across all pods.

Micrometer and Actuator vs client_golang

Spring Boot times every HTTP request for you (`http.server.requests`), and you add business metrics through Micrometer:

JAVA OrderService.java (excerpt)

```java
private final Counter ordersPlaced;

public OrderService(MeterRegistry registry) {
    this.ordersPlaced = Counter.builder("orders.placed")
            .description("Orders placed")
            .register(registry);
}

public Order place(PlaceOrder cmd) {
    // ... reserve stock, save the order ...
    ordersPlaced.increment();
    return order;
}
```

In Go, nothing is timed automatically. You write a small middleware once, on a registry you control:

GO orderly/metrics.go (excerpt)

```go
func newHTTPMetrics(reg prometheus.Registerer) *httpMetrics {
    f := promauto.With(reg) // registers each metric as it is created
    return &httpMetrics{
        requests: f.NewCounterVec(prometheus.CounterOpts{
            Name: "http_requests_total",
            Help: "HTTP requests served, by route and status code.",
        }, []string{"route", "code"}),
        duration: f.NewHistogramVec(prometheus.HistogramOpts{
            Name:    "http_request_duration_seconds",
            Help:    "Time taken to serve HTTP requests.",
            Buckets: prometheus.DefBuckets, // 5ms .. 10s
        }, []string{"route"}),
        inFlight: f.NewGauge(prometheus.GaugeOpts{
            Name: "http_requests_in_flight",
            Help: "Requests being served right now.",
        }),
    }
}
```

A `CounterVec` is a family of counters, one per combination of **label** values. Labels are the dimensions you slice by later. Names follow Prometheus conventions: base units (`_seconds`) and `_total` on counters.

The middleware records one observation per request:

GO orderly/metrics.go (excerpt)

```go
func (m *httpMetrics) middleware(next http.Handler) http.Handler {
    return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
        m.inFlight.Inc()
        defer m.inFlight.Dec()

        start := time.Now()
        rec := &statusRecorder{ResponseWriter: w, status: http.StatusOK}
        next.ServeHTTP(rec, r) // the mux sets r.Pattern in here

        route := routeOf(r)
        code := strconv.Itoa(rec.status)
        m.requests.WithLabelValues(route, code).Inc()
        m.duration.WithLabelValues(route).
            Observe(time.Since(start).Seconds())
    })
}
```

The /metrics endpoint

We create our own registry with the standard Go runtime and process collectors (goroutines, GC, memory, CPU), and serve it:

GO orderly/metrics.go and main.go (excerpts)

```go
reg := prometheus.NewRegistry()
reg.MustRegister(
    collectors.NewGoCollector(),
    collectors.NewProcessCollector(collectors.ProcessCollectorOpts{}),
)
// ...
mux.Handle("GET /metrics", promhttp.HandlerFor(reg, promhttp.HandlerOpts{}))
```

After a few requests, `curl localhost:8080/metrics` shows (trimmed):

OUTPUT

```
http_requests_in_flight 1
http_requests_total{code="200",route="GET /api/v1/inventory/{productId}"} 1
http_requests_total{code="200",route="GET /api/v1/orders/{id}"} 1
http_requests_total{code="404",route="GET /api/v1/orders/{id}"} 1
http_requests_total{code="404",route="unmatched"} 1
http_request_duration_seconds_bucket{route="GET /api/v1/orders/{id}",le="0.05"} 2
http_request_duration_seconds_sum{route="GET /api/v1/orders/{id}"} 0.03077
http_request_duration_seconds_count{route="GET /api/v1/orders/{id}"} 2
```

> 💡 **Prefer your own registry over the global one**
>
> `promhttp.Handler()` serves the global registry, which any imported package can add to. A registry you create and pass around is easier to test, since each test gets a fresh one.

Label cardinality: the one metrics mistake to avoid

Cardinality is the number of distinct label combinations. Each combination is its own **time series**, a stream of numbers Prometheus keeps in memory. Ten routes times five status codes is fifty series: fine. A `customer_id` label with a million customers is millions of series, and Prometheus slows down or runs out of memory.

GO Avoid

```
// One new time series per order, per customer, per URL. Unbounded!
requests.WithLabelValues(r.URL.Path, customerID, err.Error()).Inc()
```

GO Prefer

```
// Bounded: a fixed set of route patterns and status codes.
requests.WithLabelValues(r.Pattern, strconv.Itoa(status)).Inc()
```

Good labels have a small, fixed set of values: route patterns, status codes, `result="ok|error"`. IDs, raw URLs and error messages belong in logs and traces. Our test checks exactly this with the `testutil` package:

GO orderly/metrics_test.go (excerpt)

```
for _, path := range []string{
    "/api/v1/orders/ord-1", "/api/v1/orders/ord-2", "/nope"} {
    h.ServeHTTP(httptest.NewRecorder(),
        httptest.NewRequest("GET", path, nil))
}

// Two different IDs, but one time series: the route is the pattern.
ok := m.requests.WithLabelValues("GET /api/v1/orders/{id}", "200")
if got := testutil.ToFloat64(ok); got != 2 {
    t.Errorf("orders 200 count = %v, want 2", got)
}
if n := testutil.CollectAndCount(m.requests); n != 2 {
    t.Errorf("got %d series, want 2 (watch label cardinality!)", n)
}
```

RED and USE: what to measure

Two short checklists cover most needs:

Method	Use it for	Measure
RED	services (requests come in)	**R**ate, **E**rrors, **D**uration
USE	resources (CPU, pools, queues)	**U**tilization, **S**aturation, **E**rrors

Our middleware already gives you RED. For USE, think of a database pool: busy connections (utilization), callers waiting (saturation) and failed acquisitions (errors). In PromQL, Prometheus's query language:

```
# Rate and error ratio
sum by (route) (rate(http_requests_total[5m]))
sum(rate(http_requests_total{code=~"5.."}[5m]))
  / sum(rate(http_requests_total[5m]))

# 99th percentile latency per route
histogram_quantile(0.99,
  sum by (le, route) (rate(http_request_duration_seconds_bucket[5m])))
```

Distributed tracing with OpenTelemetry

Metrics say checkout is slow. Logs say which requests were slow. Neither says *where* the time went once a request fans out to other services. That's what tracing is for.

- A **trace** is the whole journey of one request, identified by a **trace ID**.
- A **span** is one timed step, such as "handle GET /orders/{id}" or "call inventory", with attributes and a status. Each span records its **parent**, so tools like Jaeger, Tempo or Honeycomb can draw a waterfall.
- **Context propagation** carries the trace across processes: the caller sends a W3C `traceparent` header (`00-<trace-id>-<span-id>-01`), and the callee continues the same trace.

OpenTelemetry (OTel) is the vendor-neutral standard for all of this: instrument once, send to any backend.

No bytecode agent: what's different from Java

In Java you add `-javaagent:opentelemetry-javaagent.jar`, and the agent rewrites bytecode as classes load, instrumenting Spring MVC, JDBC and Kafka without code changes.

Go compiles to a native binary, with no class loading to hook into. So you instrument explicitly: set up a **tracer provider** in `main`, wrap servers and clients with **instrumentation libraries** such as `otelhttp` (there are others for gRPC, database drivers and Kafka), and add **manual spans** where you want detail. It's a few dozen lines, and you control every span name.

> **Zero-code options exist, but they're young**
>
> The OpenTelemetry Go eBPF auto-instrumentation project (`go.opentelemetry.io/auto`) instruments a running binary from outside using Linux eBPF. Its beta was announced in 2025, its README still says "work in progress", and it needs elevated privileges. OpenTelemetry also hosts OBI, a cross-language eBPF agent (pre-1.0), and `otelc`, which adds instrumentation at build time. Worth watching; the SDK is the reliable default today.

Setting up the tracer provider

The **tracer provider** creates tracers, batches finished spans and hands them to an **exporter**. Locally we print spans with `stdouttrace`. In production we use OTLP, the OpenTelemetry protocol, usually sending to an OpenTelemetry Collector:

GO orderly/tracing.go (excerpt)

```
if os.Getenv("OTEL_EXPORTER_OTLP_ENDPOINT") != "" {
    // Reads endpoint, headers and TLS settings from OTEL_* env vars.
    return otlptracehttp.New(ctx)
}
return stdouttrace.New(stdouttrace.WithWriter(w))
```

Then the provider itself:

GO orderly/tracing.go (excerpt)

```
res, err := resource.New(ctx,
    resource.WithAttributes(semconv.ServiceName("orderly")),
    resource.WithFromEnv(), // OTEL_SERVICE_NAME etc. win
    resource.WithTelemetrySDK(),
)
if err != nil {
    return nil, err
}

tp := sdktrace.NewTracerProvider(
    sdktrace.WithBatcher(exporter), // send spans in batches
    sdktrace.WithResource(res),
    sdktrace.WithSampler(sdktrace.ParentBased(
        sdktrace.AlwaysSample())), // lower this in production
)
otel.SetTracerProvider(tp)

// Without this line, trace context is NOT sent to or read from
// other services: the default global propagator does nothing.
otel.SetTextMapPropagator(propagation.NewCompositeTextMapPropagator(
    propagation.TraceContext{}, propagation.Baggage{}))

return tp.Shutdown, nil
```

The new terms:

- The **resource** says *who* is sending: service name, SDK version, and anything in `OTEL_RESOURCE_ATTRIBUTES`.
- The **sampler** decides which traces to keep. `ParentBased` follows the caller's decision, so traces are never half-recorded. In production, `sdktrace.TraceIDRatioBased(0.1)` keeps 10%.
- The **propagator** reads and writes `traceparent`. Forgetting it is the most common reason traces break at service boundaries.
- `tp.Shutdown` flushes buffered spans. Chapter 14 places it in the shutdown sequence.

Instrumenting HTTP with otelhttp

`otelhttp.NewHandler` starts a server span per request and joins the caller's trace if a `traceparent` header arrives. It goes outermost, so the span is in the context before our logger runs:

GO orderly/main.go (excerpt)

```go
var h http.Handler = newHTTPMetrics(reg).middleware(mux)
h = requestLogger(log, h)
return otelhttp.NewHandler(h, "orderly",
    otelhttp.WithFilter(notOpsEndpoint)) // don't trace probes
```

`WithFilter` skips `/healthz`, `/readyz` and `/metrics`, because nobody wants thousands of health-check traces.

`otelhttp` names spans after the route, like `GET /api/v1/orders/{id}`, by reading `r.Pattern` after the handler returns. But `ServeMux` sets `Pattern` on the request *it* receives, and our logger passed it a copy made by `r.WithContext`. That's why `requestLogger` copies the pattern back. Without that line, every span would be named plain `GET`.

> ⚠️ **Gotcha: middleware that copies the request hides the route**
>
> `r.WithContext(...)` returns a new `*http.Request`. Fields set on that copy, like the matched `Pattern`, don't flow back to outer middleware. If an outer layer needs them, copy them back after `next.ServeHTTP` returns.

Manual spans: attributes and errors

Automatic spans show the edges of your service. For the steps inside, start your own:

GO orderly/orders.go (excerpt)

```go
var tracer = otel.Tracer("example.com/goguide/ch13/orderly")

func (a *app) loadOrder(ctx context.Context, id string) (Order, error) {
    ctx, span := tracer.Start(ctx, "order.load")
    defer span.End()
    span.SetAttributes(attribute.String("order.id", id))

    order, err := a.store.Order(ctx, id)
    if err != nil {
        span.RecordError(err)
        span.SetStatus(codes.Error, "loading order failed")
        return Order{}, err
    }
    span.SetAttributes(attribute.Int("order.item_count", len(order.Items)))
    return order, nil
}
```

Three habits to build:

- **`defer span.End()`** right after `Start`. A span that never ends is never exported.
- **Pass the new `ctx` down.** Anything started from it becomes a child span.

- **Record errors twice.** `RecordError` attaches the error as an event; `SetStatus` marks the span failed, so it shows red in your tracing UI.

High-cardinality attributes like `order.id` are fine on spans, even though they'd be a disaster as metric labels. Here's the span from the stdout exporter, trimmed:

JSON

```
{
  "Name": "order.load",
  "SpanContext": { "TraceID": "a73e190d1699efd71dfa82ce3a0a1ecc",
                   "SpanID": "f9d143ed48a328a8" },
  "Parent": { "SpanID": "7bce841ff3eb1afc" },
  "Attributes": [
    { "Key": "order.id", "Value": { "Type": "STRING", "Value": "ord-42" } },
    { "Key": "order.item_count", "Value": { "Type": "INT64", "Value": 1 } }
  ],
  "Status": { "Code": "Unset", "Description": "" }
}
```

For a missing order, the span gets `"Code": "Error"` and an `exception` event saying `order not found`. Whether "not found" is really an error is your call; many teams mark only unexpected failures, so the error rate stays meaningful.

Propagating context across HTTP calls

The order handler asks the inventory API for stock levels. To keep that call in the same trace, wrap the client transport with `otelhttp.NewTransport`, which starts a client span and injects `traceparent`:

GO orderly/orders.go (excerpt)

```
func newInventoryClient(baseURL string) *inventoryClient {
    t := http.DefaultTransport.(*http.Transport).Clone()
    t.MaxIdleConnsPerHost = 64 // the default of 2 is tiny for services
    return &inventoryClient{
        baseURL: baseURL,
        hc: &http.Client{
            Timeout:   2 * time.Second,
            Transport: otelhttp.NewTransport(t),
        },
    }
}
```

Build requests with `http.NewRequestWithContext(ctx, ...)` so the transport can find the current span. Here's the tree exported for one `GET /api/v1/orders/ord-42`. The inventory API runs in the same process in the example, but the header crosses real HTTP, exactly as between two services:

```
trace a73e190d1699efd71dfa82ce3a0a1ecc
└─ GET /api/v1/orders/{id}                server span   7bce841f...
   ├─ order.load                          internal span f9d143ed...
   └─ HTTP GET                            client span   5d6fad56...
      └─ GET /api/v1/inventory/{productId} server span   7223c19d...
```

`tracing_test.go` proves it: it installs an in-memory `tracetest.SpanRecorder`, calls the order endpoint, and asserts that both server spans share one trace ID.

Putting trace_id in your logs

Traces and logs become far more useful when you can jump between them. A small `slog.Handler` wrapper copies the IDs from the context into every record:

GO orderly/logging.go (excerpt)

```go
type traceHandler struct{ slog.Handler }

func (h traceHandler) Handle(ctx context.Context, r slog.Record) error {
    if sc := trace.SpanContextFromContext(ctx); sc.IsValid() {
        r.AddAttrs(
            slog.String("trace_id", sc.TraceID().String()),
            slog.String("span_id", sc.SpanID().String()),
        )
    }
    return h.Handler.Handle(ctx, r)
}

// WithAttrs and WithGroup must return a traceHandler too, otherwise
// child loggers (logger.With) would silently lose the trace IDs.
func (h traceHandler) WithAttrs(attrs []slog.Attr) slog.Handler {
    return traceHandler{h.Handler.WithAttrs(attrs)}
}
```

Embedding `slog.Handler` means we override only what we need (Chapter 4). Two log lines from one order request, trimmed: different request IDs, because they're two HTTP requests, but the same `trace_id`:

OUTPUT

```
{"level":"INFO","msg":"stock looked up",
  "request_id":"Z236VOMRE4N7IV3MOXTKJFUS3R","available":17,
  "trace_id":"a73e190d1699efd71dfa82ce3a0a1ecc","span_id":"7223c19dcc28bec4"}
{"level":"INFO","msg":"request completed",
  "request_id":"IMI5PHCHNJV3EESSDTRH4Z5OLV","route":"GET /api/v1/orders/{id}",
  "status":200,"duration_ms":30.77,
  "trace_id":"a73e190d1699efd71dfa82ce3a0a1ecc","span_id":"7bce841ff3eb1afc"}
```

With a little configuration in Grafana, Kibana or Datadog, you can click a slow span and see its log lines. Spring Boot 3 does the same by putting `traceId` and `spanId` into the MDC through Micrometer Tracing. If you'd rather ship logs over OTLP, the `go.opentelemetry.io/contrib/bridges/otelslog` package provides a slog handler for that; most teams start with JSON on stdout.

OTel metrics or the Prometheus client?

OpenTelemetry has a metrics API too, and `otelhttp` records HTTP metrics through it once you configure a meter provider. Which should you use?

	Prometheus client	OTel metrics
How data leaves	Prometheus scrapes `/metrics`	pushed over OTLP, or scraped via an exporter

	Prometheus client	OTel metrics
Best fit	Prometheus and Grafana stacks	OTLP-first vendors and Collectors
One SDK for all signals	no	yes

> 💡 **If you're unsure, use the Prometheus client for metrics and OTel for traces**
>
> That's this book's house pick, and it matches most Kubernetes platforms. Switch to OTel metrics when your organisation has standardised on an OpenTelemetry Collector or an OTLP-native vendor. Don't record the same metric both ways: double cost, confusing charts.

Health endpoints: liveness vs readiness

Kubernetes asks your pod two different questions, and mixing them up causes outages.

> 🧩 **Think of it like a corner shop**
>
> **Liveness** asks "is the shopkeeper conscious?" If not, call an ambulance (restart the container). **Readiness** asks "is the shop open right now?" If the delivery hasn't arrived, keep the door shut for a while (send no traffic), but skip the ambulance.

Spring Boot's Actuator exposes `/actuator/health/liveness` and `/actuator/health/readiness`. In Go, you write two small handlers:

GO orderly/health.go (excerpt)

```go
// liveness answers "is this process alive and not stuck?". It checks
// nothing external: if the database is down, restarting us won't help.
func liveness(w http.ResponseWriter, _ *http.Request) {
    w.Header().Set("Content-Type", "text/plain; charset=utf-8")
    w.Write([]byte("ok\n"))
}

// readiness answers "should the load balancer send me traffic now?".
// It runs every dependency check with a short overall deadline.
func readiness(timeout time.Duration, checks ...Check) http.HandlerFunc {
    return func(w http.ResponseWriter, r *http.Request) {
        ctx, cancel := context.WithTimeout(r.Context(), timeout)
        defer cancel()

        status, results := http.StatusOK, map[string]string{}
        for _, c := range checks {
            if err := c.Run(ctx); err != nil {
                status = http.StatusServiceUnavailable
                results[c.Name] = "failing: " + err.Error()
                continue
            }
            results[c.Name] = "ok"
        }
        // ... write status and results as JSON ...
    }
}
```

A `Check` is a name plus a `func(ctx) error`: `pool.Ping` for Postgres, `client.Ping(ctx).Err()` for Redis. The tests cover a healthy check, a failing one, and a slow one that hits the deadline, so the probe gets a 503 instead of hanging.

TERMINAL

```
$ curl -i localhost:8080/readyz
HTTP/1.1 200 OK
Content-Type: application/json

{"checks":{"store":"ok"},"status":"OK"}
```

YAML deployment.yaml (excerpt)

```yaml
livenessProbe:
  httpGet: { path: /healthz, port: 8080 }
  periodSeconds: 10
readinessProbe:
  httpGet: { path: /readyz, port: 8080 }
  periodSeconds: 5
```

> ⚠️ **Gotcha: never check dependencies in liveness**
>
> If liveness pings the database and the database has a blip, Kubernetes restarts *every* pod at once. That doesn't fix the database, and now you have two outages. During shutdown, readiness should fail *first* so traffic drains; Chapter 14 builds that.

Profiling: seeing inside a running program

Metrics might say CPU is at 90%. A **profile** says *which functions* burn that CPU or hold that memory. Go's profiler, **pprof**, is built into the runtime and cheap enough for production. Where Java reaches for JFR or async-profiler, Go has these always available:

Profile	Answers	Java analogue
`profile` (CPU)	where is CPU time spent?	JFR, async-profiler
`heap` / `allocs`	what holds memory, and what allocates it?	JFR allocations, heap dump
`goroutine`	what is every goroutine doing now?	`jstack` thread dump
`mutex` / `block`	where do goroutines wait on locks or channels?	JFR lock events

Exposing pprof safely

`net/http/pprof` serves these profiles over HTTP. We mount it on a separate admin server listening only on `127.0.0.1`:

GO orderly/main.go (excerpt)

```go
func adminRoutes() http.Handler {
    // Mutex and block profiles are off by default; turn on sampling.
    runtime.SetMutexProfileFraction(10)                // 1 in 10 contention events
    runtime.SetBlockProfileRate(int(time.Millisecond)) // ~1 per ms blocked

    mux := http.NewServeMux()
    mux.HandleFunc("/debug/pprof/", pprof.Index) // heap, goroutine, ...
    mux.HandleFunc("/debug/pprof/cmdline", pprof.Cmdline)
    mux.HandleFunc("/debug/pprof/profile", pprof.Profile) // CPU
    mux.HandleFunc("/debug/pprof/symbol", pprof.Symbol)
    mux.HandleFunc("/debug/pprof/trace", pprof.Trace) // runtime/trace
    return mux
}
```

> ⚠️ **Gotcha: importing net/http/pprof publishes it on the default mux**
>
> Tutorials often write `import _ "net/http/pprof"`, which registers the handlers on `http.DefaultServeMux`. If your public server uses that mux, anyone can download your profiles and command line. Keep pprof on a private port; in Kubernetes, reach it with `kubectl port-forward pod/<name> 6060:6060`.

Reading go tool pprof output

Let's profile Orderly while the example's tiny load generator sends requests:

TERMINAL

```
go run ./loadgen -d 12s &        # 16 workers calling GET /api/v1/orders/ord-42
go tool pprof -top -cum \
  "http://127.0.0.1:6060/debug/pprof/profile?seconds=5"
```

Here's a real story from writing this chapter. The first inventory client closed each response body without reading it to the end, and used the default transport, which keeps only two idle connections per host. The profile pointed straight at it (trimmed):

OUTPUT

```
Duration: 5s, Total samples = 24.88s (485.39%)
      flat  flat%   sum%        cum   cum%
         0     0% 49.04%      5.01s 20.14%  net/http.(*conn).serve
         0     0% 65.55%      3.24s 13.02%  net.(*sysDialer).dialTCP
         0     0% 65.55%      3.21s 12.90%  syscall.Closesocket
```

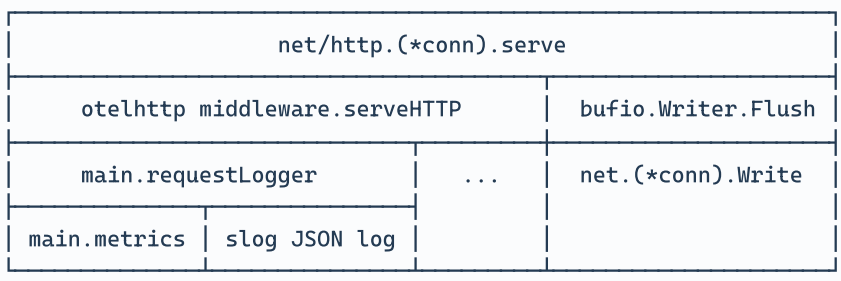
A quarter of the CPU went to opening and closing TCP connections. Draining each body before closing it and raising `MaxIdleConnsPerHost` (in the service and the load generator) made `dialTCP` vanish, and throughput on the same laptop rose from 2,195 to 6,071 requests per second.

- **flat** is time spent *in* a function itself.
- **cum** (cumulative) includes everything it calls.
- Sort by `cum` to find the expensive path, then by `flat` to find the hot function.

> 📝 **Your numbers will differ**
>
> On Windows, system calls appear under `runtime.cgocall` even with cgo disabled; on Linux you'll see `syscall.Syscall` and friends. The method is the same everywhere.

Flame graphs

`go tool pprof -http=:8081 <profile URL or file>` opens a web UI; choose *View → Flame Graph*. Each box is a function, its **width** is its share of samples, and the boxes below it are what it called. Here's a hand-drawn sketch of Orderly's shape:



The same UI shows heap profiles. Add `-sample_index=alloc_space` to see what allocates most, which is often the key to reducing GC work (Chapter 17 goes deeper).

> 💡 **Hunting goroutine leaks**
>
> `curl "http://127.0.0.1:6060/debug/pprof/goroutine?debug=1"` groups goroutines by stack trace, with counts. One stack with 40,000 goroutines and growing is a leak, usually a goroutine stuck sending on a channel nobody reads.

runtime/trace and the flight recorder

pprof samples *where* time goes. The **execution tracer** records *what happened, in order*: goroutines starting and blocking, GC pauses, network waits. It answers "why did this request wait 200 ms while the CPU was idle?". Grab five seconds from the admin port with `curl -o trace.out ".../debug/pprof/trace?seconds=5"`, then run `go tool trace trace.out`.

The catch is timing: the slow request happens once an hour, and you can't trace all day. Go 1.25 added `runtime/trace.FlightRecorder`, an aircraft-style black box. It keeps the last few seconds of trace in memory, and you save them only when something interesting happens:

GO flightrec/main.go (excerpt)

```go
fr := trace.NewFlightRecorder(trace.FlightRecorderConfig{
    MinAge:   5 * time.Second, // keep at least the last 5s
    MaxBytes: 8 << 20,         // but no more than about 8 MiB
})
if err := fr.Start(); err != nil {
    log.Fatal(err)
}
defer fr.Stop()
// ... later, when a request is slower than expected:
if elapsed > 100*time.Millisecond {
    once.Do(func() { snapshot(fr, *out) })
}
```

The `snapshot` helper creates a file and calls `fr.WriteTo(f)`. A run looks like this:

OUTPUT

```
request 3 took 21ms
request 4 took 250ms
slow request: saved 5 KiB of trace to flight.trace
request 5 took 20ms
```

`go tool trace flight.trace` then shows what every goroutine was doing in the seconds before the slow request. In a service, put the check in a middleware and allow at most one snapshot every few minutes.

Continuous profiling with Pyroscope

Continuous profiling collects low-rate profiles from every pod all the time, so you can ask "what used CPU last Tuesday at 14:05?" or compare two releases. **Grafana Pyroscope** is a popular open-source option. Its Go SDK pushes profiles from the app:

GO pyroscope/main.go (excerpt)

```go
profiler, err := pyroscope.Start(pyroscope.Config{
    ApplicationName: "orderly",
    ServerAddress:   "http://pyroscope:4040",
    ProfileTypes: []pyroscope.ProfileType{
        pyroscope.ProfileCPU, pyroscope.ProfileInuseSpace,
    },
})
if err != nil {
    log.Fatal(err)
}
defer profiler.Stop() // sends the last batch of profiles
```

Alternatively, Grafana Alloy can scrape the pprof endpoints you already have. If you're starting fresh, prefer scraping: no new dependency, and your admin port does the work.

Running the example

TERMINAL

```
cd examples/ch13-observability
go run ./orderly 2> spans.json      # JSON logs on stdout, spans in a file
curl -i localhost:8080/api/v1/orders/ord-42
curl localhost:8080/metrics | grep http_requests_total
go tool pprof -top "http://127.0.0.1:6060/debug/pprof/heap"
go test ./...                       # middleware, health and tracing tests
```

Take a moment to enjoy this one. Your service's logs, metrics and traces all line up by route and trace ID, and you understand every line that makes it work.

Recap

- **Four signals, four questions:** logs (what happened), metrics (how much, how often), traces (where the time went), profiles (which code is responsible).
- **Use `log/slog`** with the JSON handler in production, and store a request-scoped logger in the context under a typed key: Go's replacement for MDC.
- **Implement `slog.LogValuer`** on types holding sensitive data, so they redact themselves everywhere.
- **Prometheus needs a small middleware.** Use your own registry, cover RED and USE, and keep labels bounded: route patterns, never raw paths or IDs.
- **OpenTelemetry in Go is explicit:** a tracer provider, `otelhttp` on servers and clients, manual spans with `defer span.End()`, and a propagator, or traces break between services.
- **Put `trace_id` in every log line** with a small handler wrapper.
- **Liveness checks the process; readiness checks dependencies.**
- **pprof is built in.** Keep it on a private port, read `flat` vs `cum`, use flame graphs, and use the flight recorder for rare slowdowns.

Try it yourself

Exercise 1: A business metric

Add an `orderly_orders_fetched_total` counter with a `result` label (`found` or `not_found`). Increment it in `getOrder` and assert on it with `testutil.ToFloat64`. Hint: create it in a constructor that takes a `prometheus.Registerer`, the same way `newHTTPMetrics` does.

Exercise 2: Change the log level at runtime

Build the logger with a `slog.LevelVar`, and add a `PUT /debug/loglevel` handler on the **admin** port that accepts `DEBUG`, `INFO`, `WARN` or `ERROR`. Hint: `slog.Level` has an `UnmarshalText` method that parses these names.

> 🏋️ **Exercise 3: Trace the inventory loop**
>
> Wrap the loop over order items in `getOrder` in a manual span called `inventory.check`. Confirm in the exported spans that the `HTTP GET` client spans are now its children.

> 🏋️ **Exercise 4 (stretch): Flight recorder middleware**
>
> Write a middleware that keeps a `trace.FlightRecorder` running and saves a snapshot when a request takes longer than one second, at most once per minute. Hint: keep the last snapshot time in an `atomic.Int64`, and inject a clock so you can test the rate limit.

Further reading

- Structured Logging with slog: the Go team's introduction to slog and its design.
- Prometheus: Metric and label naming: the conventions behind `_total` and `_seconds`.
- Prometheus: Instrumentation: what to measure, and how to avoid cardinality trouble.
- OpenTelemetry Go documentation: getting started, instrumentation libraries and exporters.
- Go Diagnostics: the official overview of profiling, tracing and debugging.
- Profiling Go Programs: a classic, hands-on pprof walkthrough.
- Flight Recorder in Go 1.25: why the flight recorder exists and how to use it.
- Kubernetes: Liveness, readiness and startup probes: every probe option explained.

CHAPTER 14

Resilience and Graceful Shutdown

> *Things will break in production; this chapter is about making sure your users barely notice.*

🎯 **What you'll learn**

- Put a deadline on every network call, using one `context` that flows through HTTP, database and messaging code
- Retry the right failures with exponential backoff and jitter, and only when a request is safe to repeat
- Protect a struggling dependency with a circuit breaker, comparing `sony/gobreaker` and failsafe-go with Resilience4j
- Limit traffic with token buckets, cap concurrency with bulkheads, shed load and fall back gracefully
- Stop a panic in one goroutine from killing the whole process
- Shut down in the right order, and tune Kubernetes so rolling deploys drop zero requests

The big picture

Services talking over a network fail in many small ways. Packets get lost, a database pauses, a dependency is mid-deploy, a noisy client floods you, and Kubernetes restarts your pod because a node is being upgraded. That's the normal weather of distributed systems, not bad luck.

Resilience is the set of habits that keep one small failure from becoming an outage: bounding how long you wait, retrying carefully, backing off from a struggling dependency, and shutting down politely.

🧩 **Think of it like running a busy restaurant**

A good restaurant doesn't wait forever for a supplier who never answers (a **timeout**). If the line is busy, it calls back later (a **retry with backoff**). When a supplier is clearly closed, it stops calling for a while (a **circuit breaker**). A host seats guests at a steady pace (a **rate limiter**), and the grill and dessert station have separate cooks (a **bulkhead**). At closing time it stops seating people, finishes the meals already ordered, and only then turns off the lights (a **graceful shutdown**).

Spring developers get these tools from Resilience4j annotations and Spring Boot properties. In Go you'll use a few small libraries and plain code: the same patterns, with every moving part visible.

Concept	Java / Spring	Go
Timeouts	a setting per library	one `context` deadline
Retry	`@Retry`, `@Retryable`	`cenkalti/backoff`, failsafe-go

Concept	Java / Spring	Go
Circuit breaker	`@CircuitBreaker`	`sony/gobreaker`, failsafe-go
Rate limiting	Bucket4j, `@RateLimiter`	`golang.org/x/time/rate`
Bulkhead	`@Bulkhead`, thread pools	buffered channel as a semaphore
Crash in background work	that thread dies	the whole process dies
Graceful shutdown	`server.shutdown=graceful`	`NotifyContext` + `Shutdown`
Discovery and config	Spring Cloud	Kubernetes, mesh, env vars

Full code: `examples/ch14-resilience/`. Every program starts its own deliberately flaky server in-process, so you can run it all without Docker.

> **Version note**
>
> The examples pin releases that support Go 1.25: `cenkalti/backoff/v5` v5.0.3, `sony/gobreaker/v2` v2.4.0, `failsafe-go` v0.9.8 and `golang.org/x/time` v0.15.0. Newer `golang.org/x` releases require Go 1.26, so avoid `go get -u` on a Go 1.25 toolchain.

Timeouts everywhere

The most important rule in this chapter: **every call that leaves your process needs a deadline.** A call without one can wait forever, holding a goroutine, a connection and memory. Under load, thousands pile up, and a slow dependency becomes your outage.

One deadline that travels with the request

In Java every library has its own knob: `RestClient` read timeouts, HikariCP's `connectionTimeout`, `@Transactional(timeout = 5)`. They don't know about each other, so a request can spend 5 seconds in the database and another 5 calling a service.

Go has one mechanism that every well-behaved library accepts: `context.Context`, the "stop" button from Chapter 5. A deadline on it is a time budget for the whole request, and every call made with that context respects what's left.

```
incoming request, deadline 3s (set by middleware or the caller)
│
├── DB query          QueryRow(ctx, ...)    stops when the 3s are up
├── inventory call    own 500ms limit       never more than what's left
└── Kafka publish     Produce(ctx, ...)     stops when the 3s are up
```

A tighter limit for one step takes three lines:

GO

```go
ctx, cancel := context.WithTimeout(ctx, 2*time.Second)
defer cancel() // always release the timer, even on success
row := pool.QueryRow(ctx, "SELECT status FROM orders WHERE id = $1", id)
```

A child context can only shorten its parent's deadline, never extend it. A 2-second database limit inside a request with 1 second left gets 1 second, which is exactly what you want.

Clients and servers: the defaults have no timeouts

`http.DefaultClient` and `http.Get` **never time out**. Create one client with a `Timeout` and reuse it:

GO

```go
var client = &http.Client{Timeout: 10 * time.Second} // whole exchange
```

An `http.Server{}` with no timeouts waits forever for a slow client, which attackers love (Chapter 12 covers the "slowloris" attack). Here are sane defaults for a JSON API:

GO shutdown/main.go (excerpt)

```go
a.srv = &http.Server{
    Handler:           routes(&a.ready),
    ReadHeaderTimeout: 5 * time.Second,  // time to send the headers
    ReadTimeout:       10 * time.Second, // headers + body
    WriteTimeout:      15 * time.Second, // until the response is written
    IdleTimeout:       60 * time.Second, // keep-alive between requests
}
```

> ⚠️ **Gotcha: a timeout that's too long protects nothing**
>
> A 60-second timeout on a service that normally answers in 50 ms still lets requests pile up for a minute. Start from the dependency's p99 latency (the time 99% of calls finish within) and add headroom, say two or three times that.

Retries with exponential backoff and jitter

Many failures are brief: a connection reset, a pod restarting, a 503 during a deploy. Trying again a moment later often works. A careless retry, though, can turn a hiccup into a pile-up.

> 🧩 **Think of it like calling a busy friend**
>
> If the line is busy, you don't redial every second. You wait a minute, then two, then four (**exponential backoff**). And if a hundred people are calling the same friend, it helps if each waits a slightly random amount, so they don't all redial at once (**jitter**).

Without jitter, every client that failed at the same instant retries at the same instant. The recovering service gets hit by a wave of traffic called a **thundering herd**.

Only retry what is safe to repeat

A request is **idempotent** when doing it twice has the same effect as doing it once. `GET /orders/42` is. `POST /orders` is not: if the first attempt reached the server and only the response got lost, a retry places a second order.

Situation	Retry?	Why
Timeout, connection refused or reset	yes, if idempotent	probably transient
502, 503, 504	yes, if idempotent	server or proxy struggling
429 Too Many Requests	yes, slowly	honour `Retry-After`
400, 401, 403, 404, 409, 422	no	the answer won't change
`POST`, `PATCH`	no	unless the API takes idempotency keys

An **idempotency key** is a unique ID the client sends with a `POST`, often in an `Idempotency-Key` header. The server remembers it and returns the original result for a repeat, which makes retrying safe.

Java: Resilience4j. Go: cenkalti/backoff v5

In Spring you annotate the method with `@Retry(name = "inventory")` and configure it in YAML:

YAML application.yml

```yaml
resilience4j.retry.instances.inventory:
  max-attempts: 3
  wait-duration: 100ms
  enable-exponential-backoff: true
  exponential-backoff-multiplier: 2
```

In Go, `github.com/cenkalti/backoff/v5` is the long-standing backoff library. Version 5 is a single generic function: `backoff.Retry(ctx, operation, options...)`. The operation returns a value and an error, and returning `backoff.Permanent(err)` stops retrying at once.

Here's the heart of our resilient client. It retries only idempotent methods (`isIdempotent` accepts `GET`, `HEAD`, `OPTIONS`, `PUT` and `DELETE`), and every attempt goes through a circuit breaker, which we'll meet next:

GO client/client.go (excerpt)

```go
func (c *Client) Do(ctx context.Context, method, url string,
    body []byte) (*Response, error) {

    tries := uint(1)
    if isIdempotent(method) {
        tries = c.cfg.MaxTries
    }
    var last error
    op := func() (*Response, error) {
        if last != nil && !c.budget.Allow() {
            // Too many retries across the whole client right now.
            return nil, backoff.Permanent(
                fmt.Errorf("retry budget exhausted: %w", last))
        }
        resp, err := c.breaker.Execute(func() (*Response, error) {
            return c.attempt(ctx, method, url, body)
        })
        last = err
        return resp, classify(err)
    }
```

```
    return backoff.Retry(ctx, op,
        backoff.WithBackOff(c.newBackOff()),
        backoff.WithMaxTries(tries))
}
```

The policy is a small struct. `RandomizationFactor` is the jitter: with 0.5, a planned 200 ms wait becomes anything from 100 to 300 ms.

GO client/client.go (excerpt)

```go
// newBackOff returns a fresh policy for each call, because an
// ExponentialBackOff keeps state and is not safe to share.
func (c *Client) newBackOff() backoff.BackOff {
    return &backoff.ExponentialBackOff{
        InitialInterval:     c.cfg.InitialBackoff,
        RandomizationFactor: 0.5, // jitter: each wait is +/- 50%
        Multiplier:          2,   // double the wait after each failure
        MaxInterval:         c.cfg.MaxBackoff,
    }
}
```

`backoff.Retry` also stops when `ctx` is cancelled, so retries never outlive the request that started them. If a server sends `Retry-After`, return `backoff.RetryAfter(seconds)` from the operation and the library waits exactly that long.

> **Coming from Java**
>
> Notice what's missing: no proxy, no annotation, no aspect order to remember. The retry is a function call wrapped around another function call. When you want to know what happens on attempt three, you read the code.

Retry budgets: stopping retry storms

Picture three services in a chain, each making up to 3 attempts. When the last one fails, the first can trigger 3 × 3 × 3 = 27 calls to it. That's a **retry storm**: retries multiply the load on the service that's already struggling.

A **retry budget** caps retries across the whole client rather than per call. Ours is a token bucket (explained under rate limiting below): the first attempt is always allowed, but each retry needs a token. When everything is failing, the bucket empties and the client stops piling on.

GO client/client.go (excerpt)

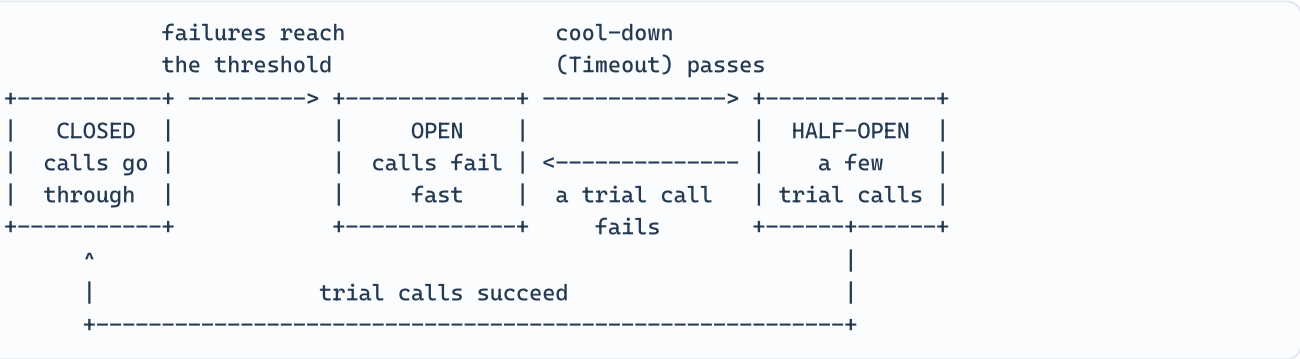
```go
budget := rate.NewLimiter(rate.Inf, 0) // no budget: allow all
if cfg.RetriesPerSec > 0 {
    budget = rate.NewLimiter(rate.Limit(cfg.RetriesPerSec), 1)
}
```

failsafe-go offers a ready-made version in its `budget` package.

Circuit breakers

Retries help with brief blips. When a dependency is properly down, retrying wastes time and keeps hitting it while it tries to recover. A **circuit breaker** notices repeated failures and stops calling for a while, failing fast instead.

> 🧩 **Think of it like the breaker in your fuse box**
>
> When a faulty kettle shorts the circuit, the breaker trips so the wiring doesn't catch fire. You don't flip it back every second. You wait, then try once. If the lights stay on, you're back to normal; if it trips again, you wait some more.



- **Closed:** normal operation; the breaker counts failures.
- **Open:** every call fails immediately without touching the dependency. It gets room to recover, and your callers get a fast answer instead of a slow timeout.
- **Half-open:** after a cool-down, a few trial calls go through. Success closes the breaker; failure opens it again.

Java: @CircuitBreaker

With Resilience4j, it's an annotation plus a fallback method, configured in YAML much like the retry:

JAVA

```
@Retry(name = "inventory")
@CircuitBreaker(name = "inventory", fallbackMethod = "cachedStock")
public Stock stock(String productId) {
    return rest.get().uri("/api/v1/inventory/{id}", productId)
            .retrieve().body(Stock.class);
}

private Stock cachedStock(String productId, Throwable cause) {
    return cache.lastKnown(productId);
}
```

Resilience4j's default aspect order puts `Retry` outside `CircuitBreaker`, so each attempt counts as one call through the breaker. Our Go client uses the same order.

Go: sony/gobreaker v2

`github.com/sony/gobreaker/v2` is small, focused and maintained. Version 2 is generic: a `CircuitBreaker[T]` returns a `T` from `Execute`, so you don't need type assertions.

GO client/client.go (excerpt)

```
st := gobreaker.Settings{
    Name:    cfg.Name,
    Timeout: cfg.OpenFor, // open -> half-open after this long
    ReadyToTrip: func(c gobreaker.Counts) bool {
        return c.ConsecutiveFailures >= cfg.TripAfter
    },
    // The caller giving up is not the dependency's fault.
    IsExcluded: func(err error) bool {
        return errors.Is(err, context.Canceled)
    },
    OnStateChange: func(_ string, from, to gobreaker.State) {
        if cfg.OnStateChange != nil {
            cfg.OnStateChange(from, to)
        }
    },
}
```

- `ReadyToTrip` decides when to open. We count consecutive failures; `Counts` also has totals if you prefer a ratio.
- `Timeout` is how long the breaker stays open (60 seconds if left at zero). `MaxRequests`, unset here, is how many trial calls half-open allows (1 by default).
- `IsExcluded` ignores some errors. A cancelled context means our caller gave up, which says nothing about the dependency.
- `OnStateChange` is the place for a log line and a metric. Your on-call engineer wants to see breakers open on a dashboard (Chapter 13).

What counts as a failure?

The breaker sees only the error returned inside `Execute`, so the single attempt decides. A 404 is a good answer ("that order doesn't exist") and must not trip the breaker; a 503 or a timeout should:

GO client/client.go (excerpt)

```
out := &Response{Status: res.StatusCode, Header: res.Header, Body: data}
if res.StatusCode >= 500 || res.StatusCode == http.StatusTooManyRequests {
    return out, &StatusError{Status: res.StatusCode}
}
return out, nil // 2xx, 3xx and 4xx are answers, not failures
```

Each attempt builds a new request with `bytes.NewReader(body)`. A Go request body is a stream that's gone once read, which is why `Do` takes a `[]byte`. The attempt also reads the whole response body itself, so its per-attempt timeout can be released safely.

When the breaker is open, `Execute` returns `gobreaker.ErrOpenState` immediately. Retrying that a few milliseconds later is pointless, so `classify` makes it permanent:

GO client/client.go (excerpt)

```go
func classify(err error) error {
    if errors.Is(err, gobreaker.ErrOpenState) ||
        errors.Is(err, gobreaker.ErrTooManyRequests) {
        return backoff.Permanent(err)
    }
    return err // nil, timeouts, connection errors, 5xx, 429: retry
}
```

Watching it work

The demo points the client at a server that fails its first two calls, and later goes down completely:

OUTPUT

```
1. GET while the server is flaky (retried):
  200 {"product_id":"MUG-BLUE-01","available":7} (server calls so far: 3)
2. POST while the server is down (never retried):
  error: server returned 503 (server calls so far: 4)
3. GET while the server is down (breaker opens):
  breaker: closed -> open
  error: circuit breaker is open (server calls so far: 6)
4. GET again (fails fast, server not called):
  error: circuit breaker is open (server calls so far: 6)
```

The `POST` went out exactly once. In step 3 the third failure in a row tripped the breaker mid-retry, so step 4 failed in microseconds without touching the server.

Testing with servers that fail on purpose

`net/http/httptest` starts a real HTTP server on a random local port. You script its failures, count the calls, and use millisecond intervals so the tests stay fast:

GO client/client_test.go (excerpt)

```go
for range 3 { // three failures trip the breaker
    c.Do(ctx, http.MethodGet, srv.URL, nil)
}
_, err := c.Do(ctx, http.MethodGet, srv.URL, nil)
if !errors.Is(err, gobreaker.ErrOpenState) {
    t.Fatalf("want ErrOpenState, got %v", err)
}
if calls.Load() != 3 {
    t.Fatalf("open breaker still called the server (%d calls)",
        calls.Load())
}
```

Other tests cover retried 503s, the never-retried `POST`, a 404, a hanging server, half-open recovery and the retry budget. That's `@Retry` plus `@CircuitBreaker`, with tests, in code you fully understand. Nice work.

The all-in-one option: failsafe-go

If you'd rather configure policies than write a wrapper, try `github.com/failsafe-go/failsafe-go`. It comes from the author of Failsafe for Java and feels like Resilience4j: retry, circuit breaker, timeout, bulkhead, rate limiter, fallback and hedging, composed in one call. Its `failsafehttp` package plugs into an `http.RoundTripper`:

GO failsafe/client.go (excerpt)

```go
// Retries 5xx, 429 and network errors, honouring Retry-After.
retry := failsafehttp.NewRetryPolicyBuilder().
    WithBackoff(10*time.Millisecond, 200*time.Millisecond).
    WithJitterFactor(0.5).
    WithMaxRetries(3).
    Build()

breaker := circuitbreaker.NewBuilder[*http.Response]().
    HandleIf(func(r *http.Response, err error) bool {
        return err != nil || r.StatusCode >= 500
    }).
    WithFailureThreshold(5). // 5 failures in a row: open
    WithDelay(10 * time.Second).
    Build()

perTry := timeout.New[*http.Response](500 * time.Millisecond)

// Outer to inner: retry( breaker( timeout( request ) ) )
rt := failsafehttp.NewRoundTripper(http.DefaultTransport,
    retry, breaker, perTry)
return &http.Client{Transport: rt}
```

> ⚠️ **Gotcha: failsafehttp retries POST too**
>
> The ready-made HTTP retry policy looks at the response and error, not the method. Use a client like this for reads only, so a `POST` is never repeated by accident.

> 💡 **If you're unsure, start with backoff + gobreaker**
>
> For a few downstream services, a small wrapper on `cenkalti/backoff` and `sony/gobreaker` is easy to read and test, with tiny dependencies. Reach for failsafe-go when you need many policies combined (hedging, adaptive limits, bulkheads). It's still pre-1.0 (v0.9.x), so expect occasional API changes.

Rate limiting

A rate limiter controls how fast requests flow. **Server-side**, it stops one client from using up all your capacity. **Client-side**, it keeps you under a partner's published limit so you never see their 429s.

> 🧩 **Think of it like a jar of tokens**
>
> A jar holds at most 5 tokens, and someone drops in a new one 10 times a second. Every request takes a token; if the jar is empty, the request is turned away or waits. The jar size allows short bursts and the refill speed sets the average. This is the **token bucket** algorithm.

Java: Bucket4j

In Spring you might keep a Bucket4j bucket per client in a `ConcurrentHashMap`:

JAVA

```java
Bucket bucket = buckets.computeIfAbsent(clientIp, ip -> Bucket.builder()
        .addLimit(Bandwidth.builder()
                .capacity(5)
                .refillGreedy(10, Duration.ofSeconds(1))
                .build())
        .build());
if (!bucket.tryConsume(1)) {
    response.setStatus(429);
    return;
}
```

Go: golang.org/x/time/rate

The Go team maintains a token bucket in `golang.org/x/time/rate`. `rate.NewLimiter(r, b)` adds `r` tokens per second to a bucket of size `b`. You ask for a token in one of three ways:

- `Allow()` answers true or false right away. Use it on servers.
- `Wait(ctx)` blocks until a token is free or the context ends. Use it in clients and workers.
- `Reserve()` tells you how long to wait, for custom scheduling.

Server-side, the example keeps one limiter per client in a map behind a mutex, and the middleware asks it for a token:

GO ratelimit/limiter.go (excerpt)

```go
func (p *PerClient) Middleware(next http.Handler) http.Handler {
    return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
        if !p.limiter(clientKey(r)).Allow() {
            w.Header().Set("Retry-After", "1")
            http.Error(w, "too many requests", http.StatusTooManyRequests)
            return
        }
        next.ServeHTTP(w, r)
    })
}
```

A `Cleanup(ttl)` method forgets idle clients; without it, the map grows forever.

> ⚠️ **Gotcha: choose the client key carefully**
>
> The example keys on the remote IP. Behind a load balancer, everyone may share one IP, and `X-Forwarded-For` can be faked unless your own proxy sets it. When you can, key on the authenticated identity, such as the JWT subject from Chapter 12.

Client-side, `Wait` is all you need. A greedy and a polite client each send 20 requests to a server allowing 10 per second with bursts of 5:

GO ratelimit/main.go (excerpt)

```go
// Client side: stay under the server's limit by waiting politely.
polite := rate.NewLimiter(rate.Limit(8), 1)

// ...and inside fire(), before each request:
if lim != nil {
    if err := lim.Wait(ctx); err != nil { // blocks until allowed
        return err.Error()
    }
}
```

OUTPUT

```
Greedy client, 20 requests as fast as possible:
  200 OK: 5, 429 Too Many Requests: 15 (took 0s)
Polite client, 20 requests at 8 per second:
  200 OK: 20, 429 Too Many Requests: 0 (took 2.4s)
```

> 📝 **Limits are per process**
>
> With 4 replicas, a per-pod limit of 10 per second means 40 overall. For a true global limit, keep counters in a shared store such as Redis, or enforce limits at your API gateway.

Bulkheads, load shedding and fallbacks

Timeouts and breakers handle slow or failing dependencies. Bulkheads and load shedding handle *too much work*, so overload in one place can't sink the whole service.

> 🧩 **Think of it like the compartments in a ship**
>
> A ship's hull is split into watertight compartments called bulkheads. If one floods, the ship stays afloat. In software, a **bulkhead** gives each dependency its own limited capacity, so a slow payment provider can't drown your inventory calls.

Bulkheads with a buffered channel

Resilience4j's `@Bulkhead` uses a semaphore or a thread pool. In Go, a buffered channel makes a fine counting semaphore: sending takes a slot, receiving gives it back.

GO bulkhead/bulkhead.go (excerpt)

```go
func (b *Bulkhead) Do(ctx context.Context,
    fn func(context.Context) error) error {

    timer := time.NewTimer(b.wait)
    defer timer.Stop()
    select {
    case b.slots <- struct{}{}: // got a slot
    case <-timer.C:
        return ErrBulkheadFull
    case <-ctx.Done():
        return ctx.Err()
    }
    defer func() { <-b.slots }() // give the slot back
    return fn(ctx)
}
```

With 2 slots and 6 slow payment calls at once, the demo prints `payments: 2 done, 4 rejected fast`. If you prefer a library, `golang.org/x/sync/semaphore` and `errgroup.Group.SetLimit` do the same job.

Bounded worker pools

A goroutine per item is cheap, but the database behind them has a fixed number of connections. A **worker pool** uses a fixed number of goroutines, and an unbuffered channel gives you **back-pressure**: the sender blocks while every worker is busy.

GO bulkhead/main.go (excerpt)

```go
jobs := make(chan int)
var wg sync.WaitGroup
for range workers {
    wg.Go(func() {
        for id := range jobs { // each worker pulls the next job
            // ...confirm order id, honouring ctx...
        }
    })
}
for id := range n {
    jobs <- id // blocks while all workers are busy: back-pressure
}
close(jobs)
wg.Wait()
```

Load shedding

When a service is overloaded, latency climbs for everyone. **Load shedding** rejects some requests early with a fast 503, so the rest still get good service. It's the Go version of capping Tomcat's `maxThreads`, except you choose what an overloaded request receives:

GO bulkhead/bulkhead.go (excerpt)

```go
func Shed(maxInFlight int, next http.Handler) http.Handler {
    slots := make(chan struct{}, maxInFlight)
    return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
        select {
        case slots <- struct{}{}:
            defer func() { <-slots }()
            next.ServeHTTP(w, r)
        default: // full: don't queue, answer right away
            w.Header().Set("Retry-After", "1")
            http.Error(w, "overloaded, try again soon",
                http.StatusServiceUnavailable)
        }
    })
}
```

Fallbacks

A **fallback** is what you return when a call fails: a cached value, a default, or a reduced feature. "7 in stock (as of a minute ago)" beats an error page. The example's `StockReader` remembers the last good answer per product:

GO bulkhead/fallback.go (excerpt)

```go
if cached, ok := s.last[id]; ok {
    // Degrade gracefully: an older number beats an error page.
    return StockLevel{ProductID: id, Available: cached, Stale: true},
        nil
}
return StockLevel{}, err // nothing to fall back to
```

OUTPUT

```
inventory up:   {ProductID:MUG-BLUE-01 Available:7 Stale:false}
inventory down: {ProductID:MUG-BLUE-01 Available:7 Stale:true}
```

> ⚠️ **Gotcha: be honest, and never fall back on writes**
>
> Mark fallback data as stale so the caller can decide what to show. Never "fall back" when placing an order or reserving stock: pretending a write succeeded is far worse than an error.

Recovering panics in goroutines

This one surprises every Java developer. In Java, an uncaught exception in a background thread kills that thread and the application carries on. In Go, **an unrecovered panic in any goroutine crashes the whole process**, along with every in-flight request.

`net/http` protects you inside handlers: if one panics, the server recovers, logs the stack and closes that connection. Goroutines *you* start (a cache warmer, an event relay) have no such safety net. A small helper adds one:

GO safefo/safefo.go

```go
// Recovered wraps fn so that a panic is logged instead of crashing
// the whole process. Use it with the go statement or wg.Go:
//
//  go Recovered(logger, "cache-warmer", warm)()
//  wg.Go(Recovered(logger, "outbox-relay", relay))
func Recovered(logger *slog.Logger, name string, fn func()) func() {
    return func() {
        defer func() {
            if r := recover(); r != nil {
                logger.Error("goroutine panicked",
                    "goroutine", name,
                    "panic", r,
                    "stack", string(debug.Stack()))
            }
        }()
        fn()
    }
}
```

The demo starts two goroutines with `sync.WaitGroup.Go` (new in Go 1.25), and one writes to a nil map. Its logger drops the time, level and stack to keep the output short:

OUTPUT

```
email-sender: sent 3 order confirmations
msg="goroutine panicked" goroutine=reports panic="assignment to entry in nil map"
main: still running, the process survived
```

The `WaitGroup.Go` documentation says the function "must not panic", one more reason to wrap long-running background work.

> **The Go way**
>
> A panic means a bug; expected failures are returned as `error` values (Chapter 4). Recovering keeps other requests alive but doesn't fix the bug. Log it, count it in a metric, alert on it and fix the cause. Don't sprinkle `recover` everywhere.

Graceful shutdown done right

Kubernetes stops pods all the time: every deploy, scale-down and node upgrade. If the process dies abruptly, users see 502s, half-finished work is lost and the last traces never arrive. A **graceful shutdown** makes each stop a non-event.

Chapter 6 showed the basic shape: start in dependency order, stop in reverse. Now we'll add readiness, draining and Kubernetes settings.

Java: one property

Since Spring Boot 3.4 graceful shutdown is on by default; before that, you set it yourself:

PROPERTIES application.properties

```
server.shutdown=graceful
spring.lifecycle.timeout-per-shutdown-phase=20s
```

On `SIGTERM`, Spring sets readiness to `REFUSING_TRAFFIC`, stops accepting requests and waits for in-flight ones. Then it stops beans in `SmartLifecycle` phase order and calls your `@PreDestroy` methods.

In Go you write that sequence yourself. It's about 40 lines, and in return you can read exactly what happens, in which order, and test it.

The order matters

```
SIGTERM arrives (Kubernetes, docker stop, Ctrl+C)
  |
  ▼
1. readiness -> 503 ......... load balancers stop sending new requests
  |  wait DrainDelay (say 5s) while the news spreads
  ▼
2. srv.Shutdown(ctx) ........ stop accepting, finish in-flight requests
  ▼
3. cancel workers, Wait ..... consumers stop, jobs finish, offsets commit
  ▼
4. flush telemetry .......... the last spans and metrics leave the process
  ▼
5. close the DB pool ........ last, because steps 2 and 3 still used it
  ▼
exit 0       (all of this within terminationGracePeriodSeconds)
```

Starting up: signals, server and workers

`signal.NotifyContext` returns a context that's cancelled on `SIGINT` (Ctrl+C) or `SIGTERM` (what Kubernetes sends). `main` then calls `run(ctx, ln, cfg, logger)`; because `run` takes a context and a listener, tests can drive it without real signals.

GO shutdown/main.go (excerpt)

```go
ctx, stop := signal.NotifyContext(context.Background(),
    os.Interrupt, syscall.SIGTERM)
defer stop()
```

Inside `run`, an `app` struct collects everything that must be shut down later:

GO shutdown/main.go (excerpt)

```
workerCtx, stopWorkers := context.WithCancel(context.Background())
a.stopWorkers = stopWorkers
defer stopWorkers()
a.workers.Go(func() { outboxRelay(workerCtx, logger) })

// ...create a.srv with the timeouts shown earlier...
serveErr := make(chan error, 1)
go func() { serveErr <- a.srv.Serve(ln) }()
a.ready.Store(true)
logger.Info("listening", "addr", ln.Addr().String())

select {
case <-ctx.Done():
    logger.Info("shutdown started")
case err := <-serveErr:
    return fmt.Errorf("server failed: %w", err)
}
return a.shutdown()
```

The workers get their *own* context, not the signal context. We decide when they stop (after HTTP has drained), instead of having them stop the instant the signal arrives.

Shutting down, step by step

The first half takes the service out of rotation and drains HTTP traffic:

GO shutdown/shutdown.go (excerpt)

```
func (a *app) shutdown() error {
    // 1. Fail readiness, then give load balancers time to notice.
    a.ready.Store(false)
    a.logger.Info("readiness set to failing", "drain", a.cfg.DrainDelay)
    time.Sleep(a.cfg.DrainDelay)

    // One deadline for the rest of the shutdown.
    ctx, cancel := context.WithTimeout(context.Background(),
        a.cfg.ShutdownTimeout)
    defer cancel()

    // 2. Stop accepting connections and finish in-flight requests.
    var errs []error
    if err := a.srv.Shutdown(ctx); err != nil {
        errs = append(errs, err)
        a.srv.Close() // deadline hit: force-close what is left
    }
    a.logger.Info("http server stopped")
```

The second half stops everything behind the HTTP layer:

GO shutdown/shutdown.go (continued)

```
    // 3. Stop background workers and wait for them to finish.
    a.stopWorkers()
    if err := waitTimeout(ctx, &a.workers); err != nil {
```

```
        errs = append(errs, err)
    }
    a.logger.Info("workers stopped")

    // 4. Flush telemetry while the network is still up.
    if err := a.tel.Shutdown(ctx); err != nil {
        errs = append(errs, err)
    }

    // 5. Close the database pool last: nothing uses it any more.
    a.db.Close()
    a.logger.Info("shutdown complete")
    return errors.Join(errs...)
}
```

- `/readyz` reads the `ready` flag, so it returns 503 from step 1 while `/healthz` (liveness) stays OK. Chapter 13 explains the two probes.
- `Server.Shutdown` closes the listeners and waits for active requests. If the deadline passes first, `Close` cuts what's left.
- `waitTimeout` waits for the `WaitGroup` but gives up when the context expires, so a stuck worker can't hang the shutdown.
- In the real service, `tel` is your OpenTelemetry provider and `db` a `*pgxpool.Pool`; their `Shutdown(ctx)` and `Close()` slot in here.

> ⚠️ **Gotcha: don't build request contexts from the signal context**
>
> If `http.Server.BaseContext` returns the signal context, every in-flight request is cancelled the moment `SIGTERM` arrives, and their database calls fail mid-request. `Shutdown` already lets in-flight requests finish. Also, `Shutdown` doesn't wait for hijacked connections such as WebSockets; use `Server.RegisterOnShutdown` to tell them to close.

Watching and testing it

The example's `-stop-after` flag behaves as if `SIGTERM` arrived after that long, which is handy on Windows. During this run a script saw `/readyz` return 200 before the stop and 503 during the drain, and an order request sent just before `Server.Shutdown` still got its 200:

TERMINAL

```
go run ./shutdown -drain 2s -stop-after 2s
```

OUTPUT

```
level=INFO msg=listening addr=127.0.0.1:8080
level=INFO msg="outbox relay started"
level=INFO msg="shutdown started"
level=INFO msg="readiness set to failing" drain=2s
level=INFO msg="http server stopped"
level=INFO msg="outbox relay stopped"
level=INFO msg="workers stopped"
level=INFO msg="telemetry flushed"
level=INFO msg="database pool closed"
level=INFO msg="shutdown complete"
```

The test plays Kubernetes by cancelling the context while a slow request is in flight. It then checks that readiness fails during the drain, the in-flight request still gets a 200, the log lines appear in the right order, and new connections are refused:

GO shutdown/main_test.go (excerpt)

```go
// Cancelling ctx plays the role of SIGTERM.
ctx, sigterm := context.WithCancel(context.Background())
// ...start run(ctx, ln, cfg, logger) and a slow request...
time.Sleep(50 * time.Millisecond)
sigterm()

// During the drain delay the server still answers, but not ready.
time.Sleep(50 * time.Millisecond)
if code, _ := get(t, base+"/readyz"); code != 503 {
    t.Errorf("readyz during drain = %d, want 503", code)
}
if code := <-inFlight; code != http.StatusOK {
    t.Errorf("in-flight request got %d, want 200", code)
}
```

A test like this catches the regressions that otherwise show up only as mysterious 502s during deploys.

Kubernetes: terminationGracePeriodSeconds and preStop

When Kubernetes deletes a pod, two things happen **at the same time**: the pod is removed from the Service's endpoints, and the container is told to stop. The endpoint change takes a few seconds to reach every kube-proxy, ingress and load balancer, so an app that exits instantly still receives requests it can't serve.

```
pod deletion starts
  |
  +--> endpoint removed ........ takes a few seconds to reach every
  |                              kube-proxy, ingress and load balancer
  |
  +--> preStop hook (sleep 5s) --+
       SIGTERM to the container  |  all of this must fit inside
       your shutdown sequence    |  terminationGracePeriodSeconds
       process exits 0         --+  (default 30s)
       still running? SIGKILL
```

That's what the drain delay is for. You can wait inside the app (our `DrainDelay`) or have Kubernetes wait before sending `SIGTERM`, using a `preStop` hook. The built-in `sleep` action is stable since Kubernetes 1.34 (on by default since 1.30) and needs no shell, which suits distroless images:

YAML deployment.yaml (excerpt)

```yaml
spec:
  terminationGracePeriodSeconds: 45
  containers:
    - name: orderly
      image: registry.example.com/orderly:1.4.2
      lifecycle:
        preStop:
          sleep:
            seconds: 5
      readinessProbe:
        httpGet: { path: /readyz, port: 8080 }
        periodSeconds: 2
```

Do the arithmetic once: preStop (5s) + HTTP drain (up to 20s) + workers and flushing (a few seconds) must stay under `terminationGracePeriodSeconds`, because the countdown includes the `preStop` time. Use either the `preStop` sleep or an in-app drain delay; stacking both only slows deploys.

> 💡 **Let a second Ctrl+C mean 'now'**
>
> Once shutdown has started, call the `stop` function from `signal.NotifyContext`. That restores default signal handling, so an impatient second Ctrl+C kills the process right away.

Microservice frameworks vs Spring Cloud

Spring Cloud gives Java teams Eureka, Config Server, Gateway, OpenFeign, LoadBalancer and CircuitBreaker. So what's the Go equivalent? Here's how the main frameworks stand in November 2026:

Project	What it is	Status
go-kit	toolkit: endpoints, transports, middleware	quiet: last release v0.13.0, May 2023
Kratos	HTTP + gRPC framework, registry, config	active (v2.9.x)
go-zero	framework + `goctl` codegen, built-in breakers	active (v1.10.x)
Dapr	sidecar: service calls, pub/sub, state, resiliency	active, CNCF graduated (2024)

go-kit isn't archived, and it taught a generation of Go developers how to structure services. But with no release since 2023 it isn't a good base for a new project. Kratos and go-zero are active and productive if your team likes an opinionated stack with code generation. Dapr runs beside your service as a sidecar and suits polyglot organisations.

What we recommend

For most teams on Kubernetes, the best "Spring Cloud for Go" is **plain libraries plus your platform**:

Spring Cloud piece	Go plus platform
Eureka / Consul discovery	Kubernetes Service DNS (`http://inventory`)
Config Server	ConfigMaps and Secrets as env vars (Chapter 10)
LoadBalancer, Gateway	kube-proxy, Ingress or Gateway API
CircuitBreaker, Retry	gobreaker + backoff, or mesh policies
mTLS between services	a service mesh such as Istio or Linkerd
Sleuth / Micrometer Tracing	OpenTelemetry (Chapter 13)

Kubernetes already does discovery, configuration and load balancing; running Eureka or a Config Server on top adds one more system to operate for no gain. Your Go code stays small: a resilient client, a rate limiter, a graceful shutdown and good telemetry.

> ⚠️ **Gotcha: retries multiply across layers**
>
> A service mesh can retry too, and Istio, for example, retries some failures by default. If the mesh retries twice and your code three times, one failing call becomes up to nine. Decide where retries live (usually one layer, close to the caller) and turn them off elsewhere.

Recap

- Every network call needs a deadline: pass `ctx` everywhere, never use `http.DefaultClient`, and set the `http.Server` timeouts.
- Retry only transient failures of idempotent requests, with exponential backoff and jitter; `backoff.Retry` does it in one call.
- Cap retries with a retry budget so they can't multiply into a storm.
- A circuit breaker (`sony/gobreaker/v2`) fails fast while a dependency is down and probes it with trial calls. Put retry outside the breaker.
- `golang.org/x/time/rate` is a token bucket: `Allow` on servers (answer 429), `Wait` in clients.
- Bulkheads, bounded worker pools and load shedding contain overload. Mark fallbacks stale and never use them for writes.
- A panic in any goroutine kills the process; wrap background goroutines with a recover helper.
- Shutdown order: fail readiness, drain, `Server.Shutdown`, stop workers, flush telemetry, close the DB, all inside `terminationGracePeriodSeconds`.

Try it yourself

🏋 Exercise 1: Honour Retry-After

Extend `client/` so a 429 or 503 carrying `Retry-After: 1` makes the client wait that long before the next try. Test it with an `httptest` server that sends the header once and then succeeds.

Hint: in `attempt`, parse the header with `strconv.Atoi` and return `backoff.RetryAfter(seconds)`.

🏋 Exercise 2: A bulkhead per dependency

Add `MaxConcurrent` to the client's `Config` and use the `Bulkhead` from `bulkhead/` to cap concurrent calls. Prove it: a slow server, ten concurrent `Do` calls, a limit of 2.

Hint: on the server, track in-flight requests with an `atomic.Int32` and record the highest value seen.

🏋 Exercise 3: Shutdown with a stuck worker

Add a worker to `shutdown/` that ignores its context. Check that shutdown still finishes within `ShutdownTimeout` and `run` returns the deadline error. Then add the "second Ctrl+C exits now" behaviour.

Hint: assert with `errors.Is(err, context.DeadlineExceeded)`.

🏋 Exercise 4 (stretch): Rate limit by customer

Key the `ratelimit/` limiter on the authenticated customer ID (set a fake one in the request context in an earlier middleware), falling back to the IP. Give premium customers a higher limit.

Hint: use a typed context key, as Chapter 12 does for JWT claims.

Further reading

- Timeouts, retries and backoff with jitter: the Amazon Builders' Library article on why jitter matters
- Addressing Cascading Failures: the Google SRE book chapter on retry storms and overload
- cenkalti/backoff v5 and sony/gobreaker v2: every option, documented
- failsafe-go documentation: all policies, how they compose, and the HTTP and gRPC integrations
- golang.org/x/time/rate: the token bucket API, including `Reserve` and `SetLimit`
- net/http Server.Shutdown: the exact guarantees of a graceful shutdown in Go
- Kubernetes pod termination: what happens, in order, when a pod is deleted
- Spring Boot graceful shutdown: the Spring side, for comparison

CHAPTER 15

Testing

> *Go ships a fast, no-nonsense test toolkit in the box, and once it clicks you'll write more tests, not fewer.*

🎯 **What you'll learn**

- Write Go tests with the `testing` package and run them with `go test`
- Turn JUnit `@ParameterizedTest` habits into clean table-driven tests
- Choose between plain assertions and testify, and between fakes and mocks
- Test HTTP handlers and clients with `httptest`, and databases with testcontainers-go
- Test time, fuzz a parser, benchmark a hot function and read coverage
- Organise unit, integration and contract tests into a sensible pyramid

The big picture

In the Java world, testing is a stack of libraries: JUnit runs things, AssertJ checks things, Mockito fakes things, and Spring's test slices glue it all together. Go takes a different route. The test runner, benchmarks, fuzzing, coverage and the race detector all come with the toolchain. There's no plugin to configure and no annotation to remember.

That doesn't mean Go teams avoid libraries. Most enterprise Go code uses **testify** for assertions and **testcontainers-go** for databases. The difference is that the foundation is small, so you can always see what's happening.

🧩 **Think of it like a kitchen that comes with the house**

The `testing` package is a kitchen that comes fully equipped: knives, pans, an oven. You can cook a proper meal on day one. testify and mock generators are the nice gadgets you buy later, like a stand mixer. Useful, but the meal doesn't depend on them.

Java ↔ Go at a glance

Concept	Java / Spring	Go
Test framework	JUnit 5	`testing` package (built in)
Runner	Surefire / Gradle `test`	`go test`
Assertions	AssertJ, Hamcrest	`if` + `t.Errorf`, or testify

Concept	Java / Spring	Go
Parameterized tests	`@ParameterizedTest`	table-driven tests + `t.Run`
Setup and teardown	`@BeforeEach`, `@AfterEach`	plain code, `t.Cleanup`, `TestMain`
Mocks	Mockito	hand-written fakes, gomock, mockery
Web layer tests	MockMvc, `@WebMvcTest`	`httptest.NewRecorder`
Stubbing remote APIs	WireMock	`httptest.NewServer`
Real database	Testcontainers Java	testcontainers-go
Fuzzing	Jazzer	`go test -fuzz`
Microbenchmarks	JMH	`testing.B` + benchstat
Coverage	JaCoCo	`go test -cover`

Full code: `examples/ch15-testing/`

Your first Go test

A Go test is an ordinary function in a file whose name ends in `_test.go`. It lives in the same folder as the code it tests. The `go` tool compiles those files only when you run tests, so they never end up in your production binary.

Files, names and `go test`

Here's a JUnit test you might write for Orderly's order total:

JAVA OrderTest.java

```java
class OrderTest {
    @Test
    void totalCentsSumsAllItems() {
        var order = new Order(List.of(
            new OrderItem(MUG_ID, 2, 1250),
            new OrderItem(TEA_ID, 1, 499)));

        assertEquals(2999, order.totalCents());
    }
}
```

And here's the Go version:

GO order/order_test.go

```go
func TestTotalCents(t *testing.T) {
    o := Order{Items: []Item{
        {ProductID: "mug", Quantity: 2, UnitPriceCents: 1250},
        {ProductID: "tea", Quantity: 1, UnitPriceCents: 499},
    }}

    got := o.TotalCents()

    if got != 2999 {
        t.Errorf("TotalCents() = %d, want %d", got, 2999)
    }
}
```

There's no class and no annotation. The rules are short:

- The function name starts with `Test` followed by a capital letter.
- It takes one parameter, `t *testing.T`, which is your handle for reporting failures and logging.
- A test passes unless it calls `t.Error`, `t.Fatal` (or their `f` variants) or panics.

Coming from Java

There's no `assertEquals` in the standard library. A Go test is plain code: an `if` plus a message saying what you got and what you wanted. It feels bare at first, and almost every Java developer reaches for an assertion library on day one. Give the plain style a week before deciding. We'll look at testify shortly.

You run tests with `go test`. The `./...` pattern means "this folder and everything below it".

TERMINAL

```
go test ./...                    # every package in the module
go test ./order/                 # one package
go test -v -run TestTotalCents ./order/
```

These flags cover nearly everything you'll do day to day:

Flag	What it does	Typical use
`-run 'Regex'`	runs only tests whose names match	focus on one test or case
`-v`	lists every test and shows `t.Log` output	debugging
`-count=1`	ignores cached results and runs again	tests that touch the outside world
`-cover`	prints statement coverage	quick health check
`-short`	makes `testing.Short()` return true	skip slow tests locally
`-race`	turns on the race detector	CI
`-timeout 2m`	fails the run if it takes longer (default 10m)	CI
`-shuffle=on`	runs tests in a random order	find hidden dependencies

> 💡 **Why did it say (cached)?**
>
> `go test ./...` remembers passing results. If neither the code nor the files and environment variables a test reads have changed, it prints `(cached)` instead of running again. That's what makes Go test runs feel so quick. When you really want a fresh run, add `-count=1`.

`go test` also runs a small, high-confidence set of `go vet` checks before the tests start. So a mismatched `Printf` verb fails your test run, which is a nice safety net.

`t.Error` versus `t.Fatal`

Go gives you two ways to fail. `t.Error` and `t.Errorf` record a failure and **keep going**. `t.Fatal` and `t.Fatalf` record a failure and **stop this test immediately**.

Here's a deliberately broken `total` function and three tests, so you can see both. (In the example folder this file sits behind a `demo_failures` build tag, so it doesn't break the normal test run.)

GO failing/total_test.go

```go
func total(prices ...int64) int64 { return 0 } // bug: forgot to add

func TestErrorKeepsGoing(t *testing.T) {
    if got := total(100, 250); got != 350 {
        t.Errorf("total(100, 250) = %d, want 350", got)
    }
    if got := total(); got != 0 {
        t.Errorf("total() = %d, want 0", got)
    }
    t.Log("still running after t.Errorf")
}

func TestFatalStops(t *testing.T) {
    if got := total(100, 250); got != 350 {
        t.Fatalf("total(100, 250) = %d, want 350", got)
    }
    t.Log("never printed")
}
```

OUTPUT

```
=== RUN   TestErrorKeepsGoing
    total_test.go:11: total(100, 250) = 0, want 350
    total_test.go:16: still running after t.Errorf
--- FAIL: TestErrorKeepsGoing (0.00s)
=== RUN   TestFatalStops
    total_test.go:21: total(100, 250) = 0, want 350
--- FAIL: TestFatalStops (0.00s)
```

> 💡 **If you're unsure, use this rule**
>
> Use `t.Fatal` when carrying on makes no sense, for example when an error means the result you're about to inspect is empty or nil. Use `t.Error` for independent checks, so one run shows you every problem at once.

> ⚠️ **Gotcha: t.Fatal only works on the test's own goroutine**
>
> `t.Fatal` stops the test by ending the goroutine that called it. If you call it inside a goroutine you started, it ends that goroutine, not the test. In goroutines, use `t.Error` and return, or send the error back on a channel.

Helpers, cleanup and context

As tests grow, you'll extract helpers. Call `t.Helper()` at the top of each one. Then a failure is reported at the line in your test that called the helper, which is the line you actually care about.

GO failing/total_test.go

```go
func assertTotal(t *testing.T, got, want int64) {
    t.Helper()
    if got != want {
        t.Errorf("total = %d, want %d", got, want)
    }
}

func TestWithHelper(t *testing.T) {
    assertTotal(t, total(100), 100) // failure is reported on this line
}
```

OUTPUT

```
=== RUN   TestWithHelper
    total_test.go:34: total = 0, want 100
--- FAIL: TestWithHelper (0.00s)
```

For teardown, Go uses `t.Cleanup`. You register a function, and it runs when the test (or subtest) finishes, pass or fail. Several cleanups run in reverse order, like `defer`.

JAVA

```java
@AfterEach
void tearDown() {
    server.shutdown();
}
```

GO

```go
srv := httptest.NewServer(h)
t.Cleanup(srv.Close)
```

The big advantage is locality: the cleanup sits right next to the thing it cleans up. A few relatives are worth knowing:

- `t.TempDir()` returns a fresh folder that is deleted after the test.
- `t.Setenv("KEY", "value")` sets an environment variable and restores it afterwards.
- `t.Context()` (Go 1.24) returns a `context.Context` that is cancelled right before cleanups run. Pass it to code under test instead of `context.Background()`, so goroutines you start get told to stop.

Table-driven tests: Go's `@ParameterizedTest`

Most business rules need many small cases: an empty order, a zero quantity, a negative price. Writing a separate test function for each gets repetitive fast. JUnit solves this with parameterized tests; Go solves it with a slice of cases and a loop.

Here is the JUnit version of "reject invalid orders":

JAVA OrderValidationTest.java

```java
@ParameterizedTest(name = "{0}")
@MethodSource("invalidOrders")
void rejectsInvalidOrders(String name, Order order, String message) {
    var ex = assertThrows(InvalidOrderException.class, order::validate);
    assertThat(ex.getMessage()).contains(message);
}

static Stream<Arguments> invalidOrders() {
    return Stream.of(
        Arguments.of("missing customer", withCustomer(null), "customer"),
        Arguments.of("no items", withItems(List.of()), "no items"),
        Arguments.of("zero quantity", withQuantity(0), "quantity"));
}
```

The Go version keeps the cases and the logic in one function:

GO order/order_test.go

```go
func TestValidate(t *testing.T) {
    tests := []struct {
        name    string
        order   Order
        wantErr string // "" means we expect no error
    }{
        {"valid order", validOrder(), ""},
        {"missing customer", withCustomer(""), "customer id is required"},
        {"no items", Order{CustomerID: "c-1"}, "no items"},
        {"zero quantity", withQuantity(0), "quantity must be positive"},
        {"negative quantity", withQuantity(-3), "got -3"},
    }

    for _, tc := range tests {
        t.Run(tc.name, func(t *testing.T) {
            t.Parallel() // safe: every case builds its own data
            err := tc.order.Validate()
            assertErrorContains(t, err, tc.wantErr)
        })
    }
}
```

Let's walk through it:

- `tests` is a slice of an **anonymous struct**: a struct type declared on the spot, with no name, because nothing else needs it.
- Each case has a `name`. `t.Run(tc.name, ...)` turns it into a **subtest**: a named child test that passes or fails on its own.

- `t.Parallel()` lets the subtests run at the same time. Since Go 1.22, every loop iteration gets its own `tc`, so the old `tc := tc` line is no longer needed.

Subtest names become part of the test's path, with spaces turned into underscores. You can run a subset with a regular expression after a slash:

TERMINAL

```
go test -v -run 'TestParsePrice/decimal' ./order/
```

OUTPUT

```
=== RUN   TestParsePrice
=== RUN   TestParsePrice/two_decimals
=== RUN   TestParsePrice/one_decimal
=== RUN   TestParsePrice/three_decimals
--- PASS: TestParsePrice (0.00s)
    --- PASS: TestParsePrice/two_decimals (0.00s)
    --- PASS: TestParsePrice/one_decimal (0.00s)
    --- PASS: TestParsePrice/three_decimals (0.00s)
PASS
```

That `TestParsePrice` table uses a **map** keyed by the case name instead of a slice. Go randomises map iteration order, so a map-based table also catches cases that accidentally depend on each other.

GO order/price_test.go

```go
tests := map[string]struct {
    in      string
    want    int64
    wantErr bool
}{
    "whole number":      {in: "7", want: 700},
    "two decimals":      {in: "12.34", want: 1234},
    "three decimals":    {in: "1.999", wantErr: true},
    "negative":          {in: "-1.00", wantErr: true},
    // ...more cases in the full file
}
```

The Go way: what makes a good table

- Give every case a short, readable name. The name is the failure message's headline.
- Keep the table to inputs and expected outputs. If cases start needing `setupMock func()` fields and `if tc.something` branches in the loop, split them into separate tests.
- Use field names (`in:`, `want:`) once a case has more than two or three fields.
- Print the input in failure messages: `ParsePrice(%q) = %d, want %d`.

Assertions: plain Go or testify?

This is the first real decision your team makes, so let's look at both honestly.

The plain standard-library style

The Go team, the Google Go Style Guide and many library authors write assertions as plain `if` statements. The convention is a message in the form "`Func(input) = got, want want`":

GO service/service_test.go

```go
if err != nil {
    t.Fatalf("PlaceOrder() unexpected error: %v", err)
}
if got.Status != order.StatusConfirmed {
    t.Errorf("status = %s, want %s", got.Status, order.StatusConfirmed)
}
```

It's more typing, but there's nothing to learn, and the failure message says exactly what you wrote. For comparing whole structs, teams using this style often add Google's `github.com/google/go-cmp/cmp` package, whose `cmp.Diff(want, got)` prints a readable diff.

testify: `assert` and `require`

testify (`github.com/stretchr/testify`, v1.12.1 at the time of writing) is the most widely used Go test library. It gives you AssertJ-style helpers in two flavours:

- `assert` functions record a failure and carry on, like `t.Errorf`.
- `require` functions stop the test, like `t.Fatalf`.

GO service/testify_test.go

```go
func TestPlaceOrder_Testify(t *testing.T) {
    repo := newFakeRepo()
    stock := &fakeStock{available: map[string]int{"mug": 5}}
    svc := New(repo, stock, fixedClock, fixedID)

    got, err := svc.PlaceOrder(t.Context(), "c-42",
        []order.Item{{ProductID: "mug", Quantity: 2, UnitPriceCents: 1250}})

    require.NoError(t, err) // stop here if this fails: got is useless
    assert.Equal(t, order.StatusConfirmed, got.Status)
    assert.Equal(t, int64(2500), got.TotalCents())
    assert.Equal(t, 3, stock.available["mug"])
    assert.Contains(t, repo.orders, "ord-1")
}
```

When an `assert.Equal` on two structs fails, testify prints a diff. Here's a trimmed real failure, comparing two item slices:

OUTPUT

```
--- FAIL: TestTestifyDiff (0.00s)
    testify_test.go:17:
        Error:   Not equal:
        ...
                 Diff:
                 --- Expected
                 +++ Actual
                 @@ -7,3 +7,3 @@
                    ProductID: (string) (len=3) "tea",
                 -  Quantity: (int) 1
                 +  Quantity: (int) 3
```

> ⚠️ **Gotcha: expected comes first**
>
> testify's order is `assert.Equal(t, expected, actual)`, the same as JUnit's `assertEquals`. Swap them and the diff labels are backwards, which is confusing at 2 a.m. Note also that `assert.Equal(t, 2500, got.TotalCents())` fails, because the untyped `2500` becomes an `int` and `TotalCents` returns `int64`. Write `int64(2500)`.

> 💡 **If you're unsure, use testify's require and assert**
>
> For application teams coming from Java, testify makes tests shorter and the failure output friendlier. Use `require` for anything the rest of the test depends on (errors, lengths before indexing) and `assert` for the rest. Prefer plain `if` statements in open-source libraries, where an extra dependency costs your users.

Suites, for when you miss `@BeforeEach`

testify's `suite` package gives you JUnit-style lifecycle hooks: `SetupTest` runs before each test method, `TearDownTest` after it, and `SetupSuite` once for the whole group.

GO service/suite_test.go

```go
type OrderServiceSuite struct {
    suite.Suite
    repo  *fakeRepo
    stock *fakeStock
    svc   *OrderService
}

// SetupTest runs before every test method, like JUnit's @BeforeEach.
func (s *OrderServiceSuite) SetupTest() {
    s.repo = newFakeRepo()
    s.stock = &fakeStock{available: map[string]int{"mug": 5}}
    s.svc = New(s.repo, s.stock, fixedClock, fixedID)
}

func (s *OrderServiceSuite) TestGetMissingOrder() {
```

```
    _, err := s.svc.GetOrder(s.T().Context(), "nope")
    s.ErrorIs(err, order.ErrNotFound)
}

// TestOrderServiceSuite is the bridge between `go test` and the suite.
func TestOrderServiceSuite(t *testing.T) {
    suite.Run(t, new(OrderServiceSuite))
}
```

Suites work well, but they're used less than you might expect. Suite methods can't use `t.Parallel()`, and shared fields make it easy for one test to leak state into the next. Most Go teams prefer a small constructor function such as `newFakeRepo()` called at the top of each test. Reach for a suite when several tests share expensive setup that `TestMain` doesn't fit.

Test doubles: fakes, mocks and when to use which

The service we're testing places an order: it validates, reserves stock and saves. In tests we don't want a real warehouse or database, so we swap in **test doubles**: stand-ins that look like the real dependency.

Small interfaces and hand-written fakes

In Spring, you'd mock the repository with Mockito:

JAVA OrderServiceTest.java

```java
@ExtendWith(MockitoExtension.class)
class OrderServiceTest {
    @Mock OrderRepository repository;
    @Mock StockClient stock;
    @InjectMocks OrderService service;

    @Test
    void savesConfirmedOrder() {
        when(stock.reserve(any(), anyInt())).thenReturn(true);
        service.placeOrder(CUSTOMER_ID, ITEMS);
        verify(repository).save(argThat(o -> o.status() == CONFIRMED));
    }
}
```

Mockito can mock any class at runtime. Go has no runtime bytecode tricks, so test doubles hang off **interfaces**. The idiom is to declare small interfaces in the package that *uses* them, listing only the methods it calls. These are called **consumer-side interfaces**.

GO service/service.go

```go
// OrderRepository is the storage the service needs. It's declared here,
// where it is used, and it only lists the methods this package calls.
type OrderRepository interface {
    Save(ctx context.Context, o order.Order) error
    Get(ctx context.Context, id string) (order.Order, error)
}

// StockReserver reserves stock for one product.
type StockReserver interface {
    Reserve(ctx context.Context, productID string, qty int) error
}
```

Because Go interfaces are satisfied implicitly (see Chapter 4), the Postgres repository satisfies `OrderRepository` without knowing it exists. And a test can satisfy it with a few lines of code. That's a **fake**: a small working implementation, usually in memory.

GO service/fakes_test.go

```go
// fakeStock tracks available stock per product.
type fakeStock struct {
    available map[string]int
}

func (f *fakeStock) Reserve(_ context.Context, productID string, qty int) error {
    if f.available[productID] < qty {
        return order.ErrInsufficientStock
    }
    f.available[productID] -= qty
    return nil
}
```

The fake repository is a mutex-protected map with a `saveErr` field you can set to simulate a failure. The test then checks *outcomes* (what's in stock, what got saved) rather than *calls*:

GO service/service_test.go

```go
func TestPlaceOrder_InsufficientStock(t *testing.T) {
    repo := newFakeRepo()
    stock := &fakeStock{available: map[string]int{"mug": 1}}
    svc := New(repo, stock, fixedClock, fixedID)

    _, err := svc.PlaceOrder(context.Background(), "c-42",
        []order.Item{{ProductID: "mug", Quantity: 2, UnitPriceCents: 1250}})

    if !errors.Is(err, order.ErrInsufficientStock) {
        t.Fatalf("error = %v, want ErrInsufficientStock", err)
    }
    if len(repo.orders) != 0 {
        t.Errorf("saved %d orders, want none", len(repo.orders))
    }
}
```

🧩 **Think of it like a flight simulator versus a script**

A fake is a flight simulator: the controls really work, so the pilot can try anything. A mock is a script: "first you'll press this button, then that one". Scripts are fine for one rehearsal, but they break the moment the pilot finds a better way to fly.

🧠 **Key idea**

Small interfaces, declared where they're used, make fakes cheap. If your fake needs twenty methods, the interface is too big; that's a design signal, not a testing problem.

Generated mocks with gomock

Sometimes you do want to assert on calls: "the payment gateway must be charged exactly once" or "we must not save when reservation fails". For that, generated mocks save typing.

📝 **Maintenance note: use go.uber.org/mock, not golang/mock**

The original `github.com/golang/mock` was archived by the Go team in 2023. Uber maintains the active fork at `go.uber.org/mock` (v0.6.0 at the time of writing). The API is the same, so migrating is a find-and-replace on import paths.

Since Go 1.24, you can pin a tool's version in `go.mod` with a `tool` directive, so every developer and CI job runs the same mockgen:

TERMINAL

```
go get -tool go.uber.org/mock/mockgen@v0.6.0
```

GO.MOD go.mod (trimmed)

```
module example.com/goguide/ch15

go 1.25.0

tool go.uber.org/mock/mockgen
```

Then a `go:generate` comment in the source says how to build the mocks. `-command` defines a short alias so the line stays readable:

GO service/service.go

```
// The mockgen tool is pinned by the "tool" line in go.mod.
//go:generate -command mockgen go tool mockgen -package=mocks
//go:generate mockgen -destination=mocks/mock.go . OrderRepository,StockReserver
```

Run `go generate ./...` and the mocks appear in `service/mocks/mock.go`. A test sets expectations, and the controller fails the test if they aren't met, or if an unexpected call arrives:

GO service/gomock_test.go

```
func TestPlaceOrder_Gomock(t *testing.T) {
    ctrl := gomock.NewController(t) // checks expectations at test end
    repo := mocks.NewMockOrderRepository(ctrl)
    stock := mocks.NewMockStockReserver(ctrl)

    // Expect exactly one reservation, and no Save because it fails.
    stock.EXPECT().
        Reserve(gomock.Any(), "mug", 2).
        Return(errors.New("warehouse offline"))

    svc := service.New(repo, stock, time.Now, func() string { return "ord-1" })
    _, err := svc.PlaceOrder(t.Context(), "c-42",
        []order.Item{{ProductID: "mug", Quantity: 2, UnitPriceCents: 1250}})

    if err == nil || err.Error() != "reserve mug: warehouse offline" {
        t.Fatalf("unexpected error: %v", err)
    }
}
```

Notice that `repo` has no expectations. If the service called `Save` anyway, gomock would fail the test with "unexpected call". That's the one thing this mock test checks that our fake test didn't.

mockery

mockery (`github.com/vektra/mockery/v3`, v3.9.0) is the other popular generator. It produces mocks built on testify's `mock` package and is driven by a YAML file at the module root:

YAML .mockery.yml

```yaml
template: testify
dir: "{{.InterfaceDir}}/mockery"
filename: "mocks.go"
pkgname: "mockery"
structname: "{{.Mock}}{{.InterfaceName}}"
packages:
  example.com/goguide/ch15/service:
    interfaces:
      OrderRepository:
      StockReserver:
```

Run it with `go run github.com/vektra/mockery/v3@v3.9.0`. The generated constructors take `t` and verify expectations automatically at cleanup:

GO service/mockery_test.go

```go
repo := mockery.NewMockOrderRepository(t) // verifies calls at cleanup
stock := mockery.NewMockStockReserver(t)
stock.EXPECT().
    Reserve(mock.Anything, "mug", 2).
    Return(errors.New("warehouse offline"))
```

> 💡 **If you're unsure, start with fakes**
>
> Write fakes by hand for your own small interfaces. Add a generator when you have many interactions to verify. Pick **gomock** if you want strict call checking out of the box, and **mockery** if your team already uses testify everywhere. Either is fine; don't use both in one codebase.

When mocks hurt

Mocks are a sharp tool. These are the signs they're cutting you:

- **Tests break on harmless refactors.** You merged two repository calls into one and twelve tests failed, though behaviour didn't change.
- **The test repeats the implementation.** Each line of code has a matching `EXPECT()` line. You're testing that the code is written the way it's written.
- **You're mocking types you don't own.** Mocking `pgx.Tx` or an HTTP client tells you nothing about whether the real thing behaves that way. Wrap it in your own small interface, or test against the real thing with testcontainers.
- **Mocks return mocks.** If setting up a test needs a mock that returns a mock, the design has too many layers.

Testing HTTP

HTTP is everywhere in enterprise services, and Go's `net/http/httptest` package makes it pleasant to test. It covers both directions: handlers you serve and clients you call.

Handler tests with `httptest.NewRecorder`

In Spring, a web-layer test spins up a slice of the context and uses MockMvc:

JAVA OrderControllerTest.java

```java
@WebMvcTest(OrderController.class)
class OrderControllerTest {
    @Autowired MockMvc mvc;
    @MockitoBean OrderService service;

    @Test
    void returns404ForUnknownOrder() throws Exception {
        when(service.getOrder(any())).thenThrow(new OrderNotFoundException());
        mvc.perform(get("/api/v1/orders/{id}", UUID.randomUUID()))
           .andExpect(status().isNotFound());
    }
}
```

In Go, a handler is an `http.Handler`: a value with a `ServeHTTP(w, r)` method. So you can call it directly. No server, no port, no context to start. `httptest.NewRecorder()` gives you a `ResponseWriter` that records the status, headers and body:

GO httpapi/handler_test.go

```go
func TestGetOrder_NotFound(t *testing.T) {
    h := NewHandler(stubService{err: order.ErrNotFound})

    req := httptest.NewRequest(http.MethodGet, "/api/v1/orders/nope", nil)
    rec := httptest.NewRecorder() // an http.ResponseWriter that records
    h.ServeHTTP(rec, req)

    if rec.Code != http.StatusNotFound {
        t.Fatalf("status = %d, want %d", rec.Code, http.StatusNotFound)
    }
    if !strings.Contains(rec.Body.String(), "order not found") {
        t.Errorf("body = %q, want it to mention the missing order", rec.Body)
    }
}
```

`NewHandler` returns the whole router, so this test also checks the route pattern `GET /api/v1/orders/{id}`. The `stubService` is a two-field struct that returns whatever order or error you give it. The full file adds a table test that maps service errors to status codes (201, 400, 409 and 422).

> **Coming from Java**
>
> These tests run in microseconds because there's no application context to build. You'll write many more of them than you wrote `@WebMvcTest` classes, and that's a good thing.

Golden files

Some outputs are large: a JSON response, a rendered email, a generated report. Spelling them out in Go string literals is painful. A **golden file** stores the expected output in `testdata/`, and the test compares against it. An `-update` flag rewrites the file when the change is intended.

GO httpapi/golden_test.go

```go
// Run `go test ./httpapi -update` to rewrite the golden files.
var update = flag.Bool("update", false, "rewrite golden files")

func assertGolden(t *testing.T, name string, got []byte) {
    t.Helper()
    path := filepath.Join("testdata", name)
    if *update {
        if err := os.WriteFile(path, got, 0o644); err != nil {
            t.Fatal(err)
        }
    }
    want, err := os.ReadFile(path)
    if err != nil {
        t.Fatalf("read golden file (run with -update to create it): %v", err)
    }
    if string(got) != string(want) {
        t.Errorf("response differs from %s\n--- got:\n%s\n--- want:\n%s",
            path, got, want)
    }
}
```

The `go` tool ignores folders named `testdata`, so it's the conventional home for fixtures. The workflow looks like this:

TERMINAL

```
go test ./httpapi/ -update      # write testdata/get_order.json
git diff httpapi/testdata/      # review the change like any other code
go test ./httpapi/              # from now on, compare against it
```

> ⚠️ **Gotcha: -update and ./...**
>
> The `-update` flag only exists in packages that define it. `go test ./... -update` fails in every other package with "flag provided but not defined". Run it on one package at a time. On Windows, also add a `.gitattributes` line such as `testdata/** -text`, so Git doesn't turn the file's line endings into CRLF and break the comparison.

Testing HTTP clients with `httptest.NewServer`

The other direction is code that *calls* an HTTP API. Java teams often use WireMock for this. Go's answer is `httptest.NewServer`, which starts a real HTTP server on a random local port with any handler you like:

GO inventory/client_test.go

```go
func TestAvailable_OK(t *testing.T) {
    // A real HTTP server on a random local port, running our handler.
    srv := httptest.NewServer(http.HandlerFunc(
```

```
        func(w http.ResponseWriter, r *http.Request) {
            if r.URL.Path != "/api/v1/inventory/mug-blue" {
                t.Errorf("unexpected path %q", r.URL.Path)
            }
            w.Header().Set("Content-Type", "application/json")
            w.Write([]byte(`{"product_id":"mug-blue","available":7}`))
        }))
    defer srv.Close()

    c := NewClient(srv.URL, srv.Client())
    got, err := c.Available(t.Context(), "mug-blue")

    if err != nil {
        t.Fatalf("Available() error: %v", err)
    }
    if got != 7 {
        t.Errorf("Available() = %d, want 7", got)
    }
}
```

This works because the client takes its base URL and `*http.Client` as constructor arguments, rather than reading a global. The same trick tests the unhappy paths: the example folder has cases for a 404, a 500 and a server that's too slow for the caller's context deadline. Testing timeouts this way is much harder with mocks.

Integration tests with testcontainers-go

Fakes prove your logic. They can't prove your SQL. For that you need a real Postgres, and testcontainers gives you one in a throwaway Docker container.

The Spring Boot version you probably know:

JAVA OrderRepositoryIT.java

```java
@SpringBootTest
@Testcontainers
class OrderRepositoryIT {
    @Container
    @ServiceConnection
    static PostgreSQLContainer<?> postgres =
        new PostgreSQLContainer<>("postgres:17-alpine");

    @Autowired OrderRepository repository;

    @Test
    void savesAndLoadsAnOrder() { /* ... */ }
}
```

Shared setup with `TestMain`

Starting a container takes a few seconds, so we start one per package, not per test. Go's hook for package-wide setup is `TestMain`. If a test package defines it, `go test` calls it instead of running the tests directly, and you call `m.Run()` when you're ready.

GO postgres/main_integration_test.go

```go
//go:build integration

package postgres_test

// testPool is shared by every test in this package.
var testPool *pgxpool.Pool

func TestMain(m *testing.M) {
    os.Exit(run(m)) // os.Exit skips defers, so the real work lives in run
}
```

The `run` function starts Postgres with the testcontainers-go Postgres module (v0.44.0), connects with pgx and applies the schema:

GO postgres/main_integration_test.go

```go
func run(m *testing.M) int {
    ctx := context.Background()
    ctr, err := tcpostgres.Run(ctx, "postgres:17-alpine",
        tcpostgres.WithDatabase("orderly"),
        tcpostgres.WithUsername("orderly"),
        tcpostgres.WithPassword("orderly"),
        tcpostgres.BasicWaitStrategies(), // wait until it accepts queries
    )
    defer testcontainers.TerminateContainer(ctr) // safe even if ctr is nil
    if err != nil {
        log.Printf("start postgres: %v", err)
        return 1
    }

    dsn, err := ctr.ConnectionString(ctx, "sslmode=disable")
    // ...error check, then pgxpool.New(ctx, dsn) and apply the schema
    return m.Run()
}
```

Each test then empties the tables first, so tests don't see each other's rows:

GO postgres/repo_integration_test.go

```go
func TestOrderRepo_GetMissing(t *testing.T) {
    cleanTables(t)
    repo := postgres.NewOrderRepo(testPool)

    _, err := repo.Get(t.Context(), uuid.NewString())

    assert.True(t, errors.Is(err, order.ErrNotFound), "got %v", err)
}

// cleanTables empties the tables so every test starts from scratch.
func cleanTables(t *testing.T) {
    t.Helper()
    _, err := testPool.Exec(t.Context(), "TRUNCATE order_items, orders")
    require.NoError(t, err)
}
```

> 💡 **Container per test, when you need it**
>
> If one test really needs its own database, start a container inside the test and call `testcontainers.CleanupContainer(t, ctr)` right after `Run`. The Postgres module also has `Snapshot` and `Restore` methods, which reset the database much faster than restarting it.

Keeping slow tests out of the fast loop

The first line, `//go:build integration`, is a **build constraint**: the file is only compiled when you ask for the `integration` tag. So everyday `go test ./...` stays fast and needs no Docker:

TERMINAL

```
go test ./...                       # unit tests only
go test -tags integration ./...     # unit + integration
go vet -tags integration ./...      # make sure tagged files still compile
```

If Docker isn't available, the integration run fails early and clearly. Here's the real output on a Windows laptop with Docker stopped:

OUTPUT

```
2026/11/02 23:15:25 start postgres: run postgres: generic container: get
provider: rootless Docker is not supported on Windows, failed to create
Docker provider
FAIL    example.com/goguide/ch15/postgres   16.360s
```

Some teams use `testing.Short()` instead of a tag: integration tests call `t.Skip` when `-short` is set. Tags are the better default because tagged files aren't even compiled without the tag, so a missing Docker can't slow anyone down.

> **Coming from Java**
>
> This is the Go cousin of Maven's Surefire/Failsafe split, where `*Test` classes run in `test` and `*IT` classes run in `verify`. Here the switch is a build tag instead of a naming pattern and a plugin.

How CI runs them

GitHub's Ubuntu runners and Azure DevOps' Microsoft-hosted Ubuntu agents have Docker installed, so testcontainers works there with no extra setup. A typical pipeline runs unit tests on every push and the integration job alongside them:

YAML .github/workflows/ci.yml (excerpt)

```yaml
integration:
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v7
    - uses: actions/setup-go@v7
      with:
        go-version-file: go.mod
    - run: go test -tags integration -count=1 ./...
```

Chapter 16 builds the full pipeline, including the race detector, linting and coverage.

Testing time

Order expiry, token lifetimes, retry back-off: enterprise code is full of time. Tests that call `time.Sleep` are slow, and tests that use the real clock are flaky. There are two good Go answers.

Inject a clock

In Java you'd inject a `java.time.Clock` and use `Clock.fixed(...)` in tests. The lightest Go equivalent is a function field: `Now func() time.Time`. Production passes `time.Now`; tests pass a function that returns a fixed time.

Orderly's reaper cancels orders that stay `PENDING` for more than 15 minutes:

GO expiry/reaper.go

```go
type Reaper struct {
    Store  Store
    MaxAge time.Duration
    Now    func() time.Time // time.Now in production, a fixed clock in tests
}

func (r *Reaper) RunOnce(ctx context.Context) (int, error) {
    cutoff := r.Now().Add(-r.MaxAge)
    // ...cancel every pending order created before cutoff
}
```

GO expiry/clock_test.go

```
now := time.Date(2026, 11, 2, 12, 0, 0, 0, time.UTC)
store := newMemStore(
    pendingAt("old", now.Add(-20*time.Minute)),
    pendingAt("fresh", now.Add(-5*time.Minute)),
)
r := &Reaper{
    Store:  store,
    MaxAge: 15 * time.Minute,
    Now:    func() time.Time { return now }, // time stands still
}
```

> 💡 **If you're unsure, inject `func() time.Time`**
>
> A function field is enough for most code. Define a `Clock` interface only when you also need fake timers or tickers, and even then, try `synctest` first.

`testing/synctest` (Go 1.25)

A fixed clock works for `RunOnce`, but what about `Run`, the loop that ticks every minute? It uses `time.NewTicker`, which an injected `Now` can't control.

Go 1.25 made the `testing/synctest` package generally available for exactly this. `synctest.Test` runs your test inside a **bubble**: an isolated group of goroutines with its own fake clock. Time inside the bubble only moves forward when every goroutine in it is blocked, and then it jumps straight to the next timer.

> 🧩 **Think of it like a snow globe with its own clock**
>
> Everything inside the globe shares one clock. When nobody inside is doing anything, the clock skips ahead to the next alarm. Sixteen minutes of "waiting" takes no real time at all.

GO expiry/synctest_test.go

```
func TestRun_CancelsAfterMaxAge(t *testing.T) {
    synctest.Test(t, func(t *testing.T) {
        // Inside the bubble, time.Now starts at 2000-01-01 00:00 UTC
        // and only moves when every goroutine here is blocked.
        store := newMemStore(pendingAt("ord-1", time.Now()))
        r := &Reaper{Store: store, MaxAge: 15 * time.Minute, Now: time.Now}

        ctx, cancel := context.WithCancel(t.Context())
        go r.Run(ctx, time.Minute)

        time.Sleep(14 * time.Minute) // returns instantly, fake clock
        synctest.Wait()              // let the reaper finish its tick
        if got := store.status("ord-1"); got != order.StatusPending {
            t.Fatalf("after 14m status = %s, want PENDING", got)
        }

        time.Sleep(2 * time.Minute)
        synctest.Wait()
        if got := store.status("ord-1"); got != order.StatusCancelled {
            t.Fatalf("after 16m status = %s, want CANCELLED", got)
```

```
        }
        cancel() // stop the reaper so the bubble can end cleanly
    })
}
```

The production code is untouched: it uses the real `time` package, and the bubble swaps the clock underneath it. `synctest.Wait()` blocks until every other goroutine in the bubble is stuck waiting, which is how we know the reaper has finished its tick before we look.

OUTPUT

```
=== RUN   TestRun_CancelsAfterMaxAge
--- PASS: TestRun_CancelsAfterMaxAge (0.00s)
```

Sixteen virtual minutes in 0.00 seconds. That's a small win worth celebrating.

> 📝 **Newer Go**
>
> Go 1.27 adds `synctest.Sleep`, which combines `time.Sleep` and `synctest.Wait` in one call. The example sticks to the Go 1.25 API so it runs on both versions.

Fuzzing

Table tests check the cases you thought of. **Fuzzing** checks the ones you didn't: the tool generates thousands of inputs per second, watches which code paths they reach, and mutates the interesting ones further. Java teams use Jazzer for this; Go has it built in since 1.18.

> 🧩 **Think of it like a tireless intern with a keyboard**
>
> You give the intern a few example inputs. They try endless variations, remember which ones made the program do something new, and keep pushing in that direction. When something breaks, they hand you the smallest input that still breaks it.

A Jazzer test looks like this:

JAVA PriceFuzzTest.java

```java
@FuzzTest
void parsePrice(FuzzedDataProvider data) {
    String input = data.consumeRemainingAsString();
    try {
        assertTrue(Prices.parse(input) >= 0);
    } catch (IllegalArgumentException expected) {
        // rejecting bad input is fine
    }
}
```

The Go version fuzzes Orderly's `ParsePrice`, which turns `"12.34"` into 1234 cents. A fuzz test starts with `Fuzz`, takes `*testing.F`, adds seed inputs and states **properties** that must hold for any input:

GO order/price_fuzz_test.go

```go
func FuzzParsePrice(f *testing.F) {
    // Seed corpus: a few valid and invalid inputs to start from.
    for _, seed := range []string{"12.34", "0.99", "7", "1.5", "", "abc"} {
        f.Add(seed)
    }
    f.Fuzz(func(t *testing.T, s string) {
        cents, err := ParsePrice(s)
        if err != nil {
            return // rejecting bad input is fine; we only check good results
        }
        if cents < 0 {
            t.Fatalf("ParsePrice(%q) = %d, want a non-negative value", s, cents)
        }
        // Round trip: formatting and parsing again must give the same cents.
        again, err := ParsePrice(FormatPrice(cents))
        if err != nil || again != cents {
            t.Fatalf("round trip of %q: got %d, %v; want %d",
                s, again, err, cents)
        }
    })
}
```

The first version of `ParsePrice` had no upper limit. Here's the real run that found the bug, in under three seconds (progress lines trimmed and long lines wrapped to fit the page):

TERMINAL

```
go test -run=NONE -fuzz=FuzzParsePrice -fuzztime=60s ./order/
```

OUTPUT

```
fuzz: elapsed: 0s, gathering baseline coverage: 6/6 completed, now fuzzing
with 20 workers
fuzz: minimizing 54-byte failing input file
--- FAIL: FuzzParsePrice (2.63s)
    --- FAIL: FuzzParsePrice (0.00s)
        price_fuzz_test.go:16: ParsePrice("2000000000000000000") =
        -29141848108050677776, want a non-negative value

    Failing input written to testdata\fuzz\FuzzParsePrice\4f1e47bcc3ff5164
    To re-run:
    go test -run=FuzzParsePrice/4f1e47bcc3ff5164
```

A long string of digits overflowed `int64` and wrapped around to a negative price. The fix is a cap inside the parsing loop:

GO order/price.go

```
cents = cents*10 + int64(r-'0')
if cents > maxPriceCents {
    return 0, fmt.Errorf("%w: %q is too large", ErrBadPrice, s)
}
```

Here's the best part. The failing input was saved to `testdata/fuzz/FuzzParsePrice/`. Commit that file, and plain `go test` runs it as a regular case forever, so the bug can't come back. Without `-fuzz`, a fuzz test only runs its seeds and saved inputs, which takes milliseconds.

> 💡 **Fuzzing in CI**
>
> Keep `go test ./...` fast: it runs the seeds automatically. Add a nightly job that runs `go test -fuzz=FuzzParsePrice -fuzztime=10m` for each fuzz target (`-fuzz` takes one target per package). Good fuzz candidates are parsers, decoders, validators and anything that reads untrusted input.

Benchmarks

Before you optimise anything, measure it. A **benchmark** is a function that runs some code many times and reports the average cost per run. Java's equivalent is JMH:

JAVA PriceBenchmark.java

```
@State(Scope.Benchmark)
public class PriceBenchmark {
    @Benchmark
    public String formatPrice() {
        return Prices.format(123456);
    }
}
```

A Go benchmark starts with `Benchmark`, takes `*testing.B` and, since Go 1.24, loops with `b.Loop()`:

GO order/bench_test.go

```
func BenchmarkTotalCents(b *testing.B) {
    items := make([]Item, 100)
    for i := range items {
        items[i] = Item{ProductID: "p", Quantity: i, UnitPriceCents: 199}
    }
    o := Order{Items: items} // setup before the loop is not timed

    for b.Loop() {
        o.TotalCents()
    }
}
```

`b.Loop()` decides how many iterations are enough, excludes the setup above the loop from the timing, and stops the compiler from optimising the call away. Older code uses `for i := 0; i < b.N; i++` with `b.ResetTimer()`; you'll still see it, but prefer `b.Loop()` in new code.

TERMINAL

```
go test -run=NONE -bench=. -benchmem ./order/
```

OUTPUT

```
cpu: 12th Gen Intel(R) Core(TM) i7-12800H
BenchmarkParsePrice-20     32408781     37.76 ns/op    0 B/op   0 allocs/op
BenchmarkTotalCents-20     18640978    110.3 ns/op     0 B/op   0 allocs/op
BenchmarkFormatPrice-20    14041850     82.60 ns/op    8 B/op   1 allocs/op
```

Reading a line: `-20` is the number of CPUs Go used, then the iteration count, the time per operation, the bytes allocated per operation and the number of allocations. `-run=NONE` skips the regular tests so only benchmarks run.

Comparing runs with benchstat

One benchmark run is noisy, especially on a laptop. **benchstat** compares several runs of "before" and "after" and tells you whether the difference is real. Here we rewrote `FormatPrice` from `fmt.Sprintf` to `strconv.AppendInt` and compared ten runs of each:

TERMINAL

```
go test -run=NONE -bench=FormatPrice -benchmem -count=10 ./order/ > old.txt
# ...change the code...
go test -run=NONE -bench=FormatPrice -benchmem -count=10 ./order/ > new.txt
go run golang.org/x/perf/cmd/benchstat@latest old.txt new.txt
```

OUTPUT

```
               |    old.txt    |               new.txt                |
               |    sec/op     |    sec/op     vs base                |
FormatPrice-20   285.80n ± 25%   58.81n ± 20%  -79.42% (p=0.000 n=10)

               |   old.txt    |              new.txt               |
               |    B/op      |    B/op     vs base                |
FormatPrice-20   16.000 ± 0%   8.000 ± 0%  -50.00% (p=0.000 n=10)
```

The `± 25%` is laptop noise, but `p=0.000` says the improvement is real. If benchstat prints `~` instead of a percentage, the difference is within the noise and you haven't proved anything yet.

> **Version note**
>
> The current benchstat needs Go 1.26 or later. On Go 1.25, `go run` downloads a newer toolchain automatically and carries on. Chapter 17 goes much deeper into benchmarks, profiles and allocation tuning.

Coverage

Coverage tells you which statements your tests executed. Java teams get this from JaCoCo; in Go it's one flag:

TERMINAL

```
go test -cover ./...
```

OUTPUT

```
ok   .../ch15/expiry      1.776s  coverage: 88.2% of statements
?    .../ch15/failing             [no test files]
ok   .../ch15/httpapi     1.866s  coverage: 92.6% of statements
ok   .../ch15/inventory   3.677s  coverage: 87.5% of statements
ok   .../ch15/order       2.723s  coverage: 97.1% of statements
     .../ch15/postgres            coverage: 0.0% of statements
ok   .../ch15/service     3.250s  coverage: 87.5% of statements
     .../ch15/service/mockery     coverage: 0.0% of statements
     .../ch15/service/mocks       coverage: 0.0% of statements
```

(The module path `example.com/goguide` is shortened to `...` to fit the page.)

`postgres` shows 0% because its only tests are behind the `integration` tag, `failing` only has tests behind its own tag, and the two mock packages are generated code with no tests of their own. For detail, write a profile and look at it per function or in the browser:

TERMINAL

```
go test -coverprofile=cover.out ./order/ ./service/
go tool cover -func=cover.out      # per-function table
go tool cover -html=cover.out      # coloured source in your browser
```

OUTPUT

```
.../ch15/order/order.go:45:      TotalCents     100.0%
.../ch15/order/order.go:54:      Validate        90.0%
.../ch15/order/price.go:19:      ParsePrice     100.0%
.../ch15/order/price.go:41:      FormatPrice    100.0%
.../ch15/service/place.go:12:    PlaceOrder      90.9%
.../ch15/service/place.go:38:    GetOrder        75.0%
.../ch15/service/service.go:37:  New            100.0%
total:                           (statements)    94.0%
```

By default, a package's tests only count coverage for that package. `-coverpkg` widens it, which shows how much of the whole module the handler tests exercise:

TERMINAL

```
go test -cover -coverpkg=./... ./httpapi/
```

OUTPUT

```
ok   .../ch15/httpapi   3.550s   coverage: 41.0% of statements in ./...
```

> 💡 **Use coverage as a flashlight, not a target**
>
> Coverage shows you code no test touches. It can't tell you whether the tests check anything useful. A healthy team looks at the uncovered lines in `-html` during review, rather than enforcing a number. If you do set a gate, something like 70 to 80% for business packages is common.

Go can also collect coverage from a running binary: build it with `go build -cover`, set the `GOCOVERDIR` environment variable, and run your end-to-end tests against it. That's handy for measuring what your integration suite really covers.

The race detector

A **data race** happens when two goroutines touch the same memory at the same time and at least one of them writes. Races cause the worst kind of bug: rare, random and impossible to reproduce on your machine. Java has no built-in equivalent; you rely on careful reviews and tools like jcstress.

Go's race detector watches memory accesses while your tests run and reports any race it sees, with the stack traces of both goroutines. You turn it on with `-race`:

TERMINAL

```
go test -race ./...
```

> ⚠️ **Gotcha: -race needs cgo, and on Windows a C compiler**
>
> The race detector is built on a C library, so it needs cgo. On a Windows machine with no C compiler (cgo is then off), you'll see this real message:
>
> OUTPUT
>
> ```
> go: -race requires cgo; enable cgo by setting CGO_ENABLED=1
> ```
>
> To run it locally on Windows, install a MinGW-w64 GCC (for example through MSYS2), put it on your `PATH` and set `CGO_ENABLED=1`. Many Windows-based teams skip that and let the Linux CI runners, which have GCC, run `-race` for them. macOS doesn't need a C compiler for `-race`.

The detector only finds races that actually happen during the run, so it's only as good as your tests' concurrency. It also makes tests slower: the Go docs say memory may grow 5 to 10 times and run time 2 to 20 times. That's why most teams run it in CI on every pull request rather than in every local loop. Chapter 16 shows where it fits in the pipeline.

Example tests that double as documentation

An **example test** is a function named `ExampleXxx` with an `// Output:` comment. `go test` runs it and checks that what it prints matches the comment. The same function appears in `go doc` and on pkg.go.dev as usage documentation, so your docs can't silently go stale.

GO order/example_test.go

```go
func ExampleParsePrice() {
    cents, err := order.ParsePrice("12.5")
    if err != nil {
        fmt.Println("error:", err)
        return
    }
    fmt.Println(cents)

    _, err = order.ParsePrice("12.345")
    fmt.Println(err)
    // Output:
    // 1250
    // bad price: "12.345"
}
```

The naming links each example to what it documents: `ExampleParsePrice` for a function, `ExampleOrder_TotalCents` for the `TotalCents` method on `Order`, and plain `Example` for the package. An example without an `// Output:` comment is compiled but not run.

> **Coming from Java**
>
> Think of it as a Javadoc `{@snippet}` that's compiled and asserted on every build. Write one for each public function that people often get wrong.

Black-box and white-box tests

Notice that `example_test.go` starts with `package order_test`, not `package order`. Go allows one extra package in a folder: the original name plus `_test`. Tests in that package can only see exported names, exactly like a real caller.

	`package order`	`package order_test`
Nickname	white-box	black-box
Can see	everything, unexported too	exported API only
Good for	tricky internals, unexported helpers	API behaviour, `Example` tests
Java cousin	test in the same package	test in another package

Black-box tests also get you out of **import cycles**. If a test needs a helper package that itself imports the package under test, only a `_test` package is allowed to import it. Our gomock and mockery tests use `package service_test` for the plainer reason: they call `service.New` exactly the way production wiring

does.

> 💡 **If you're unsure, start white-box**
>
> Most teams write `package order` tests by default, since they're simplest, and use `_test` packages for examples and API-level tests. If a black-box test needs one internal, add a file named `export_test.go` in `package order` that re-exports it, for example `var ParseForTest = parse`. Because it ends in `_test.go`, it never ships.

The test pyramid in Go

The classic test pyramid holds up well in Go, and the tools map onto it neatly. Because unit tests and `httptest` handler tests are so fast, Go codebases often have an even wider base than Java ones.

Contract testing with Pact Go, briefly

When Orderly's billing team calls our orders API, how do we make sure we don't break them? **Contract testing** answers that: the consumer records the requests it makes and the responses it expects as a "pact" file, and the provider's CI replays that pact against the real service.

Pact Go (`github.com/pact-foundation/pact-go/v2`, v2.9.0) brings this to Go. Consumer tests use the `consumer` package to describe interactions against a Pact mock server, and the provider side uses `provider.HTTPVerifier` to verify pacts in CI. Two things to know before you adopt it:

- Pact Go v2 is built on the shared Pact core written in Rust, so it needs cgo and a native library installed with its `pact-go install` command. On Windows that means a GCC toolchain, as with `-race`.
- Contract tests pay off when several teams own different services. For one team owning both sides, an integration test is usually enough.

Recap

- Tests live in `_test.go` files next to the code; `TestXxx(t *testing.T)` is all you need, and `go test ./...` runs everything.
- `t.Error` records and continues; `t.Fatal` records and stops. Use `t.Helper`, `t.Cleanup` and `t.Context` to keep tests tidy.

- Table-driven tests with named cases and `t.Run` are Go's `@ParameterizedTest`. Run one case with `-run 'TestName/case'`.
- testify's `require` and `assert` are a fine default for app teams; plain `if` statements suit libraries.
- Declare small interfaces where they're used and write fakes. Use gomock (`go.uber.org/mock`, not the archived `golang/mock`) or mockery when you need to verify calls.
- `httptest.NewRecorder` tests handlers without a server; `httptest.NewServer` gives HTTP clients a real server to talk to.
- Put testcontainers tests behind `//go:build integration`, share the container with `TestMain`, and run them in CI.
- Use `synctest` for time, `-fuzz` for parsers, `b.Loop` and benchstat for speed, `-cover` as a flashlight and `-race` in CI.

Try it yourself

🏋 Exercise 1: Grow the price table

Add three cases to `TestParsePrice` in `examples/ch15-testing/order/price_test.go`: `"0"`, `"00.10"` and `" 5"` (with a leading space). Decide what each should return, run `go test -v -run TestParsePrice ./order/`, and fix the parser or the table until they agree.

Hint: run only your new case with `-run 'TestParsePrice/leading_space'`.

🏋 Exercise 2: A fake for cancellation

Add a `CancelOrder(ctx, id)` method to `OrderService` that loads the order, refuses if it's `SHIPPED`, and saves it as `CANCELLED`. Test it with the existing `fakeRepo`, using a table with "pending order", "shipped order" and "missing order" cases.

Hint: you'll need no new interface methods. `Get` and `Save` are enough.

🏋 Exercise 3: Golden file for errors

Add a golden-file test for the 404 response of `GET /api/v1/orders/{id}`. Create the file with `-update`, look at it, then change the error message in `handler.go` and watch the test fail with a readable diff.

🏋 Exercise 4 (stretch): Fuzz a second parser

Write `ParseOrderID(s string) (int, error)` that accepts IDs like `"ORD-000123"`, with exactly six digits. Write a table test first, then a fuzz test with the property `FormatOrderID(id)` round-trips. Run the fuzzer for a minute and commit anything it finds to `testdata/`.

Hint: `strings.CutPrefix` and `fmt.Sprintf("ORD-%06d", id)` will get you most of the way.

Further reading

- testing package: the full API, including `T`, `B`, `F` and the rules for examples.
- Go Wiki: Table Driven Tests: the community's guide to the idiom.
- Go Fuzzing: how fuzzing works, the corpus format and best practices.
- Testing concurrent code with testing/synctest: the Go blog's introduction to bubbles and fake time.
- Data Race Detector: what `-race` finds, its costs and its options.
- testify: `assert`, `require`, `mock` and `suite` documentation.
- uber-go/mock: the maintained gomock and mockgen.
- testcontainers-go Postgres module: options, snapshots and wait strategies.

CHAPTER 16

Tooling and Code Quality

> *Good tooling turns "please remember to..." into "the computer already checked", and Go gives you most of it out of the box.*

🎯 **What you'll learn**

- Format, vet and lint Go code, and set up golangci-lint v2 for a team
- Set up your editor with gopls and debug programs and tests with Delve
- Generate code with `go generate` and pin tool versions with `tool` directives
- Write a practical Makefile and pre-commit hook
- Build a complete CI pipeline on GitHub Actions and Azure Pipelines
- Manage dependencies, scan for vulnerabilities and modernise old code with `go fix`

The big picture

A Java project's quality setup is usually a collection of plugins: Spotless or an IDE profile for formatting, Checkstyle and PMD for style, SpotBugs or Error Prone for bugs, SonarQube for the dashboard, and a Maven lifecycle to run them all. Each one has its own configuration file and its own opinions.

Go starts from a different place. The `go` command already formats, vets, tests, generates code, manages modules and fixes old idioms. On top of that, the community settled on a small set of extra tools. The result is that two Go projects from different companies feel remarkably alike, which makes it easy to move between teams.

🧩 **Think of it like a shared kitchen with the house rules on the wall**

In a shared kitchen, arguments start over where the spoons go. Go pins one set of rules to the wall (gofmt) and puts a friendly inspector at the door (vet and the linters). Nobody debates the spoons any more, so everyone gets on with cooking.

Java ↔ Go at a glance

Concept	Java / Spring	Go
Formatter	IDE profile, Spotless	`gofmt` / `goimports` (one style)
Bug finding	SpotBugs, Error Prone	`go vet`, staticcheck
Lint runner and gate	Checkstyle, PMD, SonarQube	golangci-lint
IDE engine	IntelliJ	gopls (VS Code, Neovim), GoLand

Concept	Java / Spring	Go
Debugger	IntelliJ debugger	Delve (`dlv`)
Code generation	annotation processors, Lombok, MapStruct	`go generate` + generators
Build lifecycle	Maven phases, Gradle tasks	`go` commands + Makefile or Taskfile
Pinning tool versions	plugin versions in `pom.xml`	`tool` directives in go.mod
Dependency tree	`mvn dependency:tree`	`go mod graph`, `go mod why`
Vulnerability scan	OWASP Dependency-Check, Snyk	govulncheck
Automated refactors	OpenRewrite	`go fix`

Full code: `examples/ch16-tooling/`

Formatting: gofmt and goimports

In most Java teams, someone eventually opens a pull request that's half real change and half reformatting. Go removed that whole category of problem by shipping one formatter with no options.

TERMINAL

```
gofmt -l .          # list files that aren't formatted
gofmt -w .          # rewrite them in place
go fmt ./...        # the same, for every package in the module
```

`gofmt` uses tabs for indentation, aligns struct fields and comments, and has no settings to argue about. As the Go proverb says: "Gofmt's style is no one's favorite, yet gofmt is everyone's favorite."

goimports does everything gofmt does and also adds missing imports, removes unused ones and sorts them into groups. Most editors run it (or the equivalent in gopls) every time you save. If your team wants a slightly stricter style, **gofumpt** (v0.12.0) is a popular superset of gofmt.

> 💡 **If you're unsure, format on save and check in CI**
>
> Turn on format-on-save in your editor, and add a CI step that fails when `gofmt -l .` prints anything. You'll never discuss formatting in a code review again.

go vet: the careful friend

`go vet` looks for code that compiles but is almost certainly wrong. It's the Go cousin of `javac -Xlint` plus a few Error Prone checks. To see it in action, here's a file full of classic mistakes:

GO Avoid

```go
func Dial(host string, port int) (net.Conn, error) {
    return net.Dial("tcp", fmt.Sprintf("%s:%d", host, port))
}

func Report(total int64) {
    fmt.Printf("total: %d cents\n", "oops")
}
```

TERMINAL

```
go vet ./...
```

OUTPUT

```
inventory\legacy.go:15:37: address format "%s:%d" does not work with IPv6
inventory\legacy.go:23:21: fmt.Printf format %d has arg "oops" of wrong type string
```

The first finding comes from the `hostport` check added in Go 1.25: `"%s:%d"` breaks for IPv6 addresses, and `net.JoinHostPort` is the fix. The second is the classic `Printf` mismatch. Remember from Chapter 15 that `go test` runs a subset of vet automatically, so the `Printf` one would fail your tests too.

staticcheck: the deeper review

staticcheck (2026.2.2, module `honnef.co/go/tools` v0.8.2) is the most respected third-party analyser. It finds real bugs, suggests simplifications and checks style. Its checks are grouped by prefix: `SA` for likely bugs, `S` for simplifications, `ST` for style and `QF` for quick fixes.

TERMINAL

```
go install honnef.co/go/tools/cmd/staticcheck@latest
staticcheck ./...
```

OUTPUT

```
inventory\legacy.go:19:9: should use strings.EqualFold instead (SA6005)
inventory\legacy.go:23:13: Printf format %d has arg #1 of wrong type string (SA5009)
```

`SA6005` caught `strings.ToLower(a) == strings.ToLower(b)`, which allocates two new strings only to compare them. You'll rarely run staticcheck on its own, though. It's built into golangci-lint, which is where most teams meet it.

golangci-lint: one runner for everything

Rather than running a dozen tools one by one, Go teams use **golangci-lint**, which runs about a hundred linters in one fast pass, sharing the parsing work between them. Think of it as Checkstyle, PMD and SpotBugs in a single binary, with SonarQube-style quality gates done in CI.

Java tool	What it does	golangci-lint equivalent
Checkstyle	style and naming rules	`revive`, the formatters
SpotBugs	likely bugs	`govet`, `staticcheck`, `errcheck`
PMD	wasteful or dead code	`unused`, `unparam`, `ineffassign`
Find Security Bugs	security smells	`gosec`

> 📝 **Version note: golangci-lint v2 changed the config**
>
> golangci-lint v2 (the current major version, v2.15.0 at the time of writing) uses a new config format. The file starts with `version: "2"`, linters live under `linters.enable` and `linters.settings`, exclusions moved to `linters.exclusions`, and formatters got their own `formatters` section. Older blog posts show the v1 format. If you inherit a v1 file, `golangci-lint migrate` converts it.

A sensible enterprise config

Here's the config from the example folder, in two halves. The first half picks the linters:

YAML .golangci.yml (part 1)

```yaml
version: "2"

run:
  timeout: 5m
  build-tags:
    - integration # lint the integration tests too

linters:
  default: standard # errcheck, govet, ineffassign, staticcheck, unused
  enable:
    - bodyclose # HTTP response bodies are closed
    - errorlint # errors are compared with errors.Is / errors.As
    - gosec # common security mistakes
    - misspell # typos in comments and strings
    - nilerr # returning nil when an error was checked
    - noctx # outgoing HTTP requests carry a context
    - revive # style rules, the successor of golint
    - sqlclosecheck # sql.Rows and statements are closed
    - unconvert # unnecessary type conversions
    - usestdlibvars # http.MethodGet instead of "GET"
```

The second half tunes them, skips generated code and configures the formatters:

YAML .golangci.yml (part 2)

```yaml
  settings:
    errcheck:
      check-type-assertions: true # flag v := x.(T) without ok
    revive:
      rules:
        - name: exported # exported names need doc comments
        - name: var-naming # userId -> userID
        - name: package-comments
        - name: error-strings # lower case, no trailing punctuation
  exclusions:
    generated: lax # skip files marked "Code generated ... DO NOT EDIT"
    presets:
      - std-error-handling # e.g. unchecked Close on readers
      - common-false-positives
    rules:
      - path: _test\.go
        linters:
          - gosec # test fixtures may use weak random numbers etc.

formatters:
  enable:
    - gofmt
    - goimports
  settings:
    goimports:
      local-prefixes:
        - example.com/goguide # our imports go in their own group
```

`golangci-lint config verify` checks the file against the official schema, which catches typos before CI does. Running it on our file of mistakes gives this (trimmed):

OUTPUT

```
inventory\legacy.go:28:9: Error return value of `f.Write` is not checked
(errcheck)
inventory\legacy.go:33:15: G107: Potential HTTP request made with variable
url (gosec)
inventory\legacy.go:15:37: hostport: address format "%s:%d" does not work
with IPv6 (govet)
inventory\legacy.go:14:1: exported: exported function Dial should have
comment or be unexported (revive)
inventory\legacy.go:19:9: SA6005: should use strings.EqualFold instead
(staticcheck)
14 issues:
* errcheck: 1
* gosec: 2
* govet: 2
* revive: 8
* staticcheck: 1
```

`golangci-lint fmt` applies the configured formatters, so one tool handles both linting and formatting.

> ⚠️ **Gotcha: install golangci-lint as a binary**
>
> The golangci-lint docs recommend the prebuilt binary and warn that `go install`, `go get` and `tool` directive installs "aren't guaranteed to work", because your Go version and dependency upgrades change what gets built. On Windows use `choco install golangci-lint` or `scoop install main/golangci-lint`; in CI use the official GitHub Action or install script, as shown later in this chapter.

> 💡 **Adopting a linter on an old codebase**
>
> Turning on twenty linters in a five-year-old service produces thousands of findings and one very sad team. Use `golangci-lint run --new-from-rev=origin/main` in CI, so only issues in new or changed code fail the build. When you must silence a finding, say why: `//nolint:gosec // the URL comes from trusted config`.

Your editor: gopls

gopls (say "go please") is the official Go language server: the engine behind completion, go-to-definition, rename, find references and inline diagnostics. VS Code's Go extension, Neovim, Emacs, Zed and Helix all use it. JetBrains GoLand has its own engine and will feel very familiar if you live in IntelliJ.

For VS Code, these workspace settings give you format-on-save, import organising and golangci-lint v2 diagnostics:

JSON .vscode/settings.json

```
{
  "go.lintTool": "golangci-lint-v2",
  "go.lintOnSave": "package",
  "[go]": {
    "editor.formatOnSave": true,
    "editor.codeActionsOnSave": {
      "source.organizeImports": "explicit"
    }
  }
}
```

> 💡 **If you're unsure, pick by habit**
>
> If IntelliJ's keymap is in your fingers, use GoLand. Otherwise, VS Code with the official Go extension is free, fast and what most Go tutorials assume. Both use Delve for debugging, so everything in the next section applies to either.

Debugging with Delve

Go's debugger is **Delve**, run as `dlv` (v1.27.2). VS Code and GoLand drive it for you when you click in the gutter, but knowing the command line pays off when you debug in a container or over SSH.

TERMINAL

```
go install github.com/go-delve/delve/cmd/dlv@latest
dlv debug ./cmd/orderly-lite                    # build and debug a program
dlv test ./internal/order -- -test.run TestDiscountCents   # debug one test
dlv attach 12345                                # attach to a running process
```

`dlv debug` and `dlv test` compile with optimisations turned off, so every variable is visible. If you build a binary yourself for debugging, add `-gcflags=all="-N -l"` for the same effect.

These commands cover most sessions:

Command	Short	What it does
`break file.go:33`	`b`	set a breakpoint
`condition 1 expr`	`cond`	only stop when `expr` is true
`continue`	`c`	run to the next breakpoint
`next` / `step` / `stepout`	`n` / `s`	step over / into / out
`print expr` / `locals`	`p`	inspect values
`stack` / `goroutines`	`bt` / `grs`	call stack, all goroutines

Here's a real session debugging Orderly's volume discount. We only care about the line with 50 or more units, so we use a **conditional breakpoint**, the same feature as right-clicking a breakpoint in IntelliJ (output trimmed):

OUTPUT

```
(dlv) break internal/order/discount.go:33
Breakpoint 1 set at 0x7ff7f20bde09 for .../order.DiscountCents()
(dlv) condition 1 it.Quantity >= 50
(dlv) continue
> [Breakpoint 1] .../order.DiscountCents() ./discount.go:33
    32:     for _, it := range items {
=>  33:         d := LineDiscountCents(it)
    34:         total += d
(dlv) print it
example.com/goguide/ch16/internal/order.Item {ProductID: "cup",
Quantity: 60, UnitPriceCents: 1000}
(dlv) print total
300
(dlv) stack
0  .../order.DiscountCents at ./discount.go:33
1  .../order.TestDiscountCents at ./discount_test.go:11
2  testing.tRunner at testing/testing.go:1934
```

It skipped the first two items and stopped at the 60-unit line, with `total` already holding the 5% discount from the earlier line. In VS Code, the same session is a `launch.json` entry; the example folder includes one for the program and one for this test:

JSON .vscode/launch.json (excerpt)

```
{
  "name": "Debug TestDiscountCents",
  "type": "go",
  "request": "launch",
  "mode": "test",
  "program": "${workspaceFolder}/internal/order",
  "args": ["-test.run", "TestDiscountCents"]
}
```

> 💡 **The goroutines view is your friend**
>
> When a service hangs, `goroutines` (or the call-stack panel in your editor) lists every goroutine and where it's blocked. A pile of goroutines waiting on the same channel or mutex is usually the whole story.

Code generation with go generate

Java hides a lot of generated code: Lombok writes getters during compilation, MapStruct writes mappers in an annotation processor, and you never see the result. Go makes generation explicit. You write a `//go:generate` comment, run `go generate ./...` yourself, and **commit the generated files** so they're reviewed like any other code.

> 🧉 **Coming from Java**
>
> `go build` never runs generators. That feels like a step backwards at first, but it means builds are fast and predictable, and you can read every line that ends up in your binary. There's no magic to debug.

Here's a common case. Orderly's event types are integer constants, and we want readable names in logs:

GO internal/order/event.go

```
//go:generate go tool stringer -type=EventType -trimprefix=Event

// EventType identifies a domain event published on orderly.orders.
type EventType int

// The events Orderly publishes. The String method is generated.
const (
    EventOrderPlaced EventType = iota + 1
    EventOrderCancelled
    EventStockReserved
)
```

`go generate ./...` writes `eventtype_string.go` with a `String()` method, so `fmt.Println(order.EventOrderPlaced)` prints `OrderPlaced` instead of `1`. The file starts with `// Code generated by "stringer ..."; DO NOT EDIT.`, which tells linters and reviewers to skip it.

These are the generators you'll meet most in enterprise Go:

Generator	Produces	Java cousin
stringer	`String()` for enums	an enum's `name()`
mockgen, mockery	mocks for interfaces	Mockito (at runtime)
sqlc	type-safe Go from SQL queries	jOOQ, Spring Data queries
oapi-codegen	server stubs and clients from OpenAPI	openapi-generator
protoc-gen-go	gRPC and Protobuf types	protobuf-maven-plugin

Chapter 9 covers sqlc (v1.31.1) and Chapter 7 covers oapi-codegen (v2.8.2) in depth.

> 💡 **Check that generated code is fresh**
>
> Add a CI step that runs `go generate ./...` and then `git diff --exit-code`. If someone changed an interface but forgot to regenerate its mock, the build fails with a clear diff.

Pinning tools with `tool` directives (Go 1.24)

Maven pins plugin versions in `pom.xml`, so everyone runs the same Surefire. Go 1.24 added the same idea: a `tool` line in go.mod, added with `go get -tool`:

TERMINAL

```
go get -tool golang.org/x/tools/cmd/stringer@v0.49.0
go tool stringer -help         # run it, at exactly the pinned version
go tool                        # list the tools this module knows about
```

GO.MOD go.mod (trimmed)

```
module example.com/goguide/ch16

go 1.25.0

tool golang.org/x/tools/cmd/stringer
```

To remove a tool, run `go get -tool golang.org/x/tools/cmd/stringer@none`.

> ⚠️ **Gotcha: a tool's requirements become your module's requirements**
>
> A `tool` line adds the tool's dependencies to your module graph, including its minimum Go version. The newest x/tools needs Go 1.26, which is why the example pins v0.49.0, the last version that supports Go 1.25. Look at what a careless upgrade did in a scratch copy:
>
> OUTPUT
>
> ```
> $ go get -u golang.org/x/tools
> go: golang.org/x/tools@v0.52.0 requires go >= 1.26.0; switching to go1.26.10
> go: upgraded go 1.25.0 => 1.26.0
> go: upgraded golang.org/x/tools v0.49.0 => v0.52.0
> ```
>
> One command quietly raised the Go version for the whole service.

So where should each tool live? This is a good default:

Kind of tool	How to run it	Examples
Small generators the build depends on	`tool` directive + `go tool`	stringer, mockgen, oapi-codegen
Big standalone tools	prebuilt binary, pinned in CI	golangci-lint
Occasional tools	`go run module/cmd@version`	govulncheck, benchstat, mockery

`go run pkg@version` builds the tool from its own go.mod, outside your module, so it can't touch your dependencies.

Task runners: Makefile, Taskfile and Maven phases

Maven gives you a lifecycle: `validate`, `compile`, `test`, `package`, `verify`, `install`, `deploy`. Go has no lifecycle, because `go build` and `go test` already know how to compile everything. What teams still want is a set of **shared verbs**, so `make test` means the same thing on every laptop and in CI.

Most Go projects use a Makefile. The one in the example folder starts with a few variables:

MAKEFILE Makefile (variables)

```
BINARY  := orderly-lite
MAIN    := ./cmd/orderly-lite
VERSION ?= $(shell git describe --tags --always --dirty 2>/dev/null || echo dev)
LDFLAGS := -s -w -X main.version=$(VERSION)
EXE     := $(shell go env GOEXE)

# The race detector needs cgo. Without a C compiler, run: make test RACE=
RACE ?= -race

.DEFAULT_GOAL := help
```

Each target is one or two `go` commands, with a `##` comment that explains it. Remember that recipe lines must start with a tab, not spaces:

MAKEFILE Makefile (some of the targets)

```
fmt-check: ## Fail if any file is not gofmt-ed
    @test -z "$$(gofmt -l .)" || { gofmt -l .; exit 1; }

lint: ## Run golangci-lint
    golangci-lint run

test: ## Unit tests with the race detector
    go test $(RACE) -count=1 ./...

test-integration: ## Unit + integration tests (needs Docker)
    go test $(RACE) -count=1 -tags integration ./...

build: ## Build bin/orderly-lite
    CGO_ENABLED=0 go build -trimpath -ldflags "$(LDFLAGS)" \
        -o bin/$(BINARY)$(EXE) $(MAIN)
```

A few details are worth pointing out:

- `-X main.version=$(VERSION)` stamps the Git version into the binary at build time, which is how `orderly-lite` knows its own version.
- `$(shell go env GOEXE)` is `.exe` on Windows and empty elsewhere, so one Makefile works on both.
- The `## comments` feed a `help` target, so running `make` with no arguments lists everything:

OUTPUT

```
help              List the targets
fmt               Format all code (gofmt + goimports)
fmt-check         Fail if any file is not gofmt-ed
vet               Run go vet
lint              Run golangci-lint
test              Unit tests with the race detector
test-integration  Unit + integration tests (needs Docker)
cover             Coverage report in coverage.html
generate          Regenerate code (stringer, mocks, ...)
build             Build bin/orderly-lite
run               Run the app locally
vuln              Scan dependencies for known vulnerabilities
tidy              Tidy go.mod and go.sum
ci                Everything CI runs
```

On a Windows laptop without a C compiler, `make test` stops with `go: -race requires cgo`, and `make test RACE=` runs the same tests without the race detector.

Taskfile, if your team is cross-platform

Make isn't installed on Windows by default (you can get it from Chocolatey or Scoop), and its tab rules trip everyone up once. **Task** (taskfile.dev, v3.54.0) is a Go-based alternative that reads a YAML file and runs the same on every OS:

YAML Taskfile.yml (excerpt)

```yaml
version: "3"

vars:
  BINARY: orderly-lite

tasks:
  build:
    desc: Build bin/orderly-lite
    sources: ["**/*.go", "go.mod", "go.sum"]
    generates: ["bin/{{.BINARY}}{{exeExt}}"]
    cmds:
      - go build -trimpath -o bin/{{.BINARY}}{{exeExt}} ./cmd/orderly-lite
```

Because it knows the sources and outputs, a second `task build` with no changes is skipped:

OUTPUT

```
task: [build] go build -trimpath -o bin/orderly-lite.exe ./cmd/orderly-lite
task: Task "build" is up to date
```

> 💡 **If you're unsure, use a Makefile**
>
> Make is everywhere on Linux and macOS, every CI image has it, and every Go developer can read one. Choose Task if most of your team develops on Windows. Whichever you pick, keep each target to a line or two of `go` commands, so people can still run them by hand.

Pre-commit hooks

CI catches problems in minutes; a Git hook catches them in seconds, before the commit even exists. The simplest version is a shell script that lives in the repository:

TERMINAL .githooks/pre-commit

```
#!/bin/sh
# Fast checks before every commit. Enable once per clone with:
#   git config core.hooksPath .githooks
set -e

files=$(git diff --cached --name-only --diff-filter=ACM -- '*.go')
[ -z "$files" ] && exit 0

unformatted=$(gofmt -l $files)
if [ -n "$unformatted" ]; then
    echo "These files need formatting (run: make fmt):"
    echo "$unformatted"
    exit 1
```

```
fi

go vet ./...
golangci-lint run --new-from-rev=HEAD
```

If your organisation already uses the **pre-commit** framework (pre-commit.com), golangci-lint publishes official hooks for it:

YAML .pre-commit-config.yaml

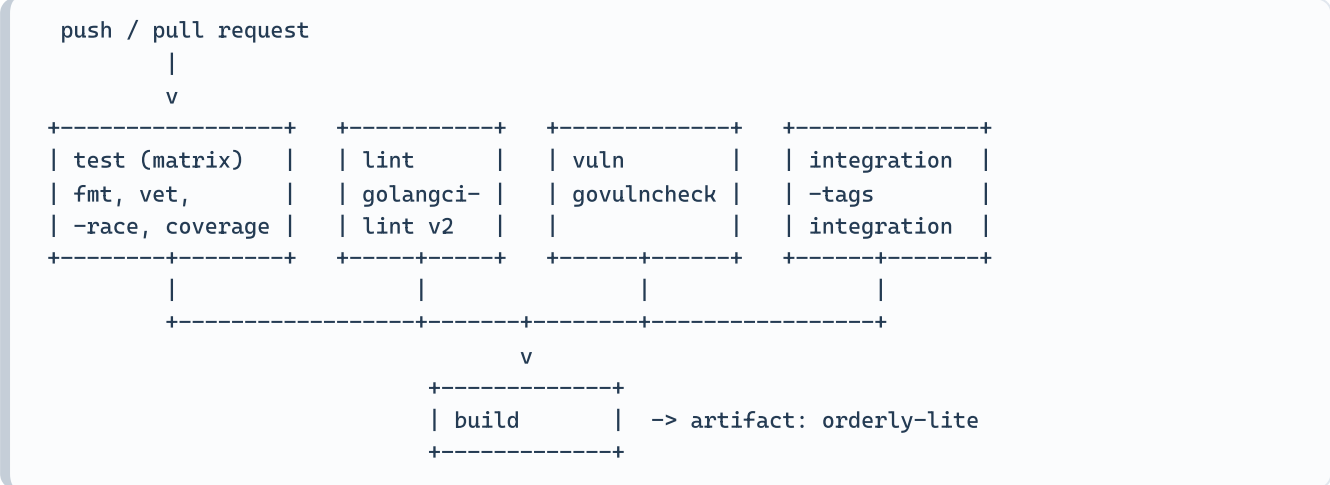
```yaml
repos:
  - repo: https://github.com/golangci/golangci-lint
    rev: v2.15.0
    hooks:
      - id: golangci-lint-fmt
      - id: golangci-lint
```

lefthook is a third option, popular with Go teams because it's a single Go binary with no Python needed.

> 💡 **Keep hooks fast**
>
> A hook that takes thirty seconds gets bypassed with `--no-verify`. Format and vet in the hook; leave the full lint, the race detector and integration tests to CI.

CI pipelines

Now let's put everything together. A good Go pipeline checks formatting, vets, lints, runs tests with the race detector and coverage, scans for vulnerabilities, runs integration tests and builds the binary.



GitHub Actions

The test job runs on two Go versions: the minimum in go.mod and the newest release. `actions/setup-go` also caches downloaded modules and build output for you.

YAML .github/workflows/ci.yml (part 1)

```yaml
name: ci
```

```yaml
on:
  push:
    branches: [main]
  pull_request:

permissions:
  contents: read

jobs:
  test:
    name: test (Go ${{ matrix.go }})
    runs-on: ubuntu-latest
    strategy:
      matrix:
        go: ["1.25.x", "stable"] # the go.mod minimum and the newest release
    steps:
      - uses: actions/checkout@v7
      - uses: actions/setup-go@v7 # also caches modules and build output
        with:
          go-version: ${{ matrix.go }}
      - name: Check formatting
        run: |
          gofmt -l . | tee unformatted.txt
          test ! -s unformatted.txt
      - name: Vet
        run: go vet ./...
```

The rest of the test job runs the tests, writes a coverage summary onto the run's summary page and keeps the coverage file:

YAML .github/workflows/ci.yml (part 2)

```yaml
      - name: Unit tests (race detector + coverage)
        run: go test -race -count=1 -coverprofile=coverage.out ./...

      - name: Coverage summary
        run: go tool cover -func=coverage.out | tail -n 1 >> "$GITHUB_STEP_SUMMARY"

      - name: Upload coverage
        if: matrix.go == 'stable'
        uses: actions/upload-artifact@v7
        with:
          name: coverage
          path: coverage.out
```

Linting, the vulnerability scan and integration tests run as separate jobs, in parallel. Ubuntu runners have GCC for `-race` and Docker for testcontainers:

YAML .github/workflows/ci.yml (part 3)

```yaml
  lint:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v7
      - uses: actions/setup-go@v7
        with:
          go-version: stable
      - uses: golangci/golangci-lint-action@v9
```

```
        with:
          version: v2.15

  integration:
    runs-on: ubuntu-latest # has Docker, so testcontainers works
    steps:
      - uses: actions/checkout@v7
      - uses: actions/setup-go@v7
        with:
          go-version: stable
      - run: go test -race -count=1 -tags integration ./...
```

The `vuln` job has the same shape, with `go run golang.org/x/vuln/cmd/govulncheck@v1.9.0 ./...` as its last step. Finally, the `build` job lists `needs: [test, lint, vuln, integration]`, so it only starts when everything else is green, then runs `make build VERSION=${{ github.sha }}` and uploads `bin/orderly-lite` as an artifact. The complete file is in the example folder.

Azure Pipelines

Many enterprises run Azure DevOps, so here's the same pipeline in `azure-pipelines.yml`. It installs an exact Go version with the `GoTool@0` task and golangci-lint with its official install script:

YAML azure-pipelines.yml (part 1)

```
trigger:
  branches:
    include: [main]
pr:
  branches:
    include: [main]

pool:
  vmImage: ubuntu-latest # has gcc (for -race) and Docker

variables:
  GO_VERSION: "1.27.3"
  GOLANGCI_LINT_VERSION: "v2.15.0"

steps:
  - task: GoTool@0
    displayName: Install Go
    inputs:
      version: $(GO_VERSION)

  - script: |
      gofmt -l . | tee unformatted.txt
      test ! -s unformatted.txt
    displayName: Check formatting

  - script: go vet ./...
    displayName: Vet
```

Azure DevOps shows test results and coverage in its own tabs, but it expects JUnit XML and Cobertura XML. **gotestsum** (v1.13.0) writes JUnit while running `go test`, and **gocover-cobertura** (v1.5.0) converts Go's coverage file:

YAML azure-pipelines.yml (part 2)

```yaml
- script: |
    curl -sSfL https://golangci-lint.run/install.sh \
      | sh -s -- -b ./.tools $(GOLANGCI_LINT_VERSION)
    ./.tools/golangci-lint run
  displayName: Lint

- script: >
    go run gotest.tools/gotestsum@v1.13.0 --junitfile test-results.xml --
    -race -count=1 -coverprofile=coverage.out ./...
  displayName: Unit tests (race detector + coverage)

- task: PublishTestResults@2
  condition: succeededOrFailed() # show results even when tests fail
  inputs:
    testResultsFormat: JUnit
    testResultsFiles: test-results.xml

- script: >
    go run github.com/boumenot/gocover-cobertura@v1.5.0
    < coverage.out > coverage.xml
  displayName: Convert coverage to Cobertura

- task: PublishCodeCoverageResults@2
  inputs:
    summaryFileLocation: coverage.xml
```

The last part scans, runs integration tests and publishes the binary:

YAML azure-pipelines.yml (part 3)

```yaml
- script: go run golang.org/x/vuln/cmd/govulncheck@v1.9.0 ./...
  displayName: Vulnerability scan

- script: go test -race -count=1 -tags integration ./...
  displayName: Integration tests (testcontainers)

- script: make build VERSION=$(Build.SourceVersion)
  displayName: Build

- task: PublishPipelineArtifact@1
  inputs:
    targetPath: bin
    artifact: orderly-lite
```

> **Key idea**
>
> CI runs the same commands you run locally. If a step only exists in YAML, developers can't reproduce failures. Keep the logic in `go` commands and Make targets, and keep the pipeline file thin.

Dependency management

Go modules (introduced in Chapter 2) are refreshingly simple next to Maven: `go.mod` lists your requirements, `go.sum` records checksums, and there's no central settings file. The questions you ask day to day are the same as in Java, though: what do I depend on, why, and is it up to date?

Question	Maven	Go
What's in my build?	`mvn dependency:list`	`go list -m all`
Why is X here?	`mvn dependency:tree -Dincludes=X`	`go mod why -m X`
Who requires X?	`mvn dependency:tree`	`go mod graph`
What can I upgrade?	`mvn versions:display-dependency-updates`	`go list -m -u all`

Here's real output from Chapter 15's example module, which pulls in testcontainers. First, why is a Docker library in our build?

TERMINAL

```
go mod why -m github.com/docker/go-units
```

OUTPUT

```
# github.com/docker/go-units
example.com/goguide/ch15/postgres
example.com/goguide/ch15/postgres.test
github.com/testcontainers/testcontainers-go
github.com/moby/moby/api/types/container
github.com/docker/go-units
```

It's only there for the Postgres package's tests, by way of testcontainers. Next, who asks for which version of pgx's connection pool library?

OUTPUT

```
$ go mod graph | grep " github.com/jackc/puddle"
example.com/goguide/ch15 github.com/jackc/puddle/v2@v2.2.3
github.com/jackc/pgx/v5@v5.12.0 github.com/jackc/puddle/v2@v2.2.3
github.com/testcontainers/testcontainers-go/modules/postgres@v0.44.0
    github.com/jackc/puddle/v2@v2.2.2
```

(The last line is wrapped to fit the page.)

Two modules ask for different versions, and Go picks v2.2.3, the **highest version anyone requires**. This rule is called minimal version selection. Unlike Maven's "nearest declaration wins", it never picks an older version than a dependency asked for, and the result is the same on every machine without a lock file. (`go.sum` stores checksums, not a version lock.)

To see what's out of date, `go list -m -u all` shows available upgrades in brackets:

OUTPUT

```
example.com/goguide/ch16
github.com/google/go-cmp v0.6.0 [v0.7.0]
golang.org/x/mod v0.39.0 [v0.42.0]
golang.org/x/tools v0.49.0 [v0.52.0]
```

Upgrading looks like this:

TERMINAL

```
go get github.com/jackc/pgx/v5@latest   # one dependency
go get -u=patch ./...                   # patch releases only, the safe option
go get -u ./...                         # everything, read the diff carefully
go mod tidy                             # drop what's unused, add what's missing
```

Automating updates and scanning for vulnerabilities

Let a bot open the upgrade pull requests, and let CI decide whether they're safe. **Dependabot** is built into GitHub:

YAML .github/dependabot.yml

```yaml
version: 2
updates:
  - package-ecosystem: gomod
    directory: /
    schedule:
      interval: weekly
    groups:
      go-modules:
        patterns: ["*"]

  - package-ecosystem: github-actions
    directory: /
    schedule:
      interval: weekly
```

Renovate works on GitHub, GitLab, Bitbucket and Azure DevOps. Its Go support is mature; add `"postUpdateOptions": ["gomodTidy"]` to `renovate.json` so it runs `go mod tidy` after each update.

govulncheck (v1.9.0) checks your code against the Go vulnerability database. Its superpower is **reachability analysis**: it reports a vulnerability only when your code can actually reach the vulnerable function, not merely when the module appears in go.mod. That means far fewer false alarms than tools like OWASP Dependency-Check.

OUTPUT

```
$ go run golang.org/x/vuln/cmd/govulncheck@v1.9.0 ./...
go: golang.org/x/vuln@v1.9.0 requires go >= 1.26.0; switching to go1.26.10
No vulnerabilities found.
```

Notice the second line. govulncheck needs Go 1.26, so the `go` command downloaded a newer toolchain to build it. Your module still targets Go 1.25.

go fix and the modernize analyzers

Go code written five years ago still compiles today, but it may use idioms that newer Go replaced: `interface{}` instead of `any`, `sort.Slice` instead of `slices.Sort`, the manual `wg.Add(1)` / `defer wg.Done()` dance instead of `wg.Go`. The **modernize** analyzers find these and rewrite them safely. Think of it as Go's OpenRewrite.

> **Newer Go**
>
> Go 1.26 completely rewrote `go fix` as the home of the modernizers, built on the same analysis framework as `go vet`. Go 1.27 added more of them. On Go 1.25, run the same analyzers with `go run golang.org/x/tools/go/analysis/passes/modernize/cmd/modernize@latest -fix ./...`, or let gopls suggest them in your editor.

Here's a real `go fix -diff` from Go 1.27.3 on our file of mistakes (trimmed to three changes):

OUTPUT

```
-   sort.Slice(skus, func(i, j int) bool { return skus[i] < skus[j] })
+   slices.Sort(skus)
...
-       wg.Add(1)
-       go func() {
-           defer wg.Done()
+       wg.Go(func() {
            reserve(id)
-       }()
+       })
...
-func Lookup(cache map[string]interface{}, key string) interface{} {
-   for i := 0; i < 3; i++ {
+func Lookup(cache map[string]any, key string) any {
+   for i := range 3 {
```

The fixers read the `go` line in go.mod and only use features your module is allowed to use. `wg.Go` appears because our module says `go 1.25.0`; a module on Go 1.24 wouldn't get it. Run `go fix ./...` after raising your Go version, review the diff and commit it on its own.

Style guides and naming

Formatting is automatic; naming and design aren't. Four documents shape how Go code "should" look:

Guide	What it is	Read it when
Effective Go	the original idioms, from the Go team	your first week
Go Code Review Comments	the common review feedback, as a checklist	before your first review

Guide	What it is	Read it when
Google Go Style Guide	thorough, with decisions and best practices	picking a team standard
Uber Go Style Guide	practical do/don't pairs for services	picking a team standard

> 💡 **If you're unsure, adopt one guide and lint for it**
>
> Read Effective Go and Code Review Comments first; they're short. Then pick Google's or Uber's guide as your team standard and write it into your review checklist. Both agree on almost everything that matters.

Naming is where Java habits show most. Here's a cheat sheet:

Concept	Java	Go
Package	`com.acme.orderly.order`	`order` (short, lower case)
Public / private	`public` / no modifier	`PlaceOrder` / `placeOrder`
Getter / setter	`getStatus()` / `setStatus()`	`Status()` / `SetStatus()`
Constant	`MAX_RETRIES`	`MaxRetries` or `maxRetries`
Acronyms	`userId`, `HttpClient`	`userID`, `HTTPClient`
Interface	`OrderRepository`, `IOrderRepo`	`OrderRepository`, `Reader`
Constructor	`new OrderService(...)`	`NewService(...)`, `order.New(...)`
Receiver	`this`	short: `(s *Service)`, never `self`
Exception / error	`OrderNotFoundException`	`ErrNotFound`, `NotFoundError`
Type in package `order`	`OrderService`	`order.Service` (no stutter)
File	`OrderService.java`	`service.go`, `order_service.go`
Local variable	`customerIdentifier`	`c` or `cust` in a short scope

The "no stutter" row deserves a word. Callers always see the package name, so `order.OrderItem` says "order" twice. That's why the examples in Chapters 15 to 17 call it `order.Item`. Short local names are fine when the scope is a few lines; the further a name travels, the more descriptive it should be.

Recap

- `gofmt` (or goimports) ends formatting debates: format on save, check `gofmt -l .` in CI.
- `go vet` and staticcheck find real bugs; golangci-lint v2 runs them all with one `version: "2"` config.
- gopls powers VS Code and most editors; Delve (`dlv`) debugs programs and tests, with conditional breakpoints.
- `go generate` is explicit and its output is committed; `go build` never runs it.
- `tool` directives pin small tools in go.mod, but they share your module's Go version; install golangci-lint as a binary and `go run` occasional tools.

- A short Makefile (or Taskfile) gives the team shared verbs; CI calls the same commands.
- Use `go mod why`, `go list -m -u all` and Dependabot or Renovate for dependencies, and govulncheck for vulnerabilities.
- `go fix` (Go 1.26+) and the modernize analyzers upgrade old idioms for you.

Try it yourself

🏋 Exercise 1: Break it, then lint it

In `examples/ch16-tooling/internal/order/discount.go`, add a function that calls `strings.ToUpper(a) == strings.ToUpper(b)` and ignores an error from `os.Remove`. Run `make lint` and read which linter reports each problem. Fix them, then add a `//nolint` with a reason to one and watch it disappear.

Hint: you need golangci-lint v2 on your `PATH`.

🏋 Exercise 2: Conditional breakpoints

Debug `TestDiscountCents` with Delve. Set a breakpoint inside `LineDiscountCents`, make it conditional on `it.ProductID == "tea"`, and print `gross` when it stops.

Hint: use `dlv test ./internal/order -- -test.run TestDiscountCents`, then `break`, `condition`, `continue`, `next` and `print`.

🏋 Exercise 3: A new event and a fresh-code check

Add `EventOrderShipped` to `EventType`, run `make generate`, and confirm the `String()` output with a new case in `TestEventTypeString`. Then add a `check-generate` target to the Makefile that fails if generated files are out of date.

Hint: `go generate ./... && git diff --exit-code`.

🏋 Exercise 4 (stretch): Your team's pipeline

Adapt `ci.yml` or `azure-pipelines.yml` for one of your own services. Add a coverage gate that fails the build below 70%, and a nightly scheduled run that executes one fuzz target for ten minutes.

Hint: parse the `total:` line from `go tool cover -func`; GitHub uses `on: schedule`, Azure uses a `schedules:` block.

Further reading

- golangci-lint configuration: every v2 option, with the linter list.
- Staticcheck documentation: what each check means and why.

- Delve documentation: command reference and editor integration.
- Go Modules Reference: the authoritative description of go.mod, `tool` directives and version selection.
- govulncheck tutorial: scanning a project step by step.
- gopls documentation: features, settings and editor setup.
- Google Go Style Guide: the most thorough guide to idiomatic Go.

CHAPTER 17

Performance and How Go Works Inside

> *Go is fast by default; this chapter shows you why, how to measure it, and the handful of habits that keep it fast under real load.*

🎯 **What you'll learn**

- Decide when performance work is worth it, and measure with benchmarks, pprof and benchstat
- Read escape analysis output and understand stack versus heap
- Explain Go's garbage collector and tune it with `GOGC` and `GOMEMLIMIT`
- Cut allocations with preallocation, `strings.Builder` and `sync.Pool`, backed by real numbers
- Describe the G-M-P scheduler and how `GOMAXPROCS` behaves in containers
- Use PGO, `bufio`, HTTP connection reuse and JSON wisely, and avoid the common mistakes

The big picture

Most enterprise services spend their time waiting: on Postgres, on Kafka, on another team's API. For those services, Go's defaults are already good, and the biggest wins come from fewer round trips, not faster code. But when you do hit a hot path (a pricing engine, a feed parser, a busy JSON API), it helps a lot to know how Go works underneath.

The good news for a Java developer is that Go has far fewer moving parts than the JVM. There's no JIT, no class loading and one garbage collector with two knobs. What you build is what runs, which makes performance easier to reason about.

🧩 **Think of it like a mechanic with a diagnostic computer**

A good mechanic doesn't rebuild the engine because it "feels slow". They plug in the diagnostic computer, read the numbers, and fix the one part that's actually worn. Benchmarks and pprof are your diagnostic computer. Everything in this chapter starts there.

Java ↔ Go at a glance

Concept	Java / JVM	Go
Microbenchmarks	JMH	`testing.B` + benchstat
Profiling	JFR, async-profiler	pprof (CPU, heap, goroutines, locks)
Stack or heap?	JIT decides at run time	compiler decides at build time

Concept	Java / JVM	Go
Garbage collector	G1, ZGC, many flags	one concurrent GC: `GOGC`, `GOMEMLIMIT`
Heap cap	`-Xmx`, `MaxRAMPercentage`	`GOMEMLIMIT`
Concurrency	platform and virtual threads	goroutines on `GOMAXPROCS` threads
Container CPUs	container support (JDK 10+)	container-aware `GOMAXPROCS` (1.25)
Startup	JVM boot, JIT warm-up	native binary, no warm-up
Optimising from profiles	JIT, at run time	PGO, at build time

Full code: `examples/ch17-performance/`. Every number below was measured on a 20-thread Intel i7-12800H laptop running Windows 11 and Go 1.25. Your numbers will differ; the ratios are what matter.

When to care, and how to measure

Here's a simple rule: **don't optimise what you haven't measured**. Look at your latency and CPU dashboards first (Chapter 13). If a service meets its targets, spend the time elsewhere. If it doesn't, find out exactly where the time goes before changing anything.

You have three measuring tools, all covered briefly in Chapter 15:

- **Benchmarks** (`testing.B` with `b.Loop()`) time one function in isolation, like JMH.
- **pprof** profiles a whole program and tells you where CPU time and memory go, like JFR.
- **benchstat** compares benchmark runs and tells you whether a difference is real or noise.

Reading a CPU profile

The example's `pgo` program prices a feed of 10,000 order lines, again and again. Here's how to profile it and see the top functions:

TERMINAL

```
go run ./pgo -rounds=20000 -cpuprofile=cpu.pprof
go tool pprof -top -nodecount=10 cpu.pprof
```

OUTPUT

```
Duration: 10.85s, Total samples = 10400ms (95.89%)
      flat  flat%   sum%        cum   cum%
    2280ms 21.92% 21.92%     3850ms 37.02%  main.lineTotal
    1400ms 13.46% 35.38%     1400ms 13.46%  indexbytebody
    1160ms 11.15% 46.54%     1160ms 11.15%  strconv.ParseUint
     940ms  9.04% 55.58%     2100ms 20.19%  strconv.ParseInt
     710ms  6.83% 62.40%     2850ms 27.40%  internal/stringslite.Cut
     590ms  5.67% 68.08%      590ms  5.67%  main.volumeDiscount.Apply
     520ms  5.00% 73.08%      520ms  5.00%  main.salesTax.Apply
     470ms  4.52% 77.60%      470ms  4.52%  strconv.Atoi
     430ms  4.13% 81.73%     6130ms 58.94%  main.parseLine
```

Two columns matter most:

- **flat** is time spent in the function's own code.
- **cum** (cumulative) includes everything it calls.

So `parseLine` does little itself (4%) but, with the `strconv` and `strings` calls under it, accounts for 59% of the run. If we wanted this program faster, parsing is where we'd look. `go tool pprof -http=:8080 cpu.pprof` opens the same data in a browser, with a flame graph.

For a running service, you don't need a flag: import `net/http/pprof` on an internal port (Chapter 13 shows how) and fetch `/debug/pprof/profile?seconds=30` from production.

Stack, heap and escape analysis

Every value lives in one of two places:

- The **stack** is a small, private scratch area for each goroutine. Space is claimed when a function starts and released, for free, when it returns.
- The **heap** is shared memory for values that must outlive the function that created them. The garbage collector has to find and free them later, which costs CPU.

Escape analysis is the compiler deciding, at build time, whether a variable can live on the fast stack or has to move to the heap.

> 🧩 **Think of it like a notepad versus a filing cabinet**
>
> The stack is a notepad on your desk: scribble on it, tear off the page when you're done, no paperwork. The heap is the office filing cabinet: anyone can use what's in it, but someone has to come round and clear out old files. The fewer files you leave in the cabinet, the less cleaning there is.

> ☕ **Coming from Java**
>
> In Java, every `new` object goes to the heap, and the JIT may later optimise some allocations away at run time. In Go, the compiler decides up front, and `&Order{}` can land on the stack if it never escapes. Better still, you can ask the compiler what it decided.

Here are some small functions from the example:

GO escape/escape.go

```go
func NewOrderValue(id string) Order {
    return Order{ID: id}
}

func NewOrderPointer(id string) *Order {
    return &Order{ID: id}
}

func SumFixed() int64 {
    prices := make([]int64, 8)
    // ...fill and sum prices
}
```

```go
func Describe(total int64) string {
    return fmt.Sprintf("total=%d", total)
}

var registry []*Order

func Register(o *Order) {
    registry = append(registry, o)
}
```

Ask the compiler with `-gcflags=-m` (trimmed to the interesting lines):

TERMINAL

```
go build -gcflags=-m ./escape/
```

OUTPUT

```
escape/escape.go:14:6: can inline NewOrderValue
escape/escape.go:20:9: &Order{...} escapes to heap
escape/escape.go:25:16: make([]int64, 8) does not escape
escape/escape.go:38:33: total escapes to heap
escape/escape.go:45:19: append escapes to heap
```

Reading it line by line:

- `NewOrderValue` returns a copy, so nothing escapes. The `Order` lives in the caller's stack frame.
- `NewOrderPointer` returns a pointer to its `Order`, which must outlive the function, so it **escapes to the heap**.
- `SumFixed`'s slice has a fixed size and never leaves the function, so it stays on the stack.
- `Describe` passes `total` to `fmt.Sprintf` as an `any`, which needs a heap copy. Formatting functions are a classic source of hidden allocations.
- `Register` appends to a package-level slice, which lives forever, so it escapes too.

> 💡 **If you're unsure, return values, not pointers**
>
> For small structs (a few fields), returning a value is often faster than returning a pointer, because it avoids the heap entirely. Use pointers when the value is large, when the caller must see changes, or when it has a mutex inside. Don't twist your design to dodge every allocation; use `-gcflags=-m` on the hot path that a profile pointed to.

The garbage collector

Go's **garbage collector** (GC) finds heap memory that nothing points to any more and makes it reusable. It's a very different design from the JVM's collectors, and four words describe it:

- **Concurrent:** it does most of its work while your goroutines keep running, with two very short stop-the-world pauses per cycle.

- **Tri-color mark-sweep:** it marks everything reachable, then sweeps away the rest (more on the colours below).
- **Non-generational:** it doesn't split the heap into young and old areas, as G1 does.
- **Non-moving:** it never moves objects, so pointers stay valid and there's no compaction step.

Tri-color marking sorts objects into three groups while the program runs:

```
start: everything is WHITE (not seen yet)

roots (stacks, globals) --> GREY (seen, children not checked)
                               |
                               v  scan its pointers
                            BLACK (seen, children checked)
                               |
                               +--> children become GREY

when no GREY objects remain:
  BLACK = reachable, keep
  WHITE = unreachable, sweep (free)
```

A **write barrier**, a small piece of code the compiler adds to pointer writes during marking, makes sure a goroutine can't hide a white object from the collector while it works.

GOGC: how much garbage before the next cycle

The GC's main knob is `GOGC`, which defaults to 100. It means: **let the heap grow by 100% of the live data before collecting again**. With 64 MB of live data, the next GC starts at about 128 MB.

> 🧩 **Think of it like emptying the office bin**
>
> `GOGC` is how full the bin gets before someone empties it. A low value means emptying it often: less mess, but more trips. A high value means fewer trips but more bin space. The rubbish produced is the same either way.

The example's `gcdemo` program keeps about 64 MB of live "cached orders" and then handles 200,000 "requests" that each create 8 KB of garbage. Here's the real trade-off on this laptop:

Setting	GC cycles	GC CPU time	Heap goal
`GOGC=50`	67	about 900 ms	79 MB
`GOGC=100` (default)	32	about 440 ms	125 MB
`GOGC=400`	9	about 133 ms	294 MB
`GOGC=off GOMEMLIMIT=200MiB`	12	about 200 ms	184 MB
`GOGC=100 GOMEMLIMIT=100MiB`	48	about 700 ms	90 MB

Doubling `GOGC` roughly halves GC CPU and costs more memory. It's that simple a trade, and there are no other heap-region settings to learn. Pauses stayed tiny throughout: in an earlier run, 67 cycles added up to about 4 ms of pause time in total.

GOMEMLIMIT: a soft ceiling

`GOGC` alone doesn't know about your container's memory limit. `GOMEMLIMIT` (Go 1.19) does: it's a **soft memory limit** for the whole Go runtime. As memory use approaches it, the GC runs more often, whatever `GOGC` says. The last row of the table shows it at work: the same `GOGC=100`, but the 100 MiB limit forced 48 cycles to stay under it.

> 💡 **If you're unsure, set GOMEMLIMIT and leave GOGC alone**
>
> In containers, set `GOMEMLIMIT` to about 90% of the container's memory limit, for example `GOMEMLIMIT=900MiB` for a 1 GiB pod, and keep `GOGC=100`. Only change `GOGC` after a profile shows GC is a real cost. For batch jobs that should use all their memory, `GOGC=off` plus `GOMEMLIMIT` is a neat pattern: no GC at all until memory gets close to the limit.

> ⚠️ **Gotcha: a limit that's too low**
>
> If the live data alone is close to `GOMEMLIMIT`, the GC runs constantly. The runtime caps GC at about half the CPU to avoid a total freeze, but your service will still slow to a crawl. The limit is a safety net, not a way to squeeze a service into less memory than it needs.

Compare that with a JVM service, where you choose a collector (G1, ZGC, Parallel), size the heap with `-Xmx` or `-XX:MaxRAMPercentage`, and possibly tune region sizes and pause targets. Go deliberately offers two knobs and asks you to fix allocation-heavy code instead.

> 📝 **Newer Go: the Green Tea collector**
>
> Go 1.25 shipped an experimental collector called Green Tea (`GOEXPERIMENT=greenteagc`), which scans small objects with better memory locality. The Go team measured 10 to 40% less GC overhead in GC-heavy programs. In Go 1.26 it became the default; Go 1.27.3 still accepts `GOEXPERIMENT=nogreenteagc` to opt out, though the Go 1.26 notes said that opt-out would eventually be removed. On our toy `gcdemo` it made no measurable difference, which is a good reminder to measure your own service.

Reducing allocations

Every heap allocation costs a little time now and creates GC work later. In a hot path, cutting allocations is usually the single most effective optimisation. Here are the main techniques, all measured with `go test -bench=. -benchmem -count=6` and summarised with benchstat (medians):

Benchmark	Before	After	Allocs
1,000 totals: grow vs preallocate	6.7 µs	2.9 µs	12 → 1
1,000-entry map: grow vs size hint	138 µs	51 µs	22 → 6
Join 200 IDs: `+=` vs `strings.Builder`	409 µs	4.5 µs	398 → 1
Record 1,000 values: `any` vs `int64`	18.6 µs	1.7 µs	999 → 0
Encode an order: new buffer vs `sync.Pool`	5.0 µs	4.2 µs	3 → 1

Preallocate slices and maps

When you know how many elements you'll add, say so up front. Otherwise `append` repeatedly allocates a bigger array and copies everything across.

GO alloc/alloc.go

```go
// TotalsPrealloc knows the final size, so it allocates exactly once.
func TotalsPrealloc(lines []Line) []int64 {
    totals := make([]int64, 0, len(lines))
    for _, l := range lines {
        totals = append(totals, int64(l.Quantity)*l.UnitPriceCents)
    }
    return totals
}
```

Maps work the same way: `make(map[string]Line, len(lines))` reserves room for every entry, so the map never has to grow and rehash.

Build strings with `strings.Builder`

Go strings can't be changed, so `s += id` copies the whole string every time. In a loop that's quadratic: joining 200 UUIDs allocated 1.5 MB. A memory profile shows exactly where:

OUTPUT

```
ROUTINE ======================== .../alloc.JoinConcat
    4.26GB     4.26GB (flat, cum) 54.77% of Total
         .          .     52:   for i, id := range ids {
         .          .     53:       if i > 0 {
    2.13GB     2.13GB     54:           s += ","
         .          .     55:       }
    2.13GB     2.13GB     56:       s += id
```

That's `go tool pprof -sample_index=alloc_space -list=JoinConcat` on a profile from `go test -bench=Join -memprofile=mem.out`. The fix uses one growing buffer, and `Grow` sizes it up front:

GO alloc/alloc.go

```go
func JoinBuilder(ids []string) string {
    var b strings.Builder
    b.Grow(len(ids) * 37) // a UUID is 36 bytes, plus a comma
    for i, id := range ids {
        if i > 0 {
            b.WriteByte(',')
        }
        b.WriteString(id)
    }
    return b.String()
}
```

That's 90 times faster and one allocation instead of 398. (For this exact job `strings.Join` is simpler still; the Builder pattern is for when you're building something more complex.)

Avoid interface boxing in hot paths

Storing a non-pointer value such as an `int64` in an interface (`any`) usually needs a small heap allocation to hold the value. This is called **boxing**, and Java developers know it from `Integer` and `Long`.

GO alloc/boxing.go

```go
type AnyRecorder struct{ values []any }

func (r *AnyRecorder) Record(v any) { r.values = append(r.values, v) }

type IntRecorder struct{ values []int64 }

func (r *IntRecorder) Record(v int64) { r.values = append(r.values, v) }
```

Recording 1,000 values made 999 allocations with `any` and none with `int64`. Why 999 and not 1,000? The runtime keeps ready-made boxes for the numbers 0 to 255, and the first value was 0. Generics (Chapter 4) are often the clean way to keep concrete types in reusable code.

`sync.Pool`, with care

`sync.Pool` keeps a set of reusable objects, so you don't allocate a fresh one for every request. Buffers are the classic case:

GO alloc/pool.go

```go
var bufPool = sync.Pool{
    New: func() any { return new(bytes.Buffer) },
}

func EncodePooled(w io.Writer, o Order) error {
    buf := bufPool.Get().(*bytes.Buffer)
    buf.Reset() // a pooled buffer still holds the last caller's bytes
    defer func() {
        if buf.Cap() <= 64<<10 { // don't keep huge buffers alive
            bufPool.Put(buf)
        }
    }()
    if err := json.NewEncoder(buf).Encode(o); err != nil {
        return err
    }
    _, err := w.Write(buf.Bytes())
    return err
}
```

Bytes allocated per call dropped from 2.1 KB to 80 bytes. Time improved only modestly (about 15%, with a lot of noise), because JSON encoding itself dominates. Keep these caveats in mind:

- **Always reset** what you take out. Leftover data from another request is a security bug waiting to happen.
- **Don't pool huge objects.** One giant request would otherwise pin megabytes forever, hence the size check.
- **The pool may be emptied at any GC.** It's a recycling bin, not a cache and not a connection pool.
- **Measure first.** A pool adds complexity; use it only where a memory profile shows the allocation matters.

Struct layout and alignment, briefly

The CPU reads memory most efficiently when an 8-byte field starts at an address divisible by 8. So the compiler inserts invisible **padding** between fields to line them up. Field order changes how much padding you get:

GO layout/layout.go

```go
type LinePadded struct {
    Gift       bool  // 1 byte, then 7 bytes of padding
    PriceCents int64 // 8 bytes
    Backorder  bool  // 1 byte, then 3 bytes of padding
    Quantity   int32 // 4 bytes
    Express    bool  // 1 byte, then 7 bytes of padding
}

type LinePacked struct {
    PriceCents int64 // 8 bytes
    Quantity   int32 // 4 bytes
    Gift       bool  // 1 byte
    Backorder  bool  // 1 byte
    Express    bool  // 1 byte, then 1 byte of padding
}
```

OUTPUT

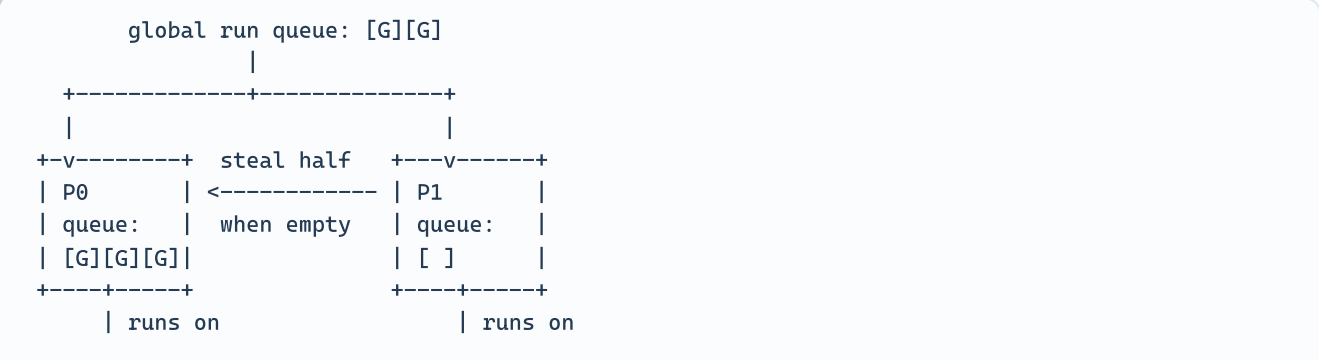
```
layout_test.go:11: LinePadded: 32 bytes, LinePacked: 16 bytes
```

Same fields, half the memory. Sorting fields from largest to smallest is a good habit, and the `fieldalignment` analyzer (available through golangci-lint's `govet` settings) can find candidates. Only bother for structs you hold millions of; for everything else, order fields so they read well.

The scheduler: goroutines, threads and G-M-P

Chapter 5 showed that goroutines are cheap. Here's why. The Go runtime runs its own scheduler that multiplexes many goroutines onto a few OS threads, using three kinds of objects:

- **G** is a goroutine: its stack, its instruction pointer and a little bookkeeping.
- **M** is a machine: an OS thread that actually executes code.
- **P** is a processor: a permission slip to run Go code, with its own queue of runnable goroutines. There are exactly `GOMAXPROCS` of them.



```
+----v-----+              +----v-----+        +-----------+
| M0 (OS   |              | M1 (OS   |        | M2 blocked|
| thread)  |              | thread)  |        | in syscall|
+----------+              +----------+        +-----------+

netpoller: parks goroutines waiting on sockets, wakes them when
data arrives (epoll on Linux, kqueue on macOS, IOCP on Windows)
```

A few behaviours make this work well:

- **Work stealing:** a P with an empty queue steals half the goroutines from a busy P, so all cores stay busy.
- **Async preemption:** since Go 1.14, the runtime can interrupt a goroutine stuck in a tight loop, so one busy goroutine can't starve the others.
- **Blocking system calls:** when a goroutine blocks in the OS (reading a file, say), its M blocks with it, and the P is handed to another thread so other goroutines keep running.
- **The netpoller:** network I/O doesn't block threads at all. A goroutine waiting on a socket is parked, and the netpoller wakes it when data arrives. That's why one Go service can hold tens of thousands of connections on a handful of threads.

> 🧩 **Think of it like a restaurant kitchen**
>
> The Ps are cooking stations, the Ms are chefs and the Gs are order tickets. Each station has its own pile of tickets. An idle chef takes tickets from a busy station. If a chef has to wait for a delivery (a system call), the station is handed to another chef, so cooking never stops.

> ☕ **Coming from Java**
>
> Java's platform threads map one-to-one onto OS threads, so you pool them. Java 21's **virtual threads** use the same M:N idea as goroutines: many lightweight threads mounted on a few carrier threads. If you've used virtual threads, goroutines will feel familiar. The differences: Go has had this since day one, so every library is written for it, and there's no equivalent of the "pinning" problem that virtual threads had inside `synchronized` blocks before Java 24.

How cheap is a parked goroutine? The `sched` program starts 100,000 goroutines that all wait on one channel:

OUTPUT

```
NumCPU: 20, GOMAXPROCS: 20
100000 parked goroutines use 781.4 MB of stack (8.0 KB each)
```

On Windows, each goroutine starts with an 8 KB stack, because the runtime reserves 4 KB of extra room for Windows system calls. On Linux, where your services run, the same goroutine starts at 2 KB. Stacks grow (and shrink) automatically as needed, unlike the fixed stacks of OS threads.

GOMAXPROCS in containers

`GOMAXPROCS` sets how many Ps exist, which means how many goroutines can run Go code at the same moment. It defaults to the number of CPUs the process can use.

Before Go 1.25, "CPUs" meant the CPUs of the whole host. A pod with a 2-CPU limit on a 64-core node got `GOMAXPROCS=64`, and the Linux scheduler throttled it hard. Teams worked around this with the `uber-go/automaxprocs` library.

Since **Go 1.25**, the runtime is container-aware:

- On Linux, it reads the cgroup **CPU limit** and lowers `GOMAXPROCS` to match.
- It re-checks periodically, so it follows changes to the limit (and to the CPUs available) while running.
- It ignores CPU **requests**; only limits count.
- Setting `GOMAXPROCS` yourself turns this off, and the `GODEBUG` settings `containermaxprocs=0` and `updatemaxprocs=0` disable each behaviour separately.

OUTPUT

```
$ GOMAXPROCS=4 go run ./sched
NumCPU: 20, GOMAXPROCS: 4
```

> 💡 **If you're unsure, let Go 1.25 decide**
>
> On Go 1.25 or later, remove `automaxprocs` and don't set `GOMAXPROCS`. If your pods have CPU requests but no CPU limits (a common Kubernetes setup), Go will use every core on the node; if that's too many, set `GOMAXPROCS` to roughly your request.

The JVM went through the same journey: container support arrived in JDK 10 (and 8u191), and `-XX:ActiveProcessorCount` overrides what the JVM detects.

Startup time and memory footprint

A Go service compiles to one native executable. There's no VM to boot, no classes to load and no JIT to warm up, so it serves requests at full speed from the first one. The example's `startup` program starts an HTTP server, calls its own `/healthz` endpoint and exits:

OUTPUT

```
healthz said "ok"; Go runtime memory: 6.3 MB
```

Measured on this laptop:

- **Binary size:** 8.4 MB, or 6.0 MB built with `-ldflags="-s -w"` (which drops debug symbols).
- **Memory:** about 6 MB mapped by the Go runtime.
- **Time:** about 55 ms from launch to exit with PowerShell's `Measure-Command`. A hello-world `.exe` takes about 45 ms on the same machine, so most of that is Windows starting a process, not Go.

How does that compare with Java? Qualitatively:

	Spring Boot on the JVM	GraalVM native image	Go
Startup	seconds, then JIT warm-up	very fast	very fast
Memory at rest	large (heap + metaspace)	small	small
Peak throughput	excellent after warm-up	good	good, from the start
Build	fast	slow, needs reflection config	fast

GraalVM native images bring Java close to Go's startup and memory, at the cost of slow builds and closed-world restrictions on reflection. Go gives you that profile by default, which is a big reason it's popular for CLIs, sidecars and fast-scaling services.

Profile-guided optimisation

Profile-guided optimisation (PGO) feeds a CPU profile from a real run back into the compiler, so it can optimise the code paths that are actually hot. The JVM's JIT does something similar while the program runs; Go does it at build time.

The workflow is short:

1. Collect a CPU profile from production (for example `/debug/pprof/profile?seconds=30`).
2. Save it as `default.pgo` in your main package's folder and commit it.
3. Build as usual. Since Go 1.21, `go build` picks up `default.pgo` automatically (`-pgo=auto`).

PGO mostly makes the compiler **inline** hot functions more aggressively and **devirtualise** hot interface calls: it adds a fast path for the concrete type the profile saw most, which can then be inlined. You can see it happen by asking for the compiler's decisions on the `pgo` package:

TERMINAL

```
go build -pgo=pgo/default.pgo   -gcflags='example.com/goguide/ch17/pgo=-m=1' ./pgo
```

OUTPUT

```
pgo/feed.go:49:18: PGO devirtualizing interface call r.Apply to volumeDiscount.Apply
pgo/feed.go:49:18: inlining call to volumeDiscount.Apply
pgo/feed.go:40:21: inlining call to lineTotal
```

So did it get faster? Here's the honest result, with the two builds' benchmarks interleaved twelve times so background load affects both equally:

OUTPUT

```
              |  nopgo.txt   |              pgo.txt               |
              |    sec/op    |   sec/op     vs base               |
PriceFeed-20    571.7µ ± 11%   608.8µ ± 9%       ~ (p=0.101 n=12)
LineTotals-20   243.0µ ±  9%   242.6µ ± 5%       ~ (p=0.932 n=12)
```

The `~` means no measurable difference. The profile explains why: most of the time is in `strconv` and `strings` parsing, which PGO can't improve much, and the devirtualised call was already cheap. The Go team reports PGO improvements of around 2 to 14% on a representative set of real programs. That's worth having, since it costs nothing, but it's not a substitute for fixing an algorithm.

> 💡 **If you're unsure, turn PGO on and forget it**
>
> Commit a production profile as `default.pgo`, refresh it every few releases, and keep building normally. Don't use a microbenchmark profile; the Go docs warn that unrepresentative profiles are poor input.

Inlining and bounds-check elimination, briefly

Two compiler optimisations come up in performance discussions:

- **Inlining** replaces a function call with the function's body, saving the call and opening up further optimisations. Go inlines small functions (below a cost budget of 80 "nodes"), and you saw `can inline` lines in the `-m` output earlier. PGO raises the budget for hot calls.
- **Bounds-check elimination** (BCE) removes the check that guards every slice index (`p[i]` panics if `i` is out of range) when the compiler can prove it's safe.

GO escape/bce.go

```go
func Sum4(p []int64) int64 {
    return p[0] + p[1] + p[2] + p[3]
}

func Sum4Hint(p []int64) int64 {
    _ = p[3] // one bounds check up front
    return p[0] + p[1] + p[2] + p[3]
}
```

OUTPUT

```
$ go build -gcflags=-d=ssa/check_bce/debug=1 ./escape/
escape/bce.go:5:10: Found IsInBounds
escape/bce.go:5:17: Found IsInBounds
escape/bce.go:5:24: Found IsInBounds
escape/bce.go:5:31: Found IsInBounds
escape/bce.go:10:7: Found IsInBounds
```

`Sum4` has four checks; `Sum4Hint` has one, because once `p[3]` is known to exist, so are `p[0]` to `p[2]`. You'll see this trick in the standard library's encoding code. In application code, leave it to the compiler unless a profile points straight at such a loop.

I/O, HTTP and JSON

These three show up in almost every enterprise service, and each has one habit that matters.

Buffer your I/O with bufio

Writing to an `*os.File` is a system call each time. Writing 1,000 small CSV lines straight to a file made 1,000 system calls. Wrapping the file in a `bufio.Writer` collects them in a 4 KB buffer:

GO iobuf/write.go

```go
func WriteOrdersBuffered(w io.Writer, n int) error {
    bw := bufio.NewWriter(w)
    if err := WriteOrders(bw, n); err != nil {
        return err
    }
    return bw.Flush() // forgetting Flush loses the last chunk
}
```

OUTPUT

```
Write1000/unbuffered-20   4.750m ±  7%
Write1000/bufio-20        182.9µ ± 10%
```

That's 4.75 ms down to 0.18 ms, about 26 times faster, from two lines of code. `bufio.Scanner` and `bufio.Reader` do the same for reading. HTTP response writers and network connections are already buffered by `net/http`.

Reuse HTTP connections

Opening a TCP connection (plus a TLS handshake) costs far more than sending a request on an existing one. Go's `http.Client` keeps idle connections for reuse, but two details trip up nearly everyone:

- You must **read the body to the end and close it**, or the connection can't go back to the pool.
- `MaxIdleConnsPerHost` defaults to **2**. With 20 goroutines calling one service, most connections are thrown away after each request.

GO httpclient/client.go

```go
func New(maxConns int) *http.Client {
    t := http.DefaultTransport.(*http.Transport).Clone() // keep sane defaults
    t.MaxIdleConns = 100                                 // across all hosts
    t.MaxIdleConnsPerHost = maxConns                     // the default is only 2
    t.IdleConnTimeout = 90 * time.Second
    return &http.Client{
        Transport: t,
        Timeout:   5 * time.Second, // never wait forever
    }
}
```

The example's test sends 1,000 requests from 20 goroutines and counts the TCP connections the server sees:

OUTPUT

```
=== RUN   TestConnectionReuse/default_transport
    client_test.go:60: 1000 requests opened 341 connections
=== RUN   TestConnectionReuse/tuned_transport
    client_test.go:60: 1000 requests opened 30 connections
```

Create one client per downstream service at startup and share it; it's safe for concurrent use. Creating a new `http.Client` per request throws the pool away every time.

JSON

`encoding/json` uses reflection, and on a busy API it often shows up in profiles. Before reaching for another library, know that the standard library itself is getting faster.

Go 1.25 includes a rewritten implementation behind `GOEXPERIMENT=jsonv2`. When you set it, the familiar `encoding/json` API runs on the new engine. Here's our order benchmark, run ten times each way on Go 1.25:

OUTPUT

```
              |     v1.txt     |                v2.txt                |
              |     sec/op     |    sec/op     vs base                |
Marshal-20      2.430µ ± 159%   3.490µ ± 14%        ~ (p=0.481 n=10)
Unmarshal-20   13.879µ ±  32%   8.221µ ±  8%  -40.76% (p=0.000 n=10)
```

Marshalling is at parity (the ±159% is laptop noise), and unmarshalling is about 40% faster with 38% fewer allocations, which matches what the Go team reported.

> **Newer Go**
>
> In Go 1.27, `encoding/json/v2` and `encoding/json/jsontext` are generally available, and the classic `encoding/json` package runs on the v2 engine by default (`GOEXPERIMENT=nojsonv2` opts out). Upgrading Go may be the cheapest JSON speed-up you'll ever get.

If JSON still dominates after that, the main alternatives are **goccy/go-json** (a drop-in replacement), **bytedance/sonic** (very fast on amd64 and arm64, using JIT and SIMD) and **easyjson** (generates code instead of using reflection). All three are maintained. Treat them as a measured last step, because each brings compatibility edge cases.

Common performance mistakes

These are the ones that show up again and again in Go code reviews, especially from teams new to the language:

- **Optimising without a profile.** Guessing is usually wrong. Measure with pprof first.
- **Creating an `http.Client` per request**, or not draining and closing response bodies.
- **Leaving `MaxIdleConnsPerHost` at 2** for a busy downstream service.
- **Building strings with `+=` in loops** instead of `strings.Builder` or `strings.Join`.
- **Growing slices and maps one element at a time** when the size is known.
- **`fmt.Sprintf` in hot paths** for simple conversions; `strconv.Itoa` and friends don't allocate an interface.
- **Unbuffered file or socket writes** in a loop.
- **Unbounded goroutines**, one per item in a million-item batch. Use a worker pool or `errgroup.SetLimit` (Chapter 5).

- **Holding a mutex while doing I/O**, so every other goroutine waits for the network.
- **Passing huge structs by value** in hot loops, or making everything a pointer "for speed" and pushing it all onto the heap.
- **Running without `GOMEMLIMIT` in containers**, then getting killed when the heap spikes.
- **Benchmarking on a busy laptop and trusting one run.** Use `-count=10` and benchstat, as we did throughout this chapter.

Recap

- Measure before optimising: benchmarks for functions, pprof for programs, benchstat to separate signal from noise.
- Escape analysis decides stack or heap at build time; `go build -gcflags=-m` shows its decisions.
- Go's GC is concurrent, tri-color, non-generational and non-moving. `GOGC` trades CPU for memory; `GOMEMLIMIT` is a soft ceiling, about 90% of the container limit.
- The cheapest allocation is the one you don't make: preallocate, use `strings.Builder`, avoid boxing in hot paths, and use `sync.Pool` carefully.
- The G-M-P scheduler runs many goroutines on `GOMAXPROCS` threads, with work stealing, preemption and a netpoller; Go 1.25 sets `GOMAXPROCS` from the container's CPU limit.
- Go starts fast with a small footprint; PGO via `default.pgo` adds a few free percent.
- Buffer I/O with `bufio`, share and tune one `http.Client` per downstream, and upgrade Go for faster JSON.

Try it yourself

> 🏋 **Exercise 1: Find the escapes**
>
> Run `go build -gcflags=-m ./escape/` in `examples/ch17-performance`. Change `Describe` to use `strconv.FormatInt` and string concatenation instead of `fmt.Sprintf`, and check whether `total escapes to heap` disappears.
>
> *Hint:* `"total=" + strconv.FormatInt(total, 10)`.

> 🏋 **Exercise 2: Tune the GC for a pod**
>
> Imagine `gcdemo` runs in a pod with a 256 MiB memory limit. Pick a `GOMEMLIMIT`, run it with `GOGC=100` and with `GOGC=off`, and compare GC cycles and GC CPU. Which would you ship, and why?

🏋️ Exercise 3: Prove a pool helps

Write a benchmark that formats an Orderly order as a CSV line into a `bytes.Buffer`, first with a fresh buffer per call and then with a `sync.Pool`. Compare them with `-count=10` and benchstat. Is the difference significant on your machine?

Hint: if benchstat prints `~`, the pool isn't worth its complexity yet.

🏋️ Exercise 4 (stretch): Profile and fix the parser

The CPU profile showed `parseLine` at 59% of the pricing run. Write a faster version that walks the string once instead of calling `strings.Cut` twice and `strconv` twice. Benchmark both, check the improvement with benchstat, and regenerate `default.pgo`.

Further reading

- A Guide to the Go Garbage Collector: the official, very readable explanation of `GOGC`, `GOMEMLIMIT` and GC costs.
- Profile-guided optimization: how PGO works, how to collect profiles and what to expect.
- Diagnostics: profiling, tracing and debugging tools in one place.
- Profiling Go Programs: the classic Go blog walkthrough of pprof.
- Container-aware GOMAXPROCS: the Go blog on the Go 1.25 change.
- runtime/pprof and net/http/pprof: the APIs for collecting profiles.
- Go 1.25 release notes: Green Tea, JSON v2 and the other changes mentioned here.

CHAPTER 18

Build, Containers and Deployment

> *Your code only helps people once it's running in production, and Go makes that last mile refreshingly short.*

🎯 **What you'll learn**

- Build small, static, reproducible binaries with `-trimpath` and `-ldflags`, and stamp a version into them
- Cross-compile for Linux, macOS, Windows and ARM from one laptop
- Write a multi-stage Dockerfile that ends in a tiny, non-root distroless image
- Run Go well on Kubernetes: probes, `GOMAXPROCS`, `GOMEMLIMIT`, graceful termination, Helm and GitOps
- Build a CI/CD pipeline in GitHub Actions and Azure Pipelines, and release Go modules
- Know when to reach for operators and serverless platforms

The big picture

In the Java world, shipping a service means a fat JAR, a base image with a Java runtime, a few JVM flags to keep memory in check, and a Helm chart. It works, but there are many moving parts between `mvn package` and a healthy pod.

Go removes one of the biggest parts: the runtime. `go build` produces one executable file with everything inside, including the garbage collector and the scheduler. That fact ripples through this whole chapter: smaller images, faster starts and simpler Dockerfiles.

🧩 **Think of it like a music box**

A fat JAR is like a song file: it's portable, but every place you take it needs a player (the JVM) of the right version. A Go binary is a music box. You wind it up and it plays, with nothing else to install. The catch is that you build one music box per kind of machine, which turns out to be very cheap in Go.

Here's the map of the chapter, from the Java tools you know to their Go counterparts.

Concept	Java / Spring	Go
Build output	Fat JAR (Spring Boot plugin)	One static binary (`go build`)
Runtime in the image	A JRE	Nothing (`CGO_ENABLED=0`)
Version info	`build-info` goal, `BuildProperties`	`-ldflags -X` , `debug.ReadBuildInfo`

Concept	Java / Spring	Go
Image without Dockerfile	Jib	ko
Release automation	Maven Release Plugin, JReleaser	GoReleaser
Memory tuning	`-XX:MaxRAMPercentage`	`GOMEMLIMIT`
CPU awareness	JVM container support	Container-aware `GOMAXPROCS` (1.25)
Operators	Java Operator SDK	controller-runtime, kubebuilder
Publishing a library	Upload to Maven Central	Push a Git tag

Full code: `examples/ch18-deploy/`

Building binaries

`go build` compiles your `main` package and everything it imports into one file. For production you'll add a few flags, and each one has a clear job.

The flags you'll use in production

TERMINAL

```
go build -trimpath -ldflags "-s -w" -o bin/server ./server
```

Here's what each part does:

- `-o bin/server` names the output file.
- `-trimpath` removes local file system paths (like `/home/alice/src/...`) from the binary, so builds are reproducible and don't leak directory names.
- `-ldflags` passes options to the linker, the tool that stitches compiled packages into one executable.
- `-s -w` tell the linker to drop the symbol table and DWARF debug information. DWARF is the data debuggers like Delve use to map machine code back to your source.

Dropping debug data sounds scary, but panics and stack traces still show function names, files and line numbers; the runtime keeps its own tables for that. You only lose the ability to attach a debugger to the production binary. Here's the saving for the demo server, built for `linux/amd64` with Go 1.25.0:

Build	Size
`go build`	8.9 MB
`go build -trimpath -ldflags "-s -w"`	6.3 MB

Stamping a version into the binary

When something breaks at 3 a.m., the first question is "which version is running?" You want the answer in the startup log, on a `/version` endpoint and in your metrics.

In Spring Boot, you'd add the `build-info` goal to the Maven plugin. It writes `META-INF/build-info.properties`, and Spring exposes it as a `BuildProperties` bean.

JAVA

```java
@RestController
class VersionController {
    private final BuildProperties build;

    VersionController(BuildProperties build) {
        this.build = build;
    }

    @GetMapping("/version")
    Map<String, String> version() {
        return Map.of("version", build.getVersion());
    }
}
```

Go has no properties file to read. Instead, the linker can overwrite the value of a package-level string variable while it builds the binary. You declare the variables with sensible defaults:

GO version/main.go

```go
// The linker overwrites these with -ldflags "-X main.name=value".
// They must be package-level string variables, not constants.
var (
    version = "dev"
    commit  = "unknown"
    date    = "unknown"
)
```

Then you set them at build time. In CI the values come from your Git tag and commit:

TERMINAL

```
go build -trimpath \
  -ldflags "-s -w -X main.version=1.4.2 -X main.commit=3f9c2e1" \
  -o bin/version ./version
./bin/version
```

OUTPUT

```
version:     1.4.2
commit:      3f9c2e1
built:       unknown
go:          go1.25.0
module:      example.com/goguide/ch18
trimpath:    true
cgo:         0
platform:    windows/amd64
GOMAXPROCS:  20
mem limit:   none
```

The name after `-X` is the package's import path plus the variable, so a variable elsewhere looks like `-X example.com/orderly/internal/platform.Version=1.4.2`.

> ⚠️ **Gotcha: -X fails silently**
>
> If you misspell the variable name, or point `-X` at a constant, the linker doesn't complain and you ship a binary that says `dev`. Add a release-pipeline check that calls `/version` (or runs the binary) and compares the result with the tag.

What Go records for you

The toolchain also embeds build information automatically, like a built-in `git-commit-id` Maven plugin. Read it inside the program with `runtime/debug.ReadBuildInfo`, or from outside with `go version -m`:

TERMINAL

```
go version -m bin/version
```

OUTPUT

```
bin/version: go1.25.0
    path    example.com/goguide/ch18/version
    mod example.com/goguide/ch18    (devel)
    build   -buildmode=exe
    build   -compiler=gc
    build   -trimpath=true
    build   CGO_ENABLED=0
    build   GOARCH=amd64
    build   GOOS=windows
    build   GOAMD64=v1
```

Inside a Git checkout you'll also see `vcs.revision` (the commit), `vcs.time` and `vcs.modified`. Since Go 1.24, the `mod` line shows a version from your Git tag, such as `v1.4.2`, with `+dirty` for uncommitted changes. The example folder isn't a Git repository, hence `(devel)`.

The example's `fromBuildInfo` function prefers the `-X` values and falls back to these recorded ones, so even a plain `go build` reports a useful commit. It takes the `*debug.BuildInfo` as a parameter, so tests can feed it anything they like.

Static binaries and CGO_ENABLED=0

cgo is Go's bridge to C code. With cgo enabled, a few standard packages (notably `net` and `os/user`) may call the system's C library, so the binary is *dynamically linked*: it expects that library on the machine it runs on. Minimal images like `scratch` and distroless `static` don't have one. The fix is one environment variable:

TERMINAL

```
CGO_ENABLED=0 go build -trimpath -ldflags "-s -w" -o bin/server ./server
```

Now Go uses its pure-Go implementations and the result is fully static: it runs on any Linux kernel of the right architecture, with nothing else installed.

> 💡 **If you're unsure, set CGO_ENABLED=0 for services**
>
> The house picks in this book (pgx, franz-go, go-redis) are pure Go. You need cgo only for libraries that wrap C code, such as `mattn/go-sqlite3` or `confluent-kafka-go`; prefer pure-Go alternatives like `modernc.org/sqlite` and `franz-go`. One exception: the race detector (`go test -race`) needs cgo, so your CI test job keeps it on.

Cross-compilation

Java's promise was "write once, run anywhere", thanks to the JVM. Go's is "write once, compile for anywhere". Set `GOOS` (the operating system) and `GOARCH` (the CPU architecture) and the compiler does the rest, with no extra toolchain.

TERMINAL

```
export CGO_ENABLED=0
GOOS=linux   GOARCH=amd64 go build -o bin/server-linux-amd64 ./server
GOOS=linux   GOARCH=arm64 go build -o bin/server-linux-arm64 ./server
GOOS=darwin  GOARCH=arm64 go build -o bin/server-darwin-arm64 ./server
GOOS=windows GOARCH=amd64 go build -o bin/server.exe ./server
```

All four took seconds on one Windows laptop, and with `-trimpath -ldflags "-s -w"` each came out between 5.9 and 6.4 MB. `go tool dist list` shows every target (48 on Go 1.25). Many teams now run cheaper ARM nodes (AWS Graviton, Azure Cobalt) while developing on x86 or Apple Silicon; in Go, the other architecture is a flag, not a project.

> ☕ **Coming from Java**
>
> GraalVM native images are the closest Java equivalent of a Go binary: fast start, no JVM. They're powerful, but builds take minutes, reflection needs configuration, and you usually build on the same OS and architecture you'll run on. Go gives you that deployment shape by default, in seconds.

Container images

A Go service image needs very little: the binary, CA certificates for outgoing HTTPS, time zone data and a non-root user. The goal is to keep the compiler and source code *out* of the final image.

A multi-stage Dockerfile

A typical Spring Boot Dockerfile builds with a JDK image and runs on a JRE image:

DOCKERFILE Spring Boot (for comparison)

```dockerfile
FROM eclipse-temurin:21-jdk AS build
WORKDIR /src
COPY . .
RUN ./mvnw -q package -DskipTests

FROM eclipse-temurin:21-jre
COPY --from=build /src/target/*.jar /app/app.jar
USER 1000
ENTRYPOINT ["java", "-XX:MaxRAMPercentage=75", "-jar", "/app/app.jar"]
```

The Go version has the same two-stage shape. A **multi-stage build** uses one image to compile and a second, much smaller image to run; only the files you copy across survive. Here's the first stage:

DOCKERFILE Dockerfile (build stage)

```dockerfile
# syntax=docker/dockerfile:1
ARG GO_VERSION=1.27
# Compile on the build machine's own platform and let Go cross-compile.
FROM --platform=$BUILDPLATFORM golang:${GO_VERSION}-alpine AS build
WORKDIR /src

# Module files first: this layer stays cached until go.mod/go.sum change.
COPY go.mod go.sum ./
RUN go mod download

COPY . .
ARG VERSION=dev
ARG TARGETOS
ARG TARGETARCH
RUN CGO_ENABLED=0 GOOS=$TARGETOS GOARCH=$TARGETARCH \
    go build -trimpath -ldflags "-s -w -X main.version=${VERSION}" \
    -o /out/server ./server
```

Three details are worth a closer look:

- **Copying `go.mod` and `go.sum` first** caches dependency downloads, so editing a `.go` file doesn't trigger a fresh download.
- **`$BUILDPLATFORM`, `TARGETOS` and `TARGETARCH`** are filled in by Docker BuildKit. With `docker buildx build --platform linux/amd64,linux/arm64`, the compiler runs natively and cross-compiles instead of running under slow CPU emulation.
- **The builder uses Go 1.27** while `go.mod` says `go 1.25.0`. That's recommended: newer toolchains build older modules, with the latest security fixes.

And here's the second stage, which is the image you actually ship:

DOCKERFILE Dockerfile (runtime stage)

```
FROM gcr.io/distroless/static-debian13:nonroot
COPY --from=build /out/server /server
# 65532 is the distroless "nonroot" user. A numeric ID lets Kubernetes
# verify runAsNonRoot.
USER 65532:65532
EXPOSE 8080
# Exec form: our binary is PID 1 and receives SIGTERM directly.
ENTRYPOINT ["/server"]
```

Build and run it like any other image:

TERMINAL

```
docker build --build-arg VERSION=1.4.2 -t orderly-demo .
docker run --rm -p 8080:8080 orderly-demo
```

> ⚠️ **Gotcha: named users and runAsNonRoot**
>
> If your Dockerfile says `USER nonroot` (a name) and your pod sets `runAsNonRoot: true`, the kubelet refuses to start the container, because it can't prove a *name* isn't root. Use the numeric ID, or set `runAsUser: 65532` in the pod. Our example does both.

Distroless or scratch?

Distroless images contain only what a program needs at runtime: no shell, no package manager. **Scratch** is Docker's name for a completely empty image. Both work for a static Go binary; they differ in what you bring yourself.

You need	distroless/static	scratch
CA certificates (HTTPS)	Included	Copy from builder
Time zone data	Included	`import _ "time/tzdata"`
Non-root user	`nonroot` (65532)	Numeric `USER` only
A `/tmp` directory	Included	Missing

The `time/tzdata` package embeds the time zone database in your binary (about 450 KB), a safety net if you call `time.LoadLocation` anywhere:

GO server/main.go

```
import (
    // Embeds the time zone database (about 450 KB) so time.LoadLocation
    // works even in a scratch image that has no /usr/share/zoneinfo.
    _ "time/tzdata"
)
```

> 💡 **If you're unsure, use distroless/static**
>
> It's about 2 MiB, already has certificates, time zones and a non-root user, and scanners understand it because it's built from Debian packages. Use `scratch` only if you want to manage those pieces yourself (see `Dockerfile.scratch`). To debug a pod with no shell, attach an ephemeral container with `kubectl debug`, or use the `:debug-nonroot` tag, which adds a BusyBox shell.

Image size and startup, compared with the JVM

Numbers depend on your service, but here's the picture you can expect.

	Spring Boot on a JRE	Go on distroless
Base image	Hundreds of MB	About 2 MiB
Final image	Usually 200 MB or more	Usually 10 to 40 MB
Startup to ready	Seconds	Milliseconds
Idle memory	Hundreds of MB	Tens of MB

Small images pull faster onto new nodes, and fast starts make rollouts and autoscaling feel instant. Spring has answers too (class data sharing, CRaC, native images), but Go gives you this without extra work.

ko: images without a Dockerfile

If you've used **Jib**, you know the idea: build an image from your build tool, with no Docker daemon or Dockerfile. **ko** does that for Go: it runs `go build`, layers the binary onto a small base image and pushes it.

TERMINAL

```
KO_DOCKER_REPO=ghcr.io/example VERSION=1.4.2 \
  ko build --bare --platform=linux/amd64,linux/arm64 ./server
```

Settings like the base image and ldflags live in `.ko.yaml`:

YAML .ko.yaml

```yaml
defaultBaseImage: gcr.io/distroless/static-debian13:nonroot
builds:
  - id: server
    main: ./server
    ldflags:
      - -s -w
      - -X main.version={{.Env.VERSION}}
```

Without that file, ko's default base image is `cgr.dev/chainguard/static`.

GoReleaser

For tools you hand out as binaries, **GoReleaser** is the standard. On a Git tag it builds every platform, creates archives and checksums, and publishes a GitHub or GitLab release. It covers what the Maven Release Plugin and JReleaser (itself inspired by GoReleaser) do in Java.

YAML .goreleaser.yaml

```yaml
version: 2
builds:
  - main: ./version
    binary: orderly-version
    env: [CGO_ENABLED=0]
    flags: [-trimpath]
    goos: [linux, darwin, windows]
    goarch: [amd64, arm64]
archives:
  - formats: [tar.gz]
    format_overrides:
      - goos: windows
        formats: [zip]
```

There are no ldflags because GoReleaser's default already sets `main.version`, `main.commit` and `main.date` (plus `-s -w`), which is why our variables have those names. Try it locally with `goreleaser release --snapshot --clean`.

Buildpacks, briefly

If you've run `mvn spring-boot:build-image`, you've used Cloud Native Buildpacks: Spring Boot calls the Paketo buildpacks. Paketo has a Go buildpack too:

TERMINAL

```
pack build orderly --buildpack paketo-buildpacks/go \
  --builder paketobuildpacks/builder-jammy-base
```

Buildpacks shine when a platform team wants one way to build many languages. For one Go service, a 25-line Dockerfile is easier to read and tweak.

> 💡 **Which image builder should you pick?**
>
> If you're unsure, use a multi-stage Dockerfile: everyone can read it and every CI system supports it. Choose ko for fast, daemonless builds in an all-Go team, GoReleaser for binaries you distribute, and buildpacks when your platform team standardises on them.

Running on Kubernetes

Kubernetes needs your service to cooperate: report its health, respect its limits and shut down politely. Spring Boot does much of this through Actuator. In Go you write a few short pieces yourself.

Liveness, readiness and startup probes

A **probe** is a check the kubelet runs against your container on a schedule. There are three kinds, and mixing them up causes classic incidents.

> 🧩 **Think of it like a restaurant kitchen**
>
> **Liveness** asks "is the chef conscious?" If not, replace the chef (restart the container). **Readiness** asks "is the kitchen taking orders?" If not, waiters stop sending customers for a while, but nobody is replaced. **Startup** asks "has the chef finished unpacking the morning delivery?" Until then, the other checks wait.

In Spring Boot, Actuator gives you `/actuator/health/liveness` and `/actuator/health/readiness`. In Go, they're two small handlers:

GO server/main.go (excerpt)

```go
// Liveness: "is this process still working?" Keep it cheap and
// never check dependencies here.
mux.HandleFunc("GET /healthz", func(w http.ResponseWriter, _ *http.Request) {
    w.WriteHeader(http.StatusOK)
})

// Readiness: "should the Service send me traffic right now?"
mux.HandleFunc("GET /readyz", func(w http.ResponseWriter, _ *http.Request) {
    if !ready.Load() {
        http.Error(w, "not ready", http.StatusServiceUnavailable)
        return
    }
    w.WriteHeader(http.StatusOK)
})
```

`ready` is an `atomic.Bool`, a boolean many goroutines can use safely. It turns `true` once the server listens and `false` when shutdown begins. The capstone's `/readyz` also pings the database (Chapter 20).

And here's how the Deployment wires them up:

YAML k8s/deployment.yaml (excerpt)

```yaml
startupProbe:
  httpGet: {path: /healthz, port: http}
  periodSeconds: 2
  failureThreshold: 15     # up to 30s to start before liveness kicks in
livenessProbe:
  httpGet: {path: /healthz, port: http}
  periodSeconds: 10
  failureThreshold: 3
readinessProbe:
  httpGet: {path: /readyz, port: http}
  periodSeconds: 5
  failureThreshold: 2
```

> ⚠️ **Gotcha: never check the database in liveness**
>
> If liveness pings Postgres and Postgres has a hiccup, Kubernetes restarts *every* pod at once, turning a blip into an outage. Dependencies belong in readiness, which takes a pod out of rotation without killing it.

Go services start in milliseconds, but the startup probe is cheap insurance for slow work before serving, such as migrations or cache warm-up.

Requests, limits and GOMAXPROCS

Requests are what the scheduler reserves for your pod. **Limits** are the ceiling: exceed the memory limit and the container is killed (OOMKilled); exceed the CPU limit and it's throttled (paused until the next scheduling period).

YAML k8s/deployment.yaml (excerpt)

```yaml
resources:
  requests:
    cpu: 250m        # a quarter of a core
    memory: 128Mi
  limits:
    cpu: "1"
    memory: 256Mi
```

`GOMAXPROCS` is the number of operating-system threads that can run Go code at once. Before Go 1.25 it defaulted to the *node's* CPU count, so a one-core pod on a 64-core node ran 64 threads, burned its quota in milliseconds and got throttled: mysterious latency spikes. Teams added `go.uber.org/automaxprocs` to fix it.

Since Go 1.25 the runtime does this itself. On Linux it uses the container's CPU limit when that's lower than the CPU count, rounding fractions up and never going below 2 (unless the machine has fewer CPUs). It also re-checks periodically, so a resized pod gets an updated value.

> 📝 **Version note: it follows your go.mod**
>
> The new behaviour follows the `go` line in your `go.mod`, like other runtime defaults. With `go 1.25.0` or later you get it automatically and can delete `automaxprocs`. It reads the CPU **limit**, not the request, and setting `GOMAXPROCS` yourself turns it off. (The JVM has been container-aware since JDK 10, so this is Go catching up.)

GOMEMLIMIT from the container's memory limit

By default, Go's garbage collector lets the heap grow to about twice the live data before collecting. It knows nothing about container limits, so a traffic spike can push it over and get the pod OOMKilled.

`GOMEMLIMIT` is a soft memory limit: as memory use approaches it, the collector runs more often to stay under. It's the Go cousin of `-XX:MaxRAMPercentage`, but a target rather than a fixed heap size. Feed it the container's limit with the **downward API**, which exposes a pod's own settings to it:

YAML k8s/deployment.yaml (excerpt)

```yaml
env:
  # The container's memory limit in bytes, e.g. "268435456".
  - name: GOMEMLIMIT
    valueFrom:
      resourceFieldRef:
        resource: limits.memory
```

`GOMEMLIMIT` takes plain bytes or a suffix such as `230MiB`. Our version program prints what the runtime ends up using:

TERMINAL

```
GOMEMLIMIT=230MiB GOMAXPROCS=2 ./bin/version
```

OUTPUT

```
GOMAXPROCS:  2
mem limit:   230 MiB
```

> 💡 **Leave a little headroom**
>
> The Go GC guide suggests 5 to 10 percent of headroom for memory the runtime doesn't track. The downward API can't do arithmetic, so the setup above uses 100 percent of the limit; as a *soft* target that's still far better than nothing. For headroom, set a fixed value instead (for a 256Mi limit, `GOMEMLIMIT=230MiB`) and keep both numbers side by side in your Helm values.

Graceful termination

During a rolling update, old pods stop while new ones take over. If a pod dies with requests in flight, customers see errors. Graceful termination means: stop receiving traffic, finish the work, then exit. Here's the sequence:

```
rollout or scale-down
       |
       v
Pod marked "Terminating" ----> removed from Service endpoints
       |                       (load balancers catch up over
       v                        the next few seconds)
preStop hook: sleep 5s
       |
       v
SIGTERM sent to PID 1 (our binary)
       |   readiness -> 503, srv.Shutdown drains requests
       v
process exits 0     ... or SIGKILL when terminationGracePeriodSeconds
                        (counted from the very start) runs out
```

The `preStop` sleep gives load balancers time to notice. Distroless has no `sleep` command, so use the built-in sleep action (on by default since Kubernetes 1.30):

YAML k8s/deployment.yaml (excerpt)

```yaml
spec:
  terminationGracePeriodSeconds: 30   # preStop + shutdown must fit
  containers:
    - name: orderly
      lifecycle:
        preStop:
          sleep:
            seconds: 5
```

In Go, catch the signal, flip readiness off and let `http.Server.Shutdown` finish in-flight requests. The shutdown timeout (20 seconds) plus the preStop sleep must fit inside the grace period:

GO server/main.go (excerpt)

```go
ctx, stop := signal.NotifyContext(context.Background(),
    os.Interrupt, syscall.SIGTERM)
defer stop()
// ... start srv.ListenAndServe in a goroutine that reports to errCh ...

select {
case err := <-errCh: // failed to start, e.g. port in use
    return err
case <-ctx.Done(): // SIGTERM arrived
}

// Stop advertising readiness, then let in-flight requests finish.
ready.Store(false)
shutdownCtx, cancel := context.WithTimeout(context.Background(),
    20*time.Second)
defer cancel()
if err := srv.Shutdown(shutdownCtx); err != nil &&
    !errors.Is(err, http.ErrServerClosed) {
    return err
}
```

In Spring Boot, `server.shutdown=graceful` (the default since 3.4) and `spring.lifecycle.timeout-per-shutdown-phase` do this for you. Chapter 14 covers the full shutdown order with background workers.

ConfigMaps and Secrets as environment variables

A **ConfigMap** holds settings; a **Secret** holds passwords and keys. Hand both to a Go service as environment variables, which `caarlos0/env` reads into a typed struct (Chapter 10).

YAML k8s/deployment.yaml (excerpt)

```yaml
envFrom:
  - configMapRef:
      name: orderly-config      # every key becomes an env var
env:
  - name: DATABASE_URL
    valueFrom:
      secretKeyRef:
        name: orderly-secrets
        key: database-url
```

It's Spring's relaxed binding (`SPRING_DATASOURCE_URL` fills `spring.datasource.url`), except Go makes the mapping explicit in struct tags. In production, create Secrets from a vault (for example with the External Secrets Operator), not from Git.

Autoscaling with the HPA

The **HorizontalPodAutoscaler** (HPA) adds or removes pods based on a metric, most often CPU use as a percentage of the CPU *request*:

YAML k8s/service-hpa.yaml (excerpt)

```yaml
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: orderly
spec:
  scaleTargetRef: {apiVersion: apps/v1, kind: Deployment, name: orderly}
  minReplicas: 2
  maxReplicas: 10
  metrics:
    - type: Resource
      resource:
        name: cpu
        target: {type: Utilization, averageUtilization: 70}
```

Because Go pods start almost instantly, scaling out takes effect quickly. For Kafka consumers, CPU is often the wrong signal; KEDA can scale on consumer lag instead.

Helm basics

Helm is a package manager for Kubernetes. A **chart** is a folder of YAML templates plus a `values.yaml` file with the settings people may change. Here's a slice of the example chart:

YAML helm/orderly/templates/deployment.yaml (excerpt)

```yaml
{{- $tag := .Values.image.tag | default .Chart.AppVersion }}
image: "{{ .Values.image.repository }}:{{ $tag }}"
resources:
  {{- toYaml .Values.resources | nindent 12 }}
```

The `{{ ... }}` parts are Go's own `text/template` syntax, filled from `values.yaml`.

Three commands cover most daily use:

TERMINAL

```
helm lint helm/orderly                          # catch template mistakes
helm template demo helm/orderly | less          # see the rendered YAML
helm upgrade --install orderly helm/orderly \
  --namespace orderly --set image.tag=1.4.2     # install or upgrade
```

The capstone's full chart is in `capstone/deploy/helm/orderly` (Chapter 20).

GitOps with Argo CD, briefly

GitOps means Git is the source of truth for your cluster. Instead of CI running `helm upgrade`, a controller in the cluster watches a Git repository and applies what it finds. **Argo CD** is the most popular one.

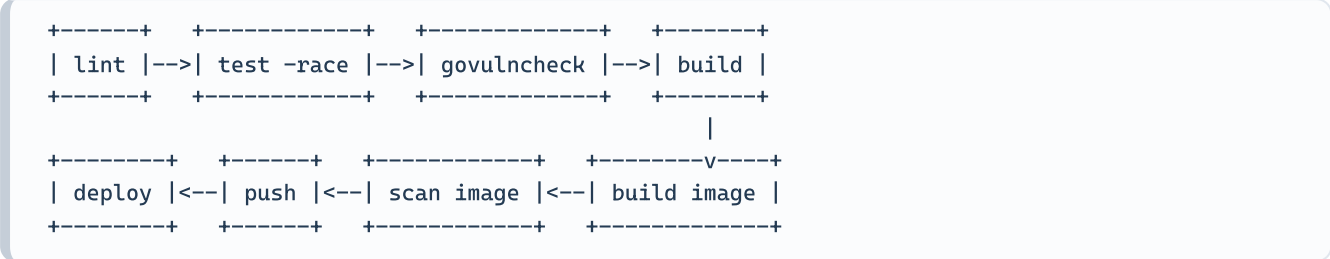
YAML k8s/argocd-application.yaml (excerpt)

```yaml
spec:
  source:
    repoURL: https://github.com/example/orderly-deploy.git
    targetRevision: main
    path: helm/orderly
  destination:
    server: https://kubernetes.default.svc
    namespace: orderly
  syncPolicy:
    automated: {prune: true, selfHeal: true}
```

CI then ends by bumping the image tag in that repository. Rollbacks become `git revert`, and every production change is a reviewable commit.

CI/CD pipelines

A good pipeline catches problems in the cheapest place first. Here's the order, which maps neatly onto a Maven pipeline:



Step	Java / Maven	Go
Lint	Checkstyle, SpotBugs	`golangci-lint`
Test	`mvn verify`	`go test -race ./...`
Dependency check	OWASP Dependency-Check	`govulncheck`
Image scan	Trivy, Grype	Trivy, Grype

govulncheck checks your dependencies against the Go vulnerability database, and analyses your call graph so it reports only vulnerabilities your code can actually reach. That means far fewer false alarms than version-only scanners.

GitHub Actions

The test job. `setup-go` caches modules by default, and hosted Ubuntu runners have a C compiler, so `-race` works:

YAML ci/github-actions.yaml (test job)

```yaml
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v7
      - uses: actions/setup-go@v7
        with:
          go-version: stable
      - name: Lint
        uses: golangci/golangci-lint-action@v9
        with:
          version: v2.15
      - name: Test with the race detector
        run: go test -race ./...
      - name: Check for known vulnerabilities
        run: go run golang.org/x/vuln/cmd/govulncheck@latest ./...
```

The image job builds locally, scans, and pushes only if the scan passes:

YAML ci/github-actions.yaml (image job, excerpt)

```yaml
      - uses: docker/setup-buildx-action@v4
      - uses: docker/build-push-action@v7
        with:
          context: .
          load: true # keep it local until the scan passes
          tags: ${{ env.IMAGE }}:${{ github.sha }}
          build-args: VERSION=${{ github.sha }}
      - uses: aquasecurity/trivy-action@v0.36.0
        with:
          image-ref: ${{ env.IMAGE }}:${{ github.sha }}
          severity: CRITICAL,HIGH
          ignore-unfixed: true
          exit-code: "1"
      # ... docker/login-action@v4, then:
      - run: docker push ${{ env.IMAGE }}:${{ github.sha }}
```

The full file adds a `deploy` job that runs `helm upgrade` behind a protected `production` environment, which gives you manual approval.

> 💡 **Pin third-party actions**
>
> Tags like `@v7` can be moved by whoever controls the repository. In production pipelines, pin actions to a full commit SHA and let Dependabot or Renovate update them.

Azure Pipelines

Azure Pipelines uses stages. `GoTool@0` installs Go; the rest are scripts:

YAML ci/azure-pipelines.yml (test stage)

```yaml
stages:
  - stage: Test
    jobs:
      - job: test
        steps:
          - task: GoTool@0
            inputs:
              version: "1.27.3"
          - script: |
              curl -sSfL https://golangci-lint.run/install.sh | sh -s v2.15.0
              ./bin/golangci-lint run
            displayName: Lint
          - script: go test -race ./...
            displayName: Test with the race detector
          - script: go run golang.org/x/vuln/cmd/govulncheck@latest ./...
            displayName: Check for known vulnerabilities
```

The image stage builds with `Docker@2`, scans with the Trivy container, then pushes:

YAML ci/azure-pipelines.yml (image stage, excerpt)

```yaml
- task: Docker@2
  inputs:
    command: build
    containerRegistry: acr-connection # a service connection
    repository: $(image)
    tags: $(tag)
    arguments: --build-arg VERSION=$(tag)
- script: >-
    docker run --rm -v /var/run/docker.sock:/var/run/docker.sock
    aquasec/trivy:0.76.1 image --exit-code 1 --ignore-unfixed
    --severity HIGH,CRITICAL $(registry)/$(image):$(tag)
  displayName: Scan image with Trivy
# ... a second Docker@2 task with command: push
```

A final `Deploy` stage uses a deployment job tied to a `production` environment (where approvals live) and the `HelmDeploy@1` task. See `ci/azure-pipelines.yml` for the whole file.

Releasing Go modules

Sometimes you ship a library, such as a shared Orderly API client. In Java that means a `groupId`, signing keys and an upload to Maven Central or Artifactory. In Go there's no upload step: **a module version is a Git tag.**

TERMINAL

```
git tag v1.3.0
git push origin v1.3.0
# Anyone can now depend on it:
go get example.com/orderly-client@v1.3.0
```

The public proxy (`proxy.golang.org`) caches the code on first request, and the checksum database records its hash so it can't silently change. For private repositories, tell Go to skip the public proxy:

TERMINAL

```
go env -w GOPRIVATE=github.com/your-company/*
```

Semantic versioning and v2+

Go modules follow semantic versioning: minor versions add features, patches fix bugs, and `v0.x` means "no promises yet". The big difference from Maven is a breaking change: in Go, a new major version gets a **new import path**. The module line ends in `/v2`, and so do the imports:

GO.MOD go.mod

```
module example.com/orderly-client/v2

go 1.25.0
```

GO

```
import orderly "example.com/orderly-client/v2"
```

This feels strange at first, and every Java developer trips on it. The payoff: `v1` and `v2` are different packages, so one program can use both during a migration. Two more tools round this out:

- `retract` in `go.mod` marks a bad version so `go get` skips it: `retract v1.3.1 // breaks PlaceOrder`.
- `go install pkg@version` installs a command without touching your `go.mod`, for example `go install github.com/pressly/goose/v3/cmd/goose@v3.27.0`.

Writing Kubernetes operators

An **operator** is a program inside the cluster that manages something for you, such as a database or certificates. Kubernetes is written in Go, and so are most operators.

> 🧩 **Think of it like a thermostat**
>
> A thermostat doesn't follow a script like "heat for ten minutes". It keeps comparing the temperature you asked for with the room's, and acts on the difference. An operator's **reconcile loop** does the same: compare desired and actual state, fix the gap, repeat whenever something changes.

Layer	Go	Java
API client	client-go	fabric8 Kubernetes client
Operator framework	controller-runtime	Java Operator SDK
Scaffolding	kubebuilder	JOSDK Maven or Quarkus tooling

In the Java Operator SDK you implement `Reconciler<R>`:

JAVA

```
@ControllerConfiguration
public class PdbReconciler implements Reconciler<Deployment> {
    @Override
    public UpdateControl<Deployment> reconcile(
            Deployment dep, Context<Deployment> context) {
        // create or update the PodDisruptionBudget for dep
        return UpdateControl.noUpdate();
    }
}
```

The controller-runtime version has the same shape. This one keeps a PodDisruptionBudget (which stops node drains from removing every replica at once) next to any Deployment labelled `orderly.example.com/pdb=true`:

GO operator/reconciler.go (excerpt)

```
func (r *PDBReconciler) Reconcile(ctx context.Context,
    req ctrl.Request) (ctrl.Result, error) {
```

```
    var dep appsv1.Deployment
    if err := r.Get(ctx, req.NamespacedName, &dep); err != nil {
        return ctrl.Result{}, client.IgnoreNotFound(err)
    }
    if dep.Labels[pdbLabel] != "true" {
        return ctrl.Result{}, nil
    }

    pdb := &policyv1.PodDisruptionBudget{ObjectMeta: metav1.ObjectMeta{
        Name: dep.Name, Namespace: dep.Namespace,
    }}
    _, err := controllerutil.CreateOrUpdate(ctx, r.Client, pdb, func() error {
        minAvailable := intstr.FromInt32(1)
        pdb.Spec.MinAvailable = &minAvailable
        pdb.Spec.Selector = dep.Spec.Selector
        return controllerutil.SetControllerReference(&dep, pdb, r.Scheme())
    })
    return ctrl.Result{}, err // an error means "retry with backoff"
}
```

`CreateOrUpdate` applies your desired state and writes only if something changed. `SetControllerReference` makes the Deployment the owner, so Kubernetes deletes the PDB along with it. The example's test runs this twice against controller-runtime's fake client to prove it's idempotent.

> 💡 **If you're unsure, start with kubebuilder**
>
> `kubebuilder init` and `kubebuilder create api` scaffold the project, custom resource types, RBAC rules and tests on top of controller-runtime. Write an operator only to encode real operational knowledge; for "deploy my app", a Helm chart is enough.

Serverless, briefly

Fast starts and small binaries make Go a great fit for serverless, where cold starts are the main cost.

AWS Lambda

AWS deprecated the old `go1.x` runtime. Today Go runs on the OS-only `provided.al2023` runtime with the `aws-lambda-go` library. The handler is a plain function, so it's easy to unit test:

GO lambda/main.go (excerpt)

```
func handle(ctx context.Context,
    req events.APIGatewayV2HTTPRequest) (response, error) {
    id := req.PathParameters["productId"]
    if id == "" {
        return response{StatusCode: http.StatusBadRequest}, nil
    }

    body, err := json.Marshal(stock{ProductID: id, Available: 42})
    if err != nil {
        return response{}, err
    }
    return response{
        StatusCode: http.StatusOK,
```

```
        Headers:    map[string]string{"Content-Type": "application/json"},
        Body:       string(body),
    }, nil
}

func main() {
    lambda.Start(handle) // blocks and serves events until shut down
}
```

The executable must be named `bootstrap`. The `lambda.norpc` tag leaves out code that only the deprecated runtime needed:

TERMINAL

```
GOOS=linux GOARCH=arm64 CGO_ENABLED=0 \
  go build -tags lambda.norpc -o bootstrap ./lambda
zip lambda.zip bootstrap
aws lambda create-function --function-name orderly-stock \
  --runtime provided.al2023 --handler bootstrap --architectures arm64 \
  --role arn:aws:iam::111122223333:role/lambda-ex \
  --zip-file fileb://lambda.zip
```

Where a Java `RequestHandler` often needs SnapStart to tame cold starts, a Go function starts fast with no tuning.

Google Cloud Run

Cloud Run runs any container that listens on the `PORT` environment variable. Our demo server reads `PORT`, so its image works unchanged:

TERMINAL

```
gcloud run deploy orderly \
  --image=europe-docker.pkg.dev/my-project/apps/orderly:1.4.2 \
  --region=europe-west1
```

Azure Functions custom handlers

Azure Functions has no native Go runtime, but **custom handlers** let any language join in. The Functions host starts your binary and forwards requests to the port in `FUNCTIONS_CUSTOMHANDLER_PORT`. For HTTP-only functions, `enableProxyingHttpRequest` passes the raw request through:

JSON host.json

```
{
  "version": "2.0",
  "customHandler": {
    "description": { "defaultExecutablePath": "handler" },
    "enableProxyingHttpRequest": true
  }
}
```

Microsoft's docs note that custom handlers aren't a general web host. For a whole service like Orderly, Azure Container Apps (ordinary containers) is the better fit.

Recap

- Build production binaries with `CGO_ENABLED=0 go build -trimpath -ldflags "-s -w"`: static, smaller and reproducible.
- Stamp versions with `-X main.version=...`; `debug.ReadBuildInfo` adds the Go version, flags and Git commit for free.
- Cross-compile with `GOOS` and `GOARCH`; in Docker, pair them with `$BUILDPLATFORM` for fast multi-arch images.
- Ship on `gcr.io/distroless/static-debian13:nonroot` with a numeric `USER`; on `scratch`, bring certificates and `time/tzdata` yourself.
- Liveness stays cheap, readiness checks dependencies, and a startup probe covers slow starts.
- Go 1.25 sets `GOMAXPROCS` from the CPU limit; set `GOMEMLIMIT` from the memory limit, ideally with headroom.
- Graceful termination is preStop sleep, SIGTERM, then `srv.Shutdown`, all inside `terminationGracePeriodSeconds`.
- Pipelines run lint, `go test -race`, `govulncheck`, build, image, scan, push and deploy; a module release is a Git tag, and v2+ changes the import path.

Try it yourself

> **Exercise 1: Add a commit to /version**
>
> The demo server only injects `version`. Add a `commit` variable, return it from `/version`, and build with `-ldflags "-X main.commit=$(git rev-parse --short HEAD)"`. Hint: copy the `fromBuildInfo` fallback from `version/buildinfo.go` so a plain `go build` still reports the commit.

> **Exercise 2: Cross-compile and inspect**
>
> Build the server for `linux/arm64` and `windows/amd64`, and run `go version -m` on each. Which `build` lines differ? Then build once with `-trimpath` and once without, both with an `-ldflags` value. Hint: look for the `-ldflags` line in each output.

> **Exercise 3: Stretch: delete the PDB when the label goes away**
>
> The operator never removes a PDB if someone deletes the `orderly.example.com/pdb` label. Extend `Reconcile` to delete it in that case, and add a fake-client test. Hint: `client.IgnoreNotFound` helps when the PDB is already gone.

Further reading

- Command go: build flags: the official reference for `-trimpath`, `-ldflags` and friends.
- A Guide to the Go Garbage Collector: how `GOGC` and `GOMEMLIMIT` work, with advice for containers.

- Go 1.25 release notes: the details of container-aware `GOMAXPROCS`.
- ko documentation: building and deploying Go images without a Dockerfile.
- Kubernetes probes: liveness, readiness and startup in depth.
- Go modules: v2 and beyond: the official guide to major version upgrades.
- The Kubebuilder Book: a hands-on tutorial for writing operators with controller-runtime.

CHAPTER 19

More Enterprise Essentials

> *Real enterprise systems need a lot of small but important pieces: admin CLIs, money that adds up, tenants that never see each other's data, audit trails, translations and private libraries. This chapter gives you a solid Go answer for each.*

🎯 **What you'll learn**

- How to build a polished admin CLI with cobra, and test it in-process
- How to ship templates and static files inside your binary with `go:embed`
- How to handle money safely with integer cents and `shopspring/decimal`
- Patterns for multi-tenancy and audit logging using context and middleware
- How to translate messages, plug in extensions, and document your packages
- How to use private company modules, and how to handle time zones correctly

The big picture

Most of this book covers the big pillars: APIs, data, messaging, observability. This chapter covers the smaller things every serious service eventually needs. Each section stands alone, so jump straight to the one you need today.

🧩 **Think of it like the small-parts drawer in a workshop**

The big machines get the attention, but every job also needs the right screws, hinges and brackets. This chapter is that drawer: small parts, each one chosen carefully so it fits and lasts.

Concern	Java / Spring	Go
Admin CLI	picocli, Spring Shell	cobra (or `flag` for tiny tools)
Bundled resources	Classpath, `ClassPathResource`	`go:embed` and `embed.FS`
Money	`BigDecimal`	`int64` cents; `shopspring/decimal` for rates
Multi-tenancy	Hibernate tenant resolver	Tenant in `context.Context`
Audit trail	Spring Data auditing, Envers	Middleware plus explicit events
Translations	`MessageSource`, `ResourceBundle`	go-i18n plus `golang.org/x/text`
Extensions	`ServiceLoader`, OSGi	Interface + registry; hashicorp/go-plugin
API docs	Javadoc	Doc comments, `Example` tests, pkg.go.dev

Concern	Java / Spring	Go
Private libraries	Nexus, Artifactory	`GOPRIVATE` plus Git authentication

Full code for this chapter: `examples/ch19-enterprise-extras/`, one folder per section. Everything runs offline.

Command-line tools with cobra

Every platform team ends up with an admin tool: something to list products, place a test order or replay an event. Go is a wonderful language for CLIs because the result is one small binary that runs anywhere. The most widely used library for them is **cobra** (v1.10.2 at the time of writing), which powers `kubectl`, `docker` and `hugo`.

The Java version: picocli

In Java you'd probably reach for picocli, which uses annotations:

JAVA

```
@Command(name = "place", description = "Place an order for one product")
class PlaceCommand implements Callable<Integer> {
    @Option(names = "--customer", required = true) String customer;
    @Option(names = "--sku", required = true) String sku;
    @Option(names = {"-q", "--qty"}, defaultValue = "1") int qty;

    @Override
    public Integer call() {
        System.out.printf("Placed %d x %s for %s%n", qty, sku, customer);
        return 0;
    }
}
```

Cobra does the same job without annotations. A command is a struct value, and flags are bound to ordinary variables.

Building `orderlyctl`

The root command holds the global flags that every subcommand inherits:

GO orderlyctl/root.go

```
func newRootCmd(c client) *cobra.Command {
    opts := &options{}
    root := &cobra.Command{
        Use:          "orderlyctl",
        Short:        "Manage Orderly products and orders",
        Version:      version, // adds a --version flag
        SilenceUsage: true,    // don't print usage for runtime errors
        // ... PersistentPreRunE validates --output ...
    }

    flags := root.PersistentFlags() // inherited by every subcommand
    flags.StringVarP(&opts.output, "output", "o", "table",
        "output format: table or json")
```

```
    // The default comes from ORDERLY_API_URL if set; the flag still wins.
    flags.StringVar(&opts.apiURL, "api-url",
        envOr("ORDERLY_API_URL", "http://localhost:8080"),
        "Orderly API URL")

    root.AddCommand(newProductsCmd(c, opts), newOrdersCmd(c, opts))
    return root
}
```

Building the tree inside a function, rather than in package-level variables as many cobra tutorials do, means each test gets a fresh command tree. The `client` parameter is a small interface; the example uses an in-memory fake, and a real build would pass an HTTP client for the Orderly API.

Here's the `orders place` subcommand. Its flags live in local variables, which the `RunE` closure reads:

GO orderlyctl/orders.go

```
func newPlaceCmd(c client, opts *options) *cobra.Command {
    var customer, sku string
    var qty int

    cmd := &cobra.Command{
        Use:     "place",
        Short:   "Place an order for one product",
        Example: "  orderlyctl orders place --customer c-42 --sku MUG-BLUE-01",
        Args:    cobra.NoArgs,
        RunE: func(cmd *cobra.Command, args []string) error {
            if qty < 1 {
                return fmt.Errorf("--qty must be at least 1, got %d", qty)
            }
            o, err := c.PlaceOrder(cmd.Context(), customer, sku, qty)
            if err != nil {
                return err
            }
            // ... print as JSON or text, depending on opts.output ...
            return nil
        },
    }

    cmd.Flags().StringVar(&customer, "customer", "", "customer ID")
    cmd.Flags().StringVar(&sku, "sku", "", "product SKU")
    cmd.Flags().IntVarP(&qty, "qty", "q", 1, "quantity to order")
    _ = cmd.MarkFlagRequired("customer")
    _ = cmd.MarkFlagRequired("sku")
    return cmd
}
```

- **`RunE`** returns an `error`, so failures flow back to `main` like any Go error. Prefer it over `Run`.
- **`Args: cobra.NoArgs`** rejects stray positional arguments; `cobra.ExactArgs(1)` (used by `orders cancel ORDER_ID`) demands exactly one.
- **`cmd.Context()`** is the context passed to `ExecuteContext` in `main`, wired to Ctrl+C with `signal.NotifyContext`. Pass it down to every API call.

Trying it out

Cobra generates help, suggestions, shell completion and error messages for you. Build the binary and give it a try:

TERMINAL

```
go build -o bin/ ./orderlyctl
./bin/orderlyctl products list
./bin/orderlyctl orders place --customer c-42 --sku MUG-BLUE-01 --qty 2
./bin/orderlyctl orders place --sku MUG-BLUE-01
```

OUTPUT

```
SKU          NAME         PRICE
MUG-BLUE-01  Blue Mug     12.99
TEE-RED-XL   Red T-Shirt  25.00
Placed order o-1001: 2 x MUG-BLUE-01 for c-42
Error: required flag(s) "customer" not set
```

To test a CLI, build a fresh root command, point its output at a buffer with `root.SetOut(&buf)`, pass arguments with `root.SetArgs(...)` and call `Execute()`. Because commands print with `cmd.OutOrStdout()`, the test sees everything they write. The `run` helper in `orderlyctl/main_test.go` does exactly this in ten lines.

> 💡 **If you're unsure, use cobra**
>
> For anything with subcommands, cobra is the safe default: it's mature, maintained, and familiar to everyone who has used `kubectl`. For a tool with three flags, the standard library's `flag` package is enough. `urfave/cli` and `alecthomas/kong` are good alternatives; kong uses struct tags and will feel closest to picocli.

Embedding files with `go:embed`

Services often need files at run time: email templates, an OpenAPI spec, SQL migrations, a small admin UI. In Java these live on the classpath inside the JAR. Go's equivalent is the `//go:embed` directive, which copies files into the binary at build time.

JAVA

```java
// Java: read a template from the classpath
var resource = new ClassPathResource("templates/receipt.txt");
String text = resource.getContentAsString(StandardCharsets.UTF_8);
```

In Go, you declare a package-level variable and put the directive right above it:

GO embedded/main.go

```go
// templateFS holds every template. The path is relative to this file's
// directory, and the files become part of the binary at build time.
//
//go:embed templates/*.tmpl
var templateFS embed.FS

// staticFS holds the admin UI's static files.
//
//go:embed static
var staticFS embed.FS
```

An `embed.FS` is a read-only file system that implements the standard `fs.FS` interface, so the rest of the standard library already knows how to use it. Templates parse straight from it, and the HTTP file server can serve it:

GO embedded/main.go

```go
var receipts = template.Must(
    template.New("receipt.txt.tmpl").
        Funcs(template.FuncMap{"cents": formatCents}).
        ParseFS(templateFS, "templates/*.tmpl"),
)

// staticHandler serves the embedded files under /static/.
func staticHandler() http.Handler {
    sub, err := fs.Sub(staticFS, "static") // drop the "static/" prefix
    if err != nil {
        panic(err)
    }
    return http.StripPrefix("/static/", http.FileServerFS(sub))
}
```

Running the example renders a receipt from the embedded template, ending with `Total: $50.98`, and lists the embedded static files.

A few rules to know:

- The directive goes directly above a **package-level** variable of type `string`, `[]byte` or `embed.FS`. For `string` or `[]byte`, add `import _ "embed"`.
- Paths are relative to the package's folder and can't contain `..`, so you can't embed files from a parent directory.
- When you embed a directory, files starting with `.` or `_` are skipped. Use the `all:` prefix (`//go:embed all:static`) to include them.
- Changing an embedded file requires a rebuild, in the same way that changing a classpath resource requires a new JAR.

> **The Go way**
>
> Embedding gives you a single, self-contained binary: no "file not found" in production because a volume wasn't mounted. Orderly embeds its SQL migrations so the service can migrate its own database on start-up (Chapter 9).

Money and decimals

Money bugs are expensive and embarrassing. The root problem is that binary floating point can't represent most decimal fractions exactly:

GO money/main.go

```go
a, b := 0.1, 0.2
fmt.Println("float64 0.1 + 0.2 =", a+b)
```

OUTPUT

```
float64 0.1 + 0.2 = 0.30000000000000004
```

(We use variables on purpose. Go evaluates constant expressions like `0.1 + 0.2` exactly at compile time, which hides the problem.) Java developers know the answer already: never use `double` for money, use `BigDecimal`. Go's standard library has no decimal type with rounding modes (`math/big` offers big integers, rationals and binary floats), so you have two good options.

Approach	Use it for	Java cousin
`int64` cents	Prices, totals, balances: almost everything	`long` cents
`shopspring/decimal`	Tax rates, FX rates, interest, percentages	`BigDecimal`
`float64`	Never for money	`double`

Integer cents: the default

Orderly stores every amount as an `int64` number of cents (`PriceCents`). Addition and multiplication are exact and fast, and the value maps straight to a Postgres `bigint`. The one thing to be careful about is division, because you can't split a cent:

GO money/main.go

```go
// split divides an amount into n parts that add up exactly, giving the
// leftover cents to the first parts (1000 / 3 = 334 + 333 + 333).
func split(cents int64, n int) []int64 {
    parts := make([]int64, n)
    base, extra := cents/int64(n), cents%int64(n)
    for i := range parts {
        parts[i] = base
        if int64(i) < extra {
            parts[i]++
        }
    }
    return parts
}
```

`shopspring/decimal` for rates

When a calculation involves a rate with many decimal places, use `github.com/shopspring/decimal` (v1.6.0 at the time of writing, actively maintained). Its values are immutable, like `BigDecimal`: every operation returns a new value.

GO money/main.go

```go
// taxCents applies a percentage rate (like "19" for 19% VAT) to an
// amount in cents and rounds half-to-even, like RoundingMode.HALF_EVEN.
func taxCents(amountCents int64, ratePercent decimal.Decimal) int64 {
    tax := decimal.NewFromInt(amountCents).
        Mul(ratePercent).
        Div(decimal.NewFromInt(100)).
        RoundBank(0) // round to whole cents
    return tax.IntPart()
}
```

OUTPUT

```
3 x 12.99 = 38.97
1000 cents split 3 ways: [334 333 333]
19% VAT on 38.97 = 7.40
38.97 USD in EUR = 35.91
```

Here's the same tax calculation in Java, for comparison:

JAVA

```java
BigDecimal tax = BigDecimal.valueOf(amountCents)
        .multiply(new BigDecimal("19"))
        .divide(BigDecimal.valueOf(100))
        .setScale(0, RoundingMode.HALF_EVEN);
```

> 💡 **If you're unsure, keep int64 cents at the edges**
>
> Store, transfer and sum money as `int64` cents. Bring in `decimal` only inside a calculation that needs a rate, round once to cents at the end, and go back to `int64`. Always create decimals from strings or integers (`RequireFromString("0.19")`), never from a `float64`. If you store `numeric` columns, `decimal.Decimal` implements `sql.Scanner` and `driver.Valuer`, and pgx has a small adapter package, `jackc/pgx-shopspring-decimal`.

`RoundBank` rounds half-to-even (banker's rounding, the usual choice in finance); `Round` rounds half away from zero. Pick one per business rule and test it with values that end in exactly .5. For heavy decimal maths, `cockroachdb/apd` is another well-maintained choice.

Multi-tenancy

If Orderly is sold as SaaS, each customer company is a **tenant**, and the one rule that must never break is: a tenant only ever sees its own data. The Go pattern is to resolve the tenant once at the edge, put it in the request's `context.Context`, and have the data layer read it from there.

> ☕ **Coming from Java**
>
> Hibernate has `CurrentTenantIdentifierResolver` and `MultiTenantConnectionProvider`, usually fed from a `ThreadLocal` set by a filter. Go replaces the `ThreadLocal` with the context, which travels explicitly through every call and also works across goroutines.

The tenant in the context

Context values need a key. Using an unexported type as the key means no other package can collide with it or overwrite it:

GO tenant/main.go

```go
// tenantKey is an unexported type, so no other package can collide
// with (or overwrite) our context value.
type tenantKey struct{}

// WithTenant returns a copy of ctx that carries the tenant ID.
func WithTenant(ctx context.Context, tenantID string) context.Context {
    return context.WithValue(ctx, tenantKey{}, tenantID)
}

// TenantFrom reads the tenant ID back out of ctx.
func TenantFrom(ctx context.Context) (string, bool) {
    id, ok := ctx.Value(tenantKey{}).(string)
    return id, ok && id != ""
}
```

A middleware resolves the tenant and rejects requests without one:

GO tenant/main.go

```go
func RequireTenant(next http.Handler) http.Handler {
    return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
        // In production, read the tenant from the verified JWT claims
        // (Chapter 12), never from a header the client controls.
        id := r.Header.Get("X-Tenant-ID")
        if id == "" {
            http.Error(w, "missing tenant", http.StatusUnauthorized)
            return
        }
        next.ServeHTTP(w, r.WithContext(WithTenant(r.Context(), id)))
    })
}
```

A repository that fails closed

The repository reads the tenant from the context itself, so a handler can't forget to pass it. If there's no tenant, it returns an error instead of everything:

GO tenant/repo.go

```go
func (r *OrderRepo) List(ctx context.Context) ([]Order, error) {
    tenantID, ok := TenantFrom(ctx)
    if !ok {
        return nil, ErrNoTenant // fail closed: never return everything
    }
    var out []Order
    for _, o := range r.rows {
        if o.TenantID == tenantID {
            out = append(out, o)
        }
    }
    return out, nil
}
```

In Postgres, that loop becomes `WHERE tenant_id = $1` on every query. The example sends three in-process requests:

OUTPUT

```
"acme"   -> 200 [{o-1 acme PENDING} {o-3 acme CONFIRMED}]
"globex" -> 200 [{o-2 globex SHIPPED}]
""       -> 401 missing tenant
```

Choosing an isolation model

Model	How it works	Good for	Watch out for
Row-level	`tenant_id` column on every table	Many small tenants	One missed `WHERE` leaks data
Schema per tenant	One Postgres schema each	Tens to hundreds	Migrations run per schema

Model	How it works	Good for	Watch out for
Database per tenant	One database (or pool) each	Few large or regulated tenants	Connection counts, cost

If you're unsure, start with **row-level** tenancy and add Postgres **row-level security** (RLS) as a safety net, so the database itself refuses cross-tenant reads even if Go code forgets a filter:

SQL

```
ALTER TABLE orders ENABLE ROW LEVEL SECURITY;
CREATE POLICY tenant_isolation ON orders
    USING (tenant_id = current_setting('app.tenant_id')::uuid);
```

Inside each transaction, the repository sets the variable from the context with `SELECT set_config('app.tenant_id', $1, true)`; the final `true` limits it to the current transaction, which matters with connection pools. Table owners and superusers bypass RLS, so connect as a separate application role (or use `FORCE ROW LEVEL SECURITY`).

> ⚠️ **Gotcha: schema names can't be query parameters**
>
> With schema-per-tenant, you'll run `SET LOCAL search_path` inside a transaction. Identifiers like schema names can't be passed as `$1` parameters, so never build them from raw input. Validate the tenant against a known list, or quote it with `pgx.Identifier{name}.Sanitize()`.

Audit logging

Auditors and security teams want to know **who did what, to which resource, when, and with what outcome**. That's different from the access log in Chapter 13, and it's worth keeping them apart:

	Access log	Audit log
Purpose	Operations and debugging	Accountability and compliance
Contents	Every request	Security-relevant actions only
Retention	Days or weeks, can be sampled	Months or years, complete
Destination	Your log platform	A separate, append-only store

Spring developers often use Spring Data's `@CreatedBy` with `AuditorAware`, or Hibernate Envers. In Go we combine two simple pieces: a middleware that records every state-changing request automatically, and explicit events from the code where business decisions happen.

The auditor

GO audit/audit.go

```go
// Record fills in the actor and request ID from ctx, so callers only
// describe the action itself.
func (a *Auditor) Record(ctx context.Context, e Event) {
    a.log.LogAttrs(ctx, slog.LevelInfo, "audit",
        slog.String("actor", ActorFrom(ctx)),
        slog.String("request_id", RequestIDFrom(ctx)),
        slog.String("action", e.Action),
        slog.String("resource", e.Resource),
        slog.String("outcome", e.Outcome),
        slog.Any("details", e.Details),
    )
}
```

`Auditor` wraps its own `*slog.Logger`, so audit events can go to a different destination than ordinary logs. Its middleware captures the response status with a small `ResponseWriter` wrapper, skips plain `GET`s, and records everything else, including every 401 and 403. Handlers add domain events with business details:

GO audit/main.go

```go
audit.Record(r.Context(), Event{
    Action:   "order.cancel",
    Resource: "order/" + id,
    Outcome:  "success",
    Details:  map[string]any{"reason": r.FormValue("reason")},
})
```

One event from the example, pretty-printed:

JSON

```json
{
    "time": "2026-11-02T17:57:20.8671608Z",
    "level": "INFO",
    "msg": "audit",
    "actor": "ada@orderly.example",
    "request_id": "req-7f3a",
    "action": "order.cancel",
    "resource": "order/o-1001",
    "outcome": "success",
    "details": {
        "reason": "changed my mind"
    }
}
```

Notice the `Z` at the end of the timestamp. The example installs a tiny `ReplaceAttr` function, `UTCTime`, so audit timestamps are always UTC, whatever time zone the server is in.

> 💡 **Audit logging checklist**
>
> - Take the actor from verified authentication, never from request input.
> - Record denials and failures, not only successes.
> - Never log secrets, tokens or full card numbers; log IDs instead.
> - Write to an append-only destination (a separate table, topic or bucket) that application code can't edit or delete.
> - If an audit record must never be lost, write it in the same database transaction as the change, using the outbox pattern from Chapter 11.

Internationalisation

Orderly's emails and error messages need to speak the customer's language, with correct plurals and number formats. In Spring you'd use a `MessageSource` backed by `messages_de.properties` and an `AcceptHeaderLocaleResolver`. In Go, two libraries share the work:

- **`nicksnyder/go-i18n`** (v2) loads message files and picks the right plural form.
- **`golang.org/x/text`** (from the Go team) handles language matching and locale-aware number formatting.

Message files

Messages live in one file per language. go-i18n reads TOML, YAML or JSON; we use JSON with no extra dependency:

JSON i18n/locales/active.en.json

```
{
  "OrderPlaced": "Thanks {{.Name}}, your order {{.OrderID}} is confirmed.",
  "ItemsInCart": {
    "description": "Cart summary shown on the checkout page",
    "one": "You have {{.Count}} item in your cart.",
    "other": "You have {{.Count}} items in your cart."
  }
}
```

The placeholders are Go templates. Plural forms (`one`, `other`, and for some languages `few` and `many`) follow the Unicode CLDR rules, the same rules ICU uses in Java.

Loading and localising

The bundle is built once at start-up, from files embedded in the binary:

GO i18n/main.go

```go
//go:embed locales/*.json
var localeFS embed.FS

// newBundle loads every message file once, at start-up.
func newBundle() (*i18n.Bundle, error) {
    bundle := i18n.NewBundle(language.English) // the fallback language
    bundle.RegisterUnmarshalFunc("json", json.Unmarshal)
    for _, file := range []string{"en", "de", "fr"} {
        path := "locales/active." + file + ".json"
        if _, err := bundle.LoadMessageFileFS(localeFS, path); err != nil {
            return nil, fmt.Errorf("load %s: %w", path, err)
        }
    }
    return bundle, nil
}
```

For each request, create a `Localizer` from the `Accept-Language` header and ask for messages by ID:

GO i18n/main.go

```go
// cartSummary picks the right plural form for the language.
func cartSummary(loc *i18n.Localizer, count int) string {
    return loc.MustLocalize(&i18n.LocalizeConfig{
        MessageID:    "ItemsInCart",
        PluralCount:  count,
        TemplateData: map[string]any{"Count": count},
    })
}
```

Here's a slice of the output for English, French and an unsupported language:

OUTPUT

```
[en-GB]
  Thanks Ada, your order o-1001 is confirmed.
  You have 0 items in your cart.
  You have 1 item in your cart.
[fr]
  Merci Ada, votre commande o-1001 est confirmée.
  Vous avez 0 article dans votre panier.
  Vous avez 3 articles dans votre panier.
[ja]
  Thanks Ada, your order o-1001 is confirmed.
```

Two details are worth a second look. French treats zero as singular ("0 article"), and the plural rules handled that without any code. Japanese isn't supported, so the bundle fell back to English. `MustLocalize` panics on a missing message, which suits messages you ship and test; use `Localize` and handle the error when message IDs come from data.

Numbers

`golang.org/x/text/message` formats numbers the local way:

OUTPUT

```
en-US  1,234,567 units in stock
de     1.234.567 units in stock
fr     1 234 567 units in stock
hi     12,34,567 units in stock
```

> **Version note: pinning x/text and go-i18n**
>
> The `golang.org/x` libraries support the two newest Go releases. With Go 1.27 out, the newest `x/text` (v0.42.0 and later) needs Go 1.26, and go-i18n v2.7.0 follows it. To keep this book's examples on `go 1.25.0`, they pin `golang.org/x/text v0.41.0` and `go-i18n/v2 v2.6.1`. On Go 1.26 or newer, use the latest of both.

Plugin architectures

Sometimes you want parts of a system to be swappable without touching the core: notification channels, payment providers, tax engines. Java has `ServiceLoader` and, in bigger systems, OSGi. Go has three options, and the first one covers most needs.

Option 1: interface plus registry (the default)

Define a small interface for the plugin contract, and a registry that maps names from configuration to constructors:

GO plugins/notify/notify.go

```go
// Notifier is the plugin contract. Keep it small.
type Notifier interface {
    Notify(ctx context.Context, m Message) error
}

// Factory builds a Notifier from plain string settings.
type Factory func(cfg map[string]string) (Notifier, error)

// Register adds a factory. Registering a name twice is a programming
// error, so it panics, like database/sql.Register does.
func (r *Registry) Register(name string, f Factory) {
    r.mu.Lock()
    defer r.mu.Unlock()
    if _, dup := r.factories[name]; dup {
        panic("notify: Register called twice for " + name)
    }
    r.factories[name] = f
}
```

Each channel lives in its own package (`email`, `webhook`) and knows nothing about the others. `main` registers the ones this build supports, and configuration decides which are switched on:

GO plugins/main.go

```go
reg := notify.NewRegistry()
reg.Register("email", email.New)
reg.Register("webhook", webhook.New)
```

OUTPUT

```
[email] from=orders@orderly.example to=ada@example.com subject="Order confirmed"
[webhook] POST https://hooks.example.com/orderly subject="Order confirmed"
skip: notify: unknown channel "sms" (have [email webhook])
```

You'll also see a variation of this in the standard library: `database/sql` drivers register themselves from an `init()` function, which is why you import them with a blank identifier, as in `import _ "github.com/jackc/pgx/v5/stdlib"`. That works, but explicit registration in `main` is easier to follow and to test, so prefer it for your own code.

Option 2: hashicorp/go-plugin, for separate processes

When plugins come from other teams or vendors, you may want them as separate binaries that can crash without taking your service down. **hashicorp/go-plugin** (v1.8.0, actively maintained) runs each plugin as a child process and talks to it over gRPC on a local connection. It's the system behind Terraform providers, Vault and Packer.

```
+---------------------+   starts process   +----------------------+
|  orderly (host)     | ------------------> |  tax-plugin binary   |
|  calls TaxEngine    |   gRPC over local  |  implements          |
|  interface          | <================> |  TaxEngine           |
+---------------------+     connection     +----------------------+
  a plugin crash is a failed call, not a crashed host
```

The cost is a protobuf contract, process management and slower calls, so use it only when isolation or independent releases really matter.

Option 3: the standard `plugin` package, rarely

Go's standard library has a `plugin` package that loads shared libraries at run time. Most teams avoid it, because:

- It only works on Linux, FreeBSD and macOS, and needs cgo.
- The host and every plugin must be built with exactly the same Go version and the same versions of every shared dependency.
- A loaded plugin can never be unloaded.

> 💡 **If you're unsure, use an interface and a registry**
>
> Compile your plugins into the binary and pick them with configuration. Rebuilding a Go service takes seconds, so "plugins" that need a rebuild are rarely a real problem.

Documentation: doc comments and runnable examples

Go documentation is plain comments written directly above declarations. There are no `@param` tags and no HTML. Tools turn them into the pages you see on pkg.go.dev, in your editor's hover, and in the `go doc` command.

JAVA

```java
/**
 * Moves qty units from available to reserved.
 * @param qty number of units, must be positive
 * @throws InsufficientStockException if not enough units are available
 * @see Stock#release(int)
 */
public void reserve(int qty) { ... }
```

The Go version is written as sentences, starting with the name being documented:

GO inventory/inventory.go

```go
// Reserve moves qty units from Available to Reserved.
//
// It returns an error wrapping [ErrInsufficientStock] if fewer than qty
// units are available, and leaves the stock unchanged.
func (s *Stock) Reserve(qty int) error {
    // ...
}
```

The square brackets are **doc links** (Go 1.19+), Go's `{@link}`: `[ErrInsufficientStock]`, `[Stock.Release]`, or `[errors.Is]` for other packages. A package comment can also use `# Headings`, lists and indented code blocks.

Examples that are also tests

The best Go documentation contains **runnable examples**. An example is a function in a `_test.go` file whose name follows a convention, `ExampleStock_Reserve` for the `Reserve` method of `Stock`, and that ends with an `// Output:` comment:

GO inventory/example_test.go

```go
// Asking for more than is available fails, and the error can be
// checked with errors.Is.
func ExampleStock_Reserve_insufficient() {
    mugs := inventory.Stock{ProductID: "p-1", Available: 1}

    err := mugs.Reserve(3)
    fmt.Println(err)
    fmt.Println(errors.Is(err, inventory.ErrInsufficientStock))
    // Output:
    // reserve 3 of p-1 (available 1): insufficient stock
    // true
}
```

`go test` runs it and fails if the printed output doesn't match, so the documentation can never go stale. pkg.go.dev shows it next to `Reserve`, with a Run button for public packages. The suffix after the second underscore (`_insufficient`) lets you add several examples for one method; it must start with a lower-case letter.

Try `go doc ./inventory Stock.Reserve` in the example folder to see the comment rendered in your terminal, with the doc link turned into plain text.

Public modules appear on **pkg.go.dev** automatically once someone fetches them through the Go proxy, as long as they have a recognised open-source licence. For private code, `go doc -http` serves the same kind of HTML pages locally, and teams that want a shared internal site can run `pkgsite` themselves.

Private modules in companies

Your company's shared libraries live in private Git repositories, so the public proxy and checksum database can't see them. Go needs two things: to be told which modules are private, and Git credentials to fetch them.

Step 1: `GOPRIVATE`

TERMINAL

```
go env -w GOPRIVATE=dev.azure.com/acme,github.acme.com
```

Modules matching these patterns are fetched directly from Git and never checked against `sum.golang.org`, so private names don't leak to public services. (`GOPRIVATE` sets the defaults for `GONOPROXY` and `GONOSUMDB`; you rarely need those directly.) If your company runs a Go proxy in Artifactory or Athens, point `GOPROXY` at it instead and set `GONOSUMDB` for your private paths.

Step 2: Git authentication

Go runs `git` to download private modules, so Git's own authentication applies. On developer machines, a Git credential manager usually handles it. In CI, the common approaches are:

- **A `url.insteadOf` rewrite** that adds a token to HTTPS URLs, or switches them to SSH.
- **A `.netrc` file** with `machine`, `login` and `password` lines. Go reads `$HOME/.netrc`, or `%USERPROFILE%\_netrc` on Windows, for the HTTPS requests it makes itself, and many Git setups honour it too.

TERMINAL

```
# CI only: inject a token for every repo in the Azure DevOps org
git config --global \
  url."https://build:${AZURE_DEVOPS_TOKEN}@dev.azure.com/acme/".insteadOf \
  "https://dev.azure.com/acme/"
```

~/.netrc

```
machine github.acme.com
login ci-bot
password <personal-access-token>
```

> ⚠️ **Gotcha: tokens in plain text**
>
> Both approaches store a token in a plain file. Use short-lived tokens with read-only scope (Azure DevOps "Code (read)", for example), keep them in your CI secret store, and never commit `.gitconfig` or `.netrc` files. Microsoft recommends Microsoft Entra tokens over long-lived personal access tokens.

The Azure DevOps `.git` suffix

Azure DevOps needs one more step, and it surprises everyone. To find the repository for an unknown host, Go first sends an HTTPS request with `?go-get=1` and reads a `<meta>` tag from the response. For private Azure DevOps repositories this lookup is a well-known source of trouble: without credentials, Azure DevOps can't confirm that the repository exists, and `go get` typically fails with an `unrecognized import path` error.

The reliable fix, and the one Microsoft documents, is to put **`.git` after the repository name in the module path**. When a path element ends in `.git`, Go skips the lookup and talks to the repository with Git directly:

GO.MOD go.mod

```
module dev.azure.com/acme/orderly/_git/orderly-shared.git

go 1.25.0
```

GO

```
import "dev.azure.com/acme/orderly/_git/orderly-shared.git/money"
```

Yes, it looks odd, with both `_git` and `.git`. That's the form Microsoft's documentation uses for HTTPS. If your team uses SSH with an `insteadOf` rule for `git@ssh.dev.azure.com:v3/...`, the path drops `_git` (`dev.azure.com/acme/orderly/orderly-shared.git`). Pick one form per module and stick to it: the module path is part of the module's identity, so changing it later breaks every importer.

GitHub Enterprise and self-hosted GitLab usually answer the `?go-get=1` lookup correctly once credentials are in place. If yours doesn't, the same `.git` suffix trick works there too.

> **Version note: GOAUTH**
>
> Go 1.24 added `GOAUTH`, which controls how the `go` command authenticates its own HTTPS requests to proxies and `?go-get=1` lookups. The default, `netrc`, reads the `.netrc` file described above; `git <dir>` reuses Git's credential helper. See `go help goauth`.

Time zones and UTC in enterprise data

Chapter 3 introduced `time.Time` and `LoadLocation`. In enterprise data the hard part is not the API but the rules, so here they are:

- **Store and compare instants in UTC.** Use Postgres `timestamptz`, call `.UTC()` before you store, and log in UTC.
- **Convert to a time zone only at the edges**: for display, for parsing what a user typed, and for calendar questions.
- **Keep the user's zone as an IANA name** like `Europe/Berlin`, never as a fixed offset like `+01:00`, because offsets change with daylight saving time.
- **Do calendar maths in the user's zone** with `AddDate`, not `Add(24 * time.Hour)`.
- **Embed the zone database** with `import _ "time/tzdata"`, because minimal container images often don't have one.

"Orders placed today" depends on where you are

A report of "today's orders in Berlin" needs the UTC boundaries of the Berlin calendar day:

GO timezones/main.go

```go
// dayBounds returns the UTC start and end of the local calendar day
// that contains t. Use it for questions like "orders placed today in
// Berlin": WHERE created_at >= start AND created_at < end.
func dayBounds(t time.Time, loc *time.Location) (start, end time.Time) {
    y, m, d := t.In(loc).Date()
    startLocal := time.Date(y, m, d, 0, 0, 0, 0, loc)
    endLocal := startLocal.AddDate(0, 0, 1) // calendar day, not 24h
    return startLocal.UTC(), endLocal.UTC()
}
```

Let's run it for an order placed late in the evening UTC, on the night Europe switches to summer time:

OUTPUT

```
stored (UTC):   2026-03-28T23:30:00Z
shown (Berlin): 2026-03-29T00:30:00+01:00
shown (NY):     2026-03-28T19:30:00-04:00
Berlin day in UTC: 2026-03-28T23:00:00Z to 2026-03-29T22:00:00Z
Add(24h):       2026-03-30 01:00:00 +0200 CEST
AddDate(0,0,1): 2026-03-30 00:00:00 +0200 CEST
slot as UTC:    2026-11-02T14:00:00Z
```

There's a lot to learn in those seven lines:

- The order was placed on **28 March** in UTC and New York, but on **29 March** in Berlin.
- That Berlin day runs from 23:00 to 22:00 UTC: it's only **23 hours** long, because clocks jump forward an hour.
- `Add(24 * time.Hour)` from Berlin midnight lands at **01:00** the next day; `AddDate(0, 0, 1)` correctly lands at midnight.
- A New York user's "09:00" delivery slot, parsed with `time.ParseInLocation`, is stored as 14:00 UTC.

> **Coming from Java**
>
> The same rules apply in Java: `Instant` for storage, `ZonedDateTime` with a `ZoneId` for display and calendar logic, `plusDays` instead of adding 24 hours. Go puts both roles in one type, `time.Time`, so the discipline is up to you: normalise with `.UTC()` when data enters your system, and use `In(loc)` only on the way out.

Recap

- Use **cobra** for CLIs with subcommands; build the command tree in a function, use `RunE` and `cmd.OutOrStdout()`, and test commands in-process.
- `go:embed` puts templates, static files and migrations inside the binary as an `embed.FS`, Go's version of classpath resources.
- Keep money in `int64` **cents**; use `shopspring/decimal` for rates, round once, and never use `float64`.
- Carry the **tenant in the context**, set by middleware from verified claims; make repositories fail closed, and add Postgres RLS as a safety net.
- Keep **audit logs** separate from access logs: middleware for every change and denial, explicit events for business actions, UTC timestamps, append-only storage.
- Use **go-i18n** for translated messages with plurals and `x/text` for language matching and number formats.
- Prefer an **interface plus registry** for plugins; reach for hashicorp/go-plugin only when you need process isolation.
- Document with **doc comments, doc links and `Example` tests**; reach private modules with `GOPRIVATE`, Git credentials and, on Azure DevOps, a `.git` suffix in the module path.

Try it yourself

> 🏋️ **Exercise 1: A `products show` command**
>
> Add `orderlyctl products show SKU` that prints one product, or fails with a clear error for an unknown SKU. Support `-o json`. Add a test using the `run` helper. *Hint:* use `Args: cobra.ExactArgs(1)` and look the product up in `ListProducts`.

> 🏋️ **Exercise 2: Translate the receipt**
>
> Add a German message file for the receipt's title and "Total" label in `i18n/locales/`, and a `ReceiptTotal` message that formats the amount. Print an English and a German receipt. *Hint:* keep amounts as cents and format them yourself; pass the formatted string as `TemplateData`.

> 🏋️ **Exercise 3: Tenant-aware audit events**
>
> Combine the `tenant` and `audit` examples: make `Auditor.Record` include the tenant from the context in every event, and add a test proving that an event recorded under tenant `acme` says so. *Hint:* `TenantFrom(ctx)` already exists; add one more `slog.String` attribute.

> 🏋️ **Exercise 4 (stretch): Weekly report boundaries**
>
> Write `weekBounds(t time.Time, loc *time.Location) (start, end time.Time)` that returns the UTC boundaries of the Monday-to-Sunday week containing `t` in `loc`. Test it with a week that includes a daylight-saving change, and check that the week is 167 or 169 hours long. *Hint:* find Monday with `t.In(loc).Weekday()`, then use `AddDate(0, 0, 7)` for the end.

Further reading

- Cobra user guide: commands, flags, hooks and completion in detail.
- Package embed: the exact rules for `//go:embed` patterns.
- shopspring/decimal: API, rounding modes and database support.
- go-i18n: message files, plurals and the `goi18n` extraction tool.
- hashicorp/go-plugin: process-based plugins over gRPC, with examples.
- Go Doc Comments: the official guide to doc comments, links, lists and headings.
- Go Modules Reference: private modules: `GOPRIVATE`, proxies and passing credentials.
- Use Go with Azure Repos: Microsoft's guide to module paths and authentication.

CHAPTER 20

Capstone: Building Orderly End to End

> *Everything you've learned, working together in one real service you can run, read and change.*

🎯 **What you'll learn**

- How the pieces from earlier chapters fit into one production-shaped Go service
- How a "place order" request travels from the first middleware to a Kafka topic
- How `main.go` acts as the composition root, replacing Spring's container
- How to write a transaction that never oversells and never deadlocks
- How the transactional outbox, error mapping, auth and graceful shutdown look in real code
- How the same app maps onto Spring Boot, and what Spring was doing for you

The big picture

For nineteen chapters we've studied Go one idea at a time. That leaves a question hanging: what does it look like when it all comes together? This chapter answers it with **Orderly**, the order and inventory service from our examples, now built for real.

Orderly is small enough to read in an afternoon (about 3,000 lines of generously commented Go, plus 1,600 lines of tests) and complete enough to deploy. It has a REST API, a PostgreSQL database, Kafka events, authentication, metrics, tracing, health checks, a container image and a Helm chart. Every line in this chapter's code blocks is copied from it, and every test in it passes.

Full code: `capstone/` (module `example.com/orderly`). Its `README.md` has step-by-step run instructions.

🧩 **Think of it like a small, well-run shop**

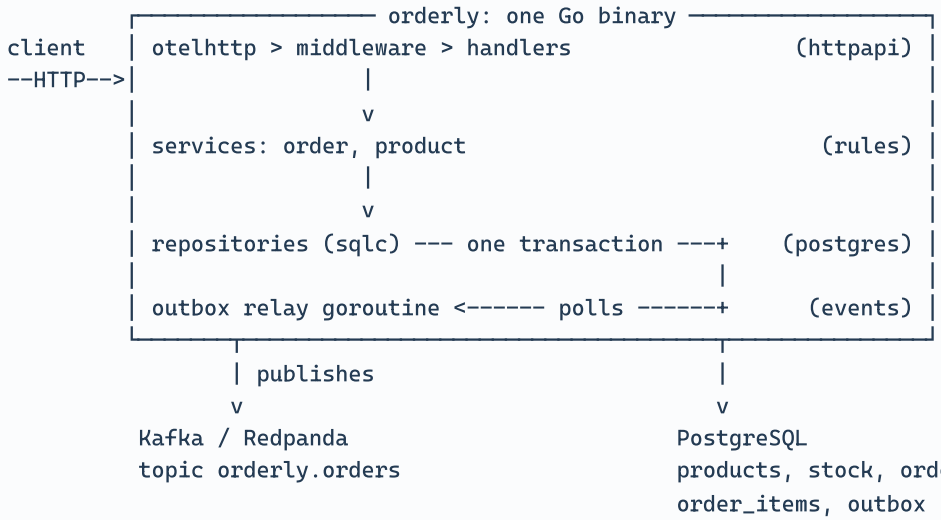
The front desk (the HTTP layer) greets customers and checks their ID. The manager (the services) applies the shop's rules. The stockroom ledger (PostgreSQL) is the single source of truth. And the outgoing mail tray (the outbox) holds letters until the postman (the relay) collects them. Each person has one job, and you can see who talks to whom.

Java ↔ Go at a glance

Concept	Java/Spring	Orderly in Go
Wiring	component scan, `@Autowired`	constructor calls in `main.go`
Web layer	`@RestController`	chi router and handler methods

Concept	Java/Spring	Orderly in Go
Transactions	`@Transactional` proxy	an `inTx` helper with a closure
Data access	JPA or jOOQ	pgx and sqlc-generated queries
Errors	`@ControllerAdvice`	one `writeError` and a mapping table
Background jobs	`@Scheduled`	a goroutine with a ticker
Config	`@ConfigurationProperties`	a struct filled by caarlos0/env

The architecture



One binary does everything: it serves HTTP, talks to PostgreSQL and runs a background goroutine that publishes events. There is no application server and no container framework, which is why the whole thing starts in well under a second.

The life of a "place order" request

Before we open any files, let's follow one request from start to finish. It's the same map you'd draw for a Spring app; only the names change.

```
POST /api/v1/orders  {"items":[{"product_id":"c43...","quantity":2}]}
 |
 ├─ otelhttp           start a trace span (or continue the caller's)
 ├─ requestID          X-Request-ID into the context and the response
 ├─ logRequests        one JSON access-log line when the request ends
 ├─ metrics            count and time it, labelled by route
 ├─ recoverPanics      a panic becomes a 500 problem, not a crash
 ├─ timeout            a 10-second deadline on the context
 ├─ auth.Middleware    verify the JWT, store the caller in the context
 |
 ├─ httpapi.placeOrder decode and validate JSON, build an order.Actor
 ├─ order.Service      merge repeated lines, new ID, status CONFIRMED
 ├─ postgres: BEGIN
 |    ├─ SELECT ... FOR UPDATE    lock stock rows, in product_id order
 |    ├─ Stock.Reserve            domain rule: never below zero
 |    ├─ UPDATE stock; INSERT orders and order_items
```

```
│    └─ INSERT outbox            OrderPlaced + StockReserved events
│  COMMIT
└─ 201 Created, Location: /api/v1/orders/52b1...

   ...moments later, in the relay goroutine...

events.Relay: BEGIN; SELECT ... FOR UPDATE SKIP LOCKED
              publish to Kafka; mark rows published; COMMIT
```

Notice how the work splits. The HTTP layer deals with HTTP: JSON, status codes and tokens. The service deals with rules. The repository deals with SQL and transactions. Kafka is touched by nobody on the request path, which is the whole point of the outbox, as we'll see.

A tour of the folders

Orderly follows the layout from Chapter 6: package by feature, with everything private under `internal/`.

```
capstone/
  cmd/orderly/main.go   the composition root: wiring and shutdown
  cmd/devtoken/         mints JWTs for local development
  internal/
    config/             env vars into a struct, validated at startup
    order/              Order, its rules and its service (the core)
    inventory/          Stock and the reserve/release rules
    product/            Product and its service
    postgres/           repositories; db/ is sqlc-generated code
    httpapi/            router, handlers, middleware, problem+json
    auth/               JWT verification and middleware
    events/             event types, outbox relay, publishers
    platform/           logging, tracing, health, metrics registry
  migrations/           goose SQL files, embedded with go:embed
  api/openapi.yaml      the API contract
  deploy/               Dockerfile, docker-compose.yaml, helm/orderly
  Makefile, sqlc.yaml, README.md
```

The most important property isn't visible in the tree: it's the direction of the imports.

```
cmd/orderly ──▶ httpapi ──▶ order, product, inventory, auth, platform
            ──▶ postgres ─▶ order, product, inventory, events
            ──▶ events ──▶ order

order, product and inventory import none of the packages above.
```

The domain packages know nothing about HTTP, SQL or Kafka, and everything else depends on them. This is hexagonal architecture without the ceremony: no `ports` and `adapters` folders, only a rule about which way the arrows point.

> **The Go way: names that read well from outside**
>
> Inside package `order` the types are `Order`, `Item` and `Status`, not `OrderItem` and `OrderStatus`. Callers write `order.Item` and `order.StatusConfirmed`, so repeating "order" would stutter. It feels odd at first after Java's `OrderItem` classes, and every Java developer trips on it, but it soon reads naturally.

main.go: the composition root

In Spring, `@SpringBootApplication` starts a container that scans your packages, creates beans and injects them into each other. You rarely see the wiring. In Go there is no container, so the wiring lives in one place you can read top to bottom: `main.go`.

A **composition root** is the single place where an application's object graph is assembled. It's the only code that knows every concrete type.

JAVA

```java
@SpringBootApplication
public class OrderlyApplication {
    public static void main(String[] args) {
        SpringApplication.run(OrderlyApplication.class, args);
    }
}
```

Here's the start of Orderly's `run` function. `main` only calls `run` and turns an error into exit code 1, so `run` can use plain `return err`.

GO cmd/orderly/main.go

```go
func run() error {
    // ctx is cancelled on Ctrl+C (SIGINT) or when Kubernetes sends SIGTERM.
    ctx, stop := signal.NotifyContext(context.Background(),
        os.Interrupt, syscall.SIGTERM)
    defer stop()

    // 1. Configuration and logging.
    cfg, err := config.Load()
    if err != nil {
        return err
    }
    logger := platform.NewLogger(os.Stdout, cfg.LogLevel).
        With("service", "orderly", "version", version)
    slog.SetDefault(logger)

    // 2. Telemetry.
    // ... platform.SetupTracing and platform.NewRegistry ...
```

Configuration comes first, because everything else needs it, and `config.Load` fails fast: a missing `ORDERLY_DB_URL` or a 10-character JWT secret stops the process before it opens a single connection. Logging and telemetry come next, so every later step can report what it's doing.

GO cmd/orderly/main.go

```
    // 3. Database, and migrations if asked for.
    pool, err := postgres.Connect(ctx, cfg.DB.URL, cfg.DB.MaxConns)
    if err != nil {
        return err
    }
    defer pool.Close() // safety net; the shutdown sequence closes it first
    if cfg.DB.MigrateOnStart {
        if err := postgres.Migrate(ctx, pool, logger); err != nil {
            return err
        }
    }

    // 4. Repositories, then services. Plain constructor calls: this is
    // all the dependency injection Orderly needs.
    orders := order.NewService(postgres.NewOrderRepository(pool))
    products := product.NewService(postgres.NewProductRepository(pool))
    stock := postgres.NewStockRepository(pool)
```

Three lines of wiring replace a component scan. If you're wondering where the interfaces are: `order.NewService` accepts an `order.Repository` interface, and `*postgres.OrderRepository` satisfies it without saying so. Go interfaces are satisfied implicitly, as Chapter 4 showed, so the concrete type and the interface never need to know about each other.

The HTTP server comes next, and its configuration is worth a close look because Spring Boot hides all of it behind Tomcat defaults:

GO cmd/orderly/main.go

```
    server := &http.Server{
        Addr: cfg.HTTP.Addr,
        // otelhttp starts a span per request (or continues the caller's
        // trace from the traceparent header).
        Handler: otelhttp.NewHandler(router, "http.server",
            otelhttp.WithFilter(func(r *http.Request) bool {
                return r.URL.Path != "/healthz" && r.URL.Path != "/readyz" &&
                    r.URL.Path != "/metrics"
            })),
        ReadHeaderTimeout: 5 * time.Second,  // slow-loris protection
        ReadTimeout:       10 * time.Second, // whole request, body included
        // Longer than the request timeout, so a handler that hits its
        // deadline still has time to send its 503 problem.
        WriteTimeout: cfg.HTTP.RequestTimeout + 5*time.Second,
        IdleTimeout:  60 * time.Second,
        ErrorLog:     slog.NewLogLogger(logger.Handler(), slog.LevelWarn),
    }
```

A zero-value `http.Server` has no timeouts at all, so a client sending one byte a minute can hold a connection forever. Chapter 7 explains each setting.

> 💡 **If you're unsure, keep main.go boring**
>
> A good `main.go` reads like a recipe: load config, build things in dependency order, start them, wait, stop them in reverse. Put logic in packages where it can be tested; keep only wiring here. If `main.go` grows past a few hundred lines, move groups of wiring into small helper functions, as Orderly does with `newPublisher` and `shutdown`.

The heart: the PlaceOrder transaction

Here's the business problem. Five blue mugs are in stock and two customers try to buy four each at the same moment. Both read "5 available", both think "fine", and both succeed. You've sold eight mugs you don't have. This is called **overselling**, and it's a classic race condition between database transactions.

> 🧩 **Think of it like a clipboard on each shelf**
>
> To count or change a shelf, you must hold its clipboard. Anyone else who wants that shelf waits until you hand it back. And everyone collects clipboards in aisle order, so two people never end up each holding one that the other needs.

The clipboard is a row lock. In PostgreSQL, `SELECT ... FOR UPDATE` reads rows and locks them until the transaction ends. Here's the sqlc query:

SQL internal/postgres/queries/orders.sql

```sql
-- name: LockStock :many
-- Locks the stock rows for the given products until the transaction ends.
-- ORDER BY product_id makes every transaction take its locks in the same
-- order, so two orders for the same products queue up instead of
-- deadlocking.
SELECT s.product_id, s.available, s.reserved, p.price_cents
FROM stock s
JOIN products p ON p.id = s.product_id
WHERE s.product_id = ANY(sqlc.arg(product_ids)::uuid[])
ORDER BY s.product_id
FOR UPDATE OF s;
```

A **deadlock** is two transactions each waiting for a lock the other holds, forever. Postgres detects it and kills one of them with an error. If customer A orders a mug and then a teapot while customer B orders a teapot and then a mug, they can deadlock. `ORDER BY s.product_id` means everyone locks in the same order, so it can't happen. `FOR UPDATE OF s` locks only the stock rows, not the products we joined for the price.

Now the repository method that uses it. Read the four numbered comments first; they're the whole story.

GO internal/postgres/orders.go

```go
func (r *OrderRepository) Create(ctx context.Context,
    o order.Order) (order.Order, error) {
    // We fill in prices below, so work on a copy of the caller's slice.
    o.Items = slices.Clone(o.Items)

    err := inTx(ctx, r.pool, func(q *db.Queries) error {
```

```
        // 1. Lock the stock rows. Other orders for these products now
        //    wait for us to commit or roll back.
        locked, err := lockStock(ctx, q, o.Items)
        if err != nil {
            return err
        }

        // 2. Check and reserve each line using the domain rule.
        for i, item := range o.Items {
            row, ok := locked[item.ProductID]
            if !ok {
                return fmt.Errorf("%w: %s", order.ErrUnknownProduct,
                    item.ProductID)
            }
            stock := toStock(row)
            if err := stock.Reserve(item.Quantity); err != nil {
                return err // inventory.ErrInsufficientStock: roll back
            }
            if err := saveStock(ctx, q, stock); err != nil {
                return err
            }
            o.Items[i].UnitPriceCents = row.PriceCents // today's price
        }
```

The `slices.Clone` line is a small kindness worth copying. A slice shares its backing array with the caller (Chapter 3), so writing prices into `o.Items` would quietly change the caller's data too. The price comes from the database inside the transaction, never from the client.

GO internal/postgres/orders.go

```
        // 3. Insert the order and its lines.
        if err := insertOrder(ctx, q, o); err != nil {
            return err
        }

        // 4. Record the events. They commit (or vanish) with the order.
        msgs, err := placedEvents(o)
        if err != nil {
            return err
        }
        return writeEvents(ctx, q, msgs...)
    })
    if err != nil {
        return order.Order{}, err
    }
    return o, nil
}
```

Where's the business rule? Not in the SQL. The repository asks the domain type to decide:

GO internal/inventory/inventory.go

```go
// Reserve moves qty units from Available to Reserved. It refuses to let
// Available go below zero, which is what prevents overselling.
func (s *Stock) Reserve(qty int) error {
    if qty <= 0 {
        return fmt.Errorf("reserve %d units: quantity must be positive", qty)
    }
    if s.Available < qty {
        return fmt.Errorf("%w: product %s has %d available, %d requested",
            ErrInsufficientStock, s.ProductID, s.Available, qty)
    }
    s.Available -= qty
    s.Reserved += qty
    return nil
}
```

So the repository owns the transaction and the domain owns the rule. `Reserve` is plain Go with no database, which is why its table-driven test runs in microseconds.

Transactions without @Transactional

In Spring you'd put `@Transactional` on a service method and a proxy would begin, commit or roll back around it. Go has no proxies, so Orderly spells it out once in a helper:

GO internal/postgres/postgres.go

```go
func inTx(ctx context.Context, pool *pgxpool.Pool,
    fn func(q *db.Queries) error) error {
    tx, err := pool.Begin(ctx)
    if err != nil {
        return fmt.Errorf("begin transaction: %w", err)
    }
    // Rollback after a successful Commit does nothing, so it is safe
    // (and simplest) to always defer it.
    defer tx.Rollback(ctx) //nolint:errcheck // the Commit error matters

    if err := fn(db.New(tx)); err != nil {
        return err
    }
    if err := tx.Commit(ctx); err != nil {
        return fmt.Errorf("commit: %w", err)
    }
    return nil
}
```

`db.New(tx)` gives the closure sqlc queries bound to the transaction. Any error returned from the closure, or a panic, triggers the deferred rollback. Nothing is hidden, and there's no equivalent of Spring's "self-invocation skips the proxy" surprise.

Coming from Java: the same lock in JPA

In Spring Data JPA you'd write a repository method with `@Lock(LockModeType.PESSIMISTIC_WRITE)` and an `order by` in the `@Query`. Hibernate turns that into the same `SELECT ... FOR UPDATE`. The SQL is identical; the difference is that in Go you can see it.

Proving it with 50 concurrent buyers

A claim like "never oversells" deserves a test. The integration test starts a real PostgreSQL with testcontainers (Chapter 15), puts 10 mugs in stock and sends 50 customers after them at once:

GO internal/postgres/postgres_integration_test.go

```go
func TestPlaceOrderNeverOversells(t *testing.T) {
    const stock, buyers = 10, 50
    mug := newProduct(t, stock)
    orders := order.NewService(postgres.NewOrderRepository(pool))

    var placed, soldOut atomic.Int32
    var placedIDs sync.Map // remembers the winners
    var wg sync.WaitGroup
    for range buyers {
        wg.Go(func() { // Go 1.25: Add, go and Done in one call
            o, err := orders.PlaceOrder(context.Background(), customer(),
                []order.ItemRequest{{ProductID: mug, Quantity: 1}})
            switch {
            case err == nil:
                placed.Add(1)
                placedIDs.Store(o.ID, true)
            case errors.Is(err, inventory.ErrInsufficientStock):
                soldOut.Add(1)
            default:
                t.Errorf("unexpected error: %v", err)
            }
        })
    }
    wg.Wait()

    assert.EqualValues(t, stock, placed.Load(), "orders placed")
    assert.EqualValues(t, buyers-stock, soldOut.Load(), "orders refused")
    // ...
}
```

A sibling test sends 40 buyers who list two products in opposite orders, the deadlock recipe, and checks that every one succeeds. Here's a real run:

OUTPUT

```
--- PASS: TestPlaceOrderNeverOversells (1.06s)
--- PASS: TestOppositeItemOrderDoesNotDeadlock (0.27s)
```

Exactly 10 winners and 40 polite refusals, every time. That's a small win worth celebrating: you've written correct concurrent database code and proved it.

The outbox relay

Placing an order must do two things: save the order and tell the world (billing, email, analytics) through Kafka. Doing both directly is called a **dual write**, and it can't be made safe. If you commit and then Kafka is down, the event is lost. If you publish first and the commit then fails, you've announced an order that doesn't exist.

The **transactional outbox** fixes it. Write the event into an `outbox` table in the same transaction as the order (step 4 above). A separate relay reads that table and publishes. The event exists if and only if the order does.

> 🧩 **Think of it like the outgoing mail tray**
>
> You write the letter at the same desk, in the same sitting, as you update the ledger, and drop it in the tray. The postman collects it later. If you're called away before he arrives, the letter is still safely in the tray.

The relay claims a batch of rows with `FOR UPDATE SKIP LOCKED`:

SQL internal/postgres/queries/outbox.sql

```sql
-- name: ClaimOutbox :many
-- SKIP LOCKED lets several relay replicas run at once: each one takes
-- the rows nobody else has locked, instead of waiting for them.
SELECT id, topic, key, event_type, payload
FROM outbox
WHERE published_at IS NULL
ORDER BY id
LIMIT sqlc.arg(batch_size)
FOR UPDATE SKIP LOCKED;
```

With three Orderly pods running, all three relays poll the same table, and `SKIP LOCKED` hands each one different rows. No leader election and no coordination service, only a database feature.

`OutboxStore.PublishPending` (in `internal/postgres/outbox.go`) runs claim, publish and mark inside one `inTx` call. If publishing fails, the transaction rolls back, the rows stay unpublished, and the next poll retries them. The relay itself is a goroutine with a ticker, the Go version of a `@Scheduled` method:

GO internal/events/relay.go

```go
func (r *Relay) Run(ctx context.Context) {
    // ... log "outbox relay started" ...
    ticker := time.NewTicker(r.cfg.PollInterval)
    defer ticker.Stop()

    for {
        r.drain(ctx)
        select {
        case <-ctx.Done():
            r.log.Info("outbox relay stopped")
            return
        case <-ticker.C:
        }
    }
}
```

`drain` keeps publishing full batches until the outbox is empty, then `select` waits for the next tick or for shutdown. One detail in `publishBatch` deserves a spotlight:

GO internal/events/relay.go

```go
func (r *Relay) publishBatch(ctx context.Context) (int, error) {
    // WithoutCancel keeps ctx's values but not its cancellation, so a
    // shutdown can't cut a batch in half. The timeout still bounds it.
    ctx, cancel := context.WithTimeout(context.WithoutCancel(ctx),
        r.cfg.BatchTimeout)
    defer cancel()

    n, err := r.store.PublishPending(ctx, r.cfg.BatchSize,
        r.publisher.Publish)
    r.published.Add(float64(n))
    return n, err
}
```

`context.WithoutCancel` (Go 1.21) lets the current batch finish when you press "stop", instead of rolling it back mid-flight. And `r.publisher.Publish` is passed as a function value, so the store never learns about Kafka.

Here's what the log publisher prints when you run Orderly without a broker (one log line, wrapped to fit the page):

OUTPUT

```
{"time":"2026-11-03T00:04:09.7788+05:30","level":"INFO",
 "msg":"event published","service":"orderly","version":"1.0.0",
 "publisher":"log","event_id":1,"topic":"orderly.orders",
 "key":"52b190a6-6850-4ed2-81c6-b92f467f08aa","event_type":"OrderPlaced",
 "payload":{"items":[{"quantity":2,
   "product_id":"c4369211-bb29-4bf2-8c16-b464ef1559bf",
   "unit_price_cents":1299}],
   "order_id":"52b190a6-6850-4ed2-81c6-b92f467f08aa",
   "customer_id":"0b6e9f4a-1c3d-4e5f-8a7b-9c0d1e2f3a4b",
   "total_cents":2598}}
```

The payload's keys come back in a different order from the Go struct because `jsonb` stores JSON in its own normalised form. JSON consumers don't care about key order, but it surprises people the first time.

> ⚠️ **Gotcha: at least once means duplicates**
>
> If the relay publishes a batch and the process dies before the commit, the rows are still unpublished, so the batch goes out again. The outbox guarantees **at-least-once** delivery, never exactly-once. Consumers must be idempotent: Orderly sends each row's ID in an `event-id` header so they can drop repeats. Also, with several relays running, two batches can interleave, so order is only reliable per partition key (the order ID).

Turning errors into problem+json

Spring 6 gave you `ProblemDetail` and `@ControllerAdvice`, so one class turns exceptions into RFC 9457 responses. Orderly does the same with a table and one function. Domain packages return plain errors such as `inventory.ErrInsufficientStock`, wrapped with context by whoever adds some.

GO internal/httpapi/problem.go

```go
var errorMappings = []struct {
    target error
    status int
    slug   string
    title  string
    detail string // optional fixed detail
}{
    {order.ErrOrderNotFound, 404, "order-not-found", "Order not found", ""},
    {inventory.ErrStockNotFound, 404, "product-not-found",
        "Product not found", ""},
    {inventory.ErrInsufficientStock, 409, "insufficient-stock",
        "Insufficient stock", ""},
    {order.ErrCannotCancel, 409, "order-not-cancellable",
        "Order cannot be cancelled", ""},
    // ... duplicate SKU, unknown product, invalid order, bad cursor ...
    {auth.ErrUnauthenticated, 401, "unauthenticated", "Unauthenticated",
        "A valid bearer token is required."},
    {auth.ErrForbidden, 403, "forbidden", "Forbidden",
```

```
        "You don't have permission to do this."},
    {context.DeadlineExceeded, 503, "timeout", "Request timed out",
        "The request took too long. Please try again."},
}
```

`writeError` walks this table with `errors.Is`, which looks through every layer of `%w` wrapping, and writes the first match as a problem. An error that matches nothing is a bug or an outage: it's logged in full with `ErrorContext` (so the line carries the request and trace IDs), and the client gets a generic 500 that reveals nothing internal.

Ordering four mugs when three are left produces this response, straight from the running service (wrapped, with the product ID shortened):

JSON

```
{"type":"https://orderly.example.com/problems/insufficient-stock",
 "title":"Insufficient stock","status":409,
 "detail":"insufficient stock: product c4369211-... has 3 available, 4 requested",
 "instance":"/api/v1/orders",
 "request_id":"75a07fc5-0003-4040-a431-c4f84f050b86"}
```

The `request_id` member matters more than it looks. When a customer reports an error, that ID finds the exact log lines in seconds.

Authentication, and "only your own orders"

Spring Security gives you a filter chain and annotations such as `@PreAuthorize("hasRole('ADMIN')")`. Orderly uses two small middlewares and one rule in the service. The token check pins the algorithm, which closes the famous `"alg": "none"` hole (Chapter 12):

GO internal/auth/auth.go

```
func (a *Authenticator) Verify(token string) (Principal, error) {
    var c claims
    _, err := jwt.ParseWithClaims(token, &c,
        func(*jwt.Token) (any, error) { return a.secret, nil },
        // Never let the token pick its own algorithm ("alg": "none"...).
        jwt.WithValidMethods([]string{jwt.SigningMethodHS256.Alg()}),
        jwt.WithIssuer(a.issuer),
        jwt.WithExpirationRequired(),
        jwt.WithLeeway(30*time.Second), // tolerate small clock drift
    )
    if err != nil {
        return Principal{}, fmt.Errorf("%w: %w", ErrUnauthenticated, err)
    }
    // ... the subject must be a UUID and the role must be known ...
    return Principal{Subject: sub, Role: c.Role}, nil
}
```

HS256 with a shared secret keeps local development easy. In production you'd verify an identity provider's public keys instead, with `coreos/go-oidc`, and only `Verify` would change.

The router shows the whole security setup in one screen. Compare that with hunting through a `SecurityFilterChain` bean and annotations spread over controllers:

GO internal/httpapi/router.go

```
    r.Route("/api/v1", func(r chi.Router) {
        // Order matters: the outer middleware sees what the inner ones
        // did, so logging and metrics also see the 500 from a panic.
        r.Use(
            logRequests(d.Logger),
            metrics.middleware,
            a.recoverPanics,
            timeout(d.RequestTimeout),
            d.Auth.Middleware(a.writeError),
        )

        r.Get("/products", a.listProducts)
        r.With(auth.RequireRole(auth.RoleAdmin, a.writeError)).
            Post("/products", a.createProduct)
        r.Get("/inventory/{productId}", a.getStock)

        r.Post("/orders", a.placeOrder)
        r.Get("/orders", a.listOrders)
        r.Get("/orders/{id}", a.getOrder)
        r.Post("/orders/{id}/cancel", a.cancelOrder)
    })
```

Notice `d.Auth.Middleware(a.writeError)`. The auth package needs to write an error response but shouldn't know about problem+json, so the HTTP layer passes in its own error writer. That one function parameter keeps the dependency arrows pointing the right way.

"Customers see only their own orders" is a business rule, so it lives in the service. The handler converts the token's principal into an `order.Actor`, and the service checks it:

GO internal/order/service.go

```
func (s *Service) GetOrder(ctx context.Context, actor Actor,
    id uuid.UUID) (Order, error) {
    o, err := s.repo.Get(ctx, id)
    if err != nil {
        return Order{}, err
    }
    if !actor.canSee(o) {
        // Say "not found" rather than "forbidden", so nobody can probe
        // for other customers' order IDs.
        return Order{}, ErrOrderNotFound
    }
    return o, nil
}
```

`ListOrders` applies the same rule by forcing the customer filter for non-admins. Because the rule lives in the service, it holds whether the request arrived over REST today or over gRPC next year.

Graceful shutdown, in the right order

When Kubernetes replaces a pod, it sends SIGTERM and waits, by default 30 seconds, before killing it. In that window a well-behaved service finishes what it started. The order of the steps is the whole trick.

> 🧩 **Think of it like closing a restaurant**
>
> Flip the sign to "closed" (readiness off). Serve the diners already seated (finish HTTP requests). Let the kitchen hand over the last deliveries (stop the relay). File the day's receipts (flush telemetry). Then lock the pantry (close the pool). Lock the pantry first and the kitchen can't finish.

GO cmd/orderly/main.go

```go
func shutdown(logger *slog.Logger, cfg config.HTTP, s shutdownSteps) error {
    ctx, cancel := context.WithTimeout(context.Background(),
        cfg.ShutdownTimeout)
    defer cancel()
    var errs []error

    // 1. Readiness off: load balancers stop sending new requests.
    s.health.SetReady(false)
    logger.Info("readiness off, draining",
        "drain_delay", cfg.DrainDelay.String())
    time.Sleep(cfg.DrainDelay)

    // 2. Stop accepting connections and finish in-flight requests.
    if err := s.server.Shutdown(ctx); err != nil {
        errs = append(errs, fmt.Errorf("http shutdown: %w", err))
    }
    logger.Info("http server stopped")

    // 3. Stop the relay. It finishes its current batch first.
    s.stopRelay()
    select {
    case <-s.relayDone:
    case <-ctx.Done():
        errs = append(errs, errors.New("outbox relay did not stop in time"))
    }
    s.closePublisher()
```

The relay gets its own context (`relayCtx`, created from `context.Background()` in `run`), not the signal context. If it shared the signal context, it would stop the instant SIGTERM arrived, while HTTP requests were still writing outbox rows. Stopping it explicitly, after the server, keeps the order right.

GO cmd/orderly/main.go

```
    // 4. Flush buffered spans to the collector.
    if err := s.shutdownTracing(ctx); err != nil {
        errs = append(errs, fmt.Errorf("tracing shutdown: %w", err))
    }

    // 5. Nothing uses the database any more: close the pool.
    s.closePool()

    logger.Info("shutdown complete")
    return errors.Join(errs...)
}
```

`errors.Join` collects every problem instead of stopping at the first, so one slow step doesn't hide another. Here's the real log from pressing Ctrl+C, with the repeated fields trimmed. All five lines carry timestamps within the same two milliseconds:

OUTPUT

```
{"time":"2026-11-03T00:15:44.0239057+05:30","level":"INFO",
 "msg":"shutdown signal received",...}
{...,"msg":"readiness off, draining",...,"drain_delay":"0s"}
{...,"msg":"http server stopped",...}
{...,"msg":"outbox relay stopped",...}
{"time":"2026-11-03T00:15:44.0256887+05:30","level":"INFO",
 "msg":"shutdown complete","service":"orderly","version":"1.0.0"}
```

On Kubernetes the Helm chart adds a `preStop` sleep of 5 seconds, so endpoint removal spreads through the cluster before SIGTERM arrives, and sets `terminationGracePeriodSeconds: 30`, comfortably more than the 5-second sleep plus the 20-second shutdown timeout. Chapter 18 explains these settings.

Run it yourself

Time to see it work. You need Go 1.25 or newer and Docker. From the `capstone` folder, one command starts PostgreSQL, Redpanda (a Kafka-compatible broker) and Orderly:

TERMINAL

```
docker compose -f deploy/docker-compose.yaml up --build
```

The compose file maps PostgreSQL to host port 5433, so it can't clash with a database you already run. Orderly migrates its schema at startup. Here's the start of the log from one of my runs (with `ORDERLY_HTTP_ADDR` set to a custom address):

OUTPUT

```
{"time":"2026-11-03T00:15:43.1425125+05:30","level":"INFO",
 "msg":"migration applied","service":"orderly","version":"1.0.0",
 "migration":1,"file":"00001_create_tables.sql","took":"37.0598ms"}
{...,"msg":"publishing events to the log (no broker configured)",...}
{...,"msg":"http server listening",...,"addr":"127.0.0.1:18080"}
{...,"msg":"outbox relay started",...,"poll_interval":"500ms",...}
```

Get a token

Every `/api/v1` call needs a JWT. The little `cmd/devtoken` program signs one with the same development secret the compose file gives the service:

TERMINAL

```
export ORDERLY_JWT_SECRET=local-dev-secret-change-me-0123456789
ADMIN=$(go run ./cmd/devtoken -role admin)
ALICE=$(go run ./cmd/devtoken -sub 0b6e9f4a-1c3d-4e5f-8a7b-9c0d1e2f3a4b)
BOB=$(go run ./cmd/devtoken)   # a random customer
```

Call the API

Create a product as the admin, with five in stock:

TERMINAL

```
curl -s -X POST localhost:8080/api/v1/products \
  -H "Authorization: Bearer $ADMIN" -H 'Content-Type: application/json' \
  -d '{"sku":"mug-blue-01","name":"Blue mug","price_cents":1299,
       "initial_stock":5}'
```

OUTPUT

```
{"id":"c4369211-bb29-4bf2-8c16-b464ef1559bf","sku":"MUG-BLUE-01",
 "name":"Blue mug","price_cents":1299,
 "created_at":"2026-11-02T18:34:08.778599Z"}
```

Now place an order as Alice (put the product ID in `$PRODUCT` first):

TERMINAL

```
curl -s -i -X POST localhost:8080/api/v1/orders \
  -H "Authorization: Bearer $ALICE" -H 'Content-Type: application/json' \
  -d "{\"items\":[{\"product_id\":\"$PRODUCT\",\"quantity\":2}]}"
```

OUTPUT

```
HTTP/1.1 201 Created
Content-Type: application/json
Location: /api/v1/orders/52b190a6-6850-4ed2-81c6-b92f467f08aa
X-Request-Id: 443ce561-1710-40ae-ba55-f274c455dc3b

{"id":"52b190a6-6850-4ed2-81c6-b92f467f08aa",
 "customer_id":"0b6e9f4a-1c3d-4e5f-8a7b-9c0d1e2f3a4b",
 "status":"CONFIRMED",
 "items":[{"product_id":"c4369211-bb29-4bf2-8c16-b464ef1559bf",
           "quantity":2,"unit_price_cents":1299}],
 "total_cents":2598,"created_at":"2026-11-02T18:34:09.494907Z"}
```

Then explore the rest of the API:

- `GET /api/v1/inventory/$PRODUCT` now shows `"available":3,"reserved":2`.
- Bob asking for Alice's order gets a 404 `order-not-found` problem.
- `POST /api/v1/orders/{id}/cancel` with `{"reason":"changed my mind"}` returns the order as `CANCELLED`, puts the stock back to 5, and publishes `OrderCancelled`.
- Cancelling it again gives a 409: `"order cannot be cancelled: it is CANCELLED"`.

To watch the events land in Redpanda, run `docker compose -f deploy/docker-compose.yaml exec redpanda rpk topic consume orderly.orders`.

> 💡 **Trace IDs without a tracing backend**
>
> Send `-H "traceparent: 00-4bf92f3577b34da6a3ce929d0e0e4736-00f067aa0ba902b7-01"` with any request. Even with trace export switched off, the access-log line for that request carries `"trace_id":"4bf92f3577b34da6a3ce929d0e0e4736"`, because otelhttp still propagates the incoming context and the log handler copies it in.

Run the tests

TERMINAL

```
make test               # unit tests, no Docker: go test ./...
make test-integration   # real PostgreSQL in Docker (testcontainers)
make lint               # gofmt, go vet with and without the tag
```

The unit tests cover the domain rules, the relay (with `testing/synctest` driving its ticker on a fake clock) and every handler through `httptest`. The handler tests use fakes with function fields, so each test plugs in only the behaviour it needs:

GO internal/httpapi/router_test.go

```go
type fakeOrders struct {
    place  func(order.Actor, []order.ItemRequest) (order.Order, error)
    get    func(order.Actor, uuid.UUID) (order.Order, error)
    list   func(order.Actor, order.ListParams) (order.Page, error)
    cancel func(order.Actor, uuid.UUID, string) (order.Order, error)
}
```

No mocking library, no generated code: a struct and four methods. Chapter 15 explains why Go teams usually prefer this.

> **Version note: Go 1.25 and govulncheck**
>
> The capstone pins `go 1.25.0`, the book's baseline, so each dependency is the newest release that still supports Go 1.25. Go supports only the two newest releases, so by late 2026 some fixes need Go 1.26: `govulncheck ./...` reports issues in `golang.org/x/net` whose fix requires `go 1.26.0`. In a real project you'd raise the `go` line and run `go get -u ./...`. This is exactly the routine upkeep that a govulncheck step in CI (Chapter 18) is there to prompt.

Where each chapter shows up

Here's a revision map: every earlier chapter, and where its ideas live in Orderly.

Ch	Concept	Where in Orderly
1	Explicit over magic	`main.go` wiring, no framework
2	Modules, `go build`, Makefile	`go.mod`, `Makefile`
3	Structs, slices, maps	`order.Order`, `slices.Clone`, `lockStock`
4	Interfaces, `%w`, generics	`httpapi.OrderService`, `errorMappings`, `env.ParseAs`
5	Goroutines, context, select	`Relay.Run`, `signal.NotifyContext`, `WithoutCancel`
6	Package by feature	`internal/` layout, import direction
7	chi, validation, problem+json	`httpapi` package
8	gRPC with Connect	an exercise below
9	pgx, sqlc, goose, transactions	`postgres`, `migrations`, `inTx`
10	Env config, fail fast	`config.Load`, `Validate` with `errors.Join`
11	Outbox, franz-go	`events`, `ClaimOutbox`
12	JWT, least privilege	`auth`, 404 for others' orders, non-root image
13	slog, Prometheus, OTel	`platform`, `httpMetrics`, otelhttp
14	Timeouts, shutdown	`http.Server`, `timeout`, `shutdown`
15	Fakes, httptest, testcontainers	`*_test.go` files
16	gofmt, vet, staticcheck	`make lint`, `sqlc generate`
17	GOMEMLIMIT, no N+1 queries	Helm chart, `OrderRepository.List`
18	Distroless, Helm, probes	`deploy/`
19	`go:embed`, money as cents	`migrations.FS`, `PriceCents int64`

The same app in Spring Boot

Your Spring Boot knowledge is an asset. Let's line the two versions up so you can translate in both directions.

Package structure

```
Spring Boot (a typical layout)          Orderly in Go
com.example.orderly                     cmd/orderly/main.go
  OrderlyApplication.java               internal/order/
  order/OrderController.java              order.go  service.go
  order/OrderService.java                 cursor.go
  order/OrderRepository.java            internal/inventory/
  order/Order.java  OrderItem.java      internal/product/
  inventory/Stock.java ...              internal/postgres/  (+ db/)
  config/SecurityConfig.java            internal/httpapi/
  outbox/OutboxPoller.java              internal/auth/
  web/GlobalExceptionHandler.java       internal/events/
src/main/resources/                     internal/platform/
  application.yml                       internal/config/
  db/migration/V1__init.sql             migrations/00001_create_tables.sql
```

The shapes are closer than you might expect. The big difference is where the edges are: in Spring, the controller, repository and entity for "order" often sit together; in Orderly, HTTP and SQL live in their own packages so the `order` package stays free of both.

Annotations and their Go equivalents

Spring	Orderly
`@SpringBootApplication`	`main.go` calling constructors
`@RestController`, `@PostMapping`	`r.Post("/orders", a.placeOrder)`
`@Valid` and Bean Validation	`validate:"..."` tags and `a.decode`
`@ControllerAdvice`, `ProblemDetail`	`errorMappings` and `writeError`
`@Transactional`	`inTx(ctx, pool, func(q) error {...})`
`@Lock(PESSIMISTIC_WRITE)`	`FOR UPDATE` in the `LockStock` query
`@PreAuthorize("hasRole('ADMIN')")`	`auth.RequireRole(auth.RoleAdmin, ...)`
`@ConfigurationProperties`	`config.Config` with `env` tags
`@Scheduled` poller	`Relay.Run` with a `time.Ticker`
`server.shutdown=graceful`	the `shutdown` function

Here's the heart of `PlaceOrder` in Spring, for comparison:

JAVA

```java
public interface StockRepository extends JpaRepository<Stock, UUID> {
    @Lock(LockModeType.PESSIMISTIC_WRITE)
    @Query("select s from Stock s where s.productId in :ids"
         + " order by s.productId")
    List<Stock> lockAll(@Param("ids") Collection<UUID> ids);
```

```
}

@Service
public class OrderService {
    // constructor injection of the repositories omitted

    @Transactional
    public Order placeOrder(UUID customerId, List<ItemRequest> items) {
        var ids = items.stream().map(ItemRequest::productId).toList();
        var stock = stockRepository.lockAll(ids);
        // reserve each line, save the order, save an OutboxEvent entity
        // ...
    }
}
```

Dependencies: pom.xml vs go.mod

A Spring Boot version of Orderly would start from a list like this:

XML pom.xml (excerpt)

```
<parent>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-parent</artifactId>
  <version>3.5.0</version>
</parent>
<dependencies>
  <dependency><groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId></dependency>
  <dependency><groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-data-jpa</artifactId></dependency>
  <dependency><groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-oauth2-resource-server</artifactId>
  </dependency>
  <dependency><groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-actuator</artifactId></dependency>
  <dependency><groupId>org.springframework.kafka</groupId>
    <artifactId>spring-kafka</artifactId></dependency>
  <!-- plus validation, Flyway, Micrometer, the PostgreSQL driver... -->
</dependencies>
```

And here are Orderly's direct dependencies, each one a library you chose by name:

GO.MOD go.mod (excerpt)

```
module example.com/orderly

go 1.25.0

require (
    github.com/caarlos0/env/v11 v11.4.1
    github.com/go-chi/chi/v5 v5.5.0
    github.com/go-playground/validator/v10 v10.30.4
    github.com/golang-jwt/jwt/v5 v5.3.1
    github.com/google/uuid v1.6.0
    github.com/jackc/pgx/v5 v5.12.0
    github.com/pressly/goose/v3 v3.27.0
    github.com/prometheus/client_golang v1.24.1
    github.com/twmb/franz-go v1.21.7
```

```
    go.opentelemetry.io/contrib/instrumentation/net/http/otelhttp v0.71.0
    go.opentelemetry.io/otel/sdk v1.46.0
    // ... plus testify and testcontainers-go for the tests
)
```

A starter pulls in dozens of libraries and auto-configures them. A Go `require` line pulls in one library that does nothing until you call it.

What Spring did for you that Go makes explicit

This is the real lesson of the capstone. None of these things disappeared; they moved from the framework into code you can read.

- **Creating and wiring objects.** Component scanning and constructor injection became about twenty lines in `main.go`.
- **Transactions.** The `@Transactional` proxy became `inTx`, a short function with a deferred rollback.
- **Exception handling.** `@ControllerAdvice` became `writeError` and a table you can test.
- **The web server.** Tomcat's defaults became an `http.Server` with every timeout written down.
- **Lifecycle.** `SmartLifecycle` phases became the five numbered steps in `shutdown`.
- **Migrations.** Flyway running on startup became `postgres.Migrate`, behind a flag, with an advisory lock so replicas don't race.
- **Observability.** Actuator and Micrometer auto-configuration became the `platform` package and two middlewares.
- **Security.** The filter chain became two middlewares you can see in `router.go`.

Explicit code costs more lines up front. In return, when something goes wrong at 3 a.m., there's no layer of magic between you and the cause. Most Java developers find that trade feels uncomfortable for a few weeks and then hard to give up.

Recap

- Orderly is one Go binary: HTTP handlers, services, PostgreSQL repositories and an outbox relay goroutine, wired together in `main.go`.
- Domain packages (`order`, `inventory`, `product`) import nothing from HTTP, SQL or Kafka; adapters depend on them, never the reverse.
- `SELECT ... FOR UPDATE` with `ORDER BY product_id` prevents overselling and deadlocks; the domain's `Reserve` method holds the rule.
- The transactional outbox writes events in the same transaction as the order; a relay using `FOR UPDATE SKIP LOCKED` publishes them at least once.
- One `writeError` function and a mapping table turn every domain error into RFC 9457 problem+json, and never leak internals.
- Authorization rules such as "only your own orders" live in the service, so they hold for every transport.
- Shutdown runs in order: readiness off, finish HTTP, stop the relay, flush telemetry, close the pool.
- Everything Spring did implicitly still happens; in Go it's code you can read, test and change.

Try it yourself

🏋 Exercise 1: Ship an order

Add `POST /api/v1/orders/{id}/ship` for admins only. Add an `Order.Ship()` domain method (only `CONFIRMED` orders can ship), a repository method that moves the units out of `Reserved`, and an `OrderShipped` event in the outbox. Write the unit test for `Ship()` first.

Hint: copy the shape of `Cancel` in `internal/postgres/orders.go`; the locking order is the same.

🏋 Exercise 2: Cache the product list in Redis

Product listings are read far more often than they change. Write a `CachedProductRepository` that wraps any `product.Repository`, caches `List` results in Redis with `redis/go-redis/v9` for 30 seconds, and clears the cache in `Create`.

Hint: because it implements the same interface, wiring it in is a one-line change in `main.go`. Chapter 9 covers the cache-aside pattern.

🏋 Exercise 3: Add a gRPC API with Connect

Define an `orderly.v1.OrderService` with `PlaceOrder` and `GetOrder` in a `.proto` file, generate code with `buf`, and implement the handler by calling the same `*order.Service`. Mount it on the existing chi router next to the REST API.

Hint: Chapter 8 shows Connect. Build the `order.Actor` from the same JWT, and notice that the ownership rule works without any change.

🏋 Exercise 4 (stretch): Payments with a saga

Make payment part of the flow. A new payments service consumes `OrderPlaced`, charges the customer and publishes `PaymentSucceeded` or `PaymentFailed`. Orderly consumes those events: success confirms the order, failure cancels it and releases the stock. That cancel is the saga's **compensating action**, the step that undoes earlier work when a later step fails.

Hint: orders now start as `PENDING`. Run a franz-go consumer group in its own goroutine, started and stopped in `main.go` like the relay, and use the `event-id` header to ignore duplicates. Chapter 11 covers consumers and sagas.

Further reading

- Organizing a Go module: the official guide to layouts like Orderly's `cmd/` and `internal/`.
- sqlc documentation: every option in `sqlc.yaml`, including the type overrides Orderly uses.
- PostgreSQL: the locking clause: exactly what `FOR UPDATE` and `SKIP LOCKED` promise.
- Pattern: Transactional outbox: Chris Richardson's write-up of the pattern behind `internal/events`.

- RFC 9457: Problem Details for HTTP APIs: the standard behind Orderly's error responses.
- franz-go kgo package: the Kafka client's API, for when you extend the relay or add a consumer.

CHAPTER 21

Your Path to Mastery

> *Reading about Go gets you started. A plan, small projects and a few good habits make you fluent.*

🎯 **What you'll learn**

- What "mastering Go" actually looks like, level by level
- A realistic 12-week plan that fits around a full-time job
- Five milestone projects that lock in each stage
- The daily and weekly habits experienced Go developers share
- The best books, sites and talks to keep growing

The big picture

Mastery in Go is less about knowing more features and more about **judgement**. The language is small, and you can learn all of its syntax in a weekend. What takes longer is the instinct for *which* simple solution to reach for: when a plain function beats an interface, when a mutex beats a channel, when to stop abstracting.

That instinct comes from building things, reading good code and getting feedback. This chapter gives you a structure for all three.

🧩 **Think of it like learning to cook in a new kitchen**

You already know how to cook (you're an experienced engineer). The new kitchen has fewer gadgets and sharper knives. At first you keep reaching for tools that aren't there. After a few weeks of real meals, you stop missing them, and you notice how much faster you move.

The four levels of Go skill

Use this table to work out where you are, and what "next" looks like.

Level	You can…	Signs you're ready to move on
1. Beginner	Write small programs, use slices, maps and structs, handle errors with `if err != nil`	`gofmt` and `go vet` rarely complain; you've stopped writing getters
2. Practitioner	Build a REST service with a database, tests and config; use goroutines safely with `context`	You design small interfaces where they're *consumed*; your tests are table-driven without thinking

Level	You can…	Signs you're ready to move on
3. Proficient	Run Go in production: observability, graceful shutdown, resilience, profiling, CI/CD	You can read a pprof profile, explain an escape-analysis report and spot a goroutine leak in review
4. Expert	Shape team conventions, design libraries others enjoy, mentor, debug runtime-level issues	People ask *you* "what's the idiomatic way?", and your answer is usually "the simpler one"

> 🧠 **Key idea**
>
> Moving up a level is mostly about *removing* things from your code: fewer layers, fewer interfaces, fewer dependencies, fewer surprises.

A 12-week plan

This plan assumes about **5 to 7 hours a week**, roughly an hour on weekdays plus a longer weekend session. If you have more time, compress it. If life gets busy, stretch it. Steady progress beats intensity.

Weeks	Read	Build	You're ready to move on when…
1	Ch 1, 2	Install Go, set up your editor, run "hello Orderly"	You can create a module and run tests from the terminal
2	Ch 3	Exercises from Chapter 3; a small Order calculator	Slices and maps no longer surprise you
3	Ch 4	**Milestone 1** (CLI tool)	You handle errors with `%w` and `errors.Is` naturally
4	Ch 5	Concurrency patterns from the chapter	You can explain when to use a channel vs a mutex
5	Ch 5 (again), 6	**Milestone 2** (concurrent worker)	Your goroutines always have a way to stop
6	Ch 7	REST endpoints with chi, validation, problem+json	Your handlers are thin and your errors map cleanly to HTTP
7	Ch 9, 10	Add Postgres with sqlc, migrations, env config	You can write a transaction without `@Transactional`
8	Ch 15	**Milestone 3** (tested CRUD service)	Tests are table-driven; fakes replace most mocks
9	Ch 11, 12	Outbox + Kafka, JWT auth	You understand at-least-once delivery and idempotency
10	Ch 13, 14	**Milestone 4** (event-driven pair)	You can trace a request across two services
11	Ch 16, 17	Profile and optimise one hot path	You've measured before *and* after a change
12	Ch 18, 19, 20	**Milestone 5** (ship it)	Your service runs in a container, with probes, metrics and graceful shutdown

> 💡 **Re-read Chapter 5**
>
> Almost everyone needs two passes at concurrency. The second time, the patterns click because you've used them. That's why it appears twice in the plan.

Five milestone projects

Each milestone is a small, complete project. Keep them in a personal Git repository. In three months you'll have a portfolio that shows real Go skill.

Milestone 1: `orderlyctl`, a command-line tool (week 3)

Build a CLI that reads products and orders from a JSON file and prints reports: total revenue, top products, orders by status.

- Practise: structs, slices, maps, sorting, errors, file I/O, `encoding/json`
- Stretch: subcommands with cobra (Chapter 19), table-driven tests
- **Done when:** `go vet` is clean, every function that can fail returns an `error`, and you have tests

Milestone 2: a concurrent stock checker (week 5)

Given 1,000 product IDs, check stock against a slow fake "warehouse API" (sleep 50 to 200 ms per call) with at most 20 calls in flight. Stop everything if the user presses Ctrl+C.

- Practise: worker pools, `errgroup` with `SetLimit`, `context` cancellation, `signal.NotifyContext`
- Stretch: add a rate limit of 50 calls per second with `golang.org/x/time/rate`
- **Done when:** it finishes far faster than doing the calls one after another, and Ctrl+C exits cleanly with no leaked goroutines

Milestone 3: the Orderly products and orders API (week 8)

A REST API for products and orders backed by Postgres.

- Practise: chi, validation, problem+json errors, pgx + sqlc, goose migrations, env config, slog
- Tests: handler tests with `httptest`, service tests with fakes, one integration test with testcontainers
- **Done when:** placing an order reserves stock inside a transaction, and two concurrent orders can't oversell

Milestone 4: an event-driven pair (week 10)

Split out a second service, `notifier`, that consumes `OrderPlaced` events and "sends" an email (a log line).

- Practise: the outbox pattern, franz-go producer and consumer, idempotent consumers, JWT auth between services
- Observability: Prometheus metrics in both, OpenTelemetry traces linked across the event
- **Done when:** you can kill and restart either service without losing or duplicating an email

Milestone 5: ship it (week 12)

Make your services production-shaped.

- A multi-stage Dockerfile producing a small, non-root image
- Kubernetes manifests or a Helm chart with liveness, readiness and startup probes
- A CI pipeline: lint, test with `-race`, `govulncheck`, build, image scan
- Graceful shutdown verified by sending `SIGTERM` during load

- **Done when:** a teammate can clone the repo and get it running from the README alone

> **The Go way**
>
> Resist the urge to build a "framework" during these projects. If you notice yourself writing a generic base handler, a reflection-based mapper or a plugin system, stop and write the plain version. Go rewards boring code.

Habits that build mastery

The plan gets you to "proficient". These habits keep you growing after that.

Read the standard library

The Go standard library is some of the best Go code ever written, and it's right there on your machine. Start with these packages, which are short, readable and full of idioms:

Package	What you'll learn from it
`strings`, `bytes`	Clean API design, small functions, performance-minded code
`io`	The power of tiny interfaces (`Reader`, `Writer`)
`errors`	How `Is`, `As` and wrapping really work
`context`	Cancellation trees, and why the API is so small
`sync`	How `Once`, `WaitGroup` and `Mutex` are built
`net/http`	A production server and client, with middleware-friendly design

In your editor, jump to the definition of any standard-library function you call. It's a habit that pays off for years.

Let the tools teach you

- Run `go vet`, `staticcheck` and `golangci-lint` (Chapter 16) on everything. Each warning is a free lesson.
- Read the explanation behind each linter rule you trip. You'll absorb idioms faster than any book can teach them.
- Run `gofmt` on save and never argue about formatting again.

Get your code reviewed by Go people

Nothing accelerates learning like a review from someone who has written Go for years. If your team is new to Go, look outside: open-source contributions, Go community forums and Gopher Slack channels are welcoming places to ask "is this idiomatic?"

Write less, delete more

Every few weeks, look back at code you wrote earlier and ask: *what can I remove?* An unused interface, a wrapper that adds nothing, a goroutine that doesn't need to exist. Deleting code is a Go superpower.

Recommended resources

You don't need all of these. Pick one book and one website to start, and come back for more when you're ready.

Books

Book	Best for
Learning Go, 2nd edition, Jon Bodner (O'Reilly, 2024)	The best all-round modern introduction; idiomatic from page one
The Go Programming Language, Donovan and Kernighan (Addison-Wesley, 2015)	Deep, precise fundamentals; still excellent despite its age
100 Go Mistakes and How to Avoid Them, Teiva Harsanyi (Manning, 2022)	Turning "it works" into "it's right"; perfect after Milestone 3
Concurrency in Go, Katherine Cox-Buday (O'Reilly, 2017)	Mastering goroutines, channels and concurrency patterns
Let's Go and *Let's Go Further*, Alex Edwards	Building real web apps and APIs step by step
Efficient Go, Bartłomiej Płotka (O'Reilly, 2022)	Performance work done properly: measure, then optimise

Websites

- **A Tour of Go** (https://go.dev/tour): an interactive walk through the language. Great for week 1.
- **Go by Example** (https://gobyexample.com): short, runnable examples of nearly everything.
- **Effective Go** (https://go.dev/doc/effective_go): the classic guide to writing Go the Go way.
- **The Go Blog** (https://go.dev/blog): authoritative deep dives on errors, generics, modules and more.
- **Go Code Review Comments** (https://go.dev/wiki/CodeReviewComments): the common review feedback, collected.
- **Google Go Style Guide** (https://google.github.io/styleguide/go/) and **Uber Go Style Guide** (https://github.com/uber-go/guide): how large teams write consistent Go.
- **Go release notes** (https://go.dev/doc/devel/release): skim each new release; Go ships one every six months.

Talks worth an evening

- **"Concurrency Is Not Parallelism"**, Rob Pike: the mental model behind goroutines (https://go.dev/blog/waza-talk).
- **"Go Proverbs"**, Rob Pike: the philosophy in 20 short sayings (https://go-proverbs.github.io).
- **"Advanced Go Concurrency Patterns"**, Sameer Ajmani: `select`, cancellation and real-world pipelines (https://go.dev/talks/2013/advconc.slide).

Practice

- **Exercism's Go track** (https://exercism.org/tracks/go): free exercises with optional human mentoring.
- **Rewrite a small Spring service you know well.** It's the single most effective exercise for a Java developer, because you already understand the problem and can focus purely on the Go.

Recap

- Go mastery is about judgement: picking the simplest thing that works, and removing what doesn't earn its place.
- The four levels are beginner, practitioner, proficient and expert. Most of the journey is from level 2 to level 3, where production skills live.
- The 12-week plan pairs reading with building. Five milestone projects turn chapters into real skill.
- Read the standard library. It is the best idiomatic Go you'll find.
- Let linters and reviewers teach you, and delete code often.
- Pick one book and one website to start, not all of them.

Try it yourself

Exercise 1: Place yourself on the map

Using the four-levels table, write down where you are today and the one skill that would move you to the next level. Put a date on it.

Exercise 2: Schedule the first two weeks

Block time in your calendar for weeks 1 and 2 of the plan, right now. Small, fixed sessions beat occasional marathons.

Exercise 3: Read one standard library file

Open the source of `strings.Builder` (jump to its definition in your editor). Read it end to end. Note one idiom you'd like to copy, for example how it detects being copied by remembering its own address (`copyCheck`).

Exercise 4 (stretch): Rewrite something you know

Pick a small Spring Boot service you've built before. Rewrite one endpoint, with its persistence and tests, in Go. Compare line counts, startup time and memory use, and note what you missed and what you didn't.

Further reading

- A Tour of Go: the official interactive introduction.
- Effective Go: the classic idioms guide.
- Go Code Review Comments: what reviewers will tell you.
- The Go Blog: deep dives straight from the Go team.

You've reached the end of the main chapters. The appendices that follow are your quick-reference companions. Keep the cheat sheet (Appendix A) and the code review checklist (Appendix C) close while you build. Happy coding, Gopher.

APPENDIX A

Java to Go Cheat Sheet

> *Keep this open in a second window while you build. It answers "what's the Go version of...?"*

This appendix collects the mappings from across the guide in one place. Each row points to the chapter that explains it in depth. The Go snippets live in `examples/appendix-a-cheatsheet/` and were compiled and tested like everything else.

> 💡 **A mapping, not a translation**
>
> These tables show the *closest* Go equivalent. Often the Go version works differently, and that's on purpose. When a row says "no direct equivalent", the linked chapter explains the Go way of solving the same problem.

Language concepts

Java	Go	Chapter
`class` / POJO / `record`	`struct` + methods	3
`implements` (explicit)	Interfaces satisfied implicitly	4
`extends` (inheritance)	Embedding (composition)	4
Constructor	`NewThing(...)` function by convention	3
`this`	Named receiver: `func (o *Order) Cancel()`	3
`public` / `private`	Capitalised = exported; lowercase = package-only	4
Package-private + modules	`internal/` directories	4, 6
`static` method / field	Package-level function / variable	3
`null`	`nil` (for pointers, slices, maps, channels, interfaces, funcs)	3
Default field values	Zero values (`0` , `""` , `false` , `nil`)	3
`Optional<T>`	`(value, ok)` , a pointer, or the zero value	3
Checked / unchecked exceptions	`error` return values	4
`throw` / `catch`	`return err` / `if err != nil`	4
Exception chaining (`cause`)	`fmt.Errorf("...: %w", err)` , `errors.Is` / `As`	4
`RuntimeException` crash	`panic` (only for real bugs)	4
`finally` / try-with-resources	`defer`	4

Java	Go	Chapter
`enum`	Typed constants + `iota` + `String()`	4
Generics with type erasure	Type parameters with constraints	4
`? extends T` wildcards	No wildcards; use constraints or interfaces	4
Annotations	Struct tags (metadata) + code generation	3, 16
`equals` / `hashCode`	`==` on comparable types; `comparable` constraint	3
`toString()`	`String() string` (the `fmt.Stringer` interface)	4
`ArrayList<T>`	Slice `[]T`	3
`HashMap<K,V>`	Map `map[K]V`	3
`HashSet<T>`	`map[T]struct{}`	3
`String` (UTF-16)	`string` (immutable UTF-8 bytes); `rune` for a character	3
`StringBuilder`	`strings.Builder`	3
Streams (`filter` / `map`)	`for` loops; `slices` and `maps` packages; iterators	3, 4
`Iterator` / `Iterable`	Range-over-func iterators (`iter.Seq`)	4
`java.time.Instant` / `Duration`	`time.Time` / `time.Duration`	3
`BigDecimal` for money	`int64` cents (or `shopspring/decimal`)	19
Classpath resources	`embed` package (`//go:embed`)	19
`Object`	`any` (alias for `interface{}`)	4
`instanceof` / casting	Type assertion `v.(T)` and type switch	4

Concurrency

Java	Go	Chapter
`Thread` / virtual thread	Goroutine (`go f()`)	5
`ExecutorService`	Goroutines + `sync.WaitGroup` , or a worker pool	5
`CountDownLatch`	`sync.WaitGroup` (`wg.Go` since 1.25)	5
`BlockingQueue`	Channel (`chan T`)	5
`synchronized` / `ReentrantLock`	`sync.Mutex`	5
`ReadWriteLock`	`sync.RWMutex`	5
`AtomicLong`	`atomic.Int64`	5
`ConcurrentHashMap`	Map + mutex (usually), or `sync.Map`	5
Lazy holder / double-checked locking	`sync.Once` , `sync.OnceValue`	5
`ThreadLocal` / `ScopedValue`	`context.Context` , passed explicitly	5
`Future.cancel` / interrupts	`context` cancellation	5

Java	Go	Chapter
`CompletableFuture.allOf` / `StructuredTaskScope`	`errgroup.Group`	5
`Semaphore`	Buffered channel or `x/sync/semaphore`	5
`ScheduledExecutorService`	`time.Ticker` in a goroutine	5, 11

Toolchain

Java	Go	Chapter
Maven / Gradle	The `go` command	2
`pom.xml` / `build.gradle`	`go.mod`	2
Lock file / checksums	`go.sum`	2
Maven Central	Module proxy (`proxy.golang.org`)	2
Multi-module build	`go.work` workspaces	2, 6
`mvn package`	`go build`	2
`mvn test`	`go test ./...`	2, 15
`java -jar app.jar`	`./app` (a single static binary)	18
Maven plugins	`tool` directives in go.mod, `go generate`	2, 16
Checkstyle / SpotBugs / PMD	`gofmt` , `go vet` , staticcheck, golangci-lint	16
Javadoc	Doc comments + `go doc` + pkg.go.dev	19
JMH	`testing.B` benchmarks	15, 17
JFR / async-profiler	`pprof` , `runtime/trace`	13, 17
IntelliJ debugger	Delve (in VS Code or GoLand)	2, 16
Fat JAR / Jib	Static binary / `ko`	18

Spring annotations and their Go equivalents

Go has no annotation processing. Every row below becomes *plain code* you can read and step through.

Spring	Go equivalent	Chapter
`@SpringBootApplication`	`func main()` that wires everything	6
`@Component` / `@Service` / `@Repository`	A struct with a `New...` constructor	6
`@Autowired` / constructor injection	Pass dependencies to `New...` in `main`	6
`@Bean` / `@Configuration`	A function in `main` (or Uber Fx `fx.Provide`)	6
`@RestController` + `@GetMapping`	`mux.HandleFunc("GET /path", h)` or chi routes	7
`@PathVariable` / `@RequestParam`	`r.PathValue("id")` / `r.URL.Query().Get("q")`	7

Spring	Go equivalent	Chapter
`@RequestBody` + `@Valid`	`json.Decoder` + go-playground/validator	7
`@ControllerAdvice` / `@ExceptionHandler`	A central error-to-response function	7
`Filter` / `HandlerInterceptor`	Middleware: `func(http.Handler) http.Handler`	7
`@Transactional`	Explicit `tx` with a `WithTx` helper	9
`@Cacheable`	Cache-aside code with go-redis or a local cache	9
`@Value` / `@ConfigurationProperties`	caarlos0/env struct tags	10
`@Profile`	Env vars or config files per environment	10
`@KafkaListener`	A franz-go consumer loop in a goroutine	11
`@Scheduled`	`time.Ticker`, robfig/cron or River	11
`@Async`	`go func() {...}()` with proper lifecycle	5, 11
`@PreAuthorize`	Authorisation middleware or service-layer checks	12
Actuator `/health`, `/metrics`	Your own `/healthz`, `/readyz` + Prometheus handler	13
`@CircuitBreaker` / `@Retry` (Resilience4j)	failsafe-go, gobreaker, backoff	14
`@PostConstruct` / `@PreDestroy`	Code in `main` before serving / graceful shutdown	6, 14
`@SpringBootTest`	`httptest` + real wiring in a test	15
`@MockBean` / Mockito	Hand-written fakes (or uber-go/mock)	15
`@ParameterizedTest`	Table-driven tests with `t.Run`	15

Frameworks and libraries

Concern	Java	Go (recommended first)	Chapter
App framework	Spring Boot	Standard library + focused libraries	1, 6
HTTP routing	Spring MVC, JAX-RS	`net/http`, **chi**, Gin, Echo	7
Reactive	WebFlux, Vert.x	Not needed: goroutines make blocking code scale	5, 7
DI container	Spring IoC, Guice, Dagger	Manual wiring, Uber Fx, Wire	6
Validation	Hibernate Validator	go-playground/validator	7
JSON	Jackson	`encoding/json`	7
OpenAPI	springdoc	oapi-codegen (spec-first), swaggo	7
gRPC	grpc-java	grpc-go, **Connect**	8
GraphQL	Spring for GraphQL, DGS	gqlgen	8
JDBC	JDBC, HikariCP	`database/sql`, **pgx** + pgxpool	9
SQL mapping	jOOQ, MyBatis, JdbcTemplate	**sqlc**, sqlx	9
ORM	Hibernate / JPA	GORM, ent	9

Concern	Java	Go (recommended first)	Chapter
Migrations	Flyway, Liquibase	**goose**, golang-migrate, Atlas	9
Caching	Caffeine, Spring Cache	ristretto / otter, go-redis	9
Config	Spring Boot config	**caarlos0/env**, koanf, Viper	10
Feature flags	Unleash, LaunchDarkly	OpenFeature Go SDK	10
Kafka	Spring Kafka	**franz-go**	11
RabbitMQ	Spring AMQP	amqp091-go	11
Messaging framework	Spring Cloud Stream	Watermill	11
Jobs / scheduling	Quartz, `@Scheduled`	River, asynq, robfig/cron	11
Workflows	Temporal Java, Camunda	Temporal Go SDK	11
Security	Spring Security	golang-jwt, go-oidc, x/oauth2, Casbin	12
Logging	SLF4J + Logback	**log/slog**, zap, zerolog	13
Metrics	Micrometer	prometheus/client_golang, OpenTelemetry	13
Tracing	OpenTelemetry Java agent	OpenTelemetry Go SDK	13
Resilience	Resilience4j	failsafe-go, gobreaker, x/time/rate	14
Testing	JUnit 5, AssertJ	`testing`, testify	15
Mocking	Mockito	Fakes, uber-go/mock, mockery	15
Integration tests	Testcontainers	testcontainers-go	15
CLI	picocli	cobra	19
Kubernetes operators	Java Operator SDK	controller-runtime, kubebuilder	18

Everyday snippets, side by side

Hello, world

JAVA

```java
public class Main {
    public static void main(String[] a) {
        System.out.println("Hello");
    }
}
```

GO

```go
package main

import "fmt"

func main() {
    fmt.Println("Hello")
}
```

A type with a method

JAVA

```java
record Product(String name,
               long priceCents) {
    String label() {
        return "%s (%d)".formatted(
            name, priceCents);
    }
}
```

GO

```go
type Product struct {
    Name       string
    PriceCents int64
}

func (p Product) Label() string {
    return fmt.Sprintf("%s (%d)",
        p.Name, p.PriceCents)
}
```

Interfaces

JAVA

```java
interface Notifier {
    void notify(String msg);
}
class Email implements Notifier {
    public void notify(String msg) {
        System.out.println(msg);
    }
}
```

GO

```go
type Notifier interface {
    Notify(msg string) error
}

// No "implements": having the
// method is enough.
type EmailNotifier struct{}

func (EmailNotifier) Notify(
    msg string) error { return nil }
```

Errors instead of exceptions

JAVA

```java
try {
    Product p = load(id);
} catch (NotFoundException e) {
    log.warn("missing {}", id);
}
```

GO

```go
p, err := load(id)
if errors.Is(err, ErrNotFound) {
    log.Printf("missing %s", id)
}
```

GO Wrapping adds context, like an exception cause

```go
func load(id string) (Product, error) {
    p, err := find(id)
    if err != nil {
        return Product{}, fmt.Errorf("load %q: %w", id, err)
    }
    return p, nil
}
```

Cleaning up resources

JAVA

```java
try (var in = Files
        .newInputStream(path)) {
    return in.readAllBytes();
}
```

GO

```go
f, err := os.Open(path)
if err != nil {
    return nil, err
}
defer f.Close()
```

Filtering and mapping a list

JAVA

```java
List<String> names = products
    .stream()
    .filter(p -> p.priceCents() > 1000)
    .map(Product::name)
    .toList();
```

GO

```go
var names []string
for _, p := range products {
    if p.PriceCents > 1000 {
        names = append(names, p.Name)
    }
}
```

Maps: lookups with a default

JAVA

```java
int qty = stock
    .getOrDefault("p2", 0);
```

GO

```go
qty, ok := stock["p2"]
if !ok {
    qty = 0 // already the zero value
}
```

Sorting

JAVA

```java
products.sort(Comparator
    .comparingLong(
        Product::priceCents));
```

GO

```go
slices.SortFunc(ps,
    func(a, b Product) int {
        return cmp.Compare(
            a.PriceCents, b.PriceCents)
    })
```

Enums

JAVA

```java
enum Status {
    PENDING, CONFIRMED, SHIPPED
}
```

GO

```go
type Status int

const (
    Pending Status = iota
    Confirmed
    Shipped
)
```

Generic functions

JAVA

```java
static <T extends Comparable<T>>
T max(T a, T b) {
    return a.compareTo(b) > 0
        ? a : b;
}
```

GO

```go
func Max[T cmp.Ordered](a, b T) T {
    if a > b {
        return a
    }
    return b
}
```

Running work concurrently

JAVA

```java
try (var ex = Executors
    .newVirtualThreadPerTaskExecutor()) {
    for (var id : ids) {
        ex.submit(() -> fetch(id));
    }
} // waits for all tasks
```

GO

```go
var wg sync.WaitGroup
for _, id := range ids {
    wg.Go(func() {
        fetch(id)
    })
}
wg.Wait()
```

JSON

JAVA

```java
var mapper = new ObjectMapper();
String json = mapper
    .writeValueAsString(p);
Product back = mapper
    .readValue(json, Product.class);
```

GO

```go
data, err := json.Marshal(p)
// handle err
var back Product
err = json.Unmarshal(data, &back)
```

Environment variables and time

JAVA

```java
String port = Optional
    .ofNullable(System.getenv("PORT"))
    .orElse("8080");
Instant deadline = Instant.now()
    .plus(Duration.ofSeconds(30));
```

GO

```go
port, ok := os.LookupEnv("PORT")
if !ok {
    port = "8080"
}
deadline := time.Now().
    Add(30 * time.Second)
```

> ⚠️ **Gotcha: Go's date format**
>
> Go formats dates with a *reference date* instead of letters like `yyyy-MM-dd`. The reference is `Mon Jan 2 15:04:05 MST 2006`, which is 1 2 3 4 5 6 7 in American order. So `yyyy-MM-dd HH:mm` becomes `"2006-01-02 15:04"`. It looks bizarre for a day, then you'll never forget it. See Chapter 3.

The ten biggest mindset shifts, in one line each

1. **Explicit over magic.** If something happens, there's a line of code that makes it happen.
2. **Composition over inheritance.** Embed and combine small pieces.
3. **Errors are values.** Handle them where they occur, and add context as they travel up.
4. **Small interfaces, defined by the consumer.** "Accept interfaces, return structs."
5. **The zero value should be useful.** Design types that work without a constructor where you can.
6. **Concurrency is cheap, but lifecycle is your job.** Every goroutine needs a way to stop.
7. **Context travels with the request.** It's always the first parameter.
8. **Libraries, not frameworks.** Compose the standard library with a few focused packages.
9. **One way to format.** `gofmt` ends style debates.
10. **Simple beats clever.** Readable code that a new teammate understands wins every time.

APPENDIX B

Common Mistakes Java Developers Make in Go

> *Every mistake here has been made by thousands of smart Java developers. Learn them from a page instead of from a production incident.*

These are the traps that come up again and again when experienced Java engineers move to Go. None of them mean you're doing badly. They're the natural result of good Java instincts applied in a language with different rules.

Each entry shows **what it looks like**, **why it hurts** and **what to do instead**. The chapter in brackets has the full story.

Design and style

1. Creating an interface for every struct

(Chapters 1, 4, 6)

In Java, `OrderService` + `OrderServiceImpl` is a common habit. In Go, an interface nobody else implements is noise.

GO Avoid

```go
type OrderService interface {
    Place(ctx context.Context, o Order) error
}

type orderServiceImpl struct{ repo OrderRepository }
```

GO Prefer

```go
// A concrete type. Consumers that need an abstraction define
// their own small interface with only the methods they use.
type OrderService struct{ repo OrderRepository }
```

Rule of thumb: define interfaces where they're *consumed*, and only when you have (or genuinely need) more than one implementation, for example a fake in tests.

2. Returning interfaces from constructors

(Chapter 4)

Returning an interface hides useful methods and forces type assertions later. "Accept interfaces, return structs" keeps things flexible for callers.

GO Prefer

```go
func NewOrderService(repo OrderRepository) *OrderService {
    return &OrderService{repo: repo}
}
```

3. Getters and setters for everything

(Chapter 1)

If a field can be read and written freely, export it. If it needs protection, keep it unexported and provide a method that *does something meaningful*, like `order.Cancel()`. Don't write `SetStatus()`. When you do need a getter, Go style drops the `Get` prefix: `o.Status()`, not `o.GetStatus()`.

4. Name stutter and Java-style names

(Chapters 1, 16)

Package names are part of every call, so `order.OrderService` reads as "order order service". Go also avoids `I` prefixes and `Impl` suffixes.

Java habit	Go style
`order.OrderService`	`order.Service`
`IRepository`	`Repository` (or better, a small consumer-side interface)
`OrderServiceImpl`	`Service`
`getCustomerId()`	`CustomerID()` (initialisms stay upper case)
package `utils`, `common`, `helpers`	Packages named for what they provide: `money`, `httpx`, `clock`

5. Mirroring Spring layers as packages

(Chapter 6)

A `controllers/`, `services/`, `repositories/` package layout spreads one feature across three packages and invites import cycles. Group by *feature* instead: `internal/order`, `internal/inventory`.

6. Global singletons and `init()` magic

(Chapters 4, 6)

Package-level `var db *sql.DB` set in `init()` feels like a Spring bean, but it makes testing hard and startup order invisible. Create dependencies in `main` and pass them explicitly.

7. Using `float64` for money

(Chapter 19)

`0.1 + 0.2` isn't `0.3` in floating point, in Go or in Java. Use `int64` cents, which is the convention throughout this guide, or a decimal library.

Errors

8. Ignoring errors

(Chapter 4)

GO Avoid

```go
data, _ := json.Marshal(order) // what if it fails?
```

The blank identifier `_` silences the compiler, not the problem. Handle the error, or at least return it. If you *truly* want to ignore one, leave a comment explaining why.

9. Logging an error and returning it

(Chapter 4)

GO Avoid

```go
if err != nil {
    log.Printf("failed to save order: %v", err)
    return err // the caller will log it again
}
```

The same failure ends up in your logs three or four times with no extra information. **Either handle it or return it**, and add context when returning:

GO Prefer

```go
if err != nil {
    return fmt.Errorf("save order %s: %w", o.ID, err)
}
```

Log once, at the edge: in the HTTP handler, the consumer loop or `main`.

10. Wrapping with `%v` instead of `%w`

(Chapter 4)

`fmt.Errorf("...: %v", err)` turns the error into text, so `errors.Is(err, ErrOrderNotFound)` stops working upstream. Use `%w` when callers may need to inspect the cause.

11. Using `panic` like `throw`

(Chapter 4)

`panic` is for programmer errors and impossible states, not for "order not found" or "invalid input". Return an `error` instead. Reserve `recover` for safety nets such as HTTP middleware that turns an unexpected panic into a 500 response.

12. The nil-interface trap

(Chapter 4)

GO Avoid

```go
func validate(o Order) error {
    var verr *ValidationError // nil pointer
    if o.Quantity <= 0 {
        verr = &ValidationError{Field: "quantity"}
    }
    return verr // never == nil, even when verr is nil!
}
```

An interface holding a typed nil pointer is **not** nil. Return a literal `nil` on the success path.

Data types

13. Slice aliasing after `append`

(Chapter 3)

Two slices can share the same backing array. Appending to one can silently overwrite data seen by the other.

GO Avoid

```go
base := make([]int, 3, 10)
a := append(base, 1) // shares base's array
b := append(base, 2) // overwrites a[3]!
```

When you need an independent copy, use `slices.Clone(s)`, or build a new slice explicitly.

14. Writing to a nil map

(Chapter 3)

GO Avoid

```go
var stock map[string]int
stock["p1"] = 5 // panic: assignment to entry in nil map
```

Reading from a nil map is fine; writing panics. Initialise with `make(map[string]int)` or a literal.

15. Expecting map iteration order

(Chapter 3)

Go deliberately randomises map iteration order. If you need a stable order, for output, tests or deterministic hashing, collect the keys and sort them (`slices.Sorted(maps.Keys(m))`).

16. Modifying the loop copy

(Chapter 3)

GO Avoid

```go
for _, item := range order.Items {
    item.Quantity++ // changes a copy, not the slice element
}
```

GO Prefer

```go
for i := range order.Items {
    order.Items[i].Quantity++
}
```

17. Pointers to everything, or pointers to interfaces

(Chapters 3, 4)

Java references everything, so Java developers often reach for `*T` everywhere. In Go, small structs are often better passed by value. A pointer to an interface (`*io.Reader`) is almost always a mistake, because interfaces already hold a pointer internally.

18. Expecting embedding to work like inheritance

(Chapter 4)

Embedding promotes methods, but there's no virtual dispatch: an embedded type's method never calls the outer type's "override". If you need polymorphism, use an interface.

Concurrency

19. Goroutines with no way to stop

(Chapter 5)

GO Avoid

```go
go func() {
    for {
        relayOutbox() // runs forever, even during shutdown
        time.Sleep(time.Second)
    }
}()
```

GO Prefer

```go
go func() {
    ticker := time.NewTicker(time.Second)
    defer ticker.Stop()
    for {
        select {
        case <-ctx.Done():
            return
        case <-ticker.C:
            relayOutbox(ctx)
        }
    }
}()
```

Every goroutine you start needs an owner, a way to stop, and someone who waits for it to finish.

20. Not passing `context`, or storing it in structs

(Chapter 5)

Without `ctx` as the first parameter, a cancelled HTTP request keeps querying your database. And a context stored in a struct field outlives the request it belonged to. Pass it explicitly, every time: `func (s *Service) Place(ctx context.Context, ...)`.

21. Forgetting to call `cancel`

(Chapter 5)

`context.WithTimeout` and `context.WithCancel` return a `cancel` function. Always `defer cancel()`, even if the timeout will fire anyway; otherwise resources linger until it does. `go vet` catches many of these.

22. Unsynchronised shared state

(Chapter 5)

Concurrent writes to a plain map can crash the whole program with `fatal error: concurrent map writes`, and that one can't be recovered. Protect shared data with a `sync.Mutex`, and run tests with `-race` in CI.

23. Copying a mutex

(Chapter 5)

GO Avoid

```go
type Counter struct {
    mu sync.Mutex
    n  int
}

func (c Counter) Inc() { // value receiver copies the mutex!
    c.mu.Lock()
    c.n++
    c.mu.Unlock()
}
```

Use a pointer receiver (`func (c *Counter) Inc()`). `go vet`'s `copylocks` check flags this.

24. Channels where a mutex is simpler

(Chapter 5)

Channels are wonderful for *passing work and ownership* between goroutines. For protecting a counter or a cache, a mutex is shorter and clearer. Pick the tool that makes the code easiest to read.

Resources and I/O

25. `defer` inside a loop

(Chapter 4)

GO Avoid

```go
for _, path := range files {
    f, _ := os.Open(path)
    defer f.Close() // nothing closes until the function returns
}
```

Move the loop body into a small function so each `defer` runs per iteration.

26. Not closing response bodies and rows

(Chapters 7, 9)

Forgetting `defer resp.Body.Close()` or `defer rows.Close()` leaks connections until your pool runs dry. Also remember to check `rows.Err()` after the loop.

27. HTTP without timeouts

(Chapters 7, 14)

`http.DefaultClient` and a bare `http.Server{}` have **no timeouts**. One slow dependency or slow client can pile up goroutines until you run out of memory. Always set `Timeout` on clients and `ReadHeaderTimeout`, `ReadTimeout`, `WriteTimeout` and `IdleTimeout` on servers.

28. Building strings with `+=` in a loop

(Chapters 3, 17)

Each `+=` on a string allocates a new one. For loops, use `strings.Builder`, the same lesson as Java's `StringBuilder`.

Data access

29. Treating GORM like Hibernate

(Chapter 9)

GORM has no persistence context, no dirty checking and no lazy-loading proxies. Changing a struct doesn't update the database, and loops that load associations one by one cause N+1 queries. If you use GORM, be explicit about `Preload` and saves, or consider sqlc, where you write the SQL yourself.

30. Hunting for `@Transactional`

(Chapter 9)

Transactions in Go are explicit: begin, pass the transaction to your queries, commit or roll back. A small `WithTx` helper makes this tidy, and you always know exactly what's inside the transaction.

Testing

31. Reaching for a mocking framework first

(Chapter 15)

A ten-line hand-written fake of a small interface is usually clearer than a generated mock with expectations. Save mock frameworks for interfaces with many methods or interaction-heavy tests.

32. `time.Sleep` in tests

(Chapter 15)

Sleeps make tests slow *and* flaky. Inject a clock, use channels to signal completion, or use `testing/synctest` (Go 1.25) for time-based concurrent code.

Tooling

33. Fighting the tools

(Chapter 16)

Turn on format-on-save, run `go vet` and a linter in CI, and treat their feedback as free mentoring. Go's tools encode years of community experience, and arguing with `gofmt` is a battle nobody wins.

> 📝 **A mistake that Go fixed for you**
>
> Before Go 1.22, a loop variable was shared across iterations, so goroutines started in a loop could all see the last value. Since Go 1.22, each iteration gets its own variable. You may still see `id := id` lines in older code. They're harmless, just no longer needed.

> 🧠 **Key idea**
>
> Most of these mistakes come from one root: bringing Java's *implicit* world into Go's *explicit* one. When in doubt, write the plain, visible version, and let the tools check you.

APPENDIX C

Go Code Review Checklist

> *A friendly list to run through before you open a pull request, or while you review someone else's.*

Use this checklist as a prompt, not a gate. Not every item applies to every change. If you're reviewing, phrase findings kindly and explain the *why*. Many Go idioms are new to people coming from Java, and a good review is one of the best ways to learn them.

> 💡 **Automate the boring half**
>
> Anything a tool can check, let the tool check: `gofmt`, `go vet`, `staticcheck`, `golangci-lint`, `govulncheck` and `go test -race` (Chapter 16). Then human reviewers can focus on design, clarity and correctness.

Readability and naming

- ☐ Code is `gofmt`-formatted and the linters pass in CI.
- ☐ Package names are short, lower case and describe what the package *provides* (no `utils`, `common`, `helpers`).
- ☐ No stutter: `order.Service`, not `order.OrderService`.
- ☐ Initialisms are consistently upper case: `ID`, `URL`, `HTTP` (`CustomerID`, not `CustomerId`).
- ☐ Names are short in small scopes and descriptive in large ones (`i` in a loop, `pendingOrders` across a function).
- ☐ Exported identifiers have doc comments that start with the identifier's name.
- ☐ Functions read top to bottom with early returns; the happy path isn't buried in `else` blocks.

Design

- ☐ Interfaces are small and defined where they're **consumed**.
- ☐ Constructors return concrete types ("accept interfaces, return structs").
- ☐ No interface exists with only one implementation and no test fake.
- ☐ Dependencies are passed in explicitly. There are no package-level mutable globals and no `init()` side effects.
- ☐ Packages are organised by feature, with no import cycles and nothing in `internal/` leaking out.
- ☐ Types have useful zero values where practical.
- ☐ No premature abstraction: generics, reflection and plugin systems are justified by real, current needs.

Errors

- ☐ No error is silently dropped with `_` unless there's a comment explaining why.
- ☐ Errors returned upward are wrapped with context: `fmt.Errorf("place order %s: %w", id, err)`.
- ☐ `%w` (not `%v`) is used where callers might call `errors.Is` or `errors.As`.
- ☐ Each error is either handled or returned, not both. It's logged once, at the edge.
- ☐ Sentinel errors and custom error types are used for conditions callers must distinguish.
- ☐ Domain errors are mapped to HTTP or gRPC status codes in one place, not scattered through handlers.
- ☐ `panic` is used only for truly impossible states; `recover` only in safety-net middleware.
- ☐ Functions return a literal `nil` error on success, never a typed nil pointer.

Concurrency

- ☐ Every goroutine has an owner, a way to stop (usually `ctx.Done()`) and something that waits for it.
- ☐ Shared mutable state is protected by a mutex or owned by a single goroutine.
- ☐ Structs containing a `sync.Mutex` are never copied (pointer receivers, no copies by value).
- ☐ Channels have a clear owner who closes them, and receivers can't block forever.
- ☐ Concurrency is bounded with worker pools, `errgroup.SetLimit` or semaphores, not one goroutine per item without limit.
- ☐ Tests run with `-race` in CI.

Context

- ☐ `ctx context.Context` is the first parameter of every function that does I/O or may block.
- ☐ Contexts aren't stored in struct fields.
- ☐ Every `WithCancel` and `WithTimeout` has a matching `defer cancel()`.
- ☐ Context values hold only request-scoped data (request ID, auth claims), never optional parameters or dependencies.
- ☐ Long-running loops check for cancellation.

Resources and I/O

- ☐ Every opened resource is closed: files, `resp.Body`, `rows`, listeners, tickers.
- ☐ `rows.Err()` is checked after iterating query results.
- ☐ No `defer` inside long loops.
- ☐ HTTP clients have timeouts; `http.DefaultClient` isn't used for outbound calls in production code.
- ☐ HTTP servers set `ReadHeaderTimeout`, `ReadTimeout`, `WriteTimeout` and `IdleTimeout`.
- ☐ Request bodies are size-limited (`http.MaxBytesReader`).

Data access

- ☐ SQL uses placeholders (or sqlc), with no string-concatenated queries.
- ☐ Transactions have a clear boundary and always roll back on error (`defer tx.Rollback(ctx)`).
- ☐ Rows that need protecting from concurrent updates are locked (`SELECT ... FOR UPDATE`) in a consistent order.
- ☐ Queries run with a context deadline.
- ☐ There are no N+1 query patterns in loops.
- ☐ Money uses integer minor units (cents) or a decimal type, never `float64`.
- ☐ Migrations are backward compatible with the currently deployed version.

Security

- ☐ Authentication is verified before any business logic runs; authorisation is checked at resource level (a customer can only read their own orders).
- ☐ JWT validation pins the algorithm and checks `exp`, `iss` and `aud`.
- ☐ Secrets come from the environment or a secret manager, never from code or committed config.
- ☐ Secrets and personal data never appear in logs (use `slog.LogValuer` for redaction).
- ☐ Randomness for tokens and IDs comes from `crypto/rand`, not `math/rand`.
- ☐ User-supplied file paths are confined (for example with `os.Root`).
- ☐ `govulncheck` passes.

Observability

- ☐ Logs are structured (`slog`) and include request and trace IDs.
- ☐ Log levels are meaningful; there's no noisy logging in hot paths.
- ☐ New endpoints or consumers have metrics (rate, errors, duration).
- ☐ Metric labels have bounded cardinality: no user IDs or order IDs as labels.
- ☐ Outbound calls propagate trace context.

Resilience and lifecycle

- ☐ Outbound calls have timeouts, and retries only apply to idempotent operations, with backoff and jitter.
- ☐ Message consumers are idempotent (at-least-once delivery is assumed).
- ☐ Graceful shutdown covers the new component: it stops accepting work, finishes in-flight work and releases resources.
- ☐ Readiness reflects real dependencies; liveness doesn't depend on downstream services.

Testing

- ☐ New behaviour has tests, table-driven where there are several cases.
- ☐ Tests use fakes behind small interfaces; mocks only where interactions really matter.
- ☐ There's no `time.Sleep` in tests: use injected clocks, channels or `testing/synctest`.
- ☐ Integration tests that need Docker are behind a build tag and run in CI.
- ☐ Test helpers call `t.Helper()`, and cleanup uses `t.Cleanup`.
- ☐ Edge cases are covered: empty input, zero values, nil, cancellation, concurrent access.

Performance

- ☐ Any optimisation is backed by a benchmark or profile, not a hunch.
- ☐ Slices and maps are preallocated when the size is known.
- ☐ There's no needless conversion between `string` and `[]byte` in hot paths.
- ☐ Large structs in hot loops are passed by pointer.

Dependencies

- ☐ New dependencies are justified, maintained and appropriately licensed. A little copying beats a little dependency.
- ☐ `go.mod` and `go.sum` are tidy (`go mod tidy` produces no diff).
- ☐ No `replace` directives are left over from local debugging.

> 🧠 **Key idea**
>
> The best Go reviews ask one question over and over: "*Could this be simpler?*" Fewer layers, fewer abstractions and more explicit code almost always win.

APPENDIX D

Glossary

> *Plain-English definitions of the Go words and enterprise terms used in this guide.*

Where a term has a close Java relative, it's shown in *italics* after the definition.

Go language and runtime

Term	What it means
`any`	An alias for `interface{}`: a value of any type. *Java:* `Object`.
Blank identifier (`_`)	A placeholder that discards a value, for example an unused return value or loop index.
Buffered channel	A channel with room for N values, so senders don't block until it's full. *Java: a bounded* `BlockingQueue`.
Build tag	A `//go:build` line that includes a file only under certain conditions, such as `integration` tests or a specific OS.
cgo	The bridge that lets Go call C code. It needs a C compiler. Many deployments disable it (`CGO_ENABLED=0`) to get static binaries.
Channel	A typed pipe for sending values between goroutines.
Closure	A function that captures variables from the scope around it. *Java: a lambda capturing effectively-final variables, except Go closures can modify them.*
`comparable`	A built-in constraint for types that support `==` and `!=`. Map keys must be comparable.
Composite literal	Syntax that builds a value in place: `Order{ID: "o1", Status: StatusPending}`.
Constraint	An interface that limits which types a type parameter accepts, such as `cmp.Ordered`. *Java: a bound like* `<T extends Comparable<T>>`.
`context.Context`	A value passed through calls that carries cancellation, deadlines and request-scoped data. *Java: closest to* `ThreadLocal` *or* `ScopedValue`*, but explicit.*
Data race	Two goroutines accessing the same memory at the same time, at least one writing, without synchronisation. Detected by `go test -race`.
`defer`	Schedules a call to run when the surrounding function returns, in last-in-first-out order. *Java:* `finally` *or try-with-resources.*
Embedding	Putting one type inside another without a field name, which promotes its methods. Composition, not inheritance.
Escape analysis	The compiler deciding whether a variable can live on the fast, short-lived stack or has to move to the garbage-collected heap.
Exported	Visible outside its package because the name starts with a capital letter. *Java:* `public`.

Term	What it means
Garbage collector (GC)	Go's concurrent, non-generational collector that frees unreachable heap memory. Tuned mainly with `GOGC` and `GOMEMLIMIT`.
`gofmt`	The standard formatter. All Go code looks the same because everyone uses it.
`GOMAXPROCS`	How many OS threads can run Go code at the same time. Container-aware by default since Go 1.25.
`GOMEMLIMIT`	A soft memory limit that makes the GC work harder as the process approaches it. Useful in containers.
Goroutine	A lightweight function running concurrently, managed by the Go runtime. Cheap enough to have hundreds of thousands. *Java: a virtual thread.*
`gopls`	The official Go language server that powers autocomplete, navigation and refactoring in editors.
`internal/`	A directory whose packages can only be imported by code rooted at its parent. Enforced by the compiler.
`iota`	A constant counter inside a `const` block, used to build enum-like sequences.
Iterator (range-over-func)	A function of type `iter.Seq[T]` that a `for ... range` loop can consume (Go 1.23+). *Java: `Iterator` / `Iterable`.*
Method set	The set of methods a type has. It determines which interfaces the type satisfies. Pointer receivers belong only to the pointer type's method set.
Module	A versioned collection of packages defined by a `go.mod` file. *Java: a Maven artifact.*
Package	A directory of `.go` files compiled together; the unit of visibility and import.
Panic	A run-time crash that unwinds the stack, running deferred calls. Can be stopped with `recover`. *Java: an uncaught `RuntimeException`.*
PGO	Profile-guided optimisation: the compiler uses a CPU profile (`default.pgo`) to optimise hot paths.
Pointer receiver	A method declared on `*T`, which can modify the value and avoids copying it.
Race detector	A tool enabled with `-race` that reports data races at run time. Requires cgo.
Receiver	The value a method is called on, named explicitly: `func (o *Order) Cancel()`. *Java: `this`.*
`recover`	Stops a panic when called inside a deferred function. Used in safety-net middleware.
Rune	A Unicode code point (`int32`). Strings are bytes; ranging over a string yields runes. *Java: roughly a `char`, but a full code point.*
`select`	A statement that waits on several channel operations at once and runs whichever is ready first.
Semantic import versioning	The rule that major version 2 and above of a module adds `/v2` (and so on) to its import path.
Sentinel error	A package-level error value that callers compare against with `errors.Is`, such as `sql.ErrNoRows`.
Slice	A growable view onto an array, with a length and capacity. *Java: `ArrayList`.*
Struct tag	A string of metadata after a struct field, such as `` `json:"id"` ``, read by libraries through reflection. *Java: a field annotation.*

Term	What it means
Toolchain	A specific Go release (compiler, runtime, standard library). `go.mod` can require a minimum one, and Go can download it automatically.
Type assertion	`v.(T)`: extracting a concrete type from an interface value. *Java: a cast after `instanceof`.*
Type parameter	A generic type placeholder: `func Max[T cmp.Ordered](a, b T) T`.
Type switch	A `switch v := x.(type)` that branches on the dynamic type of an interface value. *Java: pattern matching for `switch`.*
Vendoring	Copying dependencies into a `vendor/` directory (`go mod vendor`) for fully offline builds.
`WaitGroup`	A counter that lets one goroutine wait for a group of others to finish. *Java: `CountDownLatch`.*
Zero value	The default value of every type: `0`, `""`, `false`, `nil`, or a struct of zero values. Go has no uninitialised variables.

Enterprise and architecture terms

Term	What it means
At-least-once delivery	A messaging guarantee that every message arrives, possibly more than once. Consumers must be idempotent.
Backoff with jitter	Waiting progressively longer between retries, plus a random amount, so clients don't retry in lockstep.
Bulkhead	Isolating resources (for example, a separate worker pool per dependency) so one failure can't sink everything.
Circuit breaker	A guard that stops calling a failing dependency for a while, then cautiously tries again.
Composition root	The one place, usually `main`, where all dependencies are created and wired together.
Cursor pagination	Paging through results with an opaque "continue after this" token instead of page numbers. Stable under inserts.
Graceful shutdown	Stopping a service by refusing new work, finishing in-flight work and then releasing resources in order.
Hexagonal architecture	Keeping business logic in the centre, with databases, HTTP and queues as replaceable adapters at the edges. Also called ports and adapters.
Idempotency	Doing an operation twice has the same effect as doing it once. Essential for retries and message consumers.
Liveness probe	A Kubernetes check of "is this process stuck?" Failure means a restart.
Readiness probe	A Kubernetes check of "can this instance take traffic right now?" Failure removes it from load balancing.
Outbox pattern	Writing an event to an `outbox` table in the same transaction as the business change, then publishing it separately. No lost or phantom events.
Problem Details (RFC 9457)	A standard JSON format for HTTP API errors (`type`, `title`, `status`, `detail`).
RED method	Monitoring every service by **R**ate, **E**rrors and **D**uration.

Term	What it means
Saga	A long-running business process split into steps, each with a compensating action if a later step fails.
Structured logging	Logging key-value fields (usually JSON) instead of free text, so logs can be searched and aggregated.
Twelve-factor app	A set of conventions for cloud services, such as config in the environment, stateless processes and logs to stdout.